

The sTeX4 Package Collection*

Michael Kohlhase, Dennis Müller
FAU Erlangen-Nürnberg
<http://kwarc.info/>

2026-06-24

Contents

1	Introduction & Setup	8
1.1	What is sTeX?	8
1.2	The sTeX package	9
1.3	Math archives and the MathHub Directory	9
1.4	Setting Up the FvMf IDE	9
I	Tutorial	11
2	The Basics	12
2.1	Text symbols	15
2.1.1	Using Modules & Search in the IDE	15
2.2	Symbol References	17
2.3	Modules and Simple Symbol Declarations	20
2.4	Documenting Symbols	23
2.5	Sectioning and Reusing Document Fragments	25
2.6	Building and Exporting HTML	27
3	Mathematical Concepts	30
3.1	Simple Symbol Declarations	30
3.1.1	Semantic Macros and Notations	30
3.1.2	Types and Variables	33
3.1.3	Flexary Macros and Argument Modes	35
3.1.4	Precedences	37
3.1.5	Finishing Equality	38
3.1.6	Variable Sequences	38
3.2	Statements	38
3.2.1	Definitions	39
	Semantic Macros in Text Mode	40
	Definientia	41

*Version 4.0.1 (last revised 2026-06-24)

	Using <code>Symbols</code> Without <code>Semantic Macros</code> and Exporting Code in <code>Modules</code>	42
3.2.2	Assertions	43
3.2.3	Proofs	45
3.3	Mathematical Structures	45
3.3.1	Declaring and Using <code>Structures</code>	45
	Instantiating <code>Structures</code>	46
3.3.2	Extending <code>Structures</code> and Axioms	48
	Conservative Extensions	49
3.3.3	Nesting <code>Structures</code> and <code>\this</code>	50
3.4	Complex Inheritance and Theory Morphisms	52
3.4.1	Glueing <code>Structures</code> Together	55
3.4.2	Realizations	56
4	Extensions for Education	59
4.1	Slides and Course Notes	59
4.1.1	Introduction	59
4.1.2	Package Options	60
4.1.3	Notes and Slides	60
4.1.4	Managing and Customizing Themes	61
4.1.5	Frame Images	62
4.1.6	Ending Documents Prematurely	63
4.1.7	Global Document Variables	63
4.1.8	Excursions	63
4.2	Problems and Exercises	64
4.2.1	Background	64
4.2.2	The Package, Options, and Configuration	65
4.2.3	(Open) Problems	66
4.2.4	Solutions and Answer Classes	67
4.2.5	Grading Support	68
4.2.6	Structured Problems	69
4.2.7	Single/Multiple Choice Blocks	70
4.2.8	Filling-In Concrete Solutions	73
4.2.9	Including Problems	75
4.2.10	Testing and Spacing	75
4.3	Homework Assignments and Exams	76
4.3.1	Introduction	76
4.3.2	Package Options	76
4.3.3	Assignments	76
4.3.4	Including Assignments	76
4.3.5	Typesetting Exams	77
II	User Manual	78

5	Basics	79
5.1	Package and Class Options	79
5.2	Math Archives and the MathHub Directory	80
5.2.1	The Structure of Math Archives	81
5.2.2	MANIFEST.MF-Files	81
5.3	The <code>lib</code> -Directory	82
5.4	Basic Macros	83
6	Document Features	84
6.1	Document Fragments	84
6.2	Using and Referencing Document Fragments	85
6.3	Cross-Document References	85
7	Modules and Symbols	88
7.1	Modules	88
7.1.1	Signature Modules , Languages, and Multilinguality	89
7.2	Symbol Declarations	89
7.2.1	Returns	90
7.3	Referencing Symbols	90
7.4	Notations and Semantic Macros	92
7.4.1	Precedences and Bracketing	93
7.4.2	Notations for Argument Sequences	94
7.4.3	Semantic Macros	94
7.5	Simple Inheritance	95
7.6	Variables and Sequences	97
7.7	Structures	98
7.7.1	Semantic Macros for Structures	99
8	Statements	101
8.1	More on Definitions	102
8.2	More on Assertions	103
9	Customizing Typesetting	104
9.1	Highlighting Symbol References	104
9.2	Styling Environments and Macros	105
9.3	Custom CSS for Environments	107
10	Additional Packages	108
10.1	NotesSlides Manual	108
10.1.1	Introduction	108
10.1.2	Package Options	108
10.1.3	Notes and Slides	109
10.1.4	Customizing Header and Footer Lines	110
10.1.5	Frame Images	110
10.1.6	Ending Documents Prematurely	111
10.1.7	Global Document Variables	111
10.1.8	Excursions	112
10.2	Problem Manual	113
10.2.1	Introduction	113
10.2.2	Problems and Solutions	113

10.2.3	Markup for Added-Value Services	115
	Multiple Choice Blocks	115
	Filling-In Concrete Solutions	116
10.2.4	Including Problems	117
10.2.5	Testing and Spacing	118
10.3	HWExam Manual	118
10.3.1	Introduction	118
10.3.2	Package Options	118
10.3.3	Assignments	119
10.3.4	Including Assignments	119
10.3.5	Typesetting Exams	119
10.4	Tikzinput Manual	120
III	Documentation	122
11	Utilities	123
11.1	kpsewhich and Environment Variables	123
11.2	Logging	124
11.3	File Paths	124
11.4	Group-like Behaviours	125
11.5	Key Handling	126
11.6	Languages	127
11.7	Styling	127
11.8	Persisting Dependencies	127
12	HTML Output	128
13	Math Archives	132
14	URIs	134
14.1	Document URIs	134
14.2	Module URIs	135
14.3	Symbol URIs	136
15	Documents	138
15.1	Sectioning	139
15.2	Inter-Document References	140
15.3	Inputting From MathHub Resources	140
16	Modules	142
16.1	SMS Mode	143
16.2	Inheritance and Morphisms	145
16.3	Theory Morphisms	145

17 Symbols	147
17.1 Declarations	147
17.2 Retrieval	148
17.3 Variables	148
17.3.1 Variable Sequences	149
17.4 Expressions	149
17.4.1 Term Annotations	150
17.4.2 Symbol Arguments	151
17.4.3 Notation components	151
17.4.4 Symbol References	152
17.5 Checking	152
18 Notations	153
18.1 Automated Bracketing	154
19 Structures	156
20 Statements	157
21 Proofs	158
22 Metatheory	160
23 Others	161
24 Additional Packages	162
24.1 NotesSlides Documentation	162
24.2 Problem Documentation	162
24.3 HWExam Documentation	162
24.4 Tikzinput Documentation	162
IV Implementation	163
25 Setting up	164
26 Utilities	166
26.1 kpsewhich and Environment Variables	167
26.2 Logging	168
26.3 File Paths	169
26.4 Group-like Behaviours	173
26.5 Key Handling	174
26.6 Languages	176
26.7 Styling	177
26.8 Persisting Dependencies	179
26.9 Auxiliary Methods	181
27 HTML Output	182
28 Math Archives	187

29 URIs	194
29.1 Document URIs	194
29.2 Module URIs	197
29.3 Symbol URIs	201
30 Documents	204
30.1 Sectioning	206
30.2 Inter-Document References	211
30.3 Inputting From MathHub Resources	216
31 Modules	222
31.1 SMS Mode	230
31.2 Inheritance and Morphisms	234
31.3 Theory Morphisms	239
32 Symbols	254
32.1 Declarations	254
32.2 Retrieval	262
32.3 Variables	266
32.3.1 Variable Sequences	269
32.4 Expressions	271
32.4.1 Term Annotations	292
32.4.2 Symbol Arguments	294
32.4.3 Notation components	300
32.4.4 Symbol References	302
32.5 Checking	306
33 Notations	308
33.1 Automated Bracketing	323
34 Structures	325
35 Statements	331
36 Proofs	338
37 Metatheory	351
38 Others	354
39 Additional Packages	356
39.1 Implementation: The notesslides Package	356
39.1.1 Class and Package Options	356
39.1.2 Notes and Slides	360
39.1.3 Environment and Macro Patches	364
39.1.4 Styling Across Notes/Slides	365
39.1.5 Beamer Compatibility	366
39.1.6 TODO Excursions	367
39.2 Implementation: The problem Package	369
39.2.1 Package Options	369
39.2.2 Problems and Solutions	371

39.3	Implementation: The hwexam Package	389
39.3.1	Package Options	389
39.3.2	QR Codes	390
39.3.3	Assignments	396
39.3.4	Leftovers	400
39.4	Tikzinput Implementation	401

Index 403



$\text{\textcolor{teal}{s}TeX}$ is – by now – relatively stable and ready to use for the general public. However, it is also actively being developed further and subject to ongoing research. Some of the features described in here might not fully work as expected, some are still experimental, there might occasionally be cryptic error messages, and in general bugs are expected.

We welcome all kinds of issues you might encounter at <https://github.com/slatex/sTeX>.

If you have questions or problems with $\text{\textcolor{teal}{s}TeX}$, you can talk to us directly at <https://matrix.to/#/#stex:fau.de>.

Chapter 1

Introduction & Setup

1.1 What is sTeX?

sTeX is a system for generating human-oriented documents in either PDF or HTML format, augmented with computer-actionable semantic information (conceptually) based on the OMDoc format and ontology.

At its core is the sTeX package for L^AT_EX, that allows for semantically marking up document fragments; in particular concepts, formulae and mathematical statements (such as definitions, theorems and proofs). Running pdf_lat_ex over sTeX-annotated documents formats them into normal-looking PDF.

But sTeX also comes with a conversion pipeline into semantically annotated HTML (FTML), which can host semantic added-value services that make the documents *active* (i.e. interactive and user-adaptive) – essentially turning L^AT_EX into a document format for (mathematical) knowledge management (MKM).

The sTeX system consists of the following components:

- The sTeX package,
- the RuSTeX system for converting L^AT_EX documents to (semantically enriched) FTML,
- the FLMf system; a semantically informed back-end for managing and hosting the FTML with integrated added-value services,
- a collection of math archives of sTeX document fragments and symbols for reuse, available of mathhub.info, and
- the FLMf IDE: A VS Code extension that, besides the usual IDE functionalities like syntax highlighting, integrates FLMf and allows for accessing the public math archives on mathhub.info to make the entire toolchain easily accessible to authors.

If PDF is all you are interested in, the sTeX package is all you *need*. Either way, however, we recommend using the sTeX IDE – it very much helps with semantically annotating your L^AT_EX documents.

1.2 The $\text{\texttt{sTeX}}$ package

The $\text{\texttt{sTeX}}$ package extends $\text{\texttt{L^{A}T\textsubscript{E}X}}$ with:

- A mechanism to declare **symbols** – concepts, functions, relations, variables, etc., which can be used and referenced in text or via **semantic macros** in mathematical formulae,
- a **module system** based on logical identifiers – **modules** bundle declarations, definitions, theorems, document snippets, and **symbols** for reuse, and
- an analogous organizational structure for developing documents modularly from individual fragments and sections.

The $\text{\texttt{sTeX}}$ package has been designed to have minimal impact on other **packages** and **document classes**, or the actual document layout – formatting of semantic **environments**, **symbol** references and **semantic macros** can be fully customized.

1.3 Math archives and the MathHub Directory

To make the most of $\text{\texttt{sTeX}}$, it is strongly encouraged to follow a workflow of small document fragments and **modules** to maximize reuse.

One considerable weakness of $\text{\texttt{L^{A}T\textsubscript{E}X}}$ is the way source files are referenced: they need to be either in a **texmf** directory, or else be referenced via file paths relative to the main **.tex**-file being compiled. This is highly inconvenient if we want to collaboratively develop many highly interrelated document fragments.

$\text{\texttt{sTeX}}$ therefore adds an organizational layer on top of $\text{\texttt{L^{A}T\textsubscript{E}X}}$ ’s: **math archives** stored in a fixed **MathHub** directory anywhere on your hard drive. Referencing source files and **modules** is then done relative to the containing **math archive**, and is thus *independent* of user’s individual setups or the current **.tex**-file.

The drawback of this approach is that $\text{\texttt{sTeX}}$ needs to know the location of your **MathHub** directory. There are multiple ways to achieve that, but the simplest and recommended approach is to set an environment variable: Simply create a new directory **<path>/MathHub** somewhere on your hard drive and set the environment variable **MATHHUB** as the path to this new directory.

Alternatively, you can let the **F\textsubscript{M}\textsubscript{J} IDE** do the work for you (see section 1.4).

For more on **math archives**, see Section 1 (**Math Archives** and the **MathHub** Directory) in the $\text{\texttt{sTeX}}$ Manual.

1.4 Setting Up the **F\textsubscript{M}\textsubscript{J} IDE**

$\text{\texttt{sTeX}}$ is based on $\text{\texttt{L^{A}T\textsubscript{E}X}}$, and adds additional layers of presentational and functional markup to it. As a consequence the source files of $\text{\texttt{sTeX}}$ documents look quite different from the resulting **FTML** and **PDF** documents. Thus the best way of interacting the $\text{\texttt{sTeX}}$ document collections is via an **integrated development environment (IDE)**. In this tutorial we will use the **F\textsubscript{M}\textsubscript{J}** plugin for the **VS Code**, which you should set up as a first step (this also sets up the necessary auxiliary software).

Setting up $\text{\texttt{sTeX}}$ with the dedicated **IDE** is easy:

1. Download and install **VS Code** here: <https://code.visualstudio.com/download>

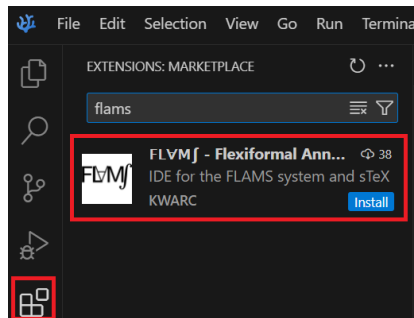


Figure 1: Installing the FLVMj IDE

2. Start **VS Code** and navigate to the *Extensions*-tab on the left. Here you can search for Extensions in the **VS Code** marketplace. Look for the FLVMj extension by *KWARC*, as in Figure 1.
3. Having done so, upon opening any folder in **VS Code** containing a `.tex`-file, you will be prompted to either download a new FLVMj or point the extension to an existing FLVMj executable. In the vast majority of cases, you'll want to download and choose a directory to install FLVMj to.

Part I

Tutorial

*The dynamic [HTML](#) version of this part can be found at
<https://mathhub.info/?a=sTeX/Documentation&d=tutorial&l=en>*

[sTeX](#) is a system for generating human-oriented documents in either [PDF](#) or [HTML](#) format, augmented with computer-actionable semantic information (conceptually) based on the [OMDoc](#) format and ontology.

In this part, we will give a broad but shallow introduction to [sTeX](#), and what you can get out of it. Additionally, this serves as an introduction to the [FvMf IDE](#), and we consequently assume that you have that one set up, as described in section 1.4.

Note that in [PDFs](#), the specific highlighting of semantically annotated text is fully customizable (see chapter 9). In this document, we use [this highlighting](#) for [notation](#) components, [this highlighting](#) for [symbol](#) references, [this highlighting](#) for (local) [variables](#) and [this highlighting](#) for definienda; i.e. new concepts being introduced.

Chapter 2

The Basics

This document itself uses [sTeX](#) and serves as a direct example for the following. You can download its source files, the generated [PDF](#) files, and the generated [HTML](#) documents directly from within the [IDE](#), by navigating to the [FLMf](#) tab in the menu on the left and finding [sTeX/Documentation](#) in the list of [math archives](#) and clicking the small “Install”-button next to it, see the screenshot on the left of Figure 2.

Once downloading is finished (this may take a while since dependencies are also downloaded), you can then browse the [.tex](#)-files in [sTeX/Documentation](#) directly from the [math archives](#) panel in the [sTeX](#) tab, as you can see in the right screenshot in Figure 2.

For example, you can now navigate to the file [tutorial/intro.en](#) to see the sources of this very chapter.

As a first example, consider the following document fragment from section 1.1:

[sTeX](#) is a system for generating human-oriented documents in either [PDF](#) or [HTML](#) format, augmented with computer-actionable semantic information (conceptually) based on the [OMDoc](#) format and ontology.

If you were to look at the generated [HTML](#) from this fragment, you could hover over the highlighted words ([sTeX](#), [PDF](#), [HTML](#), [OMDoc](#)) and get a little popup with their definitions (Figure 3). Neat, huh?

Here, in the [PDF](#), hovering will only show you a unique identifier ([MMT-URI](#)) for the word, and link to a definition on the web. Still useful, but not quite as neat, of course.

A plain [L^AT_EX](#)-version of the above document fragment, without any [sTeX](#) markup, could look like this:

Example 1

Input:

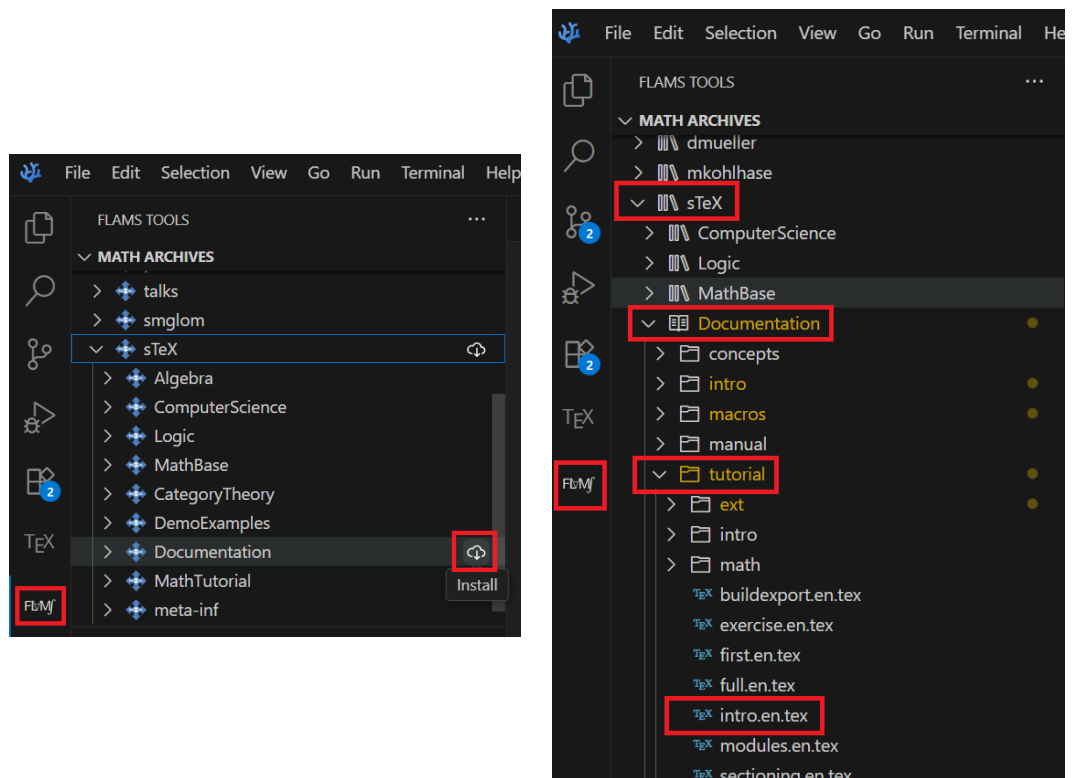


Figure 2: Installing Math Archives

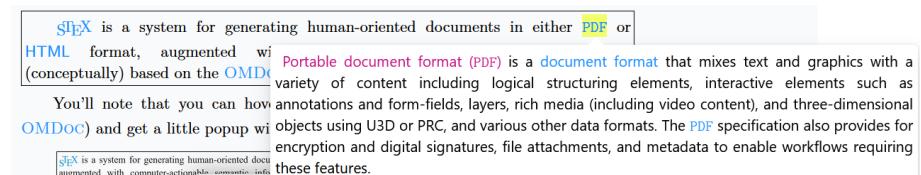


Figure 3: Definition on Hover

```

File tutorial/intro/intro1plain.en.tex
1 \documentclass{article}
2 \usepackage{stex-logo}
3 \begin{document}
4
5   \sTeX{} is a system for generating human-oriented documents
6   in either \textsf{PDF} or \textsf{HTML} format, augmented
7   with computer-actionable semantic information (conceptually)
8   based on the \textsc{OMDoc} format and ontology.
9
10 \end{document}

```

Output:

sTeX is a system for generating human-oriented documents in either PDF or HTML format, augmented with computer-actionable semantic information (conceptually) based on the OMDoc format and ontology.

(Examples like the one above always show the file the source code is in, so if you have downloaded the sTeX /Documentation [math archive](#) you can toy around with it yourself)

If you save a file in the **IDE** (regardless of whether it has unsaved changes), a preview window will pop up, showing you the **HTML** generated from the **.tex**-file; see (Figure 4).

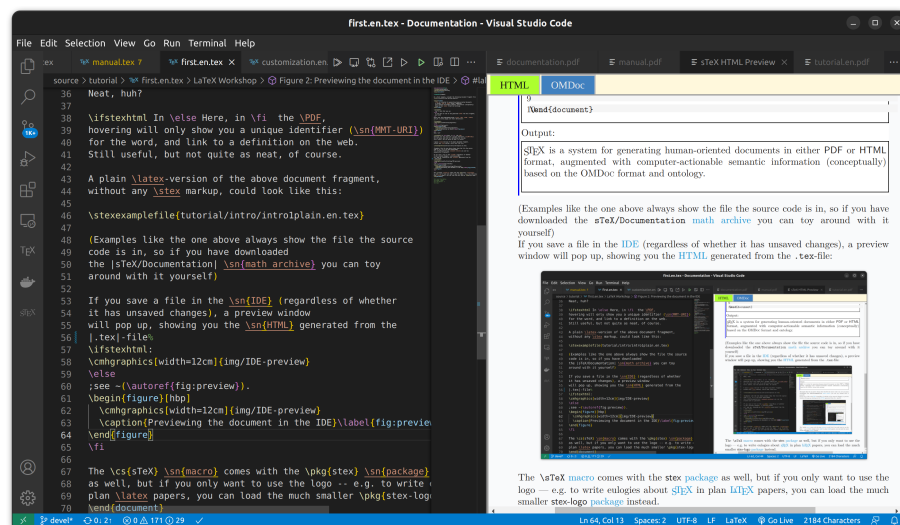


Figure 4: Previewing the document in the IDE

The sTeX macro comes with the `stex` package as well, but if you only want to use the logo – e.g. to write eulogies about sTeX in plain $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ papers, you can load the much smaller `stex-logo` package instead.

2.1 Text symbols

The most central concept behind sTeX is that of a *symbol*:

sTeX A **symbol** is a *named* concept that can be defined, documented and referenced. Examples for **symbols** are mathematical constants, functions, theorems, statements, principles – anything that has a (somewhat) precise meaning and can be referenced by name can be a **symbol**.

Before we explain how we can declare new **symbols** and associate them with definitions, **notations** and all that, let’s assume an ideal world in which others have done that job already for us – after all, sTeX is all about *reuse*, and naturally, there are sTeX **symbols** for all of the above already. Let’s start with the one for sTeX itself:

2.1.1 Using Modules & Search in the IDE

In the **VS Code IDE**, navigate to the **FLMf**-tab on the left. In the search panel, select the “Symbols” radio button and search for “**sTeX**”. The third search result should be what we’re looking for (Figure 5).

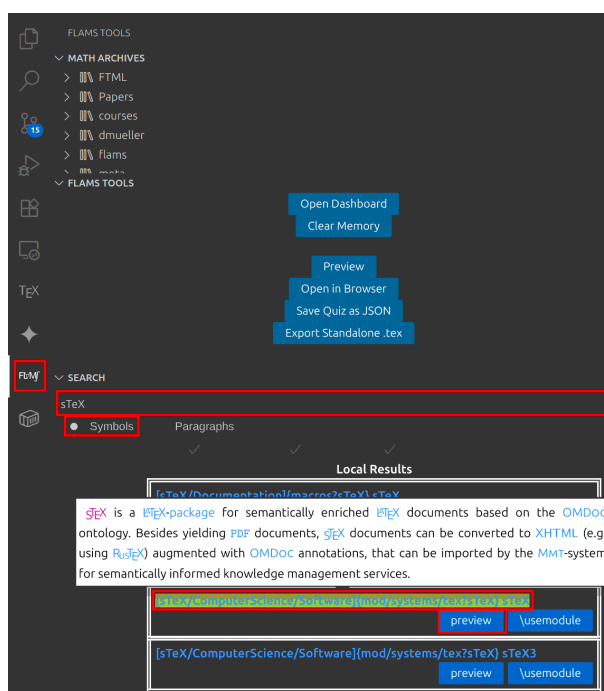


Figure 5: Search in the sTeX IDE

Search results are grouped into *local* and *remote* results. Local ones are the ones you already have in your local MathHub directory; remote ones you can download directly from within the IDE.

You can click the preview button to see the generated FTML for the document – the resulting window that pops up also has an OMDoc button you can click, which (among other things) shows you the semantic macros provided by the respective module: In this case, it tells us that there is a *text symbol* named “sTeX” with semantic macro `\stex` in the module `sTeX` in the same document. It produces the presentation “STeX” as we want (Figure 6).

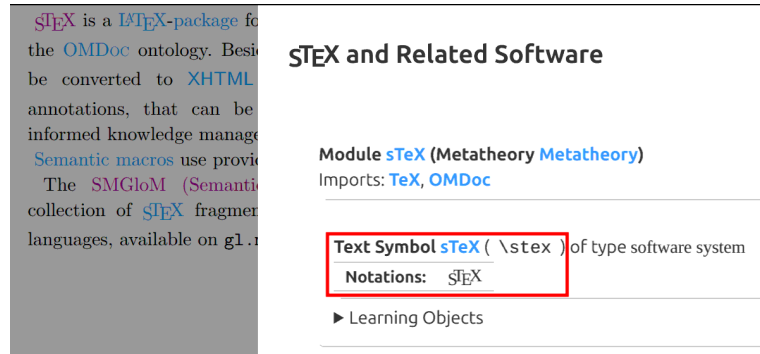


Figure 6: OMDoc Preview

STeX A *text symbol* is a *symbol* `foo` with an associated *semantic macro* `\foo`. The *macro* `\foo` is allowed in text or math mode and produces a predefined piece of text output annotated with `foo`.
The variant `\fooname` produces the same output without annotation.

If we want to use the `sTeX` symbol in a document – which we have open in the IDE – we simply click on the `\usemodule` button, and the IDE will automatically insert the line
`\usemodule[sTeX/ComputerScience/Software]{mod/systems/tex?sTeX},`
making all *symbols* in that *module* available to use – in particular, we can now use the `\stex` *semantic macro* instead of the plain, non-semantic `\sTeX` *macro* – that is, of course, after we include the `stex` *package* first.

STeX The `\usemodule` *macro* takes as *optional* argument the name of a *math archive*, and as a regular argument the path to an `sTeX` *module* (see section 7.5).

Analogously, we can also search for the `PDF`, `HTML` and `OMDoc` symbols, all of which are also *text symbols* and have the associated *semantic macros* `\PDF`, `\HTML` and `\omdoc`; the document should thus look like this:

Example 2

Input:

File tutorial/intro/intro1stex.en.tex

```
1 \documentclass{article}
2 \usepackage{stex}
3 \begin{document}
4   \usemodule[sTeX/ComputerScience/Software]{mod/systems/tex?sTeX}
5   \usemodule[sTeX/ComputerScience/Software]{mod/formats?PDF}
6   \usemodule[sTeX/ComputerScience/Software]{mod/formats?HTML}
7   \usemodule[sTeX/ComputerScience/Software]{mod/formats?OMDoc}
8
9   \stex is a system for generating human-oriented documents
10  in either \PDF or \HTML format, augmented
11  with computer-actionable semantic information (conceptually)
12  based on the \omdoc format and ontology.
13 \end{document}
```

Output:

[sTeX](#) is a system for generating human-oriented documents in either [PDF](#) or [HTML](#) format, augmented with computer-actionable semantic information (conceptually) based on the [OMDoc](#) format and ontology.

Now, our generated [HTML](#) looks a lot more interesting, with highlighting, pop-ups on hover and all that. Notably however, if we compile the file with `pdflatex`, it looks pretty much exactly as before.

That's because we haven't told [sTeX](#) what to do with semantic annotations yet – and by default, it does not do anything fancy, except for wrapping them in an `\emph`. We can customize how we want [sTeX](#) to highlight various semantic text fragments (see chapter 9). A default highlighting schema is provided in the `stex-highlighting` [package](#) – including that will

- highlight semantically annotated text in [this color](#),
- show the [MMT-URI](#) of the corresponding [symbol](#) in a tooltip on hovering over the text,
- make the text link to the place the [symbol](#) is being defined in the current document (if it is), or, alternatively,
- make it link to an external resource, if one is known. In our case, they link to `stexmmt.mathhub.info/:sTeX`, where the [HTML](#) for all the [symbols](#) we use in this document are hosted.

2.2 [Symbol](#) References

Let's continue with the next paragraph of section 1.1; for now unannotated:

[Example 3](#)

Input:

File tutorial/intro/intro2plain.en.tex

```
1 \documentclass{article}
2 \usepackage{stex}
3 \begin{document}
4
5   At its core is the \sTeX{} package for \LaTeX, that allows for
6   semantically marking up document fragments; in particular
7   concepts, formulae and mathematical statements (such as
8   definitions, theorems and proofs). Running \texttt{pdflatex}
9   over \sTeX-annotated documents formats them into normal-looking
10  \textsf{PDF}.
11
12 \end{document}
```

Output:

At its core is the $\text{\sTeX}$ package for $\text{\LaTeX}$, that allows for semantically marking up document fragments; in particular concepts, formulae and mathematical statements (such as definitions, theorems and proofs). Running `pdflatex` over $\text{\sTeX}$ -annotated documents formats them into normal-looking PDF.

We already know how to annotate “ $\text{\sTeX}$ ” and “PDF”; and if we use the search field in the IDE again, we can also find a *text symbol* for “ $\text{\LaTeX}$ ”. But if we look at the documentation, we will note that *more* is highlighted:

At its core is the $\text{\sTeX}$ package for $\text{\LaTeX}$, that allows for semantically marking up document fragments; in particular concepts, formulae and mathematical statements (such as definitions, theorems and proofs). Running `pdflatex` over $\text{\sTeX}$ -annotated documents formats them into normal-looking PDF.

The “*package*”-symbol can be found in the $\text{\LaTeX}$ module too, and searching for the keywords “formula” and “mathematics” will yield the symbols “*well-formed formula*” and “*mathematics*”, but they are not *text symbols* and “*mathematics*” and “*package*” do not even have a *semantic macro* – and the one for “*well-formed formula*” would not work outside of math mode.

Text symbols are special in that way – they are intended for *symbols* that have a specific formatting associated (such as $\text{\LaTeX}$, OMDoc, or HTML, which we prefer to typeset as sans serif). For those settings, it makes sense to associate that formatting with a *semantic macro* that does the typesetting for us.

Symbols without a text macro can be referenced with the `\syname` macro: `\syname{package}` prints the *name* of the “*package*”-symbol and annotates it accordingly, without any special formatting – in particular it is compatible with being in `\emph`, `\textbf` and similar *macros*. That takes care of *one* of the missing annotations.

More generally, the `\symref` macro can be used to annotate arbitrary text with a *symbol*: `\symref{mathematics}{mathematical}` associates the text `mathematical` with the *symbol* “*mathematics*”; thus, we get “*mathematical*” and similarly “*formulae*”.

In general, any `macro` that expects a `symbol` name can be given either

1. the *name* of the `symbol`,
2. the name of its `semantic macro`,
3. or any suffix of its `MMT-URI` containing at least the `module` name.

STEX

The second option is often short – and therefore convenient to write; for example, to achieve “`formulae`”, we can also write `\symref{wff}{formulae}`, since `\wff` is the `semantic macro` for “`well-formed formula`”.

The third option allows for distinguishing between multiple `symbols` with the same name – the `IDE` can help in the latter case, by underlining ambiguous `symbol` references in yellow, and offering the `Quick Fix` functionality to let you select and autocomplete the specific `symbol` you want to reference.

Since `\symname` and `\symref` are a lot to type for something that should ideally be used as often as possible, the `macros` `\sn` and `\sr` exist as well and behave exactly the same way. We also provide some convenience abbreviations for `\sn`; namely `\Sn` (capitalizes the first letter of the `symbol` name), `\sns` (adds an “s” at the end, for the most common pluralization of a name), and `\Sns` (both).

Using all of the above, our annotated fragment now looks like this:

Example 4

Input:

```
File tutorial/intro/intro2stex.en.tex
5 \usemodule[sTeX/ComputerScience/Software]{mod/systems/tex?sTeX}
6 \usemodule[sTeX/Logic/General]{mod/syntax?Formula}
7 \usemodule[sTeX/MathBase/General]{mod?Mathematics}
8 \usemodule[sTeX/ComputerScience/Software]{mod/formats?PDF}
9
10 At its core is the \stex \sn{package} for \latex, that allows for
11 semantically marking up document fragments; in particular
12 concepts, \sr{wff}{formulae} and \sr{mathematics}{mathematical}
13 statements (such as definitions, theorems and proofs). Running
14 \texttt{pdflatex} over \stex-annotated documents formats them
15 into normal-looking \PDF.
```

Output:

At its core is the `sTeX package` for `LaTeX`, that allows for semantically marking up document fragments; in particular concepts, `formulae` and `mathematical` statements (such as definitions, theorems and proofs). Running `pdflatex` over `sTeX`-annotated documents formats them into normal-looking `PDF`.

There’s only one problem: *the document does not compile*, with an error `Undefined control sequence`. The reason being that *some* `macro` in the `module` `Formula` uses the `\text` `macro`. We can fix that by using the `amsfonts` `package` of course, but this points to a more general problem; namely that `modules` can make use of various `LaTeX` `packages` for typesetting `symbols`.

Good practice suggests putting those packages into a *prelude* per `math archive`, which we can then import from anywhere, using the `\libinput` `macro`. For more on that, see

section 5.3.

For now, suffice it to say that we can import all [packages](#) required for the [module Formula](#) from the [math archive sTeX/Logic/General](#) by adding the line

```
\libinput[sTeX/Logic/General]{preamble}
```

before the `\begin{document}`.

2.3 Modules and Simple Symbol Declarations

Consider again the first two paragraphs of section 1.1:

[sTeX](#) is a system for generating human-oriented documents in either [PDF](#) or [HTML](#) format, augmented with computer-actionable semantic information (conceptually) based on the [OMDoc](#) format and ontology.

At its core is the [sTeX](#) package for [L^AT_EX](#), that allows for semantically marking up document fragments; in particular concepts, [formulae](#) and [mathematical](#) statements (such as definitions, theorems and proofs). Running `pdflatex` over [sTeX](#)-annotated documents formats them into normal-looking [PDF](#).

Firstly, note that the first paragraph would be perfectly suitable to serve as a pop-up definition on hover for the [sTeX](#) symbol. Secondly, what if all the [symbols](#) used in the above *didn't* already exist?

In this section, we will describe how to make your own [symbols](#) and collect them as reusable fragments in [modules](#) and [math archives](#) from scratch.

We start by creating a new [math archive](#). In the [IDE](#), switch to the [sTeX](#)-tab on the left and click the “New sTeX Archive” button (Figure 7). You will then be asked for

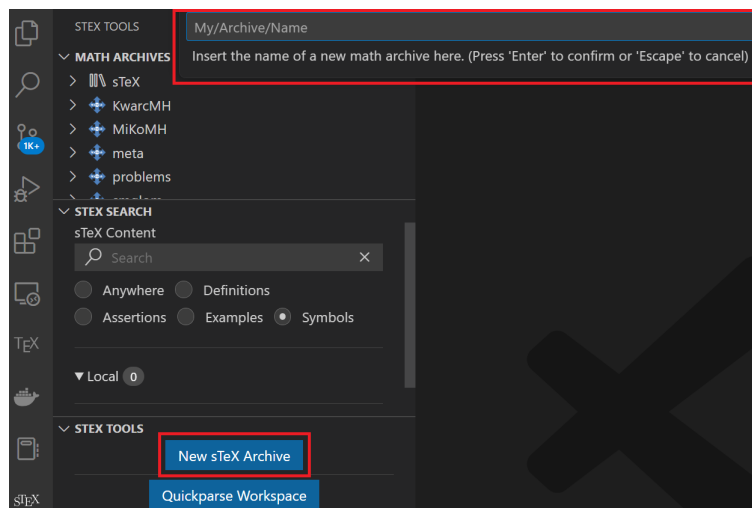


Figure 7: New [Math Archive](#) in the [IDE](#)

the name of the [archive](#), and a url-base, where the content is supposedly going to end up

online. You can safely keep the defaults for the latter. In the following, we assume that your archive is named `my/archive`.

The IDE will then create the following files and directories in your MathHub directory:

```
- my
  - archive
    - lib
      - preamble.tex
    - META-INF
    - MANIFEST.MF
    - source
    - helloworld.tex
```

...and open the file `helloworld.tex` with the content

```
1 \documentclass{stex}
2 \libinput{preamble}
3 \begin{document}
4 % A first sTeX document
5 \end{document}
```

You can now reference any newly created content in you new `archive` using for example `\usemodule[my/archive]{...}`.

Let's start with the "`LaTeX`" symbol. Rename the file `helloworld.tex` to something more meaningful, for example `latex.en.tex` – the `.en` will be picked up on by `sTeX` to signify that the fragment will be in *english* (see subsection 7.1.1).

What we want to achieve in this file is the following:

TeX is a document typesetting software developed by Donald Knuth, with a focus on mathematical formulae. It is based on a powerful and extensible **macro** expansion engine.

LaTeX is a (nowadays) default collection of **TeX macros** developed by Leslie Lamport. Among other things, **LaTeX** introduces **environments**, a distinction between preamble and document content, **packages** to bundle and distribute **macro** definitions, and **document classes**: special **packages** that govern the global layout of a document.

In particular, in the **HTML** the two paragraphs above should be shown when hovering over the **symbols** they define (as indicated by the magenta definiendum highlighting). So we need **symbols** and **semantic macros**, for: **TeX**, **macro**, **LaTeX**, **environment**, **package** and **document class**.

Symbol declarations are only allowed within **modules**:

sTeX A **module** is a *named* block that bundles **symbol** declarations for subsequent reuse.
A **module** is introduced with the **smodule**-environment.

Let's name our **module** `LaTeX`. We then wrap the contents of our document in a **smodule** environment:

```
\begin{document}
  \begin{smodule}{LaTeX}
    ...
  \end{smodule}
\end{document}
```

 Note, that the IDE immediately picks up on this and displays the full FTML URI of our new module over the `\begin{smodule}{LaTeX}` (Figure 8) –



```

my > archive > source > %X latex.tex > {} https://mathhub.info?a=my/archive&p=latex&m=L
1 \documentclass{stex}
2 \begin{document}
3   https://mathhub.info?a=my/archive&p=latex&m=LaTeX
4 \begin{smodule}{LaTeX}
5   ...
6 \end{smodule}
7 \end{document}

```

Figure 8: VS Code URI on Hover

From this, we can glimpse that the namespace of the module is `http://mathhub.info?a=my/archive&p=latex&m=LaTeX`. This implies, that to use the module somewhere else, we will have to type `\usemodule[my/archive]{latex?LaTeX}` – the `latex`-part pointing to the file and `LaTeX` referring to the actual module.

If we rename the file to `LaTeX.en.tex`, we notice that the namespace changes to `http://mathhub.info?a=my/archive&m=LaTeX`, allowing us to now use it with `\usemodule[my/archive]{LaTeX}` directly. That’s because the module name `LaTeX` and the file name `LaTeX` match now (see section 7.5, Figure 9).



```

my > archive > source > %X LaTeX.en.tex > ...
1 \documentclass{stex}
2 \begin{document}
3   https://mathhub.info?a=my/archive&m=LaTeX
4 \begin{smodule}{LaTeX}
5   ...
6 \end{smodule}
7 \end{document}

```

Figure 9: VS Code URI on Hover



Note that “`LaTeX`” and “`latex`” only differ in capitalization – if your file system is case-insensitive (as e.g. MacOS’s was until quite recently), this distinction gets murky, but remains very important especially if you want to share your [math archive](#) with others!
It is therefore *highly recommended* to treat file names as case-sensitive either way.

Within the module, we can now declare new symbols using the `\symdecl`-macro. We start with those that are not text symbols:

```

\symdecl*{macro}
\symdecl*{environment}
\symdecl*{package}
\symdecl*{document class}

```

The `*` after the `\symdecl` indicates, that we do not want a semantic macro for the symbol – otherwise, it would generate one with the same name as the symbol itself and “pollute the macro space”, so to speak.

The symbols `\TeX` and `\LaTeX`, however, have a definite way of being typeset associated with them, which can be produced using the standard `\TeX` and `\LaTeX` macros. So let's make them text symbols, using the `\textsymdecl` macro:

```
\textsymdecl{tex}{\TeX}
\textsymdecl{latex}{\LaTeX}
```

The first argument being the name of the generated macro (i.e. `\tex` and `\latex`) and the second one specifying the output to produce.



As we cannot rely on the underlying `\TeX` engine treating UniCode characters natively, symbol names can only consist of printable ASCII characters.

2.4 Documenting Symbols

We can now use the two new macros, `\symname/\sn`, `\symref/\sr` etc. to mark up the above two paragraphs. However, the system does not yet know what to show a reader when hovering over the symbol in the HTML. We can fix that by using the `sdefinition` or `sparagraph` environments.

Ignoring the former for now, which is more useful for mathematical concepts, we can use the following to mark up the first paragraph:

```
\begin{sparagraph}[style=symdoc,for={tex,macro}]
\tex is a document typesetting
software developed by Donald Knuth, with a focus on
mathematical formulae. It is based on a powerful
and extensible \sn{macro} expansion engine.
\end{sparagraph}
```

In general, the `sparagraph environment` can be used to mark up arbitrary paragraphs semantically, but the `style=symdoc` option tells `\TeX` to use this paragraph as a documentation for the symbols provided in the `for=` option.

We just used the semantic macro `\stex` and the `\sn macro` to mark up the fragment – but we can do better. Both concepts are being *introduced* in the above paragraph, and we can let `\TeX` know that that is the case:

Within an `sparagraph environment` with `style=symdoc` (or an `sdefinition environment`), we can mark up *definienda*, meaning the terms *being defined*, explicitly. Analogously to `\symname` and `\symref`, we have the macros `\definame` and `\definiendum` for that purpose.

Note that the `\tex macro` induced by the text symbol above already marks up the “`\TeX`” it produces, so wrapping it in another `\definiendum` would be redundant. However, every text symbol also generates a *second macro* with the suffix `name` that generates a non-marked-up version of the same presentation. In other words, we get the macro `\texname` for free, that produces “`\TeX`” (of course, we could just as well use the `\TeX macro`, but that one you probably know already).

Furthermore, every `\definiendum` or `\definame` automatically adds the symbol being referenced to the internal `for=-list` of the `sparagraph environment`, obviating the need to list it explicitly.

As such, we can produce a better markup like this:

```

\begin{sparagraph}[style=symdoc]
  \definiendum{tex}{\texname} is a document typesetting
  software developed by Donald Knuth, with a focus on
  mathematical formulae. It is based on a powerful
  and extensible \definame{macro} expansion engine.
\end{sparagraph}

```

Exercise

In your archive my/archive, create additional files that produce the following outputs:

Mathematics.en.tex

To do **mathematics** is to be, at once, touched by fire and bound by reason. This is no contradiction. Logic forms a narrow channel through which intuition flows with vastly augmented force.

– Jordan Ellenberg

PDF.en.tex

Portable Document Format (PDF) is a document format that mixes text and graphics with a variety of content.

HTML.en.tex

The **HyperText Markup Language (HTML)** is a representation format for web-pages.

OMDoc.en.tex

OMDoc is a document format for representing **mathematical** documents with their flexiformal semantics.

such that the following file compiles and shows the above snippets on hover:

sTeX.en.tex

```

1 \documentclass{stex}
2 \libinput{preamble}
3 \begin{document}
4 \begin{smodule}{sTeX}
5   \usemodule{OMDoc}
6   \usemodule{PDF}
7   \usemodule{HTML}
8   \textsymdecl{stex}{\sTeX}
9   \begin{sparagraph}[style=symdoc]
10    \definiendum{stex}{\stexname} is a system for generating
11    documents in either \PDF or \HTML format, augmented with
12    computer-actionable semantic information (conceptually)
13    based on the \OMDoc format and ontology.
14  \end{sparagraph}
15 \end{smodule}
16 \end{document}

```

sTeX is a system for generating documents in either **PDF** or **HTML** format, augmented with computer-actionable semantic information (conceptually) based on the **OMDoc** format and ontology.

The preamble of every file should only be

```
\documentclass{stex}
\libinput{preamble}
```

and the macros `\OMDoc`, `\PDF`, `\HTML` should produce `\textsc{OMDoc}`, `\textsf{PDF}` and `\textsf{HTML}`, respectively (but with semantic annotations of course).

Lösung: Can be found in `[sTeX/Documentation]source/tutorial/solution`

2.5 Sectioning and Reusing Document Fragments

We know now how to import and reuse the [symbols](#) of some [module](#) (using `\usemodule`). What about the actual document *content*?

Assume we want to write a new article that includes all of the fragments in `my/archive` we made so far, in a file `all.en.tex` in the same [math archive](#):

```
1 \documentclass{article}
2 \usepackage{stex}
3 \libinput{preamble}
4 \begin{document}
5   \author{Me}
6   \title{The \texttt{my/archive} Archive}
7   \maketitle
8   \tableofcontents
9   ...
10 \end{document}
```

In there, we want sections as follows:

```
- 1 Preliminaries
  (Mathematics)
- 1.1 Document Formats
  (PDF)
  (HTML)
  (OMDoc)
- 2 \TeX and Friends
  (LaTeX)
  (sTeX)
```

We could of course do the following:

```
\section{Preliminaries}
\input{Mathematics.en}
\subsection{Document Formats}
\input{PDF.en}
\input{HTML.en}
\input{OMDoc.en}
\section{\TeX and Friends}
\input{LaTeX.en}
\input{sTeX.en}
```

...but this approach has two drawbacks:

Firstly, we need to manually keep track of the section levels, by explicitly writing `\section`, `\subsection` etc. This is fine as long as we are just interested in this particular article. But what if we want to *reuse* the article’s content in another document at some point? The section levels might be entirely different then – e.g. we might want the “Preliminaries” section to be a subsection instead.

Secondly, the `\input` macro considers the file name/path provided to be either *absolute* or relative to the *current tex file being compiled* – which means that the `\input{Mathematics.en}` only works for files in the same directory as `Mathematics.en.tex`.

In short: using `\section`, `\chapter` etc. explicitly, and `\input` to reuse fragments, breaks reusability.

Instead of using `\section` and `\subsection`, $\text{\TeX}$ therefore provides the `sfragment` environment.

`\begin{sfragment}{Foo}...\end{sfragment}` inserts a sectioning header depending on the current section level and availability. These are: `\part`, `\chapter`, `\section`, `\subsection`, `\subsubsection`, `\paragraph` and `\subparagraph`. This allows us to do the following instead:

```
\begin{sfragment}{Preliminaries}
  \input{Mathematics.en}
  \begin{sfragment}{Document Formats}
    \input{PDF.en}
    \input{HTML.en}
    \input{OMDoc.en}
  \end{sfragment}
\end{sfragment}
\begin{sfragment}{\TeX and Friends}
  \input{LaTeX.en}
  \input{sTeX.en}
\end{sfragment}
```

The only problem remaining now is that if we do this, $\text{\TeX}$ will insert a `\part` for the first `sfragment`. If we want the “top-level” sectioning level to be `\section` instead, we can insert a `\setsectionlevel{section}` in the preamble.

As a more reuse-friendly replacement of `\input`, $\text{\TeX}$ provides the `\inputref` macro. Using that has two advantages: Firstly, its argument is relative to some (optionally provided, or the current) *math archive* and is thus independent of the specific location of the file relative to the currently being compiled `.tex`-file. Secondly, when converting to `HTML`, it will *not* “copy” the referenced file’s content in its entirety (as `\input` would), but instead dynamically insert the already existent (if so) `HTML` of the referenced file, avoiding content duplication and having to process the file all over again.

In general `\inputref[some/archive]{file/path}` inputs the file `file/path.tex` in the *archive* `some/archive`. As the `\input`-ed files in the example above are in the same *archive* anyway, we can simply substitute the `\inputs` by `\inputrefs` and call it a day.

Finally, we can make two more minor changes:

1. The *title* of our document is only supposed to be there, if we compile the document directly – if we were to `\inputref` our file into a “driver file” `all.en.tex`, the title and the table of contents should be omitted.

We can achieve this using the `\ifinputref` conditional: by wrapping the header in an `\ifinputref \else...\fi`, it will only be processed if the file is *not* being loaded using `\inputref`. `\ifinputref` is a “classic” $\text{\TeX}$ conditional and is treated as such in both $\text{\PDF}$ and $\text{\HTML}$ compilation. A smarter macro to use is `\IfInputref`, which takes two arguments for the *true* and *false* cases, respectively. Additionally, when compiling to $\text{\HTML}$, *both* arguments to `\IfInputref` will be processed, and the back-end will decide which of the two to present when serving a document.

2. The table of contents should also be omitted in $\text{\HTML}$ mode. To achieve that, we can use the `\ifstexhtml` conditional, which is *true* if the document is being compiled to $\text{\HTML}$, and *false* if compiled to $\text{\PDF}$.



Note, that since *both* arguments of `\IfInputref` are processed, they should *not* open $\text{\TeX}$ groups or environments!

In summary, we can modify our document to do the following:

```
\IfInputref{}{
  \author{Me}
  \title{The \texttt{my/archive} Archive}
  \maketitle
  \ifstexhtml \else \tableofcontents \fi
}
```

The final `all.en.tex` can be found in `[sTeX/Documentation]tutorial/solution/all.en.tex`.

2.6 Building and Exporting $\text{\HTML}$

So far we know how to write $\text{\sTeX}$ documents, (we assume) how to build $\text{\PDF}$ files from them (via `pdflatex` of course), and on saving documents the $\text{\IDE}$ will preview the generated $\text{\HTML}$. But if we do that with our new `all.en.tex`, we get presented with Figure 10 Where did all of our fragments go?



Figure 10: Missing Fragments in the $\text{\HTML}$ Preview

Well, they don’t exist yet as $\text{\HTML}$. The $\text{\HTML}$ Preview window in the $\text{\IDE}$ is really just that: A *preview*. But when using `\inputref`, it has to find the $\text{\HTML}$ of the `\inputrefed` fragment *somewhere*. Meaning: we have to compile all of the fragments we used to $\text{\HTML}$ first. Individually, we can compile the currently open file and all its dependencies in $\text{\VS Code}$ using the button in Figure 11.

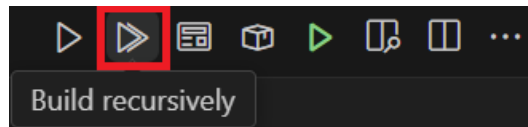


Figure 11: The Build PDF/XHTML/OMDoc Button

This will queue all dependencies of the file in a new build queue and open the [FLaMj](#) dashboard for us (see Figure 12). Clicking on the run button in the dashboard will then

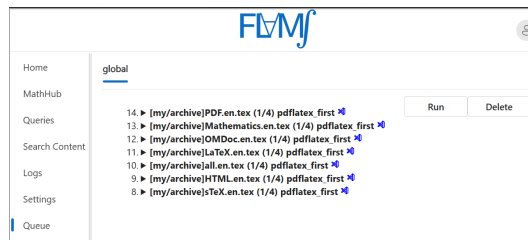


Figure 12: The FLAMS build queue

for every queued file:

1. Run `pdflatex` twice,
2. Convert the file to [HTML](#),
3. Extract all the semantics, and
4. Construct a search index.

Once that's done, saving `all.en.tex` again yields the correct [HTML](#) in the preview window.

At this point, [FLaMj](#) knows about the [HTML](#) and the semantics, and we can look at the [HTML](#) in [VS Code](#) or the [FLaMj](#) dashboard – of course, features like the fancy pop-up windows require a semantically informed back-end infrastructure, in the form of the [FLaMj](#) system. However, [FLaMj](#) can dump a standalone and self-contained [HTML](#) version for you. Let's do that now:

With our `all.en.tex` file open and everything built as above, click the **Export Standalone HTML** button in the [IDE](#) (see Figure 13).

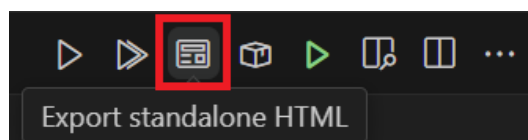


Figure 13: Exporting [HTML](#) in the [IDE](#)

In the dialog box that opens now, select an **empty** directory and [FMMf](#) will dump a standalone version of our `all.en.tex` document there. You will still not be able to open it in the browser directly without issues, because most browsers forbid javascript modules on the `file://` protocol, but opening the file via `http` will yield the desired result, and you can now upload the directory's content to wherever you might want to use it.

If you want to test this, a quick and easy way to do so is to use [VS Code](#): You can install the **Live Server** extension, open the directory and click the **Go Live** button on the lower right of the window, which will start a small web-server in the selected directory and open its `index.html` in the browser for you.

Chapter 3

Mathematical Concepts

So far, we have seen how to declare and reference [symbols](#) generate [semantic macros](#) for [text symbols](#), collect them in [modules](#) and document them properly.

But where [sTeX](#) really shines is when it comes to mathematics and related subject areas: [semantic macros](#) are significantly more useful when used for generating symbolic [notations](#) in math mode, and by associating [symbols](#) with (flexi-)formal semantics, [sTeX](#) can even *check* that your content is (to some degree) formally correct, or at least well-formed.

Alos [sTeX](#) provides specialized functionality for mathematical [statements](#): the text fragments marked as Definition, Theorem, Proof that are iconic to mathematical documents.

The example snippets in this chapter can be found in the [math archive sTeX/MathTutorial](#). If you downloaded the [sTeX/Documentation archive](#) in the [sTeX IDE](#), you already have that [archive](#). If not, you can download it from within the [IDE](#), as described in Part 1 (The Basics) in the [sTeX](#) tutorial.

3.1 Simple [Symbol](#) Declarations

We will start with [symbols](#) and [semantic macros](#) for mathematical concepts and objects and their contribution to mathematical formulae.

3.1.1 [Semantic Macros](#) and [Notations](#)

Let us start with a very fundamental concept; namely [equality](#). As you should by now know, declaring a new [symbol](#) requires a [module](#), so let's open a new one and use `\symdecl`:

```
\begin{smodule}{Equality}
  \symdecl{equal}
\end{smodule}
```

As mentioned in Section 1 ([Modules](#) and Simple [Symbol](#) Declarations), the starred variant `\symdecl*` does not create a [semantic macro](#), so presumably, the variant without a `*` *does*. And indeed, we now have a macro `\equal`, which however will produce errors if we try to use it. That's because we haven't told [sTeX](#) what to do with it yet.

STEX

A **semantic macro** is a $\text{\LaTeX}$ -macro that allows for referencing a **symbol** itself, or – in the case of e.g. a function – the *application* of a **symbol** to (one or multiple) *arguments*; primarily by invoking a **symbol**’s **notation** in *math mode*.

The command `\symdecl{macroname}` declares a new **symbol** with name `macroname` and a **semantic macro** `\macroname`. In the case where we want the name and the **semantic macro** to be distinct, the command `\symdecl{macroname}[name=some name]` declares the name of the **symbol** to be `some name` instead.

The starred variant `\symdecl*{name}` declares the concept with the given name, but does not generate a **semantic macro**.

So let’s provide equality with a **notation**. As a first step, we should let $\text{\TeX}$ know that “`equal`” takes two arguments. We might also want to shorten the **semantic macro** to e.g. `\eq`, without changing the name. Hence:

```
\symdecl{eq}[name=equal,args=2]
```

Next, we add an infix notation with the **notation macro**:

```
\notation{eq}{#1 = #2}
```

That seems like a lot to write, so for the very common case where we want to declare a **symbol** with a **semantic macro** and a **notation** all at once, the `\symdef` macro does all three by combining the optional and mandatory argument of `\symdecl` and `\notation`:

```
\symdef{eq}[name=equal,args=2]{#1 = #2}
```

and indeed, we can now use the `\eq` macro in *math mode* to invoke our new **notation**: `\eq{a}{b}` now yields $a = b$ – notably without any highlighting (and hover interaction in the **HTML**) though. Since our **semantic macro** takes *arguments*, which should be differently highlighted, we need to let our **notation** know which parts of the **notation** are highlightable components.

We can do so with the `\comp` and `\maincomp` macros:

STEX

The `\comp`-macro marks components to be highlighted in a **notation** for a **symbol** taking (one or more) arguments.

This is necessary because it is (nearly) impossible for $\text{\LaTeX}$ to figure out, which parts of a **notation** to highlight and which not on its own – in particular, the highlighting should stop for the *arguments* of a **semantic macro**.

Additionally, the `\maincomp` macro can be used to mark (at most) one **notation** component to represent the *primary* component of the **notation**.

Notations that do not take arguments, as well as **operator notations**, are automatically wrapped in `\maincomp`.

In our case, this applies only to the “`=`”, symbol, so:

```
\symdef{eq}[name=equal,args=2]{#1 \mathrel{\maincomp{=}} #2}
```



You may be wondering about the role of the `\mathrel` macro in the example above: $\text{\TeX}$ determines spacing/kerning in *math mode* by assigning a *class* to every

character. Both individual characters and whole subexpressions can be assigned one of these classes using dedicated macros. These are:



class	T_EX macro	examples
ordinary (default class)	<code>\mathord</code>	$\alpha \ i \ \Diamond$
large operator	<code>\mathop</code>	$\sum \prod \int$
opening	<code>\mathopen</code>	$([\langle$
closing	<code>\mathclose</code>	$)] \rangle$
binary relation	<code>\mathrel</code>	$\leq > =$
binary operator	<code>\mathbin</code>	$+ \cdot \circ$
punctuation	<code>\mathpunct</code>	$, ;$

T_EX “forgets” the class of an expression if it is wrapped in a `\comp` macro. It is therefore a good idea to wrap any occurrence of a `\comp` in the corresponding T_EX macro for the desired class (e.g. `\mathrel{\comp{\leq}}`).

Having done so, we can now type `\eq{a}{b}` to get $a = b$. Thanks to using `\maincomp`, we now also have an **operator notation**, which we can invoke using `\eq!`, yielding $=$.

What if we want to add more **notations**? Say we want to be able to invoke **equality** to get the variant notation $a \equiv b$ (without changing the intended meaning). If we want to be able to choose one of several **notations**, we should give the **notation** an *identifier*.

Let’s again modify our earlier **notation** by adding the identifier `eq` to the optional arguments of `\symdef`, like so:

```
\symdef{eq}[name=equal,args=2,eq]{#1 \mathrel{\maincomp{=}} #2}
```

We can now invoke the specific **notation** provided here by writing `\eq[eq]{a}{b}` to the same effect. But we can also add more **notations** using the `\notation` macro:

```
\notation{eq}[equiv]{#1 \mathrel{\maincomp{\equiv}} #2}
```

which we can now invoke with `\eq[equiv]{a}{b}`, yielding $a \equiv b$.

By default, the *first* **notation** provided for a given **symbol** is considered the *default notation*, which is invoked if the **semantic macro** is used without an optional argument – hence, `\eq{a}{b}` still yields $a = b$.

If we use the starred variant of the `\notation` macro, the **notation** is set as the new default. Hence, had we done

```
\notation*{eq}[equiv]{#1 \mathrel{\maincomp{\equiv}} #2}
```

then `\eq{a}{b}` would now yield $a \equiv b$.

Any already existing notation can be set as default using the `\setnotation` macro; e.g. instead of using `\notation*`, we could also do

```
\notation{eq}[equiv]{#1 \mathrel{\maincomp{\equiv}} #2}
\setnotation{eq}{equiv}
```

Exercise

Implement the **symbol** “equal” as above in a new **module** “Equality” and add a documentation such that hovering over the **symbol** in the **HTML** yields the following snippet:

Two objects a, b are considered **equal** (written $a = b$ or $a \equiv b$), if there is no property that distinguishes them.

Lösung: Can be found in [sTeX/MathTutorial]/mod/Equality1.en.tex

3.1.2 Types and Variables

sTeX Every **symbol** or **variable** can be assigned a **type**, signifying what “kind of object” the **symbol** represents, or what (primary) set it is contained in.

Types are particularly useful for *variables*:

sTeX A **variable** represents a *generic* or *unspecified* object.
Variables can be declared using the `\vardef`-macro, whose syntax is analogous to `\symdef`.
Note that **variables** are local to the current T_EX-group (e.g. environment).

Let’s leave our equality-module aside for now and turn our attention to something simpler: **natural numbers**. Consider the following module:

Example 5

Input:

```
\begin{smodule}{Nat}
\symdef{Nat}[name=natural numbers]{\mathbb N}
\begin{sparagraph}[style=symdoc]
  The \definame{Nat} $\defnotation{\Nat}$ are the numbers
  $0,1,2,\dots$
\end{sparagraph}
\symdef{plus}[name=addition,args=2]{#1 \mathbin{\maincomp{+}} #2}
\begin{sparagraph}[style=symdoc]
  \Definame{addition} $\defnotation{\plus{a}{b}}$
  refers to the process of adding two \sn{Nat}.
\end{sparagraph}
\end{smodule}
```

Output:

The **natural numbers** $\mathbb{N}$ are the numbers 0,1,2,...
Addition $a+b$ refers to the process of adding two **natural numbers**.

(like `\definame` and `\definiendum`, the `\defnotation` macro is only allowed in documenting environments like `sparagraph[style=symdoc]` or `sdefinition`, and highlights the **notation** components marked with `\comp` or `\maincomp` the same way as `\definame` and `\definiendum` do.)

Note, that as the `\Nat` semantic macro does not take any arguments, we do not need to wrap the notation in a `\comp` or `\maincomp`.

The above fragment uses two variables a and b . In fact, `FLMj` will consider them variables even though they are not marked up as such – but since they are not marked up, we are missing out on useful functionality.

Let’s change that by adding two variable definitions¹:

Example 6

Input:

```
\begin{sparagraph}[style=symdoc]
\vardef{va}[name=a]{a}\vardef{vb}[name=b]{b}
\Definame{addition} $\defnotation{\plus{va}{vb}}$
  refers to the process of adding two \sn{Nat}.
\end{sparagraph}
```

Output:

Addition $a+b$ refers to the process of adding two natural numbers.

Okay, so now a and b are gray, but besides that, we haven’t achieved much yet. Let’s change that by giving the variables the type $\mathbb{N}$:

Example 7

Input:

```
\begin{sparagraph}[style=symdoc]
\vardef{va}[name=a,type=\Nat]{a}\vardef{vb}[name=b,type=\Nat]{b}
\Definame{addition} $\defnotation{\plus{va}{vb}}$
  refers to the process of adding two \sn{Nat}.
\end{sparagraph}
```

Output:

Addition $a+b$ refers to the process of adding two natural numbers.

Now if we hover over the a and b (in the HTML), it will show us that their type is $\mathbb{N}$!

We can of course also assign types to symbols. In the IDE, find the symbol “function space” with semantic macro `\funspace` (in `[sTeX/MathBase/Functions]{mod?Function}`). The OMDoc preview window shows you how to use this symbol (Figure 14). This tells us that if we write `\funspace{a_1, \dots, a_n}{b}` (depending on which notation we use), we will get $a_1 \times \dots \times a_n \rightarrow b$.

We want addition to have type $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$, hence we do:

```
\symdef{plus}[name=addition,args=2,
  type=\funspace{\Nat,\Nat}{\Nat}
]{#1 \mathbin{\maincomp{+}} #2}
```

¹Technically, this is called a *variable reservation*, for those in the know.

Symbol function space (`\funspace{a_1^1, \dots, a_1^n}{i_2}`)
of type `bind( (A,B),SET)`

Notations: $a_1^1 \rightarrow \dots \rightarrow a_1^{i_1} \rightarrow i_2$ $a_1^1 \times \dots \times a_1^{i_1} \rightarrow i_2$ $a_1^1 \rightarrow \dots \rightarrow a_1^{i_1} \rightarrow i_2$ $a_1^1 \times \dots \times a_1^{i_1} \rightarrow i_2$

▼ Associated Paragraphs

Figure 14: Syntax Preview



So far (and when using the `use` button in the IDE), we have been using the `\usemodule` macro to import content. `\usemodule` is allowed anywhere and imports the referenced `module` content local to the current `TeX` group. Now that we use imported `symbols` in `types` (and since we are *in* a `module`), we need to make sure that the imported `modules` are also (transitively) *exported*, since our new `symbols` now *depend* on the imported `module`. For that we use the `\importmodule` macro within the `module`; i.e. the file should now look something like this:

```
\begin{smodule}{Nat}
  \importmodule[sTeX/MathBase/Functions]{mod?Function}
  ...
```

Note that the `HTML` is aware of this now (after you save): *Clicking* on any occurrence of `addition` now yields Figure 15.

addition

Select Definition or Example

Addition $a+b$ refers to the process of adding two `natural numbers`.

Force Notation

▼ Formal Details

Symbol addition (`\plus{a_1^1, \dots, a_1^n}`)
of type `map(N,N)`

Notations: $a_1^1 + \dots + a_1^{i_1}$

Figure 15: On-Click Popup in the `HTML`

By virtue of using `[sTeX/MathBase/Functions]{mod?Function}`, we also imported `[sTeX/MathBase/Sets]{mod?Set}`, which gives us the “*collection*” `symbol`. Let’s use this as a `type` for the `natural numbers`:

```
\symdef{Nat}[name=natural numbers,type=\collection]{\mathbb N}
```

3.1.3 Flexary Macros and Argument Modes

Here is one thing you might wonder: Writing `\plus{a}{b}` is one thing, but what if we want to produce $a + b + c + d + e$? Do we really need to write `\plus{a}{\plus{b}{\plus{c}{\dots}}}`?

Of course not. We can declare the `symbol` such that the `semantic macro` `\plus` expects a (comma-separated) *sequence* of arguments instead of two “normal” arguments.

The optional `args`-argument of `\symdecl` expects a string of characters indicating the semantic macro's **argument modes**. There are four such **modes**:

- i a **simple argument**,
- a a – (left or right) associative – **sequence argument**, represented as a single TeX-argument `{a,b,...}`,
- B A **binding argument** that expects a variable that is bound by the symbol in its application, and
- B A **binding sequence argument** of arbitrarily many bound variables by the symbol `{x,y,z,...}`.

If `args` is given as a number n instead, the semantic macro takes n arguments of mode `i`.

Example 8

- For `\plus{a,b,c}` yielding $a + b + c$, we do `\symdecl{plus}[args=a]`,
- for `\inset{a,b,c}{A}` yielding $a, b, c \in A$, we do `\symdecl{inset}[args=ai]`,
- in `\add{i}{1}{n}{f(i)}` yielding $\sum_{i=1}^n f(i)$, the variable i is **bound** in the expression, we hence do `\symdecl{add}[args=biii]`,
- in `\forall{x,y,z}{P(x,y,z)}` yielding $\forall x, y, z. P(x, y, z)$, the variables x, y, z are all **bound** by the $\forall$, we hence do `\symdecl{forall}[args=Bii]`.

So when we wrote `\symdecl{plus}[args=2]`, this was actually shorthand for `\symdecl{plus}[args=ii]`.

Let's revise our previous declaration and the syntax of the `\plus` macro:

```
\symdef{plus}[name=addition,args=a,
  type=\funspace{\Nat,\Nat}{\Nat}
]{#1 \mathbin{\maincomp{+}} #2}
\begin{sparagraph}[style=symdoc]
  \vardef{va}[name=a]{a}\vardef{vb}[name=b]{b}
  \Definame{addition} $\defnotation{\plus{\va,\vb}}$
  refers to the process of adding two \sn{\Nat}.
\end{sparagraph}
```

Now we get new errors, that are easy to explain: Our **notation** `{#1 \mathbin{\maincomp{+}} #2}` refers to *two* arguments, but our semantic macro only takes *one* (albeit a **sequence argument**). We now need to let STEX know what to do with the **sequence argument** in our **notation**. Using the `\argsep` macro, we can tell STEX to insert the *separator* “+” between the individual elements of the **argument sequence** #1:

```
\symdef{plus}[name=addition,args=a,
  type=\funspace{\Nat,\Nat}{\Nat}
]{\argsep{#1}{\mathbin{\maincomp{+}}}}
```

TODO² TODO³

Now we can finally write `$(\plus{a,b,c,d,e})$` and get $a + b + c + d + e$ – hooray!

²TODO: argmap

³TODO: argarraymap

Exercise

Analogously to the above, implement a symbol “multiplication” with semantic macro `\mult`, that takes a single sequence argument and has a default notation such that `\mult{a,b,c}` produces $a \cdot b \cdot c$.

Lösung: Can be found in [sTeX/MathTutorial]mod/Nat.en.tex

3.1.4 Precedences

If you have done the previous exercise, you now have semantic macros `\plus` and `\mult` at your disposal. We can of course nest them to produce e.g. $a + b \cdot c$ (with `$(\plus{a,\mult{b,c}})$`). If we do `$(\mult{a,\plus{b,c}})$` however, we get $a \cdot b + c$. Annoying – we now have to insert parentheses: `$(\mult{a,(\plus{b,c}))}$`... or do we?

We do *not*. Instead, we can assign *precedences* to *notations* to have sTeX insert parentheses automatically.

sTeX

`\notation` (and hence `\symdef`) take an optional argument `prec=<opprec>;<argprec1>x...<argprec n>` consisting of an **operator precedence** `<opprec>` and for each argument `k` an **argument precedence** `<argprec k>`.

All *precedences* are integers, e.g. 10 or -500. It is good practice to use *precedences* that leave enough room to smuggle values inbetween, so that we can fine-tune them later as more symbols may intervene.

The precise numbers used for *precedences* are arbitrary – what matters is which *precedence* is higher than which other *precedence* when used together.

By default, all *precedences* are 0, unless the symbol takes no arguments, in which case the **operator precedence** is `\neginfp` (negative infinity).

If we only provide a single number in `prec=`, this is taken as both the **operator precedence** and all **argument precedences**.

The *lower* a *precedence*, the *stronger* a *notation* binds its arguments. In our particular case, we want *multiplication* to bind stronger than *addition*, so we can (arbitrarily) assign them *precedences* e.g. 10 and 20:

```
\symdef{plus}[name=addition,args=a,assoc=bin,prec=20,
  type=\funspace{\Nat,\Nat}{\Nat}
]{\argsep{#1}{\mathbin{\maincomp{+}}}}
\symdef{mult}[name=multiplication,args=a,assoc=bin,prec=10,
  type=\funspace{\Nat,\Nat}{\Nat}
]{\argsep{#1}{\mathbin{\maincomp{\cdot}}}}
```

And now if we type `$(\mult{a,\plus{b,c}})$`, sTeX will automatically insert parentheses and yield $a \cdot (b + c)$ – and conversely, if we do `$(\plus{a,\mult{b,c}})$`, sTeX will *not* insert parentheses and yield $a + b \cdot c$.

3.1.5 Finishing Equality

You might wonder if – as with `addition` – we can make “`equal`” take a `sequence` argument as well. Naturally, we can:

```
1 \symdef{eq}[name=equal,args=a,eq,
2   type=\funspace{\vA,\vA}{\prop}
3 ]{\argsep{#1}{\mathrel{\maincomp=}}}
4 \notation{eq}[equiv]{\argsep{#1}{\mathrel{\maincomp\equiv}}}
```

3.1.6 Variable Sequences

There is a special kind of `variable` in `STeX` for when we want to use *sequences* of `variables`.

We can use the `\varseq` macro to declare a new sequence `variable`; in the simplest case that looks something like the following:

```
\varseq{seqn}[name=n,type=\Nat]{1,\ellipses,k}{\maincomp{n}_{#1}}
```

We have just declared a new variable sequence of `type` `N`, that ranges over indices $1, \dots, k$, with `notation` n_i for some specific index i .

If we now do `\seqn{i}`, we get n_i , and if we do `\seqn!`, we get $n_1, \dots, n_k$.

We can also do multi-dimensional sequences, e.g.

```
\varseq{seqm}[name=m,type=\Nat,args=2]
{\{1\}{1},\ellipses,{\ell}{k}}
{\maincomp{m}_{#1}^{\#2}}
```

Now `\seqm{i}{j}` produces m_i^j , and `\seqm!` produces $m_1^1, \dots, m_\ell^k$.

Of course, we can manually change the way `\seqn!` is typeset by providing an explicit `operator notation` using `op=`; e.g. if we do

```
\varseq{seqn}[name=n,type=\Nat,op={\{n_i\}_{i=1}^k}]
{1,\ellipses,k}{\maincomp{n}_{#1}}
```

then `\seqn!` produces $(n_i)_{i=1}^k$.

So far so nice, but sequence variables get especially useful in combination with `sequence arguments`: Consider for example the `\plus semantic macro` for `addition`. This expects one `sequence argument`, or alternatively, a *sequence variable*: `\plus{\seqn}` now produces $n_1 + \dots + n_k$, and `\eq{\seqm}` now produces $m_1^1 = \dots = m_\ell^k$.

TODO⁴

3.2 Statements

Now that we have `equality`, `natural numbers`, `addition` and `multiplication` at our disposal, let’s implement some *statements*. Both `addition` and `multiplication` are, for example, *associative* and *commutative*.

We could state these properties directly for the two operations, but we can also first define *associativity* and *commutativity* in general, and then assert them specifically for `addition` and `multiplication`.

⁴TODO: `seqmap`

3.2.1 Definitions

Let's define what it means to be *associative*. This means, of course, declaring a new *symbol*. Note that we don't need a *semantic macro* for *associativity*, since there is no *notation* to attach to it. We will also for now ignore its *type*. Note however, that *associativity* is still a property of (binary) operations, so it still makes sense to have the *symbol* take an *argument*; namely the operation it applies to.

We will also finally provide an actual (more or less) formal *definition* for the *symbol*, so where we used the *sparagraph environment* with `style=symdoc` before, we will now use the *sdefinition environment*, which also gives us `\definame`, `\definiendum`, `\defnotation` and all that.

A first variant of a corresponding *module* could look like this:

Example 9

Input:

```

File [sTeX/MathTutorial]props/Associative1.en.tex
4 \begin{smodule}{Associative}
5   \importmodule{mod?Equality}
6
7   \symdecl*{associative}[args=1]
8   \begin{sdefinition}[for=associative]
9     \vardef{vA}[name=A,type=\collection]{A}
10    \vardef{vop}[name=op,type=\funspace{\vA,\vA}\vA,args=a,assoc=bin]
11      {\argsep{#1}{\mathbin{\maincomp{\circ}}}}
12    %
13    A binary operation  $\fun{\vop!}{\vA,\vA}\vA$  is called
14    \definame{associative}, if
15     $\eq{
16      \vop{(\vop{a,b}),c},
17      \vop{a,(\vop{b,c})}
18    }{
19      \text{for all } \inset{a,b,c}\vA.
20    }$ 
21    \end{sdefinition}
22 \end{smodule}

```

Output:

Definition 3.2.1. A binary operation $\circ : A \times A \rightarrow A$ is called **associative**, if $(a \circ b) \circ c = a \circ (b \circ c)$ for all $a, b, c \in A$.

Note, that the *semantic macros* `\fun` and `\inset` come from `[sTeX/MathBase/Functions]mod?Function` and `[sTeX/MathBase/Sets]mod?Set`, respectively. Also note, that the *variable* declaration for `\vop` makes use of all the fun features we already discussed for *addition*.



Note that the above is more than good enough, if you merely want to produce nice-looking, “wikified” *HTML* and *PDF* documents. The rest of this subsection will cover how to add more flexiformal semantics to the above. If this seems laborious and/or difficult, keep in mind that this is to some degree experimental still, and you are not forced to go overboard with semantic annota-



tions!

But if you aim to create a “library of symbols” for mathematical concepts, then all of the possibilities that we discuss here will add value for the community. Generally, the higher the ratio of readers to authors the more any investment in semantization will pay off.

Semantic Macros in Text Mode

The first thing we can do to further improve this document is marking up the “for all” in the definition – after all, there naturally is a [symbol](#) for the [universal quantifier](#), which can be found in `[sTeX/Logic/General]mod/syntax?UniversalQuantifier` and has the [semantic macro](#) `\forall` (as to not conflict with the [TeX](#) primitive `\forall`).

The naive approach would be to replace the “for all” by e.g. `\sr{forall}{for all}`. That would (correctly) associate and highlight the text fragment with the [symbol](#) “[universal quantifier](#)”, *but* we are not just referencing the [symbol](#) here – we are actually using it, by *applying* it to the [variables](#) a, b, c and the expression $(a \circ b) \circ c = a \circ (b \circ c)$.

In *math mode*, we can just use the [semantic macro](#) `\forall` – that will take two arguments (of [modes](#) B and i) and produce the corresponding [notation](#), so that

```
$\forall{\inset{a,b,c}{\vA}}{\eq{\vop{(\vop{a,b}),c} , \vop{a,(\vop{b,c})}} }
}$
```

will produce $\forall a, b, c \in A. (a \circ b) \circ c = a \circ (b \circ c)$.

In *text mode*, however, we don’t have a specific [notation](#) – instead, the specific “[notation](#)” is whatever sentence we want to mark up semantically. In text mode, [semantic macros](#) therefore behave differently:

1. They take *precisely* one argument, regardless of how many arguments the [macro](#) would take in math mode or (equivalently) the `args` property of the [symbol](#).
2. *Within* that argument, we can use `\comp` to highlight arbitrary text fragments, and
3. we can use the `\arg` [macro](#) to mark up the *actual* arguments that the [symbol](#) is supposed to be applied to.

`\arg` takes as optional argument the index of the argument that is being marked up; if not they are used consecutively. The starred variant `\arg*` produces no output.

So we could now do

```
\forall{\comp{For all} $\arg{\inset{a,b,c}{\vA}}$, we have
$\arg{
\eq{\vop{(\vop{a,b}),c} , \vop{a,(\vop{b,c})}}
}$
}
```

which produces “For all $a, b, c \in A$, we have $(a \circ b) \circ c = a \circ (b \circ c)$ ”.

In our case though, we want to “switch the arguments around” – first comes the equation, then the [variables](#) to be bound. Hence:


```

\foral{
  $\arg[2]{
    \leq{ \vop{(\vop{a,b}),c}, \vop{a,(\vop{b,c})} }
  }$
  \comp{for all}
  $\arg[1]{ \inset{a,b,c}{\vA} }$
}

```

which produces “ $(a \circ b) \circ c = a \circ (b \circ c)$ for all $a, b, c \in A$ ”.

Definientia

Now we have a fully semantically annotated expression in the definition for “associative”. Can we let MMT know, that this expression really is *the* definition of the symbol?

Yes, we can. All we need to do is wrap the sentence in a `\definiens` macro (plural: *definientia*; like the word “*definiendum*” refers to “the term being defined”, “*definiens*” refers to “the thing the term is being defined as”).

The `\definiens` macro is only allowed within the `sdefinition` environment, and requires that the `environment` lists the `symbol` that gets the `definiens` attached explicitly in its `for=` argument. It is possible to attach `definientia` to multiple `symbols` within an `sdefinition` environment, in which case the symbol needs to be provided as an optional argument, e.g. we could do `\definiens[associative]{...}`. Since “associative” is the only `symbol` being defined in our definition, we can omit that argument.

Alternatively, as with `types` we can attach `definientia` to a `\symdecl` directly using the optional argument `def=...`.

At this point, you might justifiably wonder, why we even still need to declare `associative` with `\symdecl*` before we define it. And indeed, we don’t – the `sdefinition` environment takes the same optional arguments as the `\symdecl` macro, and if we explicitly provide a `name=` (or a `macro=`), it will generate a `symbol` for us. We can hence get rid of the `\symdecl*` and instead do:

```

1 \begin{sdefinition}[name=associative,args=1]
2   ...
3 \end{sdefinition}

```

One more problem remains: We stated that `associative` is to take one argument – but we haven’t told `TEX` what it is yet. In our case, the argument is represented by the `variable` `\vop`. In fact, chances are that arguments to symbols in `types` or `definientia` are almost always represented by some `variable`.

We can use one of two ways to a `variable` as being an argument:

1. If the `variable` (e.g. `\vop` with name `op`) was already declared prior to the `sdefinition` environment, we can use the `\varbind` macro in the `environment`; e.g. by adding `\varbind{op}`.
2. We can move (or copy) the `\vardef` for the `variable` into the `environment` and add `bind` to its optional arguments.

In total, our fully annotated definition now looks like this:

Example 10

Input:

```

File [sTeX/MathTutorial]props/Associative.en.tex
8 \begin{sdefinition}[name=associative,args=1]
9 \vardef{vA}[name=A,type=\collection]{A}
10 \vardef{vop}[name=op,type=\funspace{\vA,\vA}\vA,
11 args=a,assoc=bin,bind % <- argument for the symbol
12 ]{\argsep{#1}{\mathbin{\maincomp{\circ}}}}
13 \vardef{va}[name=a,type=\vA]{a}
14 \vardef{vb}[name=b,type=\vA]{b}
15 \vardef{vc}[name=c,type=\vA]{c}
16 %
17 A binary operation  $\fun{\vop!}{\vA,\vA}\vA$  is called
18 \define{associative}, if
19 \definiens{\foral{\arg[2]{\eq{
20 \vop{(\vop{va,vb}),vc},
21 \vop{va,(\vop{vb,vc})}}
22 }}{\comp{for all} \arg[1]{\inset{va,vb,vc}\vA}}}.
23 \end{sdefinition}
24 %

```

Output:

Definition 3.2.2. A binary operation $\circ : A \times A \rightarrow A$ is called **associative**, if $(a \circ b) \circ c = a \circ (b \circ c)$ for all $a, b, c \in A$.

And indeed, if we look at the [OMDOC](#) tab of the [HTML](#) preview, we can see that not only does [MMT](#) attach the `definiens` to the `symbol`, it has also inferred the `type` of “`associative`” from the `definiens` (Figure 16).

▼ Symbol `associative`

Definiens

$$\{A: \text{SET}\}_I (\circ: A \rightarrow A \rightarrow A) \rightarrow \forall a, b, c: A. \left(\frac{(a \circ b) \circ c = a \circ (b \circ c)}{A} \right)$$

Type

$$\{A: \text{SET}\}_I (\circ: A \rightarrow A \rightarrow A) \rightarrow \text{Prop}$$

Figure 16: Type Inferred from `Definiens`

Using Symbols Without Semantic Macros and Exporting Code in Modules

So now we don’t have a `semantic macro` for “`associative`”, but it *does* take an argument. How can we ever actually *use* the `symbol` now?

The answer is: with the `\symuse` macro. Like `\symref` and friends, `\symuse` takes a `symbol` name or the name of its `semantic macro` as argument, but behaves otherwise like using a `semantic macro` directly. So for, say, `addition`, `\symuse{addition}` and `\symuse{plus}` behave exactly like `\plus`.

In our case, this means we can do `\symuse{associative}`. “`associative`” does not have a `notation`, but in practice, we say something like “`+ is associative`” rather than using some specific mathematical `notation` for the same thing.

Combining this with what we just learned, we can now say that `addition` is `associative` by doing:

```
\symuse{associative}{\arg{\plus!}$ \comp{is associative}}
```

In fact, we would do the exact same thing every time we want to say that *some* operator is associative, so it makes sense to introduce a `macro` for this. In fact, such a `macro` is easy to define using standard `LATEX` methods. This is where `\STEXexport` becomes very handy:

In a `module`, we can put arbitrary `LATEX` code in an `\STEXexport`, and this code will be executed every time the `module` is imported via `\usemodule` or `\importmodule`. This is especially useful for `macro` definitions, and this way `modules` can almost act like `LATEX packages`!

So we can define a new `macro` `\isassociative` that applies “`associative`” to an arbitrary operation and produces the semantically marked-up text “`#1 is associative`”, and wrap that `macro` definition in an `\STEXexport`, and whenever we use the `Associative module`, we also get the `\isassociative macro`:

```
\STEXexport{
  \def\isassociative#1{
    \symuse{associative}{\arg{#1} ~is ~\comp{associative}}
  }
}
```

And now, we can do e.g. `\isassociative{\plus!$}` to produce “`+ is associative`”.



For technical reasons, `\STEXexport` processes its content in the `expl3` category code scheme – what this means is that all spaces are ignored entirely, and the characters `_` and `:` are valid characters in `macro` names.

In practice, this means you will have to use the `~` character for spaces, and if you want to use a subscript `_`, you should use the `macro` `\c_math_subscript_token` instead.

Exercise

Analogously to all the above, implement a `module` for *commutativity*; i.e the property of a binary operation that $a \circ b = b \circ a$ for all a, b . Make the `module` export a `macro` `\iscommutative` analogously to `\isassociative`.

Lösung: Can be found in `[sTeX/MathTutorial]props/Commutative.en.tex`

TODO⁵

3.2.2 Assertions

Having defined `associativity` and `commutativity`, we can now assert that both properties hold for `addition` and `multiplication`.

For *assertions* (i.e. theorems, lemmata, axioms, claims,...), `sTeX` provides the `sassertion environment`.

In the simplest case, that can look like the following:

⁵TODO: intent?

```

\begin{sassertion}
  \isassociative{\Sn{plus}}
\end{sassertion}

```

which yields

Addition is associative

Do we want this to be typeset as a **Theorem**? For that we just add a `[style=theorem]` to the `sassertion` environment, provided we have a customization for that – (see chapter 9). We can also load the `stexthm` package, which uses the `amsthm` package to provide common typesettings for the types: `theorem`, `observation`, `corollary`, `lemma`, `axiom` and `remark`.

So far, this is not too useful – after all, we could have just as well used e.g. the `amsthm` package and gone straight for the non- $\text{\LaTeX}$ variant

```

\begin{theorem}
  \isassociative{\Sn{plus}}
\end{theorem}

```

But as with `sdefinition`, we can immediately add a corresponding `symbol` in the `sassertion` environment, and have it be documented directly by the environment:

```

\begin{sassertion}[style=theorem,name=addition is associative]
  \isassociative{\Sn{plus}}
\end{sassertion}

```

And now, if we do `\sn{addition is associative}`, we get `addition is associative` with a corresponding hover pop-up (in the `HTML`).

Of course, the usefulness of this grows with more elaborate assertions. For very short assertions such as the above, we might not even want to typeset them in such a space hungry manner.

For that purpose, we provide the `\inlineass` macro (and analogously: `\inlinedef` for `sdefinition`), which takes the same optional arguments as the environment. So we could also do:

```

\inlineass[name=addition is associative]{\isassociative{\Sn{plus}}}

```

So far, `MMT` is blissfully unaware of the semantic contents of our assertions. We can easily remedy that by wrapping the expression representing the assertion in a `\conclusion` macro, analogously to the `definiens` macro in `sdefinitions`:

```

\inlineass[name=addition is associative]{
  \conclusion{\isassociative{\Sn{plus}}}
}

```

We can now see the statement in the `OMDoc` tab of the `HTML` preview (Figure 17).

Assertion	assertion
Term term	associative (+)

Figure 17: Assertion Statement in `OMDoc`

Not exactly pretty – the OMDoc tab uses notations to render content, and we did not provide any for associative.

3.2.3 Proofs



$\text{\LaTeX}$ provides the `sproof` environment for marking up *proofs*. The markup mechanism for `sproof` is still highly experimental and likely subject to change in the near future. As such, we omit a closer explanation of its usage until the syntax and functionality have sufficiently stabilized.

3.3 Mathematical Structures

A common concept in mathematics is that of a *mathematical structure* – a *tuple* of interdependent components. For example: A *monoid* is a *structure* $\langle M, \circ, e \rangle$ such that certain axioms hold; where M is a set, $\circ$ is a binary operation, and $e \in M$.

From a representational perspective, this is particularly interesting: M , $\circ$ and e in the above are not *symbols* in the same way that the previous *symbols* we considered were – they don’t represent definite objects. Instead, they are *components* of some other object, namely a monoid; where a *particular* monoid could either be a fixed object (such as $\langle \mathbb{Z}, +, 0 \rangle$) or an *indefinite* monoid; i.e. a *variable*. We call the components of a *mathematical structure* **fields**.

In this section, we will discuss how to declare and use *mathematical structures* in $\text{\LaTeX}$, build them up modularly, and connect them among each other to avoid duplication.

We will do so by considering *lattices* both algebraically and order-theoretically, and identify the two perspectives.

3.3.1 Declaring and Using Structures

The simplest kinds of *structures* are *magmas* and (*directed*) *graphs*, so we might as well start there:

Definition 3.3.1. A **magma** is a *structure* $\langle U, \circ \rangle$, where U is a *collection* and $\circ$ a binary operation $U \times U \rightarrow U$.

The obvious start is to create a new *module* `Magma`. Within this *module*, we import the `Functions` *module* so we can later assign a *type* to the operation. We can then use the `mathstructure` environment, that creates a new *symbol* “magma”:

```
\begin{smodule}{Magma}
\importmodule[sTeX/MathBase/Functions]{mod?Function}
\begin{mathstructure}{magma}
...
\end{mathstructure}
\end{smodule}
```

`mathstructure` behaves very similarly as `smodule` – within the `environment`, we can declare new `symbols`, `notations` and all that.

So within the `mathstructure`, we can add `symbols` for the two fields U and $\circ$:

```
\symdef{univ}[name=universe,type=\collection]{U}
\symdef{op}[name=operation,args=a,assoc=bin,
type=\funspace{\univ,\univ}\univ
]{\argsep{#1}{\mathbin{\maincomp{\circ}}}}
```

Once we close the `environment` (with `\end{mathstructure}`), the `symbols` are “gone”. However, we now have a new `symbol` “magma” with `semantic macro` `\magma`. Its usage is somewhat more complicated than “normal” `semantic macros`, but one thing we *can* do with it now is `\magma!`, which will produce $\langle U, \circ \rangle$.

Notably however, the `\magma` `macro` is already available *within* the `mathstructure environment` as well.

This allows us to provide an `sdefinition` using the `semantic macros` declared in the `structure`:

Example 11

Input:

File [sTeX/MathTutorial]algebra/Magma.en.tex

```
7 \begin{mathstructure}{magma}
8 \symdef{univ}[name=universe,type=\collection]{U}
9 \symdef{op}[name=operation,args=a,assoc=bin,
10 type=\funspace{\univ,\univ}\univ
11 {\argsep{#1}{\mathbin{\maincomp{\circ}}}}
12
13 \begin{sdefinition}[for={magma,univ,op}]
14 A \definame{magma} is a \sr{mathstruct}{structure} $\magma!$,
15 where $\univ$ is a \sn{collection} and $\op!$
16 a binary operation $\funspace{\univ,\univ}\univ$.
17 \end{sdefinition}
18 \end{mathstructure}
```

Output:

Definition 3.3.2. A **magma** is a `structure` $\langle U, \circ \rangle$, where U is a `collection` and $\circ$ a binary operation $U \times U \rightarrow U$.

Instantiating Structures

More importantly however, we can now declare a `variable magma`, using the optional `return=` argument. For example, we can now do

```
\vardef{vM}[name=M,return=\magma]{M}
```

and we get the `semantic macro` `\vM` with which we can do the following:

Syntax	Result
$\$ \backslash vM! \$$	M
$\$ \backslash vM\{\} \$$	$\langle U_M, \circ_M \rangle$
$\$ \backslash vM\{\text{univ}\} \$$	U_M
$\$ \backslash vM\{\text{op}\}! \$$	$\circ_M$
$\$ \backslash vM\{\text{op}\}\{a,b,c\} \$$	$a \circ_M b \circ_M c$

In other words: Given a [symbol](#) or [variable](#) with [semantic macro](#) `\foo` and `return=\struct`, then `\foo{<fn>}` behaves like the [semantic macro](#) `\fn` *within* the [mathstructure environment](#) for `struct` – but instantiated for the specific instance `foo`.

By default, [S_{TE}X](#) attaches the [symbol](#)’s (or [variable](#)’s) [operator notation](#) as a subscript suffix to the notation component marked with `\maincomp` – e.g., since the “`\circ`” in the [notation](#) for `op` is marked with `\maincomp`, doing `\$ \backslash vM\{\text{op}\}\{a,b\} \$` ultimately outputs `a \circ_{\backslash vM!} b`. Hence, we get $a \circ_M b$.

We can change the way the `\maincomp` notation component is modified, by using the optional argument `comp=` in the [semantic macro](#) for the [mathematical structure](#). For example, to not change it at all, we can do:

```
\vardef{vM}[name=M,return={\magma[comp=##1] }]{M}
```

...or to suffix it with a `'`, we can do

```
\vardef{vMp}[name=Mp,return={\magma[comp=##1'] }]{M'}
```

This allows us to do things like:

```
Let \$ \backslash vM! := \backslash vM\{\} \$ and \$ \backslash vMp! := \backslash vMp\{\} \$ \sns{magma}. Then...
```

yielding

Let $M := \langle U, \circ \rangle$ and $M' := \langle U', \circ' \rangle$ [magmas](#). Then...

We can also *assign* fields to (arbitrary) expressions, by doing `name=<tex>` in square brackets. For example we can do the following:

```
\vardef{vA}[type=\collection]{A}
```

```
\vardef{vM}[name=M,return={\magma[comp=##1][univ=\vA] }]{M}
```

```
\vardef{vMp}[name=Mp,return={\magma[comp={\##1}'] [univ=\vA] }]{M'}
```

```
Let \$ \backslash vM! := \backslash vM\{\} \$ and \$ \backslash vMp! := \backslash vMp\{\} \$ \sns{magma} on \$ \backslash vA$. ...
```

Let $M := \langle A, \circ \rangle$ and $M' := \langle A, \circ' \rangle$ [magmas](#).

Of course, we can also use `return=` with [variable](#) sequences – for example:

```
\varseq{vMs}[name=M,return={\magma[comp={\##1}_{\##1] },op=(M_i)_1^n]
{1,\ellipses,n}{\maincomp{M}_{\##1}}
```

```
Let \$ \backslash vMs! := \backslash vMs\{i\}_{1^n} \$ a sequence of \sns{magma}...
```

Let $(M_i)_1^n := \langle U_i, \circ_i \rangle_1^n$ a sequence of [magmas](#)...

Note that in the above, it seems that using `#1` in the `return` argument is allowed. Indeed, it is – the `return` statement takes the same arguments as the [semantic macro](#) itself does and is appropriately instantiated. Since the first (and only) argument to the

sequence `\vMs` is the index, when doing `\vMs{i}...` the `#1` in the `return`-statement will be replaced by `i`.

Also, note that if we want to produce M_i – i.e. the `magma` at index i in the sequence, we can do `\vMs{i}!`.



Think of the `!` as a “stop sign” - if the expression up to the `!` has an associated presentation, the `!` tells `gTeX` to “stop eating arguments” and present whatever it has until now.

3.3.2 Extending `Structures` and Axioms

It is extremely common to “build up” `structures` in a hierarchical manner by adding new fields or axioms: A *semigroup* is an associative magma. A *band* is an idempotent semigroup. A *monoid* is a semigroup with a unit. A *partial order* is an antisymmetric preorder.

We alluded to the fact earlier, that the `mathstructure environment` behaves like an `smodule` – that is literally true: Every `mathstructure foo` in a `module FooMod` is in fact also a `module ?FooMod/foo-module`. We can therefore easily extend `structures` using `\importmodule{...?FooMod/foo-module}` – but extending `structures` is so common, and using `\importmodule` tiring, that there is a shortcut: the `extstructure environment`. It takes as second argument a comma-separated list of `structure` names. That allows us to easily define `semigroups`:

Example 12

Input:

```
File [sTeX/MathTutorial]algebra/Semigroup.en.tex
8 \begin{extstructure}{semigroup}{magma}
9 \begin{sdefinition}
10 A \definame{semigroup} is a \sn{magma} $\semigroup!$,
11 where \inlineass[name=associative axiom]{
12 \conclusion{\isassociative{$\op!$}}.
13 }
14 \end{sdefinition}
15 \end{extstructure}
```

Output:

Definition 3.3.3. A `semigroup` is a `magma` $\langle U, \circ \rangle$, where $\circ$ is `associative`.

Note our usage of `\inlineass` to generate a new `symbol` for the `associative axiom`.

If we look at the `OMDOC` tab in the `HTML` preview window, we can see the output in Figure 18.

So `MMT` has decided that our statement is an *axiom*.

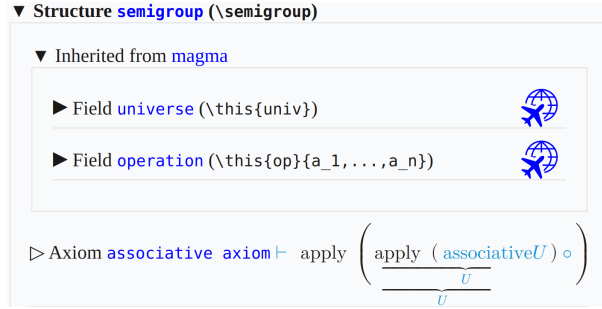


Figure 18: Axioms in OMDoc

Conservative Extensions

For **structures**, there is a *critical* distinction between *defined* and *undefined symbols*; and analogously between *theorems* and *axioms*.

Remember that **structures** are more like *templates* that are *instantiated* by particular objects. An *undefined* field in a **structure**, in that sense, is like an *obligation*: If something is supposed to be a **semigroup**, it *has to* have a **universe**, an **operation** and the **operation** needs to satisfy the **associative axiom**.

Defined fields on the other hand have a *definiens* on the basis of the remaining fields – they don’t need to be explicitly provided for something to instantiate the **structure**; if all the *undefined* fields are provided, the *defined* ones we get “for free”.

The same holds for *theorems*: If a statement is *provable* from the axioms, then we don’t need to explicitly prove it to hold for some particular instance – we have a proof already, provided the axioms hold.

The relation between axioms and theorems is not just analogous to that between undefined and defined **symbols**: It is the very same. Remember the **judgments as types** paradigm?

STEX For a **proposition** P , an assertion in **STEX** induces a **symbol** of **type** $\vdash P$. Without a proof, this **symbol** is *undefined* – and hence an *axiom*. A *proof* for P is a specific term of **type** $\vdash P$ – i.e. a potential *definiens*. To prove an assertion turns it into a *theorem*, which is to say that the **symbol** can be *defined*.

One consequence of this is: Extending a **structure** only by *defined* fields does not actually (conceptually) introduce a *new structure* – every instance of the old one *should* also be an instance of the new one. The new fields are basically just “syntactic sugar”.

There is a name for extending a **structure** only by defined fields (or theorems): A *conservative extension*.

STEX provides the **extstructure*** environment for that purpose. Unlike **extstructure**, it does *not* take a name (technically, **STEX** generates one internally). Instead, conceptually **extstructure*** modifies the extended **structure** directly, rather than generating a new **structure**. The caveat however is, that every **symbol** introduced in an **extstructure*** must be defined.

Consider the following conservative extension:

Example 13

Input:

```

File [sTeX/MathTutorial]algebra/MagmaSquare.en.tex
7 \begin{extstructure*}{magma}
8 \begin{sdefinition}[macro=sq,args=1]
9 \notation{sq}[op=\cdot^2][\{#1\}^{\comp 2}]
10 \vardef{va}[name=a,type=\univ,bind]{a}
11 Let $\inset{\va}{\univ}$. We define
12 $\defnotation{sq}{\va} := \definiens{op}{\va,\va}$.
13 \end{sdefinition}
14 \end{extstructure*}

```

Output:

Definition 3.3.4. Let $a \in U$. We define $a^2 := a \circ a$.

Via `\definiens`, the new symbol `sq` is now *defined* (note the `macro=` argument, taht generates a *semantic macro* as well). Whenever we import the containing *module*, we now have an additional field `sq` in (any extension of) `magma` – e.g., as `semigroup` extends `magma`, the following is now valid:

```

\usemodule[sTeX/MathTutorial]{algebra?MagmaSquare}
\vardef{vsg}[name=S,return=\semigroup]{S}
$\vsg{sq}{a}$

```

...producing a^2 .

3.3.3 Nesting Structures and `\this`

A perhaps not too surprising, but a notable aspect of *structures* is that fields themselves can be instances. This is important for example for implementing *vector spaces*, but can also be used to bundle things that are not normally thought of as *structures*, such as objects with certain defining properties.

Take as an example, the notion of a (magma) *homomorphism*:

Definition 3.3.5. Let $M_1 = \langle U_1, \circ_1 \rangle$ and $M_2 = \langle U_2, \circ_2 \rangle$ magmas. A **magma homomorphism** is a function $F : U_1 \rightarrow U_2$ such that $F(a \circ_1 b) = F(a) \circ_2 F(b)$ for all $a, b \in U_1$.

So a *homomorphism* is a *function* with certain properties. And *structures* can be used to “bundle” the *function* itself with both the *magmas* on whose universes the *function* operates, as well as the *axiom* that *makes* it a *homomorphism*. After all, considered as a mere *function*, $F : U_1 \rightarrow U_2$ contains no information about the operation with respect to which it is homomorphic.

The first thing to note is that we can provide `mathstructure` with an optional argument for a *name* distict from the name of its *semantic macro*. We then add two fields that *return magmas*. So far, so unexciting:

```

\begin{mathstructure}{magmahom}[magma homomorphism]
\symdef{dom}[name=domain,return={\magma[comp={##1}_1]}}{M_1}
\symdef{cod}[name=codomain,return={\magma[comp={##1}_2]}}{M_2}

```

For the `function` itself, we know how to give it a maningful `type`, already:

```

\symdef{f}[type=\funspace{\dom{univ}}{\cod{univ}},args=1]{???}

```

...but what should its `notation` be? Ideally we would want it to just be the `notation` of whatever particular instance it is – in informal mathematics, we rarely distinguish notationally between a `homomorphism` and its underlying `function` (to the point where it's not clear, whether *informally* the distinction is even meaningful). Similarly, we rarely distinguish e.g. between a `magma` (or semigroup, monoid, group, ring, vector space,...) and its underlying universe.

This is where `\this` comes into play (pun intended). Within an `mathstructure` or `exstructure`, or in the context of a particular instance of one, `\this` represents “the” instance.

We can set it in the context of `mathstructure` as a further optional argument; e.g.

```

\begin{mathstructure}{magmahom}[magma homomorphism,this=F]

```

and then use `\this` in the `notation` for the `function`. We can further provide the `homomorphism condition` as an axiom using `\inlineass`:

Example 14

Input:

```

File [sTeX/MathTutorial]algebra/Homomorphism.en.tex
9  \begin{mathstructure}{magmahom}[magma homomorphism,this=F]
10  \symdef{dom}[name=domain,return={\magma[comp={##1}_1]}}{M_1}
11  \symdef{cod}[name=codomain,return={\magma[comp={##1}_2]}}{M_2}
12  \symdef{f}[op=\this,args=1,
13    type=\funspace{\dom{univ}}{\cod{univ}}
14    ]{\this \dobrackets{#1}}
15
16  \begin{sdefinition}[for={magmahom,dom,cod,f}]
17    \vardef{va}[name=a,type=\dom{univ}]{a}
18    \vardef{vb}[name=b,type=\dom{univ}]{b}
19    Let  $\$dom!=\dom{\}$  and  $\$cod!=\cod{\}$   $\text{\sns{magma}}$ .
20    A  $\text{\definame{magmahom}}$  is a function
21     $\$fun{\f!}{\dom{univ}}{\cod{univ}}$  such that
22    \inlineass[name=homomorphism condition]{\conclusion{\forallal{
23      \$arg[2]{\eq{
24        \f{\dom{op}}{\va,\vb}}, \cod{op}{\f{\va},\f{\vb}}
25      }}\$ \comp{for all} \$arg[1]{\inset{\va,\vb}{\dom{univ}}}\$.
26    }}}
27  \end{sdefinition}
28  \end{mathstructure}

```

Output:

Definition 3.3.6. Let $M_1 = \langle U_1, \circ_1 \rangle$ and $M_2 = \langle U_2, \circ_2 \rangle$ magmas. A **magma homomorphism** is a function $F : U_1 \rightarrow U_2$ such that $F(a \circ_1 b) = F(a) \circ_2 F(b)$ for all $a, b \in U_1$.

Now if we instantiate our `magma homomorphism`:

```
\vardef{vh}[name=H,return={\magmahom[this=H] }]{H}
```

Here is a list of what we can do now:

Syntax	Result
$\$ \backslash \text{vh} ! \$$	H
$\$ \backslash \text{vh} \{ \} \$$	$\langle M_1, M_2, H \rangle$
$\$ \backslash \text{vh} \{ f \} ! \$$	H
$\$ \backslash \text{vh} \{ f \} \{ a \} \$$	$H(a)$
$\$ \backslash \text{vh} \{ \text{dom} \} ! \$$	M_1
$\$ \backslash \text{vh} \{ \text{cod} \} \{ \} \$$	$\langle U_2, \circ_2 \rangle$
$\$ \backslash \text{vh} \{ \text{cod} \} \{ \text{univ} \} \$$	U_2
$\$ \backslash \text{vh} \{ \text{dom} \} \{ \text{op} \} ! \$$	$\circ_1$
$\$ \backslash \text{vh} \{ \text{cod} \} \{ \text{op} \} \{ a, b, c \} \$$	$a \circ_2 b \circ_2 c$

Note how – as one would expect – we can treat $\backslash \text{vh} \{ \text{dom} \}$ and $\backslash \text{vh} \{ \text{cod} \}$ like any other instance of [magma](#).



Note that some of the outputs in the above table are probably not quite what we want. Determining the precise typesetting of an expression involving *nested paths* of fields is difficult, to say the least (e.g., what exactly should $\backslash \text{this}$ refer to in a deeply nested sequence of fields?).

Using instances within [structures](#) is still very useful; at the very least when defining [structures](#). When subsequently *using* [structures](#), however, accessing fields of fields (of fields (of ...)) of an instance should be avoided.

Luckily, there is rarely a need for doing so – in practice, those fields we might want to access in such a way, we usually also want to provide specific [notations](#) and talk about independently of the “containing” instance, such that introducing a new [variable](#) (or [symbol](#)), and assigning the corresponding field to that [variable](#), makes considerably more sense. And subsequently using the [variable](#) is easier than concatenating $\{ \dots \}$, too.

3.4 Complex Inheritance and Theory Morphisms



We are starting to approach seriously experimental territory.

While the theory behind all the following is relatively well understood, and their implementation in [MMT](#) is mature, the same can not be said out the implementation in [sTeX](#).

There are still kinks to be ironed out, but feel free to experiment.

We now have all the tools available to progress towards something more interesting. Here is a list of documents with respective [modules](#) and [symbols](#) we will build on in the following:

[sTeX/MathTutorial]props/Idempotent.en.tex

Definition 3.4.1. Let $e \in A$ and $\circ : A \times A \rightarrow A$. e is called **idempotent** with respect to $\circ$, if $e \circ e = e$.

Definition 3.4.2. The operation $\circ : A \times A \rightarrow A$ is called **idempotent**, if every element $a \in A$ is **idempotent** with respect to $\circ$.

[sTeX/MathTutorial]props/Distributive.en.tex

Definition 3.4.3. Let $\odot : B \times A \rightarrow A$ and $\oplus : A \times A \rightarrow A$. We say $\odot$ **distributes over** $\oplus$, if $b \odot (a_1 \oplus a_2) = (b \odot a_1) \oplus (b \odot a_2)$ for all $a_1, a_2 \in A$ and $b \in B$.

[sTeX/MathTutorial]props/Absorption.en.tex

Definition 3.4.4. Let $\odot : A \times B \rightarrow A$ and $\oplus : A \times B \rightarrow B$. We say $\odot$ **absorbs** $\oplus$, if $a_1 \odot (a_1 \oplus b) = a_1$ for all $a_1 \in A$ and $b \in B$.

[sTeX/MathTutorial]algebra/Band.en.tex

Definition 3.4.5. A **band** is an **idempotent semigroup**.

[sTeX/MathTutorial]algebra/Semilattice.en.tex

Definition 3.4.6. A **semilattice** is a **commutative band**.

[sTeX/MathTutorial]props/Reflexive.en.tex

Definition 3.4.7. A binary relation R on A is called **reflexive**, if $R(a, a)$ for all $a \in A$.

[sTeX/MathTutorial]props/Symmetric.en.tex

Definition 3.4.8. A binary relation R on A is called **symmetric**, if $R(a, b)$ implies $R(b, a)$ for all $a, b \in A$.

[sTeX/MathTutorial]props/Transitive.en.tex

Definition 3.4.9. A binary relation R on A is called **transitive**, if $R(a, b)$ and $R(b, c)$ implies $R(a, c)$ for all $a, b, c \in A$.

[sTeX/MathTutorial]props/Antisymmetric.en.tex

Definition 3.4.10. A binary relation R on A is called **antisymmetric**, if $R(a, b)$ and $R(b, a)$ implies $a = b$ for all $a, b \in A$.

[sTeX/MathTutorial]orders/Graph.en.tex

Definition 3.4.11. A **directed graph** is a structure $\langle U, R \rangle$, where U is a collection and R a binary relation on U .

Definition 3.4.12. An **(undirected) graph** is a directed graph $\langle U, R \rangle$, where R is symmetric.

[sTeX/MathTutorial]orders/Preorder.en.tex

Definition 3.4.13. A structure $\langle U, \leq \rangle$ is called a **preorder** (or **quasiorder**, or **preordered set**; in short **proset**), if $\leq$ is reflexive and transitive.

[sTeX/MathTutorial]orders/Poset.en.tex

Definition 3.4.14. A preorder $\langle U, R \rangle$ is called a **partial order** (or **poset**), if R is antisymmetric.

[sTeX/MathTutorial]orders/InfSup.en.tex

Definition 3.4.15. Let $\langle U, R \rangle$ a poset. An element $a \in U$ is called an **infimum** or **greatest lower bound** of x_1 and x_2 , if $a R x_1$, $a R x_2$, and for any x with $x R x_1$ and $x R x_2$, we have $x R a$.

Definition 3.4.16. Let $\langle U, R \rangle$ a poset. An element $a \in U$ is called a **supremum** or **least upper bound** of x_1 and x_2 , if $x_1 R a$, $x_2 R a$, and for any x with $x_1 R x$ and $x_2 R x$, we have $a R x$.



Note that **infima** and **suprema** are more generally defined on *sets* of elements. Doing so in sTeX is significantly more complicated *for now*, and will require some amount of research to make convenient – especially if we want to subsequently define *operators* on pairs of elements, as below. We therefore opt for the simpler version where it is defined as binary from the get go.

Definition 3.4.17. A poset $\langle U, R \rangle$ is called a **meet semilattice** if for every two elements a, b the **infimum** $a \wedge b$ exists.

Definition 3.4.18. A poset $\langle U, R \rangle$ is called a **join semilattice** if for every two elements a, b the **supremum** $a \vee b$ exists.

Definition 3.4.19. An **(order) semilattice** is a **meet** and **join semilattice**.

Exercise

Try to implement all of the above yourself!

3.4.1 Glueing Structures Together

We now want to progress towards **lattices**, i.e. the following:

Definition 3.4.20. A **lattice** is a **structure** $\langle U, \wedge, \vee \rangle$ such that $\langle U, \wedge \rangle$ and $\langle U, \vee \rangle$ are **semilattices**, and $\vee$ **absorbs** $\wedge$ and vice versa; i.e. $a \vee (a \wedge b) = a$ and $a \wedge (a \vee b) = a$. The operations $\wedge$ and $\vee$ are called **meet** and **join**, respectively.

So we make a new **module**, open an **extstructure environment** and... realize two problems:

1. We can't just extend **semilattice**: We need *two* copies of **semilattice** that share a universe, and importing **semilattice** twice is of course redundant.
2. We also want to *rename* the operations of the two **semilattices** to be subsequently called **join** and **meet**.

What we need is a way to *inherit* from **semilattice** while a) *modifying* the **symbols** therein, and b) not be **idempotent** – i.e. two imports from the same **structure** or **module** should not be identified. We can do that with the **\copymod macro**, which takes three arguments:

1. A *name* for the copy,
2. the **structure** or **module** to copy, and
3. a comma-separated list of renamings and redefinitions of the **symbol**. $\langle symbol \rangle = \langle def \rangle$ redefines $\langle symbol \rangle$, $\langle symbol \rangle @ \langle newname \rangle$ renames it, $\langle symbol \rangle = \langle def \rangle @ \langle newname \rangle$ (or $\langle symbol \rangle @ \langle newname \rangle = \langle def \rangle$) does both.

In our case, we want two copies of **semilattice**, which we will call **meets1** and **joins1**. In the first copy, we only want to rename **op** to **meet**. In the second, we want to rename **op** to **join**, and *also* redefine the universe to be the one from **meets1**:

```

\copymod{meetsl}{semilattice}{
  op @ meet
}
\copymod{joinsl}{semilattice}{
  univ = \univ,
  op @ join
}

```

You might have already noticed some problem with that – which of the two universes does `\univ` refer to now? (They are *defined* as equal, but `LaTeX` does not know that!) Or which of the two commutative axioms does “commutative axiom” refer to now? Everything is ambiguous now!

Not really - if you have wondered why the `\copymod` takes a *name* as argument: The name is prefixed to every *symbol* name. Hence, the universe in `joinsl` is now called `joinsl/universe`, and the one in `meetsl` is called `meetsl/universe`. Furthermore, `\copymod` by default generates no *semantic macros* for any of the imported *symbols* – except for those renamed with `@`. In fact, what the `@` syntax actually does, is to generate a *semantic macro* by that name. If we want to change the *name* (that is shown when using `\symname` et al), we add that new name in square brackets. Hence, what we really want to do is:

```

\copymod{meetsl}{semilattice}{
  univ @ univ,
  op @ [meet]meet
}
\copymod{joinsl}{semilattice}{
  univ = \univ,
  op @ [join]join
}

```

This now gives us two copies of *semilattice*, generates *semantic macros* `\univ` for `meetsl/universe`, `\meet` for `meetsl/op` and `\join` for `joinsl/op`, and renames `meetsl/op` to `meet` and `joinsl/op` to `join`.

That allows us to then add the *absorption* axioms, an *sdefinition* for *lattice* and subsequently `\lattice!` produces $\langle U, \wedge, \vee \rangle$, with all axioms inherited (see [sTeX/MathTutorial]algebra/Lattice.en.tex).

3.4.2 Realizations

A very common situation we find in connection with *mathematical structures* is that “every *this* is a *that*” (or the concrete case “*this* is a *that*”).

With what we did so far, we are in this situation regarding the algebraic definition of *semilattices* and the order-theoretic one (exemplary *meet semilattice*).

In *MMT* parlance, this corresponds to a *total (implicit) theory morphism* from “that” to “this”.

In *sTeX* words, we want to inherit from “that” by assigning all the *symbols* in “that” to concrete terms. In our case:

[sTeX/MathTutorial]algebra/SemiLatticeOrder.en.tex

Definition 3.4.21. Let $\langle U, \circ \rangle$ a *semilattice*. We let $a \mathrel{R} b$ iff $a \circ b = a$.

Satz 3.4.22. $\langle U, R \rangle$ is a *meet semilattice*.

Proof:

We need to prove the following

1. **reflexivity** ($a R a$):

We need to show $a \circ a = a$. Follows from the **idempotent axiom**.

2. **antisymmetry** ($a R b$ and $b R a$ implies $a = b$):

Assume $a \circ b = a$ and $b \circ a = b = a \circ b$ (by the **commutative axiom**). Hence, $a = b$

3. **transitivity** (If $a R b$ and $b R c$, then $a R c$.):

Assume $a \circ b = a$ and $b \circ c = b$. Then $a \circ c = (a \circ b) \circ c = a \circ (b \circ c) = a \circ b = a$. Hence, $a R c$.

4. $a \circ b$ is the **infimum** of $\{a, b\}$:

By definition (and the **commutative axiom**), $a \circ b R a$ and $a \circ b R b$. We need to show, that if $x R a$ and $x R b$, then $x R a \circ b$. Assume $x \circ a = x$ and $x \circ b = x$. Then $x \circ (a \circ b) = (x \circ a) \circ b = x \circ b = x$. Hence $x R a \circ b$

□

So to be precise, we want to provide *definientia* for all undefined **symbols** in **meet semilattice** (i.e. the **relation** and **meet**) and *proofs* for all *axioms* (**reflexive axiom**, **antisymmetric axiom**, **transitive axiom**, and **infimum axiom**), and by so obtain the fact that every **semilattice** is a **meet semilattice**.

For that purpose, we have the **\realize macro**. It behaves like **\copymod**, but does not take a name, and additionally requires that all undefined fields get assigned. So we could do the following:

Example 15

Input:

```
File [sTeX/MathTutorial]algebra/SemiLatticeOrder1.en.tex
8 \begin{extstructure*}{semilattice}
9 \interpretmod{meetsl}{meetsl}{
10   univ = \univ,
11   meet @ [meet]meet = \op!,
12   rel @ [order]order = \map{a,b}{\eq{\op{a,b},a}},
13   reflexive axiom = trivial,
14   transitive axiom = trivial,
15   antisymmetric axiom = trivial,
16   infimum axiom = trivial
17 }
18 \end{extstructure*}
19
20 \vardef{mysl}{return=\semilattice}{S}
21 $\mysl{order}\{a,b\} \quad \mysl{}\{univ,op,order\}$
```

Output:

$$a \leq_S b \quad \langle U_S, \circ_S, \leq_S \rangle$$

As we can see, we can now access the field `order`, which is renamed from `relation` in `meet semilattice` and also has the desired definiens in `MMT`. But of course this approach is very “declarative”: We do all the assigning in one `macro`, rather than narratively as what they *should* be: definitions and proofs.

If we want to achieve the more narrative version at the beginning of the subsection, we can use the `realization environment` instead. It behaves like the `\realize macro`, but allows us to do the assignments and renamings individually somewhere in the body of the `environment`, interleaved with arbitrary text. Additionally, within the `environment`, all `sTeX` features that introduce *definiencia* (like the `\definiens macro`) induce assignments instead.

To declaratively rename or assign fields, we can then use the `\assign` and `\renamedecl` macros instead. That allows us to do the following instead:

```
\begin{realization}{meetsl}
  \assign{univ}{\univ}
  \assign{meet}{\op!}
  \renamedecl{rel}[order]{order}
  ...
```

...and then use text to do the remaining assignments. For example, we can use the `sdefinition environment` to assign `rel` to the desired definiens:

```
\usestructure{meetsl}
\begin{sdefinition}[for=order]
  \varbind{va,vb}
  Let  $\$ \backslash semilattice! [univ,op] \$ a \backslash sn \{ semilattice \} .$ 
  We let  $\$ \backslash rel \{ \backslash va, \backslash vb \} \$$ 
  iff  $\$ \backslash definiens \{ \backslash eq \{ \backslash op \{ \backslash va, \backslash vb \}, \backslash va \} \} \$ .$ 
\end{sdefinition}
```

And now `sTeX` will use the `\definiens` to assign $a, b \mapsto a \circ b = a$ to the `relation` of `meet semilattice`.

Analogously, we can use the `sproof` and `subproof` environments to produce “definiencia” (i.e. proofs) for the axioms (see [sTeX/MathTutorial]algebra/SemiLatticeOrder.en.tex)

Chapter 4

Extensions for Education

The last two chapters have shown generic markup and semantization facilities in `sTeX`. As said before, investments in semantic markup pay off, iff the impact of a document is high, e.g. if there are many more readers than authors or if the semantic services afforded by the semantic markup can help reduce the help readers need to understand the material.

Educational documents constitute one category of high-impact documents which are supported by the `sTeX` ecosystem, we will cover them here. In fact, educational documents have been one of the initial document categories `sTeX` has been developed for. The idea is that if we can mark up the meaning and didactic role of learning objects, we can base learning support services on that and embed them into the documents.

Another reason educational documents are particularly interesting is that in a sense all academic communication is educational, as all documents try to “teach” the reader new concepts and results.

Concretely, we cover a document class for combining slides and course notes (section 4.1) and functionality for marking up problems and exercises (section 4.2) and for marking up homework assignments and exams (section 4.3).

4.1 Slides and Course Notes

4.1.1 Introduction

The `notesslides` document class is derived from `beamer.cls` [`beamerclass:on`], it adds a “notes version” for course notes that is more suited to printing than the one supplied by `beamer.cls`.

The `notesslides` class takes the notion of a slide frame from Till Tantau’s excellent `beamer` class and adapts its notion of frames for use in `sTeX`. To support semantic course notes, it extends the notion of mixing frames and explanatory text, but rather than treating the frames as images (or integrating their contents into the flowing text), the `notesslides` package displays the slides as such in the course notes to give students a visual anchor into the slide presentation in the course (and to distinguish the different writing styles in slides and course notes).

In practice we want to generate two documents from the same source: the slides for presentation in the lecture and the course notes as a narrative document for home study.

To achieve this, the `notesslides` class has two modes: *slides mode* and *notes mode* which are determined by the package option.

4.1.2 Package Options

The `notesslides` class takes a variety of class options:

slides notes	The options <code>slides</code> and <code>notes</code> switch between slides mode and notes mode (see subsection 10.1.3).
sectocframes topsect	If the option <code>sectocframes</code> is given, then for the <code>sfragments</code> , special frames with the <code>sfragment</code> title (and number) are generated. The <code>topsect</code> allows to specify the top section level (e.g. <code>chapter</code> , <code>section</code> , ...) for documents that are formally stand-alone, but intended to be chapters, sections, or
frameimages fiboxed	If the option <code>frameimages</code> is set, then slide mode also shows the <code>\frameimage</code> -generated frames (see ???). If also the <code>fiboxed</code> option is given, the slides are surrounded by a box.

4.1.3 Notes and Slides

frame (*env.*) Slides are represented with the `frame` environment just like in the `beamer` class, see [Tantau:ugbc] for details.

note (*env.*) The `notesslides` class adds the `note` environment for encapsulating the course note fragments. Content of this environment is only shown in *notes mode*.

By interleaving the `frame` and `note` environments, we can build course notes as shown here:

```

1 \ifnotes\maketitle\else
2 \frame[noframenumbering]\maketitle\fi
3
4 \begin{note}
5   We start this course with ...
6 \end{note}
7
8 \begin{frame}
9   \frametitle{The first slide}
10  ...
11 \end{frame}
12 \begin{note}
13   ... and more explanatory text
14 \end{note}
15
16 \begin{frame}
17   \frametitle{The second slide}
18   ...
19 \end{frame}
20 ...

```

¹EDNOTE: MK: maybe this option should be used in the other $\text{\TeX}$ classes as well. Actually I think this already is treated by `stex.sty`; where to document?

`\ifnotes` Note the use of the `\ifnotes` conditional, which allows different treatment between `notes` and `slides` mode – manually setting `\notesttrue` or `\notesfalse` is strongly discouraged however.



We need to give the title frame the `noframenumbering` option so that the frame numbering is kept in sync between the slides and the course notes.



The `beamer` class recommends not to use the `allowframebreaks` option on frames (even though it is very convenient). This holds even more in the `notesslides` case: At least in conjunction with `\newpage`, frame numbering behaves funnily (we have tried to fix this, but who knows).

`\inputref*` If we want to transclude a the contents of a file as a note, we can use a new variant `\inputref*` of the `\inputref` macro: `\inputref*{foo}` is equivalent to `\begin{note}\inputref{foo}\end{note}`.

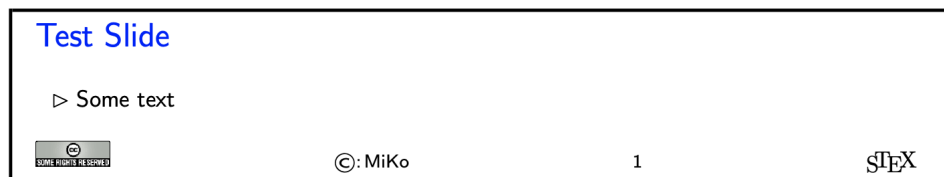
`nparagraph (env.)` There are some environments that tend to occur at the top-level of `note` environments.
`nparagraph (env.)` We make convenience versions of these: e.g. the `nparagraph` environment is just an
`ndefinition (env.)` `sparagraph` inside a `note` environment (but looks nicer in the source, since it avoids one
`nexample (env.)` level of source indenting). Similarly, we have the `nfragment`, `ndefinition`, `nexample`,
`nsproof (env.)` `nsproof`, and `nassertion` environments.
`nassertion (env.)`

4.1.4 Managing and Customizing Themes

As the `notesslides` package is based on the `beamer` class, it can in principle (in slides mode) take advantage of all `beamer` themes. Themes can be specified/loaded with the normal `\usetheme`, but the `notesslides` package provides the `\libusetheme` macro:

`\libusetheme` In analogy to the `\libinput` functionality of the `stex` package, this macro allows loading `beamer` themes from the `lib` directories leading up to the current archive.

Unfortunately, there is no `beamer` functionality of using a themed `frame` environment in `article` mode. Therefore `notesslides` has to hand-craft one – for a very limited set of themes. The `notesslides` package and class comes with a simple default theme named `sTeX` that provided by the `beamterthemesTeX.sty` style file. It is assumed as the default theme for `sTeX`-based notes and slides. The result in `notes` mode (which is like the `slides` version except that the slide height is variable) is



The footer line can be customized. In particular the logos.

\setslidelogo The default logo provided by the `notesslides` package is the $\TeX$ logo it can be customized using `\setslidelogo{<logo name>}`.

\setsource The default footer line of the `notesslides` package mentions copyright and licensing. In `notesslides \source` stores the author's name as the copyright holder. By default it is the author's name as defined in the `\author` macro in the preamble. `\setsource{<name>}` can change the writer's name.

\setlicensing For licensing, we use the Creative Commons Attribution-ShareAlike license by default to strengthen the public domain. If package `hyperref` is loaded, then we can attach a hyperlink to the license logo. `\setlicensing[<url>]{<logo name>}` is used for customization, where `<url>` is optional.

These macros can be used set up a institution-specific theme; for instance the one for FAU simply customizes the logo:

```
1 % Beamer slide theme for FAU;
2 \typeout{Beamer FAU theme}
3 \RequirePackage{beamerthemesTeX}
4 \setslidelogo[courses/FAU/admin]{PIC/FAU_Logo_Bildmarke.png}
```

This could be placed in one of the `lib` folders available to the current archive and could then be activated with

```
1 \documentclass[slides]{notesslides}
2 \libusetheme{FAU}
```

in the preamble of the main `noteslides` file.

4.1.5 Frame Images

Sometimes, we want to integrate slides as images after all – e.g. because we already have a PowerPoint presentation, to which we want to add $\TeX$ notes.

\frameimage In this case we can use `\frameimage[<opt>]{<path>}`, where `<opt>` are the options of `\includegraphics` from the `graphicx` package [**CarRah:tpp99**] and `<path>` is the file path (extension can be left off like in `\includegraphics`). We have added the `label` key that allows to give a frame label that can be referenced like a regular `beamer` frame.

The `\mhframeimage` macro is a variant of `\frameimage` with repository support. Instead of writing

```
1 \frameimage{\MathHub{fooMH/bar/source/baz/foobar}}
```

we can simply write (assuming that `\MathHub` is defined as above)

```
1 \mhframeimage[fooMH/bar]{baz/foobar}
```

Note that the `\mhframeimage` form is more semantic, which allows more advanced document management features in `MathHub`.

If `baz/foobar` is the “current module”, i.e. if we are on the MathHub path `...MathHub/fooMH/bar...`, then stating the repository in the first optional argument is redundant, so we can just use

```
1 \mhframeimage{baz/foobar}
```

`\textwarning` The `\textwarning` macro generates a warning sign: 

4.1.6 Ending Documents Prematurely

`\prematurestop`
`\afterprematurestop` For prematurely stopping the formatting of a document, $\TeX$ provides the `\prematurestop` macro. It can be used everywhere in a document and ignores all input after that – backing out of the `sfragment` environments as needed. After that – and before the implicit `\end{document}` it calls the internal `\afterprematurestop`, which can be customized to do additional cleanup or e.g. print the bibliography.

`\prematurestop` is useful when one has a driver file, e.g. for a course taught multiple years and wants to generate course notes up to the current point in the lecture. Instead of commenting out the remaining parts, one can just move the `\prematurestop` macro. This is especially useful, if we need the rest of the file for processing, e.g. to generate a theory graph of the whole course with the already-covered parts marked up as an overview over the progress; see `import_graph.py` from the `lmhtools` utilities [[lmhtools:github:on](#)].

Text fragments and modules can be made more re-usable by the use of global variables. For instance, the admin section of a course can be made course-independent (and therefore re-usable) by using variables (actually token registers) `courseAcronym` and `courseTitle` instead of the text itself. The variables can then be set in the $\TeX$ preamble of the course notes file.

4.1.7 Global Document Variables

To make document fragments more reusable, we sometimes want to make the content depend on the context. We use **document variables** for that.

`\setSGvar` `\setSGvar{<vname>}{<text>}` to set the global variable `<vname>` to `<text>` and `\useSGvar{<vname>}`
`\useSGvar` to reference it.

`\ifSGvar` With `\ifSGvar` we can test for the contents of a global variable: the macro call `\ifSGvar{<vname>}{<val>}{<ctext>}` tests the content of the global variable `<vname>`, only if (after expansion) it is equal to `<val>`, the conditional text `<ctext>` is formatted.

4.1.8 Excursions

In course notes, we sometimes want to point to an “excursion” – material that is either presupposed or tangential to the course at the moment – e.g. in an appendix. The typical setup is the following:

```

1 \excursion{founif}{../fragments/founif.en}
2 {We will cover first-order unification in}
3 ...
4 \begin{appendix}\printexcursions\end{appendix}

```

It generates a paragraph that references the excursion whose source is in the file `../fragments/founif.en.tex` and automatically books the file for the `\printexcursions` command that is used here to put it into the appendix. We will look at the mechanics now.

`\excursion` The `\excursion{<ref>}{<path>}{<text>}` is syntactic sugar for

```

1 \begin{nparagraph}[title=Excursion]
2   \activateexcursion{founif}{../ex/founif}
3   We will cover first-order unification in \sref{founif}.
4 \end{nparagraph}

```

`\activateexcursion` Here `\activateexcursion{<path>}` augments the `\printexcursions` macro by a call
`\inputref{<path>}`. In this way, the `\printexcursions` macro (usually in the appendix)
`\printexcursion` will collect up all excursions that are specified in the main text.
`\excursionref`

Sometimes, we want to reference – in an excursion – part of another. We can use `\excursionref{<label>}` for that.

`\excursiongroup` Finally, we usually want to put the excursions into an `sfragment` environment and add an introduction, therefore we provide the a variant of the `\printexcursions` macro: `\excursiongroup[id=<id>,intro=<path>]` is equivalent to

```

1 \begin{note}
2 \begin{sfragment}[id=<id>]{Excursions}
3   \inputref{<path>}
4   \printexcursions
5 \end{sfragment}
6 \end{note}

```

Contents

4.2 Problems and Exercises

4.2.1 Background

Problems/exercises are text fragments that contain a task assigned to the learner – e.g. computing the value of a specified quantity, simplifying an expression, modeling a described situation in a mathematical structure, or judging the veracity of a given statement. Problems/exercises are used in very different contexts, in

- *homework assignments and formal exams*, where they are graded and points are awarded for course credit,

- *self-study materials* that allow learners explore applications of some concepts and/or practice,
- *automated tutoring systems* to determine learner competencies to trigger computer supported learning services.

In all of these contexts, diagnosis the correctness or suitability of an answer plays a great role, e.g. for grading, the generation of feedback, or the suggestion of remedial actions that may “heal” any diagnosed competency gaps or misconceptions. To support that we might want to specify “answer classes” that classify answers received for automating/triggering feedback and grading.

There are various types of problems/exercises in practical use. They are mainly distinguished by how they support diagnosis of student answers: there are

- open problems, where expected answers are free-form, and classification into answer classes is usually a manual process; see subsection 4.2.3
- single/multiple choice questions where the answer classes and thus feedback/grading correspond to the selection pattern of items; see subsection 4.2.7.
- fill-in-the-blanks questions where we may want to specify how the answer string is interpreted; see subsection 4.2.8.

Additionally, problems and exercises often come with text fragments that serve auxiliary functions: hints, notes, and and grading specifications. Furthermore, we can specify how long solving a given problem is estimated to take and how many points will be awarded for a perfect solution – e.g. in an exam. See subsection 4.2.10 for details.

The `problem` package gives semantic markup support for all these aspects of problems/exercises with a focus on problem libraries that collect carefully formulated and tested problems/exercises, so that they can be re-used in many contexts. It also tries to support all possible stakeholders in dealing with problems/exercises:

- *learners*, who try to solve problems and may profit from feedback
- *instructors*, who pose problems in homeworks, quiz, and exams,
- *graders*, who try to assess learner’s competencies from the answers given
- *authors* of learning objects and (automated) *course generators*, who may use problems to diagnose learner’s competencies (e.g. as prerequisites).

4.2.2 The Package, Options, and Configuration

The `problem` package provides functionality for marking up problems and exercises as semantic sources from which the presentations for the various contexts can be generated. E.g. documents without solutions for paper or online exams, and the corresponding exams with master solutions for exam reviews. Similarly with/without hints, or points. Their visibility is specified in the options of the `problem` package, which can be used in any L^AT_EX class. The following is a typical preamble for a problem file:

```
\documentclass[lang=de]{article}
\usepackage[solutions,hints,pts,min]{problem}
```

Here we have specified the options `solutions` (solutions should be shown), `hints` (hints should be given), `pts` (display the points awarded for solving the problem?), `min` (display the estimated minutes for problem solving). Leaving out the options would make the corresponding functionality invisible.

The `problem` package generates content and labels according to the language specified in the `lang` key of the main document², these can be localized by in the files `ldf/problem-<lang>.ldf`⁶.

The `problem` package also supplies the `test` option, which specifies that the content is intended for use in an assessment situation, where certain additional content (e.g. exam galse) should be shown while other content (e.g. solutions) should not be.

Note that documents (or document fragments) with problems are often formatted in different versions that can be configured by these options. Therefore the following variant of the preamble prefix above can be very useful:

```
\documentclass[lang=de]{article}
\providecommand\probopts{,solutions,min}
\usepackage[hints,pts\probopts]{problem}
```

We add a level of indirection via the `\probopts` macro (`\providecommand` sets it unless it already exists). If the current file is `foo.tex`, then we can make a L^AT_EX file `foo-test.tex` that formats in test mode as

```
\newcommand\probopts{,test}\input{foo}
```

Alternatively, we do not need a file at all: we can just (e.g. in a Makefile or a PDF generation script) issue the following command in the terminal:

```
pdflatex "\def\probopts{,test}\input{foo}"
```

4.2.3 (Open) Problems

The main environment provided by the `problem` package is (surprise surprise) the `sproblem` environment. It is used to mark up problems and exercises. The following example shows the main functionality:

Example 16

Input:

²EdNOTE: MK: document the attribute somewhere

⁶The current crop of language definition files is quite limited. Please feel free to contribute the to the localization effort

```

File tutorial/ext/simple-problem.en.tex
1 \begin{document}
2 \begin{sproblem}[id=prob.elefants,pts=10,min=2,title=Fitting Elefants]
3   How many Elefants can you fit into a Volkswagen beetle?
4   \begin{hint}
5     Think positively, this is simple!
6   \end{hint}
7   \begin{exnote}
8     Justify your answer
9   \end{exnote}
10  \begin{gnote}[title=deduct 5pts if justification is missing]
11    \anscls[feedback=you forget that elephants could be small]{none}
12    \anscls[feedback=you forget the back seats]{2}
13  \end{gnote}
14  \begin{solution}
15    Four, two in the front seats, and two in the back.

```

Output:

```

Exercise (Fitting Elefants)
How many Elefants can you fit into a Volkswagen beetle?

Lösung: Four, two in the front seats, and two in the back.

```

The `sproblem` environment takes an optional key/value argument with the keys `id` as an identifier that can be reference later, `pts` for the points to be gained from this exercise in homework or quiz situations, `min` for the estimated minutes needed to solve the problem, and finally `title` for an informative title of the problem.

The `sproblem` can contain several `solution` environments, which take the well-known `id`, `style`, and `title` attributes, and also the `testspace` and `answerclass`. The former

The additional functionality is specified in the `hint` `exnote` (notes in exercises), `solution`, and `gnote` (grading notes) environments. Here, the first three are shown whereas the grading notes are hidden, since the corresponding option was not given in the `\usepackage[...]{problem}`. All of these environments can occur any number of times in the `sproblem` environment. The `solution` environment takes an optional argument that is interpreted as the identifier.

4.2.4 Solutions and Answer Classes

Most problem libraries also contain (master) solutions, which show the intended way of solving a problem. The `solution` environment allows to specify solutions. It is only formatted if the `solutions` options is specified for the `problem` package.

We observe that many problems have more than one possible solution, so the `problem` package allows multiple `solution` environments; they can be distinguished by an `id` key in the optional first argument.

We also observe that in problem libraries we may want to keep track of classes of answers that have been observed e.g. in previous assessment situations, e.g. to provide

feedback. In this setting, the `solution` environment can be used to encode (typical representative of) **answer classes** (see also ??? below).

`solution` The `solution` environment takes an optional argument that allows the keys `id`, `title` and `style` keys that we have already seen above, further more

1. `testspace=`, which is equivalent to a `\testspace{}` (see subsection 4.2.10).
2. `answerclass=`, which gives the name of an answer class, this solution corresponds to.³

EdN:3

4.2.5 Grading Support

When problems are graded by multiple persons (e.g. for large university exams), then consistency of grading is a major issue. Both within a cohort and between cohorts e.g. in successive instances of a course.

Therefore keeping records of how to grade a problem (which problem is worth how many “points” and where to deduct how many points in which situations) and making them accessible to graders in a “master solution” – a version of an exam formatted to have the known solutions/answer classes (see above) and the grading specifications is a good practice. And it makes sense to include grading information into a problem library.

The simplest and arguably most effective measure is to specify the points that can be reached for a problem in the problem itself via the ‘pts=’ key (but see subsection 4.2.9)

The `problem` package also supplies the `gnotes` environment, which is only formatted when the `gnotes=` key is given. This environment takes an optional argument that allows the `id`, `style`, and `title` keys. The latter is used to give a short version of the grading specification. More structured grading support can be expressed in `\anscls` markup which we discuss next.

We have already encountered the notion of answer classes – classes of answers that are supposed equivalent in terms of learner’s competencies inscribed in them – above. These are the natural carriers of both grading feedback and points.

In the `gnote` environment we use the `\anscls` macro for specifying such information using meta-level descriptions rather than typical representatives. It takes an optional keyword argument for the grading/feedback information and a required one for the answer class description.

We distinguish two usages of `\anscls[...]{<desc>}`: in

- *answer classes* `<desc>` is a specification of a class of answers and the `pts=` key the points to be awarded to that.
- *answer traits* `<desc>` specifies the deviation from an assumed correct solution, and the value of the `pts=` key specifies changes to the points specified in the problem itself.

Consider for instance the following situation:

```
\begin{sproblem}[pts=3]
  Give an example for a foo and explain it in one sentence.
  \begin{gnote}[title=1 pt for the example and 2 for the explanation]
    \anscls[pts=0,feedback=I did not understand this]{complete gibberish}
    \anscls[pts=-2,feedback=You forgot the explanation]{no explanation}
  \end{gnote}
\end{sproblem}
```

³EdNOTE: This mechanism needs to be further worked out or deprecated; if we keep it we probably need to add `pts` and `feedback` attributes for parity with `\anscls`.

Here we have a problem that is worth three points, and the grading note explains how to treat deviations from a correct solutions – which is not given, since there may be hundreds of examples for foos. The `\anscls` specify this out for grading support systems⁷. The first is an answer class description for the class of answers that cannot be made sense of by the graders and specifies a default grade and feedback. The second specifies the trait of a missing explanation and specifies a deduction commensurate with the grading note title and a feedback to the learner. Note that a grading support system can distinguish answer classes from traits by the form of the value of points: absolute values vs. differences like +5 or -3.

4.2.6 Structured Problems

Problems can be structured into subproblems via the `subproblem` environment. It takes the same arguments and allows the same content model as `sproblem`.

Example 17

Input:

File tutorial/ext/structured-problem.en.tex

```

1 \begin{document}
2 \begin{sproblem}[id=prob.elephants-mod,title=Fitting Elephants Modularly]
3   Consider the problem of fitting elephants into a Volkswagen beetle.
4   \begin{subproblem}[pts=1,min=2]
5     Estimate the number of elephants on the front seats.
6     \begin{solution}
7       Two on each side one.
8     \end{solution}
9   \end{subproblem}
10  \begin{subproblem}[pts=1,min=2]
11    Estimate the number of elephants on the back seats.
12    \begin{solution}
13      Three little elephants.
14    \end{solution}
15  \end{subproblem}

```

Output:

⁷like e.g. <https://gitos.rrze.fau.de/yn06uhoc/vollkorn-prototype>

Exercise (Fitting Elephants Modularly)

Consider the problem of fitting elephants into a Volkswagen beetle.

1. Estimate the number of elephants on the front seats.

Lösung: Two on each side one.

2. Estimate the number of elephants on the back seats.

Lösung: Three little elephants.

3. What is the total number?

Lösung: A family of five; we do not want elephants in the frunk.

By default, subproblems are numbered subordinate to the “mother problem”. The `problem` package does not make any assumptions about whether subproblems can be used independently from their sister problems, or on order requirements.

Note that neither `sproblem` nor `subproblem` can be nested, if you want to have further structure, you need to resort to regular L^AT_EX functionality. Note that you can add `solution` and `gnote` environments wherever you want, so that this need not force you to forego structure.

The `subproblem` environment is however very useful for modularizing problems: `subproblem` environments can appear outside of `sproblem` and in particular at the top-level of a file. This allows them to be shared between `sproblem` via `\inputref`.

4.2.7 Single/Multiple Choice Blocks

Multiple choice blocks can be formatted using the `mcb` environment, in which single choices are marked up with `\mcc` macro.

`\mcc[⟨keyvals⟩]{⟨text⟩}` takes an optional key/value argument `⟨keyvals⟩` for choice metadata and a required argument `⟨text⟩` for the proposed answer text. The following keys are supported

- `T` for correct answers, `F` (the default value) for false ones,
- `Ttext` the verdict for correct answers, `Ftext` for false ones, and
- `feedback` for a short feedback text given to the student.

What we see when this is formatted to PDF depends on the context. In solutions mode (we start the solutions in the code fragment below) we get

Example 18

Input:

```
1 \startsolutions
2 \begin{sproblem}[title=Functions,id=functions1]
3   What is the keyword to introduce a function definition in python?
4   \begin{mcb}
5     \mcc[T]{def} \mcc[F,feedback=that is for C and C++] {function}
6     \mcc[F,feedback=that is for Standard ML]{fun}
7     \mcc[F,Ftext=Noooooooooo,feedback=that is for Java]{public static void}
8   \end{mcb}
9 \end{sproblem}
```

Output:

Exercise (Functions)

What is the keyword to introduce a function definition in python?

- ☒ def
- ☐ function^a
- ☐ fun^b
- ☐ public static void^c

^a

that is for C and C++

^b

that is for Standard ML

^cNoooooooooo

that is for Java

In “exam mode” where disable solutions (here via `\stopsolutions`) we get the questions without solutions (that is what the students see during the exam/quiz).

Example 19

Input:

```
1 \stopsolutions
2 \begin{sproblem}[title=Functions,id=functions1]
3   What is the keyword to introduce a function definition in python?
4   \begin{mcb}
5     \mcc[T]{def} \mcc[F,feedback=that is for C and C++] {function}
6     \mcc[F,feedback=that is for Standard ML]{fun}
7     \mcc[F,Ftext=Noooooooooo,feedback=that is for Java]{public static void}
8   \end{mcb}
9 \end{sproblem}
```

Output:

Exercise (Functions)

What is the keyword to introduce a function definition in python?

- ☐ def
- ☐ function
- ☐ fun
- ☐ public static void

What we have seen is the default rendering of the multiple choice block, which is as an itemize like environment with check boxes. There are alternatives to that which we can choose with the `style` key in the optional attribute of the `mcb` environment. Currently the only alternative is `inline` style:

Example 20

Input:

```
1 \startsolutions
2 \begin{sproblem}[title=Functions,id=functions1]
3   What is the keyword to introduce a function definition in python?
4
5   \begin{mcb}[style=inline]
6     \mcc[T]{def} \mcc[F,feedback=that is for C and C++] {function}
7     \mcc[F,feedback=that is for Standard ML] {fun}
8     \mcc[F,Ftext=Noooooooooooo,feedback=that is for Java] {public static void}
9   \end{mcb}
10 \end{sproblem}
```

Output:

Exercise (Functions)

What is the keyword to introduce a function definition in python?

- [☒ def ☐ function^a ☐ fun^b ☐ public static void^c]

^a

that is for C and C++

^b

that is for Standard ML

^cNoooooooooooo

that is for Java

This is the solutions mode, i.e. with solutions, otherwise, the footnotes will fully disappear.

Analogously, single choice blocks are marked up by the `scc` environment and the individual choices by the `\scc` macro. In PDF, there is little difference to the multiple choice case. In interactive formats like HTML, single choice blocks are typically realized via radio buttons to enforce single choice.⁴

Often we only want to ask whether a statement is true or false. This is essentially a single choice block. $\text{\TeX}$ provides the macros `\yesTnoF`, `\yesFnoT`, `\trueTfalseF`, and `\trueFfalseT` for that. They are used as in the example below:

⁴EdNOTE: MK: are there any differences between `mcc` and `scc` we need to discuss here?

Example 21

Input:

```
1 \begin{sproblem}\startsolutions
2   Mark the following statements as true or false:
3   \begin{enumerate}
4     \item The moon is made of green cheese \trueFfalseT
5     \item \ldots
6   \end{enumerate}
7 \end{sproblem}
```

Output:

Exercise

Mark the following statements as true or false:

1. The moon is made of green cheese ☐ true ☒ false
2. ...

4.2.8 Filling-In Concrete Solutions

The next simplest situation, where we can implement auto-grading is the case where we have fill-in-the-blanks

The `\fillinsol` macro takes a single argument, which contains a concrete solution (i.e. a number, a string, ...), which generates a fill-in-box in test mode:

Example 22

Input:

```
1 \stopsolutions
2 \begin{sproblem}[id=elephants.fillin,title=Fitting Elephants]
3   How many Elephants can you fit into a Volkswagen beetle? \fillinsol{4}
4 \end{sproblem}
```

Output:

Exercise (Fitting Elephants)

How many Elephants can you fit into a Volkswagen beetle?

and the actual solution in solutions mode:

Example 23

Input:

```

1 \startsolutions
2 \begin{sproblem}[id=elephants.fillin,title=Fitting Elephants]
3   How many Elephants can you fit into a Volkswagen beetle? \fillinsol{4}
4 \end{sproblem}

```

Output:

Exercise (Fitting Elephants)

How many Elephants can you fit into a Volkswagen beetle? 4

If we do not want to leak information about the solution by the size of the blank we can also give `\fillinsol` an optional argument with a size: `\fillinsol[3cm]{12}` makes a box three cm wide.

Obviously, the required argument of `\fillinsol` can be used for auto-grading. For concrete data like numbers, this is immediate, for more complex data like strings “soft comparisons” might be in order. Indeed the default matching of answers against the string provided in the required argument of the `\fillinsol` is up to whitespace normalization.

The `problem` package allows to specify different behavior via three additional keys whose arguments are all triples of the form

```
{⟨target⟩}{⟨TF⟩}{⟨feedback⟩}
```

where `⟨target⟩` is the target specification, `⟨TF⟩` is the verdict T or F on the correctness of an answer that is accepted by `⟨target⟩`, and `⟨feedback⟩` a feedback string. In the following (somewhat contrived) example, we see all three at work:

Example 24

Input:

```

1 \begin{sproblem}
2   In 2019, Erlangen is a city of
3   \fillinsol[testspace=3cm,
4   exact={111962}T{Wow, you known your numbers, according to
5     Wikipedia that is exactly correct},
6   numrange={0-1000}F{No,that would be a village},
7   numrange={1001-100000}F{No,that would be a town},
8   numrange={100000-120000}T{Yes, it is close to 109000},
9   numrange={1200001-}F{No, that is too large},
10  regex={-[0-9]+}F{What do negative Inhabitants even mean?},
11  regex={.*}F{Please give a natural number}]
12  {111962}
13  inhabitants.
14 \end{sproblem}

```

Output:

Exercise

In 2019, Erlangen is a city of 111962^a inhabitants.

^a

type	case	verdict	feedback
exact	111962	T	
exact	111962	T	Wow, you know your numbers, according to Wikipedia that is exactly correct
numrange	0-1000	F	No, that would be a village
numrange	1001-100000	F	No, that would be a town
numrange	100000-120000	T	Yes, it is close to 109000
numrange	1200001-	F	No, that is too large
regex	-.[0-9]+	F	What do negative inhabitants even mean?
regex	.*	F	Please give a natural number

- **exact=** gives the exact string (no whitespace normalization).
- **numrange=** specifies a number range, i.e. a comma-separated list of pairs $\langle low \rangle$ - $\langle high \rangle$, where $\langle low \rangle$ and $\langle high \rangle$ are number representations or empty.
- **regex=** gives a POSIX regular expression that specifies all acceptable strings.

4.2.9 Including Problems

`\includeproblem` The `\includeproblem` macro can be used to include a problem from another file. It takes an optional `KeyVal` argument and a second argument which is a path to the file containing the problem (the macro assumes that there is only one problem in the include file). The keys `title`, `min`, and `pts` specify the problem title, the estimated minutes for solving the problem and the points to be gained, and their values (if given) overwrite the ones specified in the `problem` environment in the included file.

The sum of the points and estimated minutes (that we specified in the `pts` and `min` keys to the `problem` environment or the `\includeproblem` macro) to the log file and the screen after each run. This is useful in preparing exams, where we want to make sure that the students can indeed solve the problems in an allotted time period.

The `\min` and `\pts` macros allow to specify (i.e. to print to the margin) the distribution of time and reward to parts of a problem, if the `pts` and `pts` options are set. This allows to give students hints about the estimated time and the points to be awarded.

4.2.10 Testing and Spacing

The `problem` package is often used by the `hwexam` package, which is used to create homework assignments and exams. Both of these have a “test mode” (invoked by the package option `test`), where certain information –master solutions or feedback – is not shown in the presentation.

`\testspace` `\testspace` takes an argument that expands to a dimension, and leaves vertical space accordingly. Specific instances exist: `\testsmallspace`, `\testsmallspace`, `\testsmallspace` `\testsmallspace` give small (1cm), medium (2cm), and big (3cm) vertical space.
`\testsmallspace` `\testnewpage` makes a new page in `test` mode, and `\testemptypage` generates an empty page with the cautionary message that this page was intentionally left empty.
`\testemptypage`

TODO⁸

4.3 Homework Assignments and Exams

4.3.1 Introduction

The `hwexam` package and class supplies an infrastructure that allows to format nice-looking assignment sheets by simply including problems from problem files marked up with the `problem` package. It is designed to be compatible with `problems.sty`, and inherits some of the functionality.

4.3.2 Package Options

<code>solutions</code>	The <code>hwexam</code> package and class take the options <code>solutions</code> , <code>notes</code> , <code>hints</code> , <code>gnotes</code> , <code>pts</code> , <code>min</code> , and <code>boxed</code> that are just passed on to the <code>problems</code> package (cf. its documentation for a description of the intended behavior).
<code>notes</code>	
<code>hints</code>	
<code>gnotes</code>	
<code>pts</code>	
<code>min</code>	
<code>multiple</code>	Furthermore, the <code>hwexam</code> package takes the option <code>multiple</code> that allows to combine multiple assignment sheets into a compound document (the assignment sheets are treated as section, there is a table of contents, etc.).
<code>test</code>	Finally, there is the option <code>test</code> that modifies the behavior to facilitate formatting tests. Only in <code>test</code> mode, the macros <code>\testspace</code> , <code>\testnewpage</code> , and <code>\testemptypage</code> have an effect: they generate space for the students to solve the given problems. Thus they can be left in the L ^A T _E X source.

4.3.3 Assignments

<code>assignment</code> (<i>env.</i>)	This package supplies the <code>assignment</code> environment that groups problems into assignment sheets. It takes an optional KeyVal argument with the keys <code>number</code> (for the assignment number; if none is given, 1 is assumed as the default or — in multi-assignment documents — the ordinal of the <code>assignment</code> environment), <code>title</code> (for the assignment title; this is referenced in the title of the assignment sheet), <code>type</code> (for the assignment type; e.g. “quiz”, or “homework”), <code>given</code> (for the date the assignment was given), and <code>due</code> (for the date the assignment is due).
<code>number</code>	
<code>title</code>	
<code>type</code>	
<code>given</code>	
<code>due</code>	

4.3.4 Including Assignments

<code>\includeassignment</code>	The <code>\includeassignment</code> macro can be used to input an assignment from another file. It takes an optional KeyVal argument and a second argument which is a path to the file containing the problem (the macro assumes that there is only one <code>assignment</code> environment in the included file). The keys <code>number</code> , <code>title</code> , <code>type</code> , <code>given</code> , and <code>due</code> are just as for the <code>assignment</code> environment and (if given) overwrite the ones specified in the <code>assignment</code> environment in the included file.
---	---

⁸TODO: check what is still undescribed `problem.sty` and make examples for it.

4.3.5 Typesetting Exams

`testheading` (*env.*) The `\testheading` takes an optional keyword argument where the keys `duration` specifies a string that specifies the duration of the test, `min` specifies the equivalent in number of minutes, and `reqpts` the points that are required for a perfect grade.

```
reqpts1 \title{320101 General Computer Science (Fall 2010)}
2 \begin{testheading}[duration=one hour,min=60,reqpts=27]
3   Good luck to all students!
4 \end{testheading}
```

Will result in

Name:

Matriculation Number:

320101 General Computer Science (Fall 2010)

2026-06-24

You have one hour (sharp) for the test;

Write the solutions to the sheet.

The estimated time for solving this exam is 23 minutes, leaving you 37 minutes for revising your exam.

You can reach 23 points if you solve all problems. You will only need 27 points for a perfect score, i.e. -4 points are bonus points.

Different problems test different skills and knowledge, so do not get stuck on one problem.

	To be used for grading, do not write here								
prob.	4.1	1.1	1.2	2.1	2.2	2.3	2.4	2.5	2.6
total	0	0	0	10	0	0	0	0	0
reached									

good luck

9

⁹MK: The first three “problems” come from the stex examples above, how do we get rid of this?

Part II

User Manual

*The dynamic [HTML](https://mathhub.info/?a=sTeX/Documentation&d=manual&l=en) version of this part can be found at
`https://mathhub.info/?a=sTeX/Documentation&d=manual&l=en`*

Chapter 5

Basics

5.1 Package and Class Options

- `debug=<prefixes>`: (see Developer Manual)
- `lang=<languages>`: If set, [sTeX](#) will load the `babel` package with the provided languages. Supported languages (currently) are:

<code>ar</code>	arabic
<code>bg</code>	bulgarian
<code>de</code>	german (with option <code>ngerman</code>)
<code>en</code>	english
<code>fi</code>	finnish
<code>fr</code>	french
<code>ro</code>	romanian
<code>ru</code>	russian
<code>tr</code>	turkish (with option <code>shorthands=:!</code>)

- `mathhub=<path>`: Uses the provided file path as **MathHub** directory (see Section 1 ([Math Archives](#) and the **MathHub** Directory) in the [sTeX](#) Manual).
- `usesms/writesms`: If `writesms` is set, content loaded from external [math archives](#) (i.e. [modules](#)) is persisted in the file `\jobname.sms`.

If `usesms` is set, the content of the `.sms`-file is loaded, obviating the need to reprocess the original files.

The options are not mutually exclusive, but care should be taken if dependencies have changed between builds.

This offers two advantages:

1. If a document has many (transitive) dependencies, `usesms` should significantly speed up the build process, and
2. setting `usesms` allows for distributing the `.sms`-file to make the document *standalone*, allowing for compilation without needing imported/used modules to be present.

The options `debug`, `mathhub`, `usesms` and `writesms` can also be set by the environment variables `STEX_DEBUG`, `MATHHUB`, `STEX_USESMS` and `STEX_WRITESMS`. In fact, the `MATHHUB` environment variable is the recommended way to set the MathHub directory. This is the only option where the *package option* overrides the environment variable.

The environment variables for `USE/WRITESMS` are particularly useful, in that they allow for convenient compilation workflows. For example, the Build PDF/XHTML/OMDoc-button in the IDE does the following:

```
STEX_WRITESMS=true pdflatex <job>.tex
[bibtex|biber] <job>
STEX_USESMS=true pdflatex <job>.tex
STEX_USESMS=true pdflatex <job>.tex
```

Guaranteeing (in the first run) that all dependencies are loaded from their respective sources and persisted, and in the two subsequent runs read from the generated `.sms` file, (likely) speeding up the subsequent runs significantly.

5.2 Math Archives and the MathHub Directory

`STEX` uses [math archives](#) to organize document content modularly, without a user having to specify absolute paths, which would differ between users and machines.

All `STEX` archives need to exist in the local MathHub-directory. `STEX` knows where this folder is via one of five means:

1. If the `STEX` package is loaded with the option `mathhub=/path/to/mathhub`, then `STEX` will consider `/path/to/mathhub` as the local MathHub directory.
2. If the `mathhub` package option is *not* set, but the macro `\mathhub` exists when the `STEX`-package is loaded, then this macro is assumed to point to the local MathHub directory; i.e. `\def\mathhub{/path/to/mathhub}\usepackage{stex}` will set the MathHub directory as `path/to/mathhub`.
3. Otherwise, `STEX` will attempt to retrieve the system variable `MATHHUB`, assuming it will point to the local MathHub directory. Since this variant needs setting up only *once* and is machine-specific (rather than defined in tex code), it is compatible with collaborating and sharing tex content, and hence recommended.
4. If that too fails, `STEX` will look for a file `~/.stex/mathhub.path`. If this file exists, `STEX` will assume that it contains the path to the local MathHub-directory. This method is recommended on systems where it is difficult to set environment variables, and is used by the IDE setup.
5. Finally, if all else fails, `STEX` considers `~/MathHub` to be the MathHub directory.

The `STEX` IDE allows you to directly download [math archives](#) from `gl.mathhub.info` – currently available [archives](#) there are:

- `sTeX/*` – a group of semi-experimental documents showcasing `STEX`3.3 features,
- `smglom/*` – a vast collection of multilingual [modules](#) of concepts in mathematics and computer science. The SMGloM predates `STEX`3 and is thus largely “underannotated” with respect to (formal) semantics,
- `courses/*` – a vast collection of lecture slides and notes in computer science for courses held by Michael Kohlhase. They largely make use of SMGloM [modules](#).

5.2.1 The Structure of Math Archives

An **archive** group/name is stored in the directory <MathHub>/group/name; e.g. assuming your local MathHub-directory is set as /user/foo/MathHub, then for the sTeX/Documentation **archive** to be found by the sTeX system, it needs to be in /user/foo/MathHub/sTeX/Documentation.

Each such **archive** needs two subdirectories:

- /source – this is where all your tex files go.
- /META-INF – a directory containing a single file MANIFEST.MF, the content of which we will consider shortly.

An additional lib-directory is optional, and is discussed in section 5.3.

5.2.2 MANIFEST.MF-Files

The MANIFEST.MF in the META-INF directory consists of key-value-pairs, informing sTeX (and associated software, e.g. MMT) of various properties of an **archive**. For example, the MANIFEST.MF of the sTeX/Documentation **archive** looks like this:

```
id: sTeX/Documentation
ns: http://mathhub.info/sTeX/Documentation
narration-base: http://mathhub.info/sTeX/Documentation
format: stex
title: The sTeX Documentation
teaser: The full Documentation for the sTeX system
url-base: https://mathhub.info
dependencies:sTeX/ComputerScience/Software,sTeX/MathTutorial
ignore: */code/*|*/tikz/*|*/tutorial/solution/*
```

Many of these are in fact ignored by sTeX, but some are important:

id: The name of the **archive**, including its group (e.g. sTeX/Document). This is used by the MMT system in favor of the directory, but T_EX's limited access to the file system enforces the directory structure.

source-base or

ns: The namespace from which all **symbol** and **module** MMT-URIs in this **archive** are formed.

narration-base: The namespace from which all document MMT-URI in this repository are formed. It can safely match the **ns**-field.

url-base: A URL that is formed as a basis for *external references*. and hyperlinks. An MMT (or comparable system) instance should run there and host (sTeX-generated) HTML.

dependencies: All **archives** that this **archive** depends on. sTeX ignores this field, but MMT can pick up on them to resolve dependencies, e.g. when downloading **archives** in the IDE, which will also download all dependencies.

ignore: A regular expression of .tex files in the **source** directory that should be ignored; e.g. they will not be compiled when building a whole directory or archive in the IDE.

5.3 The lib-Directory

A **math archive** group/archive may have a **lib** directory primarily intended for preamble code, **packages**, **.bib** files, etc., the files in which can be referenced in various ways.

Additionally, a *group* of archives **group** may have an additional **archive group/meta-inf**. If this **meta-inf** archive has a **/lib**-subdirectory, they too will be considered by the following.

\libinput **\libinput** {some/file} searches for a file **some/file[.tex]** in

- the **lib**-directory of the current archive, and
- the **lib**-directory of a **meta-inf**-archive in (any of) the archive groups containing the current archive

and **\inputs** all found files in reverse order; e.g. **\libinput**{preamble} in a **.tex**-file in **sTeX/Documentation** will *first* input **.../sTeX/meta-inf/lib/preamble.tex** and then **.../sTeX/Documentation/lib/preamble.tex**.

\libinput will throw an error if *no* candidate for **some/file** is found.

\libinput[some/archive]{some/file} will do the same, but starting in the **lib** directory of the **math archive** **some/archive**.

\libusepackage **\libusepackage** [package-options]{some/file} searches for a file **some/file.sty** in the same way that **\libinput** does, but will call **\usepackage**[package-options]{path/to/some/file} instead of **\input**.
\libusepackage throws an error if not *exactly one* candidate for **some/file.sty** is found.

\addmhbibresource **\addmhbibresource** [some/archive]{some/file} searches for a file like **some/file.bib** in **some/archive**'s **lib** directory and calls **\addbibresource** to the result.

\libusetikzlibrary **\libusetikzlibrary** behaves like **\libusepackage** but looks specifically for **tikz** libraries and calls **\usetikzlibrary** on the results.

throws an error if not *exactly one* candidate for the library is found.

A good practice is to have individual **sTeX** fragments follow basically this document frame:

```
\documentclass{stex}
\libinput{preamble}
\setsectionlevel{<your preference>}
\begin{document}
  \IfInputref{}{
    ...
    \maketitle
    \ifstexhtml \else \tableofcontents \fi
  }
  ...
  \IfInputref{}{\libinput{postamble}}
\end{document}
```

Then the `preamble.tex` files can take care of loading the generally required [packages](#), setting presentation customizations etc. (per archive or archive group or both), and a `postamble.tex` can e.g. print the bibliography, index etc.

`\libusepackage` is particularly useful in such a `preamble.tex` when we want to use custom packages that are not part of a [TeX](#) distribution, or on CTAN. In this case we commit the respective packages in one of the `lib` folders and use `\libusepackage` to load them.

5.4 Basic Macros

<code>\sTeX</code>	The <code>\sTeX</code> macro produces this $\text{\TeX}$ logo. It is provided by the <code>stex-logo</code> package , included with the <code>stex</code> package .
<code>\stex</code>	

<code>\ifstexhtml</code>	The TeX conditional <code>\ifstexhtml</code> is <i>true</i> if the current compilation generates HTML , and <i>false</i> otherwise (i.e. generates PDF).
--------------------------	---

<code>\STEXinvisible</code>	<code>\STEXinvisible{<code>}</code>
-----------------------------	---

Processes `<code>`, but does not generate any output. In the [HTML](#), `<code>` is exported with `display:none`.

Can be used to declare formal content and preserve its semantics in [HTML](#) without generating output.

Chapter 6

Document Features

6.1 Document Fragments

`sfragment` (*env.*) To make reusability of document fragments more feasible, `TeX` provides the `sfragment` environment. `\begin{sfragment}[id=<id>,short=<short title>]{section title}` calls `\part`, `\chapter`, `\section`, `\subsection`, `\subsubsection`, `\paragraph` or `\subparagraph` with argument `{section title}` depending on the current *section level* and availability, and increases the level accordingly.

The `<id>` can be used for cross-document references (see Section 0.3 (Cross-Document References) in the `TeX` Manual).

`blindfragment` (*env.*) In the case where we want to increase the section level *without* producing a corresponding section header, the `blindfragment` environment can be used. This allows e.g. typesetting `\sections` before the first `\chapter`.

`\skipfragment` The `\skipfragment` macro “skips an `sfragment`”, i.e. it just steps the respective sectioning counter. This macro is useful, when we want to keep two documents in sync structurally, so that section numbers match up: Any section that is left out in one becomes a `\skipfragment`.

`\setsectionlevel` The `\setsectionlevel` macro sets the current section level to that provided as argument. This is particularly useful in the preamble of a document, as to be ignored in e.g. `\inputref` and make sure that sectioning proceeds as desired; e.g. `\setsectionlevel{section}` make sure that the first `sfragment` will be typeset as a `\section` (rather than e.g. a `\part`).

`\currentsectionlevel`
`\Currentsectionlevel` The `\currentsectionlevel` macro produces the literal string corresponding to the current section level – e.g. within a chapter (but outside of a section), `\currentsectionlevel` produces “chapter”.

The `\Currentsectionlevel` macro does the same, but capitalizes the first letter; e.g. in the above situation, `\Currentsectionlevel` produces “Chapter”.

6.2 Using and Referencing Document Fragments

`\inputref` `\inputref` [`\archive`]{`\file`}

Inputs the file `\file` in `\archive`'s `source` directory. If [`\archive`] is empty, the current `archive`'s `source` directory is used. If there is no current `archive`, `\file` is resolved relative to the current file.

The file's content is processed within a `TeX` group when using `pdflatex`. When converting to `HTML` however, the file is not processed *at all*, and instead, a reference to the file is inserted, that can be replaced by the `HTML` generated by the referenced file by e.g. the `MMT` system.

This is the recommended method to assemble documents from individual `.tex` files.

`\mhinput` Like `\inputref`, but actually calls `\input` in both `PDF` and `HTML` mode. Useful for small fragments or those without `modules`, but generally `\inputref` should be preferred.

`\ifinputref` `\ifinputref` is a `TeX` conditional for whether the current file is currently processed via
`\IfInputref` `\inputref`.

`\IfInputref` {`\true code`}{`\false code`} behaves like

`\ifinputref`{`\true code`}\else{`\false code`}\fi when using `pdflatex`; in `HTML` mode however, *both* arguments are processed and marked-up accordingly, so a hosting server (like `MMT`) can dynamically decide which parts to show or omit.

`\mhgraphics` If the `graphics` package is loaded, `\mhgraphics` takes the same arguments as `\includegraphics`,
`\cmhgraphics` with the additional optional key `archive`. It then resolves the file path in

`\mhgraphics`[`archive=some/archive`]{`some/image`} relative to the `source`-folder of the `some/archive` `archive`. If no `archive` is provided, the file path `some/image` is resolved relative to the current `archive` (if existent).

`\cmhgraphics` additionally wraps the image in a `center`-environment.

`\lstinputmhlisting` Like `\mhgraphics`, but for `\lstinputlisting` instead of `\includegraphics`. Only de-
`\clstinputmhlisting` fined if the `listings` package is loaded.

6.3 Cross-Document References

If we take features like `\inputref` and `\mhinput` (and the `sfragment environment`) seriously and build large documents modularly from individually compiling documents for sections, chapters and so on, cross-referencing becomes an interesting problem.

Say, we have a document `main.tex`, which `\inputrefs` a section `section1.tex`, which references a definition with label `some_definition` in `section2.tex` (subsequently also `\inputrefed` in `main.tex`). Then the numbering of the definition will depend on the *document context* in which the document fragment `section2.tex` occurs - in `section2.tex` itself (as a standalone document), it might be *Definition 1*, in `main.tex`

it might be *Definition 3.1*, and in `section1.tex`, the definition *does not even occur*, so it needs to be referenced by some other text.

What we would want in that instance is an equivalent of `\autoref`, that takes the document context into account to yield something like *Definition 1*, *Definition 3.1* or “*Definition 1 in the section on Foo*” respectively.

For that to work, we need to supply (up to) three pieces of information:

- The *label* of the reference target (e.g. `some_definition`),
- (optionally) the *file*/document containing the reference target (e.g. `section2`). This is not strictly necessary if the reference target occurs in the *same* document, but if not, we need to know where to find the label,
- (optionally) the document context, in which we want to refer to the reference target (e.g. `main`).

Additionally, the document in which we want to reference a label needs a title for external references.

```
\sref \sref [archive=<archive1>,file=<file1>]
      {\<label>}[archive=<archive2>,file=<file2>,title=<title>]
```

This `macro` references `<label>` (declared in `<file1>` in `math archive <archive1>`). If the object (section, figure, etc.) with that label occurs (eventually) in the same document, `\sref` will ignore the second set of optional arguments and simply defer to `\autoref` if that command exists, or `\ref` if the `hyperref` package is not included.

If the referenced object does *not* occur in the current document however, `\sref` will refer to it by the object’s name as it occurs in the file `<file2>` in `archive <archive2>`, followed by the title.

In `HTML` mode, the reference additionally links to the `HTML` of the `file1`.¹⁰

This works by storing labels during compilation in a file `<jobname>.sref`, analogous to e.g. the `.toc`. Note that this consequently requires both `file1.tex` and `file2.tex` to have been compiled previously, to generate the `.sref` file.

For example, doing

```
\sref[file=tutorial/full.en]{sec:basics}[file=tutorial.en,title=the \stex Tutorial]
```

in this very document fragment (`[sTeX/Documentation]macros/sref.en.tex`) will yield [Part I \(The Basics\) in the sTeX Tutorial](#) if compiled itself, or if compiled as part of the `sTeX` manual, and will yield the `\autoref` link [chapter 2](#) in the documentation (which includes the tutorial).

```
\srefsetin \srefsetin [<archive2>]{<file2>}{<title>}
```

Sets a default value for the optional arguments `<archive2>`, `<file2>` and `<title>` of `\sref`. If the second set of optional arguments in `\sref` are omitted, these default values are used. Particularly useful to set in a preamble.

```
\sreflabel \sreflabel {\<label>} sets a label analogous to \label{\<label>}, but for use in \sref.
```

Note that for every `sTeX macro` or `environment` that takes an optional `id=<id>` argument, the `<id>` (if non-empty) generates an `\sreflabel` automatically.

For example, `\begin{sfragment}[id=foo]{Foo}` is equivalent to `\begin{sfragment}{Foo}\sreflabel{foo}`.

`\extref` `\extref` [archive=<archive1>,file=<file1>]
`{<label>}{archive=<archive2>,file=<file2>,title=<title>}`

Like `\sref`, but with the third argument mandatory, `\extref` will *always* produce the output as if `<label>` would *not* occur in the current document.

Chapter 7

Modules and Symbols

7.1 Modules

A `module` is required to declare any new formal content such as `symbols` or `notations` (but not `variables`, which may be introduced anywhere).

`\M` → An `sTeX` `module` corresponds to an `MMT/OMDoc` *theory*. As such
`\M` → it gets assigned an `MMT-URI` (*universal resource identifier*) of the form
`\T` → `<namespace>?<module-name>`.

`smodule` (*env.*) A new module is declared using the basic syntax

`\begin{smodule}[options]{ModuleName}...\end{smodule}`.

A module is required to declare any new formal content such as `symbols` or `notations` (but not `variables`, which may be introduced anywhere).

The `smodule`-environment takes several keyword arguments, all of which are optional:

`title` (`<token list>`) to display in customizations.

`style` (`<string>*`) for use in customizations, see Part 1 (Customizing Typesetting) in the `sTeX` Manual

`id` (`<string>`) for cross-referencing, see `\sreflabel`.

`ns` (`<URI>`) the namespace to use. *Should not be used, unless you know precisely what you're doing.* If not explicitly set, is computed from the containing file and `archive`'s namespace.

`lang` (`<language>`) if not set, computed from the current file name (e.g. `foo.en.tex`).

`sig` (`<language>`) see below.

`\STEXexport` `\STEXexport{<code>}` executes `<code>` immediately and every time the current `module` is being used.



For technical reasons, `\STEXexport` processes its content in the `expl3` category code scheme – what this means is that all spaces are ignored entirely, and the characters `_` and `:` are valid characters in `macro` names.

In practice, this means you will have to use the `~` character for spaces, and if you want to use a subscript `_`, you should use the `macro` `\c_math_subscript_token` instead.

Also, note that no *global* `macro` definitions should happen in `\STEXexport`; this can lead to unexpected behaviour if the containing `module` has been used previously in the current document.

7.1.1 Signature `Modules`, Languages, and Multilinguality

if the current file is a translation of a file with the same base name but a different language suffix, setting `sig=<lang>` will preload the `module` from that language file. This helps ensuring that the (formal) content of both `modules` is (almost) identical across languages and avoids duplication.

For example, we can have a file `Foo.en.tex`, that declares and documents a `module` `Foo` (using `\begin{smodule}{Foo}`). If we put a file `Foo.de.tex` next to it, we can do `\begin{smodule}[sig=en]{Foo}` to have all the content in the `module` `Foo` (as declared in `Foo.en.tex`) available and translate its document content to german.

The `MMT` back-end, when serving `STEX` content as `HTML`, will always attempt to find documentation in the language corresponding to the context; e.g. a user's preference.

7.2 Symbol Declarations

`\symdecl` `\symdecl` `{<mname>}[<options>]`

The `\symdecl` `macro` is the simplest way to introduce a new `symbol`. If `<options>` contains `name=<name>`, then `<name>` is the *name* of the `symbol`; otherwise, `<mname>` is used for the *name*. Additionally, a `semantic macro` `\mname` is generated.

The starred variant `\symdecl*` does not generate a `semantic macro`, in which case the `name`-option is superfluous.

`↪` `\symdecl` introduces a new `MMT/OMDOC constant` in the current `module` (i.e. `↪` `MMT/OMDOC theory`). Correspondingly, they get assigned the `MMT-URI` `↪` `<module-URI>?<constant-name>`.

`\symdecl` takes the following optional arguments:

`name` see above,

`args` the arity of the `symbol` and its `semantic macro`; may be a number 0...9 or a string consisting of the characters `i`, `a`, `b` and `B` of length ≤ 9 ,

`type` the `symbol`'s `type`,

`def` the `symbol`'s definiens,

`return` the [symbol](#)’s *return code* (see below), most commonly the [semantic macro](#) of a [mathematical structure](#),
`assoc` how to resolve arguments of [mode](#) `a` or `B`; may be `pre`, `bin`, `binl`, `binr` or `conj`,
`reorder` how to reorder the arguments in [OMDoc](#) (*advanced*),
`role` [symbols](#) with certain roles are treated in particular ways in [MMT/OMDoc](#) (*advanced*),
`argtypes` [TODO](#)¹¹.

`\textsymdecl` [\textsymdecl](#){*mname*}[*options*]{*code*}

Like [\symdecl](#), but requires that the [symbol](#) has arity 0 (hence [\textsymdecl](#) does not take the `args`-option), and generates a [semantic macro](#) that takes no arguments in either text or math mode, and produces marked-up *code* as output.

Additionally, a [macro](#) [\mname](#)name is generated that produces *code* without any semantic markup.

`\symdef` [\symdef](#){*mname*}[*options*]{*notation*}

Combines the functionalities and optional arguments of [\symdecl](#) and [\notation](#) in one.

7.2.1 Returns

Assume we have a [symbol](#) `foo` with [semantic macro](#) `\foo`, (exemplary) taking two arguments, and `return=code`. If we do `\foo{a}{b}!`, the return code is simply ignored. If we do `\foo{a}{b}` *without* the `!`, here is what happens:

1. [S_{TE}X](#) will replace `#1` and `#2` in *code* by `a` and `b`, yielding *retcode*.
2. [S_{TE}X](#) will insert `<retcode>{\foo{a}{b}!}` in the input token stream.

This means that *code* should contain at most *arity of foo* argument markers, and eat precisely one argument appended to *code*.

When in doubt, we recommend only using [semantic macros](#) for [mathematical structures](#) (with only optional arguments) and `\apply` (with only optional arguments) in `return`.

7.3 Referencing Symbols

`\symref` [\symref](#){*symbol*}{*text*}

`\sr` The [\symref](#) macro (and its short version [\sr](#)) is the most general variant to mark-up arbitrary [L^AT_EX](#) code *text* with the [symbol](#).

¹¹[TODO: experimental](#)



This is as good a place as any other to explain how $\text{\texttt{S\TeX}}$ resolves a string `symbol` to an actual `symbol`.

If `\symbol` is a `semantic macro`, then $\text{\texttt{S\TeX}}$ has no trouble resolving `symbolname` to the full `MMT-URI` of the `symbol` that is being invoked.

However, especially in `\symname` (or if a `symbol` was introduced using `\symdecl*` without generating a `semantic macro`), we might prefer to use the *name* of a symbol directly for readability – e.g. we would want to write `A \symname{natural number} is...` rather than `A \symname{Nat} is...`. $\text{\texttt{S\TeX}}$ attempts to handle this case thusly:

If `symbol` does *not* correspond to a `semantic macro` `\symbol` and does *not* contain a `?`, then $\text{\texttt{S\TeX}}$ checks all `symbols` currently in scope until it finds one, whose name is `symbol`. If `symbol` is of the form `pre?name`, $\text{\texttt{S\TeX}}$ first looks through all `modules` currently in scope, whose full `MMT-URI` ends with `pre`, and then looks for a symbol with name `name` in those. This allows for disambiguating more precisely, e.g. by saying `\symname{Integers?addition}` or `\symname{RealNumbers?addition}` in the case where several `additions` are in scope.

$\text{\texttt{M\TeX}}$ $\text{\texttt{M\TeX}}$ $\text{\texttt{T\TeX}}$ `\symref{<symbol>}{<text>}` in `MMT/OMDoc` generates the term `<OMS name="{<symbol URI>}" />`.

`\symname` `\symname[pre=<pre>,post=<post>]{<symbol>}`

`\sn` If the `symbol` referenced by `<symbol>` has name `name`, this is a shortcut for

`\Symname` `\symref{<symbol>}{<pre>name<post>}`.

`\Sn` For example, given a symbol `agroup` with name `abelian group`, we can do

`\sns` `\symname[pre=Non-,post=s]{agroup}` to produce `Non-abelian groups`.

`\Sns` `\sn` is a shorter variant for `\symname`; `\Symname` and `\Sn` additionally capitalize the first letter. `\sns` and `\Sns` are short for `\sn [post=s]` and `\Sn [post=s]`, respectively.

`\srefsym` `\srefsym{<symbol>}{<text>}`

`\srefsymuri` turns `<text>` into a link to

- The documentation of `<symbol>`, if it occurs in the same document, or
- the `symbol`'s documentation online, based on the containing `math archive`'s `url-base`.

`\srefsymuri` does the same, but expects a `symbol`'s full `MMT-URI` as first argument. This is particularly useful for e.g. customizing highlighting (see chapter 9).

`\symuse` `\symuse{<symbol>}` behaves exactly like a `semantic macro` for `<symbol>`.

7.4 Notations and Semantic Macros

`\notation` `\notation{<symbol>}[<options>]{<code>}`

introduces a new `notation` for the referenced `symbol`.

The starred variant `\notation*` sets this `notation` as the (new) default `notation`.

The optional arguments are:

- `prec=<opprec>;<argprec 1>x...x<argprec n>`: An `operator precedence` and one `argument precedence` for each argument of the `semantic macro`. If no `argument precedences` are given, all `argument precedences` are equal to the `operator precedence`. By default, all `precedences` are 0, unless the `symbol` takes no argument, in which case the `operator precedence` is `\neginfprec` (negative infinity).
`prec=nobrackets` is an abbreviation for `prec=\neginfprec;\infprec x...x\infprec`.
- `op=<code>`: An `operator notation`. If none is given, the notation component marked with `\maincomp` is used. If no `\maincomp` occurs in the `notation`, the default `operator notation` is `\symname{<symbol>}`.
- `variant=<id>`: An id for this `notation`. The key `variant=` can be omitted; i.e. `\notation[foo]` is equivalent to `\notation[variant=foo]`.

`\comp` `\comp` is used to mark notation components in a `\notation` to be highlighted. Additionally, each `notation` can use `\maincomp` at most once to mark the *primary* notation component.



Ideally, `\comp` would not be necessary: Everything in a `notation` that is *not* an argument should be a notation component. Unfortunately, it is computationally expensive to determine where an argument begins and ends, and the argument markers `#n` may themselves be nested in other `macro` applications or `TEX` groups, making it ultimately almost impossible to determine them automatically while also remaining compatible with arbitrary highlighting customizations (such as tooltips, hyperlinks, colors) that users might employ, and that are ultimately invoked by `\comp`.



Note that it is required that

1. the argument markers `#n` never occur inside a `\comp`, and
2. no `semantic macros` may ever occur inside a notation.

Both criteria are not just required for technical reasons, but conceptionally meaningful:

The underlying principle is that the arguments to a `semantic macro` represent *arguments to the mathematical operation* represented by a `symbol`. For example, a `semantic macro` application `\plus{a}{b}` would represent *the actual addition of (mathematical objects) a and b*. It should therefore be impossible for *a* or *b* to be part of a notation component of `\plus`.

Similarly, a `semantic macro` can not conceptually be part of the `notation` of `\plus`,



since a **symbol** represents a *distinct (mathematical) concept with its own semantics*, and **notations** are syntactic representations of the very **symbol** to which the **notation** belongs.

If you want an argument to a **semantic macro** to be a purely syntactic parameter, then you are likely somewhat confused with respect to the distinction between the precise *syntax* and *semantics* of the **symbol** you are trying to declare (which happens quite often even to experienced **TeX** users, like us), and might want to give those another thought - quite likely, the concept you aim to implement does not actually represent a semantically meaningful (mathematical) concept, and you will want to use `\def` and similar native **LaTeX macro** definitions rather than **semantic macros**.

`\setnotation` The first **notation** provided will stay the default **notation** unless explicitly changed:
`\setnotation{\langle symbol \rangle}{\langle id \rangle}` sets the default **notation** of $\langle symbol \rangle$ to that with id $\langle id \rangle$.

7.4.1 Precedences and Bracketing

`\infprec` `\neginfprec` and `\neginfprec` represent *infinitely large* and *infinitely small* **precedences**, respectively.



TeX decides whether to insert parentheses by comparing **operator precedences** to a *downward precedence* p_d with initial value `\infprec`. When encountering a **semantic macro**, **TeX** takes the **operator precedence** p_{op} of the **notation** used and checks whether $p_{op} > p_d$. If so, **TeX** inserts parentheses.

When **TeX** steps into an argument of a **semantic macro**, it sets p_d to the respective **argument precedence** of the **notation** used.

Example 25

Consider **semantic macros** `\plus` and `\mult` taking two arguments, with **notations** $a+b$ and $a \cdot b$ respectively, and **precedences** 100 for `\plus` and 50 for `\mult`.

Consider $\mathcal{\$}\plus\{a,\mult\{b,\plus\{c,d\}\}\mathcal{\$}$ (i.e. $a + b \cdot (c + d)$). Then:

1. **TeX** starts out with $p_d = \text{\code{\infprec}}$.
2. **TeX** encounters `\plus` with $p_{op} = 100$. Since $100 \not> \text{\code{\infprec}}$, it inserts no parentheses.
3. Next, **TeX** encounters the two arguments for `\plus`. Both have no specifically provided **argument precedence**, so **TeX** uses $p_d = p_{op} = 100$ for both and recurses.
4. Next, **TeX** encounters `\mult\{b, \dots\}`, whose **notation** has $p_{op} = 50$.
5. We compare to the current downward **precedence** p_d set by `\plus`, arriving at $p_{op} = 50 \not> 100 = p_d$, so **TeX** again inserts no parentheses.

6. Since the notation of `\mult` has no explicitly set argument precedences, `\TeX` again uses the operator precedence for the arguments of `\mult`, hence sets $p_d = p_{op} = 50$ and recurses.
7. Next, `\TeX` encounters the inner `\plus{c, \dots}` whose notation has $p_{op} = 100$. We compare to the current downward precedence p_d set by `\mult`, arriving at $p_{op} = 100 > 50 = p_d$ – which finally prompts `\TeX` to insert parentheses, and we proceed as before.

`\dobracket` `\dobracket{\langle code \rangle}` wraps parentheses around `\langle code \rangle`.

`\withbrackets` `\withbrackets{\langle left \rangle}{\langle right \rangle}{\langle code \rangle}` uses the opening and closing parentheses `\langle left \rangle` and `\langle right \rangle` for the next pair of parentheses automatically inserted in `\langle code \rangle`.

7.4.2 Notations for Argument Sequences

The following macros can be used in notations that take mode `a` or `B` arguments:

`\argsep` `\argsep{\langle parameter token \rangle}{\langle separator \rangle}`

takes the elements of the argument sequence in position `\langle parameter token \rangle` and separates them by `\langle separator \rangle`.

Note that the first argument *must* be a parameter token of the form `\#k`, and the argument at position `k` of the notation has to have argument mode `a` or `B`.

`\argmap` `\argmap{\langle parameter token \rangle}{\langle code \rangle}{\langle separator \rangle}`

takes the elements of the argument sequence in position `\langle parameter token \rangle`, applies the code `\langle code \rangle` to each of them (which therefore should use `\##1`) and separates them by `\langle separator \rangle`.

For example, the notation `\argmap{\#1}{X^{\##1}}{++}` applied to the argument `\{a,b,c\}` produces $X^a + +X^b + +X^c$.

`\argarraymap` **TODO**¹²

7.4.3 Semantic Macros

Assume we have a semantic macro `\smacro` taking (exemplary) two arguments. The precise behaviour of `\smacro` depends on whether we are in text or math mode.

Math Mode `\smacro!` produces the default operator notation of its symbol. Without `!`, `\smacro` expects at least two arguments, and `\smacro{a}{b}!` produces the default notation of its symbol.

If the symbol has a return code, then `\smacro{a}{b}` continues with executing the return code. Otherwise, `\smacro{a}{b}` also simply produces the default operator notation.

The starred variants `\smacro*` and `\smacro!*` behave as in *text mode*.

Text Mode `\smacro!\{⟨arg⟩}` marks up `⟨arg⟩` similarly to how `\symref{smacro}{⟨arg⟩}` would.

Without the `!`, `\smacro` still only takes a single argument, but it is expected, that within `⟨arg⟩`, the arguments for the `symbol` are explicitly marked up. The `\comp macro` is allowed in `⟨arg⟩` to determine the components of `⟨arg⟩` to be highlighted.

`\arg` The `\arg` macro can be used to explicitly mark the arguments of a semantic macro in text mode.

By default, they are numbered consecutively; e.g. `\smacro{... \arg{a} ... \arg{b}}` determines `a` and `b` to be the (first and second) arguments.

The starred variant `\arg*` allows for marking up the arguments, but does not produce any output. This can be used to provide arguments that are not mentioned in the text we want to mark up, because they are implicitly obvious or mentioned elsewhere.

If we want to change the order of the arguments, we can provide the precise argument number as an optional argument; e.g. `\smacro{... \arg[2]{a} ... \arg[1]{b}}` determines `b` to be the first and `a` to be the second argument.

An argument number may be used repeatedly, if the corresponding argument mode is `a` or `B`.

Applications of semantic macros with arguments are translated to $\textcolor{blue}{\text{M}} \rightarrow \text{MMT/OMDOC}$ as OMA-terms with head `<OMS name="⟨symbol⟩"/>`, or $\textcolor{green}{\text{M}} \rightarrow \text{<OMBIND name="⟨symbol⟩"/>}$, depending on the absence or presence of argument mode `b` or `B` arguments.

Semantic macros with no arguments or invoked with `!` correspond to OMS directly.

7.5 Simple Inheritance

There are three macros that allow for opening a module, making its contents available for use:

`\usemodule` `\usemodule{⟨module⟩}` is allowed anywhere and makes the module's contents available up to the current `TeX` group. This is the right macro to use outside of modules, or when none of its contents use any of the used module's symbols directly (e.g. in types or definientia).

`\requiremodule` `\requiremodule{⟨module⟩}` is only allowed in modules and makes the required module's contents available within the current module. The imported symbols can be safely used in types and definientia, but not in the `return` code of symbols, and the imported content is not exported further – i.e. using the current module does not also open the required module.

`\importmodule` `\importmodule{\langle module \rangle}` is only allowed in `modules` and makes the required `module`'s contents available within the current `module`. The imported `symbols` can be safely used anywhere, and the imported content exported to any `modules` subsequently importing the current one.

$\hookrightarrow$ In `MMT`, every *document* and every `module` induces an `MMT` theory. `\usemodule`
 $\rightarrow$ induces and `MMT` `include` in the document *theory*, `\importmodule` and
 $\rightsquigarrow$ `\requiremodule` both induce an `include` in the `module`'s *theory*.

It is worth going into some detail how exactly `\usemodule`, `\importmodule` and `\requiremodule` resolve their arguments to find the desired `module` – which is closely related to the *namespace* generated for a `module`, that is used to generate its `MMT-URI`.

Ideally, `TeX` would allow for arbitrary `MMT-URIs` for `modules`, with no forced relationships between the *logical* namespace of a `module` and the *physical* location of the file declaring it – like `MMT` in fact allows for.

Unfortunately, `TeX` only provides very restricted access to the file system, so we are forced to generate namespaces systematically in such a way that they reflect the physical location of the associated files, so that `TeX` can resolve them accordingly. Largely, users need not concern themselves with namespaces at all, but for completeness sake, we describe how they are constructed:



- If `\begin{smodule}{Foo}` occurs in a file `/path/to/file/Foo[.<lang>].tex` which does not belong to an `math` *archive*, the namespace is `file://path/to/file`.
- If the same statement occurs in a file `/path/to/file/bar[.<lang>].tex`, the namespace is `file://path/to/file/bar`.

In other words: outside of `math` *archives*, the namespace corresponds to the file URI with the filename dropped iff it is equal to the `module` name, and ignoring the (optional) language suffix.

If the current file is in an *archive*, the procedure is the same except that the initial segment of the file path up to the *archive*'s *source* directory is replaced by the *archive*'s namespace URI.

Conversely, here is how namespaces/URIs and file paths are computed in import statements, exemplary `\importmodule`:



- `\importmodule{Foo}` outside of an *archive* refers to `module` `Foo` in the current namespace. Consequently, `Foo` must have been declared earlier in the same file or, if not, in a file `Foo[.<lang>].tex` in the same directory.
- The same statement *within* an *archive* refers to either the `module` `Foo` declared earlier in the same file, or otherwise to the `module` `Foo` in the *archive*'s top-level namespace. In the latter case, it has to be declared in a file `Foo[.<lang>].tex` directly in the *archive*'s *source* directory.
- Similarly, in `\importmodule{some/path?Foo}` the path `some/path` refers to either the sub-directory and relative namespace path of the current directory



and namespace outside of an `archive`, or relative to the current `archive`'s top-level namespace and `source` directory, respectively.

The `module` `Foo` must either be declared in the file `<top-directory>/some/path/Foo[.<lang>].tex`, or in `<top-directory>/some/path[.<lang>].tex` (which are checked in that order).

- Similarly, `\importmodule[Some/Archive]{some/path?Foo}` is resolved like the previous cases, but relative to the `archive` `Some/Archive` in the MathHub directory.

7.6 Variables and Sequences

`\vardef` `\vardef{<mname>}[<options>]{<notation>}`

Takes the same arguments as `\symdef`, but produces a `variable` rather than a `symbol`. `Variables` definitions are always local to the current `TeX` group and are allowed anywhere (i.e. outside of `modules`).

`<options>` may include the additional keyword `bind`, in which case the `variable` will be appropriately abstracted away in statements (see also `\varbind`).

Unlike `\symdef`, there is no starred variant `\vardef*` – `variables` always generate a `semantic macro`.

The `semantic macro` for a `variable` behaves analogously to that of a `symbol`.

$\xrightarrow{M}$ `Variables` induces the same `MMT/OMDOC` terms as `symbols` do, except for the head of the term being `<OMV name="...">` instead of `<OMS/>`.

`\varnotation` `\varnotation{<variable>}[<options>]{<notation>}`

Takes the exact same arguments as `\notation`, but attaches an additional `notation` to the `variable` `<variable>` rather than a `symbol`.

`\svar` `\svar[<name>]{<text>}`

Semantically marks up `<text>` as representing a `variable` `<name>`. The `variable` does not need to have been defined prior. If no `<name>` is given `<text>` will be used as the name.

This is useful in situations like “throwaway expressions” or remarks; e.g.

`\plus{\svar{n},\svar{m}}` means...

`\varseq` `\varseq{⟨mname⟩}[⟨options⟩]{⟨range⟩}{⟨notation⟩}`

Declares a new `variable` sequence. The `⟨options⟩` are the same as for `\vardef`. If not provided, `args=1` by default (a 0-ary sequence would just be a normal `variable`).

A `type` (given as `type=`) is interpreted to be the `type` of every element a_i of the sequence $a_1, \dots, a_n$ (not of the sequence itself). If the `type` is itself a sequence $A_1, \dots, A_n$, the assumption is that its range is the same as the one of the new sequence, and the type of every a_i in the sequence is A_i .

`⟨range⟩` needs to be a comma-separated sequence of either `args` many arguments, or `\ellipses`.

The resulting `semantic macro` is allowed anywhere `gTeX` expects an `argument mode` a or B argument.

`\ellipses` Represents ellipses in a range; produces `\ellipses` in math mode.

`\seqmap` `\seqmap{⟨code⟩}{⟨sequence⟩}`

Maps the function `⟨code⟩` (containing `#1`) over every element of the `⟨sequence⟩`.

Is allowed anywhere `gTeX` expects an `argument mode` a or B argument.

7.7 Structures

`Mathematical structure` bundle interdependent `symbols` together.

`mathstructure (env.)` `\begin{mathstructure}{⟨mname⟩}[⟨name⟩,this=⟨code⟩]` opens a new `mathematical structure` with name `⟨mname⟩` (if provided) or `⟨mname⟩` (otherwise), and `semantic macro` `\mname`. It subsequently behaves like the `smodule environment`.

`\this` The optional `this=⟨code⟩` option allows for setting the typesetting of the `\this` macro within a `mathstructure`. In particular, `\this` can be used in `notations` for `symbols` declared in the `structure`. `\this` can be thought of as representing “the” (current) instance of this `structure`.

`extstructure (env.)` `\begin{extstructure}{⟨mname⟩}[⟨name⟩,this=⟨code⟩]{⟨structs⟩}` opens a new `mathematical structure` extending the `structures` given in `⟨structs⟩` (a comma-separated list of names).

`extstructure* (env.)` `\begin{extstructure*}{⟨struct⟩}` opens a new `mathematical structure` *conservatively* extending the (single) `structure` `⟨struct⟩`. *Conservative* meaning: Every `symbol` newly introduced in this `structure` needs to have a *definiens*. The new `symbols` are attached as fields directly to `⟨struct⟩`.

`\usestructure` The `\usestructure` macro behaves like `\usemodule` for `mathematical structures`, making the `symbols` available to use directly.

$\hookrightarrow$ `mathstructure` make use of the *Theories-as-Types* paradigm (see
 $\hookrightarrow$ [MueRabKoh:tat18]):
 $\hookrightarrow$

$\hookrightarrow$ `\begin{mathstructure}{\langle name \rangle}` creates a nested **theory** with name `\langle name \rangle`-module. The **constant** `\langle name \rangle` is defined as a *dependent record type* with *manifest fields*, the fields of which are generated from (and correspond to) the **constants** in `\langle name \rangle`-module.

7.7.1 Semantic Macros for Structures

Assume we have a **mathematical structure** with semantic macro `\struct`:

Example 26

```

\begin{mathstructure}{\struct}
  \symdef{fieldda}{a}
  \symdef{fielddb}[args=2]{#1 \maincomp{b} #2}
  \symdef{fielddc}[args=2,def=\sn{fielddb}]{#1 \maincomp{c} #2}
  \inlineass[name=axiom1]{\conclusion{some axiom}}
\end{mathstructure}
\notation{\struct}{\StRuCt}

```

- If `\struct` has no **notations**, then `$_\struct!` produces $\langle a, b, c \rangle$. Otherwise, it produces the notation, i.e. `\StRuCt`. In both cases, `$_\struct\{\}\{\}` produces $\langle a, b, c \rangle$ (for reasons that will become clearer in a moment).
- `$_\struct\{\}\{\}` (or `$_\struct!` in the case where no **notations** are around) can be modified in the following ways:
 - `$_\struct\{\}\{[\langle fieldname \rangle], \dots\}` lets you pick, which precise fields to show, so e.g. `$_\struct\{\}\{fieldda,fielddb\}` produces $\langle a, b \rangle$. By default, `$_\struct\{\}\{\}` shows exactly the fields that have **semantic macros** (which are also used to access the fields in `$_\struct\{\dots\}\{\langle fieldname \rangle\}`).
 - `$_\struct\{\}\{[\langle id \rangle]\}` lets you pick the **notation** of the “**mathematical structure**” symbol to use to typeset the **structure**; e.g. `$_\struct\{\}\{[parens]\}` yields $\langle a, b, c \rangle$, and (combining both) `$_\struct\{\}\{fieldda,fielddb\}[angle]` yields $\langle a, b \rangle$.
- The two arguments in `$_\struct\{first\}\{second\}` represent 1. the term that is to be treated as an instance of `\struct`, and 2. the precise field to invoke. If the first is empty, then there is no instance. If the second is empty, **TeX** will present all of them (that are not assertions). Hence, `$_\struct\{S\}\{\}` yields $\langle a_S, b_S, c_S \rangle$, `$_\struct\{S\}\{fieldda\}` produces a_S , `$_\struct\{\}\{fieldda\}` produces a , and `$_\struct\{S\}\{fielddb\}\{x\}\{y\}` produces xb_{Sy} .
- For the sake of completion, `$_\struct\{first\}!` simply produces the given argument; e.g. `$_\struct\{S\}!` simply produces S .

More precisely: `$_\struct\{\langle code \rangle\}` acts like a “type coercion” of `\langle code \rangle` to be an instance of `\struct`.

Of course, it is (occasionally, but) rarely useful to use the **semantic macro** `\struct` by giving it two arguments *manually*; but this is what **TeX** does when using `\struct` in the **return** of a **symbol** (or **variable**).

Continuing:

- $\$ \backslash \text{struct}[\text{comp}=\langle \text{code} \rangle]\{\dots\}\{\dots\}$ applies $\langle \text{code} \rangle$ (using #1) to every occurrence of $\backslash \text{maincomp}$ in the *notations* of the fields, as a replacement for the default modification $\{\#1\}_{\backslash \text{this}}$. For example, $\$ \backslash \text{struct}[\text{comp}=\{\#1\}^{\text{Foo}}]\{\text{S}\}\{\}$ produces $\langle a^{\text{Foo}}, b^{\text{Foo}}, c^{\text{Foo}} \rangle$, and $\$ \backslash \text{struct}[\text{comp}=\{\#1\}^{\backslash \text{this}}]\{\text{S}\}\{\text{fieldb}\}\{\text{x}\}\{\text{y}\}$ yields xb^Sy .
- $\$ \backslash \text{struct}[\text{this}=\langle \text{code} \rangle]\{\dots\}\{\dots\}$ modifies the way $\backslash \text{this}$ is being typeset; i.e. the presentation of the first $\{\dots\}$ argument. For example $\$ \backslash \text{struct}[\text{this}=\text{T}]\{\text{S}\}\{\}$ produces $\langle a, b, c \rangle$ – since $\backslash \text{this}$ is not being used in the *notations* of the fields. Note that the `this=` and `comp=` variants are (as of yet) mutually incompatible.
- Finally, $\$ \backslash \text{struct}[\langle \text{fieldname} \rangle=\langle \text{code} \rangle]\{\dots\}\{\dots\}$ assigns the field $\langle \text{fieldname} \rangle$ to the term $\langle \text{code} \rangle$. $\langle \text{code} \rangle$ is subsequently used when using $\{\dots\}\{\}$, but not in fields directly. For example, $\$ \backslash \text{struct}[\text{fielda}=\text{A}]\{\text{S}\}\{\}$ produces $\langle \text{A}!, b_S, c_S \rangle$, but $\$ \backslash \text{struct}[\text{fielda}=\text{A}]\{\text{S}\}\{\text{fielda}\}$ produces a_S .

Note the insertion of ! behind the A – this is to make sure that assignments to *semantic macros* that takes arguments don't accidentally eat more than they should.

Also note that multiple assignments can be done in the same pair of [], or chained – i.e. both $\$ \backslash \text{struct}[\text{fielda}=\text{A}, \text{fieldb}=\text{B}] \dots \$$ and $\$ \backslash \text{struct}[\text{fielda}=\text{A}][\text{fieldb}=\text{B}] \dots \$$ are valid and equivalent. $\$ \backslash \text{struct}[\text{fielda}=\text{A}, \text{fielda}=\text{B}] \dots \$$ however is not – every field may be assigned at most once.

Chapter 8

Statements

`STEX` provides four environments to semantically annotate various kinds of statements:

`sdefinition (env.)` The `sdefinition environment` represents (primarily mathematical) *definitions*; in particular for `symbols`. The contents of the environment will be used as *documentation* for any `symbol` that either occurs as a `\definiendum` (or `\definame`) within the `sdefinition`, or that is listed in the optional `for=` argument of the `environment`.

If a `\definiens` occurs, this will be used by `MMT` as the formal *definiens* for the respective `symbol`.

`sassertion (env.)` The `sassertion environment` represents *assertions*, i.e. `propositions` such as *theorems*, *lemmata*, *axioms*, etc. If a `\conclusion` occurs within the `sassertion`, its argument will be used as the formal *statement* of the assertion.

`sexample (env.)` The `sexample environment` represents examples, the `for=` attribute specifies the concept this is an example for. For `counterexamples` use `style=counterexample`

`sparagraph (env.)` The `sparagraph environment` represents all other kinds of (logical) paragraphs, such as remarks, comments, transitions between topics, recaps, reminders, etc.

All of these take the same arguments:

- `for=<cs1>`: a comma-separated list of `symbols`.
- The same optional arguments as `\symdecl`, with `macro=` replacing the name of the `semantic macro`. All of them are only relevant, if either `name=` or `macro=` are provided.

As with `\symdecl`, if no `name` is given, but `macro` is, then the same name is used for both the `semantic macro` and the `symbol` itself.

If `name` is given but `macro` isn't, no `semantic macro` is generated. Subsequently, the newly generated `symbol` is added the `for-list`.

- `style`: see chapter 9.
- `title`: a title to use in various styles (see chapter 9).
- `id`: a label to use for `\sref`.

<code>\inlineass</code>	The macros <code>\inlineass</code> , <code>\inlinedef</code> and <code>\inlineex</code> behave like the <code>sassertion</code> , <code>sdefinition</code> and <code>sexample</code> environments respectively, but take the text to annotate as an argument, rather than as the body of an <code>environment</code> , and do not break paragraphs.
<code>\inlinedef</code>	
<code>\inlineex</code>	

The same macros available in the `environments` are also available in the argument of the `\inline*` macros.

<code>\varbind</code>	<code>\varbind{<cls>}</code>
-----------------------	------------------------------------

retroactively attaches the `bind` option to every `variable` provided (as a comma-separated list).

8.1 More on Definitions

In `sdefinition` (and `sparagraph` with `style=symdoc`), the following additional macros are available:

<code>\definiendum</code>	The <code>\definiendum</code> macro behaves largely like <code>\symref</code> , but it uses a dedicated highlighting for <i>definienda</i> and adds the referenced <code>symbol</code> to the <code>for=</code> list of the <code>environment</code> . <code>\definame</code> is to <code>\definiendum</code> as <code>\symname</code> is to <code>\symref</code> . Analogously, <code>\Definame</code> behaves like <code>\Symname</code> . <code>\defnotation</code> can be used in math mode to apply the <code>\definiendum</code> highlighting to <i>notations</i> .
<code>\definame</code>	
<code>\Definame</code>	
<code>\defnotation</code>	

<code>\definiens</code>	The <code>\definiens</code> macro can be used to semantically annotate the <i>definiens</i> in a <code>sdefinition</code> .
-------------------------	---

If the `sdefinition` environment has several elements in its `for` list, an optional argument `\definiens[<symbol>]{...}` can be used to tell `STEX` which `symbol`'s *definiens* this is. By default, the *first* `symbol` in the `for` list is used.

Here is how `MMT` will treat the fragment marked up with `\definiens`:

Firstly, it will attempt to translate its contents into an `MMT/OMDOC` term. This succeeds easily if `\definiens` is some `semantic macro` (applied to arguments).

Secondly, it will check for all `variables` currently in scope, that were defined with the optional argument `bind`. It then will check, whether a `symbol` is in scope, that has `role=lambda`. If so, it will use that `lambda symbol` to bind all these variables (in the order in which they were defined) in the term. If no `lambda symbol` is found, it will use the `bind symbol` that ships with `STEX`.

The final term will be attached as *definiens* to the corresponding `MMT constant`, if it was declared in the same `module` as the `\definiens` occurrence.

8.2 More on Assertions

$\backslash\text{premise}$ $\backslash\text{conclusion}$	The $\backslash\text{conclusion}$ macro can be used to mark up the actual statement within an sassertion. The $\backslash\text{premise}$ macro can be used to additionally mark up <i>premises</i>.
---	--

If the sassertion environment has several elements in its `for` list, an optional argument $\backslash\text{conclusion}[\langle\text{symbol}\rangle]\{\dots\}$ can be used to tell $\text{\LaTeX}$ which symbol 's statement this is. By default, the *first* symbol in the `for` list is used.

Here is how MMT will treat the fragments marked up with $\backslash\text{conclusion}$ and $\backslash\text{premise}$:

Firstly, it will attempt to translate the contents of $\backslash\text{conclusion}$ into an MMT/OMDOC term c . This succeeds easily if the $\backslash\text{conclusion}$ is some *semantic macro* (applied to arguments).

Secondly, it will collect all fragments marked up with $\backslash\text{premise}$ and do the same to them $(p_1, \dots, p_n)$.

It will then check, whether a symbol is in scope, that has `role=implication`. If so, it will use that *implication symbol* to attach all the premises to the conclusion, resulting in $t := \text{imply}(p_1, \dots, p_n, c)$.

Next, it will check for all *variables* currently in scope, that were defined with the optional argument `bind`. It then will check, whether a symbol is in scope, that has `role=forall`. If so, it will use that *forall symbol* to bind all these variables (in the order in which they were defined) in the term t .

Finally, it will check, whether a symbol is in scope, that has `role=judgment`. If so, it will set $t := \text{judgment}(t)$.

If no *forall symbol* is found, it will first apply the *judgment symbol* (if existent) and then use the *bind symbol* that ships with $\text{\LaTeX}$ to bind the *variables*.

The final term will be attached as *type* to the corresponding MMT *constant*, if it was declared in the same *module* as the $\backslash\text{definiens}$ occurrence.

$\hookrightarrow$ M $\rightarrow$
 $\hookrightarrow$ M $\rightarrow$
 $\hookrightarrow$ T $\rightarrow$

$\text{sproof}(\text{env.})$

TODO¹³

¹³TODO: proofs

Chapter 9

Customizing Typesetting

There are two kinds of typesetting that can be customized in $\text{\texttt{STEX}}$: **symbol** references (**notation** components, $\text{\texttt{\symref}}$, **variables**, etc.) are highlighted using a small set of **macros** that can be simply redefined by authors.

Other **macros** and **environments** usually have more complicated “typesetting rules” associated with them – often in the form of other already existing **environments** that should be used.

Lastly, in **HTML** we can provide custom CSS rules in **math archives** that determine the styling of certain **environments**, so that the actual presentation depends on the document in which the fragments are included (e.g. via $\text{\texttt{\inputref}}$), rather than the file the fragment is implemented in.

It is generally recommended to implement these customizations in a preamble in the **lib** directory (see Section 0.3 (The **lib**-Directory) in the $\text{\texttt{STEX}}$ Manual)

9.1 Highlighting **Symbol** References

 $\text{\texttt{\symrefemph}}$
 $\text{\texttt{\symrefemph@uri}}$

$\text{\texttt{\symrefemph}}$ governs how references via $\text{\texttt{\symref}}$ (or $\text{\texttt{\symname}}$, or their short variants) are highlighted;

Doing $\text{\texttt{\symref}}\{\langle symbol \rangle\}\{\langle text \rangle\}$ ultimately expands to $\text{\texttt{\symrefemph@uri}}\{\langle text \rangle\}\{\langle symbol URI \rangle\}$, the default implementation of which is just $\text{\texttt{\symrefemph}}\{\langle text \rangle\}$. The default implementation of $\text{\texttt{\symrefemph}}$, in turn, is just $\text{\texttt{\emph}}\{\langle text \rangle\}$.

If you only want to change e.g. the color of $\text{\texttt{\symrefs}}$, you only need to redefine $\text{\texttt{\symrefemph}}$, e.g. using

```
\renewcommand\symrefemph[1]{\textcolor{red}{#1}}
```

the **@uri** variant is useful if you want to link somewhere, or show the URI in a tooltip. The **stex-highlighting package** does both, using:

```
\usepackage{pdfcomment}
\protected\def\symrefemph@uri#1#2{%
  \pdftooltip{%
    \srefsymuri{#2}{\symrefemph{#1}}%
  }{%
    URI:~\detokenize{#2}%
  }%
}
```



```

}
\def\symrefemph#1{%
  \ifcsname textcolor\endcsname
    \textcolor{teal}{#1}%
  \else#1\fi
}

```

<code>\compemph</code> <code>\compemph@uri</code>	Like <code>\symrefemph</code> and <code>\symrefemph@uri</code> , but governs the highlighting of components (marked with <code>\comp</code> or <code>\maincomp</code>) in <i>notations</i> .
--	---

<code>\defemph</code> <code>\defemph@uri</code>	Like <code>\symrefemph</code> and <code>\symrefemph@uri</code> , but governs the highlighting of definienda marked with <code>\definiendum</code> (or <code>\definame</code>).
--	---

<code>\varemph</code> <code>\varemph@uri</code>	Like <code>\compemph</code> and <code>\compemph@uri</code> , but governs the highlighting of components (marked with <code>\comp</code> or <code>\maincomp</code>) in the <i>notations</i> of <i>variables</i> . The second argument to <code>\varemph@uri</code> is the <i>name</i> of the <i>variable</i> .
--	---

9.2 Styling Environments and Macros

A variety of *environments* and *macros* provided by $\text{\texttt{S\TeX}}$ are *stylable* using the *macros* `\stexstyle<name>[<style>]{...}`. These stylable *environments* and *macros* bind various of their parameters to *macros* `\this<parameter>`, which can then be used in the styles.

For example, if we have a *definition environment* that we would want to use to style our *sdefinitions*, we can do (in the simplest case)

```
\stexstyledefinition{\begin{definition}}{\end{definition}}
```

This tells $\text{\texttt{S\TeX}}$ to insert `\begin{definition}` at the beginning of every *sdefinition environment*, and `\end{definition}` at the end.

If we have a *environments theorem* and *lemma*, we probably want the *sassertion environment* to use those for theorems and lemma. We can achieve that by doing

```
\stexstyleassertion[theorem]{\begin{theorem}}{\end{theorem}}
\stexstyleassertion[lemma]{\begin{lemma}}{\end{lemma}}
```

Now if we do `\begin{sassertion}[style=theorem]`, it will wrap the *environment* with `\begin{theorem}... \end{theorem}`.

Of course, many such statements might have a title, as e.g. in

Satz 9.2.1 (Gödel's First Incompleteness Theorem). ...

In *sassertion*, we can provide that title as optional argument using `title=...`. Before calling the styling provided, *sassertion* will store that title in the macro `\thistitle`, which we can use in the styling. For example, we might prefer to pass it on to the *theorem environment*:

```
\stexstyleassertion[theorem]{\ifx\thistitle\@empty
\begin{theorem}\else\begin{theorem}[\thistitle]\fi}
{\end{theorem}}}
```

```
\stexstylemodule
\stexstylecopymodule
\stexstyleinterpretmodule
\stexstylerealization
\stexstylemathstructure
\stexstyleextstructure
\stexstyledefinition
\stexstyleassertion
\stexstyleexample
\stexstyleparagraph
\stexstyleproof
\stexstylesubproof
```

TODO¹⁴

Additionally, we can style certain [macros](#), if we want them to produce output. For example, we might (for debuggin or documentation purposes) `\symdecl` to give a short summary of the symbol.

We can achieve that by doing, for example:

```
\stexstylesymdecl[debug]{
Symbol \thisdeclname~(with arity \thisargs) of type $\thistype$.
}
```

in which case

```
\symdecl{foo}[args=2,type=\mathbb{N},style=debug]
```

will yield

Symbol foo (with arity ii) of type N.

```
\stexstyleusemodule
\stexstyleimportmodule
\stexstylerequiremodule
\stexstyleassign
\stexstylerenamedecl
\stexstyleassignMorphism
\stexstylecopymod
\stexstyleinterpretmod
\stexstylerealize
\stexstylesymdecl
\stexstyletextsymdecl
\stexstylenotation
\stexstylevarnotation
\stexstylesymdef
\stexstylevardef
\stexstylevarseq
\stexstylepsfsketch
\stexstyleMMTinclude
```

TODO¹⁵

9.3 Custom CSS for Environments

Chapter 10

Additional Packages

10.1 NotesSlides Manual

10.1.1 Introduction

The `notesslides` document class is derived from `beamer.cls` [`beamerclass:on`], it adds a “notes version” for course notes that is more suited to printing than the one supplied by `beamer.cls`.

The `notesslides` class takes the notion of a slide frame from Till Tantau’s excellent `beamer` class and adapts its notion of frames for use in the $\text{\LaTeX}$ and OMDoc. To support semantic course notes, it extends the notion of mixing frames and explanatory text, but rather than treating the frames as images (or integrating their contents into the flowing text), the `notesslides` package displays the slides as such in the course notes to give students a visual anchor into the slide presentation in the course (and to distinguish the different writing styles in slides and course notes).

In practice we want to generate two documents from the same source: the slides for presentation in the lecture and the course notes as a narrative document for home study. To achieve this, the `notesslides` class has two modes: *slides mode* and *notes mode* which are determined by the package option.

10.1.2 Package Options

The `notesslides` class takes a variety of class options:

<code>slides</code> <code>notes</code>	The options <code>slides</code> and <code>notes</code> switch between slides mode and notes mode (see subsection 4.1.3).
---	--

<code>sectocframes</code>	If the option <code>sectocframes</code> is given, then for the <code>sfragments</code> , special frames with the <code>sfragment</code> title (and number) are generated.
---------------------------	---

<code>frameimages</code> <code>fiboxed</code>	If the option <code>frameimages</code> is set, then slide mode also shows the <code>\frameimage</code> -generated frames (see ???). If also the <code>fiboxed</code> option is given, the slides are surrounded by a box.
--	---

10.1.3 Notes and Slides

frame (*env.*) Slides are represented with the **frame** environment just like in the **beamer** class, see [Tantau:ugbc] for details.

note (*env.*) The **notesslides** class adds the **note** environment for encapsulating the course note fragments.



Note that it is essential to start and end the **notes** environment at the start of the line – in particular, there may not be leading blanks – else L^AT_EX becomes confused and throws error messages that are difficult to decipher.

By interleaving the **frame** and **note** environments, we can build course notes as shown here:

```
1 \ifnotes\maketitle\else
2 \frame[noframenumbering]\maketitle\fi
3
4 \begin{note}
5   We start this course with ...
6 \end{note}
7
8 \begin{frame}
9   \frametitle{The first slide}
10  ...
11 \end{frame}
12 \begin{note}
13   ... and more explanatory text
14 \end{note}
15
16 \begin{frame}
17   \frametitle{The second slide}
18   ...
19 \end{frame}
20 ...
```

\ifnotes Note the use of the **\ifnotes** conditional, which allows different treatment between **notes** and **slides** mode – manually setting **\notestru**e or **\notesfalse** is strongly discouraged however.



We need to give the title frame the **noframenumbering** option so that the frame numbering is kept in sync between the slides and the course notes.



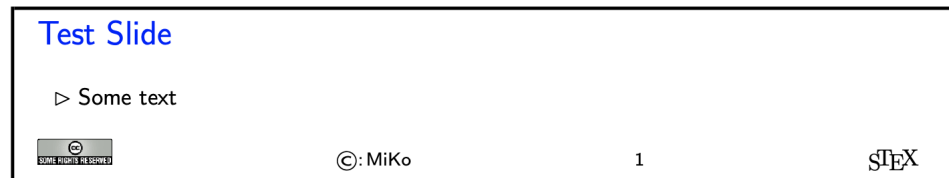
The **beamer** class recommends not to use the **allowframebreaks** option on frames (even though it is very convenient). This holds even more in the **notesslides** case: At least in conjunction with **\newpage**, frame numbering behaves funnily (we have tried to fix this, but who knows).

\inputref* If we want to transclude a the contents of a file as a note, we can use a new variant **\inputref*** of the **\inputref** macro: **\inputref*{foo}** is equivalent to **\begin{note}\inputref{foo}\end{note}**.

nparagraph (*env.*) There are some environments that tend to occur at the top-level of **note** environments. We make convenience versions of these: e.g. the **nparagraph** environment is just an **nparagraph** (*env.*) **sparagraph** inside a **note** environment (but looks nicer in the source, since it avoids one **nexample** (*env.*) level of source indenting). Similarly, we have the **nfragment**, **ndefinition**, **nexample**, **nsproof** (*env.*) **nsproof**, and **nassertion** environments. **nassertion** (*env.*)

10.1.4 Customizing Header and Footer Lines

The **notesslides** package and class comes with a simple default theme named **sTeX** that provided by the **beamterthemesTeX**. It is assumed as the default theme for **sTeX**-based notes and slides. The result in **notes** mode (which is like the **slides** version except that the slide height is variable) is



The footer line can be customized. In particular the logos.

\setslidelogo The default logo provided by the **notesslides** package is the **sTeX** logo it can be customized using **\setslidelogo{<logo name>}**.

\setsource The default footer line of the **notesslides** package mentions copyright and licensing. In **notesslides** **\source** stores the author's name as the copyright holder. By default it is the author's name as defined in the **\author** macro in the preamble. **\setsource{<name>}** can change the writer's name.

\setlicensing For licensing, we use the Creative Commons Attribution-ShareAlike license by default to strengthen the public domain. If package **hyperref** is loaded, then we can attach a hyperlink to the license logo. **\setlicensing[<url>]{<logo name>}** is used for customization, where **<url>** is optional.

10.1.5 Frame Images

Sometimes, we want to integrate slides as images after all – e.g. because we already have a PowerPoint presentation, to which we want to add **sTeX** notes.

<code>\frameimage</code> <code>\mhframeimage</code>	In this case we can use <code>\frameimage[⟨opt⟩]{⟨path⟩}</code>, where <code>⟨opt⟩</code> are the options of <code>\includegraphics</code> from the <code>graphicx</code> package [CarRah:tpp99] and <code>⟨path⟩</code> is the file path (extension can be left off like in <code>\includegraphics</code>). We have added the <code>label</code> key that allows to give a frame label that can be referenced like a regular <code>beamer</code> frame.
--	---

The `\mhframeimage` macro is a variant of `\frameimage` with repository support. Instead of writing

```
1 \frameimage{\MathHub{fooMH/bar/source/baz/foobar}}
```

we can simply write (assuming that `\MathHub` is defined as above)

```
1 \mhframeimage[fooMH/bar]{baz/foobar}
```

Note that the `\mhframeimage` form is more semantic, which allows more advanced document management features in `MathHub`.

If `baz/foobar` is the “current module”, i.e. if we are on the `MathHub` path `...MathHub/fooMH/bar...`, then stating the repository in the first optional argument is redundant, so we can just use

```
1 \mhframeimage{baz/foobar}
```

<code>\textwarning</code>	The <code>\textwarning</code> macro generates a warning sign: 
---------------------------	---

10.1.6 Ending Documents Prematurely

<code>\prematurestop</code> <code>\afterprematurestop</code>	For prematurely stopping the formatting of a document, <code>TeX</code> provides the <code>\prematurestop</code> macro. It can be used everywhere in a document and ignores all input after that – backing out of the <code>sfragment</code> environments as needed. After that – and before the implicit <code>\end{document}</code> it calls the internal <code>\afterprematurestop</code>, which can be customized to do additional cleanup or e.g. print the bibliography.
---	---

`\prematurestop` is useful when one has a driver file, e.g. for a course taught multiple years and wants to generate course notes up to the current point in the lecture. Instead of commenting out the remaining parts, one can just move the `\prematurestop` macro. This is especially useful, if we need the rest of the file for processing, e.g. to generate a theory graph of the whole course with the already-covered parts marked up as an overview over the progress; see `import_graph.py` from the `lmhtools` utilities [lmhtools:github:on].

Text fragments and modules can be made more re-usable by the use of global variables. For instance, the admin section of a course can be made course-independent (and therefore re-usable) by using variables (actually token registers) `courseAcronym` and `courseTitle` instead of the text itself. The variables can then be set in the `TeX` preamble of the course notes file.

10.1.7 Global Document Variables

To make document fragments more reusable, we sometimes want to make the content depend on the context. We use **document variables** for that.

<code>\setSGvar</code>	<code>\setSGvar{<vname>}{<text>}</code> to set the global variable <code><vname></code> to <code><text></code> and <code>\useSGvar{<vname>}</code>
<code>\useSGvar</code>	to reference it.

<code>\ifSGvar</code>	With <code>\ifSGvar</code> we can test for the contents of a global variable: the macro call <code>\ifSGvar{<vname>}{<val>}{<ctext>}</code> tests the content of the global variable <code><vname></code> , only if (after expansion) it is equal to <code><val></code> , the conditional text <code><ctext></code> is formatted.
-----------------------	---

10.1.8 Excursions

In course notes, we sometimes want to point to an “excursion” – material that is either presupposed or tangential to the course at the moment – e.g. in an appendix. The typical setup is the following:

```
1 \excursion{founif}{../fragments/founif.en}
2 {We will cover first-order unification in}
3 ...
4 \begin{appendix}\printexcursions\end{appendix}
```

It generates a paragraph that references the excursion whose source is in the file `../fragments/founif.en.tex` and automatically books the file for the `\printexcursions` command that is used here to put it into the appendix. We will look at the mechanics now.

<code>\excursion</code>	The <code>\excursion{<ref>}{<path>}{<text>}</code> is syntactic sugar for
-------------------------	---

```
1 \begin{nparagraph}[title=Excursion]
2   \activateexcursion{founif}{../ex/founif}
3   We will cover first-order unification in \sref{founif}.
4 \end{nparagraph}
```

<code>\activateexcursion</code>	Here <code>\activateexcursion{<path>}</code> augments the <code>\printexcursions</code> macro by a call <code>\inputref{<path>}</code> . In this way, the <code>\printexcursions</code> macro (usually in the appendix) will collect up all excursions that are specified in the main text.
<code>\printexcursion</code>	
<code>\excursionref</code>	

Sometimes, we want to reference – in an excursion – part of another. We can use `\excursionref{<label>}` for that.

<code>\excursiongroup</code>	Finally, we usually want to put the excursions into an <code>sfragment</code> environment and add an introduction, therefore we provide the a variant of the <code>\printexcursions</code> macro: <code>\excursiongroup[id=<id>,intro=<path>]</code> is equivalent to
------------------------------	---

```
1 \begin{note}
2 \begin{sfragment}[id=<id>]{Excursions}
3   \inputref{<path>}
4   \printexcursions
5 \end{sfragment}
6 \end{note}
```




When option `book` which uses `\pagestyle{headings}` is given and semantic macros are given in the `sfragment` titles, then they sometimes are not defined by the time the heading is formatted. Need to look into how the headings are made. This is a problem of the underlying `document-structure` package.

Local Variables: mode: latex TeX-master: ../stex-manual End:

10.2 Problem Manual

10.2.1 Introduction

The `problem` package supplies an infrastructure that allows specify problem. Problems are text fragments that come with auxiliary functions: hints, notes, and solutions. Furthermore, we can specify how long the solution to a given problem is estimated to take and how many points will be awarded for a perfect solution.

Finally, the `problem` package facilitates the management of problems in small files, so that problems can be re-used in multiple environment.

10.2.2 Problems and Solutions

<code>solutions</code> <code>notes</code> <code>hints</code> <code>gnotes</code> <code>pts</code> <code>min</code> <code>boxed</code> <code>test</code>	The <code>problem</code> package takes the options <code>solutions</code> (should solutions be output?), <code>notes</code> (should the problem notes be presented?), <code>hints</code> (do we give the hints?), <code>gnotes</code> (do we show grading notes?), <code>pts</code> (do we display the points awarded for solving the problem?), <code>min</code> (do we display the estimated minutes for problem solving). If theses are specified, then the corresponding auxiliary parts of the problems are output, otherwise, they remain invisible.
--	---

The `boxed` option specifies that problems should be formatted in framed boxes so that they are more visible in the text. Finally, the `test` option signifies that we are in a test situation, so this option does not show the solutions (of course), but leaves space for the students to solve them.

`problem` (*env.*) The main environment provided by the `problempackage` is (surprise surprise) the `problem` environment. It is used to mark up problems and exercises. The environment takes an optional `KeyVal` argument with the keys `id` as an identifier that can be reference later, `pts` for the points to be gained from this exercise in homework or quiz situations, `min` for the estimated minutes needed to solve the problem, and finally `title` for an informative title of the problem.

Example 27

Input:

```

1 \documentclass{article}
2 \usepackage[solutions,hints,pts,min]{problem}
3 \begin{document}
4   \begin{sproblem}[id=elephants,pts=10,min=2,title=Fitting Elephants]
5     How many Elephants can you fit into a Volkswagen beetle?
6     \begin{hint}
7       Think positively, this is simple!
8     \end{hint}
9     \begin{exnote}
10      Justify your answer
11    \end{exnote}
12  \begin{solution}
13    Four, two in the front seats, and two in the back.
14    \begin{gnote}
15      if they do not give the justification deduct 5 pts
16    \end{gnote}
17  \end{solution}
18 \end{sproblem}
19 \end{document}

```

Output:

Exercise (Fitting Elephants)
 How many Elephants can you fit into a Volkswagen beetle?

Lösung: Four, two in the front seats, and two in the back.

`solution (env.)` The `solution` environment can be to specify a solution to a problem. If the package option `solutions` is set or `\solutionstrue` is set in the text, then the solution will be presented in the output. The `solution` environment takes an optional KeyVal argument with the keys `id` for an identifier that can be reference `for` to specify which problem this is a solution for, and `height` that allows to specify the amount of space to be left in test situations (i.e. if the `test` option is set in the `\usepackage` statement).

`hint (env.)` The `hint` and `exnote` environments can be used in a `problem` environment to give hints
`exnote (env.)` and to make notes that elaborate certain aspects of the problem. The `gnote` (grading
`gnote (env.)` notes) environment can be used to document situations that may arise in grading.

`\startsolutions` Sometimes we would like to locally override the `solutions` option we have given to
`\stopsolutions` the package. To turn on solutions we use the `\startsolutions`, to turn them off,
`\stopsolutions`. These two can be used at any point in the documents.

`\ifsolutions` Also, sometimes, we want content (e.g. in an exam with master solutions) conditional
 on whether solutions are shown. This can be done with the `\ifsolutions` conditional.

10.2.3 Markup for Added-Value Services

The `problem` package is all about specifying the meaning of the various moving parts of practice/exam problems. The motivation for the additional markup is that we can base added-value services from these, for instance auto-grading and immediate feedback.

The simplest example of this are multiple-choice problems, where the `problem` package allows to annotate answer options with the intended values and possibly feedback that can be delivered to the users in an interactive setting. In this section we will give some infrastructure for these, we expect that this will grow over time.

Multiple Choice Blocks

`mcb` (*env.*) Multiple choice blocks can be formatted using the `mcb` environment, in which single choices are marked up with `\mcc` macro.

`\mcc` `\mcc[⟨keyvals⟩]{⟨text⟩}` takes an optional key/value argument `⟨keyvals⟩` for choice metadata and a required argument `⟨text⟩` for the proposed answer text. The following keys are supported

- `T` for true answers, `F` for false ones,
- `Ttext` the verdict for true answers, `Ftext` for false ones, and
- `feedback` for a short feedback text given to the student.

What we see when this is formatted to PDF depends on the context. In solutions mode (we start the solutions in the code fragment below) we get

Example 28

Input:

```
1 \startsolutions
2 \begin{sproblem}[title=Functions,name=functions1]
3   What is the keyword to introduce a function definition in python?
4   \begin{mcb}
5     \mcc[T]{def}
6     \mcc[F,feedback=that is for C and C++] {function}
7     \mcc[F,feedback=that is for Standard ML]{fun}
8     \mcc[F,Ftext=Noooooooooo,feedback=that is for Java]{public static void}
9   \end{mcb}
10 \end{sproblem}
```

Output:

Exercise (Functions)

What is the keyword to introduce a function definition in python?

- ☒ def
- ☐ function^a
- ☐ fun^b
- ☐ public static void^c

^a

that is for C and C++

^b

that is for Standard ML

^cNoooooooooo

that is for Java

In “exam mode” where disable solutions (here via \stopsolutions)

Example 29

Input:

```
1 \stopsolutions
2 \begin{sproblem}[title=Functions,name=functions1]
3   What is the keyword to introduce a function definition in python?
4   \begin{mcb}
5     \mcc[T]{def}
6     \mcc[F,feedback=that is for C and C++){function}
7     \mcc[F,feedback=that is for Standard ML]{fun}
8     \mcc[F,Ftext=Noooooooooo,feedback=that is for Java]{public static void}
9   \end{mcb}
10 \end{sproblem}
```

Output:

Exercise (Functions)

What is the keyword to introduce a function definition in python?

- ☐ def
- ☐ function
- ☐ fun
- ☐ public static void

we get the questions without solutions (that is what the students see during the exam/quiz).

Filling-In Concrete Solutions

The next simplest situation, where we can implement auto-grading is the case where we have fill-in-the-blanks

`\fillinsol` The `\fillinsol` macro takes a single argument, which contains a concrete solution (i.e. a number, a string, ...), which generates a fill-in-box in test mode:

Example 30

Input:

```
1 \stopsolutions
2 \begin{sproblem}[id=elephants.fillin,title=Fitting Elephants]
3   How many Elephants can you fit into a Volkswagen beetle? \fillinsol{4}
4 \end{sproblem}
```

Output:

Exercise (Fitting Elephants)

How many Elephants can you fit into a Volkswagen beetle?

Example 31

Input:

```
1 \begin{sproblem}[id=elephants.fillin,title=Fitting Elephants]
2   How many Elephants can you fit into a Volkswagen beetle? \fillinsol{4}
3 \end{sproblem}
```

Output:

Exercise (Fitting Elephants)

How many Elephants can you fit into a Volkswagen beetle?

If we do not want to leak information about the solution by the size of the blank we can also give `\fillinsol` an optional argument with a size: `\fillinsol[3cm]{12}` makes a box three cm wide.

Obviously, the required argument of `\fillinsol` can be used for auto-grading. For concrete data like numbers, this is immediate, for more complex data like strings “soft comparisons” might be in order.¹⁶

10.2.4 Including Problems

`\includeproblem` The `\includeproblem` macro can be used to include a problem from another file. It takes an optional KeyVal argument and a second argument which is a path to the file containing the problem (the macro assumes that there is only one problem in the include file). The keys `title`, `min`, and `pts` specify the problem title, the estimated minutes for solving the problem and the points to be gained, and their values (if given) overwrite the ones specified in the `problem` environment in the included file.

¹⁶For the moment we only assume a single concrete value as correct. In the future we will almost certainly want to extend the functionality to multiple answer classes that allow different feedback like in MCQ. This still needs a bit of design. Also we want to make the formatting of the answer in solutions/test mode configurable.

The sum of the points and estimated minutes (that we specified in the `pts` and `min` keys to the `problem` environment or the `\includeproblem` macro) to the log file and the screen after each run. This is useful in preparing exams, where we want to make sure that the students can indeed solve the problems in an allotted time period.

The `\min` and `\pts` macros allow to specify (i.e. to print to the margin) the distribution of time and reward to parts of a problem, if the `pts` and `pts` options are set. This allows to give students hints about the estimated time and the points to be awarded.

10.2.5 Testing and Spacing

The `problem` package is often used by the `hwexam` package, which is used to create homework assignments and exams. Both of these have a “test mode” (invoked by the package option `test`), where certain information –master solutions or feedback – is not shown in the presentation.

<code>\testspace</code>	<code>\testspace</code> takes an argument that expands to a dimension, and leaves vertical space accordingly. Specific instances exist: <code>\testsmallspace</code> , <code>\testsmallspace</code> , <code>\testsmallspace</code> give small (1cm), medium (2cm), and big (3cm) vertical space.
<code>\testsmallspace</code>	<code>\testnewpage</code> makes a new page in <code>test</code> mode, and <code>\testemptypage</code> generates an empty page with the cautionary message that this page was intentionally left empty.
<code>\testnewpage</code>	Local Variables: mode: latex TeX-master: <code>../stex-manual</code> End:
<code>\testemptypage</code>	LocalWords: solutions,notes,hints,gnotes,pts,min,boxed,test gnotes elephants,pts gnote LocalWords: 2,title exnote hint,exnote,gnote ifsolutions mcb keyvals Ttext Ftext LocalWords: Functions,name F,feedback Noooooooooo,feedback 2,title includeproblem LocalWords: elephants.fillin,title

10.3 HWExam Manual

10.3.1 Introduction

The `hwexam` package and class supplies an infrastructure that allows to format nice-looking assignment sheets by simply including problems from problem files marked up with the `problem` package. It is designed to be compatible with `problems.sty`, and inherits some of the functionality.

10.3.2 Package Options

<code>solutions</code>	The <code>hwexam</code> package and class take the options <code>solutions</code> , <code>notes</code> , <code>hints</code> , <code>gnotes</code> , <code>pts</code> , <code>min</code> , and <code>boxed</code> that are just passed on to the <code>problems</code> package (cf. its documentation for a description of the intended behavior).
<code>notes</code>	
<code>hints</code>	
<code>gnotes</code>	
<code>pts</code>	
<code>min</code>	
<code>multiple</code>	Furthermore, the <code>hwexam</code> package takes the option <code>multiple</code> that allows to combine multiple assignment sheets into a compound document (the assignment sheets are treated as section, there is a table of contents, etc.).
<code>test</code>	Finally, there is the option <code>test</code> that modifies the behavior to facilitate formatting tests. Only in <code>test</code> mode, the macros <code>\testspace</code> , <code>\testnewpage</code> , and <code>\testemptypage</code>

have an effect: they generate space for the students to solve the given problems. Thus they can be left in the L^AT_EX source.

10.3.3 Assignments

assignment (*env.*) This package supplies the **assignment** environment that groups problems into assignment sheets. It takes an optional KeyVal argument with the keys **number** (for the assignment number; if none is given, 1 is assumed as the default or — in multi-assignment documents **title** — the ordinal of the **assignment** environment), **title** (for the assignment title; this is **type** referenced in the title of the assignment sheet), **type** (for the assignment type; e.g. “quiz”, **given** or “homework”), **given** (for the date the assignment was given), and **due** (for the date the assignment is due).

10.3.4 Including Assignments

\inputassignment The **\inputassignment** macro can be used to input an assignment from another file. It takes an optional KeyVal argument and a second argument which is a path to the file containing the problem (the macro assumes that there is only one **assignment** environment in the included file). The keys **number**, **title**, **type**, **given**, and **due** are just as for the **assignment** environment and (if given) overwrite the ones specified in the **assignment** environment in the included file.

10.3.5 Typesetting Exams

testheading (*env.*) The **\testheading** takes an optional keyword argument where the keys **duration** specifies a string that specifies the duration of the test, **min** specifies the equivalent in number of minutes, and **reqpts** the points that are required for a perfect grade.

```
reqpts 1 \title{320101 General Computer Science (Fall 2010)}
        2 \begin{testheading}[duration=one hour,min=60,reqpts=27]
        3   Good luck to all students!
        4 \end{testheading}
```

Will result in

Name: Matriculation Number:

320101 General Computer Science (Fall 2010)
2026-06-24

You have one hour (sharp) for the test;
Write the solutions to the sheet.
The estimated time for solving this exam is 23 minutes, leaving you 37 minutes for revising your exam.
You can reach 23 points if you solve all problems. You will only need 27 points for a perfect score, i.e. -4 points are bonus points.

Different problems test different skills and knowledge, so do not get stuck on one problem.

	To be used for grading, do not write here								
prob.	4.1	1.1	1.2	2.1	2.2	2.3	2.4	2.5	2.6
total	0	0	0	10	0	0	0	0	0
reached									

good luck

¹⁷ Local Variables: mode: latex TeX-master: ../stex-manual End:
LocalWords: hwexam solutions,notes,hints,gnotes,pts,min gnotes testemptypage re-
qpts LocalWords: inputassignment reqpts hour,min 60,reqpts

10.4 Tikzinput Manual

image The behavior of the `ikzinput` package is determined by whether the `image` option is given. If it is not, then the `tikz` package is loaded, all other options are passed on to it and `\tikzinput{<file>}` inputs the TIKZ file `<file>.tex`; if not, only the `graphicx` package is loaded and `\tikzinput{<file>}` loads an image file `<file>.<ext>` generated from `<file>.tex`.
The selective input functionality of the `tikzinput` package assumes that the TIKZ pictures are externalized into a standalone picture file, such as the following one

¹⁷MK: The first three “problems” come from the stex examples above, how do we get rid of this?


```

1 \documentclass{standalone}
2 \usepackage{tikz}
3 \usetikzpackage{...}
4 \begin{document}
5   \begin{tikzpicture}
6     ...
7   \end{tikzpicture}
8 \end{document}

```

The `standalone` class is a minimal $\text{\LaTeX}$ class that when loaded in a document that uses the `standalone` package: the preamble and the `document` environment are disregarded during loading, so they do not pose any problems. In effect, an `\input` of the file above only sees the `tikzpicture` environment, but the file itself is standalone in the sense that we can run $\text{\LaTeX}$ over it separately, e.g. for generating an image file from it.

$\text{\tikzinput}$ $\text{\ctikzinput}$	This is exactly where the <code>tikzinput</code> package comes in: it supplies the <code>\tikzinput</code> macro, which – depending on the <code>image</code> option – either directly inputs the TIKZ picture (source) or tries to load an image file generated from it.
---	---

Concretely, if the `image` option is not set for the `tikzinput` package, then `\tikzinput[ $\langle opt \rangle$ ]{ $\langle file \rangle$ }` disregards the optional argument $\langle opt \rangle$ and inputs $\langle file \rangle.tex$ via `\input` and resizes it to as specified in the `width` and `height` keys. If it is, `\tikzinput[ $\langle opt \rangle$ ]{ $\langle file \rangle$ }` expands to `\includegraphics[ $\langle opt \rangle$ ]{ $\langle file \rangle$ }`.

`\ctikzinput` is a version of `\tikzinput` that is centered.

$\text{\mhtikzinput}$ $\text{\cmhtikzinput}$	$\text{\mhtikzinput}$ is a variant of <code>\tikzinput</code> that treats its file path argument as a relative path in a math archive in analogy to <code>\inputref</code> . To give the archive path, we use the <code>mhrepos=</code> key. Again, <code>\cmhtikzinput</code> is a version of <code>\mhtikzinput</code> that is centered.
---	--

$\text{\libusetikzlibrary}$	Sometimes, we want to supply archive-specific TIKZ libraries in the <code>lib</code> folder of the archive or the <code>meta-inf/lib</code> of the archive group. Then we need an analogon to <code>\libinput</code> for <code>\usetikzlibrary</code> . The <code>stex-tikzinput</code> package provides the <code>libusetikzlibrary</code> for this purpose.
-----------------------------	---

Local Variables: mode: latex TeX-master: ../stex-manual End:

Part III
Documentation

Chapter 11

Utilities

Various utility methods

`\stex_ignore_spaces_and_pars:`

Ignores space characters and empty lines (which would close the current paragraph)

`\stex_undefine:N` Locally undefines a macro
`\stex_undefine:c`

`\stex_str_if_starts_with_p:nn` * `\stex_str_if_starts_with:nn` `{<str1>}` `{<str2>}`
`\stex_str_if_starts_with:nnTF` *

Tests if `<str1>` starts with `<str2>`.

`\stex_str_if_ends_with_p:nn` * `\stex_str_if_ends_with:nn` `{<str1>}` `{<str2>}`
`\stex_str_if_ends_with:nnTF` *

Tests if `<str1>` ends with `<str2>`.

`\stex_deactivate_macro:Nn` `\stex_deactivate_macro:Nn` `\macro` `{<scope>}`
`\stex_reactivate_macro:N`

redefines `\macro` to throw an error, that the `\macro` is only allowed in `<scope>`. This allows for more informative error messages than `undefined control sequence`.

`\stex_reactivate_macro:N` makes the macro valid again.

11.1 kpsewhich and Environment Variables

`\stex_kpsewhich:Nn` `\stex_kpsewhich:Nn` `\macro` `{<args>}`

Calls “`kpsewhich <args>`” and stores the result in `\macro`,

T_EXhackers note: (Usually) does not require `shell-escape`, since by default, `kpsewhich` is in the list of “allowed” commands.

<code>\stex_get_env:Nn</code>	<code>\stex_get_env:Nn \macro {<envvar>}</code>
-------------------------------	---

Stores the value of the environment variable `<envvar>` in `\macro`.

11.2 Logging

<code>\stex_debug:nn</code>	<code>\stex_debug:nn {<prefix>} {<msg>}</code>
-----------------------------	--

Logs the debug message `{<msg>}` under the prefix `{<prefix>}`. A message is shown if its prefix is in a list of prefixes given either via the package option `debug=<prefixes>` or the environment variable `STEX_DEBUG=<prefixes>`, where the latter overrides the former.

11.3 File Paths

Since `STEX` needs to handle files from various *math archives* in a user-file-system-independent manner in arbitrary *MathHub* directories, we primarily use *absolute* file paths, for operations such as `\inputref`, `\usemodule`, `\importmodule`, etc.

We therefore provide macros to explicitly handle file paths independent of the underlying operating system, resolving and canonicalizing `..` segments and relative paths, etc.

<code>\stex_file_set:Nn</code>	<code>\stex_file_set:Nn \macro {<string>}</code>
--------------------------------	--

`\stex_file_set:(No|Ne)` represents an already canonicalized file path string as a `LATEX3` sequence and stores it in `\macro`.

<code>\stex_file_resolve:Nn</code>	<code>\stex_file_resolve:Nn \macro {<string>}</code>
------------------------------------	--

`\stex_file_resolve:(No|Ne)` resolves and canonicalizes the file path string `<string>` and stores the result in `\macro`.

<code>\stex_file_use:N</code>	<code>★</code> expands to a string representation of the given file path (using <code>/</code> as separator, regardless of file system).
-------------------------------	--

<code>\stex_file_split_off_ext:NN</code>	<code>\stex_file_split_off_ext:NN \target \source</code>
--	--

splits off the file extension of `\source` and stores the resulting file path in `\target`.

<code>\stex_file_split_off_lang:NN</code>	<code>\stex_file_split_off_lang:NN \target \source</code>
---	---

splits off *both* the file extension *and* language component of `\source` and stores the resulting file path in `\target`; e.g. if `\source` is `foo.en.tex`, `\target` will be `foo`. Explicitly checks that the segment before the file extension is a valid language identifier; otherwise, it will not be stripped.

<code>\c_stex_pwd_file</code>	represent the <i>parent working directory</i> and absolute path to the top-level <code>.tex</code> file currently being processed (as absolute, canonicalized paths)
-------------------------------	--

`\c_stex_home_file` represents the absolute path of the user’s home directory (as absolute, canonicalized path)

`\g_stex_current_file` the current file (as absolute, canonicalized path)

`\stex_input_with_hooks:Nn` `\stex_input_with_hooks:Nn \document-URI {<file string>}`
`\stex_input_with_hooks:Ne` Inputs the given file (as a string) by passing it on to `\input`, setting `\g_stex_current_file`, `\l_stex_current_language_str` and `\l_stex_current_document_uri`, and reverting them again afterwards

`\stex_with_file_hooks:Nnn` `\stex_with_file_hooks:Nnn \document-URI {<file string>} {<code>}`
sets `\g_stex_current_file`, `\l_stex_current_language_str` and `\l_stex_current_document_uri`, executes `<code>`, and reverts them again afterwards

`\stex_if_file_absolute_p:N` `*`
`\stex_if_file_absolute:NTF` `*`

tests whether the given file path (represented as a canonicalized L^AT_EX3 sequence) is absolute.

`\stex_if_file_starts_with:NNTF` `\stex_if_file_starts_with:NN \child \parent`
tests whether the file path `\child` is a child of `\parent`.

11.4 Group-like Behaviours

Macros for mimicking the behaviour of T_EX groups without actually opening a T_EX group. This is relevant for two purposes:

- Temporarily redefining a macro, doing a bunch of stuff, and then reverting it to its previous definition; that is what a “*pseudogroup*” does.
- Defining macros at a specific group level further up. That is what a “*metagroup*” does – code wrapped in `\stex_metagroup_do_in:n` will be repeatedly executed in `\aftergroup` calls, up to the group level of the innermost metagroup. That allows for e.g. declaring new symbols in `sdefinition` paragraphs that are valid in the entire `mathstructure` or `module`.

`\stex_pseudogroup:nn` `\stex_pseudogroup:nn {<code>} {<reset code>}`
will *expand* `<reset code>` first, then process `<code>`, then process the expanded `<reset code>`.

`\stex_pseudogroup_restore:N` ★ `\stex_pseudogroup_restore:N` `\macro`

will expand to code that will redefine `\macro` as its current definition. Only works for *token lists* and similar macros that do not take arguments.

In combination, that allows for things like

Example 32

```
1 % \stex_pseudogroup:nn{
2 % \def \foo {..}
3 % ...
4 % }\stex_pseudogroup_restore:N \foo}
5 %
```

`\stex_pseudogroup_with:nn` `\stex_pseudogroup_with:nn` `{\macro-list}` `{\code}`

will store the definitions of the macros in `\macro-list`, execute `\code`, and then revert the macros to their previous definitions without opening a TeX group. **TODO: this might be unnecessary? Check/profile**

`\stex_metagroup_new:` opens a new *metagroup*. All code executed in `\stex_metagroup_do_in:n` will survive up to the current group level. Metagroups can be nested.

`\stex_metagroup_do_in:n` Executes the given code in the current metagroup. If there is no metagroup currently, the code will simply be executed once, so there is little danger in calling this spuriously.

TeXhackers note: This repeatedly stores the provided code in a macro and uses `\aftergroup` until the group level of the current metagroup is reached.

11.5 Key Handling

Key handling mechanisms on top of `l3keys` for inheriting key sets and initializing the associated macros

`\stex_keys_define:nnnn` `\stex_keys_define:nnnn` `{\id}` `{\init}` `{\spec}` `{\exts}`

defined the key set `\id` with the `l3keys`-style key-parsing code `{\spec}`; `\init` is called every time before parsing and is intended to e.g. clear the relevant macros. `\exts` may be a comma-separated list of key set ids from which this key set inherits.

`\stex_keys_set:nn` Like `\keys_set:nn`, but additionally calling the relevant initialization code.

11.6 Languages

`\c_stex_languages_prop`
`\c_stex_language_abbrevs_prop`

Two inverse property lists; `\c_stex_languages_prop` maps language abbreviations (e.g. `en`, `de`) to their (babel) names (e.g. `english`, `ngerman`), `\c_stex_language_abbrevs_prop` does the inverse.

`\l_stex_current_language_str`

The language of the current file; may change, if a file with a different language is imported.

11.7 Styling

`\stex_new_stylable_cmd:nnnn` `\stex_new_stylable_cmd:nnnn {<macroname>} {<argument spec>} {<definition>}`
`\stex_style_apply:` `{<default>}`

Defines a new stylable command `\macroname` with `<argument spec>` (passed on to `\NewDocumentCommand`) with `<definition>` as its macro body and default typesetting style `<default>`.

`<default>` may be empty. `<definition>` should at some point use `\stex_style_apply:` to do the typesetting according to the chosen style for this macro, which by default will be defined as `<default>`.

Also generates the macro `\stexstyle<macroname>[<name>]{<code>}` to define a new typesetting style for `\macroname`.

`\stex_new_stylable_env:nnnnnn` `\stex_new_stylable_env:nnnnnn {<environment name>} {<argument spec>} {<begin code>} {<end code>} {<default begin>} {<default end>} {<prefix>}`

Defines a new stylable environment `<prefix><environment name>` with `<argument spec>` (passed on to `\NewDocumentEnvironment`) with `<begin code>` and `<end code>` as its macro body and default typesetting style dictated by `<default begin>` and `<default end>`.

`<begin code>` and `<end code>` should at some point use `\stex_style_apply:` to do the typesetting according to the chosen style for this environment.

Also generates the macro `\stexstyle<environmentname>[<name>]{<begin>}{<end>}` to define a new typesetting style for `environment name`.

A prefix `s` is e.g. used to generate the environment `sparagraph` and the macro `\stexstyleparagraph`.

11.8 Persisting Dependencies

`\stex_persist:n`
`\stex_persist:e`
`\loadsms`

Writes the given code in the `.sms`-file (if `smsmode` is on)

Chapter 12

HTML Output

Macros for dealing with HTML output.

`\stex@backend` Which engine is running; possible values are `pdflatex`, `rustex`, `tex4ht` (**TODO**) and `latexml` (**TODO**).

`\stex_if_html_backend_p: *` L^AT_EX3 conditional testing whether the engine running compiles to HTML.
`\stex_if_html_backend:TF *`

`\ifstexhtml *` L^AT_EX2 conditional testing whether the engine running compiles to HTML.

`\stex_if_do_html_p: *` Whether to (locally) produce HTML output. May be turned off locally even if the back-
`\stex_if_do_html:TF *` end produces HTML; e.g. when parsing dependencies in `\importmodule` or `\usemodule`.

`\stex_suppress_html:n` temporarily suppresses HTML output when processing the provided code.

`\stex_annotate:nn` `\stex_annotate:nn {<keyvals>} {code}`
 attaches the provided comma-separated key-value-pairs (as `key=val`) as HTML attributes to the HTML node(s) generated from `<code>`. If `<code>` results in multiple nodes, this needs to insert a new `<div>`, `<span>` or `<mrow>` node for the attributes. Multiple nested calls to `\stex_annotate:nn` may be merged into a single node.

`stex_annotate_env (env.)` like `\stex_annotate:nn`, but as an environment; i.e. `\begin{stex\_annotate\_env}{<keyvals>}{<code>}` is equivalent to `\stex_annotate:nn{<keyvals>}{<code>}`

`\stex_html_node:nnn` `\stex_html_node:nnn {<tag>} {<keyvals>} {code}`
 like `\stex_annotate:nn`, but makes sure to wrap `<code>` in a new `<tag>` node.

`stex_env_node (env.)` like `\stex_html_node:nnn`, but as an environment; i.e. `\begin{stex\_env\_node}{<tag>}{<keyvals>}{<code>}`

is equivalent to `\stex_html_node:nnn{<tag>}{<keyvals>}{<code>}`

`\stex_annotate_invisible:nn` `\stex_annotate_invisible:nn {<keyvals>} code`
`\stex_annotate_invisible:n`

like `\stex_annotate:nn`, but additionally adds the key values `data-ftml-invisible="true"` and `style="display:none;"`. `\stex_annotate_invisible:n` does not take additional attribute-value-pairs.

`\STEXinvisible` public wrapper for `\stex_annotate_invisible:n`.

`\stex_css_link:n` `\stex_css_link:n {<url>}`

inserts a `<link rel="stylesheet" href="<url>">` node in the header of the generated HTML.

`\stex_css_literal:n` `\stex_css_literal:n {<text>}`

inserts a `<style><text></style>` node in the header of the generated HTML.

`\_stex_annotate_force_break:n`

should guarantee that the HTML nodes generated by the provided code are not merged with any outer nodes with respect to attached attributes.

`\mmlintent` `\mmlintent {<value>} {<code>}`

`\mmlarg` annotates `<code>` with the provided MathML *intent* attribute (or *intent arg* attribute, respectively).

and style patching:

```

1 <@@=stex_styles>
2 \stex_if_html_backend:TF{
3   \cs_new_protected:Nn \stex_style_apply: {}
4   \cs_new_protected:Nn \_stex_styles_patch:nnnn {}
5   \stex_keys_define:nnnn{ csspatch }{
6     \str_clear:N \l_stex_keys_cls_id
7     \str_clear:N \l_stex_keys_counter_id
8     \str_clear:N \l_stex_keys_counter_parent
9   }{
10    counter      .str_set_x:N = \l_stex_keys_counter_id,
11    parent       .str_set_x:N = \l_stex_keys_counter_parent,
12    unknown      .code:n      = {
13      \exp_args:NNo \str_set:Nn \l_stex_keys_cls_id {\l_keys_key_tl}
14    }
15  }{}
16  \cs_new_protected:Npn \_stex_styles_css_patch:nnnn #1 #2 {
17    \stex_keys_set:nn{ csspatch }{ #2 }
18    \group_begin:
19    \catcode'\ =12\relax
20    \catcode'\^J=12\relax
21    \_stex_styles_patch_i:nnn {#1}
22  }
```

```

23
24 \seq_new:N \g__stex_styles_counters_seq
25
26 \cs_new:Nn \__stex_styles_patch_html_c: {
27   \stex_html_literal:n{
28     <span~data-ftml-counter="\l_stex_keys_counter_id"~style="display:none;"~
29     data-ftml-counter-parent="\l_stex_keys_counter_parent"
30     ></span>
31   }
32 }
33
34 \cs_new:Nn \__stex_styles_patch_html: {
35   \stex_html_literal:n{
36     <span~data-ftml-style="\l_stex_keys_cls_id"~style="display:none;"~
37     data-ftml-counter="\l_stex_keys_counter_id"
38     ></span>
39   }
40 }
41
42 \cs_new_protected:Nn \__stex_styles_patch_i:nnn {
43   \str_if_empty:NTF \l_stex_keys_cls_id {
44     \str_set:Nn \l_stex_keys_cls_id { #1 }
45   }{
46     \str_set:Ne \l_stex_keys_cls_id { #1-\l_stex_keys_cls_id }
47   }
48   \exp_args:No \str_case:nnF \l_stex_keys_counter_parent {
49     {}{}
50     {part}{\str_set:Nn \l_stex_keys_counter_parent { 0 }}
51     {chapter}{\str_set:Nn \l_stex_keys_counter_parent { 1 }}
52     {section}{\str_set:Nn \l_stex_keys_counter_parent { 2 }}
53     {subsection}{\str_set:Nn \l_stex_keys_counter_parent { 3 }}
54     {subsubsection}{\str_set:Nn \l_stex_keys_counter_parent { 4 }}
55     {paragraph}{\str_set:Nn \l_stex_keys_counter_parent { 5 }}
56     {subparagraph}{\str_set:Nn \l_stex_keys_counter_parent { 6 }}
57   }{
58     \msg_error:nnx{stex}{error/notasection}\l_stex_keys_counter_parent
59   }
60   \str_set:Ne \l__stex_styles_first_str { &~.ftml-title-paragraph~ { display:inline-block
61   \str_if_empty:NF \l_stex_keys_counter_id {
62     %\str_put_left:Ne \l__stex_styles_first_str { counter-increment:~ ftml-\l_stex_keys_c
63     \exp_args:NNo \seq_if_in:NnF \g__stex_styles_counters_seq \l_stex_keys_counter_id {
64       \seq_gpush:No \g__stex_styles_counters_seq \l_stex_keys_counter_id
65       \cs_if_eq:ccTF{@onlypreamble}{@notprerr}{
66         \__stex_styles_patch_html_c:
67         % in body
68       }{
69         \exp_args:Ne \AtBeginDocument \__stex_styles_patch_html_c:
70         % in preamble
71       }
72     }
73     \cs_if_eq:ccTF{@onlypreamble}{@notprerr}{
74       \__stex_styles_patch_html:
75       % in body
76     }{

```

```

77     \exp_args:Ne \AtBeginDocument \__stex_styles_patch_html:
78     % in preamble
79   }
80 }
81 \exp_args:Ne \stex_css_literal:n { .ftml-\l_stex_keys_cls_id { \l__stex_styles_first_st
82
83 % TODO export counter reset somehow
84
85 \group_end:
86 }
87 }{
88 \cs_new_protected:Nn \__stex_styles_patch:nnnn {
89   \str_if_empty:nTF {#2}{
90     \tl_set:cn{\__stex_styles_style_#1_start:}{#3}
91     \tl_set:cn{\__stex_styles_style_#1_end:}{#4}
92   }{
93     \tl_set:cn{\__stex_styles_style_#1_#2_start:}{#3}
94     \tl_set:cn{\__stex_styles_style_#1_#2_end:}{#4}
95   }
96 }
97 \cs_new_protected:Nn \__stex_styles_css_patch:nnnn {}
98 }

```

Chapter 13

Math Archives

`\c_stex_mathhub_files` represents the comma-separated, absolute path *strings* of the user’s MathHub directories.
`\mathhub` If the `\mathhub` macro is set when the `stex` package is loaded, this one is used for finding archives. Otherwise it is determined from the `MATHHUB` environment variable, if set, or from the path in `<HOME>/stex/mathhub.path`, if existent, or set to `<HOME>/MathHub` if all else fails. In all cases, `\mathhub` will be defined.

MathHub directories are scanned in order; i.e. earlier paths are prioritized over later ones.

A *math archive* is represented as a triple `{<archive id>}{<base uri>}{<file path string>}`, where an invariant is that `<file path string>` is of the form `<mh>/<archive id>` for some directory `<mh>` in `\c_stex_mathhub_files`.

Such a triple is stored in the macro `\c_stex_mathhub_<id>_archive` when the archives metadata is first loaded.

`\stex_archive_id:N` * expands to the id of the given archive (a macro defined as a triple as described above)

`\stex_archive_base:N` * expands to the base URI of the given archive; either given as a macro defined as a triple
`\stex_archive_base:n` * as described above, or as the archive’s id. The latter requires the archive to be loaded beforehand!

`\stex_archive_path:N` * expands to the file path *as a string* of the given archive; either given as a macro defined
`\stex_archive_path:n` * as a triple as described above, or as the archive’s id. The latter requires the archive to be loaded beforehand!

`\stex_in_archive:nn` `\stex_in_archive:nn {<archive id>} {<code>}`

Temporarily sets the current archive `\l_stex_current_archive` to `<archive id>`, executes `<code>` and reverts back to the previous archive. Passes the id of the *now current* archive to `<code>` as `#1`: If the first argument is empty, this will be the current archive’s id without changing it. Otherwise, the archive with the desired id will be loaded; throws an error, if it does not exist.

<code>\c_stex_no_archive_str</code>	The archive ID <code>\c_stex_no_archive_str=no/archive</code> is used for documents and other content that is not contained in a <i>math archive</i> . <code>\c_stex_no_archive</code> is the corresponding archive triple.
<code>\c_stex_no_archive</code>	

<code>\c_stex_main_archive</code>	Point to the main file's archive and the <i>current</i> archive, respectively, if existent; undefined otherwise.
<code>\l_stex_current_archive</code>	

<code>\stex_source_path:n</code>	<code>\stex_source_path:n</code> <code>{\langle relative path \rangle}</code>
<code>\stex_source_path:nn</code>	<code>\stex_source_path:nn</code> expands to the absolute file path (string) of the given <code>\langle relative path \rangle</code> relative to the source directory of the current archive – or relative to the current file, if we are not in an archive. In the latter case, the file path will <i>not</i> be canonicalized as to remain expandable.

The variant `\stex_source_path:nn` takes the ID of an archive as first argument (instead of the current archive).

Chapter 14

URIs

There are kinds of URIs used in IMMT/OMDoc:

- *Base URIs*, i.e. namespaces,
- *Archive URIs* of the form `<base uri>?a=<archive id>`,
- *Document URIs* of the form `<archive uri>[&p=<path>]&d=<name>&l=<language>`,
- *Document element URIs* of the form `<document uri>&e=<name>`,
- *Module URIs* of the form `<archive uri>[&p=<path>]&m=<name>`, and
- *Declaration URIs* of the form `<module uri>&s=<name>`.

We do not need all of these in `STeX` itself: the *base uri* of any URI is uniquely determined by the archive ID, and we conceptually merge document URIs and document element URIs.

We therefore represent document (element) URIs as 5-tuples `{<archive id>}{<path>}{<name>}{<language>}{<element name>}`, where `<path>` and `<element name>` may be empty; module URIs as triples `{<archive id>}{<path>}{<name>}`, and declaration URIs as 4-tuples `{<archive id>}{<path>}{<module name>}{<name>}`.

`\stex_use_archive_uri:n` ★ expands to the string representation of the archive URI of the archive with the given ID. Requires, that the archive has been loaded first

14.1 Document URIs

`\stex_use_document_uri:N` ★ expands to the string representation of the document uri (represented as a macro containing a 5-tuple as above). The variant `\stex_document_uri:n` expects a 5-tuple directly.

`\stex_document_uri_archive:N` ★ expands to the archive ID of the document URI

`\stex_document_uri_path:N *`

expands to the path of the document URI (possibly empty)

`\stex_document_uri_name:N *`

expands to the document name of the document URI

`\stex_document_uri_language:N *`

expands to the language of the document URI

`\stex_document_uri_element:N *`

expands to the element name of the document (element) URI (possibly empty)

`\stex_document_uri_with_language:Nn *`

expands to the document URI with the given language

`\stex_new_document_element_uri:n *`

expands to a new document element URI in the current document

`\stex_document_uri_from_archive_file:Nn \stex_uri_from_archive_file:Nn \macro {<relative path>}`

Constructs the document URI of the file *<relative path>* in the current archive and stores it in `\macro`

`\c_stex_main_document_uri`
`\l_stex_current_document_uri`

store the document URIs of the top-level file being processed and the *current* file, respectively.

14.2 Module URIs

`\stex_use_module_uri:N *` expands to the string representation of the module uri (represented as a macro containing
`\stex_use_module_uri:n *` a triple as above). The variant `\stex_module_uri:n` expects a triple directly.

`\stex_module_uri_archive:N *`

expands to the archive ID of the module URI

`\stex_module_uri_path:N *` expands to the path of the module URI (possibly empty)
`\stex_module_uri_path:n *`

`\stex_module_uri_name:N` * expands to the name of the module URI
`\stex_module_uri_name:n` *

`\stex_module_uri_split_name:NNN`
`\stex_module_uri_split_name:NNn`

sets #1 as the parent module of the module URI and #2 as the last name

`\stex_module_uri_as_qm:n` *

`\stex_new_module_uri:n` * `\stex_new_module_uri:n` {*<name>*}

expands to a new module URI triple with the given *<name>* based on the current document URI or module URI, if existent

`\stex_uri_from_pair:Nnn` `\stex_uri_from_pair:Nnn` {*<archive id>*} {*<module path>*}

resolves the pair [*<archive id>*]{*<module path>*} to a module URI and stores the result in #1

`\stex_uri_from_pair:Nnn` like `\stex_uri_from_pair:Nnn`, but takes a token list as argument that may or may not be of the form [archive]module

14.3 Symbol URIs

`\stex_use_symbol_uri:N` * expands to the string representation of the module uri (represented as a macro containing a 4-tuple as above). The variant `\stex_use_symbol_uri:n` expects a 4-tuple directly.
`\stex_use_symbol_uri:n` *

`\stex_new_symbol_uri:n` * `\stex_new_symbol_uri:n` {*<name>*}

expands to a new symbol URI tuple with the given *<name>* based on the current module URI, which is assumed to exist(!).

`\stex_new_symbol_uri:nn` * expands to a new symbol URI tuple with the given *<name>* based on the *given* module URI (either as macro or as tuple), which is assumed to exist(!).

`\stex_symbol_uri_archive:N` *

expands to the archive ID of the symbol URI

`\stex_symbol_uri_path:N` * expands to the path of the symbol URI (possibly empty)

`\stex_symbol_uri_module:N` ★
`\stex_symbol_uri_module:n` ★

expands to the URI of the containing module of the symbol URI

`\stex_symbol_uri_name:N` ★ expands to the name of the symbol URI
`\stex_symbol_uri_name:n` ★

Chapter 15

Documents

`\STEXftmlink`
`\STEXlinkftml`
`\STEXsetlinkftml`

`\STEXlinkftml` inserts the following link/disclaimer:

*The **FTML** version of this document can be found at*
[http://unknown.source?a=no/archive&p=/home/jazzpirate/work/
Software/sTeX/sTeX/doc&d=stex-doc&l=en](http://unknown.source?a=no/archive&p=/home/jazzpirate/work/Software/sTeX/sTeX/doc&d=stex-doc&l=en)

The precise text can be set using `\STEXsetlinkftml{<text>}`; The link on its own can be inserted using `\STEXftmlink`.

`\stexdoctitle` `\stexdoctitle {<title>}`

Sets `<title>` to be the title of the document. Only the first call of this command does something.

15.1 Sectioning

`\l_stex_current_section_level_int`
`\l_stex_current_section_level_str`

integer keeping track of the current sectioning level:

0 part

1 chapter

2 section

3 subsection

4 subsubsection

5 paragraph

>5 subparagraph

`\setsectionlevel` sets `\l_stex_current_section_level_int` to the corresponding integer value. `\l_stex_current_section_level_str` stores a string representation of the same value, but will initially contain `document`.

`\currentsectionlevel` will leave the current section level as text in the token input. The variant `\Currentsectionlevel` will produce the capitalized version.

If the `xspace` package is loaded, this will insert an `\xspace` afterwards.

In HTML mode, this will insert an annotation, so it can be adapted dynamically.

`sfragment` (*env.*) A section at the current section level; preferred over the primitives `\section`, `\subsection`, etc., because a) environments are better suited for such structuring, and b) it allows for adapting the inserted section header based on document context, rather than it being hardcoded – e.g. what’s a subsection in one document may be a section in another.

`titlefragment` (*env.*) like `sfragment`, but will use `\maketitle` instead of (one of) the sectioning macros, if no title has been used yet.

`blindfragment` (*env.*) skips one sectioning level in its body so that e.g. subsection 0.1 can be used before the first section

`\skipfragment` Skips one counter at the current sectioning level; e.g. increase the subsection counter to 2 if called immediately after `\section` (or, rather, after `\begin{sfragment}`).

`\setsectionlevel` `\setsectionlevel {<name>}`

Sets the current section level to the one specified (e.g. `\setsectionlevel{subsection}`). Should ideally be called at most once in the preamble of a document.

15.2 Inter-Document References

[id=foo] sets the current fragment's (e.g. Definition 3.1 (Foo)) name to foo, resulting in a DocumentElementURI ..&e=foo. It then delegates to \label{...&e=foo} (which defines \r@...&e=foo), stores ::&e=foo in \g_stex_sref_label_foo, and writes \STeXInternalSRefLabel{foo}{...&e=foo}{definition.3.1}{Foo} to the sref file.

15.3 Inputting From MathHub Resources

\inputref \inputref [*archive id*] {*filepath*}

Inputs the referenced file in the referenced archive (or current archive, if empty) in a tex group, while setting all the relevant macros for the current archive, document URI, etc.

In HTML, will instead just insert an annotation, so that the HTML of the referenced document can be dynamically inserted instead.

\mhinput \inputref [*archive id*] {*filepath*}

Like \inputref, but *always* actually inputs the referenced file (also in HTML mode!) and without opening a tex group.

\ifinputref L^AT_EX2 conditional; is true, if we currently are in an \inputref or \mhinput.

\IfInputref \IfInputref {*true*} {*false*}

Executes *true* if we currently are in an \inputref or \mhinput, otherwise executes *false*.

In HTML, *both* branches are executed(!) and inserted in the HTML with corresponding attributes, so that the relevant parts can be shown or hidden.

\libinput \libinput [*archive id*] {*file name*}

inputs all files *file name*.tex in the lib directories of the current archive. Will throw an error if we are not in an archive or no suitable file is found.

\libusepackage \libusepackage [*package options*] {*package name*}

inputs all packages *package name*.sty in the lib directories of the current archive. Will throw an error if we are not in an archive, or there is not *exactly one* file *package name*.sty somewhere in the lib directories.

Note that unlike \libinput, the optional argument does *not* refer to an archive ID, since this would conflict with package options.

Will give a spurious warning that package mathhub/directory/foo was requested, but package foo was found. This is a side-effect of using absolute file paths, which is necessary to make the package findable in the first place.

\libusetikzlibrary \libusetikzlibrary [*archive id*] {*name*}

like \libusepackage, but for \usetikzlibrary.

\addmhbibresource	\addmhbibresource [<i>archive id</i>] { <i>file name</i> }
	Calls \addbibresource on all <i>file name</i> .bib files in the lib directories of the current archive. Will throw an error if we are not in an archive or no suitable file is found.
\mhgraphics \cmhgraphics	\mhgraphics adds the archive key to the optional arguments of \includegraphics to allow for using images in math archives (relative to its source directory). \cmhgraphics additionally wraps the image in a \begin{center} . Only defined if the graphicx package is loaded (but may be loaded after the stex package).
\lstinputmhlisting \clstinputmhlisting	like \mhgraphics , but for \lstinputlisting . Only defined if the listings package is loaded (but may be loaded after the stex package).
\mhtikzinput \cmhtikzinput	like \mhgraphics , but for \tikzinput . Only defined if the tikzinput package is loaded (but may be loaded after the stex package).

Chapter 16

Modules

`\l_stex_current_module_uri` stores the URI of the current module, if existent

`\l_stex_all_modules_seq` stores the URIs of all modules *currently in scope*.

`smodule (env.)` Parses the optional arguments, (if relevant) inserts HTML annotations, and then delegates to `\stex_module_setup:n`

`\stex_module_setup:n` Sets up a new module with the given name by checking whether it is a nested module, and if not, has a signature etc. Loads signature and metatheory, if necessary **TODO: deprecate in favor of every document being a module**

`\stex_module_setup_top_nosig:n` Sets up a new top-level module with the given name and no signature. Does not load a meta theory. **TODO: deprecate in favor of every document being a module**

`\stex_close_module:` closes the current module.

`\stex_every_module:n` adds code to be executed every time a new module is opened (e.g. activating certain macros).

`\stex_do_up_to_module:n` Execute code in the current module (i.e. as if the `\begin{<smodule>}` was the current
`\stex_do_up_to_module:e` tex group)

`\stex_execute_in_module:n` Execute code in the current module (i.e. as if the `\begin{<smodule>}` was the current tex
`\stex_execute_in_module:e` group) and also adds it to the initialization code of the module; i.e. exports all macros defined.

`\stex_if_in_module_p:` * conditional whether we currently are in a module. **TODO: deprecate in favor of every document being a module**
`\stex_if_in_module:` *TF* *

`\stex_if_module_exists_p:` *N* *
`\stex_if_module_exists:` *NTF* *
`\stex_if_module_exists_p:` *n* *
`\stex_if_module_exists:` *nTF* *

conditional whether a module with the given URI exists

`\stex_activate_module:` *N*
`\stex_activate_module:` *n*

`\setmetatheory` `\setmetatheory` [*archive id*] {*module*}

sets the metatheory of all subsequent modules to the specified one

`\STEXexport` `\STEXexport` {*code*}

executes the provided code every time the current module is opened (including immediately). Processes its content in the `\ExplSyntaxOn` category code scheme.

`\stex_structural_feature_module:` *nn*
`\stex_structural_feature_module_end:`

16.1 SMS Mode

`\STEX` has to extract formal content (i.e. modules and their symbols) from `\LaTeX`-files, that may otherwise contain arbitrary code, including macros that may not be defined unless the file is fully processed by `\TeX`. Those modules and symbols also may depend on other modules that have not yet been loaded. The naive way to achieve this, which would be to just suppress output (e.g. by storing it in a box register) and then `\input` the required file, does not work thanks to `\TeX`'s limited *file stack*, which would overflow quickly for modules that have a deeply nested list of dependencies.

To solve those problems, `\STEX` reads dependencies in what we call *sms mode*, which can be summarized thusly:

- In a first pass, we parse the file token by token, ignoring everything other than a select list of macros and environments that introduce dependencies (such as `\importmodule` and `\begin{smodule}[sig=...]`). Instead of loading those, we remember them for later.
- After the file has been fully parsed thusly, the dependencies found are loaded, again in sms-mode.
- In a second pass, we parse the file *again* in the same way, but this time execute all macros that are explicitly allowed in sms mode, such as `\importmodule`, `\symdecl`, `\notation`, `\symdef`, etc.

- all this parsing happens additionally in a `\setbox\throwaway\ vbox{...}`-block to suppress any accidental output.

This means that T_EX’s input stack never grows by more than +1, but still behaves *as if* the dependencies were loaded recursively, at the detriment of being somewhat slow.

`\stex_sms_allow:N` registers the provided macro to be allowed in sms mode.

This only works, if the macro takes no arguments and/or does not touch the subsequent tokens.

`\stex_sms_allow_escape:N` registers the provided macro to be allowed in sms mode.

If the macro is subsequently encountered in sms-mode, parsing is halted and it can process arguments as desired. It then needs to continue parsing manually though, by calling `\stex_smsmode_do:` as (usually) its last token.

`\stex_sms_allow_env:n` registers the provided environment to be allowed in sms mode.

As with `\stex_sms_allow_escape:N`, the `\begin{envname}` is escaped, hence the begin-code of the environment needs to call `\stex_smsmode_do:`. Since `\end{envname}` never takes arguments, it does not need to be escaped.

`\stex_sms_allow_import:Nn` `\stex_sms_allow_import:Nn \macro {<code>}`

registers the provided macro to be allowed in the *first pass* in sms mode. The provided `<code>` is executed at the start of the first pass, to define macros accordingly.

`\stex_sms_allow_import_env:nn` `\stex_sms_allow_import_env:nn \envname {<code>}`

registers the provided environment to be allowed in the *first pass* in sms mode. The provided `<code>` is executed at the start of the first pass, to define macros accordingly.

`\g_stex_sms_import_code` inserted between the first and second pass; macros/environments allowed via `\stex_sms_allow_import_*` should store their “results” in here.

`\stex_if_smsmode_p: *` tests for whether we are currently in sms-mode.
`\stex_if_smsmode:TF *`

`\stex_file_in_smsmode:Nn` `\stex_file_in_smsmode:Nn \document-URI {<file string>}`

Loads `{<file string>}` in sms mode, setting `\l_stex_current_document_uri` etc. accordingly and reverting them afterwards.

`\stex_smsmode_do:` should be called in escaped macros or environments allowed in sms mode; switches back to parsing file contents ignoring everything not explicitly allowed in sms mode.
 Does nothing outside of sms mode, so can be called safely.

16.2 Inheritance and Morphisms

```
\stex_require_module:N
\stex_require_module_noerr:N
\stex_require_module_noerr:n
```

makes sure that the module with the given URI is in scope. If not, it will attempt to load the module from disk. `\stex_require_module:N` throws an error if the module can't be found; `\stex_require_module_noerr:N` silently fails.

The `\*:n`-variant takes the URI tuple directly rather than a macro.

```
\usemodule \usemodule [<archive ID>] {<module>}
```

loads the specified module without exporting it further.

```
\importmodule \importmodule [<archive ID>] {<module>}
```

loads the specified module and exports it further.

```
\requiremodule \requiremodule [<archive ID>] {<module>}
```

loads the specified module without exporting it further, but generates FTML equivalent to `\importmodule`.

```
\stex_add_morphism:nnnn \stex_add_morphism:nnnn {<name>} {<module URI>} {<kind>} {<contents>}
```

adds a morphism to the current module

16.3 Theory Morphisms

```
\stex_structural_feature_morphism:nnnnn
\stex_structural_feature_morphism_with_macros:nnnnn
\stex_structural_feature_morphism_end:
```

```

#1 : name
#2 : kind
archive ID
#3 : module spec
#4 : additional attributes
```

```
\stex_iterate_morphisms:nn
```

```
\stex_get_in_morphism:n
```

```
copymodule (env.) (and \copymod)
```

```
interpretmodule (env.) (and \interpretmod)
```

\assign

\renamedekl

\assignMorphism

Chapter 17

Symbols

17.1 Declarations

`\symdecl` `\symdecl[*] {⟨iname⟩} [⟨options⟩]`

takes care of the optional arguments, styling, generates the symbol, defines the relevant macros and adds them to the current module.

`\symdef` `\symdef {⟨iname⟩} [⟨options⟩] {⟨notation⟩}`

`\textsymdecl`

`\stex_symdecl_do:` does the actual work. Requires all the `\l_stex_key_*` macros are set.

`\_stex_symdecl_html:` not namespaced so it can be reused in e.g. `sdefinitions` that generate new symbols. Requires that the relevant `\l_stex_key_*` macros and `\l_stex_macroname_str` are set.

`\stex_add_symbol:nnnnnnN` #1 : {⟨Macro name⟩}
 #2 : {⟨Name⟩}
 #3 : {⟨arity⟩}
 #4 : {(⟨Arg num⟩){⟨Arg str⟩}*}
 #5 : Definiens
 #6 : type
 #7 : Return
 #8 : Command
 adds a new symbol to the current module.

`\stex_has_definiens_p:N` ★
`\stex_has_definiens:N $\underline{TF}$` ★
`\stex_has_definiens_p:n` ★
`\stex_has_definiens:n $\underline{TF}$` ★

17.2 Retrieval

`\stex_iterate_symbols:n` iterates over all symbols currently in scope. The provided code will receive nine arguments: The URI of the containing module and the eight parameters as in `\stex_add_symbol:nnnnnnnN`.

iteration is stopped in the code using `\stex_iterate_break:n` or `\stex_iterate_break:n`.

`\stex_iterate_symbols:nn` `\stex_iterate_symbols:nn {<modules>} {<code>}`

iterates over all symbols in the comma-separated list of module URIs in `<modules>`. The provided code will receive nine arguments: The URI of the containing module and the eight parameters as in `\stex_add_symbol:nnnnnnnN`.

iteration is stopped in the code using `\stex_iterate_break:n` or `\stex_iterate_break:n`.

`\stex_get_symbol:n` attempts to find the symbol with the given name/id/URI suffix. If not successful, will execute the F code or throw an error. Otherwise, defines the following macros:

- `\l_stex_get_symbol_uri,`
- `\l_stex_get_symbol_arity_int,`
- `\l_stex_get_symbol_args_tl,`
- `\l_stex_get_symbol_def_tl,`
- `\l_stex_get_symbol_type_tl,`
- `\l_stex_get_symbol_return_tl,`
- `\l_stex_get_symbol_invoke_cs`

17.3 Variables

`\l_stex_variables_prop` contains all variables currently in scope.

`\vardef`

`\newvar`

`\newseq`

`\stex_get_var:n`

17.3.1 Variable Sequences

`\varseq`

17.4 Expressions

`\symuse` retrieves a symbol by name/id and invokes it as if using a semantic macro.

`\_stex_next_symbol:n` code to be executed the next time a symbol is invoked.

`\_stex_invoke_symbol:NnnnnnN`
`\_stex_invoke_symbol:NeooooN`
`\_stex_invoke_symbol:nnnnnnN`

invokes the symbol. Takes as arguments the following data, and defines the corresponding macros accordingly:

- `\l_stex_current_symbol_uri`,
- `\l_stex_current_symbol_arity_int`,
- `\l_stex_current_symbol_args_tl`,
- `definiens` (currently not used),
- `\l_stex_current_symbol_type_tl`,
- `\l_stex_current_symbol_return_tl`,
- the invocation macro (is called after setup).

Also defines `\l_stex_current_full_tl` and `\l_stex_current_display_tl`

For a “normal” symbol defined via `\symdecl` or `\symdef`, `\l_stex_current_symbol_invoke_cs` is defined as `\stex_invoke_symbol:.`

Opens a new $\text{\TeX}$ group that needs to be closed by `\l_stex_current_symbol_invoke_cs!`

`\stex_invoke_symbol:` Invokes `\l_stex_current_symbol_uri`; assumes all the `\l_stex_current_*`-macros have been defined prior.

`\stex_invoke_text_symbol:` Invokes `\l_stex_current_symbol_uri` as a symbol declared via `\textsymdecl`; assumes all the `\l_stex_current_*`-macros have been defined prior.

`\stex_invoke_sequence:` Invokes `\l_stex_current_symbol_uri` as a sequence variable declared via `\varseq`; assumes all the `\l_stex_current_*`-macros have been defined prior.

`\stex_invoke_structure:` Invokes a mathstructure symbol; assumes all the `\l_stex_current_*`-macros have been defined prior.

`\_stex_invoke_variable:nnnnnn`

invokes the variable. Takes as arguments the following data, and defines the corresponding macros accordingly:

- variable name,
- `\l_stex_current_symbol_arity_int`,
- `\l_stex_current_symbol_args_tl`,
- definiens (currently not used),
- `\l_stex_current_symbol_type_tl`,
- `\l_stex_current_symbol_return_tl`,
- the invocation macro (is called after setup).

`\svar`

17.4.1 Term Annotations

`\_stex_eat_exclamation_point:`

removes spurious ! characters

`\_stex_term_oms:nn` `\_stex_term_oms:nn {<notation id>} {<code>}`
 annotates `<code>` with `OMID="\l_stex_current_symbol_uri"`

`\_stex_term_omv:nn` `\_stex_term_omv:nn {<notation id>} {<code>}`
 annotates `<code>` with `OMV="\l_stex_current_symbol_uri"`
TODO: This shouldn't use `\l_stex_current_symbol_uri`!

`\_stex_term_oma:nn` `\_stex_term_oma:nn {<notation id>} {<code>}`
 annotates `<code>` with `OMA="\l_stex_current_symbol_uri"`

`\_stex_term_omb:nn` `\_stex_term_omb:nn {<notation id>} {<code>}`
 annotates `<code>` with `OMBIND="\l_stex_current_symbol_uri"`

17.4.2 Symbol Arguments

`\_stex_term_arg:nnnnn` marks an argument of a symbol application, sets the precedence accordingly, sets `\l_stex_allow_semantic_bool` etc.

#1 : argument number
 #2 : argument mode
 #3 : precedence
 #4 : argument name (WiP)
 #5 : code / body

`\_stex_term_arg:nnn` `\_stex_term_arg:nnn` `{\mode}` `{\num}` `{\code}`

inserts the HTML annotations for the argument

`\_stex_term_arg_aB:nnnnn` takes care of sequence arguments

`\seqmap`

17.4.3 Notation components

`\comp`
`\compemph@uri`
`\compemph`

`\varemp@uri`
`\varemp`

`\defemph@uri`
`\defemph`

`\symrefemph@uri`
`\symrefemph`

The following commands do the actual attributes:

`\_do_comp:nnN` `\_do_comp:nnN` `{\key-suffix}` `{\html-value}` `\comp-macro` `{\code}`

annotated `\code` with `data-ftml-{\key-suffix}` using the highlighting dictated by `\comp-macro`.

17.4.4 Symbol References

`\symref`
`\sr`

`\symname`
`\sn`
`\sns`
`\Symname`
`\Sn\Sns`

`\varref`
`\varname`
`\Varname`

`\definiendum`
`\definame`
`\Definame`
`\defnotation`

`\foldexpr`

17.5 Checking

`\stex_if_check_terms_p:` $\star$ conditional whether terms (types and definientia of symbols) should get syntactically
`\stex_if_check_terms:` $\underline{TF}$ $\star$ checked. In HTML mode, this is always false, since they get written to the HTML
 anyway.

`\stex_check_term:` nn typesets the given expression in a `\tiny\marginpar`, checking its syntactic validity. Does
 nothing, if term checking is not explicitly activated.

`\_stex_check_terms:` checks the type and definiens components of a symbol declaration (i.e. `\stex_key_-`
`type_tl`, `\stex_key_def_tl` and `\stex_key_return_tl!`).

Chapter 18

Notations

`\notation`

`\varnotation`

`\stex_notation_parse:n` parses the body of a notations and saves it in `\l_stex_notation_macrocode_cs`

`\_stex_notation_add:` adds the fully parsed notation to the current module

<code>\stex_add_notation:nnnnn</code>	#1 :	URI
<code>\stex_add_notation:ooeoo</code>	#2 :	variant
	#3 :	arity
	#4 :	macro body
	#5 :	op
		adds the notation to the current module

`\_stex_map_args:N`
`\_stex_map_notation_args:N`

`\_stex_var_notation_macro:`

`\setnotation`
`\_stex_notation_set_default:n`

<code>\stex_iterate_notations:nn</code>	<code>\stex_iterate_notations:nn {<modules>} {<code>}</code>
---	--

iterates over all notations in the comma-separated list of module URIs and executes `{<code>}` with #1,#2,#3,#4,#5 as the notation parameters.

<code>\stex_use_notation:nnTF</code>	<code>\stex_use_notation:nn {<uri>} {<id>} {<exists code>} {<missing code>}</code>
--------------------------------------	--

<code>\stex_use_op_notation:nnTF</code>	sets <code>\l_stex_notation_cs</code>
---	---------------------------------------

<code>_stex_notation_check:</code>

<code>_stex_notation_do_html:n</code>
--

<code>_stex_notation_make_args:</code>

<code>\stex_do_default_notation:</code>
<code>\stex_do_default_notation_op:</code>

<code>\STEXInternalNotation</code>	#1 : notation id
	#2 : operator precedence
	#3 : intent (WiP)
	#4 : arguments
	#5 : notation code
	#6 : continuation

<code>\argsep</code>

<code>\argmap</code>

<code>\argarraymap</code>

18.1 Automated Bracketing

<code>\infprec</code>	infinite and negative infinite precedences.
-----------------------	---

<code>\neginfprec</code>

<code>_stex_maybe_brackets:nn</code>	<code>_stex_maybe_brackets:nn {<prec>} {<body>}</code>
---------------------------------------	---

inserts parentheses around `<body>` iff precedences and context call for it.

`\dobrackets` actually inserts parentheses around its argument.

`\withbrackets` `\withbrackets {⟨left⟩} {⟨right⟩} {⟨body⟩}`
sets `⟨left⟩` and `⟨right⟩` as the parentheses to use in `⟨body⟩`.

`\dowithbrackets` `\dowithbrackets {⟨left⟩} {⟨right⟩} {⟨body⟩}`
combines `\withbrackets` and `\dobrackets`.

Chapter 19

Structures

```
mathstructure (env.)  
  
extstructure (env.)
```

```
\this
```

```
\stex_get_mathstructure:n sets \l_stex_get_structure_module_uri or throws an error if no structure by the given  
name/id exists.
```

```
\usestructure
```

Chapter 20

Statements

`\stex_do_for_list:` resolves each id in the `for={...}` comma-separated list of ids `\l_stex_key_for_clist`. Stores the full resolved URIs in `\l_stex_fors_seq`.

`\stex_new_statement:nnn` `\stex_new_statement:nnn {<env name>} {<macro name>} {<code>}`
defines a new statement environment `s<env name>` and the optional macro-variant `\inline<macro name>` (may be empty). `{<code>}` does arbitrary additional setup and is called in the `\begin` part of the environment and the macro after optional arguments have been parsed. e.g. `\stex_new_statement:nnn{definition}{def}{...}` defines the `sdefinition` environment and the `\inlinedef` macro.

`sdefinition (env.)` (and `\inlinedef`)

`sassertion (env.)` (and `\inlineass`)

`sexample (env.)` (and `\inlineex`)

`sparagraph (env.)`

`definiens`

`\stex_add_definiens:nn` marks #2 as the definiens of the symbol with uri #1. Also marks the symbol as being defined, iff the symbol is declared in the current module.

`\varbind`

`\conclusion`

`\premise`

Chapter 21

Proofs

`sproof (env.)`

`subproof (env.)`

`spfsketchenv (env.)`

`\spfsketch`

`spfstepenv (env.)`

`spfblock (env.)`

`\spfstep`

`\assumption`

`\conclude`

`\eqstep`

`\yield`

`\spfjust`

`\spfby`

\spfarg

\sproofend

\stexcommentfont

Chapter 22

Metatheory

TODO

Chapter 23

Others

Chapter 24

Additional Packages

24.1 NotesSlides Documentation

TODO

24.2 Problem Documentation

TODO

24.3 HWExam Documentation

TODO

24.4 Tikzinput Documentation

TODO

Part IV

Implementation

Chapter 25

Setting up

Setup code for the document class

```
1 <*cls>
2 %%%%%%%%% stex.dtx %%%%%%%%%
3
4 \RequirePackage{expl3,l3keys2e}
5 \ProvidesExplClass{stex}{2026/06/24}{4.1.0}{sTeX document class}
6 \IfFileExists{stex-expl-compat.sty}{
7   \usepackage{stex-expl-compat}
8 }{}
9
10 \DeclareOption*{\PassOptionsToPackage{\CurrentOption}{stex}}
11 \ProcessOptions
12
13 \RequirePackage{stex}
14 \str_set:Nn \g_stex_document_kind_str{fragment}
15
16 \LoadClass{article}
17 </cls>
```

Setup code for the package

```
18 <*package>
19 \RequirePackage{expl3,l3keys2e,ltxcmds}
20 \ProvidesExplPackage{stex}{2026/06/24}{4.1.0}{sTeX package}
21 \IfFileExists{stex-expl-compat.sty}{
22   \usepackage{stex-expl-compat}
23 }{}
24 \RequirePackage{stex-logo} % externalized for backwards-compatibility reasons
25 \RequirePackage{standalone}
26
27 \bool_new:N \c_stex_use_sref_bool
28 \message{^^J*~This~is~sTeX~version~4.1.0~*^^J}
```

Package options:

```
29 \keys_define:nn { stex / package } {
30   debug      .str_set_x:N = \c_stex_debug_clist ,
31   lang       .clist_set:N = \c_stex_languages_clist ,
32   mathhub    .tl_set_x:N = \mathhub ,
33   usesms     .bool_set:N = \c_stex_persist_mode_bool ,
```

```

34  writesms .bool_set:N = \c_stex_persist_write_mode_bool ,
35  forcenousesms .bool_set:N = \c_stex_persist_force_bool ,
36  checkterms .bool_set:N = \c_stex_check_terms_bool ,
37  metadata .bool_set:N = \c_stex_metadata_bool ,
38  image .bool_set:N = \c_tikzinput_image_bool,
39  nofrontmatter .bool_set:N = \c_stex_no_frontmatter_bool,
40  unknown .code:n = {}
41 }
42 \exp_args:NNo \clist_set:Nn \c_stex_debug_clist \c_stex_debug_clist
43 \ProcessKeysOptions { stex / package }

Error messages:
44 \input{stex-en.ldf}

```

Chapter 26

Utilities

`\stex_ignore_spaces_and_pars:`

```
45 \cs_new_protected:Nn \stex_ignore_spaces_and_pars:{
46   \begingroup\catcode13=10\relax
47   \@ifnextchar\par{
48     \endgroup\expandafter\stex_ignore_spaces_and_pars:\@gobble
49   }{
50     \endgroup
51   }
52 }
```

(End of definition for \stex_ignore_spaces_and_pars:. This function is documented on page 123.)
sfunction

`\stex_undefine:N`

`\stex_undefine:c`

```
53 \cs_new_protected:Nn \stex_undefine:N {
54   \cs_set_eq:NN #1 \tex_undefined:D
55 }
56 \cs_generate_variant:Nn \stex_undefine:N {c}
```

(End of definition for \stex_undefine:N. This function is documented on page 123.)
sfunction

`\stex_str_if_starts_with_p:nn`

`\stex_str_if_starts_with:nnTF`

```
57 \prg_new_conditional:Nnn \stex_str_if_starts_with:nn {p,T,F,TF} {
58   \exp_args:Ne \str_if_eq:nnTF {
59     \str_range:nnn{#1}{1}{\str_count:n{#2}}
60   }{#2}\prg_return_true: \prg_return_false:
61 }
```

(End of definition for \stex_str_if_starts_with:nnTF. This function is documented on page 123.)
sfunction

`\stex_str_if_ends_with_p:nn`

`\stex_str_if_ends_with:nnTF`

```
62 \prg_new_conditional:Nnn \stex_str_if_ends_with:nn {p,T,F,TF} {
63   \exp_args:Ne \str_if_eq:nnTF {
64     \str_range:nnn{#1}{- \str_count:n{#2}}{-1}
65   }{#2}\prg_return_true: \prg_return_false:
66 }
```

(End of definition for `\stex_str_if_ends_with:nnTF`. This function is documented on page 123.)

```
sfunction
```

```
\stex_deactivate_macro:Nn
\stex_reactivate_macro:N
67 \cs_new_protected:Nn \stex_deactivate_macro:Nn {
68   \tl_set_eq:cN{\tl_to_str:n{#1}---orig}#1
69   \cs_set_protected:Npn#1{
70     \msg_error:nnnn{stex}{error/deactivated-macro}{#1}{#2}
71   }
72 }
73 \cs_new_protected:Nn \stex_reactivate_macro:N {
74   \cs_set_eq:Nc #1{\tl_to_str:n{#1}---orig}
75 }
```

(End of definition for `\stex_deactivate_macro:Nn` and `\stex_reactivate_macro:N`. These functions are documented on page 123.)

```
sfunction
```

26.1 kpsewhich and Environment Variables

```
76 <@@=stex_kpse>
```

```
\stex_kpsewhich:Nn
77 \cs_new_protected:Nn \stex_kpsewhich:Nn {
78   \group_begin:
79   \catcode'\ =12
80   \sys_get_shell:nnN { kpsewhich ~ #2 } { } \l_tmpa_tl
81   \tl_gset_eq:NN \l_tmpa_tl \l_tmpa_tl
82   \group_end:
83   \exp_args:NNo\str_set:Nn #1 \l_tmpa_tl
84   \tl_trim_spaces:N #1
85 }
```

(End of definition for `\stex_kpsewhich:Nn`. This function is documented on page 123.)

```
sfunction
```

```
\stex_get_env:Nn
86 \sys_if_platform_windows:TF{
87   \cs_new_protected:Nn \stex_get_env:Nn {\group_begin:
88     \escapechar=-1\catcode'\ =12
89     \exp_args:NNe \stex_kpsewhich:Nn #1 {-expand-var~\c_percent_str#2\c_percent_str}
90     \exp_args:NNx \str_if_eq:VnT #1 {\c_percent_str #2 \c_percent_str}{
91       \str_clear:N #1
92     }
93     \exp_args:NNx\use:nn\group_end:{
94       \str_set:Nn \exp_not:N #1 { #1 }
95     }
96   }
97 }{
98   \cs_new_protected:Nn \stex_get_env:Nn {
99     \stex_kpsewhich:Nn #1 {-var-value~#2}
100   }
101 }
```

(End of definition for `\stex_get_env:Nn`. This function is documented on page 124.)

sfunction

26.2 Logging

```

102 <@@=stex_debug>

\stex_debug:nn

103 \cs_new_protected:Nn \stex_debug:nn {
104   \exp_args:NNo \clist_if_in:NnTF \c_stex_debug_clist { \tl_to_str:n{all} }{
105     \__stex_debug_:nn{#1}{#2}
106   }{
107     \exp_args:NNo \clist_if_in:NnT \c_stex_debug_clist { \tl_to_str:n{#1} }{
108       \__stex_debug_:nn{#1}{#2}
109     }
110   }
111 }

112
113 \cs_new_protected:Nn \__stex_debug_:nn {
114   \msg_set:nnn{stex}{debug / #1}{
115     \Debug~#1:~#2\
116   }
117   \msg_none:nn{stex}{debug / #1}
118 }

We check an environment variable for debugging and set things up:

119 \stex_get_env:Nn\__stex_debug_env_str{STEX_DEBUG}
120 \str_if_empty:NTF\__stex_debug_env_str {
121   \clist_set_eq:NN \l__stex_debug_cl \c_stex_debug_clist
122 }{
123   \clist_set:No \l__stex_debug_cl {\__stex_debug_env_str}
124 }
125 \clist_clear:N \c_stex_debug_clist
126 \clist_map_inline:Nn \l__stex_debug_cl {
127   \exp_args:NNo \clist_put_right:Nn \c_stex_debug_clist
128   { \tl_to_str:n{#1} }
129 }
130
131 \exp_args:NNo \clist_if_in:NnTF \c_stex_debug_clist {\tl_to_str:n{all}} {
132   \msg_redirect_module:nnn{ stex }{ none }{ warning }
133   \stex_debug:nn{all}{Logging-everything!}
134 }{
135   \clist_map_inline:Nn \c_stex_debug_clist {
136     \msg_redirect_name:nnn{ stex }{ debug / #1 }{ warning }
137     \stex_debug:nn{#1}{Logging~#1}
138   }
139 }
```

(End of definition for `\stex_debug:nn`. This function is documented on page 124.)

sfunction

26.3 File Paths

```

140 <@@=stex_path>

\stex_file_set:Nn
\stex_file_set:No
\stex_file_set:Ne
141 \cs_new_protected:Nn \stex_file_set:Nn {
142   \str_if_empty:nTF {#2} { \seq_clear:N #1 }{
143     \exp_args:NNno \seq_set_split:Nnn #1 / { \tl_to_str:n{#2} }
144   }
145 }
146 \cs_generate_variant:Nn \stex_file_set:Nn {No, Ne}

(End of definition for \stex_file_set:Nn. This function is documented on page 124.)
sfunction

\stex_file_resolve:Nn
\stex_file_resolve:No
\stex_file_resolve:Ne
147 \sys_if_platform_windows:TF{
148   \cs_new_protected:Npn \__stex_path_win_take:w #1#2#3 \__stex_path_: {
149     \uppercase{ \str_set:Nn \l__stex_path_str{#1}}
150     \str_set:Ne \l__stex_path_win_drive {\l__stex_path_str #2}
151     \str_set:Nn \l__stex_path_str{#3}
152   }
153   \cs_new_protected:Nn \stex_file_resolve:Nn {
154     \str_set:Nn \l__stex_path_str {#2}
155     \str_clear:N \l__stex_path_win_drive
156     \exp_args:NNno \str_replace_all:Nnn \l__stex_path_str \c_backslash_str /
157     \exp_args:Ne \str_if_eq:nnT {\str_item:Nn \l__stex_path_str 2} : {
158       \exp_after:wN \__stex_path_win_take:w \l__stex_path_str \__stex_path_:
159     }
160     \stex_file_set:No #1 \l__stex_path_str
161     \__stex_path_canonicalize:N #1
162     \str_if_empty:NF \l__stex_path_win_drive {
163       \seq_pop_left:NN #1 \l__stex_path_str
164       \seq_put_left:No #1 \l__stex_path_win_drive
165     }
166     %\stex_debug:nn{files}{Set-\tl_to_str:n{#1}~to~\stex_file_use:N #1}
167   }
168 }{
169   \cs_new_protected:Nn \stex_file_resolve:Nn {
170     \str_set:Nn \l__stex_path_str {#2}
171     \stex_file_set:No #1 \l__stex_path_str
172     \__stex_path_canonicalize:N #1
173     % \stex_debug:nn{files}{Set-\tl_to_str:n{#1}~to~\stex_file_use:N #1}
174   }
175 }
176 \cs_generate_variant:Nn \stex_file_resolve:Nn {No, Ne}

Auxiliary methods:
177 \cs_new_protected:Nn \__stex_path_canonicalize:N {
178   \seq_if_empty:NF #1 {
179     \seq_pop:NN #1 \l__stex_path_can_str
180     \seq_clear:N \l__stex_path_can_seq
181     \str_if_empty:NTF \l__stex_path_can_str {
182       \seq_map_function:NN #1 \__stex_path_dodots:n
183       \seq_put_left:Nn \l__stex_path_can_seq {}

```

```

184     }{
185       \seq_push:No #1 \l__stex_path_can_str
186       \seq_map_function:NN #1 \__stex_path_dodots:n
187     }
188     \seq_set_eq:NN #1 \l__stex_path_can_seq
189   }
190 }
191
192 \cs_new_protected:Nn \__stex_path_dodots:n {
193   \str_if_empty:nF{#1}{
194     \str_if_eq:nnF {#1} {.} {
195       \str_if_eq:nnTF {#1} {...} {
196         \seq_if_empty:NF \l__stex_path_can_seq {
197           \seq_pop_right:NN \l__stex_path_can_seq \l__stex_path_can_str
198         }
199       }{
200         \seq_put_right:Nn \l__stex_path_can_seq {#1}
201       }
202     }
203   }
204 }

```

(End of definition for `\stex_file_resolve:Nn`. This function is documented on page 124.)

sfunction

`\stex_file_use:N`

```

205 \cs_new:Nn \stex_file_use:N {
206   \seq_use:Nn #1 /
207 }

```

(End of definition for `\stex_file_use:N`. This function is documented on page 124.)

sfunction

`\stex_file_split_off_ext:NN`

```

208 \cs_new_protected:Nn \stex_file_split_off_ext:NN {
209   \seq_set_eq:NN #1 #2
210   \seq_pop_right:NN #1 \l__stex_path_str
211   \seq_set_split:NnV \l__stex_path_seq . \l__stex_path_str
212   \seq_pop_right:NN \l__stex_path_seq \l__stex_path_str
213   \seq_put_right:Ne #1 {\seq_use:Nn \l__stex_path_seq .}
214 }

```

(End of definition for `\stex_file_split_off_ext:NN`. This function is documented on page 124.)

sfunction

`\stex_file_split_off_lang:NN`

```

215 \cs_new_protected:Nn \stex_file_split_off_lang:NN {
216   \seq_set_eq:NN #1 #2
217   \seq_pop_right:NN #1 \l__stex_path_str
218   \seq_set_split:NnV \l__stex_path_seq . \l__stex_path_str
219   \seq_pop_right:NN \l__stex_path_seq \l__stex_path_str
220
221   \seq_pop_right:NN \l__stex_path_seq \l__stex_path_str
222   \exp_args:NNo \prop_if_in:NnF \c_stex_languages_prop \l__stex_path_str {

```

```

223   \seq_put_right:No \l__stex_path_seq \l__stex_path_str
224 }
225
226   \seq_put_right:Ne #1 {\seq_use:Nn \l__stex_path_seq .}
227 }

```

(End of definition for \stex_file_split_off_lang:NN. This function is documented on page 124.)

sfunction

```

\c_stex_pwd_file We determine the pwd
\c_stex_main_file
228 \sys_if_platform_windows:TF{
229   \stex_get_env:Nn \l__stex_path_str{CD}
230 }{
231   \stex_get_env:Nn \l__stex_path_str{PWD}
232 }
233 \stex_file_resolve:No \c_stex_pwd_file \l__stex_path_str
234 \seq_set_eq:NN \c_stex_main_file \c_stex_pwd_file
235 \seq_put_right:Ne \c_stex_main_file {\jobname\tl_to_str:n{.tex}}
236
237 \stex_debug:nn {files} {PWD:~\stex_file_use:N \c_stex_pwd_file}

```

(End of definition for \c_stex_pwd_file and \c_stex_main_file. These variables are documented on page 124.)

```

\c_stex_home_file
238 \sys_if_platform_windows:TF{
239   \stex_get_env:Nn \l__stex_path_str {homedrive\c_percent_str\c_percent_str homedrive}
240 }{
241   \stex_get_env:Nn \l__stex_path_str {HOME}
242 }
243 \stex_file_resolve:No \c_stex_home_file \l__stex_path_str

```

(End of definition for \c_stex_home_file. This variable is documented on page 125.)

```

\g_stex_current_file
244 \seq_gset_eq:NN \g_stex_current_file \c_stex_main_file

```

(End of definition for \g_stex_current_file. This variable is documented on page 125.)

```

\stex_input_with_hooks:Nn
\stex_input_with_hooks:Ne
245 \cs_new_protected:Nn \stex_input_with_hooks:Nn {
246   \stex_with_file_hooks:Nnn #1 {#2} {\input{#2}}
247 }
248 \cs_generate_variant:Nn \stex_input_with_hooks:Nn {Ne}

```

(End of definition for \stex_input_with_hooks:Nn. This function is documented on page 125.)

sfunction

```

\stex_with_file_hooks:Nnn
249 \cs_new_protected:Nn \stex_with_file_hooks:Nnn {
250   \stex_pseudogroup:nn {
251     \stex_file_resolve:Nn \l__stex_path_seq {#2}
252     \seq_gset_eq:NN \g_stex_current_file \l__stex_path_seq
253     \tl_set_eq:NN \l_stex_current_document_uri #1
254     \str_set:Ne \l_stex_current_language_str { \stex_document_uri_language:N \l_stex_current

```

```

255     #3
256   }{
257     \tl_gset:Nn \exp_not:N \g_stex_current_file { \exp_args:No \exp_not:n \g_stex_current_f
258     \stex_pseudogroup_restore:N \l_stex_current_language_str
259     \stex_pseudogroup_restore:N \l_stex_current_document_uri
260   }
261 }

```

(End of definition for \stex_with_file_hooks:Nnn. This function is documented on page 125.)

sfunction

\stex_if_file_absolute_p:N
\stex_if_file_absolute:N $\underline{NTF}$

```

262 \sys_if_platform_windows:TF {
263   \prg_new_conditional:Nnn \stex_if_file_absolute:N {p, T, F, TF} {
264     \seq_if_empty:NTF #1 \prg_return_false: {
265       \exp_args:Ne \tl_if_empty:nTF {\seq_item:Nn #1 1} \prg_return_true: {
266         \exp_args:Ne \str_if_eq:nnTF { \exp_args:Ne \str_item:nn {\seq_item:Nn #1 1} 2} :
267         \prg_return_true: \prg_return_false:
268       }
269     }
270   }
271 }{
272   \prg_new_conditional:Nnn \stex_if_file_absolute:N {p, T, F, TF} {
273     \seq_if_empty:NTF #1 \prg_return_false: {
274       \exp_args:Ne \tl_if_empty:nTF {\seq_item:Nn #1 1}
275       \prg_return_true: \prg_return_false:
276     }
277   }
278 }

```

(End of definition for \stex_if_file_absolute:N $\underline{NTF}$. This function is documented on page 125.)

sfunction

\stex_if_file_starts_with:N $\underline{NTF}$

```

279 \prg_new_protected_conditional:Nnn \stex_if_file_starts_with:NN {T,F,TF} {
280   \seq_set_eq:NN \l__stex_path_a_seq #1
281   \seq_set_eq:NN \l__stex_path_b_seq #2
282   \tl_clear:N \l__stex_path_return_tl
283   \bool_while_do:nn{
284     \bool_not_p:n{
285       \bool_lazy_any_p:n{
286         {\seq_if_empty_p:N \l__stex_path_a_seq}
287         {\seq_if_empty_p:N \l__stex_path_b_seq}
288         {\bool_not_p:n{\tl_if_empty_p:N \l__stex_path_return_tl}}
289       }
290     }
291   }{
292     \seq_pop_left:NN \l__stex_path_a_seq \l__stex_path_a_tl
293     \seq_pop_left:NN \l__stex_path_b_seq \l__stex_path_b_tl
294     \str_if_eq:NNF \l__stex_path_a_tl \l__stex_path_b_tl {
295       \tl_set:Nn \l__stex_path_return_tl {\prg_return_false:}
296     }
297   }
298   \tl_if_empty:NTF \l__stex_path_return_tl {

```

```

299     \seq_if_empty:NTF \l__stex_path_b_seq \prg_return_true: \prg_return_false:
300   } \l__stex_path_return_tl
301 }

```

(End of definition for `\stex_if_file_starts_with:NNTF`. This function is documented on page 125.)

```
sfunction
```

26.4 Group-like Behaviours

```

302 <@@=stex_groups>

```

`\stex_pseudogroup:nn`

```

303 \cs_new_protected:Npn \stex_pseudogroup:nn {
304   \exp_args:Nne \use:nn
305 }

```

(End of definition for `\stex_pseudogroup:nn`. This function is documented on page 125.)

```
sfunction
```

`\stex_pseudogroup_restore:N`

```

306 \cs_new:Nn \stex_pseudogroup_restore:N {
307   \tl_if_exist:NTF #1 {
308     \tl_set:Nn \exp_not:N #1 { \exp_args:No \exp_not:n #1 }
309   }{
310     \stex_undefine:N \exp_not:N #1
311   }
312 }

```

(End of definition for `\stex_pseudogroup_restore:N`. This function is documented on page 126.)

```
sfunction
```

`\stex_pseudogroup_with:nn`

```

313 \cs_new_protected:Nn \stex_pseudogroup_with:nn {
314   \tl_map_inline:nn{#1}{
315     \cs_set_eq:cN{__stex_groups_\tl_to_str:n{##1}}##1
316   }
317   #2
318   \tl_map_inline:nn{#1}{
319     \cs_set_eq:Nc##1{__stex_groups_\tl_to_str:n{##1}}
320     \stex_undefine:c{__stex_groups_\tl_to_str:n{##1}}
321   }
322 }

```

(End of definition for `\stex_pseudogroup_with:nn`. This function is documented on page 126.)

```
sfunction
```

`\stex_metagroup_new:` Current metagroup group level

```

323 \int_new:N \l__stex_groups_lvl_int

```

start a new metagroup at the current group level

```

324 \cs_new_protected:Nn \stex_metagroup_new: {
325   \int_set_eq:NN \l__stex_groups_lvl_int \currentgrouplevel
326 }

```

(End of definition for `\stex_metagroup_new:`. This function is documented on page 126.)

sfunction

```

\stex_metagroup_do_in:n
\stex_metagroup_do_in:e
327 \cs_new_protected:Npn \stex_metagroup_do_in:n {
328   \int_compare:nNnTF \l__stex_groups_lvl_int = \currentgrouplevel
329     \use:n \__stex_groups_do_in:n
330 }
331 \cs_generate_variant:Nn \stex_metagroup_do_in:n {e}
332
333 \cs_new_protected:Nn \__stex_groups_do_in:n {
334   \tl_if_exist:cTF{g__stex_groups\_the\currentgrouplevel\_tl}{
335     \exp_args:Nno \tl_gput_right:cn{g__stex_groups\_the\currentgrouplevel\_tl}
336   }{
337     \exp_args:Nno \tl_gset:cn{g__stex_groups\_the\currentgrouplevel\_tl}
338   }{ #1 }
339   \bool_if_exist:cF {l__stex_groups\_the\currentgrouplevel\_bool} {
340     \group_insert_after:N \__stex_groups_do:
341     \bool_set_true:c {l__stex_groups\_the\currentgrouplevel\_bool}
342   }
343   #1
344 }
345
346 \cs_new_protected:Nn \__stex_groups_do: {
347   \tl_if_exist:cT{g__stex_groups\_int_eval:n{\currentgrouplevel+1}_tl}{
348     \exp_args:Nno \exp_args:No \stex_metagroup_do_in:n {
349       \csname g__stex_groups\_int_eval:n{\currentgrouplevel+1}_tl \endcsname
350     }
351     \cs_undefine:c{g__stex_groups\_int_eval:n{\currentgrouplevel+1}_tl}
352   }
353   \bool_if_exist:cF {l__stex_groups\_the\currentgrouplevel\_bool} {
354     \group_insert_after:N \__stex_groups_do:
355     \bool_set_true:c {l__stex_groups\_the\currentgrouplevel\_bool}
356   }
357 }

```

(End of definition for `\stex_metagroup_do_in:n`. This function is documented on page 126.)

sfunction

26.5 Key Handling

358 $\langle @@=\text{stex_keys} \rangle$

`\stex_keys_define:nnnn`

```

359 \cs_new_nopar:Nn \stex_keys_define:nnnn {
360   \tl_gset:cn {__stex_keys_keys_#1_pre_tl}{#2}
361   \tl_gset:cn {__stex_keys_keys_#1_def_tl}{#3}
362   \tl_if_empty:nF{#4}{
363     \clist_map_inline:nn{#4}{
364       \tl_set_eq:Nc \l__stex_keys_tl {__stex_keys_keys_##1_pre_tl}
365       \tl_gput_left:co{__stex_keys_keys_#1_pre_tl} \l__stex_keys_tl
366       \tl_set_eq:Nc \l__stex_keys_tl {__stex_keys_keys_##1_def_tl}
367       \tl_gput_left:cn{__stex_keys_keys_#1_def_tl} ,

```

```

368     \tl_gput_left:co{__stex_keys_keys_#1_def_tl} \l__stex_keys_tl
369   }
370 }
371 \tl_set_eq:Nc \l__stex_keys_tl {__stex_keys_keys_#1_def_tl}
372 \exp_args:Nno \keys_define:nn {stex / #1} {\l__stex_keys_tl}
373 }

```

(End of definition for `\stex_keys_define:nnnn`. This function is documented on page 126.)

sfunction

`\stex_keys_set:nn`

```

374 \cs_new_nopar:Nn \stex_keys_set:nn {
375   \use:c{__stex_keys_keys_#1_pre_tl}
376   \keys_set:nn {stex / #1} { #2 }
377 }

```

(End of definition for `\stex_keys_set:nn`. This function is documented on page 126.)

sfunction

Some ubiquitous key sets:

```

378 \stex_keys_define:nnnn{archive}{file}{
379   \str_clear:N \l_stex_key_archive_str
380   \str_clear:N \l_stex_key_file_str
381 }{
382   archive .str_set_x:N = \l_stex_key_archive_str ,
383   file .str_set_x:N = \l_stex_key_file_str
384 }{}
385
386 \stex_keys_define:nnnn{id}{
387   \str_clear:N \l_stex_key_id_str
388 }{
389   id .str_set_x:N = \l_stex_key_id_str
390 }{}
391
392 \stex_keys_define:nnnn{title}{
393   \tl_clear:N \l_stex_key_title_tl
394 }{
395   title .tl_set:N = \l_stex_key_title_tl
396 }{}
397
398 \stex_keys_define:nnnn{style}{
399   \clist_clear:N \l_stex_key_style_clist
400 }{
401   style .clist_set:N = \l_stex_key_style_clist
402 }{}
403
404 \stex_keys_define:nnnn{deprecate}{
405   \str_clear:N \l_stex_key_deprecate_str
406 }{
407   deprecate .str_set_x:N = \l_stex_key_deprecate_str
408 }{}
409
410 \stex_keys_define:nnnn{uses}{}{
411   uses .code:n = {
412     \clist_map_inline:nn{#1}{

```

```

413     \stex_str_if_starts_with:nnTF{##1} [{
414         \__stex_keys_split_at_bracket:w ##1 \stex_end:
415     } {
416         \usemodule{##1}
417     }
418 }
419 }
420 } {}
421
422 \cs_new_protected:Npn \__stex_keys_split_at_bracket:w [ #1 ] #2 \stex_end: {
423     \usemodule[##1]{##2}
424 }

```

26.6 Languages

```

425 <@@=stex_lang>

```

`\c_stex_languages_prop`

`\c_stex_language_abbrevs_prop`

```

426 \exp_args:NNx \prop_const_from_keyval:Nn \c_stex_languages_prop { \tl_to_str:n {
427     en = english ,
428     de = ngerman ,
429     ar = arabic ,
430     bg = bulgarian ,
431     ru = russian ,
432     fi = finnish ,
433     ro = romanian ,
434     tr = turkish ,
435     fr = french ,
436     sl = slovenian
437 }}
438
439 \exp_args:NNx \prop_const_from_keyval:Nn \c_stex_language_abbrevs_prop { \tl_to_str:n {
440     english   = en ,
441     ngerman   = de ,
442     arabic    = ar ,
443     bulgarian = bg ,
444     russian   = ru ,
445     finnish   = fi ,
446     romanian  = ro ,
447     turkish   = tr ,
448     french    = fr ,
449     slovenian = sl ,
450 }}
451 % todo: chinese simplified (zhs)
452 %         chinese traditional (zht)

```

(End of definition for `\c_stex_languages_prop` and `\c_stex_language_abbrevs_prop`. These variables are documented on page 127.)

`\l_stex_current_language_str`

```

453 \str_new:N \l_stex_current_language_str

```

(End of definition for `\l_stex_current_language_str`. This variable is documented on page 127.)

Loading babel (if necessary), depending on the `lang` package option:


```

454 \clist_if_empty:NF \c_stex_languages_clist {
455   \bool_set_false:N \l__stex_lang_turkish_bool
456   \seq_clear:N \l_tmpa_seq
457   \clist_map_inline:Nn \c_stex_languages_clist {
458     \str_if_empty:NF \l_stex_current_language_str {
459       \str_set:Nn \l_stex_current_language_str {#1}
460     }
461     \str_set:Ne \l_tmpa_str {#1}
462     \str_if_eq:nnT {#1}{tr}{
463       \bool_set_true:N \l__stex_lang_turkish_bool
464     }
465     \prop_get:NoNTF \c_stex_languages_prop \l_tmpa_str \l_tmpa_str {
466       \tl_set_rescan:Nno \l_tmpa_str {} \l_tmpa_str
467       \seq_put_right:No \l_tmpa_seq \l_tmpa_str
468     } {
469       \msg_error:nmx{stex}{error/unknownlanguage}{\l_tmpa_str}
470     }
471   }
472   \stex_debug:nn{lang} {Languages:~\seq_use:Nn \l_tmpa_seq {,~} }
473   \bool_if:NTF \l__stex_lang_turkish_bool {
474     \exp_args:NNe \use:nn \RequirePackage
475     {[main=\seq_use:Nn \l_tmpa_seq, ,shorthands=:!]}{babel}
476   }{
477     \exp_args:NNe \use:nn \RequirePackage
478     {[main=\seq_use:Nn \l_tmpa_seq, ]}{babel}
479   }
480 }

```

26.7 Styling

```

481 <@@=stex_styles>

```

```

\stex_new_stylable_cmd:nnnn

```

```

\stex_style_apply:

```

```

482 \cs_new_protected:Nn \stex_new_stylable_cmd:nnnn {
483   \exp_after:wN \newcommand \cs:w stexstyle#1 \cs_end:[2] [] {
484     \__stex_styles_patch:nnn{#1}{##1}{##2}
485   }
486   \exp_after:wN \NewDocumentCommand\cs:w #1\cs_end:{#2}{
487     \cs_set:Npn \stex_style_apply: {
488       \__stex_styles_apply_patch:n{#1}
489     }
490     #3
491   }
492   \tl_set:cn {\__stex_styles_style_#1:} { #4 }
493 }
494
495 \cs_new_protected:Nn \__stex_styles_patch:nnn {
496   \str_if_empty:nTF {#2}{
497     \tl_set:cn{\__stex_styles_style_#1:}{#3}
498   }{
499     \tl_set:cn{\__stex_styles_style_#1_#2:}{#3}
500   }
501 }

```

```

502
503 \cs_new_protected:Nn \__stex_styles_apply_patch:n {
504   \clist_if_empty:NTF \l_stex_key_style_clist {
505     \tl_clear:N \thisstyle
506     \use:c{__stex_styles_style_#1:}
507   }{
508     \clist_get:NN \l_stex_key_style_clist \thisstyle
509     \tl_if_exist:cTF{__stex_styles_style_#1_\thisstyle :}{
510       \use:c{__stex_styles_style_#1_\thisstyle :}
511     }{
512       \use:c{__stex_styles_style_#1:}
513     }
514   }
515 }

```

(End of definition for `\stex_new_stylable_cmd:nnnn` and `\stex_style_apply:`. These functions are documented on page 127.)

sfunction

`\stex_new_stylable_env:nnnnnnn`

```

516 \cs_new_protected:Nn \stex_new_stylable_env:nnnnnnn {
517   \exp_after:wN \newcommand \cs:w stexstyle#1 \cs_end:[3] []{
518     \__stex_styles_patch:nnnn{#1}{##1}{##2}{##3} % <- defined after \stex_if_html_backend
519   }
520   \exp_after:wN \newcommand \cs:w stexcss#1 \cs_end:[1] []{
521     \__stex_styles_css_patch:nnnn{#1}{##1}
522   }
523   \NewDocumentEnvironment{#7#1}{#2}{
524     \cs_set:Npn \stex_style_apply: {
525       \stex_apply_patch_begin:n{#1}
526     }
527     #3
528   }{
529     \stex_if_html_backend:F{
530       \cs_set:Npn \stex_style_apply: {
531         \stex_apply_patch_end:n{#1}
532       }
533     }
534     #4
535   }
536   \tl_set:cn {__stex_styles_style_#1_start:} { #5 }
537   \tl_set:cn {__stex_styles_style_#1_end:} { #6 }
538 }
539
540
541 \cs_new_protected:Nn \stex_apply_patch_begin:n {
542   \clist_if_empty:NTF \l_stex_key_style_clist {
543     \tl_clear:N \thisstyle
544     \use:c{__stex_styles_style_#1_start:}
545   }{
546     \clist_get:NN \l_stex_key_style_clist \thisstyle
547     \stex_debug:nn{styling}{dominant~style:~\thisstyle}
548     \tl_if_exist:cTF{__stex_styles_style_#1_\thisstyle _start:}{
549       \use:c{__stex_styles_style_#1_\thisstyle _start:}

```

```

550     }{
551       \use:c{__stex_styles_style_#1_start:}
552     }
553   }
554 }
555
556 \cs_new_protected:Nn \_stex_apply_patch_end:n {
557   \tl_if_empty:NTF \thisstyle {
558     \use:c{__stex_styles_style_#1_end:}
559   }{
560     \tl_if_exist:cTF{__stex_styles_style_#1\_thisstyle_end:}{
561       \use:c{__stex_styles_style_#1\_thisstyle_end:}
562     }{
563       \use:c{__stex_styles_style_#1_end:}
564     }
565   }
566 }

```

(End of definition for `\stex_new_stylable_env:nnnnnnn`. This function is documented on page 127.)

sfunction

26.8 Persisting Dependencies

```

567 <@@=stex_persist>

```

We check the environment variables:

```

568 \stex_get_env:Nn\__stex_persist_env_str{STEX_USESMS}
569 \str_if_empty:NF\__stex_persist_env_str{
570   \exp_args:No \str_if_eq:nnF \__stex_persist_env_str{false}{
571     \bool_set_true:N \c_stex_persist_mode_bool
572   }
573 }
574 \stex_get_env:Nn\__stex_persist_env_str{STEX_WRITESMS}
575 \str_if_empty:NF\__stex_persist_env_str{
576   \exp_args:No \str_if_eq:nnF \__stex_persist_env_str{false}{
577     \bool_set_true:N \c_stex_persist_write_mode_bool
578   }
579 }
580
581 \bool_if:NT \c_stex_persist_force_bool {
582   \bool_set_false:N \c_stex_persist_mode_bool
583 }
584
585 \iow_new:N \c__stex_persist_sms_iow

```

`\stex_persist:n` defined later; requires `\stex_if_html_backend:TF`

`\stex_persist:e`
`\loadsms` (End of definition for `\stex_persist:n` and `\loadsms`. These functions are documented on page 127.)

sfunction

Is called at the end of the .sty-file:

```

586
587 \cs_new_protected:Npn \loadsms #1 {
588   \stex_str_if_ends_with:nnTF{#1}{.sms}{
589     \__stex_persist_load_file:n{#1}

```

```

590   }{
591     \stex_str_if_ends_with:nnTF{#1}{.tex}{
592       \__stex_persist_load_file:n{#1}
593     }{
594       \__stex_persist_load_file:n{#1.sms}
595     }
596   }
597 }
598
599 \cs_new_protected:Nn \_stex_persist_read_now: {
600   \bool_if:NTF \c_stex_persist_mode_bool {
601     \bool_if:NTF \c_stex_persist_write_mode_bool
602     \__stex_persist_read_and_write:
603     {
604       \__stex_persist_load_file:n{\jobname.sms}
605     }
606   }{
607     \bool_if:NT \c_stex_persist_write_mode_bool \__stex_persist_write_only:
608   }
609 }
610
611 \cs_new_protected:Nn \__stex_persist_load_file:n {
612   \file_if_exist:nT{#1}{
613     \group_begin:
614     \cs_set:Npn \stex_persist:n ##1 {}
615     \cs_set:Npn \stex_persist:e ##1 {}
616     \stex_debug:nn{persist}{restoring~from~sms~file}
617     \catcode'\:=11\relax
618     \catcode'\_:=11\relax
619     \catcode'\ =10\relax
620     \cs:w @ @ input \cs_end:#1\relax
621   \group_end:
622 }
623 }
624
625 \cs_new_protected:Nn \__stex_persist_write_only: {
626   \iow_open:Nn \c__stex_persist_sms_iow {\jobname.sms}
627   \AtEndDocument{ \iow_close:N \c__stex_persist_sms_iow }
628 }
629
630 \cs_new_protected:Nn \__stex_persist_read_and_write: {
631   \file_if_exist:nTF{\jobname.sms}{
632     \ior_open:Nn \g_tmpa_ior {\jobname.sms}
633     \iow_open:Nn \g_tmpa_iow {\jobname.sms2}
634     \ior_str_map_inline:Nn \g_tmpa_ior {
635       \iow_now:Nn \g_tmpa_iow {##1}
636     }
637     \iow_close:N \g_tmpa_iow
638     \ior_close:N \g_tmpa_ior
639     \__stex_persist_write_only:
640     \ior_open:Nn \g_tmpa_ior {\jobname.sms2}
641     \ior_str_map_inline:Nn \g_tmpa_ior {
642       \iow_now:Nn \c__stex_persist_sms_iow {##1}
643     }

```

```

644 \ior_close:N \g_tmpa_ior
645 \__stex_persist_load_file:n{\jobname.sms2}
646 }\__stex_persist_write_only:
647 }

```

26.9 Auxiliary Methods

```

648 <@@=stex_aux>

```

`\_stex_do_deprecation:n`

```

649 \cs_new:Nn \_stex_do_deprecation:n {
650   \str_if_empty:NF \l_stex_key_deprecate_str {
651     \msg_warning:nxxx{stex}{warning/deprecated}{#1}{\l_stex_key_deprecate_str}
652   }
653 }

```

(End of definition for _stex_do_deprecation:n. This function is documented on page ??.)

`\_stex_do_id:`

```

654 \cs_new_protected:Nn \_stex_do_id: {
655   \stex_if_smsmode:F {
656     \str_if_empty:NF \l_stex_key_id_str {
657       \exp_args:No \stex_ref_new_doc_target:n \l_stex_key_id_str
658     }
659   }
660 }

```

(End of definition for _stex_do_id:. This function is documented on page ??.)

Chapter 27

HTML Output

```
661 <@=stex_annotate>
\stex@backend We determine and \input the backend config file:
662 \ifcstype if@rustex\endcstype\else
663   \expandafter\newif\cstype if@rustex\endcstype
664   \@rustexfalse
665 \fi
666
667 \stex_get_env:Nn\__stex_annotate_env_str{STEX_FORCE_PDF}
668 \exp_args:No \str_if_eq:nnTF \__stex_annotate_env_str {true} {
669   \def\stex@backend{pdflatex}
670 }{
671   \tl_if_exist:NF\stex@backend{
672     \if@rustex
673       \def\stex@backend{rustex}
674     \else
675       \cs_if_exist:NTF\HCode{
676         \def\stex@backend{tex4ht}
677       }{
678         \def\stex@backend{pdflatex}
679       }
680     \fi
681   }
682 }
683
684 \input{stex-backend-\stex@backend.cfg}

(End of definition for \stex@backend. This variable is documented on page 128.)

\stex_if_html_backend_p: is provided by the backend config file.
\stex_if_html_backend:TF (End of definition for \stex_if_html_backend:TF. This function is documented on page 128.)
sfunction

\ifstexhtml
685 \bool_new:N \l__stex_annotate_do_output_bool
686
687 \newif\ifstexhtml
688 \stex_if_html_backend:TF {
```

```

689 \stexhtmltrue
690 \bool_set_true:N \l__stex_annotate_do_output_bool
691 }{
692 \stexhtmlfalse
693 \bool_set_false:N \l__stex_annotate_do_output_bool
694 }

```

(End of definition for `\ifstexhtml`. This function is documented on page 128.)
sfunction

```

\stex_if_do_html_p:
\stex_if_do_html:TF
695 \prg_new_conditional:Nnn \stex_if_do_html: {p,T,F,TF} {
696 \bool_if:NTF \l__stex_annotate_do_output_bool
697 \prg_return_true: \prg_return_false:
698 }

```

(End of definition for `\stex_if_do_html:TF`. This function is documented on page 128.)
sfunction

```

\stex_suppress_html:n
699 \cs_new_protected:Nn \stex_suppress_html:n {
700 \stex_pseudogroup:nn{
701 \bool_set_false:N \l__stex_annotate_do_output_bool
702 #1
703 }{
704 \stex_if_do_html:T {
705 \bool_set_true:N \l__stex_annotate_do_output_bool
706 }
707 }
708 }

```

(End of definition for `\stex_suppress_html:n`. This function is documented on page 128.)
sfunction

`\stex_annotate:nn` is provided by the backend config file.

(End of definition for `\stex_annotate:nn`. This function is documented on page 128.)
sfunction

`stex_annotate_env (env.)` is provided by the backend config file.

`\stex_html_node:nnn` is provided by the backend config file.

(End of definition for `\stex_html_node:nnn`. This function is documented on page 128.)
sfunction

`stex_env_node (env.)` is provided by the backend config file.

`\stex_annotate_invisible:nn` is provided by the backend config file.

```

\stex_annotate_invisible:n

```

(End of definition for `\stex_annotate_invisible:nn` and `\stex_annotate_invisible:n`. These functions are documented on page 129.)
sfunction

`\STEXinvisible`

```
709 \cs_new_protected:Npn \STEXinvisible #1 {  
710   \stex_annotate_invisible:n { #1 }  
711 }
```

(End of definition for \STEXinvisible. This function is documented on page 129.)
sfunction

`\stex_css_link:n` is provided by the backend config file.

(End of definition for \stex_css_link:n. This function is documented on page 129.)
sfunction

`\stex_css_literal:n` is provided by the backend config file.

(End of definition for \stex_css_literal:n. This function is documented on page 129.)
sfunction

`\_stex_annotate_force_break:n` is provided by the backend config file.

(End of definition for _stex_annotate_force_break:n. This function is documented on page 129.)
sfunction

`\mmlintent`

```
\mmlarg 712 \stex_if_html_backend:TF {  
713   \cs_new_protected:Npn \mmlintent #1 #2 {  
714     \stex_annotate:nn{\mml:intent={#1}}{#2}  
715   }  
716   \cs_new_protected:Npn \mmlarg #1 #2 {  
717     \stex_annotate:nn{\mml:arg={#1}}{#2}  
718   }  
719 }{  
720   \cs_new_protected:Npn \mmlintent #1 #2 { #2 }  
721   \cs_new_protected:Npn \mmlarg #1 #2 { #2 }  
722 }
```

(End of definition for \mmlintent and \mmlarg. These functions are documented on page 129.)
sfunction

We can now define `\stex_persist:n`:

```
723 <@@=stex_persist>  
724 \bool_if:NTF \c_stex_persist_write_mode_bool {  
725   \stex_if_html_backend:TF{  
726     \cs_new:Npn \stex_persist:n #1 {}  
727     \cs_new:Npn \stex_persist:e #1 {}  
728   }{  
729     \cs_new_protected:Nn \stex_persist:n {  
730       \iow_now:Nn \c__stex_persist_sms_iow {#1}  
731     }  
732     \cs_generate_variant:Nn \stex_persist:n {e}  
733   }  
734 }{  
735   \cs_new:Npn \stex_persist:n #1 {}  
736   \cs_new:Npn \stex_persist:e #1 {}  
737 }
```


and style patching:

```

738 <@@=stex_styles>
739 \stex_if_html_backend:TF{
740   \cs_new_protected:Nn \stex_style_apply: {}
741   \cs_new_protected:Nn \__stex_styles_patch:nnnn {}
742   \stex_keys_define:nnnn{ csspatch }{
743     \str_clear:N \l_stex_keys_cls_id
744     \str_clear:N \l_stex_keys_counter_id
745     \str_clear:N \l_stex_keys_counter_parent
746   }{
747     counter      .str_set_x:N = \l_stex_keys_counter_id,
748     parent       .str_set_x:N = \l_stex_keys_counter_parent,
749     unknown      .code:n      = {
750       \exp_args:NNo \str_set:Nn \l_stex_keys_cls_id {\l_keys_key_tl}
751     }
752   }{}
753   \cs_new_protected:Npn \__stex_styles_css_patch:nnnn #1 #2 {
754     \stex_keys_set:nn{ csspatch }{ #2 }
755     \group_begin:
756     \catcode'\ =12\relax
757     \catcode'\^J=12\relax
758     \__stex_styles_patch_i:nnn {#1}
759   }
760
761   \seq_new:N \g__stex_styles_counters_seq
762
763   \cs_new:Nn \__stex_styles_patch_html_c: {
764     \stex_html_literal:n{
765       <span~data-ftml-counter="\l_stex_keys_counter_id"~style="display:none;"~
766       data-ftml-counter-parent="\l_stex_keys_counter_parent"
767       ></span>
768     }
769   }
770
771   \cs_new:Nn \__stex_styles_patch_html: {
772     \stex_html_literal:n{
773       <span~data-ftml-style="\l_stex_keys_cls_id"~style="display:none;"~
774       data-ftml-counter="\l_stex_keys_counter_id"
775       ></span>
776     }
777   }
778
779   \cs_new_protected:Nn \__stex_styles_patch_i:nnn {
780     \str_if_empty:NTF \l_stex_keys_cls_id {
781       \str_set:Nn \l_stex_keys_cls_id { #1 }
782     }{
783       \str_set:Ne \l_stex_keys_cls_id { #1-\l_stex_keys_cls_id }
784     }
785     \exp_args:No \str_case:nnF \l_stex_keys_counter_parent {
786       {}{}
787       {part}{\str_set:Nn \l_stex_keys_counter_parent { 0 }}
788       {chapter}{\str_set:Nn \l_stex_keys_counter_parent { 1 }}
789       {section}{\str_set:Nn \l_stex_keys_counter_parent { 2 }}
790       {subsection}{\str_set:Nn \l_stex_keys_counter_parent { 3 }}

```

```

791     {subsubsection}{\str_set:Nn \l_stex_keys_counter_parent { 4 }}
792     {paragraph}{\str_set:Nn \l_stex_keys_counter_parent { 5 }}
793     {subparagraph}{\str_set:Nn \l_stex_keys_counter_parent { 6 }}
794   }{
795     \msg_error:nnx{stex}{error/notasection}\l_stex_keys_counter_parent
796   }
797   \str_set:Ne \l__stex_styles_first_str { &~.ftml-title-paragraph~ { display:inline-block
798   \str_if_empty:NF \l_stex_keys_counter_id {
799     %\str_put_left:Ne \l__stex_styles_first_str { counter-increment:~ ftml-\l_stex_keys_c
800     \exp_args:NNo \seq_if_in:NnF \g__stex_styles_counters_seq \l_stex_keys_counter_id {
801       \seq_gpush:No \g__stex_styles_counters_seq \l_stex_keys_counter_id
802       \cs_if_eq:ccTF{@onlypreamble}{@notprerr}{
803         \__stex_styles_patch_html_c:
804         % in body
805       }{
806         \exp_args:Ne \AtBeginDocument \__stex_styles_patch_html_c:
807         % in preamble
808       }
809     }
810     \cs_if_eq:ccTF{@onlypreamble}{@notprerr}{
811       \__stex_styles_patch_html:
812       % in body
813     }{
814       \exp_args:Ne \AtBeginDocument \__stex_styles_patch_html:
815       % in preamble
816     }
817   }
818   \exp_args:Ne \stex_css_literal:n { .ftml-\l_stex_keys_cls_id { \l__stex_styles_first_st
819
820   % TODO export counter reset somehow
821
822   \group_end:
823 }
824 }{
825   \cs_new_protected:Nn \__stex_styles_patch:nnnn {
826     \str_if_empty:NTF {#2}{
827       \tl_set:cn{__stex_styles_style_#1_start:}{#3}
828       \tl_set:cn{__stex_styles_style_#1_end:}{#4}
829     }{
830       \tl_set:cn{__stex_styles_style_#1_#2_start:}{#3}
831       \tl_set:cn{__stex_styles_style_#1_#2_end:}{#4}
832     }
833   }
834   \cs_new_protected:Nn \__stex_styles_css_patch:nnnn {}
835 }

```

Chapter 28

Math Archives

```
\c_stex_mathhub_files
\mathhub
836 <@@=stex_mathhub>
837 \str_if_empty:NTF\mathhub{
838   \stex_get_env:Nn \l__stex_mathhub_str {MATHHUB}
839   \str_if_empty:NTF \l__stex_mathhub_str {
840     \ior_open:NnTF \g_tmpa_ior{\stex_file_use:N \c_stex_home_file/.stex/mathhub.path}{
841       \group_begin:
842         \escapechar=-1\catcode'\=12
843         \ior_str_get:NN \g_tmpa_ior \l__stex_mathhub_str
844         \str_gset_eq:NN \l__stex_mathhub_str \l__stex_mathhub_str
845       \group_end:
846       \ior_close:N \g_tmpa_ior
847       \stex_debug:nn{mathhub}{MathHub-directory~determined~from~home~directory}
848     }{
849       \str_clear:N \l__stex_mathhub_str
850     }
851   }{
852     \stex_debug:nn{mathhub}{MathHub-directory~determined~from~environment~variable}
853   }
854 }{
855   \str_set_eq:NN \l__stex_mathhub_str \mathhub
856 }
857
858 \str_if_empty:NTF \l__stex_mathhub_str {
859   \msg_warning:nn{stex}{warning/nomathhub}
860   \stex_file_set:Ne \l_tmpa_seq {\stex_file_use:N \c_stex_home_file \tl_to_str:n{/MathHub}}
861   \seq_clear:N \c_stex_mathhub_files
862   \seq_push:Ne \c_stex_mathhub_files {\stex_file_use:N \l_tmpa_seq}
863 }{
864   \seq_clear:N \c_stex_mathhub_files
865   \seq_set_split:NnV \l_tmpa_seq , \l__stex_mathhub_str
866   \seq_map_inline:Nn \l_tmpa_seq {
867     \stex_file_resolve:Nn \l__stex_mathhub_str {#1}
868     \stex_if_file_absolute:NF \l__stex_mathhub_str {
869       \stex_file_resolve:Ne \l__stex_mathhub_str {
870         \stex_file_use:N \c_stex_pwd_file / #1
871       }

```

```

872     }
873     \seq_put_right:Ne \c_stex_mathhub_files {
874         \stex_file_use:N \l__stex_mathhub_str
875     }
876 }
877 }
878
879 \exp_args:NNe \str_set:Nn \mathhub {\seq_use:Nn \c_stex_mathhub_files ,}
880 \stex_debug:nn{mathhub}{MATHHUBS:~\mathhub}

```

(End of definition for `\c_stex_mathhub_files` and `\mathhub`. These variables are documented on page 132.)

`\stex_archive_id:N`

```

881 \cs_new:Npn \stex_archive_id:N {
882     \exp_after:wN \use_i:nnn
883 }

```

(End of definition for `\stex_archive_id:N`. This function is documented on page 132.)

sfunction

`\stex_archive_base:N`

`\stex_archive_base:n`

```

884 \cs_new:Npn \stex_archive_base:N {
885     \exp_after:wN \use_ii:nnn
886 }
887
888 \cs_new:Nn \stex_archive_base:n {
889     \exp_args:NNe \use:nn \use_ii:nnn { \use:c {c_stex_mathhub_#1_archive} }
890 }

```

(End of definition for `\stex_archive_base:N` and `\stex_archive_base:n`. These functions are documented on page 132.)

sfunction

`\stex_archive_path:N`

`\stex_archive_path:n`

```

891 \cs_new:Npn \stex_archive_path:N {
892     \exp_after:wN \use_iii:nnn
893 }
894
895 \cs_new:Nn \stex_archive_path:n {
896     \exp_args:NNe \use:nn \use_iii:nnn { \use:c {c_stex_mathhub_#1_archive} }
897 }

```

(End of definition for `\stex_archive_path:N` and `\stex_archive_path:n`. These functions are documented on page 132.)

sfunction

`\stex_in_archive:nn`

```

898 \cs_new_protected:Nn \stex_in_archive:nn {
899     \stex_pseudogroup:nn{
900         \cs_set:Npn \l__stex_mathhub_cs ##1 {#2}
901         \tl_if_empty:nTF{#1}{
902             \tl_if_exist:NTF \l_stex_current_archive {
903                 \exp_args:Ne \l__stex_mathhub_cs {\stex_archive_id:N \l_stex_current_archive }
904             }{

```

```

905     \l__stex_mathhub_cs {}
906   }
907   }{
908     \__stex_mathhub_set_current:n{#1}
909     \l__stex_mathhub_cs {#1}
910   }
911   }{
912     \stex_pseudogroup_restore:N \l_stex_current_archive
913     \cs_set:Npn \exp_not:N \l__stex_mathhub_cs ##1 {
914       \exp_args:No \exp_not:n {\l__stex_mathhub_cs {##1}}
915     }
916   }
917 }
918
919 \cs_set:Npn \l__stex_mathhub_cs #1 {}

```

Auxiliary macros for loading archives:

```

920 \cs_new_protected:Nn \__stex_mathhub_set_current:n {
921   \__stex_mathhub_require:n { #1 }
922   \stex_debug:nn{mathhub}{switching~to~archive~#1}
923   \tl_set_eq:Nc \l_stex_current_archive {
924     c_stex_mathhub_#1_archive
925   }
926 }
927
928 \cs_new_protected:Nn \_stex_set_meta_archive: {
929   \prop_if_exist:cF { c_stex_mathhub_FTML/meta_archive } {
930     \seq_if_empty:NF \c_stex_mathhub_files {
931       \stex_debug:nn{mathhub}{Opening~archive:~FTML/meta}
932       \__stex_mathhub_do_manifest:nn { FTML/meta }{
933         \tl_gset:ce {c_stex_mathhub_FTML/meta_archive}{
934           {\tl_to_str:n{FTML/meta}}
935           {\tl_to_str:n{http://mathhub.info}}
936         }
937       }
938     }
939   }
940 }
941
942 }
943
944 \cs_new_protected:Nn \__stex_mathhub_require:n {
945   \prop_if_exist:cF { c_stex_mathhub_#1_archive } {
946     \seq_if_empty:NTF \c_stex_mathhub_files {
947       \msg_fatal:nn{stex}{warning/nomathhub}
948     }{
949       \stex_debug:nn{mathhub}{Opening~archive:~#1}
950       \__stex_mathhub_do_manifest:nn { #1 }{
951         \msg_fatal:nnee{stex}{error/noarchive}
952         {#1}{\seq_use:Nn \c_stex_mathhub_files ,}
953       }
954     }
955   }
956 }

```

Attempts to find the archive's manifest file:

```

957 \cs_new_protected:Nn \__stex_mathhub_do_manifest:nn {
958   \seq_map_inline:Nn \c_stex_mathhub_files {
959     \__stex_mathhub_find_manifest:nn {##1}{#1}
960     \str_if_empty:NF \l__stex_mathhub_manifest_str \seq_map_break:
961   }
962   %\exp_args:Ne \__stex_mathhub_find_manifest:n {\stex_file_use:N \c_stex_mathhub_file / #1}
963   \str_if_empty:NTF \l__stex_mathhub_manifest_str { #2 }{
964     \__stex_mathhub_parse_manifest:n {#1}
965   }
966 }
967
968 \cs_new_protected:Nn \__stex_mathhub_find_manifest:nn {
969   \str_clear:N \l__stex_mathhub_manifest_str
970   \seq_set_split:Nnn \l_tmpa_seq / { #1 }
971   \seq_set_split:Nnn \l__stex_mathhub_seq / {#1 / #2}
972   \bool_set_true:N \l__stex_mathhub_bool
973   \bool_while_do:Nn \l__stex_mathhub_bool {
974     \tl_if_eq:NNTF \l__stex_mathhub_seq \l_tmpa_seq {
975       \bool_set_false:N \l__stex_mathhub_bool
976     }{
977       \__stex_mathhub_check_manifest:
978       \bool_if:NT \l__stex_mathhub_bool {
979         \seq_pop_right:NN \l__stex_mathhub_seq \l__stex_mathhub_tl
980       }
981     }
982   }
983 }
984
985 \cs_new_protected:Nn \__stex_mathhub_check_manifest: {
986   \__stex_mathhub_check_manifest:n {MANIFEST.MF}
987   \bool_if:NT \l__stex_mathhub_bool {
988     \__stex_mathhub_check_manifest:n {META-INF/MANIFEST.MF}
989     \bool_if:NT \l__stex_mathhub_bool {
990       \__stex_mathhub_check_manifest:n {meta-inf/MANIFEST.MF}
991     }
992   }
993 }
994
995 \cs_new_protected:Nn \__stex_mathhub_check_manifest:n {
996   \stex_debug:nn{mathhub}{Checking~\stex_file_use:N \l__stex_mathhub_seq / #1}
997   \file_if_exist:nT {\stex_file_use:N \l__stex_mathhub_seq / #1} {
998     \bool_set_false:N \l__stex_mathhub_bool
999     \str_set:Ne \l__stex_mathhub_manifest_str {\stex_file_use:N \l__stex_mathhub_seq / #1}
1000   }
1001 }

```

Parses the archive's manifest file:

```

1002 \ior_new:N \c__stex_mathhub_manifest_ior
1003
1004 \str_set:Nn \l_tmpa_str {https://}
1005
1006 \exp_after:wN \cs_new:Npn \exp_after:wN \__stex_mathhub_replace_https:w \l_tmpa_str #1 \__s
1007   http:// #1

```

```

1008 }
1009
1010 \cs_new_protected:Nn \__stex_mathhub_parse_manifest:n {
1011   \ior_open:Nn \c__stex_mathhub_manifest_ior \l__stex_mathhub_manifest_str
1012   \str_set:Ne \l__stex_mathhub_file_str {\stex_file_use:N \l__stex_mathhub_seq}
1013   \str_set:Nn \l__stex_mathhub_id_str {#1}
1014   \str_set:Nn \l__stex_mathhub_base_str {http://mathhub.info}
1015
1016   \ior_map_inline:Nn \c__stex_mathhub_manifest_ior {
1017     \exp_args:NNNo \exp_args:NNNx
1018     \seq_set_split:Nnn \l__stex_mathhub_seq \c_colon_str {\tl_to_str:n{##1}}
1019     \seq_pop_left:NNT \l__stex_mathhub_seq \l__stex_mathhub_key {
1020       \exp_args:NNo \str_set:Nn \l__stex_mathhub_key \l__stex_mathhub_key
1021       \str_set:Ne \l__stex_mathhub_val {\seq_use:Nn \l__stex_mathhub_seq :}
1022       \str_case:Nn \l__stex_mathhub_key {
1023         {id} { \str_set_eq:NN \l__stex_mathhub_id_str \l__stex_mathhub_val }
1024         {url-base} { \str_set_eq:NN \l__stex_mathhub_base_str \l__stex_mathhub_val }
1025       }
1026     }
1027   }
1028   \ior_close:N \c__stex_mathhub_manifest_ior
1029   \exp_args:No \stex_str_if_starts_with:nnT \l__stex_mathhub_base_str {https://} {
1030     \str_set:Ne \l__stex_mathhub_base_str { \exp_after:wN \__stex_mathhub_replace_https:w \
1031   }
1032   \tl_gset:ce { c_stex_mathhub_#1_archive } { {\l__stex_mathhub_id_str} {\l__stex_mathhub_b
1033   \stex_debug:nn{mathhub}{Result:~\use:c{c_stex_mathhub_#1_archive}}
1034   \str_if_eq:nnF {#1}{main}{
1035     \stex_persist:e {
1036       \__stex_mathhub_restore:nn{\l__stex_mathhub_id_str}{\l__stex_mathhub_base_str}
1037     }
1038   }
1039 }

```

The `.sms` file persisting content should not store the file path of the archive, since that depends on the directory layout of the system that generated it. We therefore, when restoring archive macros, check if we find the archive on the local system, and if not, leave the directory path empty. This will throw an error iff some functionality at some point *requires* the archive to actually exist locally.

```

1040 \cs_set_protected:Nn \__stex_mathhub_restore:nn {
1041   \__stex_mathhub_do_manifest:nn { #1 }{}
1042   \tl_if_exist:cF { c_stex_mathhub_#1_archive }{
1043     \tl_set:ce{ c_stex_mathhub_#1_archive }{
1044       {\tl_to_str:n{#1}}
1045       {\tl_to_str:n{#2}}
1046     }
1047   }
1048 }
1049 }
1050

```

(End of definition for `\stex_in_archive:nn`. This function is documented on page 132.)

sfunction

```

\c_stex_no_archive_str
\c_stex_no_archive

```

```

1051 \str_const:Nn \c_stex_no_archive_str { no/archive }
1052
1053 \tl_const:Ne \c_stex_no_archive_uri {
1054   {\c_stex_no_archive_str}
1055   {\tl_to_str:n{http://unknown.source}}
1056   {}
1057 }
1058
1059 \tl_set_eq:cN {c_stex_mathhub_ no/archive _ archive}\c_stex_no_archive_uri

```

(End of definition for `\c_stex_no_archive_str` and `\c_stex_no_archive`. These variables are documented on page 133.)

```

\c_stex_main_archive
\l_stex_current_archive

```

```

1060 \seq_map_inline:Nn \c_stex_mathhub_files {
1061   \seq_set_split:Nnn \l_tmpa_seq / {#1}
1062   \stex_if_file_starts_with:NNT \c_stex_pwd_file \l_tmpa_seq {
1063     \seq_set_eq:NN \l_tmpb_seq \c_stex_pwd_file
1064     \seq_map_inline:Nn \l_tmpa_seq { \seq_pop_left:NN \l_tmpb_seq \l_tmpa_tl }
1065     \exp_args:Nne \_stex_mathhub_find_manifest:nn {#1} { \stex_file_use:N \l_tmpb_seq }
1066     \str_if_empty:NTF \l__stex_mathhub_manifest_str {
1067       \stex_debug:nn{mathhub}{Not~currently~in~a~MathHub~archive}
1068     }{
1069       \_stex_mathhub_parse_manifest:n { main }
1070       \tl_set_eq:NN \c_stex_main_archive \c_stex_mathhub_main_archive
1071       \cs_undefine:N \c_stex_mathhub_main_archive
1072       \tl_set_eq:cN { c_stex_mathhub_ \stex_archive_id:N \c_stex_main_archive _archive }
1073       \c_stex_main_archive
1074       \prop_set_eq:NN \l_stex_current_archive \c_stex_main_archive
1075       \stex_debug:nn{mathhub}{Current~archive:~
1076         \stex_archive_id:N \c_stex_main_archive
1077       }
1078       \stex_persist:e {
1079         \_stex_mathhub_restore:nn{\stex_archive_id:N \c_stex_main_archive}{\stex_archive_b
1080         \tl_gset_eq:Nc \exp_not:N \c_stex_main_archive {c_stex_mathhub_ \stex_archive_id:N
1081         \tl_gset_eq:NN \exp_not:N \l_stex_current_archive \exp_not:N \c_stex_main_archive
1082       }
1083       %\bool_if:NT \c_stex_persist_write_mode_bool {
1084       %   \tl_put_right:Ne \_stex_persist_read_now: {
1085       %     \stex_persist:n {c_stex_mathhub_ \stex_archive_id:N \c_stex_main_archive _archi
1086       %       \tl_gset_eq:cN{c_stex_mathhub_ \stex_archive_id:N \c_stex_main_archive _manife
1087       %       \tl_gset_eq:NN \exp_not:N \l_stex_current_archive \exp_not:N \c_stex_main_archi
1088       %     }
1089       %   }
1090       %}
1091   }
1092   \seq_map_break:
1093 }
1094 }
1095 \tl_if_exist:NF \c_stex_main_archive {
1096   \stex_debug:nn{mathhub}{Not~currently~in~a~MathHub~directory}
1097 }

```

(End of definition for `\c_stex_main_archive` and `\l_stex_current_archive`. These variables are documented on page 133.)


```

\stex_source_path:n
\stex_source_path:nn
1098 \cs_new:Nn \stex_source_path:n {
1099   \tl_if_exist:NTF \l_stex_current_archive {
1100     \stex_archive_path:N \l_stex_current_archive / source \tl_if_empty:nF{#1}{/ #1}
1101   }{
1102     \stex_file_use:N \g_stex_current_file / .. \tl_if_empty:nF{#1}{/ #1}
1103   }
1104 }
1105 \cs_new:Nn \stex_source_path:nn {
1106   \tl_if_empty:nTF{#1}{
1107     \stex_source_path:n {#2}
1108   }{
1109     \tl_if_exist:cTF {c_stex_mathhub_ #1 _archive } {
1110       \stex_archive_path:n {#1} / source \tl_if_empty:nF{#2}{/ #2}
1111     }{
1112       \stex_file_use:N \g_stex_current_file / .. \tl_if_empty:nF{#2}{/ #2}
1113     }
1114   }
1115 }

```

(End of definition for \stex_source_path:n and \stex_source_path:nn. These functions are documented on page 133.)

sfunction

Chapter 29

URIs

```
1116 <@@=stex_uris>
```

```
\stex_use_archive_uri:n
```

```
1117 \cs_new:Nc \stex_use_archive_uri:n {  
1118   \exp_not:N \stex_archive_base:n{#1} \tl_to_str:n{?a=} #1  
1119 }
```

(End of definition for \stex_use_archive_uri:n. This function is documented on page 134.)
sfunction

29.1 Document URIs

```
\stex_use_document_uri:N
```

```
\stex_use_document_uri:n
```

```
1120 \cs_new:Nc \stex_use_document_uri:N {  
1121   \exp_after:wN \__stex_uris_doc:nnnnn #1  
1122 }  
1123 \cs_new:Nc \stex_use_document_uri:n {  
1124   \__stex_uris_doc:nnnnn #1  
1125 }  
1126 \cs_new:Nc \__stex_uris_doc:nnnnn {  
1127   \exp_not:N \stex_archive_base:n{#1} \tl_to_str:n{?a=} #1  
1128   \exp_not:N \tl_if_empty:nF{#2}{  
1129     \c_ampersand_str\tl_to_str:n{p=}#2  
1130   }  
1131   \c_ampersand_str\tl_to_str:n{d=}#3  
1132   \c_ampersand_str\tl_to_str:n{l=}#4  
1133   \exp_not:N \tl_if_empty:nF{#5}{  
1134     \c_ampersand_str\tl_to_str:n{e=}#5  
1135   }  
1136 }
```

(End of definition for \stex_use_document_uri:N and \stex_use_document_uri:n. These functions are documented on page 134.)
sfunction

```
\stex_document_uri_archive:N
```

```
1137 \cs_new:Nc \stex_document_uri_archive:N {  
1138   \exp_after:wN \use_i:nnnnn #1
```

```

1139 }
(End of definition for \stex_document_uri_archive:N. This function is documented on page 134.)
sfunction

```

`\stex_document_uri_path:N`

```

1140 \cs_new:Nn \stex_document_uri_path:N {
1141   \exp_after:wN \use_ii:nnnnn #1
1142 }
(End of definition for \stex_document_uri_path:N. This function is documented on page 135.)
sfunction

```

`\stex_document_uri_name:N`

```

1143 \cs_new:Nn \stex_document_uri_name:N {
1144   \exp_after:wN \use_iii:nnnnn #1
1145 }
(End of definition for \stex_document_uri_name:N. This function is documented on page 135.)
sfunction

```

`\stex_document_uri_language:N`

```

1146 \cs_new:Nn \stex_document_uri_language:N {
1147   \exp_after:wN \use_iv:nnnnn #1
1148 }
(End of definition for \stex_document_uri_language:N. This function is documented on page 135.)
sfunction

```

`\stex_document_uri_element:N`

```

1149 \cs_new:Nn \stex_document_uri_element:N {
1150   \exp_after:wN \use_v:nnnnn #1
1151 }
(End of definition for \stex_document_uri_element:N. This function is documented on page 135.)
sfunction

```

`\stex_document_uri_with_language:Nn`

```

1152 \cs_new:Npn \stex_document_uri_with_language:Nn {
1153   \exp_after:wN \__stex_uris_with_language:nnnnnn
1154 }
1155 \cs_new:Nn \__stex_uris_with_language:nnnnnn {
1156   {#1}{#2}{#3}{#6}{}
1157 }
(End of definition for \stex_document_uri_with_language:Nn. This function is documented on page 135.)
sfunction

```

`\stex_new_document_element_uri:n`

```

1158 \cs_new:Npn \stex_new_document_element_uri:n {
1159   \exp_after:wN \__stex_uris_with_elem:nnnnnn \l_stex_current_document_uri
1160 }
1161 \cs_new:Nn \__stex_uris_with_elem:nnnnnn {
1162   {#1}{#2}{#3}{#4}{ \tl_if_empty:nTF{#5}{#6}{#5/#6}}
1163 }

```

(End of definition for \stex_new_document_element_uri:n. This function is documented on page 135.)

sfunction

\stex_document_uri_from_archive_file:Nn

```

1164 \cs_new_protected:Nn \stex_document_uri_from_archive_file:Nn {
1165   \stex_file_set:Nn \l__stex_uris_file {#2}
1166   \stex_debug:nn{URI}{relative~path:~\stex_file_use:N \l__stex_uris_file}
1167   \__stex_uris_doc_from_archive_file:NN #1 \l__stex_uris_file
1168 }
1169
1170 \cs_new_protected:Nn \__stex_uris_doc_from_archive_file:NN {
1171   \__stex_uris_split_file:N #2
1172   \prop_if_exist:NTF \l_stex_current_archive {
1173     \str_set:Ne \l__stex_uris_archive {
1174       \stex_archive_id:N \l_stex_current_archive
1175     }
1176   }{
1177     \str_set_eq:NN \l__stex_uris_archive \c_stex_no_archive_str
1178   }
1179   \tl_set:Ne #1 {
1180     { \l__stex_uris_archive }
1181     { \stex_file_use:N \l__stex_uris_path_seq }
1182     { \l__stex_uris_name_str }
1183     { \l__stex_uris_lang_str }
1184   }{}
1185 }
1186 }
1187
1188 \cs_new_protected:Nn \__stex_uris_split_file:N {
1189   \seq_set_eq:NN \l__stex_uris_path_seq #1
1190   \seq_pop_right:NN \l__stex_uris_path_seq \l__stex_uris_name_str
1191   \seq_set_split:NnV \l_tmpa_seq . \l__stex_uris_name_str
1192   \seq_pop_right:NN \l_tmpa_seq \l__stex_uris_lang_str % .tex?
1193   \seq_if_empty:NTF \l_tmpa_seq {
1194     \str_set_eq:NN \l__stex_uris_name_str \l__stex_uris_lang_str
1195     \str_set_eq:NN \l__stex_uris_lang_str \l_stex_current_language_str
1196   }\__stex_uris_split_file_i:
1197 }
1198
1199 \cs_new_protected:Nn \__stex_uris_split_file_i: {
1200   \cs_if_eq:NNTF \l__stex_uris_lang_str \q_no_value {
1201     \str_set_eq:NN \l__stex_uris_lang_str \l_stex_current_language_str
1202   }{
1203     \prop_if_in:NoF \c_stex_languages_prop \l__stex_uris_lang_str {
1204       \seq_pop_right:NN \l_tmpa_seq \l__stex_uris_lang_str % actual language maybe?
1205       \cs_if_eq:NNTF \l__stex_uris_lang_str \q_no_value {
1206         \str_set_eq:NN \l__stex_uris_lang_str \l_stex_current_language_str
1207       }{
1208         \prop_if_in:NoF \c_stex_languages_prop \l__stex_uris_lang_str {
1209           \seq_put_right:No \l_tmpa_seq \l__stex_uris_lang_str
1210           \str_set:Nn \l__stex_uris_lang_str {en}
1211         }
1212       }
1213     }

```

```

1214 }
1215 \str_set:Ne \l__stex_uris_name_str {\seq_use:Nn \l_tmpa_seq .}
1216 }
1217
1218 \cs_new_protected:Nn \__stex_uris_split_file_ii: {
1219
1220 }

```

(End of definition for `\stex_document_uri_from_archive_file:Nn`. This function is documented on page 135.)

sfunction

```

\c_stex_main_document_uri
\l_stex_current_document_uri
1221 \tl_if_exist:NTF \l_stex_current_archive {
1222   \str_set:Ne \l_tmpa_str { \stex_archive_path:N \l_stex_current_archive }
1223   \seq_set_split:NnV \l_tmpa_seq / \l_tmpa_str
1224   \seq_set_eq:NN \l_tmpb_seq \c_stex_main_file
1225   \seq_map_inline:Nn \l_tmpa_seq {
1226     \seq_pop_left:NN \l_tmpb_seq \l_tmpa_tl
1227   }
1228   \seq_pop_left:NN \l_tmpb_seq \l_tmpa_tl
1229   \__stex_uris_doc_from_archive_file:NN \l_stex_current_document_uri \l_tmpb_seq
1230 }{
1231   \__stex_uris_doc_from_archive_file:NN \l_stex_current_document_uri \c_stex_main_file
1232 }
1233
1234 \tl_set_eq:NN \c_stex_main_document_uri \l_stex_current_document_uri
1235 \str_set:Ne \l_stex_current_language_str { \stex_document_uri_language:N \l_stex_current_do
1236
1237 \stex_debug:nn{URIs}{Current~document::~ \stex_use_document_uri:N \c_stex_main_document_uri}

```

(End of definition for `\c_stex_main_document_uri` and `\l_stex_current_document_uri`. These variables are documented on page 135.)

29.2 Module URIs

```

\stex_use_module_uri:N
\stex_use_module_uri:n
1238 \cs_new:Nn \stex_use_module_uri:N {
1239   \exp_after:wN \__stex_uris_module:nnn #1
1240 }
1241 \cs_new:Nn \stex_use_module_uri:n {
1242   \__stex_uris_module:nnn #1
1243 }
1244 \cs_new:Ne \__stex_uris_module:nnn {
1245   \exp_not:N \stex_archive_base:n{#1} \tl_to_str:n{?a=} #1
1246   \exp_not:N \tl_if_empty:nF{#2}{
1247     \c_ampersand_str\tl_to_str:n{p=}#2
1248   }
1249   \c_ampersand_str\tl_to_str:n{m=}#3
1250 }

```

(End of definition for `\stex_use_module_uri:N` and `\stex_use_module_uri:n`. These functions are documented on page 135.)

sfunction

`\stex_module_uri_archive:N`

```
1251 \cs_new:Nn \stex_module_uri_archive:N {
1252   \exp_after:wN \use_i:nnn #1
1253 }
```

(End of definition for \stex_module_uri_archive:N. This function is documented on page 135.)
sfunction

`\stex_module_uri_path:N`

`\stex_module_uri_path:n`

```
1254 \cs_new:Nn \stex_module_uri_path:N {
1255   \exp_after:wN \use_ii:nnn #1
1256 }
1257 \cs_new:Nn \stex_module_uri_path:n {
1258   \use_ii:nnn #1
1259 }
```

(End of definition for \stex_module_uri_path:N and \stex_module_uri_path:n. These functions are documented on page 135.)
sfunction

`\stex_module_uri_name:N`

`\stex_module_uri_name:n`

```
1260 \cs_new:Nn \stex_module_uri_name:N {
1261   \exp_after:wN \use_iii:nnn #1
1262 }
1263
1264 \cs_new:Nn \stex_module_uri_name:n {
1265   \use_iii:nnn #1
1266 }
```

(End of definition for \stex_module_uri_name:N and \stex_module_uri_name:n. These functions are documented on page 136.)
sfunction

`\stex_module_uri_split_name:NNN`

`\stex_module_uri_split_name:NNn`

```
1267 \cs_new_protected:Npn \stex_module_uri_split_name:NNN #1 {
1268   \exp_after:wN \__stex_uris_parent:NNnnn \exp_after:wN #1 \exp_after:wN
1269 }
1270
1271 \cs_new_protected:Nn \stex_module_uri_split_name:NNn {
1272   \__stex_uris_parent:NNnnn #1 #2 #3
1273 }
1274
1275 \cs_new_protected:Nn \__stex_uris_parent:NNnnn {
1276   \seq_set_split:Nnn #1 / {#5}
1277   \seq_pop_right:NN #1 #2
1278   \seq_if_empty:NTF #1 {
1279     \tl_set:Ne #1 { {#3} {#4} {#3}}
1280     \tl_clear:N #2
1281   }{
1282     \tl_set:Ne #1 { {#3} {#4} {\seq_use:Nn #1 /} }
1283   }
1284 }
```

(End of definition for `\stex_module_uri_split_name:NNN` and `\stex_module_uri_split_name:NNn`. These functions are documented on page 136.)

sfunction

`\stex_module_uri_as_qm:n`

```
1285 \cs_new:Nn \stex_module_uri_as_qm:n {
1286   \__stex_uris_as_qm:nnn #1
1287 }
1288 \cs_new:Nn \__stex_uris_as_qm:nnn {
1289   #1 / #2 ? #3
1290 }
```

(End of definition for `\stex_module_uri_as_qm:n`. This function is documented on page 136.)

sfunction

`\stex_new_module_uri:n`

```
1291 \cs_new:Npn \stex_new_module_uri:n {
1292   \stex_if_in_module:TF {
1293     \exp_after:wN \__stex_uris_new_nested_mod:nnnn \l_stex_current_module_uri
1294   }{
1295     \exp_after:wN \__stex_uris_new_mod:nnnnnn \l_stex_current_document_uri
1296   }
1297 }
1298
1299 \cs_new:Nn \__stex_uris_new_mod:nnnnnn {
1300   {#1}
1301   \str_if_eq:nnTF{#3}{#6}{
1302     {#2}{#3}
1303   }{
1304     {#2 \tl_if_empty:nF{#2}/ #3}{ \tl_to_str:n{ #6 } }
1305   }
1306 }
1307 \cs_new:Nn \__stex_uris_new_nested_mod:nnnn {
1308   {#1}
1309   {#2}
1310   {#3 / \tl_to_str:n{#4}}
1311 }
```

(End of definition for `\stex_new_module_uri:n`. This function is documented on page 136.)

sfunction

`\stex_uri_from_pair:Nnn`

```
1312 \bool_new:N \l_stex_relative_import_bool
1313 \cs_new_protected:Nn \stex_uri_from_pair:Nnn {
1314   \bool_set_false:N \l_stex_relative_import_bool
1315   \stex_debug:nn{uri}{resolving~<#2,#3>~in~\stex_use_document_uri:N \l_stex_current_document_uri}
1316   \seq_set_split:Nne \l_tmpa_seq ? { \tl_to_str:n {#3} }
1317   \seq_pop_right:NN \l_tmpa_seq \l__stex_uris_name_str
1318   \str_clear:N \l__stex_uris_path_str
1319   \seq_if_empty:NF \l_tmpa_seq {
1320     \seq_pop_right:NN \l_tmpa_seq \l__stex_uris_path_str
1321   }
1322   \tl_if_empty:nTF { #2 }{
1323     \str_if_empty:NTF \l__stex_uris_path_str
```

```

1324     \__stex_uris_maybe_relative:N \__stex_uris_absolute:N
1325     #1
1326   }{
1327     \__stex_uris_pair_in_archive:Nn #1 { #2 }
1328   }
1329 }
1330 %\cs_generate_variant:Nn \stex_uri_from_pair:Nnn {Nee}
1331
1332 \cs_new_protected:Nn \__stex_uris_maybe_relative:N {
1333   \tl_set:Nc \l_tmpb_tl {
1334     \stex_document_uri_path:N \l_stex_current_document_uri
1335   }
1336   \tl_set:Nc \l_tmpa_tl {
1337     {
1338       \stex_document_uri_archive:N \l_stex_current_document_uri
1339     }
1340     {
1341       \l_tmpb_tl
1342       \exp_args:NNo \exp_args:Nne \str_if_eq:nnF \l__stex_uris_name_str {
1343         \stex_document_uri_name:N \l_stex_current_document_uri
1344       }{
1345         \str_if_empty:NF \l_tmpb_tl / \stex_document_uri_name:N \l_stex_current_document_uri
1346       }
1347     }
1348     { \l__stex_uris_name_str }
1349   }
1350   \stex_if_module_exists:NTF \l_tmpa_tl {
1351     \tl_set_eq:NN #1 \l_tmpa_tl
1352     \bool_set_true:N \l_stex_relative_import_bool
1353     \stex_debug:nn{uri}{returning~relative~\stex_use_module_uri:N #1}
1354   }{
1355     \__stex_uris_absolute:N #1
1356   }
1357 }
1358
1359 \cs_new_protected:Nn \__stex_uris_absolute:N {
1360   \tl_if_exist:NTF \l_stex_current_archive {
1361     \tl_set:Nc #1 {
1362       { \stex_archive_id:N \l_stex_current_archive }
1363       { \l__stex_uris_path_str }
1364       { \l__stex_uris_name_str }
1365     }
1366     \stex_debug:nn{uri}{returning~\stex_use_module_uri:N #1}
1367   }{
1368     \__stex_uris_pair_no_archive:N #1
1369   }
1370 }
1371
1372 \cs_new_protected:Nn \__stex_uris_pair_no_archive:N {
1373   \stex_file_resolve:Nc \l_tmpa_seq {
1374     \stex_file_use:N \g_stex_current_file / .. / \l__stex_uris_path_str
1375   }
1376   \tl_set:Nc #1 {
1377     { \c_stex_no_archive_str }

```



```

1378     { \stex_file_use:N \l_tmpa_seq }
1379     { \l__stex_uris_name_str }
1380   }
1381   \stex_debug:nn{uri}{returning~\stex_use_module_uri:N #1}
1382 }
1383
1384 \cs_new_protected:Nn \__stex_uris_pair_in_archive:Nn {
1385   \tl_set:Nc #1 {
1386     { \tl_to_str:n{ #2 } }
1387     { \l__stex_uris_path_str }
1388     { \l__stex_uris_name_str }
1389   }
1390   \stex_debug:nn{uri}{returning~\stex_use_module_uri:N #1}
1391 }

```

(End of definition for \stex_uri_from_pair:Nnn. This function is documented on page 136.)

sfunction

\stex_uri_from_pair:Nn

```

1392 \cs_new_protected:Nn \stex_uri_from_pair:Nn {
1393   \peek_charcode:NTF [ {
1394     \__stex_uris_parse_i:Nw #1
1395   }{
1396     \__stex_uris_parse_ii:Nw #1
1397   } #2 \__stex_uris_end:
1398 }
1399
1400 \cs_new_protected:Npn \__stex_uris_parse_i:Nw #1 [#2] #3 \__stex_uris_end: {
1401   \stex_uri_from_pair:Nnn #1 {#2} {#3}
1402 }
1403
1404 \cs_new_protected:Npn \__stex_uris_parse_ii:Nw #1 #2 \__stex_uris_end: {
1405   \stex_uri_from_pair:Nnn #1 {} {#2}
1406 }

```

(End of definition for \stex_uri_from_pair:Nn. This function is documented on page 136.)

sfunction

29.3 Symbol URIs

\stex_use_symbol_uri:N

\stex_use_symbol_uri:n

```

1407 \cs_new:Nn \stex_use_symbol_uri:N {
1408   \exp_after:wN \__stex_uris_symbol:nnnn #1
1409 }
1410 \cs_new:Nn \stex_use_symbol_uri:n {
1411   \__stex_uris_symbol:nnnn #1
1412 }
1413 \cs_new:Nc \__stex_uris_symbol:nnnn {
1414   \exp_not:N \stex_archive_base:n{#1} \tl_to_str:n{?a=} #1
1415   \exp_not:N \tl_if_empty:nF{#2}{
1416     \c_ampersand_str\tl_to_str:n{p=}#2
1417   }
1418   \c_ampersand_str\tl_to_str:n{m=}#3

```

```

1419 \c_ampersand_str\tl_to_str:n{s=}#4
1420 }

```

(End of definition for \stex_use_symbol_uri:N and \stex_use_symbol_uri:n. These functions are documented on page 136.)

```
sfunction
```

```
\stex_new_symbol_uri:n
```

```

1421 \cs_new:Nn \stex_new_symbol_uri:n {
1422   \l_stex_current_module_uri { \tl_to_str:n{ #1 } }
1423 }

```

(End of definition for \stex_new_symbol_uri:n. This function is documented on page 136.)

```
sfunction
```

```
\stex_new_symbol_uri:nn
```

```

1424 \cs_new:Nn \stex_new_symbol_uri:nn {
1425   #1 { #2 }
1426 }

```

(End of definition for \stex_new_symbol_uri:nn. This function is documented on page 136.)

```
sfunction
```

```
\stex_symbol_uri_archive:N
```

```

1427 \cs_new:Nn \stex_symbol_uri_archive:N {
1428   \exp_after:wN \use_i:nnnn #1
1429 }

```

(End of definition for \stex_symbol_uri_archive:N. This function is documented on page 136.)

```
sfunction
```

```
\stex_symbol_uri_path:N
```

```

1430 \cs_new:Nn \stex_symbol_uri_path:N {
1431   \exp_after:wN \use_ii:nnnn #1
1432 }

```

(End of definition for \stex_symbol_uri_path:N. This function is documented on page 136.)

```
sfunction
```

```
\stex_symbol_uri_module:N
```

```
\stex_symbol_uri_module:n
```

```

1433 \cs_new:Nn \stex_symbol_uri_module:N {
1434   \exp_after:wN \__stex_uris_first_three:nnnn #1
1435 }
1436 \cs_new:Nn \stex_symbol_uri_module:n {
1437   \__stex_uris_first_three:nnnn #1
1438 }
1439
1440 \cs_new:Nn \__stex_uris_first_three:nnnn {
1441   {#1}{#2}{#3}
1442 }

```

(End of definition for \stex_symbol_uri_module:N and \stex_symbol_uri_module:n. These functions are documented on page 137.)

```
sfunction
```

`\stex_symbol_uri_name:N`

`\stex_symbol_uri_name:n`

```
1443 \cs_new:Nn \stex_symbol_uri_name:N {  
1444   \exp_after:wN \use_iv:nnnn #1  
1445 }  
1446 \cs_new:Nn \stex_symbol_uri_name:n {  
1447   \use_iv:nnnn #1  
1448 }
```

(End of definition for `\stex_symbol_uri_name:N` and `\stex_symbol_uri_name:n`. These functions are documented on page 137.)

sfunction

Chapter 30

Documents

```
1449 (@@=stex_doc)

\STEXftmllink
\STEXlinkftml
\STEXsetlinkftml

1450 \cs_new_protected:Npn \STEXftmllink {
1451   \exp_args:Ne\url{
1452     \stex_use_document_uri:N \l_stex_current_document_uri
1453   }
1454 }
1455
1456 \cs_new_protected:Npn \STEXsetlinkftml #1 {
1457   \tl_gset:Nn \g__stex_doc_ftml_link_text_tl { #1 }
1458 }
1459
1460 \tl_gset:Nn \g__stex_doc_ftml_link_text_tl{
1461   The~
1462   \stex_get_symbol:nF{FTML}{~}
1463   \tl_if_empty:NTF \l_stex_get_symbol_uri {FTML}{\sn{FTML}}
1464   {~ version ~ of ~ this ~ document ~ can ~ be ~ found ~ at\\
1465   \STEXftmllink
1466 }
1467
1468 \NewDocumentCommand \STEXlinkftml { 0{ } } {
1469   \ifstexhtml\else
1470     \begin{center}
1471       \emph{
1472         \tl_if_empty:NTF{#1}\g__stex_doc_ftml_link_text_tl{#1}
1473       }
1474     \end{center}
1475   \fi
1476 }
```

(End of definition for \STEXftmllink, \STEXlinkftml, and \STEXsetlinkftml. These functions are documented on page 138.)

sffunction

\stexdoctitle Initial definition (before \begin{document}):

```
1477 \tl_new:N \g__stex_doc_title_tl
1478
1479 \str_new:N \g_stex_document_kind_str
```

```

1480 \tl_new:N \g_stex_document_kind_parameters_tl
1481 \str_set:Nn \g_stex_document_kind_str{article}
1482 \stex_if_html_backend:T{
1483   \AtBeginDocument{
1484     \exp_args:Nne\stex_html_node:nnn{span}{
1485       data-ftml-document-kind=\g_stex_document_kind_str
1486       \g_stex_document_kind_parameters_tl
1487     }{}
1488   }
1489 }
1490
1491 \cs_new_protected:Npn \stexdoctitle #1 {
1492   \tl_gset:Nn \g__stex_doc_title_tl { #1 }
1493   \global\def\stexdoctitle##1{}
1494 }

```

At begin document, we switch to:

```

1495 \cs_new_protected:Nn \__stex_doc_set_title:n {
1496   \stex_if_smsmode:F{
1497     \gdef\stexdoctitle##1{}
1498     \stex_debug:nn{title}{Setting~title~to:~\tl_to_str:n{#1}}
1499     \tl_gset:Nn \g__stex_doc_title_tl { #1 }
1500     \__stex_doc_title_html:
1501   }
1502 }
1503
1504 \cs_new_protected:Nn \__stex_doc_title_html: {
1505   \stex_if_do_html:T{
1506     \stex_annotate_invisible:nn{data-ftml-doctitle={}}{ \hbox{\g__stex_doc_title_tl} }
1507   }
1508 }
1509
1510 \AtBeginDocument {
1511   \tl_if_empty:NTF \g__stex_doc_title_tl {
1512     \cs_set_eq:NN \stexdoctitle \__stex_doc_set_title:n
1513   }{
1514     \gdef\stexdoctitle#1{}
1515     \__stex_doc_title_html:
1516   }
1517
1518   \cs_set_eq:NN \__stex_doc_maketitle: \maketitle
1519   \protected\gdef\maketitle{
1520     \tl_if_empty:NF \@title {
1521       \exp_args:No \stexdoctitle \@title
1522     }
1523     \__stex_doc_maketitle:
1524   }
1525 }

```

(End of definition for `\stexdoctitle`. This function is documented on page 138.)

sfunction

30.1 Sectioning

`\l_stex_current_section_level_int`
`\l_stex_current_section_level_str`

```
1526 \int_new:N \l_stex_current_section_level_int
1527 \str_set:Nn \l_stex_current_section_level_str {document}
```

(End of definition for `\l_stex_current_section_level_int` and `\l_stex_current_section_level_str`. These variables are documented on page 139.)

`\currentsectionlevel`
`\Currentsectionlevel`

```
1528 \cs_set_protected:Npn \currentsectionlevel {
1529   \stex_if_do_html:TF{
1530     \mode_if_vertical:T{\hbox_unpack:N\c_empty_box}
1531     \stex_annotate:nn{data-ftml-currentsectionlevel={},data-ftml-capitalize=false}{}
1532   }{
1533     \l_stex_current_section_level_str
1534   }
1535   \tl_if_exist:NT\xspace\xspace
1536 }
1537
1538 \cs_set_protected:Npn \Currentsectionlevel {
1539   \stex_if_do_html:TF{
1540     \mode_if_vertical:T{\hbox_unpack:N\c_empty_box}
1541     \stex_annotate:nn{data-ftml-currentsectionlevel={},data-ftml-capitalize=true}{}
1542   }{
1543     \exp_after:wN \str_uppercase:n \l_stex_current_section_level_str
1544   }
1545   \tl_if_exist:NT\xspace\xspace
1546 }
```

(End of definition for `\currentsectionlevel` and `\Currentsectionlevel`. These functions are documented on page 139.)

sfunction

`sfragment (env.)`

```
1547 \stex_keys_define:nnnn{ sfragment }{
1548   \tl_clear:N \l_stex_key_short_tl
1549 }{
1550   short .tl_set:N = \l_stex_key_short_tl
1551 }{id}
1552
1553 \NewDocumentEnvironment{sfragment}{0{} m}{
1554   \stex_keys_set:nn{sfragment}{#1}
1555   \__stex_doc_do_section:n{#2}
1556   \stex_do_id: % TODO
1557 }{
1558   \_sfragment_end:
1559 }
1560
1561 \cs_new_protected:Npn \__stex_doc_do_section:n {
1562   \int_case:nnF \l_stex_current_section_level_int {
1563     {0}{\cs_if_exist:NTF \thepart {\_sfragment_do_level:nn{part}}{
1564       \int_incr:N \l_stex_current_section_level_int
1565       \__stex_doc_do_section:n
```

```

1566     }}
1567     {1}{\cs_if_exist:NTF \thechapter {\_sfragment_do_level:nn{chapter}}{
1568         \int_incr:N \l_stex_current_section_level_int
1569         \__stex_doc_do_section:n
1570     }}
1571     {2}{\_sfragment_do_level:nn{section}}
1572     {3}{\_sfragment_do_level:nn{subsection}}
1573     {4}{\_sfragment_do_level:nn{subsubsection}}
1574     {5}{\_sfragment_do_level:nn{paragraph}}
1575 }{\_sfragment_do_level:nn{subparagraph}}
1576 }
1577
1578 \stex_if_html_backend:TF {
1579     \cs_new_protected:Nn \_sfragment_do_level:nn {
1580         \stexdoctitle{#2}
1581         \par
1582         \begin{stex_env_node}{section}{data-ftml-section={\int_use:N \l_stex_current_section_level_int}
1583             \noindent\stex_html_node:nnn{h1}{data-ftml-title={}}{
1584                 \stex_annotate_force_break:n{#2}
1585             }\par
1586         }
1587     \cs_new_protected:Nn \_sfragment_end: {
1588         \end{stex_env_node}
1589     }
1590 }{
1591     \cs_new_protected:Nn \_sfragment_do_level:nn {
1592         \stexdoctitle{#2}
1593         \tl_if_empty:NTF \l_stex_key_short_tl {
1594             \use:c{#1}
1595         }{
1596             \exp_args:Nne \use:nn{\use:c{#1}}{[\exp_args:No \exp_not:n \l_stex_key_short_tl]}
1597         }{#2}
1598
1599         \int_compare:nNnF \l_stex_current_section_level_int = 6{
1600             \int_incr:N \l_stex_current_section_level_int
1601         }
1602         \tl_set:Nn\l_stex_current_section_level_str{#1}
1603     }
1604     \cs_new_protected:Nn \_sfragment_end: {}
1605 }

```

`titlefragment (env.)`

```

1606 \bool_new:N \l__stex_doc_titlefragment_bool
1607
1608 \NewDocumentEnvironment{titlefragment}{0}{m}{
1609     \stex_keys_set:nn{sfragment}{#1}
1610     \cs_if_exist:NTF \maketitle {
1611         \bool_set_true:N \l__stex_doc_titlefragment_bool
1612         \title{#2}
1613     \tl_if_empty:NF \l_stex_key_short_tl {
1614         \cs_if_exist:NT \shorttitle {
1615             \exp_args:No \shorttitle \l_stex_key_short_tl
1616         }
1617     }

```

```

1618     \stexdoctitle{#2}
1619     \maketitle
1620   }{
1621     \bool_set_false:N \l__stex_doc_titlefragment_bool
1622     \__stex_doc_do_section:n{#2}
1623     \_stex_do_id:
1624   }
1625 }{
1626   \bool_if:NF \l__stex_doc_titlefragment_bool \_sfragment_end:
1627 }

```

blindfragment (*env.*)

```

1628 \cs_new_protected:Nn \__stex_doc_skip_section_i: {
1629   \int_case:nn \l_stex_current_section_level_int {
1630     {0}{\cs_if_exist:NF \thepart {
1631       \int_incr:N \l_stex_current_section_level_int \__stex_doc_skip_section_i:
1632     }}
1633     {1}{\cs_if_exist:NF \thechapter {
1634       \int_incr:N \l_stex_current_section_level_int \__stex_doc_skip_section_i:
1635     }}
1636   }
1637   \int_compare:nNnF \l_stex_current_section_level_int = 6{
1638     \int_incr:N \l_stex_current_section_level_int
1639   }
1640 }
1641
1642 \stex_if_html_backend:TF {
1643   \cs_new_protected:Nn \__stex_doc_skip_section: {
1644     \__stex_doc_skip_section_i:
1645     \begin{stex_annotate_env}{data-ftml-skipsection={\int_use:N \l_stex_current_section_level_int}}
1646     \_stex_annotate_force_break:n{}
1647   }
1648 }{
1649   \cs_set_eq:NN \__stex_doc_skip_section: \__stex_doc_skip_section_i:
1650 }
1651
1652 \NewDocumentEnvironment{blindfragment}{}{
1653   \__stex_doc_skip_section:
1654 }{
1655   \stex_if_html_backend:T{
1656     \stex_annotate_invisible:n{~}
1657     \end{stex_annotate_env}
1658   }
1659 }

```

\skipfragment

```

1660 \cs_new_protected:Npn \skipfragment {
1661   \int_case:nnF \l_stex_current_section_level_int {
1662     {0}{\cs_if_exist:NTF \thepart {\stepcounter{part}}{
1663       \int_incr:N \l_stex_current_section_level_int
1664       \skipfragment
1665     }}
1666     {1}{\cs_if_exist:NTF \thechapter {\stepcounter{chapter}}{
1667       \int_incr:N \l_stex_current_section_level_int

```



```

1668     \skipfragment
1669   }}
1670   {2}{\stepcounter{section}}
1671   {3}{\stepcounter{subsection}}
1672   {4}{\stepcounter{subsubsection}}
1673   {5}{\stepcounter{paragraph}}
1674   }{\stepcounter{subparagraph}}
1675 }

```

(End of definition for \skipfragment. This function is documented on page 139.)

sfunction

\setsectionlevel

```

1676 \cs_new_protected:Npn \setsectionlevel #1 {
1677   \str_case:nnF{#1}{
1678     {part}{\int_set:Nn \l_stex_current_section_level_int 0}
1679     {chapter}{\int_set:Nn \l_stex_current_section_level_int 1}
1680     {section}{\int_set:Nn \l_stex_current_section_level_int 2}
1681     {subsection}{\int_set:Nn \l_stex_current_section_level_int 3}
1682     {subsubsection}{\int_set:Nn \l_stex_current_section_level_int 4}
1683     {paragraph}{\int_set:Nn \l_stex_current_section_level_int 5}
1684   }{
1685     \int_set:Nn \l_stex_current_section_level_int 6
1686   }
1687   \stex_if_do_html:T{
1688     \cs_if_eq:NNTF\@onlypreamble\@notprerr{
1689       \stex_html_literal:n{<span~ data-ftml-sectionlevel="\int_use:N\l_stex_current_section_level_int}
1690     }{
1691       \AtBeginDocument{
1692         \stex_html_literal:n{<span~ data-ftml-sectionlevel="\int_use:N\l_stex_current_section_level_int}
1693       }
1694     }
1695   }
1696 }

```

In HTML mode, we make sure to record the top section level:

```

1697 \stex_if_html_backend:T{
1698   \cs_new_protected:Nn \__stex_doc_check_topsect: {
1699     \int_case:nnF \l_stex_current_section_level_int {
1700       {0}{\cs_if_exist:NTF \thepart {
1701         \stex_annotate_invisible:nn{data-ftml-sectionlevel=0}{}}
1702       }{
1703         \int_incr:N \l_stex_current_section_level_int
1704         \__stex_doc_check_topsect:
1705       }}
1706     {1}{\cs_if_exist:NTF \thechapter {
1707       \stex_annotate_invisible:nn{data-ftml-sectionlevel=1}{}}
1708     }{
1709       \int_incr:N \l_stex_current_section_level_int
1710       \__stex_doc_check_topsect:
1711     }}
1712   }{
1713     \stex_annotate_invisible:nn{data-ftml-sectionlevel={\int_use:N\l_stex_current_section_level_int}}
1714   }
1715 }

```

```

1716 \AtBeginDocument{\__stex_doc_check_topsect:
1717 \cs_if_exist:NT \thechapter {
1718 \int_case:nnT \l_stex_current_section_level_int{
1719 {0}{}}
1720 {1}{}}
1721 }{
1722 \stex_css_literal:n {
1723 .ftml-section {
1724 &~.ftml-title-section { &::before {
1725 content:~counter(ftml-chapter)~"."~counter(ftml-section)~"";
1726 } }
1727 }
1728 .ftml-subsection {
1729 &~.ftml-title-subsection { &::before {
1730 content:~counter(ftml-chapter)~"."~counter(ftml-section)~"."~counter(ftml-sub
1731 } }
1732 }
1733 .ftml-subsubsection {
1734 &~.ftml-title-subsubsection { &::before {
1735 content:~counter(ftml-chapter)~"."~counter(ftml-section)~"."~counter(ftml-sub
1736 } }
1737 }
1738 }
1739 }
1740 }
1741 }
1742 }

```

(End of definition for `\setsectionlevel`. This function is documented on page 139.)

sfunction

We make sure `\frontmatter` and `\backmatter` are always defined. **TODO: this seems like a hack and probably shouldn't be here**

```

1743 \bool_if:NF \c_stex_no_frontmatter_bool {
1744 \cs_new_protected:Nn \__stex_doc_x_matter: {
1745 \cs_if_exist:NTF\frontmatter{
1746 \let\__stex_doc_orig_frontmatter\frontmatter
1747 \let\__stex_doc_orig_endfrontmatter\endfrontmatter
1748 \let\endfrontmatter\relax
1749 \let\frontmatter\relax
1750 }{
1751 \tl_set:Nn\__stex_doc_orig_frontmatter{
1752 \clearpage
1753 %\@mainmatterfalse
1754 \pagenumbering{roman}
1755 }
1756 \tl_set:Nn\__stex_doc_orig_endfrontmatter{}
1757 }
1758 \cs_if_exist:NTF\backmatter{
1759 \let\__stex_doc_orig_backmatter\backmatter
1760 \let\__stex_doc_orig_endbackmatter\endbackmatter
1761 \let\backmatter\relax
1762 \let\endbackmatter\relax
1763 }{
1764 \tl_set:Nn\__stex_doc_orig_backmatter{

```

```

1765     \clearpage
1766     %\@mainmatterfalse
1767     \pagenumbering{roman}
1768   }
1769 }
1770 \newenvironment{frontmatter}{
1771   \__stex_doc_orig_frontmatter
1772 }{
1773   \cs_if_exist:NTF\mainmatter{
1774     \mainmatter
1775   }{
1776     \clearpage
1777     %\@mainmattertrue
1778     \pagenumbering{arabic}
1779   }
1780 }
1781 \newenvironment{backmatter}{
1782   \__stex_doc_orig_backmatter
1783 }{
1784   \cs_if_exist:NTF\mainmatter{
1785     \mainmatter
1786   }{
1787     \clearpage
1788     %\@mainmattertrue
1789     \pagenumbering{arabic}
1790   }
1791 }
1792 }
1793 \AddToHook{begindocument/end}\__stex_doc_x_matter:
1794 }

```

30.2 Inter-Document References

1795 <@@=stex_refs>

The .sref-file:

```

1796 \iow_new:N \c__stex_refs_iow
1797 \AtBeginDocument{ \iow_open:Nn \c__stex_refs_iow {\jobname.sref} }
1798 \AtEndDocument{\iow_close:N \c__stex_refs_iow}
1799 \str_set:Ne \c__stex_refs_iow_str { \stex_file_use:N \c_stex_pwd_file / }

```

Keys:

```

1800 \stex_keys_define:nnnn{sref / 1}{
1801   \tl_clear:N \l_stex_key_pre_tl
1802   \tl_clear:N \l_stex_key_post_tl
1803   \tl_clear:N \l_stex_key_fallback_tl
1804 }{
1805   % TODO get rid of this
1806   fallback .code:n = {},
1807   pre      .code:n = {},
1808   post     .code:n = {}
1809 }{archive file}
1810 \stex_keys_define:nnnn{sref / 2}{}{}{archive file, title}

```

Target declarations:

`\sreflabel`

`\stex_ref_new_doc_target:n`

```
1811 \cs_new_protected:Nn \stex_ref_new_doc_target:n {
1812   \stex_if_smsmode:F{
1813     \tl_set:Nc \l__stex_refs_new_tl { \stex_new_document_element_uri:n { #1 } }
1814     \exp_args:Nc \label { \stex_use_document_uri:N \l__stex_refs_new_tl }
1815     \exp_args:Nne \STeXInternalNewSRefLabel{#1}{ \stex_use_document_uri:N \l__stex_refs_new_tl }
1816     \iow_now:Nc \auxout {
1817       \STeXInternalNewSRefLabel{#1}{ \stex_use_document_uri:N \l__stex_refs_new_tl }
1818     }
1819     \iow_now:Nc \c__stex_refs_iow {
1820       \exp_not:N \STeXInternalSRefLabel
1821       { #1 }
1822       { \stex_use_document_uri:N \l__stex_refs_new_tl }
1823       { \exp_not:o \@currentHref }
1824       { \exp_not:o \@currentlabelname }
1825     }
1826   }
1827 }
1828
1829 \NewDocumentCommand \sreflabel {m} { \stex_ref_new_doc_target:n{#1} }
```

(End of definition for \sreflabel and \stex_ref_new_doc_target:n. These functions are documented on page 86.)

sfunction

`\stex_ref_new_sym_target:n`

```
1830 \cs_new_protected:Nn \stex_ref_new_sym_target:n {
1831   \stex_if_smsmode:F{
1832     \cs_if_exist:cTF{r@\tl_to_str:n{#1}}{
1833       \cs_if_exist:cT{__stex_sref_aux_sym_ \tl_to_str:n{#1}}{
1834         \cs_undefine:c{__stex_sref_aux_sym_ \tl_to_str:n{#1}}
1835         \exp_args:Nc \label { \tl_to_str:n{#1} }
1836       }
1837     }{ \exp_args:Nc \label { \tl_to_str:n{#1} } }
1838   }
1839 }
```

(End of definition for \stex_ref_new_sym_target:n. This function is documented on page ??.)

sfunction

Inserting References:

`\sref`

```
1840 \NewDocumentCommand \sref { 0{} m 0{} }{
1841   \stex_keys_set:nn { sref / 1 }{ #1 }
1842   \tl_clear:N \l__stex_refs_tmp
1843   \str_if_empty:NTF \l_stex_key_file_str {
1844     \__stex_refs_local:n
1845   }{
1846     \__stex_refs_remote:nn{#3}
1847   }{#2}
1848 }
1849
1850 \cs_new_protected:Nn \__stex_refs_local:n {
1851   \stex_debug:nn{sref}{Checking~#1}
```

```

1852 \tl_if_exist:cTF{ g_stex_sref_label_ #1 }{
1853   %% TODO INSERT FTML ANNOTATION
1854   \exp_args:Ne \__stex_refs_local_ref:n {\use:c{ g_stex_sref_label_ #1 }}
1855 }{
1856   \tl_if_empty:NTF \l_stex_key_fallback_tl { ??? }{ \l_stex_key_fallback_tl }
1857 }
1858 }
1859
1860 \cs_new_protected:Nn \__stex_refs_local_ref:n {
1861   \cs_if_exist:cTF{autoref}{
1862     \l_stex_key_pre_tl \autoref{#1} \l_stex_key_post_tl
1863   }{
1864     \message{^^J^^J
1865       sTeX~Warning: \c_backslash_str sref ~ with ~ local ~ target ~ used ~ without ~ hyperr
1866       ^^J^^J}
1867     \l_stex_key_pre_tl \ref{#1} \l_stex_key_post_tl
1868   }
1869 }
1870
1871 \cs_new_protected:Nn \__stex_refs_remote:nn {
1872   \str_set:Nn \l__stex_refs_key { #2 }
1873   \tl_set:Nn \l__stex_refs_in { #1 }
1874   \exp_args:No \stex_in_archive:nn \l_stex_key_archive_str {
1875     \exp_args:NNo \stex_document_uri_from_archive_file:Nn \l__stex_refs_uri {\l_stex_key_fi
1876     \stex_debug:nn{sref}{Checking~#2~from~\stex_use_document_uri:N \l__stex_refs_uri}
1877     \tl_if_eq:NNTF \l__stex_refs_uri \c_stex_main_document_uri {
1878       \stex_debug:nn{sref}{From~current~document;~delegating~to~\string\autoref}
1879       \__stex_refs_local:n{ #2 }
1880     }{
1881       \str_set:Ne \l__stex_refs_uri { \stex_use_document_uri:N \l__stex_refs_uri }
1882       \str_set:Ne \l__stex_refs_file { \exp_args:No \stex_source_path:n {\l_stex_key_file_s
1883       \exp_args:No \IfFileExists \l__stex_refs_file {
1884         \__stex_refs_find:
1885       }{
1886         \tl_if_empty:NTF \l__stex_refs_tmp {
1887           \stex_debug:nn{sref}{Not~found}
1888           \tl_if_empty:NTF \l_stex_key_fallback_tl { ??? }{ \l_stex_key_fallback_tl }
1889         }{
1890           \str_set_eq:NN \l__stex_refs_uri \l__stex_refs_tmp
1891           \stex_debug:nn{sref}{Found~ \l__stex_refs_uri}
1892           \__stex_refs_maybe_in:
1893         }
1894       }
1895     }
1896   }
1897
1898 \cs_new_protected:Nn \__stex_refs_find: {
1899   \setbox0\vbox\bgroup
1900   \cs_set_eq:NN \STeXInternalSRefLabel \__stex_refs_check_i:nnnn
1901   \use:c{@ @ input}{\l__stex_refs_file}
1902   \exp_args:NNe \use:nn
1903   \egroup {
1904     \tl_set:Nn \exp_not:N \l__stex_refs_tmp { \l__stex_refs_tmp }
1905   }

```

```

1906 }
1907
1908 \cs_new_protected:Nn \__stex_refs_check_i:nnnn {
1909   \exp_args:No \str_if_eq:nnT{ \l__stex_refs_key }{#1}{
1910     \exp_args:Nno \stex_str_if_starts_with:nnT{#2}{\l__stex_refs_uri}{
1911       \str_set:Nn \l__stex_refs_tmp {#2}
1912     }
1913   }
1914 }
1915 }
1916
1917 \cs_new_protected:Nn \__stex_refs_maybe_in: {
1918   \tl_if_empty:NTF\l__stex_refs_in {
1919     \cs_if_exist:cTF{r@\l__stex_refs_uri}{
1920       \exp_args:No \__stex_refs_local_ref:n \l__stex_refs_uri
1921     }{
1922       \tl_if_empty:NTF \l__stex_refs_default_file {
1923         \exp_args:No \__stex_refs_local_ref:n \l__stex_refs_uri
1924       }{
1925         \tl_set_eq:NN \l_stex_key_title_tl \l__stex_refs_default_title
1926         \str_set_eq:NN \l__stex_refs_file \l__stex_refs_default_file
1927         \str_if_eq:NNTF \l__stex_refs_default_file \c__stex_refs_iow_str {
1928           \exp_args:No \__stex_refs_local_ref:n \l__stex_refs_uri
1929         }{
1930           \stex_debug:nn{sref}{in~title=\exp_args:No\exp_not:n\l_stex_key_title_tl,file=\l__
1931             \__stex_refs_in:
1932         }
1933       }
1934     }
1935   }{
1936     \stex_debug:nn{sref}{in~\exp_args:No\exp_not:n\l__stex_refs_in}
1937     \exp_args:No \stex_in_archive:nn \l_stex_key_archive_str {
1938       \str_set:Ne \l__stex_refs_file { \exp_args:No \stex_source_path:n {\l_stex_key_file_s
1939     }
1940     \exp_args:Nno \stex_keys_set:nn{ sref / 2 }{ \l__stex_refs_in }
1941     \__stex_refs_in:
1942   }
1943 }
1944
1945 \cs_new_protected:Nn \__stex_refs_in: {
1946   \tl_clear:N \l__stex_refs_tmp
1947   \exp_args:No \IfFileExists \l__stex_refs_file {
1948     \__stex_refs_find_in:
1949   }{
1950     \stex_debug:nn{sref}{not~found:~\l__stex_refs_file}
1951   }
1952   \tl_if_empty:NTF\l__stex_refs_tmp{
1953     \stex_debug:nn{sref}{Not~found}
1954     \tl_if_empty:NTF \l_stex_key_fallback_tl { ??? }{ \l_stex_key_fallback_tl }
1955   }{
1956     \stex_debug:nn{sref}{found:~\meaning\l__stex_refs_tmp}
1957     \tl_set:Ne \l__stex_refs_tmp {
1958       \exp_after:wN \__stex_refs_in_text:nn \l__stex_refs_tmp
1959     }

```

```

1960     \cs_if_exist:NT \href {\exp_args:No \href \l__stex_refs_uri } \l__stex_refs_tmp
1961   }
1962 }
1963
1964 \cs_new:Nn \__stex_refs_in_text:nn {
1965   \__stex_refs_in_text:w #1 \__stex_refs_stop:
1966   \tl_if_empty:nF{#2}{~(\exp_not:n{#2})}
1967   \tl_if_empty:NF \l_stex_key_title_tl {
1968     {}~in~\exp_args:No \exp_not:n \l_stex_key_title_tl
1969   }
1970 }
1971
1972 \cs_new:Npn \__stex_refs_in_text:w #1.#2 \__stex_refs_stop: {
1973   \__stex_refs_uppercase:n #1~#2~
1974 }
1975
1976 \cs_new:Nn \__stex_refs_uppercase:n { \uppercase{#1}}
1977
1978 \cs_new_protected:Nn \__stex_refs_find_in: {
1979   \stex_debug:nn{sref}{finding~\l__stex_refs_uri;~in~\l__stex_refs_file...?}
1980   \setbox0\vbox\bgroup
1981     \catcode'\J=9\relax
1982     \cs_set_eq:NN \STeXInternalSRefLabel \__stex_refs_check_in:nnnn
1983     \use:c{@ @ input}{\l__stex_refs_file}
1984     \exp_args:NNe \use:nn
1985     \egroup {
1986       \tl_set:Nn \exp_not:N \l__stex_refs_tmp { \exp_args:No \exp_not:n \l__stex_refs_tmp }
1987     }
1988 }
1989
1990 \cs_new_protected:Nn \__stex_refs_check_in:nnnn {
1991   \exp_args:No \str_if_eq:nnT{ \l__stex_refs_uri }{#2}{
1992     \tl_set:Nn \l__stex_refs_tmp { {#3}{#4} }
1993     \endinput
1994   }
1995 }

```

(End of definition for \sref. This function is documented on page 86.)

```

sfunction
TODO

1996 %\int_new:N \l__stex_refs_unnamed_counter_int
1997
1998
1999
2000
2001 \cs_new:Npn \STeXInternalSRefLabel #1 #2 #3 #4 {}
2002
2003 \cs_new_protected:Npn \STeXInternalNewSRefLabel #1 #2 {
2004   \tl_gset:ce{ g_stex_sref_label_ #1 }{ \tl_to_str:n{ #2 } }
2005 }
2006
2007 \tl_new:N \l__stex_refs_default_title
2008 \str_new:N \l__stex_refs_default_file
2009 \str_new:N \l__stex_refs_default_archive

```

```

2010
2011 \newcommand\srefsetin[3][]{
2012   \str_set:Ne \l__stex_refs_default_file { #2 }
2013   \tl_if_empty:nTF{#1}{
2014     \str_set:Ne \l__stex_refs_default_archive { \stex_archive_id:N \c_stex_main_archive }
2015   }{
2016     \str_set:Nn \l__stex_refs_default_archive { #1 }
2017   }
2018   \exp_args:No \stex_in_archive:nn \l__stex_refs_default_archive {
2019     \str_set:Ne \l__stex_refs_default_file { \exp_args:No \stex_source_path:n {\l_stex_key_
2020   }
2021   \str_set:Nn \l__stex_refs_default_title { #3 }
2022 }
2023 \NewDocumentCommand \extref { 0{} m m }{???}
2024
2025 \cs_new_protected:Nn \stex_ref_new_symbol:n {}
2026
2027
2028 \NewDocumentCommand \srefsym { m m }{
2029   \stex_get_symbol:n { #1 }
2030   \exp_args:Ne \srefsymuri { \stex_use_symbol_uri:N \l_stex_get_symbol_uri }{ #2 }
2031 }
2032
2033 \cs_new_protected:Npn \srefsymuri #1 {
2034   \cs_if_exist:cTF{r@{tl_to_str:n{#1}}}{
2035     \cs_if_exist:NT \hyperlink { \hyperlink{#1} }
2036   }{
2037     \cs_if_exist:NT \href { \href{#1} }
2038   }
2039 }

```

30.3 Inputting From MathHub Resources

```

2040 <@@=stex_inputs>
\inputref
2041 \NewDocumentCommand \inputref { 0{} m }{
2042   \stex_in_archive:nn{#1}{
2043     \stex_document_uri_from_archive_file:Nn \l__stex_inputs_uri {#2}
2044     \__stex_inputs_ref:n{#2}
2045   }
2046 }
2047
2048 \stex_if_html_backend:TF{
2049   \str_set:Nn \c__stex_inputs_ref_pre_str {<span~data-ftml-inputref="}
2050   \str_set:Nn \c__stex_inputs_ref_post_str {"~><!--></span>}
2051   \cs_new_protected:Nn \__stex_inputs_ref_i:n {
2052     \IfFileExists{\stex_source_path:n {#1}}{
2053       %\relax
2054       %\noindent\vbox{
2055         \exp_args:Ne\stex_html_literal:n{
2056           \c__stex_inputs_ref_pre_str
2057           \stex_use_document_uri:N\l__stex_inputs_uri

```



```

2058         \c__stex_inputs_ref_post_str
2059     }
2060     %}
2061 }{
2062     \stex_input_with_hooks:Ne\l__stex_inputs_uri {\stex_source_path:n {#1}}
2063 }
2064 }
2065 \cs_new_protected:Nn \__stex_inputs_ref:n {
2066     \ifnum\@listdepth>0
2067         \noindent\vbox{\vskip-26pt\relax\__stex_inputs_ref_i:n{#1}}\par
2068         %\begin{stex_env_node}{div}{style:foo=bar}\vskip-700pt\end{stex_env_node}
2069     \else
2070         \__stex_inputs_ref_i:n{#1}
2071     \fi
2072 }
2073
2074 }{
2075     \cs_new_protected:Nn \__stex_inputs_ref:n {
2076         \begingroup
2077         \inputreftrue
2078         \let \l_stex_metatheory_uri \c_stex_default_metatheory
2079         %\seq_clear:N \l_stex_all_modules_seq
2080         \stex_undefine:N \l_stex_current_module_uri
2081         \stex_debug:nn{inputref}{Inputrefing~document~\stex_use_document_uri:N \l__stex_input
2082         \stex_input_with_hooks:Ne\l__stex_inputs_uri{\stex_source_path:n {#1}}
2083         \endgroup\medskip
2084     }
2085 }

```

(End of definition for \inputref. This function is documented on page 140.)

sfunction

\mhinput

```

2086 \NewDocumentCommand \mhinput { 0{} m}{
2087     \stex_in_archive:nn {#1} {
2088         \stex_document_uri_from_archive_file:Nn \l__stex_inputs_uri {#2}
2089         \ifinputref
2090             \stex_input_with_hooks:Ne\l__stex_inputs_uri{\stex_source_path:n {#2}}
2091         \else
2092             \inputreftrue
2093             \stex_input_with_hooks:Ne\l__stex_inputs_uri{\stex_source_path:n {#2}}
2094             \inputreffalse
2095         \fi
2096     }
2097 }

```

(End of definition for \mhinput. This function is documented on page 140.)

sfunction

\ifinputref

```

2098 \newif \ifinputref \inputreffalse

```

(End of definition for \ifinputref. This function is documented on page 140.)

sfunction

\IfInputref

```

2099 \stex_if_html_backend:TF{
2100   \newcommand \IfInputref[2]{
2101     \stex_annotate:nn{data-ftml-ifinputref=true}{#1}
2102     \stex_annotate:nn{data-ftml-ifinputref=false}{#2}
2103   }
2104 }{
2105   \newcommand \IfInputref[2]{
2106     \ifinputref #1 \else #2 \fi
2107   }
2108 }

```

(End of definition for \IfInputref. This function is documented on page 140.)
sfunction

\libinput

```

2109 \newcommand \libinput [2] [] {
2110   \stex_in_archive:nn{#1}{
2111     \__stex_inputs_up_archive:nn{#2}{tex}
2112     \stex_debug:nn{lib}{Result:~\seq_use:Nn \l__stex_inputs_libinput_files_seq ,}
2113     \seq_if_empty:NTF \l__stex_inputs_libinput_files_seq {
2114       \msg_error:nnnn{stex}{error/nofile}{\libinput}{#2.tex}
2115     }{
2116       \seq_map_function:NN \l__stex_inputs_libinput_files_seq \input
2117     }
2118   }
2119 }

```

(End of definition for \libinput. This function is documented on page 140.)
sfunction

\libusepackage

```

2120 \newcommand\libusepackage[2] []{
2121   \__stex_inputs_up_archive:nn{#2}{sty}
2122   \int_compare:nNnTF {\seq_count:N \l__stex_inputs_libinput_files_seq} = 1 {
2123     \str_set:Ne \l__stex_inputs_tmp_str {\seq_item:Nn \l__stex_inputs_libinput_files_seq 1}
2124     \exp_args:Nne \use:n {\usepackage[#1]} {
2125       \str_range:Nnn\l__stex_inputs_tmp_str 1 {-5}
2126     }
2127   }{
2128     \msg_fatal:nnnn{stex}{error/nofile}{\libusepackage}{#2.sty}
2129   }
2130 }

```

(End of definition for \libusepackage. This function is documented on page 140.)
sfunction

\libusetikzlibrary

```

2131 \newcommand \libusetikzlibrary [2] [] {
2132   \cs_if_exist:NF \usetikzlibrary {
2133     \msg_error:nnx{stex}{error/notikz}{\tl_to_str:n{\libusetikzlibrary}}
2134   }
2135   \tl_if_empty:NTF{#1}{
2136     \__stex_inputs_usetikzlibrary:n{#2}

```

```

2137   }{
2138     \stex_in_archive:nn{#1}{\__stex_inputs_usetikzlibrary:n{#2}}
2139   }
2140 }
2141
2142 \cs_new_protected:Nn \__stex_inputs_usetikzlibrary:n{
2143   \__stex_inputs_up_archive:nn{tikzlibrary#1}{code.tex}
2144   \int_compare:nNnTF {\seq_count:N \l__stex_inputs_libinput_files_seq} = 1 {
2145     \exp_args:Nne \__stex_inputs_usetikzlibrary_i:nn{#1}{ \seq_item:Nn \l__stex_inputs_libi
2146   }{
2147     \msg_fatal:nnnn{stex}{error/nofile}{\libusetikzlibrary}{tikzlibrary#1.code.tex}
2148   }
2149 }
2150
2151 \cs_new_protected:Nn \__stex_inputs_usetikzlibrary_i:nn {
2152   \pgfkeys@spdef\pgf@temp{#1}
2153   \expandafter\ifx\csname tikz@library@\pgf@temp @loaded\endcsname\relax%
2154   \expandafter\global\expandafter\let\csname tikz@library@\pgf@temp @loaded\endcsname=\pgfu
2155   \expandafter\edef\csname tikz@library@#1@atcode\endcsname{\the\catcode'\@}
2156   \expandafter\edef\csname tikz@library@#1@barcode\endcsname{\the\catcode'\|}
2157   \expandafter\edef\csname tikz@library@#1@dollarcode\endcsname{\the\catcode'\$}
2158   \catcode'\@=11
2159   \catcode'\|=12
2160   \catcode'\$=3
2161   \pgfutil@InputIfFileExists{#2}{\{}}{\}
2162   \catcode'\@=\csname tikz@library@#1@atcode\endcsname
2163   \catcode'\|=\csname tikz@library@#1@barcode\endcsname
2164   \catcode'\$=\csname tikz@library@#1@dollarcode\endcsname
2165 }

```

(End of definition for \libusetikzlibrary. This function is documented on page 140.)

sfunction

\addmhbibresource

```

2166 \newcommand \addmhbibresource [2][] {
2167   \stex_in_archive:nn{#1}{
2168     \__stex_inputs_up_archive:nn{#2}{bib}
2169     \stex_debug:nn{lib}{Result:-\seq_use:Nn \l__stex_inputs_libinput_files_seq ,}
2170     \seq_if_empty:NTF \l__stex_inputs_libinput_files_seq {
2171       \msg_error:nnnn{stex}{error/nofile}{\addmhbibresource}{#2.bib}
2172     }{
2173       \seq_map_function:NN \l__stex_inputs_libinput_files_seq \addbibresource
2174     }
2175   }
2176 }

```

(End of definition for \addmhbibresource. This function is documented on page 141.)

sfunction

__stex_inputs_up_archive:nn Iterates up the current archive's directory in search for lib files with name #1.#2

```

2177 \cs_new_protected:Nn \__stex_inputs_up_archive:nn {
2178   \tl_if_exist:NF \l_stex_current_archive {
2179     \msg_error:nnn{stex}{error/notinarchive}{\libinput
2180   }

```

```

2181 \seq_clear:N \l__stex_inputs_libinput_files_seq
2182 \seq_set_split:Nne \l__stex_inputs_path_seq / {\stex_archive_path:N \l_stex_current_archi
2183 \seq_set_split:Nne \l__stex_inputs_id_seq / {\stex_archive_id:N \l_stex_current_archive}
2184 \seq_map_inline:Nn \l__stex_inputs_id_seq {
2185   \seq_pop_right:NN \l__stex_inputs_path_seq \l_tmpa_tl
2186 }
2187 \stex_debug:nn{lib}{Checking~#1~in~\seq_use:Nn \l__stex_inputs_path_seq /}
2188
2189 \bool_while_do:nn { ! \seq_if_empty_p:N \l__stex_inputs_id_seq }{
2190   \str_set:Ne \l__stex_inputs_path_str {\stex_file_use:N \l__stex_inputs_path_seq / meta-
2191   \stex_debug:nn{lib}{Checking~\l__stex_inputs_path_str}
2192   \IfFileExists{ \l__stex_inputs_path_str }{
2193     \exp_args:NNo \seq_if_in:NnF \l__stex_inputs_libinput_files_seq \l__stex_inputs_path_
2194     \seq_put_right:No \l__stex_inputs_libinput_files_seq \l__stex_inputs_path_str
2195   }
2196 }{}
2197 \seq_pop_left:NN \l__stex_inputs_id_seq \l__stex_inputs_path_str
2198 \seq_put_right:No \l__stex_inputs_path_seq \l__stex_inputs_path_str
2199 }
2200
2201 \str_set:Ne \l__stex_inputs_path_str {\stex_file_use:N \l__stex_inputs_path_seq / lib / #
2202 \stex_debug:nn{lib}{Checking~\l__stex_inputs_path_str}
2203 \IfFileExists{ \l__stex_inputs_path_str }{
2204   \exp_args:NNo \seq_if_in:NnF \l__stex_inputs_libinput_files_seq \l__stex_inputs_path_
2205   \seq_put_right:No \l__stex_inputs_libinput_files_seq \l__stex_inputs_path_str
2206 }
2207 }{}
2208 }

```

(End of definition for `\__stex_inputs_up_archive:nn`.)

`\mhgraphics`
`\cmhgraphics`

```

2209 \ltx@ifpackageloaded{graphicx}{\use:n}{\AtEndOfPackageFile{graphicx}}{
2210   \define@key{Gin}{archive}{
2211     \stex_in_archive:nn{#1}{}
2212     \str_set:Ne \Gin@mhrepos {
2213       \stex_source_path:nn{#1}{}
2214     }
2215   }
2216   \providecommand\mhgraphics[2][]{
2217     \str_set:Ne \Gin@mhrepos {
2218       \stex_source_path:n{}
2219     }
2220     \setkeys{Gin}{#1}
2221     \includegraphics[#1]{ \Gin@mhrepos / #2 }
2222   }
2223   \providecommand\cmhgraphics[2][]{\begin{center}\mhgraphics[#1]{#2}\end{center}}
2224 }

```

(End of definition for `\mhgraphics` and `\cmhgraphics`. These functions are documented on page 141.)

`sfunction`

`\lstinputmhlisting`
`\clstinputmhlisting`

```

2225 \ltx@ifpackageloaded{listings}{\use:n}{\AtEndOfPackageFile{listings}}{

```

```

2226 \define@key{lst}{archive}{
2227   \stex_in_archive:nn{#1}{}
2228   \str_set:Ne \lst@mhrepos{
2229     \stex_source_path:nn{#1}{}
2230   }
2231 }
2232 \newcommand\lstinputmhlisting[2][]{\%
2233   \str_set:Ne \lst@mhrepos{
2234     \stex_source_path:n{}}
2235 }
2236 \setkeys{lst}{#1}%
2237 \lstinputlisting[#1]{\lst@mhrepos / #2}}
2238 \newcommand\clstinputmhlisting[2][]{\begin{center}\lstinputmhlisting[#1]{#2}\end{center}}
2239 }

```

(End of definition for \lstinputmhlisting and \clstinputmhlisting. These functions are documented on page 141.)

sfunction

\mhtikzinput
\cmhtikzinput

```

2240 \ltx@ifpackageloaded{tikzinput}{\use:n}{\AtEndOfPackageFile{tikzinput}}{
2241   \define@key{Gin}{archive}{
2242     \stex_in_archive:nn{#1}{}
2243     \str_set:Ne \Gin@mhrepos{
2244       \stex_source_path:nn{#1}{}
2245     }
2246     \str_set:Ne \l__stex_inputs_gin_repo_str {#1}
2247   }
2248   \newcommand\mhtikzinput[2][]{\%
2249     \str_clear:N \l__stex_inputs_gin_repo_str
2250     \str_set:Ne \Gin@mhrepos{
2251       \stex_source_path:n{}}
2252   }
2253   \setkeys{Gin}{#1}%
2254   \exp_args:No \stex_in_archive:nn \l__stex_inputs_gin_repo_str {
2255     \tikzinput[#1]{\Gin@mhrepos / #2}
2256   }
2257 }
2258 \newcommand\cmhtikzinput[2][]{\begin{center}\mhtikzinput[#1]{#2}\end{center}}
2259 }

```

(End of definition for \mhtikzinput and \cmhtikzinput. These functions are documented on page 141.)

sfunction

Chapter 31

Modules

```
2260 (@@=stex_modules)
```

A module consists of four separate macros, storing its *symbols*, *incoming morphisms* (e.g. `\importmodules`), *notations*, and “initialization code”, respectively:

```
2261 \cs_new:Nn \__stex_modules_macro_short:nnn {
2262   #1 ? #2 ? #3
2263 }
2264 \cs_new:Nn \_stex_symbol_macro:N {
2265   c_stex_module_ \exp_after:wN \__stex_modules_macro_short:nnn #1 _symbols_prop
2266 }
2267 \cs_new:Nn \_stex_symbol_macro:n {
2268   c_stex_module_ \__stex_modules_macro_short:nnn #1 _symbols_prop
2269 }
2270 \cs_new:Nn \_stex_morphisms_macro:N {
2271   c_stex_module_ \exp_after:wN \__stex_modules_macro_short:nnn #1 _morphisms_prop
2272 }
2273 \cs_new:Nn \_stex_morphisms_macro:n {
2274   c_stex_module_ \__stex_modules_macro_short:nnn #1 _morphisms_prop
2275 }
2276 \cs_new:Nn \_stex_notations_macro:N {
2277   c_stex_module_ \exp_after:wN \__stex_modules_macro_short:nnn #1 _notations_prop
2278 }
2279 \cs_new:Nn \_stex_notations_macro:n {
2280   c_stex_module_ \__stex_modules_macro_short:nnn #1 _notations_prop
2281 }
2282 \cs_new:Nn \_stex_module_code_macro:N {
2283   c_stex_module_ \exp_after:wN \__stex_modules_macro_short:nnn #1 _code
2284 }
2285 \cs_new:Nn \_stex_module_code_macro:n {
2286   c_stex_module_ \__stex_modules_macro_short:nnn #1 _code
2287 }
```

`\l_stex_current_module_uri` initially undefined.

(End of definition for `\l_stex_current_module_uri`. This variable is documented on page 142.)

`\l_stex_all_modules_seq`

```
2288 \seq_new:N \l_stex_all_modules_seq
```

(End of definition for `\l_stex_all_modules_seq`. This variable is documented on page 142.)

smodule (*env*.)

```

2289 \tl_new:N \l_stex_metatheory_uri
2290
2291 \stex_keys_define:nnnn{smodule}{
2292   \str_clear:N \l_stex_key_sig_str
2293 }{
2294   meta .code:n = {
2295     \tl_if_empty:nTF {#1}{
2296       \tl_clear:N \l_stex_metatheory_uri
2297     }{
2298       \stex_uri_from_pair:Nn \l_stex_metatheory_uri { #1 }
2299     }
2300   },
2301   %ns .code:n = {
2302     % \stex_uri_resolve:Ne \l_stex_current_ns_uri { #1 }
2303     %} ,
2304   %lang .code:n = {
2305     % \stex_set_language:n { #1 }
2306     %} ,
2307   sig .str_set_x:N = \l_stex_key_sig_str ,
2308   %creators .code:n = {} , % todo ?
2309   %contributors .code:n = {} , % todo ?
2310 }{id, title, style, deprecate}
2311
2312 \stex_if_html_backend:T {
2313   \cs_new_protected:Nn \__stex_modules_html_annots: {
2314     \exp_args:Ne \stex_annotate_env {
2315       data-ftml-module={\stex_module_uri_name:N \l_stex_current_module_uri}
2316       %data-ftml-language={ \l_stex_current_language_str},
2317       \str_if_empty:NF\l_stex_key_sig_str {, data-ftml-signature={\l_stex_key_sig_str}}
2318       \tl_if_empty:NF \l_stex_metatheory_uri {,
2319         data-ftml-metatheory={\stex_use_module_uri:N \l_stex_metatheory_uri}
2320       }
2321     }
2322     \stex_annotate_invisible:n{}
2323   }
2324 }
2325
2326 \stex_new_stylable_env:nnnnnn {module} {0{} m}{
2327   \stex_keys_set:nn { smodule }{ #1 }
2328   \tl_set_eq:NN \thistitle \l_stex_key_title_tl
2329   \tl_if_empty:NF \thistitle {
2330     \exp_args:No \stexdoctitle \thistitle
2331   }
2332   \stex_module_setup:n { #2 }
2333
2334   \stex_if_do_html:T \__stex_modules_html_annots:
2335   \stex_if_smsmode:F {
2336     \str_set:Ne \thismoduleuri {\stex_use_module_uri:N \l_stex_current_module_uri }
2337     \str_set:Nn \thismodulename {#2}
2338     \stex_style_apply:
2339   }
2340   \stex_smsmode_do:
2341 }{

```

```

2342 \stex_close_module:
2343 \stex_if_smsmode:F \stex_style_apply:
2344 \stex_if_do_html:T{ \endstex_annotate_env }
2345 }{}{}{s}

```

`\stex_module_setup:n`

```

2346 \cs_new_protected:Npn \stex_module_setup:n {
2347   \stex_if_in_module:TF \__stex_modules_nested:n \__stex_modules_top:n
2348 }
2349
2350 \cs_new_protected:Nn \__stex_modules_top:n {
2351   \str_if_empty:NTF \l_stex_key_sig_str
2352     \stex_module_setup_top_nosig:n \__stex_modules_top_sig:n {#1}
2353   \g_stex_every_module_tl
2354   \tl_if_empty:NF \l_stex_metatheory_uri {
2355     \stex_debug:nn{modules}{loading~metatheory~\stex_use_module_uri:N \l_stex_metatheory_uri}
2356     \__stex_modules_load_meta:
2357   }
2358 }

```

(End of definition for \stex_module_setup:n. This function is documented on page 142.)

sfunction

`\stex_module_setup_top_nosig:n`

```

2359 \cs_new_protected:Nn \stex_module_setup_top_nosig:n {
2360   \stex_metagroup_new:
2361   \tl_set:Ne \l__stex_modules_uri { \stex_new_module_uri:n {#1} }
2362   \stex_debug:nn{module}{URI:\stex_use_module_uri:N \l__stex_modules_uri}
2363   \exp_args:No \stex_if_module_exists:NTF \l__stex_modules_uri {
2364     \stex_debug:nn{modules}{(already exists)}
2365   }{
2366     \tl_gclear:c{ \stex_module_code_macro:N \l__stex_modules_uri }
2367     \prop_gclear:c{ \stex_morphisms_macro:N \l__stex_modules_uri }
2368     \prop_gclear:c{ \stex_symbol_macro:N \l__stex_modules_uri }
2369     \prop_gclear:c{ \stex_notations_macro:N \l__stex_modules_uri }
2370   }
2371   \tl_set_eq:NN \l_stex_current_module_uri \l__stex_modules_uri
2372   \seq_put_right:No \l_stex_all_modules_seq \l__stex_modules_uri
2373 }

```

(End of definition for \stex_module_setup_top_nosig:n. This function is documented on page 142.)

sfunction

`\__stex_modules_load_meta:` loads the metatheory:

```

2374 \bool_new:N \l_stex_in_meta_bool
2375
2376 \cs_new_protected:Nn \__stex_modules_load_meta: {
2377   \exp_after:wN \stex_execute_in_module:n \exp_after:wN {
2378     \exp_after:wN \__stex_modules_load_meta_i:n \exp_after:wN { \l_stex_metatheory_uri }
2379   }
2380 }
2381
2382 \cs_new_protected:Nn \__stex_modules_load_meta_i:n {
2383   \bool_if:NTF \l_stex_in_meta_bool {

```



```

2384 \stex_activate_module:n {#1}
2385 }{
2386 \bool_set_true:N \l_stex_in_meta_bool
2387 \stex_activate_module:n {#1}
2388 \bool_set_false:N \l_stex_in_meta_bool
2389 }
2390 }

```

(End of definition for _stex_modules_load_meta:.)

_stex_modules_top_sig:n sets up a new module with a signature:

```

2391 \cs_new_protected:Nn \_stex_modules_top_sig:n {
2392 \stex_debug:nn{modules}{Signature:\l_stex_key_sig_str}
2393 \stex_metagroup_new:
2394 \tl_set:Nc \l__stex_modules_uri { \stex_new_module_uri:n {#1} }
2395 \stex_debug:nn{module}{URI:\stex_use_module_uri:N \l__stex_modules_uri}
2396 \stex_if_module_exists:NTF \l__stex_modules_uri {
2397 \stex_debug:nn{modules}{(already exists)}
2398 \stex_activate_module:N \l__stex_modules_uri
2399 }{
2400 \stex_debug:nn{modules}{(needs loading)}
2401 \_stex_modules_load_sig:
2402 }
2403 \tl_set_eq:NN \l_stex_current_module_uri \l__stex_modules_uri
2404 }
2405
2406 \cs_new_protected:Nn \_stex_modules_load_sig: {
2407 \stex_file_split_off_lang:NN \l__stex_modules_sigfile \g_stex_current_file
2408 \tl_set:Nc \l__stex_modules_sig {
2409 \exp_args:NNo \stex_document_uri_with_language:Nn
2410 \l_stex_current_document_uri \l_stex_key_sig_str
2411 }
2412 \stex_debug:nn{modules}{Loading~signature~\stex_use_document_uri:N \l__stex_modules_sig;~
2413 \exp_args:NNe \stex_file_in_smsmode:Nn \l__stex_modules_sig {
2414 \stex_file_use:N \l__stex_modules_sigfile . \l_stex_key_sig_str . tex
2415 }
2416 \stex_activate_module:N \l__stex_modules_uri
2417 }

```

(End of definition for _stex_modules_top_sig:n.)

_stex_modules_nested:n sets up a new nested module:

```

2418 \cs_new_protected:Nn \_stex_modules_nested:n {
2419 \stex_metagroup_new:
2420 \stex_debug:nn{module}{Nested~module~#1~in~\stex_use_module_uri:N \l_stex_current_module_uri}
2421 \tl_set:Nc \l__stex_modules_uri { \stex_new_module_uri:n{#1} }
2422 \stex_debug:nn{module}{URI:\stex_use_module_uri:N \l__stex_modules_uri}
2423 \exp_args:No \stex_if_module_exists:NTF \l__stex_modules_uri {
2424 \stex_debug:nn{modules}{(already exists)}
2425 }{
2426 \tl_gclear:c{ \_stex_module_code_macro:N \l__stex_modules_uri }
2427 \prop_gclear:c{ \_stex_morphisms_macro:N \l__stex_modules_uri }
2428 \prop_gclear:c{ \_stex_symbol_macro:N \l__stex_modules_uri }
2429 \prop_gclear:c{ \_stex_notations_macro:N \l__stex_modules_uri }
2430 }

```

```

2431 \tl_set_eq:NN \l_stex_current_module_uri \l__stex_modules_uri
2432 \seq_put_left:No \l_stex_all_modules_seq \l__stex_modules_uri
2433 }

```

(End of definition for `\__stex_modules_nested:n`.)

`\stex_close_module:`

```

2434 \cs_new_protected:Nn \stex_close_module: {
2435   \bool_if:NT \c_stex_persist_write_mode_bool \__stex_modules_persist_module:
2436   \stex_debug:nn{module}{
2437     Closing~module~\stex_use_module_uri:N \l_stex_current_module_uri^^J
2438     Code:~\cs_meaning:c{ \_stex_module_code_macro:N \l_stex_current_module_uri }^^J
2439     Imports:~ \exp_after:wN \prop_to_keyval:N \cs:w \_stex_morphisms_macro:N \l_stex_curr
2440     Declarations:~ \exp_after:wN \prop_to_keyval:N \cs:w \_stex_symbol_macro:N \l_stex_curr
2441     Notations:~ \exp_after:wN \prop_to_keyval:N \cs:w \_stex_notations_macro:N \l_stex_curr
2442   }
2443 }

```

Persist and restore modules in the `.sms`-file:

```

2444 \cs_new_protected:Nn \__stex_modules_persist_module: {
2445   \stex_persist:e {
2446     \__stex_modules_restore_module:nnnn {\l_stex_current_module_uri}{
2447       \exp_after:wN \prop_to_keyval:N \cs:w
2448       \_stex_morphisms_macro:N \l_stex_current_module_uri
2449       \cs_end:
2450     }{
2451       \exp_after:wN \prop_to_keyval:N \cs:w
2452       \_stex_symbol_macro:N \l_stex_current_module_uri
2453       \cs_end:
2454     }{
2455       \exp_after:wN \exp_after:wN \exp_after:wN \exp_not:n
2456       \exp_after:wN \exp_after:wN \exp_after:wN
2457       { \cs:w \_stex_module_code_macro:N \l_stex_current_module_uri \cs_end: }
2458     }{}
2459     \prop_map_function:cN{\_stex_notations_macro:N \l_stex_current_module_uri}
2460     \__stex_modules_persist_not:nn
2461     \exp_not:N \STEXRestoreNotsEnd {}
2462   }
2463 }
2464
2465 \cs_new:Nn \__stex_modules_persist_not:nn {
2466   \exp_not:n{#2}
2467 }
2468
2469
2470 \cs_new:Nn \__stex_modules_stringify_uri:nnn {
2471   {\tl_to_str:n{#1}}
2472   {\tl_to_str:n{#2}}
2473   {\tl_to_str:n{#3}}
2474 }
2475 \cs_new:Nn \__stex_modules_stringify_uri:nnnn {
2476   {\tl_to_str:n{#1}}
2477   {\tl_to_str:n{#2}}
2478   {\tl_to_str:n{#3}}
2479   {\tl_to_str:n{#4}}

```

```

2480 }
2481
2482 \cs_new_protected:Nn \__stex_modules_restore_module:nnnn {
2483   \tl_set:Ne \l__stex_modules_uri { \__stex_modules_stringify_uri:nnn #1 }
2484   \stex_debug:nn{persist}{restoring~\stex_use_module_uri:N \l__stex_modules_uri}
2485   \prop_gset_from_keyval:cn{ \stex_morphisms_macro:N \l__stex_modules_uri }{#2}
2486   \prop_gset_from_keyval:cn{ \stex_symbol_macro:N \l__stex_modules_uri }{#3}
2487   \prop_map_inline:cn{ \stex_symbol_macro:N \l__stex_modules_uri }{
2488     \stex_ref_new_symbol:n{#1?##1}
2489   }
2490   \def \l_tmpa_tl { #4 }
2491   \exp_args:Nno \tl_gset:cn{ \stex_module_code_macro:N \l__stex_modules_uri } \l_tmpa_tl
2492   \prop_gc_clear:c{ \stex_notations_macro:N \l__stex_modules_uri }
2493   \group_begin:
2494     \catcode'_=8\relax
2495     \catcode'_{=12\relax
2496     \__stex_modules_restore_not:s:n
2497   }
2498
2499 \quark_new:N \STEXRestoreNotsEnd
2500
2501 \cs_new_protected:Nn \__stex_modules_restore_not:s:n {
2502   \__stex_modules_restore_not:s_i:n
2503 }
2504
2505 \cs_new_protected:Nn \__stex_modules_restore_not:s_i:n {
2506   \tl_if_eq:nnTF{#1}{\STEXRestoreNotsEnd}{
2507     \group_end:
2508   }{
2509     \__stex_modules_restore_not:s_ii:nnnn {#1}
2510   }
2511 }
2512
2513 \cs_new_protected:Nn \__stex_modules_restore_not:s_ii:nnnn {
2514   \tl_set:Nn \l__stex_modules_tl {{#4}{#5}}
2515   % eliminate ## => #
2516   \exp_after:wN \def \exp_after:wN \l__stex_modules_tl \exp_after:wN {\l__stex_modules_tl}
2517   \exp_args:NNe\use:nn\prop_gput:cnn{
2518     { \stex_notations_macro:N \l__stex_modules_uri }
2519     { \stex_use_symbol_uri:n{#1}\tl_to_str:n{?#2}}{
2520       { \__stex_modules_stringify_uri:nnnn #1 } { \tl_to_str:n{#2}} { #3 }
2521       \exp_args:No \exp_not:n \l__stex_modules_tl
2522     }
2523   }
2524   \__stex_modules_restore_not:s_i:n
2525 }

```

(End of definition for \stex_close_module:. This function is documented on page 142.)

sfunction

\stex_every_module:n

```

2526 \tl_new:N \g_stex_every_module_tl
2527 \cs_new_protected:Nn \stex_every_module:n {
2528   \tl_gput_right:Nn \g_stex_every_module_tl { #1 }
2529 }

```

(End of definition for `\stex_every_module:n`. This function is documented on page 142.)

sfunction

`\stex_do_up_to_module:n`

`\stex_do_up_to_module:e`

```
2530 \cs_new_protected:Nn \stex_do_up_to_module:n {
2531   \stex_metagroup_do_in:n {#1}
2532 }
2533 \cs_generate_variant:Nn \stex_do_up_to_module:n {e}
```

(End of definition for `\stex_do_up_to_module:n`. This function is documented on page 142.)

sfunction

`\stex_execute_in_module:n`

`\stex_execute_in_module:e`

```
2534 \cs_new_protected:Nn \stex_execute_in_module:n { \stex_if_in_module:TF {
2535   \tl_gput_right:cn { \_stex_module_code_macro:N \l_stex_current_module_uri } { #1 }
2536   \stex_debug:nn{modules}{extending~activation~for~\stex_use_module_uri:N \l_stex_current_m
2537   \stex_do_up_to_module:n { #1 }
2538 }{ #1 }}
2539 \cs_generate_variant:Nn \stex_execute_in_module:n {e}
```

(End of definition for `\stex_execute_in_module:n`. This function is documented on page 142.)

sfunction

`\stex_if_in_module_p:`

`\stex_if_in_module:TF`

```
2540 \prg_new_conditional:Nnn \stex_if_in_module: {p, T, F, TF} {
2541   \tl_if_exist:NTF \l_stex_current_module_uri
2542   \prg_return_true: \prg_return_false:
2543 }
```

(End of definition for `\stex_if_in_module:TF`. This function is documented on page 143.)

sfunction

`\stex_if_module_exists_p:N`

`\stex_if_module_exists:NTF`

`\stex_if_module_exists_p:n`

`\stex_if_module_exists:nTF`

```
2544 \prg_new_conditional:Nnn \stex_if_module_exists:N {p, T, F, TF} {
2545   \tl_if_exist:cTF { \_stex_module_code_macro:N #1 }
2546   \prg_return_true: \prg_return_false:
2547 }
2548 \prg_new_conditional:Nnn \stex_if_module_exists:n {p, T, F, TF} {
2549   \tl_if_exist:cTF { \_stex_module_code_macro:n {#1} }
2550   \prg_return_true: \prg_return_false:
2551 }
```

(End of definition for `\stex_if_module_exists:NTF` and `\stex_if_module_exists:nTF`. These functions are documented on page 143.)

sfunction

`\stex_activate_module:N`

`\stex_activate_module:n`

```
2552 \cs_new_protected:Nn \stex_activate_module:N {
2553   \exp_args:No \stex_activate_module:n #1
2554 }
2555
2556 \cs_new_protected:Nn \stex_activate_module:n {
2557   \seq_if_in:NnF \l_stex_all_modules_seq {#1} {
2558     \stex_debug:nn{modules}{Activating~module~\stex_use_module_uri:n{#1}}
```

```

2559 \seq_put_right:Nn \l_stex_all_modules_seq {#1}
2560 \stex_pseudogroup:nn{
2561   \tl_set:Nn \l_stex_current_module_uri {#1}
2562   \tl_if_exist:cF { \_stex_module_code_macro:N \l_stex_current_module_uri }{
2563     \msg_error:nnx{stex}{error/unknownmodule}{\stex_use_module_uri:n {#1} }
2564   }
2565   \_stex_activate_symbols:
2566   \_stex_activate_notations:
2567   \stex_debug:nn{modules}{Executing:~\expandafter\meaning\csname\_stex_module_code_macro:N \l_stex_current_module_uri }
2568   \use:c{ \_stex_module_code_macro:N \l_stex_current_module_uri }
2569 }{
2570   \stex_pseudogroup_restore:N \l_stex_current_module_uri
2571 }
2572 \stex_debug:nn{modules}{Activated~module~\stex_use_module_uri:n {#1} }
2573 }
2574 }

```

(End of definition for `\stex_activate_module:N` and `\stex_activate_module:n`. These functions are documented on page 143.)

sfunction

`\setmetatheory`

```

2575 \NewDocumentCommand \setmetatheory {0{} m}{
2576   \stex_debug:nn{metatheory}{Setting~metatheory~[#1]#2}
2577   \stex_uri_from_pair:Nnn \l_stex_import_uri { #1 }{ #2 }
2578   \stex_debug:nn{metatheory}{Here:\stex_use_module_uri:N \l_stex_import_uri
2579 }
2580   \tl_set_eq:NN \l_stex_metatheory_uri \l_stex_import_uri
2581   \begingroup
2582     \stex_require_module:N \l_stex_metatheory_uri
2583   \endgroup
2584   \stex_smsmode_do:
2585 }

```

(End of definition for `\setmetatheory`. This function is documented on page 143.)

sfunction

`\STEXexport`

```

2586 \cs_new_protected:Npn \STEXexport {
2587   \ExplSyntaxOn
2588   \__stex_modules_export:n
2589 }
2590 \cs_new_protected:Nn \__stex_modules_export:n {
2591   \stex_ignore_spaces_and_pars:#1\ExplSyntaxOff
2592   \tl_gput_right:cn { \_stex_module_code_macro:N \l_stex_current_module_uri } {
2593     \stex_ignore_spaces_and_pars: #1
2594   }
2595   \stex_smsmode_do:
2596 }
2597
2598 \stex_deactivate_macro:Nn \STEXexport {module~environments}
2599 \stex_every_module:n {\stex_reactivate_macro:N \STEXexport}

```

(End of definition for `\STEXexport`. This function is documented on page 143.)

sfunction

```

\stex_structural_feature_module:nn
\stex_structural_feature_module_end:
2600 \cs_new_protected:Nn \stex_structural_feature_module:nn {
2601   \stex_if_do_html:TF {
2602     \exp_args:Nne \begin{stex_annotate_env} {
2603       data-ftml-feature-#2={#1}
2604       \str_if_empty:NF \l_stex_macroname_str {,
2605         data-ftml-macroname={\l_stex_macroname_str}
2606       }
2607     }
2608     \stex_annotate_invisible:n{ }
2609   }\group_begin:
2610   \stex_module_setup:n {#1}
2611 }
2612
2613 \cs_new_protected:Nn \stex_structural_feature_module_end: {
2614   \tl_gset_eq:NN \g_stex_last_feature_str \l_stex_current_module_str
2615   \stex_close_module:
2616   \stex_if_do_html:TF{
2617     \end{stex_annotate_env}
2618   }\group_end:
2619 }

(End of definition for \stex_structural_feature_module:nn and \stex_structural_feature_module_end:.)
These functions are documented on page 143.)
sfunction

```

31.1 SMS Mode

```

2620 <@@=stex_smsmode>

\stex_sms_allow:N

2621 \tl_new:N \g__stex_smsmode_allowed_tl
2622
2623 \cs_new_protected:Nn \stex_sms_allow:N {
2624   \tl_gput_right:Nn \g__stex_smsmode_allowed_tl {#1}
2625 }

(End of definition for \stex_sms_allow:N. This function is documented on page 144.)
sfunction

\stex_sms_allow_escape:N

2626 \tl_new:N \g__stex_smsmode_allowed_escape_tl
2627
2628 \cs_new_protected:Nn \stex_sms_allow_escape:N {
2629   \tl_gput_right:Nn \g__stex_smsmode_allowed_escape_tl {#1}
2630 }

(End of definition for \stex_sms_allow_escape:N. This function is documented on page 144.)
sfunction

\stex_sms_allow_env:n

2631 \seq_new:N \g__stex_smsmode_allowedenvs_seq
2632
2633 \cs_new_protected:Nn \stex_sms_allow_env:n {

```

```

2634 \seq_gput_right:Ne \g__stex_smsmode_allowedenvs_seq {\tl_to_str:n{#1}}
2635 }

```

(End of definition for \stex_sms_allow_env:n. This function is documented on page 144.)

sfunction

Some initial allowed macros:

```

2636 \stex_sms_allow:N \makeatletter
2637 \stex_sms_allow:N \makeatother
2638 \stex_sms_allow:N \ExplSyntaxOn
2639 \stex_sms_allow:N \ExplSyntaxOff
2640 \stex_sms_allow:N \rustexBREAK
2641 \stex_sms_allow_env:n{smodule}
2642 \stex_sms_allow_escape:N \setmetatheory
2643 \stex_sms_allow_escape:N \STEXexport

```

\stex_sms_allow_import:Nn

```

2644 \tl_new:N \g__stex_smsmode_allowed_import_tl
2645 \tl_new:N \g__stex_smsmode_import_setup_tl
2646
2647 \cs_new_protected:Nn \stex_sms_allow_import:Nn {
2648   \tl_gput_right:Nn \g__stex_smsmode_allowed_import_tl {#1}
2649   \tl_gput_right:Nn \g__stex_smsmode_import_setup_tl {#2}
2650 }

```

(End of definition for \stex_sms_allow_import:Nn. This function is documented on page 144.)

sfunction

\stex_sms_allow_import_env:nn

```

2651 \seq_new:N \g__stex_smsmode_allowed_import_env_seq
2652
2653 \cs_new_protected:Nn \stex_sms_allow_import_env:nn {
2654   \seq_gput_right:Ne \g__stex_smsmode_allowed_import_env_seq {\tl_to_str:n{#1}}
2655   \tl_gput_right:Nn \g__stex_smsmode_import_setup_tl {#2}
2656 }
2657
2658 %\stex_sms_allow_import_env:nn{smodule}{ }

```

(End of definition for \stex_sms_allow_import_env:nn. This function is documented on page 144.)

sfunction

\g_stex_sms_import_code

```

2659 \tl_new:N \g_stex_sms_import_code

```

(End of definition for \g_stex_sms_import_code. This variable is documented on page 144.)

\stex_if_smsmode_p:

\stex_if_smsmode:TF

```

2660 \bool_new:N \g__stex_smsmode_bool
2661
2662 \prg_new_conditional:Nnn \stex_if_smsmode: { p, T, F, TF } {
2663   \bool_if:NTF \g__stex_smsmode_bool \prg_return_true: \prg_return_false:
2664 }

```

(End of definition for \stex_if_smsmode:TF. This function is documented on page 144.)

sfunction

\stex_file_in_smsmode:Nn

```

2665
2666 \seq_new:N \l__stex_smsmode_cycles_seq
2667
2668 \cs_new_protected:Nn \stex_file_in_smsmode:Nn {
2669   \seq_gclear:N \l__stex_smsmode_importmodules_seq
2670   \seq_gclear:N \l__stex_smsmode_sigmodules_seq
2671   \tl_clear:N \g_stex_sms_import_code
2672   \seq_if_in:NnF \l__stex_smsmode_cycles_seq {#2} {
2673     \group_begin:
2674     \seq_push:Nn \l__stex_smsmode_cycles_seq {#2}
2675     \let \l_stex_metatheory_uri \c_stex_default_metatheory
2676     \seq_clear:N \l_stex_all_modules_seq
2677     \stex_undefine:N \l_stex_current_module_uri
2678     \cs_set:Npn \stex_check_term:nn ##1 ##2 {}
2679     \stex_debug:nn{sms}{Loading~#2~in~\stex_use_document_uri:N #1}
2680     \stex_with_file_hooks:Nnn #1 {#2}{
2681       \__stex_smsmode_in_smsmode:n {
2682         \group_begin:
2683         \let \__stex_smsmode_do_aux_curr:N \__stex_smsmode_do_aux_imports:N
2684         \g__stex_smsmode_import_setup_tl
2685         \__stex_smsmode_start_smsmode:n{#2}
2686         \group_end:
2687         %{
2688           \g_stex_sms_import_code
2689         }%
2690         %\group_begin:
2691         \let \__stex_smsmode_do_aux_curr:N \__stex_smsmode_do_aux_normal:N
2692         \__stex_smsmode_start_smsmode:n{#2}
2693         %\group_end:
2694       }
2695     }
2696   \group_end:
2697 }
2698
2699
2700 \cs_new_protected:Nn \__stex_smsmode_in_smsmode:n {
2701   \stex_suppress_html:n {
2702     \vbox_set:Nn \l_tmpa_box {
2703       \bool_set_true:N \g__stex_smsmode_bool
2704       #1
2705     }
2706   }
2707 }
2708
2709 \quark_new:N \q__stex_smsmode_break
2710
2711 \cs_new_protected:Nn \__stex_smsmode_start_smsmode:n {
2712   \everyeof{\q__stex_smsmode_break\exp_not:N}
2713   \let\stex_smsmode_do:\__stex_smsmode_smsmode_do:
2714   \exp_after:wN \exp_after:wN \exp_after:wN
2715   \stex_smsmode_do:
2716   \cs:w @ @ input\cs_end: "#1" \relax
2717 }

```


(End of definition for \stex_file_in_smsmode:Nn. This function is documented on page 144.)

sfunction

\stex_smsmode_do:

```

2718 \let\stex_smsmode_do:\relax
2719
2720 \cs_new_protected:Nn \__stex_smsmode_smsmode_do: { \__stex_smsmode_do:w }
2721
2722 \cs_new_protected:Npn \__stex_smsmode_do:w #1 {
2723   \exp_args:Ne \tl_if_empty:nTF { \tl_tail:n{ #1 } }{
2724     %\__stex_smsmode_check_cs:NNn \__stex_smsmode_do_aux:N \__stex_smsmode_do:w { #1 }
2725     \__stex_smsmode_check_cs:N {#1}
2726   }{
2727     \__stex_smsmode_do:w #1
2728   }
2729 }
2730
2731 \cs_new_protected:Nn \__stex_smsmode_check_cs:N {
2732   %\stex_debug:nn{temp}{Checking \exp_not:N#1 \relax}
2733   \exp_after:wN\if\exp_after:wN\relax\noexpand #1 ~
2734   \exp_after:wN \__stex_smsmode_do_aux:N \exp_after:wN#1\else
2735   \exp_after:wN \__stex_smsmode_do:w \fi
2736 }
2737
2738 %\cs_new_protected:Nn \__stex_smsmode_check_cs:NNn {
2739 %  \message{^^JHERE: \tl_to_str:n{#1~and~#2~and~#3}}
2740 %  \exp_after:wN\if\exp_after:wN\relax\exp_not:N#3
2741 %  \exp_after:wN#1\exp_after:wN#3\else
2742 %  \exp_after:wN#2\fi
2743 %}
2744
2745 \cs_new_protected:Nn \__stex_smsmode_do_aux:N {
2746   \cs_if_eq:NNF #1 \q__stex_smsmode_break {
2747     \__stex_smsmode_do_aux_curr:N #1
2748   }
2749 }
2750
2751 \cs_new_protected:Nn \__stex_smsmode_do_aux_normal:N {
2752   \tl_if_in:NnTF \g__stex_smsmode_allowed_tl {#1} {
2753     \stex_debug:nn{sms}{Executing~\tl_to_str:n{#1}}
2754     #1\__stex_smsmode_do:w
2755   }{
2756     \tl_if_in:NnTF \g__stex_smsmode_allowed_escape_tl {#1} {
2757       \stex_debug:nn{sms}{Executing~escaped~\tl_to_str:n{#1}}
2758       #1
2759     }{
2760       \cs_if_eq:NNTF \begin #1 {
2761         \__stex_smsmode_check_begin:Nn \g__stex_smsmode_allowedenvs_seq
2762       }{
2763         \cs_if_eq:NNTF \end #1 {
2764           \__stex_smsmode_check_end:Nn \g__stex_smsmode_allowedenvs_seq
2765         }{
2766           \__stex_smsmode_do:w
2767         }

```

```

2768     }
2769   }
2770 }
2771 }
2772
2773 \cs_new_protected:Nn \__stex_smsmode_do_aux_imports:N {
2774   \tl_if_in:NnTF \g__stex_smsmode_allowed_import_tl {#1} {
2775     \stex_debug:nn{sms}{Executing~\tl_to_str:n{#1}~in~import}
2776     #1
2777   }{
2778     \cs_if_eq:NNTF \begin #1 {
2779       \__stex_smsmode_check_begin:Nn \g__stex_smsmode_allowed_import_env_seq
2780     }{
2781       \cs_if_eq:NNTF \end #1 {
2782         \__stex_smsmode_check_end:Nn \g__stex_smsmode_allowed_import_env_seq
2783       }{
2784         \__stex_smsmode_do:w
2785       }
2786     }
2787   }
2788 }
2789
2790 \cs_new_protected:Nn \__stex_smsmode_check_begin:Nn {
2791   \seq_if_in:NeTF #1 { \tl_to_str:n{#2} }{
2792     \stex_debug:nn{sms}{Environment~#2}
2793     \begin{#2}
2794   }{
2795     \__stex_smsmode_do:w
2796   }
2797 }
2798 \cs_new_protected:Nn \__stex_smsmode_check_end:Nn {
2799   \seq_if_in:NeTF #1 { \tl_to_str:n{#2} }{
2800     \stex_debug:nn{sms}{End~Environment~#2}
2801     \end{#2}\__stex_smsmode_do:w
2802   }{
2803     \__stex_smsmode_do:w
2804   }
2805 }

```

(End of definition for `\stex_smsmode_do:.` This function is documented on page 144.)

sfunction

31.2 Inheritance and Morphisms

```

2806 <@@=stex_import>

\stex_require_module:N
\stex_require_module_noerr:N
\stex_require_module_noerr:n
2807 \cs_new_protected:Nn \stex_require_module:N {
2808   \stex_if_module_exists:NF #1 {
2809     \__stex_import_load_module:NF #1 {
2810       \msg_error:nnx{stex}{error/unknownmodule}{\stex_use_module_uri:N #1}
2811     }
2812   }
2813   \stex_activate_module:N #1

```

```

2814 }
2815
2816 \cs_new_protected:Nn \stex_require_module_noerr:N {
2817   \stex_if_module_exists:NF #1 {
2818     \__stex_import_load_module:NF #1 {}
2819     \str_if_empty:NF \l__stex_import_file_str {
2820       \stex_activate_module:N #1
2821     }
2822   }
2823 }
2824
2825 \cs_new_protected:Nn \stex_require_module_noerr:n {
2826   \tl_set:Nn \l__stex_import_uri { #1 }
2827   \stex_require_module_noerr:N \l__stex_import_uri
2828 }
2829
2830 \cs_new_protected:Npn \__stex_import_load_module:NF #1 #2 {
2831   \tl_set_eq:NN \l__stex_import_uri #1
2832   \tl_set:Ne \l__stex_import_archive_str { \stex_module_uri_archive:N #1 }
2833   \tl_set:Ne \l__stex_import_path_str { \stex_module_uri_path:N #1 }
2834   \tl_set:Ne \l__stex_import_name_str { \stex_module_uri_name:N #1 }
2835   \str_if_eq:NNTF \l__stex_import_archive_str \c_stex_no_archive_str {
2836     \str_set_eq:NN \l__stex_import_pre_str \l__stex_import_path_str
2837     \__stex_import_load_check:n {#2}
2838   }{
2839     \exp_args:No \stex_in_archive:nn \l__stex_import_archive_str {
2840       \str_set:Ne \l__stex_import_pre_str {
2841         \exp_args:Ne \stex_source_path:n{ \l__stex_import_path_str }
2842       }
2843       \exp_args:No \__stex_import_load_check:n {#2}
2844     }
2845   }
2846 }
2847
2848 \cs_new_protected:Nn \__stex_import_load_check:n {
2849   \str_clear:N \l__stex_import_file_str
2850   \str_set_eq:NN \l__stex_import_language_str \l_stex_current_language_str
2851   \__stex_import_check_file:nnn{ / \l__stex_import_name_str .tex }{}{
2852     \__stex_import_check_file:nnn{/ \l__stex_import_name_str . \l_stex_current_language_str
2853     \__stex_import_check_file:nnn{/ \l__stex_import_name_str .en.tex}{
2854       \str_set:Nn \l__stex_import_language_str {en}
2855     }{
2856       \__stex_import_load_check_i:n{#1}
2857     }
2858   }
2859 }
2860 \__stex_import_load_check_ii:
2861 }
2862
2863 \cs_new_protected:Nn \__stex_import_load_check_i:n {
2864   \exp_args:NNNo \seq_set_split:Nnn \l_tmpa_seq / \l__stex_import_path_str
2865   \seq_pop_right:NN \l_tmpa_seq \l__stex_import_name_str
2866   \str_set:Ne \l__stex_import_path_str { \seq_use:Nn \l_tmpa_seq /}
2867   \__stex_import_check_file:nnn{.tex}{}{

```

```

2868 \__stex_import_check_file:nnn{. \l_stex_current_language_str .tex}{ }{
2869 \__stex_import_check_file:nnn{.en.tex}{
2870 \str_set:Nn \l__stex_import_language_str {en}
2871 }{
2872 #1
2873 }
2874 }
2875 }
2876 }
2877
2878 \cs_new_protected:Nn \__stex_import_load_check_ii: {
2879 \str_if_empty:NF \l__stex_import_file_str {
2880 \stex_if_smsmode:TF{
2881 \exp_args:NNo \exp_args:Nne \str_if_eq:nnTF{\l__stex_import_file_str}{\stex_file_use:
2882 \stex_debug:nn{imports}{Skipping~current~file}
2883 }{
2884 \IfFileExists{ \l__stex_import_file_str }{
2885 \__stex_import_load_file:
2886 }{ }
2887 }
2888 }{
2889 \IfFileExists{ \l__stex_import_file_str }{
2890 \__stex_import_load_file:
2891 }{ }
2892 }
2893 }
2894 }
2895
2896 \cs_new_protected:Npn \__stex_import_check_file:nnn #1 #2 {
2897 \stex_debug:nn{imports}{Checking~ \l__stex_import_pre_str #1}
2898 \IfFileExists{ \l__stex_import_pre_str #1 }{
2899 \stex_debug:nn{imports}{Success}
2900 \str_set:Ne \l__stex_import_file_str { \l__stex_import_pre_str #1 }
2901 #2
2902 }
2903 }
2904
2905 \cs_new_protected:Nn \__stex_import_load_file: {
2906 \tl_set:Ne \l__stex_import_doc_uri {
2907 { \l__stex_import_archive_str }
2908 { \l__stex_import_path_str }
2909 { \l__stex_import_name_str }
2910 { \l__stex_import_language_str }
2911 {}
2912 }
2913 \exp_args:NNo \stex_file_in_smsmode:Nn \l__stex_import_doc_uri \l__stex_import_file_str
2914 \stex_if_module_exists:NF \l__stex_import_uri {
2915 \msg_error:nnx{stex}{error/unknownmodule}{\stex_use_module_uri:N \l__stex_import_uri}
2916 }
2917 }

```

(End of definition for `\stex_require_module:N`, `\stex_require_module_noerr:N`, and `\stex_require_module_noerr:n`. These functions are documented on page 145.)

sfunction

\usemodule

```

2918 \stex_new_stylable_cmd:nnnn {usemodule} { 0{} m } {
2919   \stex_uri_from_pair:Nnn \l_stex_import_uri { #1 }{ #2 }
2920   \stex_require_module:N \l_stex_import_uri
2921   \__stex_import_usemodule:
2922 }{\aftergroup\ignorespaces}
2923
2924 \cs_new_protected:Nn \__stex_import_usemodule: {
2925   \stex_debug:nn{usemodule}{Done.}
2926   \stex_if_do_html:T {
2927     \hbox{\stex_annotate_invisible:nn
2928       {data-ftml-usemodule=\stex_use_module_uri:N \l_stex_import_uri } {}}
2929   }
2930   \stex_if_smsmode:F{
2931     \group_begin:
2932     \str_set:Ne \thismoduleuri {\stex_use_module_uri:N \l_stex_import_uri }
2933     \str_set:Ne \thismodulename {\stex_module_uri_name:N \l_stex_import_uri}
2934     \tl_clear:N \thisstyle
2935     \stex_style_apply:
2936     \group_end:
2937   }
2938 }
2939 \stex_deactivate_macro:Nn \usemodule {document~environment}
2940 \AtBeginDocument{\stex_reactivate_macro:N \usemodule}

```

(End of definition for \usemodule. This function is documented on page 145.)

sfunction

\importmodule

```

2941 \stex_new_stylable_cmd:nnnn{importmodule} { 0{} m } {
2942   \__stex_import_import_module:nn {#1}{#2}
2943   \stex_smsmode_do:
2944 }{\aftergroup\ignorespaces}
2945
2946 \stex_deactivate_macro:Nn \importmodule {module~environments}
2947 \stex_every_module:n {\stex_reactivate_macro:N \importmodule}
2948 \stex_sms_allow_escape:N \importmodule
2949
2950 \cs_new_protected:Nn \__stex_import_import_module:nn {
2951   \stex_uri_from_pair:Nnn \l_stex_import_uri { #1 }{ #2 }
2952   \stex_require_module:N \l_stex_import_uri
2953   \__stex_import_import_module_i:n {#1}
2954 }
2955
2956 \cs_new_protected:Nn \__stex_import_import_module_i:n {
2957   \stex_execute_in_module:e{
2958     \stex_activate_module:n { \l_stex_import_uri }
2959   }
2960   \stex_add_morphism:nonn
2961     {}{\l_stex_import_uri}{import}{}
2962   \stex_if_do_html:T {
2963     \stex_annotate_invisible:nn
2964       {data-ftml-import=\stex_use_module_uri:N \l_stex_import_uri } {}
2965   }

```

```

2966 \stex_if_smsmode:F{
2967   \group_begin:
2968   \tl_set:Nn \thisarchive {#1}
2969   \str_set:Ne \thismoduleuri {\stex_use_module_uri:N \l_stex_import_uri }
2970   \str_set:Ne \thismodulename {\stex_module_uri_name:N \l_stex_import_uri}
2971   \tl_clear:N \thisstyle
2972   \stex_style_apply:
2973   \group_end:
2974 }
2975 }

```

In sms mode, we change the definition:

```

2976 \cs_new_protected:Nn \__stex_import_import_module_presms:nn {
2977   \stex_uri_from_pair:Nnn \l_stex_import_uri { #1 }{ #2 }
2978   \bool_if:NF \l_stex_relative_import_bool {
2979     \tl_gput_right:Ne \g_stex_sms_import_code {
2980       \stex_require_module_noerr:n { \l_stex_import_uri }
2981     }
2982   }
2983 }
2984
2985 \stex_sms_allow_import:Nn \importmodule {
2986   \stex_reactivate_macro:N \importmodule
2987   \let \__stex_import_import_module:nn \__stex_import_import_module_presms:nn
2988 }

```

(End of definition for `\importmodule`. This function is documented on page 145.)

sfunction

`\requiremodule`

```

2989 \stex_new_stylable_cmd:nnnn {requiremodule} { 0{} m } {
2990   \stex_uri_from_pair:Nnn \l_stex_import_uri { #1 }{ #2 }
2991   \stex_require_module:N \l_stex_import_uri
2992   \__stex_import_requiremodule:
2993 }{}%{\aftergroup\ignorespaces}
2994 \stex_deactivate_macro:Nn \requiremodule {module~environments}
2995 \stex_every_module:n {\stex_reactivate_macro:N \requiremodule}
2996
2997 \cs_new_protected:Nn \__stex_import_requiremodule: {
2998   \stex_debug:nn{requiremodule}{Done.}
2999   \stex_if_do_html:T {
3000     \hbox{\stex_annotate_invisible:nn
3001       {data-ftml-import=\stex_use_module_uri:N \l_stex_import_uri } {}}
3002   }
3003   \stex_if_smsmode:F{
3004     \group_begin:
3005     \str_set:Ne \thismoduleuri {\stex_use_module_uri:N \l_stex_import_uri }
3006     \str_set:Ne \thismodulename {\stex_module_uri_name:N \l_stex_import_uri}
3007     \tl_clear:N \thisstyle
3008     \stex_style_apply:
3009     \group_end:
3010   }
3011 }

```

(End of definition for `\requiremodule`. This function is documented on page 145.)

sfunction

`\stex_add_morphism:nnnn`

```

3012 \cs_new_protected:Nn \stex_add_morphism:nnnn {
3013   \exp_args:Nne \prop_gput:cnn{ \stex_morphisms_macro:N \l_stex_current_module_uri }{
3014     \tl_if_empty:nTF{#1}{[\stex_use_module_uri:n {#2}]}{#1}
3015   }{#1}{#2}{#3}{#4}}
3016 }
3017 \cs_generate_variant:Nn \stex_add_morphism:nnnn {nonn,oooe}

```

(End of definition for `\stex_add_morphism:nnnn`. This function is documented on page 145.)

sfunction

31.3 Theory Morphisms

3018 `<@@=stex_morphisms>`

`\stex_structural_feature_morphism:nnnnn`

`\stex_structural_feature_morphism_with_macros:nnnnn`

`\stex_structural_feature_morphism_end:`

```

3019 \tl_new:N \l_stex_current_domain_uri
3020 \bool_new:N \l__stex_morphisms_with_macros_bool
3021
3022 \cs_new_protected:Npn \stex_structural_feature_morphism_with_macros:nnnnn {
3023   \bool_set_true:N \l__stex_morphisms_with_macros_bool
3024   \__stex_morphisms_morphism:nnnnn
3025 }
3026 \cs_new_protected:Npn \stex_structural_feature_morphism:nnnnn {
3027   \bool_set_false:N \l__stex_morphisms_with_macros_bool
3028   \__stex_morphisms_morphism:nnnnn
3029 }
3030
3031 \cs_new_protected:Nn \__stex_morphisms_morphism:nnnnn {
3032   \tl_clear:N \l_stex_current_domain_uri
3033   \stex_debug:nn{morphisms}{new-morphism:{#1}{#2}{#3}{#4}{#5}}
3034   \tl_if_empty:nTF{#3}{
3035     \stex_get_mathstructure:n{#4}
3036     \tl_set_eq:NN \l_stex_current_domain_uri \l_stex_get_structure_module_uri
3037   }
3038   \tl_if_empty:NT \l_stex_current_domain_uri {
3039     \stex_uri_from_pair:Nnn \l_stex_import_uri { #3 }{ #4 }
3040     \group_begin:
3041     \stex_require_module:N \l_stex_import_uri
3042     \exp_args:Nne \use:nn \group_end: {
3043       \tl_set:Nn \exp_not:N\l_stex_current_domain_uri {\l_stex_import_uri}
3044     }
3045   }
3046   \tl_if_empty:nTF{#1}{
3047     \msg_error:nn{stex}{error/morphism-needs-name}
3048   }
3049   \str_set:Nn \l_tmpa_str {#1}
3050
3051   \stex_debug:nn{morphisms}{#2:~\l_tmpa_str;~for~\stex_use_module_uri:N \l_stex_current_dom
3052

```

```

3053 \str_set:Nn \l__stex_morphisms_feature_str {#2}
3054 \str_set_eq:NN \l_stex_feature_name_str \l_tmpa_str
3055
3056 \stex_if_do_html:TF {
3057   \begin{stex_annotate_env} {
3058     data-ftml-feature-#2={\l_tmpa_str},
3059     data-ftml-domain={\stex_use_module_uri:N \l_stex_current_domain_uri}
3060     #5
3061   }
3062   \stex_annotate_invisible:n{}
3063 }\group_begin:
3064 \__stex_morphisms_setup:
3065 \__stex_morphisms_reactivate:
3066 \stex_metagroup_new:
3067 }
3068
3069 \cs_new_protected:Nn \__stex_morphisms_setup: {
3070   \seq_clear:N \l_stex_morphism_symbols_seq
3071   \seq_clear:N \l_stex_morphism_renames_seq
3072   \seq_clear:N \l_stex_morphism_morphisms_seq
3073   \seq_clear:N \l_stex_morphism_assigns_seq
3074   \__stex_morphisms_do_decls:
3075   \exp_args:No \__stex_morphisms_do:n \l_stex_current_domain_uri
3076 }
3077
3078 \cs_new_protected:Nn \__stex_morphisms_do_decls: {
3079   \exp_args:No \stex_iterate_symbols:nn \l_stex_current_domain_uri {
3080     \exp_args:NNe \seq_put_right:Nn \l_stex_morphism_symbols_seq {
3081       { \stex_new_symbol_uri:nn{##1}{##3} }
3082       { {##2}{##4}{##5}{##6}{##7}{##8}##9 }
3083     }
3084   }
3085 }
3086
3087
3088 \cs_new_protected:Nn \__stex_morphisms_do:n {
3089   %\seq_clear:N \l__stex_morphisms_dones_seq
3090   \prop_map_inline:cn {\_stex_morphisms_macro:n{#1}}{
3091     %\seq_if_in:NnF \l__stex_morphisms_dones_seq {##2}{
3092       % \seq_push:Nn \l__stex_morphisms_dones_seq {##2}
3093       \stex_debug:nn{morphisms}{Doing ~ \tl_to_str:n{##2}}
3094       \__stex_morphisms_do_morph:nnnn ##2
3095     % }
3096   }
3097 }
3098
3099 \cs_new_protected:Nn \__stex_morphisms_do_morph:nnnn {
3100   \tl_if_empty:nF{#3}{
3101     \seq_put_right:Nn \l_stex_morphism_morphisms_seq {{#1}{#2}{#3}}
3102   }
3103   \__stex_morphisms_do:n{#2}
3104 }
3105
3106

```



```

3107 \cs_new_protected:Nn \stex_structural_feature_morphism_end: {
3108   \tl_gset_eq:NN \l_stex_current_domain_uri \l_stex_current_domain_uri
3109   \seq_gset_eq:NN \l_stex_morphism_symbols_seq \l_stex_morphism_symbols_seq
3110   \seq_gset_eq:NN \l_stex_morphism_renames_seq \l_stex_morphism_renames_seq
3111   \seq_gset_eq:NN \l_stex_morphism_assigns_seq \l_stex_morphism_assigns_seq
3112   \seq_gset_eq:NN \l_stex_morphism_morphisms_seq \l_stex_morphism_morphisms_seq
3113   \bool_gset_eq:NN \l__stex_morphisms_with_macros_bool \l__stex_morphisms_with_macros_bool
3114   \stex_if_do_html:TF{
3115     \end{stex_annotate_env}
3116   }\group_end:
3117   \__stex_morphisms_do_elaboration:
3118 }
3119
3120 \cs_new_protected:Nn \__stex_morphisms_do_elaboration: {
3121   \stex_debug:nn{morphisms}{
3122     Elaborating:^^J\meaning \l_stex_morphism_symbols_seq
3123     ^^J^^J
3124     Renamings:^^J
3125     \meaning \l_stex_morphism_renames_seq
3126     ^^J^^J
3127     Assignments:^^J
3128     \meaning \l_stex_morphism_assigns_seq
3129   }
3130
3131   \seq_map_inline:Nn \l_stex_morphism_symbols_seq {
3132     \__stex_morphisms_elab:nn ##1
3133   }
3134
3135   \stex_add_morphism:oooo
3136     \l_stex_feature_name_str
3137     \l_stex_current_domain_uri
3138     \l__stex_morphisms_feature_str
3139     {\seq_map_function:NN \l_stex_morphism_renames_seq \__stex_morphisms_rename:n}
3140 }
3141
3142 \cs_new:Nn \__stex_morphisms_rename:n{
3143   \__stex_morphisms_rename:nn #1
3144 }
3145
3146 \cs_new:Nn \__stex_morphisms_rename:nn{
3147   {#1}{\use_ii:nn#2}
3148 }
3149
3150 \cs_new_protected:Nn \__stex_morphisms_elab:nn {
3151   \tl_clear:N \l__stex_morphisms_tmp
3152   \seq_map_inline:Nn \l_stex_morphism_renames_seq {
3153     \__stex_morphisms_elab_check_rename:nnn{#1}##1
3154   }
3155   \tl_if_empty:NTF \l__stex_morphisms_tmp {
3156     \exp_args:Ne \__stex_morphisms_elab_i:nnn {\stex_symbol_uri_name:n {#1}}{#1}
3157   }{
3158     \stex_debug:nn{morphisms}{Generating~1~\l__stex_morphisms_tmp}
3159     \use_i:nn{ \exp_args:Nno \use:nn {\__stex_morphisms_add_symbol:nnnnnnnnN {#1}} \l__stex
3160   }#2

```

```

3161
3162 \exp_args:No\stex_iterate_notations:nn\l_stex_current_domain_uri {
3163   \str_if_eq:nnT{#1}{##1}{
3164     \exp_args:Ne \stex_add_notation:nnnnn {
3165       \exp_args:Ne \stex_new_symbol_uri:n {\exp_after:wN \use_ii:nn \l__stex_morphisms_tm
3166         }{##2}{##3}{##4}{##5}
3167     }
3168   }
3169 }
3170
3171 \cs_new_protected:Nn \__stex_morphisms_elab_i:nnn {
3172   \tl_set:Ne \l__stex_morphisms_tmp {{{\l_stex_feature_name_str / #1}}}
3173   \stex_debug:nn{morphisms}{Generating-2~\l_stex_feature_name_str / #1}
3174   \bool_if:NTF \l__stex_morphisms_with_macros_bool {
3175     \exp_args:Nnno \__stex_morphisms_add_symbol:nnnnnnnnN {#2} {#3}
3176   }{
3177     \exp_args:Nnno \__stex_morphisms_add_symbol:nnnnnnnnN {#2} {}
3178   }
3179   {\l_stex_feature_name_str / #1}
3180 }
3181
3182 \cs_new_protected:Npn \__stex_morphisms_add_symbol:nnnnnnnnN #1 {
3183   \tl_clear:N \l__stex_morphisms_tmp_b
3184   \seq_map_inline:Nn \l_stex_morphism_assigns_seq {
3185     \__stex_morphisms_elab_check_assign:nnnnn{#1}##1
3186   }
3187   \tl_if_empty:NTF \l__stex_morphisms_tmp_b
3188   \stex_add_symbol:nnnnnnnN
3189   \__stex_morphisms_add_symbol_i:nnnnnnnnN
3190 }
3191
3192 \cs_new_protected:Npn \__stex_morphisms_add_symbol_i:nnnnnnnnN #1 #2 #3 #4 #5 {
3193   \stex_add_symbol:nnnnnnnnN {#1}{#2}{#3}{#4}{assigned}
3194 }
3195
3196 \cs_new_protected:Nn \__stex_morphisms_elab_check_assign:nnnnn {
3197   \tl_if_eq:nnT{#1}{{#2}{#3}{#4}{#5}}{
3198     \seq_map_break:n{
3199       \tl_set:Nn \l__stex_morphisms_tmp_b {assigned}
3200     }
3201   }
3202 }
3203
3204 \cs_new_protected:Nn \__stex_morphisms_elab_check_rename:nnn {
3205   \tl_if_eq:nnT{#1}{#2}{
3206     \seq_map_break:n{
3207       \tl_set:Nn \l__stex_morphisms_tmp {#3}
3208     }
3209   }
3210 }
3211
3212 \cs_new_protected:Nn \__stex_morphisms_do_for_list: {
3213   \seq_clear:N \l_stex_fors_seq
3214   \clist_map_inline:Nn \l_stex_key_for_clist {

```

```

3215 \exp_args:N\stex_get_in_morphism:n{\tl_to_str:n{##1}}
3216 \seq_put_right:No \l_stex_fors_seq
3217 {\l_stex_get_symbol_uri}
3218 }
3219 }
3220
3221
3222 \cs_new_protected:Nn \__stex_morphisms_definiens_impl:nn {
3223 \tl_if_empty:NF {#1} {
3224 \stex_get_in_morphism:n { #1 }
3225 \tl_set_eq:NN \l_stex_current_def_uri \l_stex_get_symbol_uri
3226 }
3227 \tl_if_empty:NT \l_stex_current_def_uri {
3228 \msg_error:nn{stex}{error/definiensfor}
3229 }
3230 \stex_debug:nn{definiens}{Checking-\stex_use_symbol_uri:N \l_stex_current_def_uri }
3231 \stex_if_do_html:TF{
3232 \exp_after:wN \stex_metagroup_do_in:n \exp_after:wN {
3233 \exp_after:wN \seq_put_right:Nn \exp_after:wN \l_stex_morphism_assigns_seq
3234 \exp_after:wN { \l_stex_current_def_uri }
3235 }
3236 \mode_leave_vertical: \stex_annotate:nn{data-ftml-assign={\stex_use_symbol_uri:N \l_st
3237 \stex_annotate_force_break:n{
3238 \stex_annotate:nn{data-ftml-definiens={}}{#2}
3239 }
3240 }
3241 }{
3242 \exp_args:No \__stex_morphisms_add_definiens:nn \l_stex_current_def_uri {#2}
3243 \stex_if_smsmode:F{#2}
3244 }
3245 }
3246
3247 \cs_new_protected:Nn \__stex_morphisms_add_definiens:nn {
3248 \tl_set:Nn \l_stex_get_symbol_uri {#1}
3249 \stex_debug:nn{morphisms}{Adding-definiens~\tl_to_str:n{#1~#2}}
3250 %\__stex_morphisms_set_definiens_macros: #1\__stex_morphisms_break:
3251 \stex_assign_do:n{#2}
3252 }
3253
3254 %\cs_new_protected:Npn \__stex_morphisms_set_definiens_macros: #1?#2?#3\__stex_morphisms_br
3255 % \str_set:Nn \l_stex_get_symbol_uri { #1?#2} % TODO
3256 % \str_set:Nn \l_stex_get_symbol_name_str {#3}
3257 % \exp_args:Nne\use:nn{\__stex_morphisms_set_definiens_macros_i:nnnnnnn}{
3258 % \prop_item:Nn \l_stex_morphism_symbols_prop {[#1?#2]/[#3]}
3259 % }
3260 %}
3261
3262 %\cs_new_protected:Nn \__stex_morphisms_set_definiens_macros_i:nnnnnnn {
3263 % \tl_set:Nn \l_stex_get_symbol_def_tl{#4}
3264 %}
3265
3266
3267 \cs_new_protected:Nn \__stex_morphisms_rename_all: {
3268 % TODO

```

```

3269 }
3270
3271
3272 \cs_new_protected:Nn \stex_structural_feature_morphism_check_total: {
3273   \seq_map_inline:Nn \l_stex_morphism_symbols_seq {
3274     \__stex_morphisms_total_check:nn ##1
3275   }
3276 }
3277
3278 \cs_new_protected:Nn \__stex_morphisms_total_check:nn {
3279   \__stex_morphisms_total_check:nnnnnnnN {#1} #2
3280 }
3281
3282 \cs_new_protected:Nn \__stex_morphisms_total_check:nnnnnnnN {
3283   \tl_if_empty:nT{#5}{
3284     \seq_if_in:NnF \l_stex_morphism_assigns_seq {#1} {
3285       \msg_error:nnxx{stex}{error/needsdefiniens}{\stex_use_symbol_uri:n {#1}}{total~morphi
3286     }
3287   }
3288 }
3289
3290 \cs_new:Npn \__stex_morphisms_split_qm:w #1 ? #2 ? #3 { #3 }
3291
3292
3293
3294 \cs_new_protected:Npn \__stex_morphisms_reactivate: {
3295   \stex_deactivate_macro:Nn \symdecl {module~environments}
3296   \stex_deactivate_macro:Nn \textsymdecl {module~environments}
3297   \stex_deactivate_macro:Nn \symdef {module~environments}
3298   \stex_deactivate_macro:Nn \notation {module~environments}
3299   \stex_deactivate_macro:Nn \importmodule {module~environments}
3300   \stex_deactivate_macro:Nn \requiremodule {module~environments}
3301   \stex_deactivate_macro:Nn \smodule {outside-of~morphisms}
3302   \stex_reactivate_macro:N \assign
3303   \stex_reactivate_macro:N \assignMorphism
3304   \stex_reactivate_macro:N \renamedec1
3305   \cs_set_eq:NN \stex_do_for_list: \__stex_morphisms_do_for_list:
3306   \cs_set_eq:NN \stex_add_definiens:nn \__stex_morphisms_add_definiens:nn
3307   \cs_set_eq:NN \stex_definiens_impl:nn \__stex_morphisms_definiens_impl:nn
3308 }

```

(End of definition for \stex_structural_feature_morphism:nnnnn, \stex_structural_feature_morphism_~with_macros:nnnnn, and \stex_structural_feature_morphism_end:. These functions are documented on page 145.)

sfunction

\stex_iterate_morphisms:nn

```

3309 \cs_new_protected:Nn \stex_iterate_morphisms:nn {
3310   \seq_clear:N \l__stex_morphisms_mods_seq
3311   \bool_set_true:N \l__stex_morphisms_continue_bool
3312   \cs_set:Npn \__stex_morphisms_cs:nnnn ##1 ##2 ##3 ##4 ##5 {
3313     #2
3314     \bool_if:NT \l__stex_morphisms_continue_bool {
3315       \str_if_eq:nnTF{##1}{[#2]}{
3316         \tl_put_right:Nn \l__stex_morphisms_todo_tl {

```

```

3317         \__stex_morphisms_iterate:nn{##5}{##2}
3318     }
3319 }{
3320     \tl_put_right:Nn \l__stex_morphisms_todo_tl {
3321         \__stex_morphisms_iterate:nn{##5 / ##1}{##2}
3322     }
3323 }
3324 }
3325 }
3326 \cs_set:Npn \stex_iterate_break:n ##1 {
3327     \bool_set_false:N \l__stex_morphisms_continue_bool
3328     \prop_map_break:n{##1}
3329 }
3330 \__stex_morphisms_iterate:nn{}{##1}
3331 }
3332 \cs_new_protected:Nn \__stex_morphisms_iterate:nn {
3333     \tl_clear:N \l__stex_morphisms_todo_tl
3334     \seq_if_in:NnF \l__stex_morphisms_mods_seq {#1 #2}{
3335         \seq_put_right:Nn \l__stex_morphisms_mods_seq {#1 #2}
3336         \prop_map_inline:cn{\_stex_morphisms_macro:n{#2}}{
3337             \__stex_morphisms_cs:nnnn ##2 {#1}
3338             % TODO
3339             % ##1: name or [mpath]
3340             % ##2 = {####1}{####2}{####3}{####4}
3341             % ####1 = name
3342             % ####2 = mpath
3343             % ####3 = type
3344             % ####4 = {origname}{newname}*
3345         }
3346         \bool_if:NT \l__stex_morphisms_continue_bool \l__stex_morphisms_todo_tl
3347     }
3348 }

```

(End of definition for `\stex_iterate_morphisms:nn`. This function is documented on page 145.)

sfunction

`\stex_get_in_morphism:n`

```

3349 \cs_new_protected:Nn \stex_get_in_morphism:n {
3350     \stex_debug:nn{morphisms}{Getting-in~morphism:~#1}
3351     \tl_clear:N \l_stex_get_symbol_uri
3352     \str_set:Nn \l__stex_morphisms_get_str {#1}
3353     \seq_map_inline:Nn \l_stex_morphism_symbols_seq {
3354         \__stex_morphisms_get_check:nn ##1
3355     }
3356     \tl_if_empty:NT \l_stex_get_symbol_uri {
3357         \seq_map_inline:Nn \l_stex_morphism_renames_seq {
3358             \__stex_morphisms_renamed_check:nn##1
3359         }
3360     }
3361     \tl_if_empty:NT \l_stex_get_symbol_uri {
3362         \msg_error:nnxx{stex}{error/unknownsymbolin}{#1}{
3363             morphism~\l_stex_feature_name_str
3364         }
3365     }

```

```

3366 }
3367
3368
3369 \cs_new_protected:Nn \__stex_morphisms_get_check:nn {
3370   \__stex_morphisms_check_name:nnnn #1 #2
3371 }
3372
3373 \cs_new_protected:Nn \__stex_morphisms_check_name:nnnn {
3374   % TODO module name, path, archive, macroname ?
3375   \exp_args:No \str_if_eq:nnTF \l__stex_morphisms_get_str {#4} {
3376     \tl_set:Nn \l_stex_get_symbol_uri {{{#1}{#2}{#3}{#4}}}
3377     \__stex_morphisms_set:nnnnnnN
3378   }{
3379     \__stex_morphisms_macro:nnnnnnN{{{#1}{#2}{#3}{#4}}}
3380   }
3381 }
3382
3383 \cs_new_protected:Nn \__stex_morphisms_set:nnnnnnN {
3384   \seq_map_break:n{
3385     \str_set:Nn \l_stex_get_symbol_macro_str{#1}
3386     \int_set:Nn \l_stex_get_symbol_arity_int {#2}
3387     \tl_set:Nn \l_stex_get_symbol_args_tl {#3}
3388     \tl_set:Nn \l_stex_get_symbol_def_tl {#4}
3389     \tl_set:Nn \l_stex_get_symbol_type_tl {#5}
3390     \tl_set:Nn \l_stex_get_symbol_return_tl {#6}
3391     \tl_set:Nn \l_stex_get_symbol_invoke_cs {#7}
3392   }
3393 }
3394
3395 \cs_new_protected:Npn \__stex_morphisms_macro:nnnnnnN #1 #2 {
3396   \exp_args:No \str_if_eq:nnTF \l__stex_morphisms_get_str {#2}{
3397     \tl_set:Nn \l_stex_get_symbol_uri{#1}
3398     \__stex_morphisms_set:nnnnnnN
3399   }\__stex_morphisms_gobble:nnnnnnN
3400   {#2}
3401 }
3402
3403 \cs_new_protected:Nn \__stex_morphisms_gobble:nnnnnnN {}
3404
3405 \cs_new_protected:Nn \__stex_morphisms_renamed_check:nn {
3406   \stex_debug:nn{here}{\tl_to_str:n{{{#1}{#2}}}:~\l__stex_morphisms_get_str}
3407   \__stex_morphisms_renamed_check:nnnnnn #1 #2
3408 }
3409
3410 \cs_new_protected:Nn \__stex_morphisms_renamed_check:nnnnnn {
3411   \exp_args:No \str_if_eq:nnTF \l__stex_morphisms_get_str{#5}{
3412     \seq_map_break:n{
3413       \str_set:Nn \l__stex_morphisms_get_str {#4}
3414       \seq_map_inline:Nn \l_stex_morphism_symbols_seq {
3415         \__stex_morphisms_get_check:nn ##1
3416       }
3417     }
3418   }{
3419     \exp_args:No \str_if_eq:nnT \l__stex_morphisms_get_str{#6}{

```

```

3420     \seq_map_break:n{
3421       \str_set:Nn \l__stex_morphisms_get_str {#4}
3422       \seq_map_inline:Nn \l_stex_morphism_symbols_seq {
3423         \__stex_morphisms_get_check:nn ##1
3424       }
3425     }
3426   }
3427 }
3428 }

```

(End of definition for `\stex_get_in_morphism:n`. This function is documented on page 145.)

sfunction

`copymodule (env.)`

```

3429 \stex_new_stylable_env:nnnnnnn {copymodule}{m 0{} m}{
3430   \__stex_morphisms_begin_copy:Nnnn\stex_structural_feature_morphism:nnnnn {#1}{#2}{#3}
3431 }{
3432   \stex_if_smsmode:F {
3433     \stex_style_apply:
3434   }
3435   \stex_structural_feature_morphism_end:
3436 }{}{}{}
3437
3438 \stex_new_stylable_env:nnnnnnn {copymodule*}{m 0{} m}{
3439   \cs_set:Npn \stex_style_apply: {
3440     \stex_apply_patch_begin:n{copymodule}
3441   }
3442   \__stex_morphisms_begin_copy:Nnnn\stex_structural_feature_morphism_with_macros:nnnnn {#1}
3443 }{
3444   \cs_set:Npn \stex_style_apply: {
3445     \stex_apply_patch_end:n{copymodule}
3446   }
3447   \stex_if_smsmode:F {
3448     \stex_style_apply:
3449   }
3450   \stex_structural_feature_morphism_end:
3451 }{}{}{}
3452
3453 \cs_new_protected:Nn \__stex_morphisms_begin_copy:Nnnn {
3454   #1{#2}{morphism}{#3}{#4}{,data-ftml-total=false}
3455   \stex_if_smsmode:F {
3456     \tl_set:Nn \thiscopyname { #2 }
3457     \tl_set:Nn \thismoduleuri {\stex_use_module_uri:N \l_stex_current_domain_uri}
3458     \stex_style_apply:
3459   }
3460   \stex_smsmode_do:
3461 }
3462
3463
3464 \stex_deactivate_macro:Nn \copymodule {module-environments}
3465 \stex_every_module:n {
3466   \stex_reactivate_macro:N \copymodule
3467 }
3468 \stex_sms_allow_env:n{copymodule}

```

```

3469
3470 \exp_after:wN \stex_deactivate_macro:Nn \csname copymodule*\endcsname {module~environments}
3471 \stex_every_module:n {
3472   \exp_after:wN\stex_reactivate_macro:N \csname copymodule*\endcsname
3473 }
3474 \stex_sms_allow_env:n{copymodule*}
3475
3476 \stex_new_stylable_cmd:nnnn{copymod}{s m O{} m m}{
3477   \IfBooleanTF#1\stex_structural_feature_morphism_with_macros:nnnnn
3478   \stex_structural_feature_morphism:nnnnn{#2}{morphism}{#3}{#4}{,data-ftml-total={false}}
3479   \clist_map_function:nN{#5}\__stex_morphisms_parse_assign:n
3480   \stex_if_smsmode:F {
3481     \tl_set:Nn \thiscopyname { #2 }
3482     \tl_set:Nn \thismoduleuri {\stex_use_module_uri:N \l_stex_current_domain_uri}
3483     \stex_style_apply:
3484   }
3485   \stex_structural_feature_morphism_end:
3486   \stex_smsmode_do:
3487 }{}
3488
3489 \stex_deactivate_macro:Nn \copymod {module~environments}
3490 \stex_every_module:n {
3491   \stex_reactivate_macro:N \copymod
3492 }
3493 \stex_sms_allow_escape:N\copymod

```

interpretmodule (env.)

```

3494 \stex_new_stylable_env:nnnnnnn {interpretmodule}{m O{} m}{
3495   \__stex_morphisms_begin_interpret:Nnn\stex_structural_feature_morphism:nnnnn {#1}{#2}{#3}
3496 }{
3497   \stex_structural_feature_morphism_check_total:
3498   \stex_if_smsmode:F {
3499     \stex_style_apply:
3500   }
3501   \stex_structural_feature_morphism_end:
3502 }{}{}{}
3503
3504 \stex_new_stylable_env:nnnnnnn {interpretmodule*}{m O{} m}{
3505   \cs_set:Npn \stex_style_apply: {
3506     \stex_apply_patch_begin:n{interpretmodule}
3507   }
3508   \__stex_morphisms_begin_interpret:Nnn\stex_structural_feature_morphism_with_macros:nnnnn
3509 }{
3510   \cs_set:Npn \stex_style_apply: {
3511     \stex_apply_patch_end:n{interpretmodule}
3512   }
3513   \stex_structural_feature_morphism_check_total:
3514   \stex_if_smsmode:F {
3515     \stex_style_apply:
3516   }
3517   \stex_structural_feature_morphism_end:
3518 }{}{}{}
3519
3520 \cs_new_protected:Nn \__stex_morphisms_begin_interpret:Nnnn {

```



```

3521 #1{#2}{morphism}{#3}{#4}{,data-ftml-total=true}
3522 \stex_if_smsmode:F {
3523   \tl_set:Nn \thiscopyname { #2 }
3524   \tl_set:Nn \thismoduleuri {\stex_use_module_uri:N \l_stex_current_domain_uri}
3525   \stex_style_apply:
3526 }
3527 \stex_smsmode_do:
3528 }
3529
3530 \stex_deactivate_macro:Nn \interpretmodule {module~environments}
3531 \stex_every_module:n {
3532   \stex_reactivate_macro:N \interpretmodule
3533 }
3534 \stex_sms_allow_env:n{interpretmodule}
3535
3536 \exp_after:wN \stex_deactivate_macro:Nn \csname interpretmodule*\endcsname {module~environm
3537 \stex_every_module:n {
3538   \exp_after:wN \stex_reactivate_macro:N \csname interpretmodule*\endcsname
3539 }
3540 \stex_sms_allow_env:n{interpretmodule*}
3541
3542 \stex_new_stylable_cmd:nnnn{interpretmod}{s m 0{} m m}{
3543   \IfBooleanTF#1\stex_structural_feature_morphism_with_macros:nnnnn
3544   \stex_structural_feature_morphism:nnnnn{#2}{morphism}{#3}{#4}{,data-ftml-total=true}
3545   \clist_map_function:nN{#5}\__stex_morphisms_parse_assign:n
3546   \stex_if_smsmode:F {
3547     \tl_set:Nn \thiscopyname { #2 }
3548     \tl_set:Nn \thismoduleuri {\stex_use_module_uri:N \l_stex_current_domain_uri}
3549     \stex_style_apply:
3550   }
3551   \stex_structural_feature_morphism_check_total:
3552   \stex_structural_feature_morphism_end:
3553   \stex_smsmode_do:
3554 }{}
3555
3556 \stex_deactivate_macro:Nn \interpretmod {module~environments}
3557 \stex_every_module:n {
3558   \stex_reactivate_macro:N \interpretmod
3559 }
3560 \stex_sms_allow_escape:N\interpretmod

```

\assign

```

3561 \stex_new_stylable_cmd:nnnn {assign} { m m }{
3562   \stex_get_in_morphism:n{#1}
3563   \stex_if_do_html:TF{
3564     \stex_annotate_invisible:n{\hbox{$\stex_assign_do:n{#2}$}}
3565   }{
3566     \stex_assign_do:n{#2}
3567   }
3568   \stex_smsmode_do:
3569 }{}
3570 \stex_sms_allow_escape:N\assign
3571 \stex_deactivate_macro:Nn \assign {morphism~environments}
3572

```

```

3573 \cs_new_protected:Nn \_stex_assign_do:n{
3574   \stex_debug:nn{assign}{Assigning~\stex_use_symbol_uri:N \l_stex_get_symbol_uri~to~\tl_to_
3575   \stex_check_term:nn{assignment}{#1}
3576   \stex_if_do_html:T{
3577     \stex_annotate_invisible:nn{data-ftml-assign={\stex_use_symbol_uri:N \l_stex_get_symbol
3578     \_stex_annotate_force_break:n{
3579       \mode_if_math:TF{\stex_annotate:nn{data-ftml-definiens={}}{#1}}{\hbox{$\stex_annota
3580     }
3581   }
3582 }
3583 \exp_after:wN \stex_metagroup_do_in:n \exp_after:wN {
3584   \exp_after:wN \seq_put_right:Nn \exp_after:wN \l_stex_morphism_assigns_seq
3585   \exp_after:wN { \l_stex_get_symbol_uri }
3586 }
3587 }
3588
3589 \cs_new_protected:Nn \__stex_morphisms_parse_assign:n {
3590   \str_clear:N \l__stex_morphisms_name_str
3591   \str_clear:N \l__stex_morphisms_newname_str
3592   \tl_clear:N \l__stex_morphisms_ass_tl
3593   \stex_debug:nn{morphisms}{Parsing~#1}
3594   \exp_args:NNe \seq_set_split:Nnn \l__stex_morphisms_seq {\tl_to_str:n{0}} {#1}
3595   \int_compare:nNnTF {\seq_count:N \l__stex_morphisms_seq} = 1 {
3596     \stex_debug:nn{morphisms}{No~0}
3597     \seq_pop_left:NN \l__stex_morphisms_seq \l__stex_morphisms_next_tl
3598   }{
3599     \seq_pop_left:NN \l__stex_morphisms_seq \l__stex_morphisms_name_str
3600     \stex_debug:nn{morphisms}{Name:~\l__stex_morphisms_name_str}
3601     \exp_args:NNo \str_set:Nn \l__stex_morphisms_name_str \l__stex_morphisms_name_str
3602     \tl_set:Ne \l__stex_morphisms_next_tl {\seq_use:Nn \l__stex_morphisms_seq 0}
3603   }
3604   \exp_args:NNNo \seq_set_split:Nnn \l__stex_morphisms_seq = \l__stex_morphisms_next_tl
3605   \str_if_empty:NTF \l__stex_morphisms_name_str {
3606     \seq_pop_left:NN \l__stex_morphisms_seq \l__stex_morphisms_name_str
3607     \exp_args:NNo \str_set:Nn \l__stex_morphisms_name_str \l__stex_morphisms_name_str
3608     \tl_set:Ne \l__stex_morphisms_ass_tl {\seq_use:Nn \l__stex_morphisms_seq =}
3609   }{
3610     \seq_pop_left:NN \l__stex_morphisms_seq \l__stex_morphisms_newname_str
3611     \exp_args:NNo \str_set:Nn \l__stex_morphisms_newname_str \l__stex_morphisms_newname_str
3612     \tl_set:Ne \l__stex_morphisms_ass_tl {\seq_use:Nn \l__stex_morphisms_seq =}
3613   }
3614   \__stex_morphisms_do_parsed_assign:
3615 }
3616
3617 \cs_new_protected:Nn \__stex_morphisms_do_parsed_assign: {
3618   \exp_args:No \stex_get_in_morphism:n \l__stex_morphisms_name_str
3619   \str_if_empty:NF \l__stex_morphisms_newname_str {
3620     \stex_debug:nn{morphisms}{renaming~\l__stex_morphisms_name_str;~to~\l__stex_morphisms_n
3621     \exp_after:wN \__stex_morphisms_do_parsed_newname: \l__stex_morphisms_newname_str \__st
3622   }
3623   \tl_if_empty:NF \l__stex_morphisms_ass_tl {
3624     \stex_debug:nn{morphisms}{assigning~\l__stex_morphisms_name_str;~to~\exp_args:No \tl_to
3625     \exp_args:No \_stex_assign_do:n \l__stex_morphisms_ass_tl
3626   }

```

```

3627 }
3628
3629 \cs_new_protected:Nn \__stex_morphisms_do_parsed_newname: {
3630   \peek_charcode:NTF [ {
3631     \__stex_morphisms_do_parsed_newname:w
3632   }{
3633     \__stex_morphisms_do_parsed_newname:w []
3634   }
3635 }
3636
3637 \cs_new_protected:Npn \__stex_morphisms_do_parsed_newname:w [#1] #2 \__stex_morphisms_end:
3638   \stex_renamedec1_do:nn{#1}{#2}
3639 }

```

(End of definition for \assign. This function is documented on page 146.)

sfunction

\renamedec1

```

3640 \stex_new_stylable_cmd:nnnn {renamedec1} { m 0{} m }{
3641   \stex_get_in_morphism:n{#1}
3642   \stex_renamedec1_do:nn{#2}{#3}
3643   \stex_smsmode_do:
3644 }{}
3645 \stex_sms_allow_escape:N\renamedec1
3646 \stex_deactivate_macro:Nn \renamedec1 {morphism~environments}
3647
3648 \cs_new_protected:Nn \stex_renamedec1_do:nn {
3649   \stex_debug:nn{renamedec1}{Renaming~\stex_use_symbol_uri:N \l_stex_get_symbol_uri~to~[#1]
3650   \stex_if_do_html:T{
3651     \exp_args:Ne \stex_annotate_invisible:nn{
3652       data-ftml-rename={\stex_use_symbol_uri:N \l_stex_get_symbol_uri},
3653       data-ftml-macroname={#2}
3654       \str_if_empty:nF{#1}{ ,data-ftml-to={#1} }
3655     }}
3656   }
3657   \stex_metagroup_do_in:e {
3658     \seq_push:Nn \exp_not:N \l_stex_morphism_renames_seq
3659     {{\l_stex_get_symbol_uri}{#2}{
3660       \tl_if_empty:nTF{#1}{
3661         \l_stex_feature_name_str/\stex_symbol_uri_name:N \l_stex_get_symbol_uri
3662       }{#1}
3663     }}}
3664   }
3665 }

```

(End of definition for \renamedec1. This function is documented on page 146.)

sfunction

\assignMorphism

```

3666 \stex_new_stylable_cmd:nnnn {assignMorphism} { m m }{
3667   \str_clear:N \l__stex_morphisms_morphism_dom_str
3668   \exp_args:No \stex_iterate_morphisms:nn\l_stex_current_domain_uri{
3669     \stex_debug:nn{assignMorphism}{
3670       Checking:~#1~vs:^^J##1^^J##2^^J##3^^J##4

```

```

3671     }
3672     \str_if_eq:nnTF{#1}{##1}{
3673         \__stex_morphisms_do_morph_assign:nnn{##1}{##2}{##4}
3674     }{
3675         \stex_str_if_ends_with:nnT{##2}{#1}{
3676             \__stex_morphisms_do_morph_assign:nnn{##1}{##2}{##4}
3677         }
3678     }
3679 }
3680 \str_if_empty:NT \l__stex_morphisms_morphism_dom_str {
3681     \msg_error:nnn{stex}{error/nomorphism}{#1}
3682 }
3683 \bool_set_false:N \l_tmpa_bool
3684 \stex_iterate_morphisms:nn \l_stex_current_module_str {
3685     \stex_debug:nn{assignMorphism}{
3686         Checking:~#2~vs:^^J##1^^J##2^^J##3^^J##4
3687     }
3688     \str_if_eq:nnTF{#2}{##1}{
3689         \stex_debug:nn{assignMorphism}{match!}
3690         \stex_iterate_break:n{
3691             \stex_annotate_invisible:nn{
3692                 data-ftml-assignmorphismfrom={\l__stex_morphisms_morphism_dom_str}
3693                 data-ftml-assignmorphismto={\l_stex_current_module_str?##1}
3694             }{}
3695             \bool_set_true:N \l_tmpa_bool
3696         }
3697     }{
3698         \stex_str_if_ends_with:nnT{##2}{#2}{
3699             \stex_debug:nn{assignMorphism}{match!}
3700             \stex_iterate_break:n{
3701                 \stex_annotate_invisible:nn{
3702                     data-ftml-assignmorphismfrom={\l__stex_morphisms_morphism_dom_str},
3703                     data-ftml-assignmorphismto={\l_stex_current_module_str?##1}
3704                 }{}
3705                 \bool_set_true:N \l_tmpa_bool
3706             }
3707         }
3708     }
3709 }
3710 \bool_if:NF \l_tmpa_bool {
3711     \msg_error:nnn{stex}{error/nomorphism}{#2}
3712 }
3713 }{}
3714 \stex_sms_allow_escape:N \assignMorphism
3715 \stex_deactivate_macro:Nn \assignMorphism {morphism~environments}
3716
3717 \cs_new_protected:Nn \__stex_morphisms_do_morph_assign:nnn {
3718     \stex_iterate_break:n{
3719         \str_set:Nx \l__stex_morphisms_morphism_dom_str { \l_stex_current_domain_str ? #1 }
3720         \stex_debug:nn{assignMorphism}{match!}
3721         \stex_iterate_symbols:nn{#2}{
3722             \stex_debug:nn{assignMorphism}{removing~##1?##3}
3723             % TODO: non-trivial assignments
3724             \prop_remove:Nn \l_stex_morphism_symbols_prop {

```

```

3725         [[#1]] / [[#3]]
3726     }
3727 }
3728 }
3729 }

```

(End of definition for \assignMorphism. This function is documented on page 146.)

sfunction

Chapter 32

Symbols

3730 <@@=stex_syms>

32.1 Declarations

Keys used in various symbol declarations:

```
3731 \stex_keys_define:nnnn{symargs}{
3732   \str_clear:N \l_stex_key_args_str
3733   \str_clear:N \l_stex_key_role_str
3734   \str_clear:N \l_stex_key_reorder_str
3735   \str_clear:N \l_stex_key_assoc_str
3736 }{
3737   args      .str_set:N = \l_stex_key_args_str ,
3738   reorder   .str_set:N = \l_stex_key_reorder_str ,
3739   assoc     .choices:nn = {bin,binl,binr,pre,conj,pwconj}
3740   { \str_set:Ne \l_stex_key_assoc_str \l_keys_choice_tl},
3741   role      .str_set:N = \l_stex_key_role_str
3742 }{}
3743
3744 \stex_keys_define:nnnn{decl}{
3745   \str_clear:N \l_stex_key_name_str
3746   \str_clear:N \l_stex_key_args_str
3747   \tl_clear:N \l_stex_key_type_tl
3748   \tl_clear:N \l_stex_key_def_tl
3749   \tl_clear:N \l_stex_key_return_tl
3750   \str_clear:N \l_stex_key_wikidata_str
3751   \clist_clear:N \l_stex_key_argtypes_clist
3752 }{
3753   name      .str_set:N = \l_stex_key_name_str ,
3754   return     .tl_set:N = \l_stex_key_return_tl ,
3755   argtypes   .clist_set:N = \l_stex_key_argtypes_clist ,
3756   wikidata   .str_set:N = \l_stex_key_wikidata_str ,
3757   type       .tl_set:N = \l_stex_key_type_tl ,
3758   def        .tl_set:N = \l_stex_key_def_tl ,
3759   align      .code:n = {},
3760   gf         .code:n = {}
3761 }{style,deprecate,symargs}
```

\symdecl

```

3762 \str_new:N \l_stex_macroname_str
3763
3764 \stex_new_stylable_cmd:nnnn {symdecl} { s m 0{}} {
3765   \stex_keys_set:nn{decl}{#3}
3766   \IfBooleanTF #1 {
3767     \str_clear:N \l_stex_macroname_str
3768   }{
3769     \str_set:Ne \l_stex_macroname_str { #2 }
3770   }
3771   \__stex_syms_top:n{#2}
3772   \stex_if_smsmode:F \__stex_syms_style:
3773   \stex_smsmode_do:
3774 }{}
3775
3776 \stex_deactivate_macro:Nn \symdecl {module~environments}
3777 \stex_every_module:n {\stex_reactivate_macro:N \symdecl}
3778 \stex_sms_allow_escape:N \symdecl
3779
3780 \cs_new_protected:Nn \__stex_syms_style: {
3781   \group_begin:
3782     \tl_set:Ne \thisdecluri {\stex_use_symbol_uri:N \l_stex_current_symbol_uri}
3783     \tl_set_eq:NN \thisdeclname \l_stex_key_name_str
3784     \tl_set_eq:NN \thistype \l_stex_key_type_tl
3785     \tl_set_eq:NN \thisdefiniens \l_stex_key_def_tl
3786     \tl_set_eq:NN \thisargs \l_stex_key_args_str
3787     \tl_clear:N \thisstyle
3788     \stex_style_apply:
3789   \group_end:
3790 }

```

(End of definition for \symdecl. This function is documented on page 147.)

sfunction

\symdef

```

3791 \stex_new_stylable_cmd:nnnn {symdef} { m 0{ } m } {
3792   \stex_keys_set:nn{symdef}{#2}
3793   \str_set:Ne \l_stex_macroname_str { #1 }
3794   \__stex_syms_top:n{#1}
3795   \stex_debug:nn{symdef}{Doing~\stex_use_symbol_uri:N \l_stex_current_symbol_uri}
3796   \str_set_eq:NN \l_stex_get_symbol_uri \l_stex_current_symbol_uri
3797   \stex_notation_parse:n{#3}
3798   \stex_if_check_terms:T{ \_stex_notation_check: }
3799   \_stex_notation_add:
3800   \stex_if_do_html:T{
3801     \_stex_notation_do_html:n{\stex_use_symbol_uri:N \l_stex_current_symbol_uri}
3802   }
3803   \stex_if_smsmode:F \__stex_syms_def_style:
3804   \stex_smsmode_do:
3805 }{}
3806
3807 \stex_deactivate_macro:Nn \symdef {module~environments}
3808 \stex_every_module:n {\stex_reactivate_macro:N \symdef}
3809 \stex_sms_allow_escape:N \symdef
3810

```

```

3811 \cs_new_protected:Nn \__stex_syms_def_style: {
3812   \group_begin:
3813   \tl_set:Nx \thisdecluri {\stex_use_symbol_uri:N \l_stex_current_symbol_uri}
3814   \tl_set_eq:NN \thisdeclname \l_stex_key_name_str
3815   \tl_set_eq:NN \thistype \l_stex_key_type_tl
3816   \tl_set_eq:NN \thisdefiniens \l_stex_key_def_tl
3817   \tl_set_eq:NN \thisargs \l_stex_key_args_str
3818   \tl_clear:N \thisstyle
3819   \str_set_eq:NN\thisnotationvariant\l_stex_key_variant_str
3820   \def\thisnotation{
3821     \exp_args:Nne \use:nn{\l_stex_notation_macrocode_cs}{}}{
3822     \stex_notation_make_args:
3823   }
3824 }
3825 \stex_style_apply:
3826 \group_end:
3827 }

```

(the symdef keyset is defined after notations as

`\stex_keys_define:nnnn{symdef}{-}{-}{decl,notation}`)

(End of definition for `\symdef`. This function is documented on page 147.)

sffunction

`\textsymdecl`

```

3828 \stex_keys_define:nnnn{textsymdecl}{
3829   \str_clear:N \l_stex_key_name_str
3830   \tl_clear:N \l_stex_key_type_tl
3831   \tl_clear:N \l_stex_key_def_tl
3832 }{
3833   name      .str_set:N = \l_stex_key_name_str ,
3834   type      .tl_set:N  = \l_stex_key_type_tl ,
3835   def       .tl_set:N  = \l_stex_key_def_tl,
3836   gf        .code:n    = {}
3837 }{style,deprecate}
3838
3839 \stex_new_stylable_cmd:nnnn {textsymdecl} {m 0{ } m} {
3840   \stex_keys_set:nn{symdef}{-}
3841   \stex_keys_set:nn{textsymdecl}{#2}
3842   \str_set:Ne \l_stex_macroname_str { #1 }
3843   \str_if_empty:NT \l_stex_key_name_str {
3844     \str_set:Nn \l_stex_key_name_str {#1}
3845   }%{
3846   % \str_set:Ne \l_stex_key_name_str {\l_stex_key_name_str-sym}
3847   %}
3848   \str_set:Nn \l_stex_key_role_str {textsymdecl}
3849   \tl_set:Ne \l_stex_current_symbol_uri {
3850     \exp_args:No \stex_new_symbol_uri:n \l_stex_key_name_str
3851   }
3852   \stex_symdecl_do:
3853   \stex_check_terms:
3854   \exp_args:Nne \use:nn {\stex_add_symbol:nnnnnnN}{
3855     {\l_stex_macroname_str}
3856     {\l_stex_key_name_str}
3857     {0}{-}

```



```

3858     {\tl_if_empty:NF \l_stex_key_def_tl{DEFED} }
3859     {}% type
3860     {\use:c{#1name_nospace}}}% return
3861     \stex_invoke_text_symbol:
3862 }
3863 \exp_args:Nc \stex_ref_new_symbol:n
3864 { \stex_use_symbol_uri:N \l_stex_current_symbol_uri }
3865 \stex_if_do_html:T {
3866     \stex_symdecl_html:
3867 }
3868
3869 \int_set:Nn \l_stex_get_symbol_arity_int 0
3870 \tl_clear:N \l_stex_key_op_tl
3871 \str_clear:N \l_stex_key_intent_str
3872 \str_clear:N \l_stex_key_prec_str
3873 \tl_set_eq:NN \l_stex_get_symbol_uri \l_stex_current_symbol_uri
3874 \stex_notation_parse:n{\hbox{#3}}
3875 \stex_notation_add:
3876 \stex_if_do_html:T {
3877     \def\comp{\_comp}
3878     \stex_notation_do_html:n{ \stex_use_symbol_uri:N \l_stex_current_symbol_uri }
3879 }
3880 \stex_execute_in_module:e{
3881     \__stex_syms_set_textsymdecl_macro:nnn{#1}{ \stex_use_symbol_uri:N \l_stex_current_symbol_uri }
3882     \exp_not:n{#3}}
3883 }
3884
3885 \stex_if_smsmode:F{
3886     \group_begin:
3887     \tl_set:Nc \thisdecluri { \stex_use_symbol_uri:N \l_stex_current_symbol_uri }
3888     \tl_set_eq:NN \thisdeclname \l_stex_key_name_str
3889     \tl_clear:N \thisstyle
3890     \stex_style_apply:
3891     \group_end:
3892 }
3893 \stex_smsmode_do:
3894 }{}
3895
3896 \stex_deactivate_macro:Nn \textsymdecl {module~environments}
3897 \stex_every_module:n { \stex_reactivate_macro:N \textsymdecl }
3898 \stex_sms_allow_escape:N \textsymdecl
3899
3900 \cs_new_protected:Nn \__stex_syms_set_textsymdecl_macro:nnn {
3901     \cs_set_protected:cpn{#1name_nospace}{#3}
3902     \cs_set_protected:cpn{#1name}{
3903         \mode_if_vertical:T{\hbox_unpack:N\c_empty_box}
3904         \mode_if_math:T{\hbox{\let\xspace\relax #3}
3905         \mode_if_math:F{\cs_if_exist:NT\xspace\xspace}
3906     }
3907 }

```

(End of definition for `\textsymdecl`. This function is documented on page 147.)

sfunction

`\__stex_syms_top:n` shared behaviour for `\symdecl` and `\symdef`; calls `\stex_symdecl_do:`, optionally checks

the term components, adds the symbol to the current module and produces the FTML for the declaration.

```

3908 \cs_new_protected:Nn \__stex_syms_top:n {
3909   \str_if_empty:NT \l_stex_key_name_str {
3910     \str_set:Ne \l_stex_key_name_str { #1 }
3911   }
3912   \tl_set:Ne \l_stex_current_symbol_uri {
3913     \exp_args:No \stex_new_symbol_uri:n \l_stex_key_name_str
3914   }
3915
3916   \stex_symdecl_do:
3917   \__stex_check_terms:
3918   \__stex_syms_add_decl:
3919   \stex_if_do_html:T \stex_symdecl_html:
3920 }

```

(End of definition for __stex_syms_top:n.)

Adds the symbol to the current module:

```

3921 \cs_new_protected:Nn \__stex_syms_add_decl: {
3922   \exp_args:Nne \use:nn {\stex_add_symbol:nnnnnnN}{
3923     {\l_stex_macroname_str}
3924     {\l_stex_key_name_str}
3925     {\int_use:N \l_stex_get_symbol_arity_int}
3926     {\l_stex_get_symbol_args_tl}
3927     {\tl_if_empty:NF \l_stex_key_def_tl{DEFED} }
3928     {}
3929     {\exp_args:No \exp_not:n \l_stex_key_return_tl}
3930     \stex_invoke_symbol:
3931   }
3932   \exp_args:Ne \stex_ref_new_symbol:n
3933     {\stex_use_symbol_uri:N \l_stex_current_symbol_uri}
3934 }

```

\stex_symdecl_do:

```

3935 \cs_new_protected:Nn \stex_symdecl_do: {
3936   \stex_do_deprecation:n \l_stex_key_name_str
3937   \__stex_syms_parse_arity:
3938   \__stex_syms_do_args:
3939 }
3940
3941 \cs_new:Nn \stex_return_args:nn {
3942   {\svar{ARGUMENT_#1}\stex_eat_exclamation_point:}
3943 }
3944
3945 \cs_new_protected:Nn \__stex_syms_do_args: {
3946   \tl_clear:N \l_stex_get_symbol_args_tl
3947   \int_step_inline:nn \l_stex_get_symbol_arity_int {
3948     \tl_put_right:Nn \l_stex_get_symbol_args_tl {##1}
3949     \tl_put_right:Ne \l_stex_get_symbol_args_tl {
3950       \str_item:Nn \l_stex_key_args_str {##1}
3951     }
3952   }
3953 }

```

Constructs the arity string of the symbol:

```

3954 \int_new:N \l_stex_assoc_args_count
3955
3956 \cs_new_protected:Nn \__stex_syms_parse_arity: {
3957   \int_zero:N \l_stex_get_symbol_arity_int
3958   \int_zero:N \l_stex_assoc_args_count
3959   \str_map_inline:Nn \l_stex_key_args_str {
3960     \str_case:nnF ##1 {
3961       0 { \str_map_break: }
3962       1 { \str_map_break:n{
3963         \int_set:Nn \l_stex_get_symbol_arity_int {1}
3964         \str_set:Nn \l_stex_key_args_str {i}
3965       } }
3966       2 { \str_map_break:n{
3967         \int_set:Nn \l_stex_get_symbol_arity_int {2}
3968         \str_set:Nn \l_stex_key_args_str {ii}
3969       } }
3970       3 { \str_map_break:n{
3971         \int_set:Nn \l_stex_get_symbol_arity_int {3}
3972         \str_set:Nn \l_stex_key_args_str {iii}
3973       } }
3974       4 { \str_map_break:n{
3975         \int_set:Nn \l_stex_get_symbol_arity_int {4}
3976         \str_set:Nn \l_stex_key_args_str {iiii}
3977       } }
3978       5 { \str_map_break:n{
3979         \int_set:Nn \l_stex_get_symbol_arity_int {5}
3980         \str_set:Nn \l_stex_key_args_str {iiiii}
3981       } }
3982       6 { \str_map_break:n{
3983         \int_set:Nn \l_stex_get_symbol_arity_int {6}
3984         \str_set:Nn \l_stex_key_args_str {iiiiii}
3985       } }
3986       7 { \str_map_break:n{
3987         \int_set:Nn \l_stex_get_symbol_arity_int {7}
3988         \str_set:Nn \l_stex_key_args_str {iiiiiii}
3989       } }
3990       8 { \str_map_break:n{
3991         \int_set:Nn \l_stex_get_symbol_arity_int {8}
3992         \str_set:Nn \l_stex_key_args_str {iiiiiiii}
3993       } }
3994       9 { \str_map_break:n{
3995         \int_set:Nn \l_stex_get_symbol_arity_int {9}
3996         \str_set:Nn \l_stex_key_args_str {iiiiiii}
3997       } }
3998       i {\int_incr:N \l_stex_get_symbol_arity_int}
3999       b {\int_incr:N \l_stex_get_symbol_arity_int}
4000       a {\int_incr:N \l_stex_get_symbol_arity_int \int_incr:N \l_stex_assoc_args_count}
4001       B {\int_incr:N \l_stex_get_symbol_arity_int \int_incr:N \l_stex_assoc_args_count}
4002     }{
4003       \msg_error:nnxx{stex}{error/wrongargs}{-}{##1}
4004     }
4005   }
4006 }
```

(End of definition for `\stex_symdecl_do`:. This function is documented on page 147.)

sfunction

`\_stex_symdecl_html`:

```

4007 \cs_new_protected:Nn \_stex_symdecl_html: {
4008   \stex_annotate_invisible:n{
4009     \hbox{\exp_args:Ne \stex_annotate:nn {
4010       data-ftml-symdecl = { \l_stex_key_name_str},
4011       data-ftml-args = {\l_stex_key_args_str}
4012       \str_if_empty:NF \l_stex_macroname_str {,
4013         data-ftml-macroname={\l_stex_macroname_str}
4014       }
4015       \str_if_empty:NF \l_stex_key_wikidata_str {,
4016         data-ftml-wikidata={\l_stex_key_wikidata_str}
4017       }
4018       \str_if_empty:NF \l_stex_key_assoc_str {,
4019         data-ftml-assoctype={\l_stex_key_assoc_str}
4020       }
4021       \str_if_empty:NF \l_stex_key_reorder_str {,
4022         data-ftml-reorderargs={\l_stex_key_reorder_str}
4023       }
4024       \str_if_empty:NF \l_stex_key_role_str {,
4025         data-ftml-role={\l_stex_key_role_str}
4026       }
4027     }{\stex_annotate_force_break:n{
4028       \bool_set_true:N \stex_in_invisible_html_bool
4029       \tl_if_empty:NF \l_stex_key_type_tl {
4030         $\stex_annotate:nn{data-ftml-type={}}{\l_stex_key_type_tl}$
4031       }
4032       \tl_if_empty:NF \l_stex_key_def_tl {
4033         $\stex_annotate:nn{data-ftml-definiens={}}{\l_stex_key_def_tl}$
4034       }
4035       \tl_if_empty:NF \l_stex_key_return_tl{
4036         \exp_args:Nno \use:n{
4037           \cs_generate_from_arg_count:NNnn \l__stex_syms_cs
4038           \cs_set:Npn \l_stex_get_symbol_arity_int} \l_stex_key_return_tl
4039           \tl_set:Ne \l__stex_syms_args_tl {\_stex_map_args:N \_stex_return_args:nn}
4040           $\stex_annotate:nn{data-ftml-returntype={}}{
4041             \exp_after:wN \l__stex_syms_cs \l__stex_syms_args_tl!
4042           }$
4043         }
4044       \clist_if_empty:NF \l_stex_key_argtypes_clist {
4045         \stex_annotate:nn{data-ftml-argtypes={}}{\_stex_annotate_force_break:n{
4046           \clist_map_inline:Nn \l_stex_key_argtypes_clist {
4047             $\stex_annotate:nn{data-ftml-type={}}{##1}$
4048           }
4049         }}
4050       }
4051     }}
4052   }}
4053 }
```

(End of definition for `\_stex_symdecl_html`:. This function is documented on page 147.)

sfunction

\stex_add_symbol:nnnnnnN

```

4054 \cs_new_protected:Nn \stex_add_symbol:nnnnnnN {
4055   \stex_debug:nn{declaration}{New~declaration:~\stex_use_module_uri:N \l_stex_current_modul
4056   Macro:#1^^JArity:#3~(#4)^^J
4057   Def:~\tl_to_str:n{#5}^^J
4058   Type:~\tl_to_str:n{#6}^^J
4059   Returns:~\tl_to_str:n{#7}^^J
4060   Invokes:~\tl_to_str:n{#8}
4061 }
4062 \prop_gput:cnn{ \_stex_symbol_macro:N \l_stex_current_module_uri }
4063 {#2}{{#1}{#2}{#3}{#4}{#5}{#6}{#7}{#8}}
4064 \tl_if_empty:nF{#1}{
4065   \stex_do_up_to_module:n {
4066     \__stex_syms_activate_i:nnnnnnnn{#1}{#2}{#3}{#4}{#5}{#6}{#7}{#8}
4067   }
4068 }
4069 }

```

Generate semantic macros etc.:

```

4070 \cs_new_protected:Nn \_stex_activate_symbols: {
4071   \stex_debug:nn{activating}{All~symbols:~\cs_meaning:c{ \_stex_symbol_macro:N \l_stex_curr
4072   \prop_map_inline:cn{ \_stex_symbol_macro:N \l_stex_current_module_uri }{
4073     \__stex_syms_maybe_activate:nnnnnnnn ##2
4074   }
4075 }
4076
4077 \cs_new_protected:Nn \__stex_syms_maybe_activate:nnnnnnnn {
4078   \str_if_empty:nF{#1}{
4079     \__stex_syms_activate_i:nnnnnnnn {#1}{#2}{#3}{#4}{#5}{#6}{#7}{#8}
4080   }
4081 }
4082
4083 \cs_new_protected:Nn \__stex_syms_activate_i:nnnnnnnn {
4084   \stex_debug:nn{activating}{macro~#1:\stex_use_module_uri:N \l_stex_current_module_uri ? #
4085   \tl_to_str:n{{#3}{#4}{#5}{#6}{#7}{#8}}
4086 }
4087 \cs_set:cpe{#1} {
4088   \_stex_invoke_symbol:nnnnnnN
4089   {\exp_args:No \stex_new_symbol_uri:nn {\l_stex_current_module_uri} {#2} }
4090   \exp_not:n{{#3}{#4}{#5}{#6}{#7}{#8}}
4091 }
4092 }

```

(End of definition for \stex_add_symbol:nnnnnnN. This function is documented on page 147.)

sfunction

\stex_has_definiens_p:N

\stex_has_definiens:N $\underline{TF}$

\stex_has_definiens_p:n

\stex_has_definiens:n $\underline{TF}$

```

4093 \prg_new_conditional:Nnn \stex_has_definiens:N { p, T, F, TF } {
4094   \exp_args:No \stex_has_definiens:nTF #1 \prg_return_true: \prg_return_false:
4095 }
4096 \prg_new_conditional:Nnn \stex_has_definiens:n { p, T, F, TF } {
4097   \exp_args:Nne \use:nn{\__stex_syms_has_definiens:nnnnnnN}{\exp_args:Nne \prop_item:cn{
4098     \exp_args:Ne \_stex_symbol_macro:n {\stex_symbol_uri_module:n{#1}}
4099   }{

```

```

4100     \stex_symbol_uri_name:n {#1}
4101   }}
4102 }
4103
4104 \cs_new:Nn \__stex_syms_has_definiens:nnnnnnnN {
4105   \tl_if_empty:nTF{#5}\prg_return_false:\prg_return_true:
4106 }

```

(End of definition for `\stex_has_definiens:NTF` and `\stex_has_definiens:nTF`. These functions are documented on page 147.)

sfunction

32.2 Retrieval

```

\stex_iterate_symbols:n
  \stex_iterate_break:
  \stex_iterate_break:n
4107 \cs_new_protected:Nn \stex_iterate_symbols:n {
4108   \stex_pseudogroup_with:nn{\__stex_syms_sym_cs:nnnnnnnnN\stex_iterate_break:\stex_iterate_
4109     \cs_set:Npn \__stex_syms_sym_cs:nnnnnnnnN
4110     ##1 ##2 ##3 ##4 ##5 ##6 ##7 ##8 ##9 { #1 }
4111     \cs_set:Npn \stex_iterate_break: {
4112       \prop_map_break:n{\seq_map_break:}
4113     }
4114     \cs_set:Npn \stex_iterate_break:n ##1 {
4115       \prop_map_break:n{\seq_map_break:n{##1}}
4116     }
4117     \seq_map_inline:Nn \l_stex_all_modules_seq {
4118       \prop_map_inline:cn{\_stex_symbol_macro:n {##1}}{
4119         \__stex_syms_sym_cs:nnnnnnnnN {##1} #####2
4120       }
4121     }
4122   }
4123 }

```

(End of definition for `\stex_iterate_symbols:n`, `\stex_iterate_break:`, and `\stex_iterate_break:n`. These functions are documented on page 148.)

sfunction

```

\stex_iterate_symbols:nn
4124 \cs_new_protected:Nn \stex_iterate_symbols:nn {
4125   \seq_clear:N \l__stex_syms_mods_seq
4126   \stex_pseudogroup_with:nn{\__stex_syms_sym_cs:nnnnnnnnN}{
4127     \cs_set:Npn \__stex_syms_sym_cs:nnnnnnnnN
4128     ##1 ##2 ##3 ##4 ##5 ##6 ##7 ##8 ##9 { #2 }
4129     \clist_map_function:nN {#1} \__stex_syms_it_decl_i:n
4130   }
4131 }
4132
4133 \cs_new_protected:Nn \__stex_syms_it_decl_i:n {
4134   \seq_if_in:NnF \l__stex_syms_mods_seq {#1} {
4135     \seq_put_left:Nn \l__stex_syms_mods_seq {#1}
4136     \prop_map_inline:cn{\_stex_morphisms_macro:n {#1}}{
4137       \__stex_syms_it_decl_check:nnnn ##2
4138     }

```

```

4139     \prop_map_inline:cn{\_stex_symbol_macro:n {#1} }{
4140       \__stex_syms_sym_cs:nnnnnnnnN {#1} ##2
4141     }
4142   }
4143 }
4144
4145 \cs_new_protected:Nn \__stex_syms_it_decl_check:nnnn {
4146   \tl_if_empty:nT{#1}{
4147     \__stex_syms_it_decl_i:n {#2}
4148   }
4149 }

```

(End of definition for \stex_iterate_symbols:nn. This function is documented on page 148.)

sfunction

```

\stex_get_symbol:n
\stex_get_symbol:nTF
\stex_get_symbol:nF
4150 \int_new:N \l_stex_get_symbol_arity_int
4151
4152 \cs_new_protected:Nn \stex_get_symbol:n {
4153   \stex_get_symbol:nF{ #1 }{
4154     \msg_error:nnn{stex}{error/unknownsymbol}{#1}
4155   }
4156 }
4157
4158 \cs_new_protected:Npn \stex_get_symbol:nTF #1 #2 #3 {
4159   \tl_clear:N \l_stex_get_symbol_uri
4160   \cs_if_exist:cTF { #1 }{
4161     \cs_set_eq:Nc \l__stex_syms_cs { #1 }
4162     % command name
4163     \exp_args:Ne \tl_if_empty:nTF { \cs_argument_spec:N \l__stex_syms_cs }{
4164       % ...that takes no arguments
4165       \exp_args:Ne \cs_if_eq:NNTF {\tl_head:N \l__stex_syms_cs}
4166         \stex_invoke_symbol:nnnnnnN
4167         \__stex_syms_get_symbol_from_cs:
4168         {\__stex_syms_get_symbol_from_string:n { #1 }}
4169     }{
4170       \__stex_syms_get_symbol_from_string:n { #1 }
4171     }
4172   }{
4173     \__stex_syms_get_symbol_from_string:n { #1 }
4174   }
4175   \tl_if_empty:nTF \l_stex_get_symbol_uri { #3 } { #2 }
4176 }
4177
4178 \cs_new_protected:Npn \stex_get_symbol:nF #1 #2 {
4179   \stex_get_symbol:nTF{ #1 }{}{ #2 }
4180 }
4181
4182 \cs_new_protected:Nn \__stex_syms_get_symbol_from_cs: {
4183   \stex_pseudogroup_with:nn{\_stex_invoke_symbol:nnnnnnN}{
4184     \cs_set:Npn \_stex_invoke_symbol:nnnnnnN ##1 ##2 ##3 ##4 ##5 ##6 ##7 {
4185       \tl_set:Nn \l_stex_get_symbol_uri { ##1 }
4186       \int_set:Nn \l_stex_get_symbol_arity_int {##2}
4187       \tl_set:Nn \l_stex_get_symbol_args_tl {##3}

```

```

4188     \tl_set:Nn \l_stex_get_symbol_def_tl {##4}
4189     \tl_set:Nn \l_stex_get_symbol_type_tl {##5}
4190     \tl_set:Nn \l_stex_get_symbol_return_tl {##6}
4191     \tl_set:Nn \l_stex_get_symbol_invoke_cs {##7}
4192   }
4193   \l__stex_syms_cs
4194 }
4195 }
4196
4197 \cs_new_protected:Nn \__stex_syms_get_symbol_from_string:n {
4198   \stex_debug:nn{symbols}{Getting~from~string~#1...}
4199   \seq_set_split:Nnn \l__stex_syms_seq ? {#1}
4200   \seq_pop_right:NN \l__stex_syms_seq \l__stex_syms_name
4201   \seq_if_empty:NTF \l__stex_syms_seq {
4202     \exp_args:No \__stex_syms_get_from_one_string:n {#1}
4203   }{
4204     \exp_args:NNe \exp_args:Nno \__stex_syms_get_symbol_from_modules:nn {
4205       \seq_use:Nn \l__stex_syms_seq ?
4206     } \l__stex_syms_name
4207   }
4208 }
4209
4210 \cs_new_protected:Nn \__stex_syms_sym_from_str_i:nnnn {
4211   \bool_lazy_any:nTF{
4212     {\str_if_eq_p:nn{#2}{#3}}
4213     {\str_if_eq_p:nn{#2}{#4}}
4214     {\stex_str_if_ends_with_p:nn{#4}{/#2}}
4215   }{
4216     \__stex_syms_sym_i_finish:nnnnnnN{#1}{#4}
4217   }{
4218     \__stex_syms_sym_i_gobble:nnnnnn
4219   }
4220 }
4221 \cs_new_protected:Nn \__stex_syms_sym_i_gobble:nnnnnn {}
4222
4223 \cs_new_protected:Nn \__stex_syms_sym_i_finish:nnnnnnN {
4224   \prop_map_break:n{\seq_map_break:n{
4225     \tl_set:Ne \l_stex_get_symbol_uri { \stex_new_symbol_uri:nn {#1} {#2}}
4226     \int_set:Nn \l_stex_get_symbol_arity_int {#3}
4227     \tl_set:Nn \l_stex_get_symbol_args_tl {#4}
4228     \tl_set:Nn \l_stex_get_symbol_def_tl {#5}
4229     \tl_set:Nn \l_stex_get_symbol_type_tl {#6}
4230     \tl_set:Nn \l_stex_get_symbol_return_tl {#7}
4231     \tl_set:Nn \l_stex_get_symbol_invoke_cs {#8}
4232   }}
4233 }
4234
4235 \cs_new_protected:Nn \__stex_syms_get_symbol_from_modules:nn {
4236   %\seq_set_split:Nnn \l_tmpa_seq ? {#1}
4237   %\seq_pop_right:NN \l_tmpa_seq \l__stex_syms_name_str
4238   %\str_set:Ne \l__stex_syms_path_str {\seq_use:Nn \l_tmpa_seq ?}
4239   \stex_debug:nn{symbols}{Getting~#2~in~#1...}
4240   \seq_map_inline:Nn \l_stex_all_modules_seq {
4241     %\stex_debug:nn{symbols}{comparing~\stex_module_uri_as_qm:n{##1}~and~#1}

```



```

4242 \exp_args:Ne \stex_str_if_ends_with:nnT{\stex_module_uri_as_qm:n{##1}}{#1}{
4243 \prop_map_inline:cn{\_stex_symbol_macro:n {##1} }{
4244 \_stex_syms_sym_from_str_i:nnnn{##1}{#2} ####2
4245 }
4246 }
4247
4248 %\str_if_empty:NTF \l__stex_syms_path_str {
4249 % \exp_args:NNe \exp_args:Nno \stex_str_if_ends_with:nnT {\stex_module_uri_name:n{##1}}
4250 %}{
4251 % \exp_args:NNNe \exp_args:Nno \str_if_eq:nnT\l__stex_syms_name_str{
4252 % \stex_module_uri_name:n {##1}
4253 % }
4254 %}{
4255 % \str_if_empty:NTF \l__stex_syms_path_str {
4256 % \prop_map_inline:cn{\_stex_symbol_macro:n {##1} }{
4257 % \_stex_syms_sym_from_str_i:nnnn{##1}{#2} ####2
4258 % }
4259 % }{
4260 % \stex_debug:nn{symbols}{Comparing~\l__stex_syms_path_str;~with~\tl_to_str:n{##1}}
4261 % \exp_args:NNe \exp_args:Nno \stex_str_if_ends_with:nnT{\stex_module_uri_path:n {##1}}
4262 % \prop_map_inline:cn{\_stex_symbol_macro:n {##1} }{
4263 % \_stex_syms_sym_from_str_i:nnnn{##1}{#2} ####2
4264 % }
4265 % }
4266 % }
4267 %}
4268 }
4269 }
4270
4271 \prg_new_conditional:Nnn \_stex_syms_uri_match:n {T,F,TF} {
4272 \str_if_eq:nnT
4273 }
4274
4275
4276 \cs_new_protected:Nn \_stex_syms_get_from_one_string:n {
4277 \stex_debug:nn{symbols}{Getting~#1~anywhere...}
4278 \stex_iterate_symbols:n{
4279 %\stex_debug:nn{symbols}{>#1==#2~|~#1==#3<...}
4280 \bool_lazy_any:nT{
4281 {\str_if_eq_p:nn{#1}{##2}}
4282 {\str_if_eq_p:nn{#1}{##3}}
4283 {\stex_str_if_ends_with_p:nn{##3}{/#1}}
4284 }{
4285 \stex_iterate_break:n{
4286 \tl_set:Ne \l_stex_get_symbol_uri { \stex_new_symbol_uri:nn {##1} {##3}}
4287 \int_set:Nn \l_stex_get_symbol_arity_int {##4}
4288 \tl_set:Nn \l_stex_get_symbol_args_tl {##5}
4289 \tl_set:Nn \l_stex_get_symbol_def_tl {##6}
4290 \tl_set:Nn \l_stex_get_symbol_type_tl {##7}
4291 \tl_set:Nn \l_stex_get_symbol_return_tl {##8}
4292 \tl_set:Nn \l_stex_get_symbol_invoke_cs {##9}
4293 }
4294 }
4295 }

```

4296 }

(End of definition for `\stex_get_symbol:n`, `\stex_get_symbol:nTF`, and `\stex_get_symbol:nF`. These functions are documented on page 148.)

sfunction

32.3 Variables

4297 <@@=stex_vars>

`\l_stex_variables_prop`

4298 \prop_new:N \l_stex_variables_prop

(End of definition for `\l_stex_variables_prop`. This variable is documented on page 148.)

`\vardef`

```

4299 \bool_new:N \l__stex_vars_bind_bool
4300 \cs_new_protected:Nn \_stex_variable:nnnnnnnN {}
4301
4302 \stex_new_stylable_cmd:nnnn {vardef} { m 0{} m} {
4303   \stex_keys_set:nn{vardef}{#2}
4304   \str_set:Ne \l_stex_macroname_str { #1 }
4305   \str_if_empty:NT \l_stex_key_name_str {
4306     \str_set:Ne \l_stex_key_name_str { #1 }
4307   }
4308
4309   \stex_symdecl_do:
4310   \stex_if_check_terms:T \_stex_check_terms:
4311   \__stex_vars_add:
4312   \__stex_vars_macro:
4313   \stex_if_do_html:T \__stex_vars_html:
4314
4315   \int_set:Nn \l_stex_get_symbol_arity_int {\l_stex_get_symbol_arity_int}
4316   \stex_debug:nn{vardef}{Doing~\l_stex_key_name_str}
4317   \tl_set_eq:NN \l_stex_get_symbol_return_tl \l_stex_key_return_tl
4318   \stex_notation_parse:n{#3}
4319   \stex_if_check_terms:T{ \_stex_notation_check: }
4320   \_stex_var_notation_macro:
4321   \stex_if_do_html:T {
4322     \def\comp{\_varcomp}
4323     \_stex_notation_do_html:n \l_stex_key_name_str
4324   }
4325   \group_begin:
4326   \tl_set_eq:NN \thisvarname \l_stex_key_name_str
4327   \tl_clear:N \thisstyle
4328   \str_set_eq:NN\thisnotationvariant\l_stex_key_variant_str
4329   \def\thisnotation{
4330     \exp_args:Nne \use:nn{\l_stex_notation_macrocode_cs}{ }{
4331       \_stex_notation_make_args:
4332     }
4333   }
4334   \stex_style_apply:
4335   \group_end:\ignorespaces
4336 }{}

```

(End of definition for `\vardef`. This function is documented on page 148.)

sfunction

Adds a new variable to the current scope:

```

4337 \cs_new_protected:Nn \__stex_vars_add: {
4338   \exp_args:NNNo \exp_args:NNne
4339   \prop_put:Nnn \l_stex_variables_prop \l_stex_key_name_str {
4340     {\l_stex_macroname_str}
4341     {\l_stex_key_name_str}
4342     {\int_use:N \l_stex_get_symbol_arity_int}
4343     {\l_stex_get_symbol_args_tl}
4344     {\exp_args:No \exp_not:n \l_stex_key_def_tl}
4345     {\exp_args:No \exp_not:n \l_stex_key_type_tl}
4346     {\exp_args:No \exp_not:n \l_stex_key_return_tl}
4347     \stex_invoke_symbol:
4348   }
4349 }
```

Defines the macro for a variable:

```

4350 \cs_new_protected:Nn \__stex_vars_macro: {
4351   \tl_set:ce{\l_stex_macroname_str}{
4352     \stex_invoke_variable:nnnnnnN
4353     {\l_stex_key_name_str}
4354     {\int_use:N \l_stex_get_symbol_arity_int}
4355     {\l_stex_get_symbol_args_tl}
4356     {\exp_args:No \exp_not:n \l_stex_key_def_tl}
4357     {\exp_args:No \exp_not:n \l_stex_key_type_tl}
4358     {\exp_args:No \exp_not:n \l_stex_key_return_tl}
4359     \stex_invoke_symbol:
4360   }
4361 }
```

Insert the HTML for a variable declaration:

```

4362
4363 \cs_new_protected:Nn \__stex_vars_html: {
4364   \stex_if_do_html:T {
4365     \hbox\bgroup\exp_args:Ne \stex_annotate_invisible:nn {
4366       data-ftml-vardef = {\l_stex_key_name_str},
4367       data-ftml-args = {\l_stex_key_args_str}
4368       \str_if_empty:NF \l_stex_macroname_str {,
4369         data-ftml-macroname={\l_stex_macroname_str}
4370       }
4371       \str_if_empty:NF \l_stex_key_assoc_str {,
4372         data-ftml-assoctype={\l_stex_key_assoc_str}
4373       }
4374       \str_if_empty:NF \l_stex_key_role_str {,
4375         data-ftml-role={\l_stex_key_role_str}
4376       }
4377       \str_if_empty:NF \l_stex_key_reorder_str {,
4378         data-ftml-reorderargs={\l_stex_key_reorder_str}
4379       }
4380       \bool_if:NT \l__stex_vars_bind_bool {,
4381         data-ftml-bind={true}
4382       }
4383     }\}
```

```

4384 \stex_annotate_force_break:n{
4385   \bool_set_true:N \stex_in_invisible_html_bool
4386   \tl_if_empty:NF \l_stex_key_type_tl {
4387     \stex_annotate:nn{data-ftml-type={}}{${\l_stex_key_type_tl$}
4388   }
4389   \tl_if_empty:NF \l_stex_key_def_tl {
4390     \stex_annotate:nn{data-ftml-definiens={}}{${\l_stex_key_def_tl$}
4391   }
4392   \tl_if_empty:NF \l_stex_key_return_tl{
4393     \exp_args:Nno \use:n{
4394       \cs_generate_from_arg_count:NNnn \l__stex_vars_cs
4395       \cs_set:Npn \l_stex_get_symbol_arity_int} \l_stex_key_return_tl
4396       \tl_set:Ne \l__stex_vars_args_tl {\stex_map_args:N \stex_return_args:nn}
4397       ${\stex_annotate:nn{data-ftml-returntype={}}}{
4398         \exp_after:wN \l__stex_vars_cs \l__stex_vars_args_tl!}$
4399     }
4400     \tl_if_empty:NF \l_stex_key_argtypes_clist {
4401       \stex_annotate:nn{data-ftml-argtypes={}}{
4402         \stex_annotate_force_break:n{
4403           \clist_map_inline:Nn \l_stex_key_argtypes_clist {
4404             ${\stex_annotate:nn{data-ftml-type={}}}{##1}$
4405           }
4406         }
4407       }
4408     }
4409   }
4410 } \egroup
4411 }
4412 }

```

`\newvar`

```

4413 \NewDocumentCommand\newvar{ m O{} }{
4414   \vardef{v#1}[name=#1,#2]{#1}
4415 }

```

(End of definition for `\newvar`. This function is documented on page 148.)

sfunction

`\newseq`

```

4416 \NewDocumentCommand\newseq{ m O{} m }{
4417   \varseq{v#1}[name=#1,#2]{#3}{\maincomp{#1}}\c_math_subscript_token{##1}}
4418 }

```

(End of definition for `\newseq`. This function is documented on page 148.)

sfunction

`\stex_get_var:n`

```

4419 \cs_new_protected:Nn \stex_get_var:n {
4420   \str_clear:N \l_stex_get_variable_str
4421   \__stex_vars_get_var:n{#1}
4422   \str_if_empty:NT \l_stex_get_variable_str {
4423     \msg_error:nnn{stex}{error/unknownsymbol}{#1}
4424   }
4425 }

```

```

4426
4427 \cs_new_protected:Nn \__stex_vars_get_var:n {
4428   \prop_map_inline:Nn \l_stex_variables_prop {
4429     \__stex_vars_check_var:nnnnnnnnN {#1} ##2
4430   }
4431 }
4432
4433 \cs_new_protected:Nn \__stex_vars_check_var:nnnnnnnnN {
4434   \str_if_eq:nnTF{#1}{#2}{
4435     \prop_map_break:n{\__stex_vars_set_vars:nnnnnnN {#3}{#4}{#5}{#6}{#7}{#8}{#9}}
4436   }{
4437     \str_if_eq:nnT{#1}{#3}{
4438       \prop_map_break:n{\__stex_vars_set_vars:nnnnnnN {#3}{#4}{#5}{#6}{#7}{#8}{#9}}
4439     }
4440   }
4441 }
4442
4443 \cs_new_protected:Nn \__stex_vars_set_vars:nnnnnnN {
4444   \stex_debug:nn{symbols}{Variable~#1~found}
4445   \cs_set:Npn \_stex_variable:nnnnnnnnN ##1 ##2 ##3 ##4 ##5 ##6 ##7 ##8 {}
4446   \str_set:Nn \l_stex_get_variable_str {#1}
4447   \int_set:Nn \l_stex_get_symbol_arity_int {#2}
4448   \tl_set:Nn \l_stex_get_symbol_args_tl {#3}
4449   \tl_set:Nn \l_stex_get_symbol_def_tl {#4}
4450   \tl_set:Nn \l_stex_get_symbol_type_tl {#5}
4451   \tl_set:Nn \l_stex_get_symbol_return_tl {#6}
4452   \tl_set:Nn \l_stex_get_symbol_invoke_cs {#7}
4453 }

```

(End of definition for `\stex_get_var:n`. This function is documented on page 149.)

sfunction

32.3.1 Variable Sequences

```

4454 <@@=stex_seqs>

\varseq

4455 \stex_new_stylable_cmd:nnnn {varseq}{m 0}{ m m} {
4456   \stex_keys_set:nn{symdef}{#2}
4457   \str_set:Ne \l_stex_macroname_str { #1 }
4458   \str_if_empty:NT \l_stex_key_name_str {
4459     \str_set:Ne \l_stex_key_name_str { #1 }
4460   }
4461   \str_if_empty:NT \l_stex_key_args_str {
4462     \str_set:Nn \l_stex_key_args_str {1}
4463   }
4464   \stex_symdecl_do:
4465
4466   \tl_set_eq:NN \l_stex_get_symbol_return_tl \l_stex_key_return_tl
4467   \clist_set:Nn \l__stex_seqs_range_clist {#3}
4468   \tl_if_empty:NTF \l_stex_key_op_tl {
4469     \stex_notation_parse:n{#4}
4470     \tl_clear:N \l_stex_key_op_tl
4471   }{

```

```

4472     \stex_notation_parse:n{#4}
4473   }
4474   \stex_if_do_html:T \__stex_seqs_html:
4475   \stex_if_check_terms:T \_stex_check_terms:
4476   \__stex_seqs_add:
4477   \__stex_seqs_macro:
4478   \stex_if_check_terms:T \_stex_notation_check:
4479   \_stex_var_notation_macro:
4480   \stex_if_do_html:T {
4481     \def\comp{\_varcomp}
4482     \_stex_notation_do_html:n \l_stex_key_name_str
4483   }
4484   \group_begin:
4485   \tl_set_eq:NN \thisvarname \l_stex_key_name_str
4486   \tl_clear:N \thisstyle
4487   \str_set_eq:NN\thisnotationvariant\l_stex_key_variant_str
4488   \def\thisnotation{
4489     \l_stex_notation_macrocode_cs{}!
4490   }
4491   \stex_style_apply:
4492   \group_end:\ignorespaces
4493 }{}
4494
4495 \cs_new_protected:Nn \__stex_seqs_add: {
4496   \exp_args:NNNo \exp_args:NNne
4497   \prop_put:Nnn \l_stex_variables_prop \l_stex_key_name_str {
4498     {\l_stex_macroname_str}
4499     {\l_stex_key_name_str}
4500     {\int_use:N \l_stex_get_symbol_arity_int}
4501     {\l_stex_get_symbol_args_tl}
4502     {\exp_args:No \exp_not:n \l_stex_key_def_tl}
4503     {\exp_args:No \exp_not:n \l__stex_seqs_range_clist}
4504     {\exp_args:No \exp_not:n \l_stex_key_return_tl}
4505     \stex_invoke_sequence:
4506   }
4507 }
4508
4509 \cs_new_protected:Nn \__stex_seqs_macro: {
4510   \tl_set:ce{\l_stex_macroname_str}{
4511     \_stex_invoke_variable:nnnnnnN
4512     {\l_stex_key_name_str}
4513     {\int_use:N \l_stex_get_symbol_arity_int}
4514     {\l_stex_get_symbol_args_tl}
4515     {\exp_args:No \exp_not:n \l_stex_key_def_tl}
4516     {\exp_args:No \exp_not:n \l__stex_seqs_range_clist}
4517     {\exp_args:No \exp_not:n \l_stex_key_return_tl}
4518     \stex_invoke_sequence:
4519   }
4520 }
4521
4522 \cs_new_protected:Nn \__stex_seqs_html: {
4523   \exp_args:Ne \stex_annotate_invisible:nn {
4524     data-ftml-varseq = {\l_stex_key_name_str},
4525     data-ftml-args = {\l_stex_key_args_str}

```

```

4526 \str_if_empty:NF \l_stex_macroname_str {,
4527   data-ftml-macroname={\l_stex_macroname_str}
4528 }
4529 \str_if_empty:NF \l_stex_key_assoc_str {,
4530   data-ftml-assoctype={\l_stex_key_assoc_str}
4531 }
4532 \str_if_empty:NF \l_stex_key_role_str {,
4533   data-ftml-role={\l_stex_key_role_str}
4534 }
4535 \str_if_empty:NF \l_stex_key_reorder_str {,
4536   data-ftml-reorderargs={\l_stex_key_reorder_str}
4537 }
4538 }{\hbox\bgroup
4539   \stex_annotate_force_break:n{
4540
4541     $\stex_annotate:nn{data-ftml-seqrange={}}{
4542       \exp_args:Nno\symuse{Metatheory?sequence-range}\l__stex_seqs_range_clist
4543     }$
4544
4545     \tl_if_empty:NF \l_stex_key_type_tl {
4546       \stex_annotate:nn{data-ftml-type={}}{${\l_stex_key_type_tl$}
4547     }
4548     \tl_if_empty:NF \l_stex_key_def_tl {
4549       \stex_annotate:nn{data-ftml-definiens={}}{${\l_stex_key_def_tl$}
4550     }
4551     \tl_if_empty:NF \l_stex_key_return_tl{
4552       \exp_args:Nno \use:n{
4553         \cs_generate_from_arg_count:NNnn \l__stex_seqs_cs
4554         \cs_set:Npn \l_stex_get_symbol_arity_int} \l_stex_key_return_tl
4555         \tl_set:Nx \l__stex_seqs_args_tl {\stex_map_args:N \stex_return_args:nn}
4556         \stex_annotate:nn{data-ftml-returntype={}}{
4557           $\exp_after:wN \l__stex_seqs_cs \l__stex_seqs_args_tl!$}
4558       }
4559       \tl_if_empty:NF \l_stex_key_argtypes_clist {
4560         \stex_annotate:nn{data-ftml-argtypes={}}{
4561           \stex_annotate_force_break:n{
4562             \clist_map_inline:Nn \l_stex_key_argtypes_clist {
4563               \stex_annotate:nn{data-ftml-type={}}{####1$}
4564             }
4565           }
4566         }
4567       }
4568     }
4569   \egroup}
4570 }

```

(End of definition for \varseq. This function is documented on page 149.)

sfunction

32.4 Expressions

```

4571 <@@=stex_expr>
\symuse

```

```

4572 \cs_new_protected:Npn \symuse #1 {
4573   \stex_get_symbol:n{#1}
4574   \exp_args:Nno \use:n {\_stex_invoke_symbol:NeooooN
4575   \l_stex_get_symbol_uri
4576   {\int_use:N \l_stex_get_symbol_arity_int}
4577   \l_stex_get_symbol_args_tl
4578   \l_stex_get_symbol_def_tl
4579   \l_stex_get_symbol_type_tl
4580   \l_stex_get_symbol_return_tl}
4581   \l_stex_get_symbol_invoke_cs
4582 }

```

(End of definition for \symuse. This function is documented on page 149.)
sfunction

_stex_next_symbol:n

```

4583 \tl_new:N \l__stex_expr_reset_tl
4584
4585 \cs_new_protected:Nn \_stex_next_symbol:n {
4586   \tl_set:Nc \l_stex_every_symbol_tl {
4587     \tl_gset:Nn \exp_not:N \l__stex_expr_reset_tl {
4588       \tl_set:Nn \exp_not:N \l_stex_every_symbol_tl {
4589         \exp_args:No \exp_not:n \l_stex_every_symbol_tl
4590       }
4591       \tl_gset:Nn \exp_not:N \l__stex_expr_reset_tl {
4592         \exp_args:No \exp_not:n \l__stex_expr_reset_tl
4593       }
4594     }
4595     \tl_set:Nn \exp_not:N \l_stex_every_symbol_tl {
4596       \exp_args:No \exp_not:n \l_stex_every_symbol_tl
4597     }
4598     \exp_not:n{ \aftergroup \l__stex_expr_reset_tl }
4599     \exp_not:N \l_stex_every_symbol_tl
4600     \exp_not:n{ #1 }
4601   }
4602 }
4603 \cs_generate_variant:Nn \_stex_next_symbol:n {e}

```

(End of definition for _stex_next_symbol:n. This function is documented on page 149.)
sfunction

_stex_invoke_symbol:NnnnnnN
_stex_invoke_symbol:NeooooN
_stex_invoke_symbol:nnnnnnN

- \l_stex_current_symbol_uri,
- \l_stex_current_symbol_arity_int,
- \l_stex_current_symbol_args_tl,
- definiens (currently not used),
- \l_stex_current_symbol_type_tl,
- \l_stex_current_symbol_return_tl,

- the invocation macro (is called after setup).

```

4604 \cs_new_protected:Nn \_stex_invoke_symbol:nnnnnnN {
4605   \bool_if:NTF \l_stex_allow_semantic_bool{
4606     \group_begin:
4607     \tl_set:Nn \l_stex_current_symbol_uri {#1}
4608     \tl_set:Nn \l_stex_current_full_tl { \stex_use_symbol_uri:N \l_stex_current_symbol_uri
4609     \tl_set:Nn \l_stex_current_display_tl {
4610       \stex_symbol_uri_name:N \l_stex_current_symbol_uri
4611     }
4612     \__stex_expr_invoke:nnnnN{#2}{#3}{#5}{#6}{#7}
4613   }{
4614     \msg_error:nnxx{stex}{error/notallowed}{
4615       \stex_use_symbol_uri:n{#1}
4616     }{
4617       \l_stex_current_full_tl
4618     }
4619   }
4620 }
4621
4622 \cs_new_protected:Npn \_stex_invoke_symbol:NnnnnnnN {
4623   \exp_args:No \_stex_invoke_symbol:nnnnnnN
4624 }
4625 \cs_generate_variant:Nn \_stex_invoke_symbol:NnnnnnnN {NeooooN}

```

Whether semantic macros are allowed currently. This is false e.g. in the body of a semantic macro outside of one of its arguments:

```

4626 \bool_new:N \l_stex_allow_semantic_bool
4627 \bool_set_true:N \l_stex_allow_semantic_bool

```

Code to execute every time a semantic macro is invoked:

```

4628 \tl_set:Nn \l_stex_every_symbol_tl {
4629   \bool_set_false:N \l_stex_allow_semantic_bool
4630 }

```

The current top-level term; may be undefined:

```

4631 \tl_new:N \l_stex_current_term_tl

```

Keeps track of all set up so it can be reexecuted; this may be necessary to e.g. typeset `\this` in a struct field:

```

4632 \tl_new:N \l_stex_current_redo_tl

```

Setup for symbols; takes: arity, args, type, return, invoke. Calls invoke at the end.

In HTML mode, we make sure we enter horizontal mode *before* any annotations are inserted

```

4633 \cs_new_protected:Nn \__stex_expr_invoke:nnnnN {
4634   \stex_if_html_backend:T{\relax\ifvmode\indent\fi}
4635   \__stex_expr_setup:nnnnn{\_comp}{#1}{#2}{#4}{#3}
4636   \cs_set_eq:NN \_stex_term_oms_or_omv:nn \_stex_term_oms:nn
4637   \tl_put_right:Nn \l_stex_current_redo_tl{
4638     \cs_set_eq:NN \_stex_term_oms_or_omv:nn \_stex_term_oms:nn
4639   }
4640   #5
4641 }

```

general setup; takes: comp, arity, args, return, type

```

4642 \cs_new_protected:Nn \__stex_expr_setup:nnnnn {
4643   \tl_clear:N \l_stex_return_notation_tl
4644   \tl_set:Nn \l_stex_current_redo_tl {
4645     \let \this \stex_current_this:
4646     \def\comp{#1}
4647     \def\maincomp{\comp}
4648     \str_set:Nn \l_stex_current_arity_str{ #2 }
4649     \tl_set:Nn \l_stex_current_args_tl{ #3 }
4650     \tl_set:Nn \l_stex_current_return_tl{ #4 }
4651     \tl_set:Nn \l_stex_current_type_tl{ #5 }
4652     \tl_clear:N \l_stex_current_term_tl
4653   }
4654   \tl_put_right:N \l_stex_current_redo_tl \l_stex_every_symbol_tl
4655   \tl_put_left:Ne \l_stex_current_redo_tl {
4656     \tl_set:Nn \exp_not:N \l_stex_current_full_tl { \l_stex_current_full_tl}
4657   }
4658   \l_stex_current_redo_tl
4659 }

```

(End of definition for `\stex_invoke_symbol:NnnnnnN` and `\stex_invoke_symbol:nnnnnnN`. These functions are documented on page 149.)

sfunction

`\stex_invoke_symbol:` Math or text mode?

```

4660 \cs_new_protected:Nn \stex_invoke_symbol: {
4661   \stex_debug:nn{expressions}{Invoking~ \l_stex_current_full_tl}
4662   \mode_if_math:TF \__stex_expr_math: \__stex_expr_text:
4663 }

```

(End of definition for `\stex_invoke_symbol:`. This function is documented on page 149.)

sfunction

`\stex_invoke_text_symbol:`

```

4664 \cs_new_protected:Nn \stex_invoke_text_symbol: {
4665   \stex_if_html_backend:T{\relax\ifvmode\indent\fi}
4666   \stex_term_oms_or_omv:nn{}{\maincomp{\let\xspace\relax\l_stex_current_return_tl}}
4667   \group_end:\mode_if_math:F{\cs_if_exist:NT\xspace\xspace}
4668 }

```

(End of definition for `\stex_invoke_text_symbol:`. This function is documented on page 149.)

sfunction

`\stex_invoke_sequence:`

```

4669 \cs_new_protected:Nn \stex_invoke_sequence: {
4670   \peek_charcode_remove:NTF ! {
4671     \peek_charcode:NTF [ \__stex_expr_seq_op:w { \__stex_expr_seq_op:w [] }
4672     }\__stex_expr_seq_first:
4673   }
4674
4675   \cs_new_protected:Npn \__stex_expr_seq_op:w [#1] {
4676     \stex_use_op_notation:nnTF \l_stex_current_full_tl{#1}{
4677       %\stex_debug:nn{vars}{Here: \meaning\l_stex_notation_cs}
4678       \stex_maybe_brackets:nn{\neginfprec}{

```

```

4679     \_stex_term_oms_or_omv:nn{#1}
4680     {\l_stex_notation_cs}\group_end:
4681   }
4682   ){
4683     \_stex_expr_get_index_notation:n{#1}
4684     \peek_charcode:NTF [ \_stex_expr_range:w { \_stex_expr_range:w[] }
4685   }
4686 }
4687
4688 \cs_new_protected:Nn \_stex_expr_get_index_notation:n {
4689   \stex_use_notation:nnTF \l_stex_current_full_tl{#1}{}{
4690     \stex_do_default_notation:
4691     \cs_set_eq:NN \l_stex_notation_cs \l_stex_default_notation
4692   }
4693 }
4694
4695 \cs_new_protected:Npn \_stex_expr_range:w [#1] {
4696   \bool_set_true:N \l_stex_allow_semantic_bool
4697   \clist_clear:N \l__stex_expr_clist
4698   \clist_map_function:NN \l_stex_current_type_tl \_stex_expr_seq_arg:n
4699   \stex_annotate:nn{
4700     data-ftml-term=OMV,
4701     data-ftml-head={\l_stex_current_full_tl},
4702     data-ftml-notationid={}
4703   }{
4704     \l__stex_expr_clist
4705   }
4706   \group_end:
4707 }
4708
4709 \cs_new_protected:Nn \_stex_expr_seq_arg:n {
4710   \tl_if_eq:nnTF{#1}{\ellipses}{
4711     \clist_put_right:Nn \l__stex_expr_clist {
4712       \ellipses
4713     }
4714   }{
4715     \clist_put_right:Nn \l__stex_expr_clist {
4716       \exp_args:No \str_if_eq:nnTF \l_stex_current_arity_str {1}{
4717         \group_begin:
4718         \l_stex_notation_cs \group_end: {#1}
4719       }{
4720         \group_begin:
4721         \l_stex_notation_cs \group_end: #1
4722       }
4723     }
4724   }
4725 }
4726 }
4727
4728 \cs_new_protected:Nn \_stex_expr_seq_first: {
4729   \exp_args:Nne \use:nn{
4730     \cs_generate_from_arg_count:NNnn \l_stex_notation_cs \cs_set:Npn
4731     \l_stex_current_arity_str}{
4732     \tl_set:Nn \exp_not:N \l__stex_expr_first_args_tl {

```

```

4733     \int_step_function:nN \l_stex_current_arity_str \__stex_expr_do_first_arg:n
4734   }
4735   \exp_not:N \__stex_expr_do_first_next:
4736 }
4737 \l_stex_notation_cs
4738 }
4739
4740 \cs_new:Nn \__stex_expr_do_first_arg:n {{\exp_not:n{## #1}}}
4741
4742 \cs_new_protected:Nn \__stex_expr_do_first_next: {
4743   \peek_charcode_remove:NTF ! {
4744     \peek_charcode:NTF [ \__stex_expr_do_one:w {\__stex_expr_do_one:w []}
4745   }{
4746     \peek_charcode:NTF [ \__stex_expr_do_all:w {\__stex_expr_do_all:w []}
4747   }
4748 }
4749
4750 \cs_new_protected:Npn \__stex_expr_do_one:w [#1] {
4751   \__stex_expr_get_index_notation:n{#1}
4752   \exp_args:Nno\use:nn{\l_stex_notation_cs\group_end:}\l__stex_expr_first_args_tl
4753 }
4754
4755 \cs_new_protected:Npn \__stex_expr_do_all:w [#1] {
4756   \exp_args:Nno\use:nn{\__stex_expr_notation:w [#1]}\l__stex_expr_first_args_tl
4757 }

```

(End of definition for `\stex_invoke_sequence:..` This function is documented on page 150.)

sfunction

`\stex_invoke_structure:`

```

4758 \cs_new_protected:Nn \stex_invoke_structure: {
4759   \tl_set:Nn \l__stex_expr_set_comp_tl {\__stex_expr_set_thiscomp:}
4760   \__stex_expr_struct_top:n {}
4761 }
4762
4763 \cs_new_protected:Nn \__stex_expr_set_thiscomp: {
4764   \exp_args:Ne \tl_if_eq:NNF {\tl_head:N \maincomp} \_thiscomp {
4765     \edef\maincomp {\_thiscomp{\comp}}
4766   }
4767 }
4768
4769 \cs_new_protected:Nn \__stex_expr_struct_top:n {
4770   \stex_debug:nn{structure}{
4771     invoking~structure~{\l_stex_current_type_tl}<\tl_to_str:n{#1}>
4772   }
4773   \peek_charcode:NTF [ {
4774     \__stex_expr_merge:nw{#1}
4775   }{
4776     \__stex_expr_struct_type:n {#1}
4777     \tl_set:Nn \l_stex_struct_this_tl {}
4778     \peek_charcode_remove:NTF ! {
4779       \peek_charcode:NTF [ {
4780         \__stex_expr_maybe_notation:w
4781       }{

```

```

4782     \_stex_expr_maybe_notation:w []
4783   }
4784   }{
4785     \_stex_expr_invoke_this:n
4786   }
4787 }
4788 }
4789
4790 \cs_new_protected:Npn \_stex_expr_merge:nw #1 [ #2 ] {
4791   \exp_args:Ne \stex_str_if_starts_with:nnTF {\tl_to_str:n{#2}}{comp=}{
4792     \_stex_expr_set_customcomp: #2 \_stex_expr_end:
4793     \_stex_expr_struct_top:n{#1}
4794   }{
4795     \exp_args:Ne \stex_str_if_starts_with:nnTF {\tl_to_str:n{#2}}{this=}{
4796       \_stex_expr_set_thisnotation: #2 \_stex_expr_end:
4797       \_stex_expr_struct_top:n{#1}
4798     }{
4799       \tl_if_empty:nTF{#1}{
4800         \_stex_expr_struct_top:n{#2}
4801       }{
4802         \tl_if_empty:nTF{#2}{
4803           \_stex_expr_struct_top:n{#1}
4804         }{
4805           \_stex_expr_struct_top:n{#1,#2}
4806         }
4807       }
4808     }
4809   }
4810 }
4811
4812 \cs_new_protected:Npn \_stex_expr_set_thisnotation: this= #1 \_stex_expr_end: {
4813   \tl_set:Nn \l_stex_return_notation_tl { \comp{#1} }
4814   \tl_set:Nn \l__stex_expr_set_comp_tl {}
4815 }
4816
4817 \cs_new_protected:Npn \_stex_expr_set_customcomp: comp= #1 \_stex_expr_end: {
4818   \tl_set:Nn \l__stex_expr_set_comp_tl {
4819     \_stex_expr_set_custom_comp:n{#1}
4820   }
4821   \tl_set:Nn \l_stex_return_notation_tl { \comp{} }
4822 }

```

The structure type:

```

4823 \cs_new_protected:Nn \_stex_expr_struct_type:n {
4824   \_stex_expr_do_assign_list:n{#1}
4825   \clist_if_empty:NTF \l__stex_expr_fields_clist {
4826     \int_compare:nNnTF {\clist_count:N \l_stex_current_type_tl}
4827       = 1 {
4828       \tl_set:Ne \l_stex_expr_current_type_tl {
4829         \tl_set:Nn \exp_not:N \l_stex_current_full_tl { \l_stex_current_full_tl }
4830         \exp_args:No \exp_not:n \l_stex_current_redo_tl
4831         \_stex_term oms_or_omv:nn{}{}
4832       }
4833     }{
4834       \exp_args:No \_stex_expr_make_type:n \l_stex_current_type_tl

```

```

4835     }
4836   }{
4837     \int_compare:nNnTF {\clist_count:N \l_stex_current_type_tl}
4838       = 1 {
4839         \__stex_expr_make_type:n {}
4840       }{
4841         \exp_args:No \__stex_expr_make_type:n \l_stex_current_type_tl
4842       }
4843     }
4844   }
4845
4846   \cs_new_protected:Nn \__stex_expr_do_assign_list:n {
4847     \clist_clear:N \l__stex_expr_fields_clist
4848     \tl_if_empty:nF {#1} {
4849       \keyval_parse:NNn\TODO\__stex_expr_do_assign:nn{#1}
4850     }
4851   }
4852
4853   \cs_new_protected:Nn \__stex_expr_do_assign:nn {
4854     \clist_put_right:Nn \l__stex_expr_fields_clist {{#1}{#2}}
4855   }
4856
4857   \cs_new_protected:Nn \__stex_expr_make_type:n {
4858     \tl_if_empty:nTF{#1}{
4859       \seq_clear:N \l_tmpa_seq
4860     }{
4861       \seq_set_split:Nnn \l_tmpa_seq ,{#1}
4862       \seq_pop_right:NN \l_tmpa_seq \l_tmpa_tl
4863       \seq_reverse:N \l_tmpa_seq
4864     }
4865     \tl_set:Ne \l__stex_expr_current_type_tl {
4866       \symuse{Metatheory?module~type~merge}{
4867         {
4868           \exp_args:No \exp_not:n \l_stex_current_redo_tl
4869           \stex_term_oms_or_omv:nn{}{}}
4870       }
4871     \seq_map_function:NN \l_tmpa_seq \__stex_expr_make_mod:n
4872     \clist_if_empty:NF \l__stex_expr_fields_clist {
4873       ,\symuse{Metatheory?anonymous~record}{
4874         \exp_args:Ne \tl_tail:n{
4875           \clist_map_function:NN \l__stex_expr_fields_clist \__stex_expr_make_oml:n
4876         }
4877       }
4878     }
4879   }
4880 }
4881 }
4882
4883 \cs_new:Nn \__stex_expr_make_mod:n {
4884   ,\symuse{Metatheory?module~type}{
4885     \exp_args:Ne \stex_annotate:nn{data-ftml-term=OMMOD,data-ftml-head={\stex_use_module_ur
4886   }
4887 }
4888

```

```

4889
4890 \cs_new:Nn \__stex_expr_make_oml:n {
4891   \__stex_expr_make_oml:nn #1
4892 }
4893 \cs_new:Nn \__stex_expr_make_oml:nn {
4894   ,\stex_annotate:nn{
4895     data-ftml-term=OML,
4896     data-ftml-head={#1}
4897   }{
4898     \stex_annotate_force_break:n{
4899       \stex_annotate:nn{data-ftml-definiens={}}{\exp_not:n{#2!}}
4900     }
4901   }
4902 }

```

Insert the structure type as a term:

```

4903 \cs_new:Nn \__stex_expr_current_type: {
4904   %\exp_args:No \exp_not:n \l_stex_current_redo_tl
4905   \tl_set:Nn \exp_not:N \l_stex_current_term_tl {
4906     \exp_args:No\exp_not:n\l__stex_expr_current_type_tl
4907   }
4908   \stex_term_oms_or_omv:nn{}{}
4909 }

```

The structure type itself:

```

4910 \cs_new_protected:Npn \__stex_expr_maybe_notation:w [ #1 ] {
4911   \tl_set_eq:NN \l_stex_current_term_tl \l__stex_expr_current_type_tl
4912   \stex_use_notation:nnTF \l_stex_current_full_tl {#1}{
4913     \l_stex_notation_cs\group_end:
4914   }{
4915     \__stex_expr_make_prop:
4916     \__stex_expr_make_prop_assign:
4917     \__stex_expr_present_i:w [ #1 ]
4918   }
4919 }
4920
4921 \cs_new_protected:Nn \__stex_expr_present: {
4922   \peek_charcode:NTF [ {
4923     \__stex_expr_present_i:w
4924   }{
4925     \__stex_expr_present:nn{}{}
4926   }
4927 }
4928
4929 \cs_new_protected:Npn \__stex_expr_present_i:w [ #1 ] {
4930   \int_compare:nNnTF{\clist_count:n{#1}} = 1 {
4931     \__stex_expr_present:nn{}{#1}
4932   }{
4933     \peek_charcode:NTF [ {
4934       \__stex_expr_present_ii:nw{#1}
4935     }{
4936       \__stex_expr_present:nn{#1}{}
4937     }
4938   }
4939 }

```

```

4940
4941 %First: clist, second:notation-id
4942 \cs_new_protected:Npn \__stex_expr_present_ii:nw #1 [#2] {
4943   \__stex_expr_present:nn{#1}{#2}
4944 }
4945
4946 \cs_new_protected:Nn \__stex_expr_present:nn {
4947   \clist_clear:N \l__stex_expr_clist
4948   \tl_if_empty:nTF{#1}{
4949     \cs_set:Npn \l__stex_expr_cs ##1 ##2 ##3 {
4950       \tl_if_empty:nF{##2}{
4951         \__stex_expr_present_entry:nn {##1}{##3}
4952       }
4953     }
4954   }{
4955     \cs_set:Npn \l__stex_expr_cs ##1 ##2 ##3 {
4956       \exp_args:Ne \clist_if_in:nnT{\tl_to_str:n{#1}}{##1}{
4957         \__stex_expr_present_entry:nn {##1}{##3}
4958       }
4959     }
4960   }
4961   \prop_map_inline:Nn \l__stex_expr_prop {
4962     \l__stex_expr_cs {##1} ##2
4963   }
4964   \stex_term_oms_or_omv:nn{}{
4965     \exp_args:Nno \use:n{
4966       \bool_set_true:N \l_stex_allow_semantic_bool
4967       \symuse{Metatheory?mathematical~structure}[#2]
4968     }{\l__stex_expr_clist}
4969   }\group_end:
4970 }
4971
4972 \cs_new_protected:Nn \__stex_expr_present_entry:nn {
4973   \seq_if_in:NnTF \l__stex_expr_assigned_seq {#1}{
4974     \clist_put_right:Nn \l__stex_expr_clist {#2!}
4975   }{
4976     \exp_args:Nne \clist_put_right:Nn \l__stex_expr_clist {
4977       \stex_next_symbol:n {
4978         \exp_args:No \exp_not:n \l__stex_expr_set_comp_tl
4979         \tl_set:Nn \exp_not:N \l_stex_struct_this_tl {
4980           \exp_args:No \exp_not:n \l_stex_struct_this_tl
4981         }
4982         \exp_not:n {
4983           \tl_set_eq:NN \this \l_stex_struct_this_tl
4984         }
4985         \tl_set:Nn \exp_not:N \l_stex_return_notation_tl {
4986           \exp_args:No \exp_not:n \l_stex_return_notation_tl
4987         }
4988       }
4989       \exp_not:n{#2!}
4990     }
4991   }
4992 }
4993

```



```

4994 %\cs_new_protected:Nn \__stex_expr_set_custom_comp:n {
4995 % \exp_args:Ne \tl_if_eq:NNF {\tl_head:N \maincomp} \_customthiscomp {
4996 % \cs_set_protected:Npx \_customthiscomp ##1 {
4997 % \group_begin:
4998 % \bool_set_true:N \l_stex_allow_semantic_bool
4999 % \exp_not:n{
5000 % \cs_set:Npn \l__stex_expr_comp_cs ##1 {
5001 % #1
5002 % }
5003 % \def\maincomp
5004 % }{\comp}
5005 % \exp_not:N\l__stex_expr_comp_cs{\comp{##1}}
5006 % \group_end:
5007 % }
5008 % \def\maincomp {\_customthiscomp}
5009 % }
5010 %}

5011
5012 \cs_new_protected:Nn \__stex_expr_set_custom_comp:n {
5013 \exp_args:Ne \tl_if_eq:NNF {\tl_head:N \maincomp} \_customthiscomp {
5014 \stex_debug:nn{structs}{Setting~custom~comp:~\tl_to_str:n{##1}}
5015 \exp_args:Nne \__stex_expr_set_custom_i:nn{##1}{\comp}
5016 \def\maincomp {\_customthiscomp}
5017 }
5018 }
5019
5020 \cs_new_protected:Nn \__stex_expr_set_custom_i:nn {
5021 \cs_set_protected:Npn \_customthiscomp ##1 {
5022 \group_begin:
5023 \bool_set_true:N \l_stex_allow_semantic_bool
5024 \cs_set:Npn \l__stex_expr_comp_cs ####1 { #1 }
5025 \def \maincomp {#2}
5026 \l__stex_expr_comp_cs{#2{##1}}
5027 \group_end:
5028 }
5029 }

this (of type structure):
5030 \cs_new_protected:Nn \__stex_expr_invoke_this:n {
5031 \peek_charcode_remove:NTF ! {
5032 \exp_args:Nne\use:nn{
5033 \group_end:\symuse{Metatheory?of~type}[invisible]{
5034 \tl_if_empty:nTF{##1}{\_stex_annotate_force_break:n{}}{##1}
5035 }
5036 }{
5037 {
5038 \tl_set:Nn \exp_not:N \l_stex_current_full_tl { \l_stex_current_full_tl}
5039 \__stex_expr_current_type:
5040 }
5041 }
5042 }{
5043 \__stex_expr_invoke_maybe_field:nn{##1}
5044 }
5045 }
5046

```

```

5047 \cs_new_protected:Nn \__stex_expr_invoke_maybe_field:nn {
5048   \__stex_expr_make_prop:
5049   \__stex_expr_set_this:n{#1}
5050   \tl_if_empty:nTF{#2}{
5051     \__stex_expr_make_prop_assign:
5052     \__stex_expr_present:
5053   }{
5054     \__stex_expr_invoke_field:n{#2}
5055   }
5056 }
5057
5058 \cs_new_protected:Nn \__stex_expr_set_this:n {
5059   \tl_if_empty:nTF{#1}{
5060     %\tl_put_right:Nn \l_stex_current_redo_tl {
5061       % \tl_clear:N \l_stex_struct_this_tl
5062     %}
5063   }{
5064     \tl_set:Ne \l_stex_struct_this_tl {{
5065       \bool_set_true:N \l_stex_allow_semantic_bool
5066       \bool_set_true:N \l_stex_in_this_bool
5067       \tl_set:Nn \exp_not:N \this {
5068         \exp_args:No \exp_not:n \this
5069       }
5070       \tl_set:Nn \exp_not:N \l_stex_current_full_tl { \l_stex_current_full_tl }
5071       \exp_not:n{#1}
5072     }}
5073     \tl_set_eq:NN \this \l_stex_struct_this_tl
5074     % \l_stex_return_notation_tl
5075   }
5076 }
5077
5078 \cs_new_protected:Nn \__stex_expr_get_field_name:n {
5079   \str_set:Ne \l_stex_expr_field_name_str {
5080     \exp_args:Nne \use:n {\exp_after:wN \use_i:nn \use:n}
5081     {\prop_item:Nn \l__stex_expr_prop {#1}}
5082   }
5083   \str_if_empty:NT \l_stex_expr_field_name_str {
5084     \str_set:Nn \l_stex_expr_field_name_str {#1}
5085   }
5086 }
5087
5088 \cs_new_protected:Nn \__stex_expr_invoke_field:n {
5089   \prop_if_in:NnTF \l__stex_expr_prop {#1}{
5090     \__stex_expr_get_field_name:n{#1}
5091     \tl_clear:N \l__stex_expr_more_nextsymbol_tl
5092     %\exp_args:Nne \seq_if_in:NnF \l__stex_expr_assigned_seq {\tl_to_str:n{#1}}{
5093       \tl_set:Ne \l__stex_expr_more_nextsymbol_tl {
5094         \tl_set:Nn \exp_not:N \l_stex_struct_this_tl {
5095           \exp_args:No \exp_not:n \l_stex_struct_this_tl
5096         }
5097         \exp_not:n {
5098           \tl_set_eq:NN \this \l_stex_struct_this_tl
5099         }
5100       \tl_set:Nn \exp_not:N \l_stex_return_notation_tl {

```

```

5101         \exp_args:No \exp_not:n \l_stex_return_notation_tl
5102     }
5103     \exp_args:No \exp_not:n \l__stex_expr_set_comp_tl
5104 }
5105 %}
5106 \exp_args:NNe \use:nn \group_end: {
5107     \_stex_next_symbol:n {
5108         \exp_args:No \exp_not:n \l__stex_expr_redo_tl
5109         \tl_set:Nn \exp_not:N \l_stex_current_term_tl {
5110             \symuse{Metatheory?record-field}{
5111                 \symuse{Metatheory?of~type}{
5112                     \exp_args:No\tl_if_empty:nTF \l_stex_struct_this_tl {\_stex_annotate_force_br
5113                 }{
5114                     \tl_set:Nn \exp_not:N \l_stex_current_full_tl { \l_stex_current_full_tl}
5115                     \__stex_expr_current_type:
5116                 }
5117             }{
5118                 \stex_annotate:nn{data-ftml-term=OML,data-ftml-head={\l__stex_expr_field_name_s
5119             }
5120         }
5121         \exp_args:No \exp_not:n \l__stex_expr_more_nextsymbol_tl
5122     }
5123     \exp_not:N \use_ii:nn
5124     \prop_item:Nn \l__stex_expr_prop {#1}
5125 }
5126 }{
5127     \msg_error:nnn{stex}{error/unknownfield}{#1}
5128 }
5129 }
5130
5131 \cs_new_protected:Nn \__stex_expr_make_prop: {
5132     \prop_clear:N \l__stex_expr_prop
5133     \seq_clear:N \l__stex_expr_seq
5134     \seq_clear:N \l__stex_expr_assigned_seq
5135     \tl_clear:N \l__stex_expr_redo_tl
5136     \__stex_expr_prop_do_decls:
5137 }
5138
5139 \cs_new_protected:Nn \__stex_expr_prop_do_decls: {
5140     \group_begin:
5141     \stex_debug:nn{expressions}{constructing-record}
5142     \seq_clear:N \l_stex_all_module_seq
5143     \exp_args:Ne \stex_activate_module:n {\clist_item:Nn \l_stex_current_type_tl 1}
5144     \exp_args:No \stex_iterate_symbols:nn \l_stex_current_type_tl {
5145         % uri, macro name, name, arity, args, type, def, return, macro
5146         \tl_if_empty:nTF{##2}{
5147             \exp_args:Nne\__stex_expr_do_decl_nomacro:nnnnnnnn{##3}
5148         }{
5149             \exp_args:Nne\__stex_expr_do_decl:nnnnnnnn{##2}
5150         }
5151         {\stex_new_symbol_uri:nn{##1}{##3}}{##3}{##4}{##5}{##6}{##7}{##8}##9
5152     }
5153     \exp_after:wN \tl_set:Nn \exp_after:wN \l__stex_expr_after_tl \exp_after:wN {
5154         \exp_after:wN \tl_set:Nn \exp_after:wN \l__stex_expr_prop \exp_after:wN { \l__stex_ex

```

```

5155     }
5156     \__stex_expr_prop_do_notations:
5157     \stex_debug:nn{expressions}{record:~\meaning\l__stex_expr_after_tl}
5158     \tl_put_right:Ne \l__stex_expr_after_tl {\tl_set:Nn \exp_not:N \l__stex_expr_redo_tl {\
5159     \exp_after:wN \group_end: \l__stex_expr_after_tl
5160 }
5161
5162 % id, uri, name, arity, args, type, def, return, macro
5163 \cs_new_protected:Nn \__stex_expr_do_decl_nomacro:nnnnnnnn {
5164   \prop_if_in:NnF \l__stex_expr_prop {#1} {
5165     \seq_put_left:Ne \l__stex_expr_seq {\tl_to_str:n{#2}}
5166     \prop_put:Nnn \l__stex_expr_prop {#1}{
5167       {}{
5168         \stex_invoke_symbol:nnnnnnN
5169         {#2}
5170         {#4}
5171         {#5}{#6}{#7}{#8}#9
5172       }
5173     }
5174   }
5175 }
5176
5177 \cs_new_protected:Nn \__stex_expr_do_decl:nnnnnnnn {
5178   \prop_if_in:NnF \l__stex_expr_prop {#1} {
5179     \seq_put_left:Ne \l__stex_expr_seq {\tl_to_str:n{#2}}
5180     \prop_put:Nnn \l__stex_expr_prop {#1}{
5181       {#3}{
5182         \stex_invoke_symbol:nnnnnnN
5183         {#2}
5184         {#4}
5185         {#5}{#6}{#7}{#8}#9
5186       }
5187     }
5188   }
5189 }
5190
5191 \cs_new_protected:Nn \__stex_expr_make_prop_assign: {
5192   \clist_if_empty:NF \l__stex_expr_fields_clist {
5193     \clist_map_inline:Nn \l__stex_expr_fields_clist {
5194       \__stex_expr_make_prop_assign:nn ##1
5195     }
5196   }
5197 }
5198
5199 \cs_new_protected:Nn \__stex_expr_make_prop_assign:nn {
5200   \prop_if_in:NnTF \l__stex_expr_prop {#1}{
5201     \exp_args:NNe \seq_put_right:Nn \l__stex_expr_assigned_seq {\tl_to_str:n{#1}}
5202     \exp_args:Nne \use:nn {\__stex_expr_make_prop_assign_replace:nnnn {#1}{#2}}
5203     {\prop_item:Nn \l__stex_expr_prop {#1}}
5204   }{
5205     \msg_error:nnn{stex}{error/unknownfieldass}{#1}
5206   }
5207 }
5208 \cs_new_protected:Nn \__stex_expr_make_prop_assign_replace:nnnn {

```

```

5209 \prop_put:Nnn \l__stex_expr_prop {#1}{#{3}{#2}}
5210 \tl_if_empty:nF{#3}{
5211   \tl_set:cn{#1}{ #2 }
5212   \tl_put_right:Nn \l__stex_expr_redo_tl {
5213     \tl_set:cn{#1}{ #2 }
5214   }
5215 }
5216 }
5217
5218 \cs_new_protected:Nn \__stex_expr_prop_do_notations: {
5219   \exp_args:No \stex_iterate_notations:nn \l_stex_current_type_tl {
5220     \exp_args:NNe \seq_if_in:NnT \l__stex_expr_seq {\tl_to_str:n{##1}}{
5221       \tl_set:Nn \l_tmpa_tl {
5222         \cs_if_exist:cF{l_stex_notation_\stex_use_symbol_uri:n{##1}?##2_cs}{
5223           \tl_set:cn{l_stex_notation_\stex_use_symbol_uri:n{##1}?##2_cs}{##4}
5224         }
5225         \cs_if_exist:cF{l_stex_notation_\stex_use_symbol_uri:n{##1}?_cs}{
5226           \tl_set:cn{l_stex_notation_\stex_use_symbol_uri:n{##1}?_cs}{##4}
5227         }
5228       }
5229       \exp_args:NNo \tl_put_right:Nn \l__stex_expr_redo_tl \l_tmpa_tl
5230       \exp_args:NNo \tl_put_right:Nn \l__stex_expr_after_tl \l_tmpa_tl
5231       \tl_if_empty:nF{##5}{
5232         \tl_set:Nn \l_tmpa_tl {
5233           \cs_if_exist:cF{l_stex_notation_\stex_use_symbol_uri:n{##1}?##2_op_cs}{
5234             \tl_set:cn{l_stex_notation_\stex_use_symbol_uri:n{##1}?##2_op_cs}{##5}
5235           }
5236           \cs_if_exist:cF{l_stex_notation_\stex_use_symbol_uri:n{##1}?_op_cs}{
5237             \tl_set:cn{l_stex_notation_\stex_use_symbol_uri:n{##1}?_op_cs}{##5}
5238           }
5239         }
5240         \exp_args:NNo \tl_put_right:Nn \l__stex_expr_redo_tl \l_tmpa_tl
5241         \exp_args:NNo \tl_put_right:Nn \l__stex_expr_after_tl \l_tmpa_tl
5242       }
5243     }
5244   }
5245 }

```

(End of definition for `\stex_invoke_structure:`. This function is documented on page 150.)

sfunction

`\_stex_invoke_variable:nnnnnn`

```

5246 \cs_new_protected:Nn \_stex_invoke_variable:nnnnnnN {
5247   \bool_if:NTF \l_stex_allow_semantic_bool{
5248     \group_begin:
5249     \tl_set:Nn \l_stex_current_full_tl { #1 }
5250     \tl_set:Nn \l_stex_current_display_tl { #1 }
5251     \stex_if_html_backend:T{\relax\ifvmode\indent\fi}
5252     \__stex_expr_setup:nnnnn{\_varcomp}{#2}{#3}{#6}{#5}
5253     \cs_set_eq:NN \_stex_term_oms_or_omv:nn \_stex_term_omv:nn
5254     \tl_put_right:Nn \l_stex_current_redo_tl {
5255       \cs_set_eq:NN \_stex_term_oms_or_omv:nn \_stex_term_omv:nn
5256     }
5257     #7

```

```

5258   }{
5259     \msg_error:nnxx{stex}{error/notallowed}{#1}{\l_stex_current_full_tl}
5260   }
5261 }

```

(End of definition for `\_stex_invoke_variable:nnnnnn`. This function is documented on page 150.)

sfunction

`\svar`

```

5262 \NewDocumentCommand \svar {0{ } m}{
5263   \group_begin:
5264     \tl_if_empty:nTF{#1}{
5265       \tl_set:Nn \l_stex_current_full_tl { #2 }
5266       \tl_set:Nn \l_stex_current_display_tl { #2 }
5267     }{
5268       \tl_set:Nn \l_stex_current_full_tl { #1 }
5269       \tl_set:Nn \l_stex_current_display_tl { #1 }
5270     }
5271     \bool_if:NTF \l_stex_allow_semantic_bool{
5272       \tl_clear:N \l_stex_current_term_tl
5273       \stex_term_omv:nn{}{\_varcomp{#2}}
5274     }{
5275       \msg_error:nnxx{stex}{error/notallowed}{Variable}{\l_stex_current_full_tl}
5276     }
5277   \group_end:
5278 }

```

(End of definition for `\svar`. This function is documented on page 150.)

sfunction

Now for the brunt of the work:

Math

Modifiers (! or *)?

```

5279 \cs_new_protected:Nn \_stex_expr_math: {
5280   \stex_debug:nn{expressions}{math-mode}
5281   \peek_charcode_remove:NTF ! {
5282     % operator
5283     \peek_charcode_remove:NTF * \_stex_expr_op_custom:n {
5284       % op notation
5285       \peek_charcode:NTF [ \_stex_expr_op_notation:w {
5286         \_stex_expr_op_notation:w []
5287       }
5288     }
5289   }{
5290     \peek_charcode_remove:NTF * \_stex_expr_custom:n {
5291       % normal
5292       \peek_charcode:NTF [ \_stex_expr_notation:w {
5293         \_stex_expr_notation:w []
5294       }
5295     }
5296   }
5297 }

```

Text

Operator?

```
5298 \cs_new_protected:Nn \__stex_expr_text: {  
5299   \stex_debug:nn{expressions}{text~mode}  
5300   \peek_charcode_remove:NTF ! \__stex_expr_op_custom:n \__stex_expr_custom:n  
5301 }
```

Notation

Operator?

```
5302 \cs_new_protected:Npn \__stex_expr_notation:w [ #1 ] {  
5303   \peek_charcode_remove:NTF ! {  
5304     \__stex_expr_op_notation:w [ #1]  
5305   }{  
5306     \__stex_expr_full_notation:n { #1}  
5307   }  
5308 }
```

(Possibly) complex notation taking arguments with optional particular identifier:

```
5309 \cs_new_protected:Nn \__stex_expr_full_notation:n {  
5310   \stex_use_notation:nnTF \l_stex_current_full_tl { #1 } {  
5311     \stex_debug:nn{expressions}{using~notation~" #1 " ~for~\l_stex_current_full_tl}  
5312     \tl_if_empty:NTF \l_stex_current_return_tl {  
5313       \stex_debug:nn{expressions}{return~empty}  
5314       \l_stex_notation_cs{\group_end:\stex_eat_exclamation_point:}  
5315     }{  
5316       \stex_debug:nn{expressions}{return?}  
5317       \exp_after:wN \__stex_expr_maybe_return:n \exp_after:wN {  
5318         \l_stex_notation_cs{  
5319       }  
5320     }  
5321   }{  
5322     \stex_debug:nn{expressions}{notation~" #1 " ~for~\l_stex_current_full_tl{ }~does~not~exist;  
5323     \stex_do_default_notation:  
5324     \tl_if_empty:NTF \l_stex_current_return_tl {  
5325       \l_stex_default_notation{\group_end:\stex_eat_exclamation_point:}  
5326     }{  
5327       \exp_after:wN  
5328       \__stex_expr_maybe_return:n  
5329       \exp_after:wN  
5330       {\l_stex_default_notation { } }  
5331     }  
5332   }  
5333 }
```

Operator notation:

```
5334 \cs_new_protected:Npn \__stex_expr_op_notation:w [ #1 ] {  
5335   \stex_debug:nn{expressions}{op~notation~for~\l_stex_current_full_tl}  
5336   \stex_use_op_notation:nnTF \l_stex_current_full_tl { #1 } {  
5337     \stex_maybe_brackets:nn{\neginfprec}{  
5338       \stex_term_oms_or_omv:nn{ #1 }  
5339       {\l_stex_notation_cs}  
5340     }  
5341     \group_end:  
5342   }{
```

```

5343 \int_compare:nNnTF \l_stex_current_arity_str = 0 {
5344   \tl_clear:N \l_stex_current_return_tl
5345   \__stex_expr_notation:w [#1]
5346 }{
5347   \stex_do_default_notation_op:
5348   \_stex_maybe_brackets:nn{\neginfprec}{
5349     \_stex_term_oms_or_omv:nn{#1}
5350     {\l_stex_default_notation}
5351   }
5352   \group_end:
5353 }
5354 }
5355 }

```

Custom Notations

Operator:

```

5356 \cs_new_protected:Nn \__stex_expr_op_custom:n {
5357   \stex_debug:nn{expressions}{custom-op}
5358   \bool_set_true:N \l_stex_allow_semantic_bool
5359   \_stex_term_oms_or_omv:nn{}{\maincomp{#1}}
5360   \group_end:
5361 }

```

Complex:

```

5362 \int_new:N \l__stex_expr_arg_counter_int
5363
5364 \cs_new_protected:Nn \__stex_expr_custom:n {
5365   \stex_debug:nn{custom}{custom-notation-for-\l_stex_current_full_tl}
5366   \stex_pseudogroup:nn{
5367     \bool_set_true:N \l_stex_allow_semantic_bool
5368     \prop_gclear:N \l__stex_expr_customs_prop
5369     \seq_gclear:N \l__stex_expr_customs_seq
5370     \int_gzero:N \l__stex_expr_arg_counter_int
5371     \tl_if_empty:NF \l_stex_current_args_tl {
5372       \exp_after:wN \__stex_expr_add_prop_arg:nnw \l_stex_current_args_tl \_stex_args_end:
5373       \cs_set_eq:NN \arg \__stex_expr_arg:n
5374     }
5375     \tl_set_eq:NN \l_stex_get_symbol_args_tl \l_stex_current_args_tl
5376     \cs_set_eq:NN \__stex_expr_do_ab_next:nn \_stex_term_oma:nn
5377     \_stex_map_args:N \__stex_expr_check_b:nn
5378     \__stex_expr_do_ab_next:nn{}{#1}
5379   }{
5380     \prop_if_exist:NT \l__stex_expr_customs_prop {
5381       \prop_gset_from_keyval:Nn \exp_not:N \l__stex_expr_customs_prop {
5382         \prop_to_keyval:N \l__stex_expr_customs_prop
5383       }
5384     }
5385     \int_gset:Nn \l__stex_expr_arg_counter_int { \int_use:N \l__stex_expr_arg_counter_int}
5386     \seq_if_exist:NT \l__stex_expr_customs_seq {
5387       \seq_gset_split:Nnn \exp_not:N \l__stex_expr_customs_seq , {
5388         \seq_use:Nn \l__stex_expr_customs_seq ,
5389       }
5390     }
5391   }

```



```

5392 % TODO check that all arguments are present
5393 \group_end:
5394 }
5395
5396 \cs_new_protected:Npn \__stex_expr_add_prop_arg:nnw #1 #2 #3\__stex_args_end: {
5397   \prop_gput:Nnn \l__stex_expr_customs_prop {#1} {}
5398   \seq_gput_right:Nn \l__stex_expr_customs_seq {#2}
5399   \tl_if_empty:nF{#3}{\__stex_expr_add_prop_arg:nnw #3 \__stex_args_end:}
5400 }
5401
5402 \cs_new:Nn \__stex_expr_check_b:nn {
5403   \str_case:nn #2 {
5404     b {\cs_set_eq:NN \__stex_expr_do_ab_next:nn \stex_term_omb:nn}
5405     B {\cs_set_eq:NN \__stex_expr_do_ab_next:nn \stex_term_omb:nn}
5406   }
5407 }
5408
5409 Arguments:
5410 \NewDocumentCommand \__stex_expr_arg:n {s O{} m} {
5411   \IfBooleanTF #1 {
5412     \stex_annotate_invisible:n{
5413       \__stex_expr_arg_inner:nn{#2}{#3}
5414     }
5415   }{
5416     \__stex_expr_arg_inner:nn{#2}{#3}
5417   }
5418 }
5419
5420 \cs_new_protected:Nn \__stex_expr_arg_inner:nn {
5421   \tl_if_empty:nTF{#1}{
5422     \int_gincr:N \l__stex_expr_arg_counter_int
5423     \exp_args:Ne \__stex_expr_check:nTF{ \int_use:N \l__stex_expr_arg_counter_int }{
5424       \__stex_expr_arg_do:oon \l_tmpa_tl \l_tmpb_tl
5425     }{
5426       \__stex_expr_arg_inner:nn{}
5427     }{ #2 }
5428   }{
5429     \__stex_expr_check:nTF {#1}{
5430       \__stex_expr_arg_do:oon \l_tmpa_tl \l_tmpb_tl { #2 }
5431     }{
5432       \exp_args:No \str_case:nnTF \l_tmpb_tl {
5433         {a}{
5434           \exp_args:NNne \prop_gput:Nnn \l__stex_expr_customs_prop {#1}{
5435             \l_tmpa_tl X
5436           }
5437           \tl_set:Nx \l_tmpa_tl { #1 \int_eval:n {\tl_count:N \l_tmpa_tl + 1} }
5438         }{
5439           \exp_args:NNne \prop_gput:Nnn \l__stex_expr_customs_prop {#1}{
5440             \l_tmpa_tl X
5441           }
5442           \tl_set:Ne \l_tmpa_tl { #1 \int_eval:n {\tl_count:N \l_tmpa_tl + 1} }
5443         }
5444       }{
5445         \__stex_expr_arg_do:oon \l_tmpa_tl \l_tmpb_tl { #2 }

```

```

5445     }{
5446     \msg_error:nnxx{stex}{error/invalidarg}{#1}{\l_stex_current_full_tl}
5447     }
5448   }
5449 }
5450 }
5451
5452 \prg_new_conditional:Nnn \__stex_expr_check:n {TF} {
5453   \exp_args:NNe \prop_get:NnNTF \l__stex_expr_customs_prop {#1} \l_tmpa_tl {
5454     \tl_set:Ne \l_tmpb_tl {\seq_item:Nn \l__stex_expr_customs_seq {#1} }
5455     \tl_if_empty:NTF \l_tmpa_tl {
5456       \exp_args:NNe \prop_gput:Nnn \l__stex_expr_customs_prop
5457       { #1 }{X}
5458       \exp_args:No \str_case:nnF \l_tmpb_tl {
5459         {a}{
5460           \tl_set:Ne \l_tmpa_tl{ #1 1 }
5461         }
5462         {B}{
5463           \tl_set:Ne \l_tmpa_tl{ #1 1 }
5464         }
5465       }{
5466         \tl_set:Ne \l_tmpa_tl{ #1 }
5467       }
5468       \prg_return_true:
5469     }{
5470       \prg_return_false:
5471     }
5472   }{
5473     \msg_error:nnxx{stex}{error/invalidarg}{#1}{\l_stex_current_full_tl}
5474     \prg_return_false:
5475   }
5476 }
5477
5478 % #1 argnum #2 argmode #3 code
5479 \cs_new_protected:Nn \__stex_expr_arg_do:nnn {
5480   \stex_debug:nn{custom}{Doing~argument~#1~of~mode~#2:~\tl_to_str:n{#3}}
5481   \group_begin:
5482     \bool_set_true:N \l_stex_allow_semantic_bool
5483     \stex_term_arg:nnn {#2}{#1}{#3}
5484   \group_end:
5485 }
5486 \cs_generate_variant:Nn \__stex_expr_arg_do:nnn {oon}

```

Return code

```

5487 \cs_new_protected:Nn \__stex_expr_maybe_return:n {
5488   \tl_set:Nn \l__stex_expr_return_this_tl {#1}
5489   \tl_clear:N \l__stex_expr_return_args_tl
5490   \exp_args:Nne \use:n {
5491     \cs_generate_from_arg_count:NNnn \__stex_expr_ret_cs:
5492     \cs_set:Npn \l_stex_current_arity_str } {
5493       \int_step_function:nN \l_stex_current_arity_str \__stex_expr_return_arg:n
5494       \__stex_expr_return_next:
5495     }

```

```

5496 \__stex_expr_ret_cs:
5497 }
5498
5499 \cs_new:Nn \__stex_expr_return_arg:n {
5500 \tl_put_right:Nn \exp_not:N \l__stex_expr_return_args_tl {{## #1}}
5501 }
5502
5503 \cs_new_protected:Nn \__stex_expr_return_next: {
5504 \peek_charcode_remove:NTF ! {
5505 \exp_after:wN \l__stex_expr_return_this_tl \l__stex_expr_return_args_tl \group_end:
5506 }\__stex_expr_return:
5507 }
5508
5509 \cs_new_protected:Nn \__stex_expr_return: {
5510 \tl_set:Ne \l__stex_expr_return_this_tl {
5511 \tl_if_empty:NTF \l_stex_return_notation_tl {
5512 \exp_after:wN \exp_after:wN \exp_after:wN
5513 \exp_not:n \exp_after:wN \exp_after:wN \exp_after:wN {
5514 \exp_after:wN \l__stex_expr_return_this_tl \l__stex_expr_return_args_tl
5515 }
5516 }{
5517 \l_stex_return_notation_tl
5518 }
5519 }
5520 \stex_debug:nn{return}{Notation:~\meaning\l__stex_expr_return_this_tl}
5521
5522 \exp_after:wN
5523 \tl_put_left:Nn \exp_after:wN \l__stex_expr_return_this_tl \exp_after:wN {
5524 \exp_after:wN \group_begin: \l_stex_current_redo_tl
5525 }
5526 \exp_args:Nne \use:n {
5527 \cs_generate_from_arg_count:NNnn \__stex_expr_ret_cs:
5528 \cs_set:Npn \l_stex_current_arity_str } {
5529 \exp_args:No \exp_not:n \l_stex_current_return_tl
5530 }
5531 \stex_debug:nn{return}{
5532 \meaning\__stex_expr_ret_cs:^^J
5533 \meaning\l__stex_expr_return_this_tl^^J
5534 \exp_args:No \exp_not:n \l__stex_expr_return_args_tl^^J
5535 }
5536 \exp_args:Nne \use:nn {
5537 \exp_after:wN \group_end: \__stex_expr_ret_cs:
5538 }{
5539 \exp_args:No \exp_not:n \l__stex_expr_return_args_tl
5540 {
5541 \exp_args:No \exp_not:n \l__stex_expr_return_this_tl
5542 \group_end:
5543 }
5544 }
5545 }

```

32.4.1 Term Annotations

`\stex_eat_exclamation_point:`

```
5546 \cs_new_protected:Nn \stex_eat_exclamation_point: {
5547   \peek_charcode_remove:NT ! \stex_eat_exclamation_point:
5548 }
```

(End of definition for \stex_eat_exclamation_point:. This function is documented on page 150.)

sfunction

We occasionally allow for semantic macros where they otherwise shouldn't be, if we are in “invisible” parts:

```
5549 \bool_new:N \stex_in_invisible_html_bool
```

`\stex_term_oms:nn`

```
5550 \stex_if_html_backend:TF {
5551   \cs_new_protected:Nn \stex_term_oms:nn {
5552     \bool_if:NTF\l_stex_in_this_bool{#2}{
5553       \__stex_expr_check_comp:
5554       \tl_if_empty:NTF \l_stex_current_term_tl {
5555         \__stex_expr_annotate:nnn{OMID}{#1}{#2}
5556       }{
5557         \__stex_expr_do_headterm:nn{#1}{#2}
5558       }
5559     }
5560   }
5561 }{
5562   \cs_new_protected:Nn \stex_term_oms:nn {\bool_if:NF\l_stex_in_this_bool\__stex_expr_check_comp:}
5563 }
5564
5565 \cs_set_eq:NN \stex_term_oms_or_omv:nn \stex_term_oms:nn
```

(End of definition for \stex_term_oms:nn. This function is documented on page 150.)

sfunction

`\stex_term_omv:nn`

```
5566 \stex_if_html_backend:TF {
5567   \cs_new_protected:Nn \stex_term_omv:nn {
5568     \bool_if:NTF\l_stex_in_this_bool{#2}{
5569       \__stex_expr_check_comp:
5570       \tl_if_empty:NTF \l_stex_current_term_tl {
5571         \__stex_expr_annotate:nnn{OMV}{#1}{#2}
5572       }{
5573         \__stex_expr_do_headterm:nn{#1}{#2}
5574       }
5575     }
5576   }
5577 }{
5578   \cs_new_protected:Nn \stex_term_omv:nn {\bool_if:NF\l_stex_in_this_bool\__stex_expr_check_comp:}
5579 }
```

(End of definition for \stex_term_omv:nn. This function is documented on page 150.)

sfunction

In the case where the “symbol” (OMS or OMV) is the field of some structure or otherwise a complex expression, we have to insert the full term representing the operator:

```

5580 \stex_if_html_backend:TF {
5581   \cs_new_protected:Nn \__stex_expr_do_headterm:nn {
5582     \bool_if:NTF \stex_in_invisible_html_bool {
5583       {\bool_set_true:N \l_stex_allow_semantic_bool
5584        \ensuremath{\l_stex_current_term_tl}
5585       }
5586     }{
5587       \__stex_expr_annotate:nnn{complex}{#1}{
5588         \stex_annotate_invisible:nn{data-ftml-headterm={}}{
5589           {\bool_set_true:N \l_stex_allow_semantic_bool
5590            \ensuremath{\l_stex_current_term_tl}
5591           }
5592         }
5593       }#2
5594     }
5595   }
5596 }
5597 }{
5598   \cs_new_protected:Nn \__stex_expr_do_headterm:nn { #2 }
5599 }

```

`\_stex_term_oma:nn`

```

5600 \stex_if_html_backend:TF {
5601   \cs_new_protected:Nn \_stex_term_oma:nn {
5602     \bool_if:NTF \l_stex_in_this_bool{#2}{
5603       \__stex_expr_check_comp:
5604       \__stex_expr_annotate:nnn{OMA}{#1}{
5605         \tl_if_empty:NF \l_stex_current_term_tl {
5606           \stex_annotate_invisible:nn{data-ftml-headterm={}}{
5607             \bool_set_true:N \l_stex_allow_semantic_bool
5608             \l_stex_current_term_tl
5609           }}
5610       }
5611     }#2
5612   }
5613 }
5614 }
5615 }{
5616   \cs_new_protected:Nn \_stex_term_oma:nn {\bool_if:NF \l_stex_in_this_bool \__stex_expr_check_comp:}
5617 }

```

(End of definition for `\_stex_term_oma:nn`. This function is documented on page 150.)

sfunction

`\_stex_term_omb:nn`

```

5618 \stex_if_html_backend:TF {
5619   \cs_new_protected:Nn \_stex_term_omb:nn {
5620     \bool_if:NTF \l_stex_in_this_bool{#2}{
5621       \__stex_expr_check_comp:
5622       \__stex_expr_annotate:nnn{OMBIND}{#1}{
5623         \tl_if_empty:NF \l_stex_current_term_tl {
5624           \stex_annotate_invisible:nn{data-ftml-headterm={}}{
5625             \bool_set_true:N \l_stex_allow_semantic_bool
5626             \l_stex_current_term_tl

```

```

5627         }}
5628     }
5629     #2
5630 }
5631 }
5632 }
5633 }{
5634 \cs_new_protected:Nn \_stex_term_omb:nn {\bool_if:NF\l_stex_in_this_bool\__stex_expr_check:
5635 }

```

(End of definition for `\_stex_term_omb:nn`. This function is documented on page 150.)

sfunction

Does the actual annotating: **TODO: This shouldn't use `\l_stex_current_symbol_uri`!**

```

5636 % kind, notation id, body
5637 \stex_if_html_backend:TF {
5638   \cs_new_protected:Nn \__stex_expr_annotate:nnn {
5639     \exp_args:Ne \stex_annotate:nn{
5640       data-ftml-term=#1,
5641       data-ftml-head={\l_stex_current_full_tl},
5642       data-ftml-notationid={#2},
5643     }{
5644       \_stex_annotate_force_break:n{#3}
5645     }
5646   }
5647 }{
5648   \cs_new_protected:Nn \__stex_expr_annotate:nnn { #3 }
5649 }
5650

```

32.4.2 Symbol Arguments

`\_stex_term_arg:nnnnn` #1 : argument number
 #2 : argument mode
 #3 : precedence
 #4 : argument name (WiP)
 #5 : code / body

```

5651 \cs_new_protected:Nn \_stex_term_arg:nnnnn {
5652   \group_begin:
5653     \tl_clear:N \l_stex_current_symbol_uri
5654     \tl_clear:N \l_stex_current_full_tl
5655     \tl_clear:N \l_stex_current_display_tl
5656     \tl_clear:N \l_stex_current_term_tl
5657     \int_set:Nn \l_stex_notation_downprec { #3 }
5658     \bool_set_true:N \l_stex_allow_semantic_bool
5659     \_stex_term_arg:nnn {#2}{#1}{#5}
5660   \group_end:
5661 }

```

(End of definition for `\_stex_term_arg:nnnnn`. This function is documented on page 151.)

sfunction

`\_stex_term_arg:nnn`

```

5662 \cs_new_protected:Nn \__stex_expr_check_comp: {
5663   \bool_if:NT \g_stex_in_comp_bool {
5664     \msg_error:nn{stex}{error/argincomp}
5665   }
5666 }
5667 \stex_if_html_backend:TF {
5668   \cs_new_protected:Nn \_stex_term_arg:nnn {
5669     \stex_if_do_html:TF{
5670       \bool_if:NTF\l_stex_in_this_bool{#3}{
5671         \_stex_annotate_force_break:n{
5672           \stex_annotate:nn{
5673             data-ftml-arg={#2}, data-ftml-argmode={#1}
5674           }{
5675             \_stex_annotate_force_break:n{
5676               \__stex_expr_check_comp:
5677               #3
5678             }
5679           }
5680         }
5681       }{ #3 }
5682     }
5683   }{
5684     \cs_new_protected:Nn \_stex_term_arg:nnn {\bool_if:NF\l_stex_in_this_bool\__stex_expr_che
5685   }

```

(End of definition for `\_stex_term_arg:nnn`. This function is documented on page 151.)

sfunction

`\_stex_term_arg_aB:nnnnn` The following macros fill up `\l_stex_aB_args_seq` and ultimately call `\_stex_term_do_aB_clist:`. By default, this does:

```

5686 \cs_new:Nn \__stex_expr_do_aB_clist: {
5687   \_stex_annotate_force_break:n{
5688     \seq_use:Nn \l_stex_aB_args_seq {
5689       \mathpunct{\comp{,}}
5690     }
5691   }
5692 }
5693
5694 \cs_set_eq:NN \_stex_term_do_aB_clist: \__stex_expr_do_aB_clist:
5695
5696 \cs_new_protected:Nn \_stex_is_sequentialized:n {
5697   \group_begin: #1 \group_end:
5698 }

```

Setup:

```

5699 \int_new:N \l__stex_expr_count_int
5700 \cs_new_protected:Nn \_stex_term_arg_aB:nnnnn {
5701   \stex_debug:nn{sequences}{Processing~argument:~ \tl_to_str:n{#5}}
5702   \tl_if_empty:NTF{#5}{
5703     \_stex_term_arg:nnnnn{#1}{#2}{#3}{#4}{#5}
5704   }{
5705     \seq_clear:N \l_stex_aB_args_seq
5706     \int_zero:N \l__stex_expr_count_int

```

```

5707 \clist_map_inline:nn{#5}{
5708   \__stex_expr_aB_arg:nnnnn{##1}{#1}{#2}{#3}{#4}
5709 }
5710 \_stex_term_do_aB_clist:
5711 }
5712 }

```

We “pattern match” the sequence argument - is it a comma-separated list? A sequence variable macro? `\argmap?` `\seqmap?`

```

5713 % 1: code 2: argnum 3: argmode 4: precedence 5: argname
5714 \cs_new_protected:Npn \__stex_expr_aB_arg:nnnnn #1 #2 #3 {
5715   \int_incr:N \l__stex_expr_count_int
5716   \stex_debug:nn{expressions}{matching~argument~ #2 #3:~ \tl_to_str:n{#1}}
5717   \__stex_expr_is_varseq:nTF{#1}{
5718     \stex_debug:nn{expressions}{=sequence~variable}
5719     \exp_after:wN \exp_after:wN \exp_after:wN
5720     \__stex_expr_assoc_seq:nnnnnnn
5721     \exp_after:wN
5722     \__stex_expr_gobble:nnnnnnnn #1 \__stex_expr_end:
5723   }{
5724     \__stex_expr_is_seqmap:nTF{#1}{
5725       \stex_debug:nn{expressions}{=seqmap}
5726       \exp_args:NNe \use:nn \__stex_expr_do_seqmap:nnnnnn { \tl_tail:n{#1}}
5727     }{
5728       \stex_debug:nn{expressions}{=simple}
5729       \__stex_expr_aB_simple_arg:nnnnn{#1}
5730     }
5731   }
5732   {#2}{#3}
5733 }
5734
5735 \cs_new:Npn \__stex_expr_gobble:nnnnnnnn #1 #2 #3 #4 #5 #6 #7 #8 #9 \__stex_expr_end: {
5736   {#2} #3 {#6}
5737 }

```

Is it a varseq?

```

5738 \prg_new_conditional:Nnn \__stex_expr_is_varseq:n {TF} {
5739   \int_compare:nNnTF {\tl_count:n{#1}} = 1 {
5740     \exp_args:Ne \cs_if_eq:NNTF {\exp_args:No \tl_head:n{#1}}
5741     \_stex_invoke_variable:nnnnnnN {
5742       \exp_args:Ne \cs_if_eq:NNTF {\exp_args:No \tl_item:nn{#1}{8}}
5743       \stex_invoke_sequence:
5744       \prg_return_true:\prg_return_false:
5745     }\prg_return_false:
5746   }\prg_return_false:
5747 }

```

Is it a seqmap?

```

5748 \prg_new_conditional:Nnn \__stex_expr_is_seqmap:n {TF} {
5749   \int_compare:nNnTF {\tl_count:n{#1}} = 3 {
5750     \exp_args:Ne \tl_if_eq:nnTF {\tl_head:n{#1}} {\seqmap}
5751     \prg_return_true:\prg_return_false:
5752   }\prg_return_false:
5753 }

```


Is it an argsep?

```

5754 \prg_new_conditional:Nnn \__stex_expr_is_argsep:n {TF} {
5755   \int_compare:nNnTF {\tl_count:n{#1}} = 3 {
5756     \exp_args:Ne \tl_if_eq:nnTF {\tl_head:n{#1}} {\argsep}
5757     \prg_return_true:\prg_return_false:
5758   }\prg_return_false:
5759 }

```

Simple argument:

```

5760 \cs_new_protected:Nn \__stex_expr_aB_simple_arg:nnnnn{
5761   \seq_put_right:Ne \l_stex_aB_args_seq {
5762     \stex_term_arg:nnnnn{#2\int_use:N\l__stex_expr_count_int}{#3}{#4}{#5}{
5763       \exp_not:n{
5764         \tl_set_eq:NN \stex_term_do_aB_clist: \__stex_expr_do_aB_clist:
5765         #1
5766       }
5767     }
5768   }
5769 }

```

Sequence variable:

```

5770 % 1: name 2: arity 3: clist 4: argnum 5: argmode 6: precedence 7: argname
5771 \cs_new_protected:Nn \__stex_expr_assoc_seq:nnnnnnn {
5772   \group_begin:
5773   \seq_clear:N \l_stex_aB_args_seq
5774   \tl_set:Nn \l_stex_current_full_tl{#1}
5775   \tl_set:Nn \l_stex_current_display_tl{#1}
5776   \__stex_expr_assoc_make_seq:nnn{#1}{#3}{#2}
5777   \exp_args:NNe \use:nn \group_end: {
5778     \seq_put_right:Nn \exp_not:N \l_stex_aB_args_seq {
5779       \stex_is_sequentialized:n{
5780         \stex_term_arg:nnnnn{#4\int_use:N\l__stex_expr_count_int}{#5}{#6}{#7}{
5781           \bool_set_true:N \l_stex_allow_semantic_bool
5782           \tl_if_empty:NF \l_stex_current_term_tl {
5783             \tl_set:Nn \exp_not:N \l_stex_current_term_tl {
5784               \exp_args:No \exp_not:n \l_stex_current_term_tl
5785             }
5786           }
5787           \stex_annotate:nn{
5788             data-ftml-term=OMV,
5789             data-ftml-head={#1},
5790             data-ftml-notationid={}
5791           }{
5792             \stex_annotate_force_break:n{
5793               \stex_term_do_aB_clist:
5794             }
5795           }
5796         }
5797       }
5798     }
5799   }
5800 }
5801
5802 % #1: name, #2: clist, #3:arity
5803 \cs_new_protected:Nn \__stex_expr_assoc_make_seq:nnn {

```

```

5804 \stex_use_notation:nnTF {#1}{}{
5805   \cs_set_eq:NN \l__stex_expr_cs \l_stex_notation_cs
5806 }{
5807   \stex_do_default_notation:
5808   \cs_set_eq:NN \l__stex_expr_cs \l_stex_default_notation
5809 }
5810 \clist_map_inline:nn{#2}{
5811   \tl_if_eq:nnTF{##1}{\ellipses}{
5812     \seq_put_right:Nn \l_stex_aB_args_seq { ##1 }
5813   }{
5814     \int_compare:nNnTF {#3} = 1 {
5815       \tl_set:Nn \l__stex_expr_iarg_tl { {##1} }
5816     }{
5817       \tl_set:Nn \l__stex_expr_iarg_tl { ##1 }
5818     }
5819     \seq_put_right:Ne \l_stex_aB_args_seq {
5820       \group_begin:
5821       \exp_not:n {
5822         \tl_set_eq:NN \_stex_term_do_aB_clist: \_stex_expr_do_aB_clist:
5823         \def\comp{\_varcomp}
5824         \tl_set:Nn \l_stex_current_full_tl{#1}
5825       }
5826       \exp_after:wN \exp_after:wN \exp_after:wN \exp_not:n
5827       \exp_after:wN \exp_after:wN \exp_after:wN {
5828         \exp_after:wN \l__stex_expr_cs \exp_after:wN \group_end: \l__stex_expr_iarg_tl
5829       }
5830     }
5831   }
5832 }
5833 }

seqmap:
5834 % 1: fun 2: clist 3: argnum 4: argmode 5: precedence 6: argname
5835 \cs_new_protected:Nn \_stex_expr_do_seqmap:nnnnnn {
5836   \group_begin:
5837   \cs_set:Npn \l_tmpa_cs ##1 {#1}
5838   \seq_clear:N \l_stex_aB_args_seq
5839   \clist_map_inline:nn{#2}{
5840     \_stex_expr_is_varseq:nTF{##1}{
5841       \exp_after:wN
5842       \_stex_expr_varseq_in_map:nnnnnnnn ##1
5843     }{
5844       \seq_put_right:Nn \l_stex_aB_args_seq {##1}
5845     }
5846   }
5847   \seq_clear:N \l__stex_expr_old_seq
5848   \seq_map_inline:Nn \l_stex_aB_args_seq {
5849     \tl_if_eq:nnTF{##1}{\ellipses}{
5850       \seq_put_right:Nn \l__stex_expr_old_seq {##1}
5851     }
5852     % TODO \_stex_is_sequentialized:n
5853     {
5854       \exp_args:NNo \seq_put_right:Nn \l__stex_expr_old_seq {
5855         \l_tmpa_cs {##1}
5856       }

```

```

5857     }
5858   }
5859   \seq_set_eq:NN \l_stex_aB_args_seq \l__stex_expr_old_seq
5860   \tl_set:Ne \l_tmpa_tl {
5861     \symuse{Metatheory?bind}{
5862       \svar[MAP_DUMMY]{\exp_not:N\mathcal{m}}
5863     }{
5864       \exp_args:No\exp_not:n{\l_tmpa_cs{\svar[MAP_DUMMY]{\mathcal{m}}}}
5865     }
5866   }
5867   \exp_args:NNe \use:nn \group_end: {
5868     \seq_put_right:Nn \exp_not:N \l_stex_aB_args_seq {
5869       \_stex_is_sequentialized:n{
5870         \_stex_term_arg:nnnnn{#3\int_use:N\l__stex_expr_count_int}{#4}{#5}{#6}{
5871           \bool_set_true:N \l_stex_allow_semantic_bool
5872           \tl_set:Nn \exp_not:N \l_stex_current_full_tl
5873             {\l_stex_current_full_tl}
5874           \tl_set:Nn \exp_not:N \l_stex_current_term_tl {
5875             \exp_not:N \symuse{Metatheory?sequence-map}
5876             {\exp_not:n{#2}}
5877             {\exp_args:No \exp_not:n \l_tmpa_tl}
5878           }
5879           \__stex_expr_do_headterm:nn{}{
5880             \_stex_term_do_aB_clist:
5881           }
5882         }
5883       }
5884     }
5885   }
5886 }
5887
5888 \cs_new_protected:Nn \__stex_expr_varseq_in_map:nnnnnnnn {
5889   \__stex_expr_assoc_make_seq:nnn{#2}{#6}{#3}
5890 }
5891

```

(End of definition for `\_stex_term_arg:aB:nnnnn`. This function is documented on page 151.)

sfunction

`\seqmap`

```

5892 \cs_new_protected:Npn \seqmap #1 #2 {
5893   \symuse{Metatheory?sequence-expression}{\seqmap{#1}{#2}}
5894   % \cs_set:Npn \l_tmpa_cs ##1 {#1}
5895   % \tl_set:Ne \l_tmpa_tl {
5896   %   \symuse{Metatheory?bind}{
5897   %     \svar[MAP_DUMMY]{\exp_not:N\mathcal{m}}
5898   %   }{
5899   %     \exp_args:No\exp_not:n{\l_tmpa_cs{\svar[MAP_DUMMY]{\mathcal{m}}}}
5900   %   }
5901   % }
5902   % \__stex_expr_annotate:nnn{complex}{OMS}{
5903   %   \stex_annotate_invisible:nn{data-ftml-headterm={}}{
5904   %     {\bool_set_true:N \l_stex_allow_semantic_bool
5905   %       \ensuremath{

```

```

5906 %           \exp_args:Nnno \symuse{Metatheory?sequence~map}{#2}\l_tmpa_tl
5907 %           }
5908 %       }
5909 %   }
5910 %       \symuse{Metatheory?sequence~expression}{\seqmap{#1}{#2}}
5911 %   }
5912 }

```

(End of definition for `\seqmap`. This function is documented on page 151.)

sfunction

32.4.3 Notation components

```

5913 <@@=stex_comps>

\comp
\compemph@uri
\compemph
5914 \cs_set_protected:Npn \comp {}
5915
5916 \cs_new_protected:Npn \compemph@uri #1 #2 {
5917     \compemph{ #1 }
5918 }
5919
5920 \cs_new_protected:Npn \compemph #1 { #1 }
5921
5922 \cs_new_protected:Npn \_comp {
5923     \_do_comp:nnNn {comp}{}\compemph@uri
5924 }
5925
5926 \cs_new_protected:Npn \_thiscomp #1 #2 {
5927     {\tl_set:cn{this}{}}#1{#2}\c_math_subscript_token{
5928         \group_begin:
5929         \bool_set_true:N \l_stex_allow_semantic_bool
5930         \bool_set_true:N \l_stex_in_this_bool
5931         \l_stex_struct_this_tl
5932         \group_end:
5933     }
5934 }
5935 }
5936
5937 \def\maincomp{\comp}

```

(End of definition for `\comp`, `\compemph@uri`, and `\compemph`. These functions are documented on page 151.)

sfunction

```

\varemp@uri
\varemp
5938 \cs_new_protected:Npn \varemp@uri #1 #2 {
5939     \varemp{#1}
5940 }
5941
5942 \cs_new_protected:Npn \varemp #1 { #1 }
5943
5944 \cs_new_protected:Npn \_varcomp {
5945     \_do_comp:nnNn {comp}{}\varemp@uri
5946 }

```

(End of definition for \varemp@uri and \varemp. These functions are documented on page 151.)
sfunction

```
\defemph@uri
\defemph
5947 \cs_new_protected:Npn \defemph@uri #1 #2 {
5948   \defemph{#1}
5949 }
5950
5951 \cs_new_protected:Npn \defemph #1 {
5952   \ifmmode\else\expandafter\textbf\fi{#1}
5953 }
5954
5955 \cs_new_protected:Npn \_defcomp {
5956   \_do_comp:nnNn {defcomp}{ }\defemph@uri
5957 }
```

(End of definition for \defemph@uri and \defemph. These functions are documented on page 151.)
sfunction

```
\symrefemph@uri
\symrefemph
5958 \cs_new_protected:Npn \symrefemph@uri #1 #2 {
5959   \symrefemph{#1}
5960 }
5961
5962 \cs_new_protected:Npn \symrefemph #1 { \emph{#1} }
```

(End of definition for \symrefemph@uri and \symrefemph. These functions are documented on page 151.)

sfunction

The following commands do the actual attributes:

```
\_do_comp:nnNn
5963 \bool_new:N \g_stex_in_comp_bool
5964
5965 \cs_new_protected:Nn \_do_comp:nnNn {
5966   \stex_pseudogroup_with:nn{\comp}{
5967     \def\comp##1{##1}
5968     \bool_set_true:N \g_stex_in_comp_bool
5969     \stex_if_html_backend:TF {
5970       \stex_annotate:nn { data-ftml-#1 = {#2}}{ #4 }
5971     }{
5972       \tl_if_empty:NTF \l_stex_current_full_tl {
5973         #4
5974       }{
5975         #3 { #4 } \l_stex_current_full_tl
5976       }
5977     }
5978   }
5979   \bool_set_false:N \g_stex_in_comp_bool
5980 }
5981 }
```

(End of definition for _do_comp:nnNn. This function is documented on page 151.)
sfunction

32.4.4 Symbol References

Keys:

```

5982 \stex_keys_define:nnnn{symname}{
5983   \tl_clear:N \l_stex_key_pre_tl
5984   \tl_clear:N \l_stex_key_post_tl
5985   %\tl_clear:N \l_stex_key_root_tl
5986 }{
5987   pre .tl_set:N = \l_stex_key_pre_tl ,
5988   post .tl_set:N = \l_stex_key_post_tl ,
5989   %root .code:n = {}% .tl_set:N = \l_stex_key_root_tl
5990 }{}
5991
5992 \stex_keys_define:nnnn{symref}{
5993   %\tl_clear:N \l_stex_key_root_tl
5994 }{
5995   root .code:n = {}% .tl_set:N = \l_stex_key_root_tl
5996 }{}

```

\symref

\sr

```

5997 \NewDocumentCommand \symref { 0{ } m m } {
5998   \group_begin:
5999   \stex_keys_set:nn{symname}{#1}
6000   \stex_get_symbol:n{#2}
6001   \__stex_comps_do_ref:nNn{#3}\symrefemph@uri\stex_term_oms:nn
6002 }
6003 \let\sr\symref

```

(End of definition for \symref and \sr. These functions are documented on page 152.)

sfunction

\symname

\sn

\sns

\Symname

\Sn\Sns

```

6004 \cs_new:Nn \__stex_comps_split_slash:n {
6005   \__stex_comps_split_slash_i:nw #1//
6006 }
6007
6008 \cs_new:Npn \__stex_comps_split_slash_i:nw #1/#2/ {
6009   \tl_if_empty:nTF{#2}{#1}{
6010     \__stex_comps_split_slash_i:nw #2/
6011   }
6012 }
6013
6014 \NewDocumentCommand \symname { 0{ } m } {
6015   \group_begin:
6016   \stex_keys_set:nn{symname}{#1}
6017   \stex_get_symbol:n{#2}
6018   \tl_set:Nn \l_stex_current_full_tl { \stex_use_symbol_uri:N \l_stex_get_symbol_uri }
6019   \tl_set:Ne \l_stex_current_display_tl {
6020     \exp_args:Ne \__stex_comps_split_slash:n {\stex_symbol_uri_name:N \l_stex_get_symbol_uri
6021   }
6022   \__stex_comps_do_ref:nNn{
6023     \l_stex_key_pre_tl \l_stex_current_display_tl \l_stex_key_post_tl
6024   }\symrefemph@uri\stex_term_oms:nn
6025 }

```

```

6026 \let\sn\symname
6027 \protected\def\sns{\symname[post=s]}
6028
6029 \cs_new:Nn \__stex_comps_uppercase:n { \uppercase{#1}}
6030
6031 \NewDocumentCommand \Symname { 0{} m} {
6032   \group_begin:
6033   \stex_keys_set:nn{\symname}{#1}
6034   \stex_get_symbol:n{#2}
6035   \tl_set:Nn \l_stex_current_full_tl { \stex_use_symbol_uri:N \l_stex_get_symbol_uri }
6036   \tl_set:Ne \l_stex_current_display_tl {
6037     \exp_args:Ne \__stex_comps_split_slash:n {\stex_symbol_uri_name:N \l_stex_get_symbol_uri
6038   }
6039   \__stex_comps_do_ref:nNn{
6040     \l_stex_key_pre_tl \exp_args:NNe \use:nn \__stex_comps_uppercase:n \l_stex_current_disp
6041   }\symrefemph@uri\stex_term_oms:nn
6042 }
6043 \let\Sn\Symname
6044 \protected\def\Sns{\Symname[post=s]}

```

(End of definition for \symname and others. These functions are documented on page 152.)

sfunction

```

\varref
\varname
\Varname
6045 \NewDocumentCommand \varref { 0{} m m} {
6046   \group_begin:
6047   \stex_keys_set:nn{\symname}{#1}
6048   \stex_get_var:n{#2}
6049   \__stex_comps_do_ref:nNn{#3}\varemp@uri{
6050     \tl_set:Nn \l_stex_current_full_tl { \l_stex_get_variable_str }
6051     \tl_set:Nn \l_stex_current_display_tl {\l_stex_get_variable_str}
6052     \def\comp{\_varcomp}
6053     \stex_term_omv:nn
6054   }
6055 }
6056
6057 \NewDocumentCommand \varname { 0{} m} {
6058   \group_begin:
6059   \stex_keys_set:nn{\symname}{#1}
6060   \stex_get_var:n{#2}
6061   \__stex_comps_do_ref:nNn{
6062     \l_stex_key_pre_tl\l_stex_get_variable_str\l_stex_key_post_tl
6063   }\varemp@uri{
6064     \tl_set:Nn \l_stex_current_full_tl { \l_stex_get_variable_str }
6065     \tl_set:Nn \l_stex_current_display_tl {\l_stex_get_variable_str}
6066     \def\comp{\_varcomp}
6067     \stex_term_omv:nn
6068   }
6069 }
6070
6071 \NewDocumentCommand \Varname { 0{} m} {
6072   \group_begin:
6073   \stex_keys_set:nn{\symname}{#1}
6074   \stex_get_var:n{#2}

```

```

6075 \_stex_comps_do_ref:nNn{
6076   \l_stex_key_pre_tl\exp_after:wN\_stex_comps_uppercase:n\l_stex_get_variable_str\l_stex
6077 } \vareph@uri{
6078   \tl_set:Nn \l_stex_current_full_tl { \l_stex_get_variable_str }
6079   \tl_set:Nn \l_stex_current_display_tl {\l_stex_get_variable_str}
6080   \def\comp{\_varcomp}
6081   \_stex_term_omv:nn
6082 }
6083 }

```

(End of definition for `\varref`, `\varname`, and `\Varname`. These functions are documented on page 152.)

sfunction

`\definiendum`

`\definame`

`\Definame`

`\defnotation`

```

6084 \stex_keys_define:nnnn{defname}{-}{-}{
6085   gf      .code:n      = {},
6086   root    .code:n      = {}
6087 }{symname}
6088
6089 \stex_keys_define:nnnn{definiendum}{-}{-}{
6090   gf      .code:n      = {},
6091   root    .code:n      = {}
6092 }{-}
6093
6094 \NewDocumentCommand \definiendum { O{} m m } {
6095   \stex_keys_set:nn{definiendum}{#1 }
6096   \_stex_comps_do_defref:nn{#2}{#3}
6097 }
6098 \stex_deactivate_macro:Nn \definiendum {definition~environments}
6099
6100 \NewDocumentCommand \definame { O{} m } {
6101   \stex_keys_set:nn{definame}{#1}
6102   \_stex_comps_do_defref:nn{#2}{
6103     \l_stex_key_pre_tl\l_stex_current_display_tl\l_stex_key_post_tl
6104   }
6105 }
6106 \protected\def\definames{\definame[post=s]}
6107 \stex_deactivate_macro:Nn \definame {definition~environments}
6108
6109
6110 \NewDocumentCommand \Definame { O{} m } {
6111   \stex_keys_set:nn{definame}{#1}
6112   \_stex_comps_do_defref:nn{#2}{
6113     \l_stex_key_pre_tl \exp_args:NNe \use:nn \_stex_comps_uppercase:n \l_stex_current_disp
6114   }
6115 }
6116 \protected\def\Definames{\Definame[post=s]}
6117 \stex_deactivate_macro:Nn \Definame {definition~environments}
6118
6119
6120 \NewDocumentCommand \defnotation{ m } {
6121   \_stex_next_symbol:n { \def\comp{\_defcomp}}#1
6122 }
6123 \stex_deactivate_macro:Nn \defnotation {definition~environments}

```


(End of definition for `\definiendum` and others. These functions are documented on page 152.)

sfunction

Does the actual referencing:

```

6124 \cs_new_protected:Nn \__stex_comps_do_ref:nNn {
6125   \stex_if_html_backend:T{\relax\ifvmode\indent\fi}
6126   \bool_if:NTF \l_stex_allow_semantic_bool{
6127     \tl_set_eq:NN \l_stex_current_symbol_uri \l_stex_get_symbol_uri
6128     \tl_set:Nn \l_stex_current_full_tl { \stex_use_symbol_uri:N \l_stex_current_symbol_uri
6129       %\str_set:Ne \l_stex_current_symbol_name_str { \stex_symbol_uri_name:N \l_stex_current_
6130       %\str_if_in:NnT \l_stex_current_symbol_name_str / {
6131       % \str_set:Ne \l_stex_current_symbol_name_str {
6132       %   \exp_after:wN \__stex_comps_slash:w \l_stex_current_symbol_name_str
6133       %   /\_stex_args_end:
6134       % }
6135     %}
6136     \tl_clear:N \l_stex_current_term_tl
6137     \def\comp{\_comp}
6138     \let\compemph@uri#2
6139     #3{\_comp{#1}}
6140   }{
6141     \msg_error:nnxx{stex}{error/notallowed}{#1}{\l_stex_current_full_tl}
6142   }
6143   \group_end:
6144 }

\definame et al:

6145 \cs_new_protected:Nn \__stex_comps_do_defref:nn {
6146   \stex_if_html_backend:T{\relax\ifvmode\indent\fi}
6147   \group_begin:
6148   \stex_get_symbol:n{#1}
6149   \bool_if:NTF \l_stex_allow_semantic_bool{
6150     \tl_set:Nn \l_stex_current_full_tl { \stex_use_symbol_uri:N \l_stex_get_symbol_uri }
6151     \tl_set:Nn \l_stex_current_display_tl {
6152       \stex_symbol_uri_name:N \l_stex_get_symbol_uri
6153     }
6154     %\str_if_in:NnT \l_stex_get_svariable_str / {
6155     % \str_set:Ne \l_stex_get_variable_str {
6156     %   \exp_after:wN \__stex_comps_slash:w \l_stex_get_variable_str
6157     %   /\_stex_args_end:
6158     % }
6159     %}
6160     \exp_args:Ne \stex_ref_new_sym_target:n \l_stex_current_full_tl
6161     \_do_comp:nnNn {definiendum}{\l_stex_current_full_tl}\defemph@uri{#2}
6162   }{
6163     \msg_error:nnxx{stex}{error/notallowed}{#1}{\l_stex_current_full_tl}
6164   }
6165   \group_end:
6166 }

6167

6168 \cs_new:Npn \__stex_comps_slash:w #1/#2/#3\_stex_args_end: {
6169   \tl_if_empty:nTF{#3}{
6170     #2
6171   }{
6172     \__stex_comps_slash:w #2 / #3 \_stex_args_end:

```

```

6173   }
6174 }

\foldexpr

6175 \stex_if_html_backend:TF {
6176   \NewDocumentCommand \foldexpr { O{} mm } {
6177     \stex_annotate:nn{
6178       data-ftml-fold-expression={\exp_args:Ne \str_if_eq:nnT{#1}{show}{show}}
6179     }{
6180       \stex_annotate:nn{
6181         data-ftml-fold-short={ }
6182       }{#2}
6183       #3
6184     }
6185   }
6186 }{
6187   \NewDocumentCommand \foldexpr { O{} mm } {
6188     \exp_args:Ne \str_if_eq:nnTF{#1}{hide}{
6189       \underbrace{...}{#2}
6190     }{
6191       \underbrace{#3}{#2}
6192     }
6193   }
6194 }

```

(End of definition for `\foldexpr`. This function is documented on page 152.)

sfunction

32.5 Checking

```

\stex_if_check_terms_p:
\stex_if_check_terms:TF
6195 <@@=stex_check>
6196 \stex_if_html_backend:TF {
6197   \prg_new_conditional:Nnn \stex_if_check_terms: {p, T, F, TF} {
6198     \prg_return_false:
6199   }
6200 }{
6201   \stex_get_env:Nn\__stex_check_env_str{STEX_CHECKTERMS}
6202   \str_if_empty:NF\__stex_check_env_str{
6203     \exp_args:No \str_if_eq:nnF \__stex_check_env_str{false}{
6204       \bool_set_true:N \c_stex_check_terms_bool
6205     }
6206   }
6207   \bool_if:NTF \c_stex_check_terms_bool {
6208     \prg_new_conditional:Nnn \stex_if_check_terms: {p, T, F, TF} {
6209       \prg_return_true:
6210     }
6211   }{
6212     \prg_new_conditional:Nnn \stex_if_check_terms: {p, T, F, TF} {
6213       \prg_return_false:
6214     }
6215   }
6216 }

```

(End of definition for `\stex_if_check_terms:TF`. This function is documented on page 152.)

sfunction

`\stex_check_term:nn`

```

6217 \stex_if_check_terms:TF{
6218   \cs_new_protected:Nn \stex_check_term:nn {
6219     \marginpar {\tiny Checking~#1:~
6220     \group_begin:
6221       $#2$
6222     \group_end:
6223   }
6224 }
6225 }{
6226   \cs_new_protected:Nn \stex_check_term:nn {}
6227 }
```

(End of definition for `\stex_check_term:nn`. This function is documented on page 152.)

sfunction

`\_stex_check_terms:`

```

6228 \stex_if_check_terms:TF{
6229   \cs_new_protected:Nn \_stex_check_terms: {
6230     \stex_debug:nn{check_terms}{Checking~type...}
6231     \tl_if_empty:NF \l_stex_key_type_tl {
6232       \stex_check_term:nn{type}\l_stex_key_type_tl
6233     }
6234     \stex_debug:nn{check_terms}{Checking~definiens...}
6235     \tl_if_empty:NF \l_stex_key_def_tl {
6236       \stex_check_term:nn{definiens}\l_stex_key_def_tl
6237     }
6238     \stex_debug:nn{check_terms}{Checking~return...}
6239     \tl_if_empty:NF \l_stex_key_return_tl {
6240       \stex_check_term:nn{return}\l_stex_key_return_tl!}
6241     }
6242     \stex_debug:nn{check_terms}{Checking~argument~types...}
6243     \group_begin:\l_stex_key_argtypes_clist\group_end:
6244     \tl_if_empty:NF \l_stex_key_argtypes_clist {
6245       \stex_check_term:nn{argument~types}\l_stex_key_argtypes_clist}
6246     }
6247   }
6248 }{
6249   \cs_new_protected:Nn \_stex_check_terms: {}
6250 }
6251
```

(End of definition for `\_stex_check_terms:.` This function is documented on page 152.)

sfunction

Chapter 33

Notations

```
6252 <@@=stex_notations>

      keys:
6253 \stex_keys_define:nnnn{notation}{
6254   \str_clear:N \l_stex_key_variant_str
6255   \str_clear:N \l_stex_key_prec_str
6256   \str_clear:N \l_stex_key_op_tl
6257   \str_clear:N \l_stex_key_intent_str
6258   \clist_clear:N \l_stex_key_intent_args_clist
6259 }{
6260   variant      .str_set_x:N = \l_stex_key_variant_str ,
6261   prec         .str_set_x:N = \l_stex_key_prec_str ,
6262   op           .tl_set:N     = \l_stex_key_op_tl ,
6263   intent       .str_set:N    = \l_stex_key_intent_str ,
6264   argnames     .clist_set:N  = \l_stex_key_intent_args_clist ,
6265   unknown      .code:n       = {
6266     \str_if_empty:NTF \l_keys_key_str {
6267       \str_set:Nc \l_stex_key_variant_str {\l_keys_key_tl}
6268     }{
6269       \str_set_eq:NN \l_stex_key_variant_str \l_keys_key_str
6270     }
6271   }
6272 }{style}
6273
6274 \stex_keys_define:nnnn{symdef}{ }{ }{decl,notation}
6275
6276 \stex_keys_define:nnnn{vardef}{
6277   \bool_set_false:N \l__stex_notations_bind_bool
6278 }{
6279   bind .bool_set:N = \l__stex_notations_bind_bool
6280 }{symdef}
6281
\notation

6282 \stex_new_stylable_cmd:nnnn {notation} { s m O{ } m } {
6283   \stex_keys_set:nn{notation}{#3}
6284   \stex_get_symbol:n{#2}
6285   \stex_notation_parse:n{#4}
6286   \stex_if_check_terms:T{ \_stex_notation_check: }
```

```

6287 \_stex_notation_add:
6288 \stex_if_do_html:T {
6289   \def\comp{\_comp}
6290   \_stex_notation_do_html:n{\stex_use_symbol_uri:N \l_stex_get_symbol_uri}
6291 }
6292 \IfBooleanTF#1{
6293   \_stex_notation_set_default:n{
6294     \l_stex_get_symbol_uri
6295   }
6296 }{}
6297 \stex_if_smsmode:F{
6298   \group_begin:
6299   \__stex_notations_styledefs:
6300   \stex_style_apply:
6301   \group_end:
6302 }
6303 \stex_smsmode_do:
6304 }{}
6305
6306 \stex_deactivate_macro:Nn \notation {module~environments}
6307 \stex_every_module:n {\stex_reactivate_macro:N \notation}
6308 \stex_sms_allow_escape:N \notation
6309
6310 \cs_new_protected:Nn \__stex_notations_styledefs: {
6311   \str_set_eq:NN\thisnotationvariant\l_stex_key_variant_str
6312   \str_set:Nn \thisdeclname {\stex_symbol_uri_name:N \l_stex_get_symbol_uri}
6313   \tl_set:Ne \thisdecluri {\stex_use_symbol_uri:N \l_stex_get_symbol_uri}
6314   \def\thisnotation{
6315     \exp_args:Nne \use:nn{\l_stex_notation_macrocode_cs}{ }{
6316       \_stex_notation_make_args:
6317     }
6318   }
6319 }

```

(End of definition for \notation. This function is documented on page 153.)

sfunction

\varnotation

```

6320 \stex_new_stylable_cmd:nnnn {\varnotation} { s m O{} m} {
6321   \stex_keys_set:nn{notation}{#3}
6322   \stex_get_var:n{#2}
6323   \str_set_eq:NN \l_stex_key_name_str \l_stex_get_variable_str
6324   \stex_notation_parse:n{#4}
6325   \stex_if_check_terms:T{ \_stex_notation_check: }
6326   \_stex_var_notation_macro:
6327   \IfBooleanTF#1{
6328     \_stex_notation_set_default:n{\l_stex_get_variable_str}
6329   }{}
6330   \group_begin:
6331   \tl_set_eq:NN \thisvarname \l_stex_get_variable_str
6332   \tl_clear:N \thisstyle
6333   \str_set_eq:NN\thisnotationvariant\l_stex_key_variant_str
6334   \def\thisnotation{
6335     \exp_args:Nne \use:nn{\l_stex_notation_macrocode_cs}{ }{

```

```

6336     \_stex_notation_make_args:
6337   }
6338 }
6339 \stex_style_apply:
6340 \group_end:
6341 }{}

```

(End of definition for \varnotation. This function is documented on page 153.)

sfunction

\stex_notation_parse:n

```

6342 \cs_new_protected:Nn \stex_notation_parse:n {
6343   \tl_if_empty:NF \l_stex_key_op_tl {
6344     \tl_set:Ne \l_stex_key_op_tl { \exp_not:N\maincomp {
6345       \exp_args:No \exp_not:n \l_stex_key_op_tl
6346     } }
6347   }
6348   \seq_clear:N \l__stex_notations_precs_seq
6349   \tl_clear:N \l_stex_notation_args_tl
6350   \int_compare:nNnTF \l_stex_get_symbol_arity_int = 0 {
6351     \__stex_notations_const_precs:
6352     \tl_if_empty:NT \l_stex_key_op_tl {
6353       \tl_set:Nn \l_stex_key_op_tl { \maincomp{#1} }
6354     }
6355   }{
6356     \__stex_notations_fun_precs:
6357     \__stex_notations_do_missing_args:n{#1}
6358     \tl_if_empty:NT \l_stex_key_op_tl {
6359       \hbox_set:Nn \l_tmpa_box {
6360         \tl_clear:N \l_stex_current_full_tl
6361         \tl_clear:N \l_stex_current_display_tl
6362         \cs_set:Npn \l_tmpa_cs ##1 ##2 ##3 ##4 ##5 ##6 ##7 ##8 ##9 { #1 }
6363         \cs_set:Npn \maincomp ##1 {
6364           \tl_gset:Nn \l_stex_key_op_tl { \maincomp{##1} }
6365           ##1
6366         }
6367         \cs_set:Npn \argsep ##1 ##2 {##1 ##2}
6368         \cs_set:Npn \argmap ##1 ##2 ##3 {##1 ##3}
6369         \cs_set:Npn \argarraymap ##1 ##2 ##3 ##4 {
6370           ##1 ##2
6371         }
6372         \stex_suppress_html:n{ $\l_tmpa_cs abcdefghj$ }
6373       }
6374     }
6375   }
6376   \exp_args:NNe
6377   \tl_set:Nn \l_stex_notation_macrocode_cs {
6378     \STEXInternalNotation
6379     { \l_stex_key_variant_str }
6380     { \l__stex_notations_opprec_tl }
6381     { \l_stex_key_intent_str }
6382     { \l_stex_notation_args_tl }
6383     {
6384       \int_compare:nNnTF \l_stex_get_symbol_arity_int = 0

```

```

6385         { \exp_not:n { \maincomp{ #1 } } }
6386         { \exp_not:n { #1 } \l__stex_notations_missing_tl }
6387     }
6388 }
6389 \stex_debug:nn{notation}{Notation:~\meaning\l_stex_notation_macrocode_cs}
6390 }

precedences:
6391 \cs_new_protected:Nn \__stex_notations_const_precs: {
6392     \str_if_empty:NTF \l_stex_key_prec_str {
6393         \tl_set:No \l__stex_notations_opprec_tl { \neginfprec }
6394     }{
6395         \str_if_eq:onTF \l_stex_key_prec_str {nobrackets}{
6396             \tl_set:No \l__stex_notations_opprec_tl { \neginfprec }
6397         }{
6398             \tl_set_eq:NN \l__stex_notations_opprec_tl \l_stex_key_prec_str
6399         }
6400     }
6401 }
6402
6403 \cs_new_protected:Nn \__stex_notations_fun_precs: {
6404     \str_if_empty:NTF \l_stex_key_prec_str {
6405         \tl_set:No \l__stex_notations_opprec_tl { \neginfprec }
6406     }{
6407         \str_if_eq:onTF \l_stex_key_prec_str {nobrackets}{
6408             \tl_set:No \l__stex_notations_opprec_tl { \neginfprec }
6409         }{
6410             \tl_set_eq:NN \l__stex_notations_opprec_tl \l_stex_key_prec_str
6411         }
6412     }
6413     \str_if_empty:NTF \l_stex_key_prec_str {
6414         \tl_set:Nn \l__stex_notations_opprec_tl { 0 }
6415         \int_step_inline:nn \l_stex_get_symbol_arity_int {
6416             \seq_put_right:Nn \l__stex_notations_precs_seq {0}
6417         }
6418     }{
6419         \str_if_eq:onTF \l_stex_key_prec_str {nobrackets}{
6420             \stex_debug:nn{notation}{No~brackets}
6421             \tl_set:No \l__stex_notations_opprec_tl { \neginfprec }
6422             \int_step_inline:nn \l_stex_get_symbol_arity_int {
6423                 \exp_args:NNo \seq_put_right:Nn \l__stex_notations_precs_seq \infprec
6424             }
6425         }\__stex_notations_parse_precs:
6426     }
6427     \__stex_notations_do_argnames:
6428 }
6429
6430 \cs_new_protected:Nn \__stex_notations_parse_precs: {
6431     \stex_debug:nn{notation}{parsing~precedence~\l_stex_key_prec_str}
6432     \seq_set_split:NnV \l__stex_notations_seq ; \l_stex_key_prec_str
6433     \seq_pop_left:NNTF \l__stex_notations_seq \l__stex_notations_str {
6434         \tl_set_eq:NN \l__stex_notations_opprec_tl \l__stex_notations_str
6435         \seq_pop_left:NNT \l__stex_notations_seq \l__stex_notations_str {
6436             \exp_args:NNo \seq_set_split:NnV \l__stex_notations_seq
6437                 {\tl_to_str:n{x}} \l__stex_notations_str

```

```

6438     }
6439   }{
6440     \tl_set:No \l__stex_notations_opprec_tl { 0 }
6441   }
6442   \int_step_inline:nn \l_stex_get_symbol_arity_int {
6443     \seq_pop_left:NNTF \l__stex_notations_seq \l__stex_notations_str {
6444       \seq_put_right:No \l__stex_notations_precs_seq \l__stex_notations_str
6445     }{
6446       \seq_put_right:No \l__stex_notations_precs_seq \l__stex_notations_opprec_tl
6447     }
6448   }
6449 }

```

Experimental; named arguments:

```

6450 \cs_new_protected:Nn \__stex_notations_do_argnames: {
6451   \tl_clear:N \l_stex_notation_args_tl
6452   \stex_map_args:N \__stex_notations_do_argname:nn
6453 }
6454
6455 \cs_new_protected:Nn \__stex_notations_do_argname:nn {
6456   \clist_if_empty:NNTF \l_stex_key_intent_args_clist {
6457     \tl_put_right:Ne \l_stex_notation_args_tl {
6458       #1#2{\seq_item:Nn \l__stex_notations_precs_seq #1}{
6459         \str_if_empty:NF \l_stex_key_intent_str {#1}
6460       }
6461     }
6462   }{
6463     \tl_put_right:Ne \l_stex_notation_args_tl {
6464       #1#2{\seq_item:Nn \l__stex_notations_precs_seq #1}
6465       {\c_dollar_str\clist_item:Nn \l_stex_key_intent_args_clist 1}
6466     }
6467     \clist_pop:NN \l_stex_key_intent_args_clist \l_tmpa_tl
6468   }
6469 }

```

inserts hidden arguments that don't occur in the body of the notation:

```

6470 \cs_new:Nn \__stex_notations_do_missing_args:n {
6471   \str_set:Nn \l__stex_notations_missing_str {#1}
6472   \exp_args:NNe \seq_set_split:NnV \l_tmpa_seq {\c_hash_str\c_hash_str\c_hash_str\c_hash_str}
6473   \str_clear:N \l__stex_notations_missing_str
6474   \seq_map_inline:Nn \l_tmpa_seq {
6475     \tl_put_right:Nn \l__stex_notations_missing_str { ##1 }
6476   }
6477   \tl_clear:N \l__stex_notations_missing_tl
6478   \stex_map_args:N \__stex_notations_add_missing_args:nn
6479 }
6480
6481 \cs_new:Nn \__stex_notations_add_missing_args:nn {
6482   % TODO this skips arguments if e.g. ##1 occurs rather than #1!!
6483   \exp_args:NNe \str_if_in:NnF \l__stex_notations_missing_str {\c_hash_str\c_hash_str#1}{
6484     \tl_put_right:Nn \l__stex_notations_missing_tl{\STEXinvisible{## #1}}
6485   }
6486 }

```

(End of definition for `\stex_notation_parse:n`. This function is documented on page 153.)

sfunction

_stex_notation_add:

```

6487 \cs_new_protected:Nn \_stex_notation_add: {
6488   \stex_add_notation:ooeoo
6489   {\l_stex_get_symbol_uri}
6490   \l_stex_key_variant_str
6491   {\int_use:N \l_stex_get_symbol_arity_int}
6492   \l_stex_notation_macrocode_cs
6493   \l_stex_key_op_tl
6494 }

```

(End of definition for _stex_notation_add:. This function is documented on page 153.)

sfunction

\stex_add_notation:nnnnn

\stex_add_notation:ooeoo

```

6495 \cs_new:Nn \__stex_notations_macro:nn {
6496   l_stex_notation_ #1?#2 _cs
6497 }
6498 \cs_new:Nn \__stex_notations_op_macro:nn {
6499   l_stex_notation_ #1?#2 _op_cs
6500 }
6501
6502 \cs_new_protected:Nn \stex_add_notation:nnnnn {
6503   \exp_args:Nne \stex_debug:nn{notations}{Adding~notation:^^J
6504     #1~\c_hash_str#2~#3^^J
6505     \tl_to_str:n{#4}^^J\tl_to_str:n{#5}^^J
6506     to~\stex_use_module_uri:N \l_stex_current_module_uri
6507   }
6508   \prop_gput:cen{ \_stex_notations_macro:N \l_stex_current_module_uri }
6509   {\stex_use_symbol_uri:n{#1}?#2}{\{#1\}{#2\}{#3\}{#4\}{#5}}
6510   \stex_do_up_to_module:n {
6511     \__stex_notations_set_macro:nnnnn{#1}{#2}{#3}{#4}{#5}
6512     %\__stex_notations_activate_not:nn{#1}{#2}
6513   }
6514 }
6515 \cs_generate_variant:Nn \stex_add_notation:nnnnn {ooeoo}
6516
6517 \cs_new_protected:Nn \_stex_activate_notations: {
6518   \stex_debug:nn{activating}{All~notations:~\cs_meaning:c{ \_stex_notations_macro:N \l_stex
6519   \prop_map_inline:cn{ \_stex_notations_macro:N \l_stex_current_module_uri }{
6520     \__stex_notations_set_macro:nnnnn ##2
6521   }
6522 }
6523
6524 \cs_new_protected:Nn \__stex_notations_activate_not:nn {
6525   \bool_set_false:N \l_tmpa_bool
6526   \stex_debug:nn{notations}{activating~\stex_use_symbol_uri:n{#1}?#2}
6527   \prop_map_inline:cn{ \_stex_notations_macro:N \l_stex_current_module_uri }{
6528     %\stex_debug:nn{notations}{##1?}
6529     \exp_args:Ne \str_if_eq:nnT{\stex_use_symbol_uri:n{#1}?#2}{##1}{
6530       \prop_map_break:n{
6531         %\stex_debug:nn{notations}{Found:~\tl_to_str:n{##2}}
6532         \bool_set_true:N \l_tmpa_bool

```

```

6533     \_stex_notations_set_macro:nnnnn ##2
6534   }
6535 }
6536 }
6537 \bool_if:NF \l_tmpa_bool {
6538   \errmessage{Woop}
6539 }
6540 }
6541
6542 \cs_new_protected:Nn \_stex_notations_set_macro:nnnnn {
6543   \tl_set:cn {\_stex_notations_macro:nn{\stex_use_symbol_uri:n{#1}}{#2}}{#4}
6544   \cs_if_exist:cF{\_stex_notations_macro:nn{\stex_use_symbol_uri:n{#1}}{}}{
6545     \tl_set:cn {\_stex_notations_macro:nn{\stex_use_symbol_uri:n{#1}}{}}{#4}
6546   }
6547   \tl_if_empty:nF{#5}{
6548     \tl_set:cn{\_stex_notations_op_macro:nn{\stex_use_symbol_uri:n{#1}}{#2}}{#5}
6549     \cs_if_exist:cF{\_stex_notations_op_macro:nn{\stex_use_symbol_uri:n{#1}}{}}{
6550       \tl_set:cn{\_stex_notations_op_macro:nn{\stex_use_symbol_uri:n{#1}}{}}{#5}
6551     }
6552   }
6553 }

```

(End of definition for \stex_add_notation:nnnnn. This function is documented on page 153.)

sfunction

_stex_map_args:N

_stex_map_notation_args:N

```

6554 \cs_new:Nn \_stex_map_args:N {
6555   \tl_if_empty:NF \l_stex_get_symbol_args_tl {
6556     \exp_after:wN \_stex_notations_map_args_i:w \exp_after:wN
6557     #1 \l_stex_get_symbol_args_tl \_stex_notations_args_end:
6558   }
6559 }
6560 \cs_new:Npn \_stex_notations_map_args_i:w #1 #2 #3 #4 \_stex_notations_args_end: {
6561   #1 {#2} {#3}
6562   \tl_if_empty:nF{#4}{
6563     \_stex_notations_map_args_i:w #1 #4 \_stex_notations_args_end:
6564   }
6565 }
6566
6567 \cs_new:Nn \_stex_map_notation_args:N {
6568   \tl_if_empty:NF \l_stex_notation_args_tl {
6569     \exp_after:wN \_stex_notations_map_args_ii:w \exp_after:wN
6570     #1 \l_stex_notation_args_tl \_stex_notations_args_end:
6571   }
6572 }
6573 \cs_new:Npn \_stex_notations_map_args_ii:w #1 #2 #3 #4 #5 #6 \_stex_notations_args_end: {
6574   #1 {#2} {#3} {#4} {#5}
6575   \tl_if_empty:nF{#6}{
6576     \_stex_notations_map_args_ii:w #1 #6 \_stex_notations_args_end:
6577   }
6578 }

```

(End of definition for _stex_map_args:N and _stex_map_notation_args:N. These functions are documented on page 153.)

sfunction

`\_stex_var_notation_macro:`

```

6579 \cs_new_protected:Nn \_stex_var_notation_macro: {
6580   \tl_set_eq:cN {\_stex_notations_macro:nn{\l_stex_key_name_str}{\l_stex_key_variant_str}}
6581   \cs_if_exist:cF {\_stex_notations_macro:nn{\l_stex_key_name_str}{}}{
6582     \tl_set_eq:cN{\_stex_notations_macro:nn{\l_stex_key_name_str}{}}
6583     \l_stex_notation_macrocode_cs
6584   }
6585   \tl_if_empty:NF \l_stex_key_op_tl {
6586     \tl_set_eq:cN {\_stex_notations_op_macro:nn{\l_stex_key_name_str}{\l_stex_key_variant_
6587     \cs_if_exist:cF{\_stex_notations_op_macro:nn{\l_stex_key_name_str}{}}{
6588       \cs_set_eq:cN{\_stex_notations_op_macro:nn{\l_stex_key_name_str}{}}
6589       \l_stex_key_op_tl
6590     }
6591   }
6592 }

```

(End of definition for `\_stex_var_notation_macro:`. This function is documented on page 153.)

`sfunction`

`\setnotation`

`\_stex_notation_set_default:n`

```

6593 \cs_new_protected:Nn \_stex_notation_set_default:n{
6594   \stex_if_in_module:TF{
6595     \stex_add_notation:ooeo{#1}{
6596       {\int_use:N \l_stex_get_symbol_arity_int}
6597       \l_stex_notation_macrocode_cs
6598       \l_stex_key_op_tl
6599     }{
6600       \cs_set_eq:cN {\_stex_notations_macro:nn {\stex_use_symbol_uri:N \l_stex_get_symbol_ur
6601       \l_stex_notation_macrocode_cs
6602       \tl_if_empty:NF \l_stex_key_op_tl {
6603         \cs_set_eq:cN{\_stex_notations_op_macro:nn{\stex_use_symbol_uri:N \l_stex_get_symbol
6604         \l_stex_key_op_tl
6605       }
6606     }
6607   }
6608
6609   \cs_new_protected:Npn \setnotation #1 #2 {
6610     \stex_get_symbol:n{#1}
6611     \cs_if_exist:cTF{\_stex_notations_macro:nn{\stex_use_symbol_uri:N \l_stex_get_symbol_uri
6612     \tl_set_eq:Nc \l_stex_notation_macrocode_cs {\_stex_notations_macro:nn{\stex_use_symbol
6613     \cs_if_exist:cTF{\_stex_notations_op_macro:nn{\stex_use_symbol_uri:N \l_stex_get_symbol
6614     \tl_set_eq:Nc \l_stex_key_op_tl {\_stex_notations_op_macro:nn{\stex_use_symbol_uri:N
6615     }{
6616       \tl_clear:N \l_stex_key_op_tl
6617     }
6618     \_stex_notation_set_default:n{
6619       \l_stex_get_symbol_uri
6620     }
6621   }{
6622     \msg_error:nnxx{stex}{error/unknownnotation}{#2}{
6623       \stex_use_symbol_uri:N \l_stex_get_symbol_uri
6624     }
6625   }
6626 }

```

(End of definition for `\setnotation` and `\_stex_notation_set_default:n`. These functions are documented on page 153.)

sfunction

`\stex_iterate_notations:nn`

```

6627 \cs_new_protected:Nn \stex_iterate_notations:nn {
6628   \seq_clear:N \l__stex_notations_mods_seq
6629   \stex_pseudogroup_with:nn{\__stex_notations_not_cs:nnnnn}{
6630     \cs_set:Npn \__stex_notations_not_cs:nnnnn
6631       ##1 ##2 ##3 ##4 ##5 { #2 }
6632     \clist_map_function:nN {#1} \__stex_notations_it_not_i:n
6633   }
6634 }
6635
6636 \cs_new_protected:Nn \__stex_notations_it_not_i:n {
6637   \seq_if_in:NnF \l__stex_notations_mods_seq {#1} {
6638     \seq_put_left:Nn \l__stex_notations_mods_seq {#1}
6639     \prop_map_inline:cn{\_stex_notations_macro:n{#1}}{
6640       \__stex_notations_not_cs:nnnnn ##2
6641     }
6642     \prop_map_inline:cn{\_stex_morphisms_macro:n{#1}}{
6643       \__stex_notations_it_not_check:nnnn ##2
6644     }
6645   }
6646 }
6647 \cs_new_protected:Nn \__stex_notations_it_not_check:nnnn {
6648   \tl_if_empty:nT{#1}{
6649     \__stex_notations_it_not_i:n {#2}
6650   }
6651 }
```

(End of definition for `\stex_iterate_notations:nn`. This function is documented on page 154.)

sfunction

`\stex_use_notation:nnTF`

`\stex_use_op_notation:nnTF`

```

6652 \cs_new_protected:Npn \stex_use_notation:nnTF #1 #2 #3 #4 {
6653   \cs_if_exist:cTF{\_stex_notations_macro:nn{#1}{#2}}{
6654     \cs_set_eq:Nc \l_stex_notation_cs {\_stex_notations_macro:nn{#1}{#2}}
6655     #3
6656   }{#4}
6657 }
6658 \cs_new_protected:Npn \stex_use_op_notation:nnTF #1 #2 #3 #4 {
6659   \cs_if_exist:cTF{\_stex_notations_op_macro:nn{#1}{#2}}{
6660     \cs_set_eq:Nc \l_stex_notation_cs {\_stex_notations_op_macro:nn{#1}{#2}}
6661     #3
6662   }{#4}
6663 }
```

(End of definition for `\stex_use_notation:nnTF` and `\stex_use_op_notation:nnTF`. These functions are documented on page 154.)

sfunction

`\_stex_notation_check:`

```

6664 \cs_new_protected:Nn \_stex_notation_check: {
```

```

6665 \stex_check_term:nn{notation}{
6666   \cs_set:Npn \comp ##1 {##1}
6667   \stex_debug:nn{check}{Checking~notation:~\cs_meaning:N \l_stex_notation_macrocode_cs ^^
6668   \exp_args:Nne \use:nn{\l_stex_notation_macrocode_cs}{}}{
6669     \stex_notation_make_args:
6670   }
6671 }
6672 }

```

(End of definition for `\_stex_notation_check:`. This function is documented on page 154.)

sfunction

`\_stex_notation_do_html:n`

```

6673 \cs_new_protected:Nn \_stex_notation_do_html:n {
6674   \stex_annotate_invisible:n{\_stex_annotate_force_break:n{\hbox{\stex_annotate:nn {
6675     data-ftml-notation={#1},
6676     data-ftml-notationfragment={\l_stex_key_variant_str},
6677     data-ftml-precedence={\l__stex_notations_opprec_tl},
6678     data-ftml-argprec={\seq_use:Nn \l__stex_notations_precs_seq ,}
6679   }}{
6680     \cs_set_protected:Npn \argsep ##1 ##2 {
6681       \stex_annotate:nn{data-ftml-argsep={}}{
6682         ##1 \tl_if_empty:nTF{##2}{\!\,}{##2}
6683       }
6684     }
6685     \cs_set_protected:Npn \argmap ##1 ##2 ##3 {
6686       \cs_set:Npn \__stex_notations_map_cs: #####1 { ##2 }
6687       \stex_annotate:nn{data-ftml-argmap={}}{
6688         \__stex_notations_map_cs:{##1} \stex_annotate:nn{data-ftml-argmap-sep={}}{##3}
6689       }
6690     }
6691     \cs_set_protected:Npn \maincomp {
6692       \do_comp:nnNn {maincomp}{}\compemph@uri
6693     }
6694     \stex_debug:nn{notation}{Doing~notation~HTML:\meaning\l_stex_get_symbol_args_tl}
6695     $
6696     \tl_clear:N \l_stex_current_full_tl
6697     \stex_annotate:nn{data-ftml-notationcomp={}}{
6698       \exp_args:Nne \use:nn {
6699         \l_stex_notation_macrocode_cs {}
6700       }{
6701         \_stex_map_args:N \__stex_notations_make_arg_html:nn
6702       }
6703     }
6704     $
6705     \tl_if_empty:NF \l_stex_key_op_tl {
6706       $
6707       \tl_clear:N \l_stex_current_full_tl
6708       \stex_annotate:nn{data-ftml-notationopcomp={}}{
6709         \_stex_term_oms:nn{\l_stex_key_variant_str}{\l_stex_key_op_tl}
6710       }
6711       $
6712     }
6713   }}

```

```

6714   }}
6715 }
6716
6717 \cs_new:Nn \__stex_notations_make_arg_html:nn {
6718   {\stex_annotate:nn{data-ftml-argnum=#1}{x}}
6719 }

```

(End of definition for `\_stex_notation_do_html:n`. This function is documented on page 154.)

sfunction

`\_stex_notation_make_args:`

```

6720 \cs_new:Nn \_stex_notation_make_args: {
6721   \_stex_map_notation_args:N \_stex_notations_make_arg:nnnn
6722 }
6723
6724 \cs_new:Nn \__stex_notations_make_arg:nnnn {
6725   \str_case:nnF #2 {
6726     a {{
6727       a\c_math_subscript_token{#1,1},
6728       a\c_math_subscript_token{#1,2}
6729     }}
6730     B {{
6731       B\c_math_subscript_token{#1,1},
6732       B\c_math_subscript_token{#1,2}
6733     }}
6734   }{{
6735     \_stex_term_arg:nnnn{#1}{#2}{#3}{#4}
6736     {{#2}\c_math_subscript_token{#1}}
6737   }}
6738 }

```

(End of definition for `\_stex_notation_make_args:.` This function is documented on page 154.)

sfunction

`\stex_do_default_notation:`

`\stex_do_default_notation_op:`

```

6739 \cs_new_protected:Nn \stex_do_default_notation: {
6740   \stex_do_default_notation_op:
6741   \tl_if_empty:NTF \l_stex_current_args_tl {
6742     \tl_clear:N \l__stex_notations_args_tl
6743   }\_stex_notations_default_args:
6744   \tl_set:Ne \l_stex_default_notation {\STEXInternalNotation{}{0}{}}{\l__stex_notations_args_tl}
6745   \exp_args:No \exp_not:n \l_stex_default_notation
6746   }}
6747 }
6748
6749 \cs_new_protected:Nn \stex_do_default_notation_op: {
6750   \_stex_notations_make_name:
6751   \tl_set:Ne \l_stex_default_notation {\exp_not:N \maincomp{ \exp_not:N \mathrm {\l__stex_notations_args_tl}}
6752 }
6753
6754 \cs_new_protected:Nn \__stex_notations_default_args: {
6755   \_stex_notations_make_name:
6756   \tl_set_eq:NN \l_stex_get_symbol_args_tl \l_stex_current_args_tl
6757   \tl_set:Ne \l__stex_notations_args_tl {

```

```

6758 \stex_map_args:N \__stex_notations_augment_arg:nn
6759 }
6760 \tl_put_right:Nn \l_stex_default_notation {\comp{}}
6761 \seq_clear:N \l_tmpa_seq
6762 \int_step_inline:nn \l_stex_current_arity_str {
6763   \seq_put_right:Nn \l_tmpa_seq {#### #1}
6764 }
6765 \tl_put_right:Ne \l_stex_default_notation {
6766   \seq_use:Nn \l_tmpa_seq {\mathpunct{\comp{,}}}}
6767 }
6768 \tl_put_right:Nn \l_stex_default_notation {\comp{}}
6769 }
6770
6771 \cs_new:Nn \__stex_notations_augment_arg:nn {
6772   #1#2{0}{}}
6773 }
6774
6775 \cs_new_protected:Nn \__stex_notations_make_name: {
6776   \exp_args:NNNe \seq_set_split:Nnn \l_tmpa_seq / \l_stex_current_display_tl
6777   \seq_pop_right:NN \l_tmpa_seq \l__stex_notations_name_str
6778 }

```

(End of definition for `\stex_do_default_notation:` and `\stex_do_default_notation_op:`. These functions are documented on page 154.)

sfunction

`\STEXInternalNotation`

```

6779 \cs_new_protected:Npn \STEXInternalNotation #1 #2 #3 #4 #5 #6 {
6780   \__stex_notations_process:nnnnnn{#1}{#2}{#3}{#4}{#5}{
6781     \l__stex_notations_code_tl
6782     #6
6783   }
6784 }
6785
6786 \cs_new_protected:Npn \__stex_notations_process:nnnnnn #1 #2 #3 #4 {
6787   \tl_if_empty:nTF{#4}{
6788     \__stex_notations_simple:nnnnn{#1}{#2}{#3}
6789   }{
6790     \__stex_notations_complex:nnnnnn{#1}{#2}{#3}{#4}
6791   }
6792 }
6793
6794 \cs_new_protected:Nn \__stex_notations_simple:nnnnn {
6795   \stex_debug:nn{Notation~code}{\tl_to_str:n{#4}}
6796   \tl_set:Nn \l__stex_notations_code_tl {
6797     \cs_set:Npn \l__stex_notations_cs {
6798       \stex_maybe_brackets:nn{#2}{
6799         \stex_term_oms_or_omv:nn{#1}{#4}
6800       }
6801     }
6802     \l__stex_notations_cs
6803   }
6804   #5
6805 }

```

```

6806
6807 \cs_new_protected:Nn \__stex_notations_complex:nnnnnn {
6808   \stex_debug:nn{Notation~code}{\tl_to_str:n{#5}}
6809   \int_zero:N \l_tmpa_int
6810   \tl_set:Nn \l__stex_notations_pre_tl {\cs_set_eq:NN \stex_term_oma_or_omb:nn \stex_term_
6811   \tl_set:Nn \l__stex_notations_code_tl {
6812     \cs_generate_from_arg_count:NNnn \l__stex_notations_cs \cs_set:Npn \l_tmpa_int
6813     {
6814       \stex_maybe_brackets:nn{#2}{
6815         \stex_term_oma_or_omb:nn{#1}{
6816           \bool_set_false:N \l_stex_brackets_dones_bool
6817           #5
6818         }
6819       }
6820     }
6821     \l__stex_notations_cs
6822   }
6823   \tl_set:Nn \l__stex_notations_after_tl{
6824     \exp_args:NNo
6825     \tl_put_left:Nn \l__stex_notations_code_tl \l__stex_notations_pre_tl
6826     \tl_put_left:Ne \l__stex_notations_code_tl {
6827       \int_set:Nn \l_tmpa_int {\int_use:N \l_tmpa_int}
6828     }
6829     #6
6830   }
6831   \__stex_notations_parse_args:nnnnw #4 \__stex_notations_args_end:
6832 }
6833
6834 \cs_new_protected:Npn \__stex_notations_parse_args:nnnnw #1 #2 #3 #4 #5 \__stex_notations_a
6835   \tl_if_empty:nTF{#5}{
6836     \__stex_notations_add_last:nnnnn{#1}{#2}{#3}{#4}
6837   }{
6838     \__stex_notations_add_next:nnnnnn{#1}{#2}{#3}{#4}{#5}
6839   }
6840 }
6841
6842 \cs_new_protected:Nn \__stex_notations_add_next:nnnnnn {
6843   \__stex_notations_add:nnnnn{#1}{#2}{#3}{#4}{#6}
6844   \__stex_notations_parse_args:nnnnw #5 \__stex_notations_args_end:
6845 }
6846
6847 \cs_new_protected:Nn \__stex_notations_add_last:nnnnn {
6848   \__stex_notations_add:nnnnn{#1}{#2}{#3}{#4}{#5}
6849   \stex_debug:nn{sequences}{Executing~notation:~\meaning\l__stex_notations_code_tl}
6850   \l__stex_notations_after_tl
6851 }
6852
6853 \cs_new_protected:Nn \__stex_notations_add:nnnnn {
6854   \int_incr:N \l_tmpa_int
6855   \str_case:nn{#2}{
6856     i {
6857       \tl_put_right:Nn \l__stex_notations_code_tl {
6858         {\stex_term_arg:nnnnn{#1}{#2}{#3}{#4}{#5}}
6859       }

```



```

6860 }
6861 b {
6862   \tl_set:Nn \l__stex_notations_pre_tl {
6863     \cs_set_eq:NN \_stex_term_oma_or_omb:nn \_stex_term_omb:nn
6864   }
6865   \tl_put_right:Nn \l__stex_notations_code_tl {
6866     {\_stex_term_arg:nnnnn{#1}{#2}{#3}{#4}{#5}}
6867   }
6868 }
6869 a {
6870   \tl_put_right:Nn \l__stex_notations_code_tl {
6871     {\_stex_term_arg_aB:nnnnn{#1}{#2}{#3}{#4}{#5}}
6872   }
6873 }
6874 B {
6875   \tl_set:Nn \l__stex_notations_pre_tl {
6876     \cs_set_eq:NN \_stex_term_oma_or_omb:nn \_stex_term_omb:nn
6877   }
6878   \tl_put_right:Nn \l__stex_notations_code_tl {
6879     {\_stex_term_arg_aB:nnnnn{#1}{#2}{#3}{#4}{#5}}
6880   }
6881 }
6882 }
6883 }

```

(End of definition for \STEXInternalNotation. This function is documented on page 154.)

sfunction

\argsep

```

6884 \cs_new_protected:Npn \argsep #1 #2 {
6885   \__stex_notations_check_aB_arg:Nn\argsep{#1}
6886   \stex_pseudogroup_with:nn{\_stex_term_do_aB_clist:}{
6887     \tl_set:Nn \_stex_term_do_aB_clist: {
6888       \seq_use:Nn \l_stex_aB_args_seq {#2}
6889     }
6890     #1
6891   }
6892 }
6893
6894 \cs_new_protected:Nn \__stex_notations_check_aB_arg:Nn {
6895   \exp_args:Ne \cs_if_eq:NNF {\tl_head:n{#2}}
6896   \_stex_term_arg_aB:nnnnn {
6897     \msg_error:nnx{stex}{error/assocarg}{\tl_to_str:n{#1}}
6898   }
6899 }

```

(End of definition for \argsep. This function is documented on page 154.)

sfunction

\argmap

```

6900 \cs_new_protected:Npn \argmap #1 #2 #3 {
6901   \__stex_notations_check_aB_arg:Nn\argmap{#1}
6902   \stex_pseudogroup_with:nn{
6903     \_stex_term_do_aB_clist:

```

```

6904 \_stex_notations_map_cs:
6905 }{
6906 \cs_set:Npn \_stex_notations_map_cs: ##1 { #2 }
6907 \tl_set:Nn \_stex_term_do_aB_clist: {
6908 \seq_clear:N \l_tmpa_seq
6909 \seq_map_inline:Nn \l_stex_aB_args_seq {
6910 \tl_if_eq:nnTF{##1}{\ellipses}{
6911 \seq_put_right:Nn \l_tmpa_seq \ellipses
6912 }{
6913 \seq_put_right:Nx \l_tmpa_seq {
6914 \exp_after:wN \exp_not:n \exp_after:wN { \_stex_notations_map_cs: {##1} }
6915 }
6916 }
6917 }
6918 \seq_set_eq:NN \l_stex_aB_args_seq \l_tmpa_seq
6919 \seq_use:Nn \l_stex_aB_args_seq {#3}
6920 }
6921 #1
6922 }
6923 }

```

(End of definition for \argmap. This function is documented on page 154.)

sfunction

\argarraymap

```

6924 \int_new:N \l_stex_notations_clist_count_int
6925 \cs_new_protected:Npn \argarraymap #1 #2 #3 #4 {
6926 \_stex_notations_check_aB_arg:Nn\argarraymap{#1}
6927 \stex_pseudogroup_with:nn{
6928 \_stex_term_do_aB_clist:
6929 \_stex_notations_map_cs:
6930 }{
6931 \cs_set:Npn \_stex_notations_map_cs: ##1 { #3 }
6932 \int_set:Nn \l_stex_notations_clist_count_int {\exp_args:No\clist_count:n{\tl_to_str:n{
6933 \tl_set:Nn \_stex_term_do_aB_clist: {
6934 \tl_clear:N \l_tmpa_tl
6935 \int_zero:N \l_tmpa_int
6936 \seq_map_inline:Nn \l_stex_aB_args_seq {
6937 \int_incr:N \l_tmpa_int
6938 \int_compare:nNnT \l_tmpa_int > \l_stex_notations_clist_count_int {
6939 \int_set:Nn \l_tmpa_int 1
6940 }
6941 \tl_put_right:Ne \l_tmpa_tl {
6942 \exp_after:wN \exp_not:n \exp_after:wN { \_stex_notations_map_cs: {##1} }
6943 \clist_item:nn{#4}\l_tmpa_int
6944 }
6945 }
6946 \seq_set_eq:NN \l_stex_aB_args_seq \l_tmpa_seq
6947 \begin{array}{#2}
6948 \l_tmpa_tl
6949 \end{array}
6950 }
6951 #1
6952 }
6953 }

```

(End of definition for `\argarraymap`. This function is documented on page 154.)
sfunction

33.1 Automated Bracketing

Variables for current (downwards) precedence, set of parentheses etc.:

```

6954 \int_new:N \l_stex_notation_downprec
6955 \tl_set:Nn \l__stex_notations_left_bracket_str (
6956 \tl_set:Nn \l__stex_notations_right_bracket_str )
6957 \bool_new:N \l_stex_brackets_dones_bool

\infprec
\neginfprec
6958 \tl_const:Ne \infprec {\int_use:N \c_max_int}
6959 \tl_const:Ne \neginfprec {-\int_use:N \c_max_int}
6960
6961 \int_set:Nn \l_stex_notation_downprec \infprec

```

(End of definition for `\infprec` and `\neginfprec`. These variables are documented on page 154.)

`\_stex_maybe_brackets:nn`

```

6962 \cs_new_protected:Nn \_stex_maybe_brackets:nn {
6963   \bool_if:NTF \l_stex_brackets_dones_bool {
6964     \bool_set_false:N \l_stex_brackets_dones_bool
6965     #2
6966   } {
6967     \stex_debug:nn{brackets}{#1>\int_eval:n \l_stex_notation_downprec?}
6968     \int_compare:nNnTF { #1 } > \l_stex_notation_downprec {
6969       %\bool_if:NTF \l_stex_inarray_bool { #2 }{
6970         \dobrackets {
6971           #2
6972         }
6973         %}
6974       }{
6975         #2
6976       }
6977     }
6978   }

```

(End of definition for `\_stex_maybe_brackets:nn`. This function is documented on page 154.)
sfunction

`\dobrackets`

```

6979 \cs_new_protected:Npn \dobrackets #1 {
6980   \group_begin:
6981     \bool_set_true:N \l_stex_brackets_dones_bool
6982     \mathopen{\tl_if_exist:NT\l_stex_current_symbol_uri\comp
6983       \l__stex_notations_left_bracket_str
6984     }
6985     #1
6986   \group_end:
6987   \mathclose{\tl_if_exist:NT\l_stex_current_symbol_uri\comp
6988     \l__stex_notations_right_bracket_str
6989   }
6990 }

```

(End of definition for \dobrackets. This function is documented on page 155.)

sfunction

\withbrackets

```
6991 \cs_new_protected:Npn \withbrackets #1 #2 #3 {  
6992   \stex_pseudogroup:nn{  
6993     \tl_set:Nn \l__stex_notations_left_bracket_str { #1 }  
6994     \tl_set:Nn \l__stex_notations_right_bracket_str { #2 }  
6995     #3  
6996   }{  
6997     \stex_pseudogroup_restore:N \l__stex_notations_left_bracket_str  
6998     \stex_pseudogroup_restore:N \l__stex_notations_right_bracket_str  
6999   }  
7000 }
```

(End of definition for \withbrackets. This function is documented on page 155.)

sfunction

\dowithbrackets

```
7001 \cs_new_protected:Npn \dowithbrackets #1 #2 #3 {  
7002   \withbrackets{#1}{#2}{\dobrackets{#3}}  
7003 }
```

(End of definition for \dowithbrackets. This function is documented on page 155.)

sfunction

Chapter 34

Structures

```
7004 <@@=stex_structures>

      Keys:
7005 \stex_keys_define:nnnn{mathstructure}{
7006   \tl_clear:N \l_stex_current_this_tl
7007   \str_clear:N \l__stex_structures_name_str
7008 }{
7009   this .tl_set:N = \l_stex_current_this_tl ,
7010   unknown .code:n = {
7011     \str_if_empty:NTF \l_keys_key_str {
7012       \str_set:Ne \l__stex_structures_name_str {\l_keys_key_tl}
7013     }{
7014       \str_set_eq:NN \l__stex_structures_name_str \l_keys_key_str
7015     }
7016   }
7017 }{}
```

`mathstructure (env.)`

```
7018 \stex_new_stylable_env:nnnnnnn {mathstructure}{m 0{}}{
7019   \__stex_structures_begin:nn{#1}{#2}
7020   \stex_smsmode_do:
7021 }{
7022   \stex_structural_feature_module_end:
7023   \__stex_structures_do externals:
7024 }{}{}{}
7025
7026 \stex_sms_allow_env:n{mathstructure}
7027 \stex_deactivate_macro:Nn \mathstructure {module~environments}
7028 \stex_every_module:n {\stex_reactivate_macro:N \mathstructure}
```

Shared code for `mathstructure` and `extstructure`:

```
7029 \cs_new_protected:Nn \__stex_structures_begin:nn {
7030   \stex_keys_set:nn {mathstructure}{#2}
7031   \str_if_empty:NT \l__stex_structures_name_str {
7032     \str_set:Nn \l__stex_structures_name_str {#1}
7033   }
7034   \def\comp{\_comp}
7035
7036   \tl_set:Ne \l__stex_structures_struct_uri {
```

```

7037 \exp_args:No \stex_new_module_uri:n \l__stex_structures_name_str
7038 }
7039
7040 \exp_args:Nne \use:nn { \stex_add_symbol:nnnnnnN }
7041 { {#1}{\l__stex_structures_name_str}{0}{\defed}{\l__stex_structures_struct_uri}}
7042 {} \stex_invoke_structure:
7043 \str_set:Ne \l_stex_macroname_str {#1}
7044
7045 \exp_args:No \stex_structural_feature_module:nn
7046 {\l__stex_structures_name_str}{structure}
7047 }
7048
7049 \cs_new_protected:Nn \__stex_structures_doexternals: {
7050 \tl_set:Nn \l__stex_structures_replace_this_tl {####1}
7051 }
7052

```

extstructure (env.)

```

7053 \stex_new_stylable_env:nnnnnn {extstructure}{m O{} m}{
7054 \seq_clear:N \l__stex_structures_imports_seq
7055 \clist_map_inline:nn{#3}{
7056 \stex_get_mathstructure:n{##1}
7057 \clist_map_inline:Nn \l_stex_get_symbol_type_tl {
7058 \stex_if_module_exists:nT{####1}{
7059 \seq_put_right:Nn \l__stex_structures_imports_seq{####1}
7060 }
7061 }
7062 }
7063 \__stex_structures_begin:nn{#1}{#2}
7064 \seq_map_inline:Nn \l__stex_structures_imports_seq{
7065 \stex_if_do_html:T {
7066 \hbox{\stex_annotate_invisible:nn
7067 {data-ftml-import={\stex_use_module_uri:n{##1}}}{}}
7068 }
7069 \stex_add_morphism:nonn
7070 {}{##1}{import}{}
7071 \stex_execute_in_module:e{
7072 \stex_activate_module:n {##1}
7073 }
7074 }
7075 \stex_smsmode_do:
7076 }{
7077 \stex_structural_feature_module_end:
7078 \__stex_structures_doexternals:
7079 }{}{}
7080
7081 \stex_sms_allow_env:n{extstructure}
7082 \stex_deactivate_macro:Nn \extstructure {module~environments}
7083 \stex_every_module:n {
7084 \stex_reactivate_macro:N \extstructure
7085 }
7086
7087 \stex_new_stylable_env:nnnnnn {extstructure*}{m}{
7088 \__stex_structures_new_extstruct_name:

```

```

7089 \seq_clear:N \l__stex_structures_imports_seq
7090 \stex_get_mathstructure:n{#1}
7091
7092 \clist_map_inline:Nn \l_stex_get_symbol_type_tl {
7093   \stex_if_module_exists:nT{##1}{
7094     \seq_put_right:Nn \l__stex_structures_imports_seq{##1}
7095   }
7096 }
7097
7098 \stex_module_uri_split_name:NNN \l__stex_structures_parent_uri \l__stex_structures_last_n
7099
7100 \stex_execute_in_module:e{
7101   \__stex_structures_extend_structure:nnnn{\l__stex_structures_parent_uri}{\l__stex_struct
7102 }
7103
7104 \exp_args:No \__stex_structures_begin:nn\l__stex_structures_exstruct_name_str{
7105
7106 \seq_map_inline:Nn \l__stex_structures_imports_seq {
7107   \stex_if_do_html:T {
7108     \stex_annotate_invisible:nn
7109     {data-ftml-import= {\stex_use_module_uri:n{##1}}}{ }
7110   }
7111   %\stex_add_morphism:nonn
7112   % {}{##1}{import}{ }
7113   %\stex_execute_in_module:e{
7114   % \stex_activate_module:n {##1}
7115   %}
7116   \stex_activate_module:n {##1}
7117 }
7118
7119 \stex_smsmode_do:
7120 }{
7121 \prop_map_inline:cn{\_stex_symbol_macro:N \l_stex_current_module_uri }{
7122   \__stex_structures_check_def:nnnnnnn ##2
7123 }
7124 \stex_structural_feature_module_end:
7125 %\exp_args:Ne \stex_do_up_to_module:n {
7126 % \stex_activate_module:n {\exp_args:No \stex_new_module_uri:n {\l__stex_structures_exst
7127 %}
7128 \__stex_structures_do externals:
7129 }{}{}{}
7130
7131 \stex_sms_allow_env:n{extstructure*}
7132 \exp_after:wN \stex_deactivate_macro:Nn
7133 \cs:w extstructure*\cs_end: {module~environments}
7134 \stex_every_module:n {
7135   \exp_after:wN \stex_reactivate_macro:N \cs:w extstructure*\cs_end:
7136 }
7137
7138 \cs_new_protected:Nn \__stex_structures_extend_structure:nnnn {
7139   \stex_debug:nn{ext}{Extending~\stex_use_module_uri:n {#1} / #2-by~\stex_use_module_uri:n
7140   \exp_args:Nne \tl_put_right:cn{\_stex_module_code_macro:n{#4}}{
7141     \stex_activate_module:n{#3}
7142   }

```

```

7143 \exp_args:Nne \prop_put:cnn{\_stex_morphisms_macro:n{#4}}{\stex_use_module_uri:n{#3}}{
7144   {}{#3}{extstruct}}{
7145 }
7146 \exp_args:Nne \use:nn {\_stex_structures_extend_ii:nnnn}{
7147   \exp_args:Nne \use:nn \_stex_structures_extend_i:nnnnnnnnNn {
7148     \prop_item:cn{\_stex_symbol_macro:n {#1} } { #2 }
7149     } { #3 }
7150   }{#1}{#2}
7151 }
7152
7153 \cs_new_protected:Nn \_stex_structures_extend_ii:nnnn {
7154   \prop_put:cnn{\_stex_symbol_macro:n {#3} } { #4 }{#2}
7155   \tl_set:ce{#1}{
7156     \stex_invoke_symbol:nnnnnnN {\stex_new_symbol_uri:nn {#3}{#4}}
7157     \use_none:nn #2
7158   }
7159 }
7160
7161 \cs_new:Nn \_stex_structures_extend_i:nnnnnnnnNn {
7162   {#1}{#{#1}{#2}{#3}{#4}{#5}{#6,#9}{#7}#8}
7163 }
7164
7165 \cs_new_protected:Nn \_stex_structures_check_def:nnnnnnnn {
7166   \tl_if_empty:nT{#5}{
7167     \msg_error:nnnn{stex}{error/needsdefinien}{#2}{extstructure*}
7168   }
7169 }
7170
7171 \cs_new_protected:Nn \_stex_structures_new_extstruct_name: {
7172   \stex_do_up_to_module:n {
7173     \str_set:Ne \l__stex_structures_extname_count {\int_eval:n{\l__stex_structures_extname_
7174   }
7175   \str_set:Ne \l__stex_structures_exstruct_name_str {EXTSTRUCT_\l__stex_structures_extname_
7176 }
7177
7178 \stex_every_module:n{ \str_set:Nn \l__stex_structures_extname_count 0}

```

\this

```

7179 \bool_new:N \l_stex_in_this_bool
7180
7181 \cs_new_protected:Npn \stex_current_this: {
7182   { \bool_set_true:N \l_stex_allow_semantic_bool
7183     \tl_if_empty:NTF \l_stex_current_this_tl {}{}{
7184       \tl_set:Nn \l_stex_current_full_tl {
7185         \exp_args:Ne \stex_use_symbol_uri:n {
7186           \exp_args:No \stex_new_symbol_uri:n \l__stex_structures_name_str
7187         }
7188       }
7189       \str_set_eq:NN \l_stex_current_display_tl \l__stex_structures_name_str
7190       \bool_set_true:N \l_stex_in_this_bool
7191       \maincomp{\l_stex_current_this_tl}
7192       \bool_set_false:N \l_stex_in_this_bool
7193     }
7194   }

```



```

7195 }
7196
7197 \let \this \stex_current_this:

```

(End of definition for \this. This function is documented on page 156.)

```

sfunction

```

\stex_get_mathstructure:n

```

7198 \cs_new_protected:Nn \stex_get_mathstructure:n {
7199   \stex_get_mathstructure_maybe:nTF{#1}{#{
7200     \msg_error:nnn{stex}{error/unknownstructure}{#1}
7201   }}
7202 }
7203 \cs_new_protected:Npn \stex_get_mathstructure_maybe:nTF #1 #2 #3 {
7204   \tl_clear:N \l_stex_get_structure_module_uri
7205   \stex_get_symbol:nTF{#1}{
7206     \exp_args:No \tl_if_eq:NNTF \l_stex_get_symbol_invoke_cs \stex_invoke_structure: {
7207       \tl_set:Ne \l_stex_get_structure_module_uri { \exp_after:wN \__stex_structures_get:w
7208         #2
7209       }{
7210         #3
7211       }
7212     }{
7213       #3
7214     }
7215   }
7216
7217   \cs_new:Npn \__stex_structures_get:w #1 , #2 \stex_end: { #1 }

```

(End of definition for \stex_get_mathstructure:n. This function is documented on page 156.)

```

sfunction

```

\usestructure

```

7218 \cs_new_protected:Npn \usestructure #1 {
7219   \stex_if_smsmode:TF{
7220     \stex_get_mathstructure_maybe:nTF{ #1 }{
7221       \stex_debug:nn{usestructure}{Using~structure~#1:~\stex_use_module_uri:N \l_stex_get_str
7222       \__stex_structures_do_usestructure:
7223     }{
7224       \stex_debug:nn{usestructure}{Structure~#1~not~found~in~sms~mode}
7225     }
7226   }{
7227     \stex_get_mathstructure:n{ #1 }
7228     \stex_debug:nn{usestructure}{Using~structure~#1:~\stex_use_module_uri:N \l_stex_get_str
7229     \__stex_structures_do_usestructure:
7230   }
7231   \stex_smsmode_do:
7232 }
7233 \stex_sms_allow_escape:N \usestructure
7234
7235 \cs_new_protected:Nn \__stex_structures_do_usestructure: {
7236   \seq_clear:N \l__stex_structures_imports_seq
7237   \clist_map_inline:Nn \l_stex_get_symbol_type_tl {
7238     \stex_if_module_exists:nT{##1}{

```

```

7239     \seq_put_right:Nn \l__stex_structures_imports_seq{##1}
7240   }
7241 }
7242 \seq_map_inline:Nn \l__stex_structures_imports_seq {
7243   \stex_if_do_html:T {
7244     \hbox{\stex_annotate_invisible:nn
7245       {data-ftml-usemodule=\stex_use_module_uri:n {##1}} {}}
7246   }
7247   \stex_activate_module:n {##1}
7248 }
7249 }

```

(End of definition for \usestructure. This function is documented on page 156.)
 sfunction

Chapter 35

Statements

```
7250 <@@=stex_statements>
```

Keys:

```
7251 \stex_keys_define:nnnn{statement}{  
7252   \str_clear:N \l_stex_key_name_str  
7253   \str_clear:N \l_stex_key_macroname_str  
7254   \clist_clear:N \l_stex_key_for_clist  
7255   \str_clear:N \l_stex_key_args_str  
7256   \tl_clear:N \l_stex_key_type_tl  
7257   \tl_clear:N \l_stex_key_def_tl  
7258   \tl_clear:N \l_stex_key_return_tl  
7259   \clist_clear:N \l_stex_key_argtypes_clist  
7260 }{  
7261   name      .str_set:N = \l_stex_key_name_str ,  
7262   for       .clist_set:N = \l_stex_key_for_clist ,  
7263   macro     .str_set:N = \l_stex_key_macroname_str ,  
7264   % start   .str_set:N = \l_stex_key_title_str , % TODO remove  
7265   type      .tl_set:N = \l_stex_key_type_tl ,  
7266   judgment  .code:n = {},  
7267   from      .code:n= {}, % TODO remove  
7268   to        .code:n={} % TODO remove  
7269 }{id,title,style,symargs}
```

`\stex_do_for_list:`

```
7270 \cs_new_protected:Npn \stex_do_for_list: {  
7271   \seq_clear:N \l_stex_fors_seq  
7272   \clist_map_inline:Nn \l_stex_key_for_clist {  
7273     \exp_args:Ne\stex_get_symbol:n{\tl_to_str:n{##1}}  
7274     \seq_put_right:No \l_stex_fors_seq  
7275     {\l_stex_get_symbol_uri}  
7276   }  
7277 }
```

(End of definition for \stex_do_for_list:. This function is documented on page 157.)

sfunction

`\stex_new_statement:nnn`

```
7278 \cs_new_protected:Nn \stex_new_statement:nnn {  
7279   \stex_new_stylable_env:nnnnnnn {#1}{0}}{
```

```

7280 \stex_keys_set:nn{statement}{##1}
7281 #3
7282
7283 \stex_if_smsmode:F {
7284   \exp_args:Nne \begin{stex_annotate_env}{
7285     \__stex_statements_html_keyvals:nn{#1}{false}
7286   }
7287   \tl_set_eq:NN \thistitle \l_stex_key_title_tl
7288   \str_set_eq:NN \thisname \l_stex_key_name_str
7289   \clist_set_eq:NN \thisfor \l_stex_key_for_str
7290   \stex_if_html_backend:TF {
7291     \noindent
7292     \stex_annotate:nn{data-ftml-title={}, style:display={none}}{\_stex_annotate_force_b
7293     \tl_if_empty:NF \l_stex_key_title_tl{~}
7294     \l_stex_key_title_tl
7295   }}
7296   }
7297   \stex_style_apply:
7298 }
7299 \_stex_do_id:
7300 \stex_smsmode_do:
7301 }{
7302   \stex_if_smsmode:F {
7303     \stex_if_html_backend:F \stex_style_apply:
7304     \end{stex_annotate_env}
7305   }
7306   }{}{}{s}
7307   \stex_sms_allow_env:n{s#1}
7308
7309   \tl_if_empty:NF{#2}{\__stex_statements_make_macro:nnn{#1}{#2}{#3}}
7310 }
7311
7312 \cs_new_protected:Nn \__stex_statements_make_macro:nnn {
7313   \exp_after:wN \NewDocumentCommand \cs:w inline#2\cs_end: { 0{} m}{
7314     \group_begin:
7315     \stex_keys_set:nn{statement}{##1}
7316     #3
7317     \_stex_do_id:
7318     \stex_if_smsmode:F{
7319       \exp_args:Ne \stex_annotate:nn{\__stex_statements_html_keyvals:nn{#1}{true}}{
7320         \_stex_annotate_force_break:n{##2}
7321       }
7322     }
7323     \group_end:
7324     %\stex_if_smsmode:TF \stex_smsmode_do: \stex_ignore_spaces_and_pars:
7325     \stex_smsmode_do:
7326   }
7327   \exp_after:wN \stex_sms_allow_escape:N\cs:w inline#2\cs_end:
7328 }
7329
7330 \cs_new:Nn \__stex_statements_html_keyvals:nn {
7331   data-ftml-#1={},
7332   data-ftml-inline={#2},
7333   \seq_if_empty:NF \l_stex_fors_seq {,

```

```

7334     data-ftml-fors={\seq_map_indexed_function:Nn \l_stex_fors_seq \_stex_comma_sep_uri:nn }
7335   }
7336   \str_if_empty:NF \l_stex_key_id_str {,
7337     data-ftml-id={\l_stex_key_id_str}%{\stex_uri_use:N \l_stex_current_doc_uri ? \l_stex_ke
7338   }
7339   \clist_if_empty:NF \l_stex_key_style_clist {,
7340     data-ftml-styles={\l_stex_key_style_clist}
7341   }
7342 }
7343
7344 \cs_new:Nn \_stex_comma_sep_uri:nn {
7345   \int_compare:nNnF{#1}=1 {,}
7346   \stex_use_symbol_uri:n{#2}
7347 }

```

(End of definition for `\stex_new_statement:nnn`. This function is documented on page 157.)

sfunction

`\_stex_statements_setup:n` sets up a statement that may declare a new symbol with the given `role` by processing the optional arguments, calling `\stex_symdecl_do:`, etc.

```

7348 \cs_new_protected:Nn \_stex_statements_setup:n {
7349   \str_if_empty:NF \l_stex_key_macroname_str {
7350     \str_if_empty:NT \l_stex_key_name_str {
7351       \str_set_eq:NN \l_stex_key_name_str \l_stex_key_macroname_str
7352     }
7353   }
7354   \_stex_do_for_list:
7355   \tl_clear:N \l_stex_current_def_uri
7356
7357   \str_if_empty:NTF \l_stex_key_name_str \_stex_statements_setup_noname: {
7358     \_stex_statements_setup_named:n{#1}
7359   }
7360 }
7361
7362 \cs_new_protected:Nn \_stex_statements_setup_named:n {
7363   \_stex_statements_force_id:
7364   \seq_put_right:Ne \l_stex_fors_seq {
7365     \l_stex_current_module_uri {\l_stex_key_name_str}
7366   }
7367   \str_set_eq:NN \l_stex_macroname_str \l_stex_key_macroname_str
7368   \str_set:Nn \l_stex_key_role_str {#1}
7369   \stex_symdecl_do:
7370   \exp_args:Nne \use:nn {\stex_add_symbol:nnnnnnN}{
7371     {\l_stex_key_macroname_str}{\l_stex_key_name_str}
7372     {\int_use:N \l_stex_get_symbol_arity_int}
7373     {\l_stex_get_symbol_args_tl}
7374     {#1}{}}{\stex_invoke_symbol:
7375   }
7376   \stex_if_do_html:T \_stex_symdecl_html:
7377   \tl_set:Ne \l_stex_current_def_uri {\exp_args:No \stex_new_symbol_uri:nn \l_stex_current_
7378   \stex_debug:nn{statement}{name:~\stex_use_symbol_uri:N \l_stex_current_def_uri}
7379 }
7380
7381 \cs_new_protected:Nn \_stex_statements_setup_noname: {

```

```

7382 \stex_debug:nn{statement}{no~name}
7383 \int_compare:nNnTF {\seq_count:N \l_stex_fors_seq} = 1 {
7384   \tl_set:Ne \l_stex_current_def_uri {\seq_item:Nn \l_stex_fors_seq 1}
7385   \stex_debug:nn{statement}{for:~\stex_use_symbol_uri:N \l_stex_current_def_uri}
7386 }{
7387   \stex_debug:nn{statement}{no~for}
7388 }
7389 }
7390
7391 \cs_new_protected:Nn \__stex_statements_force_id: {
7392   %\str_if_empty:NT \l_stex_key_id_str {
7393   %   \stex_ref_new_id:n{ }
7394   %   \str_set_eq:NN \l_stex_key_id_str \l__stex_refs_str
7395   %}
7396 }

```

(End of definition for __stex_statements_setup:n.)

__stex_statements_setup_def: Sets up all the macros allowed in definition-like environments:

```

7397 \cs_new_protected:Nn \__stex_statements_setup_def: {
7398   \stex_if_smsmode:F{
7399     \seq_map_inline:Nn \l_stex_fors_seq {
7400       \exp_args:Ne \stex_ref_new_sym_target:n{\stex_use_symbol_uri:n{##1}}
7401     }
7402   }
7403   \stex_reactivate_macro:N \definiendum
7404   \stex_reactivate_macro:N \defnotation
7405   \stex_reactivate_macro:N \definame
7406   \stex_reactivate_macro:N \Definame
7407   \stex_reactivate_macro:N \varbind
7408   \stex_reactivate_macro:N \definiens
7409 }

```

(End of definition for __stex_statements_setup_def:.)

sdefinition (env.)

```

7410 \stex_new_statement:nnn{definition}{def}{
7411   \__stex_statements_force_id:
7412   \__stex_statements_setup:n{ }
7413   \__stex_statements_setup_def:
7414 }

```

sassertion (env.)

```

7415 \stex_new_statement:nnn{assertion}{ass}{
7416   \__stex_statements_setup:n{assertion}
7417   \stex_if_smsmode:F{
7418     \seq_map_inline:Nn \l_stex_fors_seq {
7419       \exp_args:Ne \stex_ref_new_sym_target:n{\stex_use_symbol_uri:n{##1}}
7420     }
7421   }
7422   \stex_reactivate_macro:N \varbind
7423   \stex_reactivate_macro:N \conclusion
7424   \stex_reactivate_macro:N \premise
7425   \stex_reactivate_macro:N \definiendum

```

```

7426 \stex_reactivate_macro:N \defnotation
7427 \stex_reactivate_macro:N \definame
7428 \stex_reactivate_macro:N \Definame
7429 }

```

sexample (*env.*)

```

7430 \stex_new_statement:nnn{example}{ex}{
7431 \stex_if_smsmode:F {\_stex_statements_setup:n{example}}
7432 }

```

sparagraph (*env.*)

```

7433 \stex_new_statement:nnn{paragraph}{}{
7434 \clist_if_in:NnTF \l_stex_key_style_clist {symdoc}{
7435 \_stex_statements_force_id:
7436 \_stex_statements_setup:n{
7437 \_stex_statements_setup_def:
7438 }{
7439 \_stex_statements_setup:n{
7440 }
7441 }

```

definiens

```

7442 \NewDocumentCommand \definiens { 0{} m }{
7443 \group_begin:
7444 \_stex_definiens_impl:nn{#1}{#2}
7445 \group_end:
7446 \stex_smsmode_do:
7447 }
7448
7449 \cs_new_protected:Nn \_stex_definiens_impl:nn {
7450 \tl_if_empty:nF {#1} {
7451 \stex_get_symbol:n { #1 }
7452 \tl_set_eq:NN \l_stex_current_def_uri \l_stex_get_symbol_uri
7453 }
7454 \tl_if_empty:NT \l_stex_current_def_uri {
7455 \msg_error:nn{stex}{error/definiensfor}
7456 }
7457 \stex_debug:nn{definiens}{Checking-\stex_use_symbol_uri:N \l_stex_current_def_uri }
7458
7459 \exp_args:No \_stex_add_definiens:nn \l_stex_current_def_uri {#2}
7460 }
7461
7462 \stex_deactivate_macro:Nn \definiens {definition~environments}
7463 \stex_sms_allow_escape:N \definiens

```

(End of definition for definiens. This function is documented on page 157.)

sfunction

_stex_add_definiens:nn

```

7464 \cs_new_protected:Nn \_stex_add_definiens:nn {
7465 \stex_if_do_html:TF {
7466 \stex_annotate:nn{data-ftml-definiens={\stex_use_symbol_uri:n{#1}}}{
7467 #2 %\_stex_annotate_force_break:n{ #2 }
7468 }

```

```

7469   }{ \stex_if_smsmode:F{#2} }
7470   \stex_if_in_module:T {
7471     \exp_args:Ne \str_if_eq:noT {\stex_symbol_uri_module:n{#1}}\l_stex_current_module_uri{
7472       \__stex_statements_add_definiens:n{#1}
7473     }
7474   }
7475 }
7476
7477 \cs_new_protected:Nn \__stex_statements_add_definiens:n {
7478   \stex_debug:nn{definiens}{Adding~definiens~to~\stex_use_symbol_uri:n{#1}}
7479   \exp_args:Nne \prop_gput:cne{\exp_args:Ne \stex_symbol_macro:n {\stex_symbol_uri_module:
7480     {\stex_symbol_uri_name:n {#1}} {
7481       \exp_args:Nne \use:nn { \__stex_statements_definiens_inner:nnnnnnN }{ \exp_args:Nne
7482     }
7483   }
7484
7485   \cs_new:Nn \__stex_statements_definiens_inner:nnnnnnN {
7486     {#1}{#2}{#3}{#4}{defed}{#6}{#7}{#8}
7487   }

```

(End of definition for `\stex_add_definiens:nn`. This function is documented on page 157.)

sfunction

`\varbind`

```

7488 \NewDocumentCommand \varbind {m} {
7489   \clist_map_inline:nn {#1} {
7490     \stex_get_var:n {##1}
7491     \stex_if_do_html:T {
7492       \stex_annotate_invisible:nn {data-ftml-bind=\l_stex_get_variable_str}{ }
7493     }
7494   }
7495 }
7496 \stex_deactivate_macro:Nn \varbind {definition~or~assertion~environments}

```

(End of definition for `\varbind`. This function is documented on page 157.)

sfunction

`\conclusion`

```

7497 \NewDocumentCommand \conclusion { 0{ } m } {
7498   \group_begin:
7499   \tl_if_empty:nF {#1} {
7500     \stex_get_symbol:n { #1 }
7501     \tl_set_eq:NN \l_stex_current_def_uri \l_stex_get_symbol_uri
7502   }
7503   \tl_if_empty:NT \l_stex_current_def_uri {
7504     \msg_error:nn{stex}{error/conclusionfor}
7505   }
7506   \stex_annotate:nn{ data-ftml-conclusion={\stex_use_symbol_uri:N \l_stex_current_def_uri}}
7507   #2 %\stex_annotate_force_break:n{ #2 }
7508 }
7509 \group_end:
7510 }
7511 \stex_deactivate_macro:Nn \conclusion {assertion~environments}

```


(End of definition for `\conclusion`. This function is documented on page 157.)
`sfunction`

`\premise`

```

7512 \NewDocumentCommand \premise {0{} m} {
7513   \tl_if_empty:nF {#1} {
7514     \stex_debug:nn{Here:}{Variable~#1}
7515     \exp_args:Nne\use:nn{\vardef}{v#1}[name=#1]{#1}
7516   }
7517   \stex_annotate:nn{data-ftml-premise={#1}}{#2}
7518 }
7519 \stex_deactivate_macro:Nn \premise {assertion~environments}

```

(End of definition for `\premise`. This function is documented on page 157.)
`sfunction`

Chapter 36

Proofs

```
7520 <@@=stex_proof>
      Keys:
7521
7522 \stex_keys_define:nnnn{ spfcommon }{
7523   \tl_clear:N \l_stex_key_for_clist
7524   \tl_clear:N \l_stex_key_term_tl
7525   \tl_clear:N \l_stex_key_method_tl
7526   \tl_clear:N \l_stex_key_just_tl
7527 }{
7528   for      .clist_set:N = \l_stex_key_for_clist ,
7529   term     .tl_set:N    = \l_stex_key_term_tl,
7530   method   .tl_set:N    = \l_stex_key_method_tl,
7531   just     .tl_set:N    = \l_stex_key_just_tl,
7532 }{id,style,title}
7533
7534 \bool_set_false:N \l_stex_key_showname_bool
7535
7536 \stex_keys_define:nnnn{ spfnamed }{
7537   \str_clear:N \l_stex_key_name_str
7538   \str_clear:N \l_stex_key_numname_str
7539   \bool_set_false:N \l_stex_key_showname_bool
7540 }{
7541   name      .str_set_x:N = \l_stex_key_name_str,
7542   numname   .str_set_x:N = \l_stex_key_numname_str,
7543   showname  .bool_set:N  = \l_stex_key_showname_bool
7544 }{spfcommon}
7545
7546 \cs_new_protected:Nn \__stex_proof_do_spf_name: {
7547   \str_if_empty:NTF \l_stex_key_numname_str {
7548     \str_if_empty:NTF \l_stex_key_name_str {
7549       \bool_set_false:N \l_stex_key_showname_bool
7550     } {
7551       \exp_args:Nne\use:nn{\vardef}{\v\l_stex_key_name_str}[name=\l_stex_key_name_str]{\l_s
7552     }
7553   }{
7554     \bool_set_false:N \l_stex_key_showname_bool
7555     \__stex_proof_number_as_string:N \l__stex_proof_counter_str
7556     \exp_args:Nne \use:nn {\vardef}{\{\l_stex_key_numname_str}[name=\l__stex_proof_counter_s
```

```

7557 }
7558 }
7559
7560 \cs_new_protected:Nn \__stex_proof_do_spf_name_i: {
7561   \str_if_empty:NTF \l__stex_proof_key_numname_str {
7562     %\str_if_empty:NF \l__stex_proof_key_name_str {
7563       \exp_args:Nne\use:nn{\vardef}{\{v\l__stex_proof_key_name_str}[name=\l__stex_proof_key_
7564       %}
7565     }{
7566       \exp_args:Nne \use:nn {\vardef}{\{\l__stex_proof_key_numname_str}[name=\l__stex_proof_co
7567     }
7568   }
7569
7570 \cs_new_protected:Nn \__stex_proof_do_spf_varname: {
7571   \bool_if:NT \l_stex_key_showname_bool {
7572     ~($\csname v\l_stex_key_name_str\endcsname$)~
7573   }
7574 }
7575
7576 \stex_keys_define:nnnn{ spftop }{
7577   \tl_set_eq:NN \l_stex_key_proofend_tl \__stex_proof_proof_box_tl
7578   \bool_set_false:N \l_stex_key_hide_bool
7579   \tl_clear:N \l_stex_key_continues_tl
7580   \tl_clear:N \l_stex_key_functions_tl
7581   \tl_clear:N \l_stex_key_from_tl
7582 }{
7583   proofend      .tl_set:N      = \l_stex_key_proofend_tl,
7584   hide          .code:n        = {
7585     \str_if_eq:eeT{#1}{true}{
7586       \bool_set_true:N \l_stex_key_hide_bool
7587     }
7588   },
7589   continues     .tl_set:N      = \l_stex_key_continues_tl, % <- unused
7590   functions     .tl_set:N      = \l_stex_key_functions_tl, % <- unused
7591   from          .tl_set:N      = \l_stex_key_from_tl % <- unused
7592 }{spfcommon}
7593
7594 \stex_keys_define:nnnn{ subproof }{ }{ }{spftop,spfnamed}
7595
7596 \bool_set_true:N \l__stex_proof_inc_counter_bool

```

For proofs, we will have to have deeply nested structures of enumerated list-like environments. However, L^AT_EX only allows `enumerate` environments up to nesting depth 4 and general list environments up to listing depth 6. This is not enough for us. Therefore we have decided to go along the route proposed by Leslie Lamport to use a single top-level list with dotted sequences of numbers to identify the position in the proof tree. Unfortunately, we could not use his `pf.sty` package directly, since it does not do automatic numbering, and we have to add keyword arguments all over the place, to accomodate semantic information.

```

7597 \intarray_new:Nn\l__stex_proof_counter_intarray{50}
7598
7599 \cs_new_protected:Npn \__stex_proof_insert_number: {
7600   \int_set:Nn \l_tmpa_int {1}
7601   \bool_while_do:nn {

```

```

7602     \int_compare_p:nNn {
7603         \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int
7604     } > 0
7605 }{
7606     \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int .
7607     \int_incr:N \l_tmpa_int
7608 }
7609 }
7610 \cs_new_protected:Nn \__stex_proof_number_as_string:N {
7611     \str_clear:N #1
7612     \int_set:Nn \l_tmpa_int {1}
7613     \bool_while_do:nn {
7614         \int_compare_p:nNn {
7615             \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int
7616         } > 0
7617     }{
7618         \str_put_right:Ne #1 {\intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int .}
7619         \int_incr:N \l_tmpa_int
7620     }
7621 }
7622
7623 \cs_new_protected:Npn \__stex_proof_inc_counter: {
7624     \int_set:Nn \l_tmpa_int {1}
7625     \bool_while_do:nn {
7626         \int_compare_p:nNn {
7627             \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int
7628         } > 0
7629     }{
7630         \int_incr:N \l_tmpa_int
7631     }
7632     \int_compare:nNnF \l_tmpa_int = 1 {
7633         \int_decr:N \l_tmpa_int
7634     }
7635     \intarray_gset:Nnn \l__stex_proof_counter_intarray \l_tmpa_int {
7636         \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int + 1
7637     }
7638 }
7639
7640 \cs_new_protected:Npn \__stex_proof_add_counter: {
7641     \int_set:Nn \l_tmpa_int {1}
7642     \bool_while_do:nn {
7643         \int_compare_p:nNn {
7644             \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int
7645         } > 0
7646     }{
7647         \int_incr:N \l_tmpa_int
7648     }
7649     \intarray_gset:Nnn \l__stex_proof_counter_intarray \l_tmpa_int { 1 }
7650 }
7651
7652 \cs_new_protected:Npn \__stex_proof_remove_counter: {
7653     \int_set:Nn \l_tmpa_int {1}
7654     \bool_while_do:nn {
7655         \int_compare_p:nNn {

```

```

7656     \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int
7657   } > 0
7658   ){
7659     \int_incr:N \l_tmpa_int
7660   }
7661   \int_decr:N \l_tmpa_int
7662   \intarray_gset:Nnn \l__stex_proof_counter_intarray \l_tmpa_int { 0 }
7663 }

sproof (env.)

7664 \bool_set_false:N \l__stex_proof_in_spfblock_bool
7665
7666 \cs_new_protected:Nn \__stex_proof_begin_proof:nn {\par
7667   \intarray_gzero:N \l__stex_proof_counter_intarray
7668   \intarray_gset:Nnn \l__stex_proof_counter_intarray 1 1
7669   \stex_keys_set:nn{spf}top}{#1}
7670   \_stex_do_for_list:
7671   \stex_if_do_html:T {
7672     \__stex_proof_html_env_proof:
7673   }
7674   \seq_map_inline:Nn \l_stex_fors_seq {
7675     \stex_debug:nn{definiens}{Adding~definiens~to~##1}
7676     \stex_if_do_html:TF{
7677       \STEXinvisible{\hbox{\_stex_add_definiens:nn {##1}{proven}}}}
7678     }{
7679       \_stex_add_definiens:nn {##1}{\STEXinvisible{proven}}
7680     }
7681   }
7682   \stex_if_do_html:F\stex_style_apply:
7683   %\_stex_do_id:
7684   \stex_reactivate_macro:N \subproof
7685   \stex_reactivate_macro:N \spfstep
7686   \stex_reactivate_macro:N \conclude
7687   \stex_reactivate_macro:N \assumption
7688   \stex_reactivate_macro:N \eqstep
7689   \stex_reactivate_macro:N \yield
7690   \stex_reactivate_macro:N \spfescape
7691   \stex_reactivate_macro:N \spfblock
7692   \stex_reactivate_macro:N \spfjust
7693   \stex_reactivate_macro:N \spfby
7694   \stex_reactivate_macro:N \spfarg
7695   \noindent\stex_annotate:nn{data-ftml-title={}}{\_stex_annotate_force_break:n{#2}}\par
7696   \stex_if_do_html:TF{
7697     \use:c{stex_annotate_env}{data-ftml-proofbody={}}
7698     \_stex_annotate_force_break:n{}}
7699   ){
7700     \bool_if:NT \l_stex_key_hide_bool{\setbox0\vbox\bgroup}
7701   }
7702 }

7703
7704 \cs_new_protected:Nn \__stex_proof_html_env_proof: {
7705   \exp_args:Nne \use:c{stex_annotate_env}{
7706     data-ftml-proof={}}
7707   \seq_if_empty:NF \l_stex_fors_seq {,

```

```

7708     data-ftml-fors={\seq_map_indexed_function:NN \l_stex_fors_seq \_stex_comma_sep_uri:nn
7709   }
7710   \bool_if:NT \l_stex_key_hide_bool {,
7711     data-ftml-proofhide=true
7712   }
7713 }
7714 \__stex_proof_html:
7715 }
7716
7717 \cs_new_protected:Nn \__stex_proof_html_env_subproof: {
7718   \str_if_empty:NF \l_stex_key_numname_str {
7719     \__stex_proof_number_as_string:N \l__stex_proof_counter_str
7720   }
7721   \exp_args:Nne \use:c{stex_annotate_env}{
7722     data-ftml-subproof={}
7723     \seq_if_empty:NF \l_stex_fors_seq {,
7724       data-ftml-fors={\seq_map_indexed_function:NN \l_stex_fors_seq \_stex_comma_sep_uri:nn
7725     }
7726     \bool_if:NT \l_stex_key_hide_bool {,
7727       data-ftml-proofhide=true
7728     }
7729     \str_if_empty:NTF \l_stex_key_numname_str {
7730       \str_if_empty:NF \l_stex_key_name_str {,
7731         data-ftml-stepname={\l_stex_key_name_str}
7732       }
7733     }{
7734       , data-ftml-stepname={\l__stex_proof_counter_str}
7735     }
7736   }
7737   \__stex_proof_html:
7738 }
7739
7740 \cs_new_protected:Nn \__stex_proof_html: {
7741   \stex_annotate_force_break:n{
7742     \tl_set:N\l_tmpa_tl {\l_stex_key_term_tl\l_stex_key_method_tl\l_stex_key_just_tl}
7743     \tl_if_empty:NF\l_tmpa_tl{
7744       \stex_annotate_invisible:n{\hbox{
7745         \tl_if_empty:NF \l_stex_key_term_tl {
7746           $\stex_annotate:nn{data-ftml-proofterm={}}{\l_stex_key_term_tl}$
7747         }
7748         \tl_if_empty:NF \l_stex_key_just_tl {
7749           $\stex_annotate:nn{data-ftml-spfjust={}}{\l_stex_key_just_tl}$
7750         }
7751         \tl_if_empty:NF \l_stex_key_method_tl {
7752           \stex_annotate:nn{data-ftml-proofmethod={}}{\l_stex_key_method_tl}
7753         }
7754       }}}
7755   }
7756 }
7757
7758 \cs_new_protected:Nn \__stex_proof_start_comment: {
7759   \bool_if:NT \l__stex_proof_in_spfblock_bool {
7760     \par
7761     \group_begin:\stexcommentfont

```

```

7762 }
7763 }
7764 \cs_new_protected:Nn \__stex_proof_end_comment: {
7765   \bool_if:NT \l__stex_proof_in_spfblock_bool {
7766     \par
7767     \group_end:
7768   }
7769 }
7770 \cs_new_protected:Npn \spfescape #1{
7771   \__stex_proof_end_comment: #1 \__stex_proof_start_comment:
7772 }
7773 \stex_deactivate_macro:Nn \spfescape {sproof~environments}
7774
7775 \stex_new_stylable_env:nnnnnnn{proof}{0}{ m}{
7776   \__stex_proof_begin_proof:nn{#1}{#2}
7777   \bool_set_true:N\l__stex_proof_in_spfblock_bool
7778   %\__stex_proof_start_list:n{}
7779   \__stex_proof_start_comment:
7780 }{
7781   \stex_if_do_html:TF{
7782     %\__stex_proof_end_list:
7783     \use:c{endstex_annotate_env}\use:c{endstex_annotate_env}
7784   }\stex_style_apply:
7785 }{
7786   \emph{\sproofautorefname :}~
7787 }{
7788   \sproofend
7789 }{s}
7790
7791 \AddToHook{env/sproof/end}{
7792   \__stex_proof_end_comment:
7793   \stex_if_do_html:F{\bool_if:NT \l_stex_key_hide_bool\egroup}
7794 }
7795
7796
7797 \stex_new_stylable_env:nnnnnnn{proof*}{0}{ }{
7798   \__stex_proof_begin_proof:nn{#1}{ }
7799   \bool_set_false:N\l__stex_proof_in_spfblock_bool
7800 }{
7801   \stex_if_do_html:TF{\use:c{endstex_annotate_env}\use:c{endstex_annotate_env}}\stex_style_
7802 }{
7803   \emph{Proof:}~
7804 }{
7805   \sproofend
7806 }{s}
7807
7808 \cs_new_protected:Nn \__stex_proof_start_list:n {
7809   %\par
7810   \group_begin:\list{}{
7811     \setlength\topsep{0pt}
7812     \setlength\parsep{0pt}
7813     \setlength\rightmargin{0pt}
7814   }\item[#1]
7815 }

```

```

7816
7817 \cs_new_protected:Nn \__stex_proof_end_list: {
7818   %\par
7819   \endlist\group_end:
7820 }

```

subproof (*env.*)

```

7821 \str_set_eq:NN \subproofautorefname \spfstepautorefname
7822
7823 \cs_new_protected:Nn \__stex_proof_do_after_subproof:N {
7824   \bgroup
7825   \expandafter \__stex_proof_do_after_subproof_i:nn #1
7826   \egroup
7827   \__stex_proof_do_spf_name_i:
7828 }
7829 \cs_new_protected:Nn \__stex_proof_do_after_subproof_i:nn {
7830   \exp_args:Nno \stex_keys_set:nn{subproof}{#1}
7831   \str_gset_eq:NN \l__stex_proof_key_name_str \l_stex_key_name_str
7832   \str_gset_eq:NN \l__stex_proof_key_numname_str \l_stex_key_numname_str
7833   \bool_gset_eq:NN \l__stex_proof_key_showname_bool \l_stex_key_showname_bool
7834   \str_gset:Nn \l__stex_proof_counter_str {#2}
7835 }
7836 \cs_new_protected:Npn \__stex_proof_set_after_subproof:n {
7837   \str_if_empty:NTF \l_stex_key_numname_str {
7838     \str_if_empty:NTF \l_stex_key_name_str \use_none:n \__stex_proof_set_after_subproof_i:n
7839   }\__stex_proof_set_after_subproof_i:n
7840 }
7841 \cs_new_protected:Npn \__stex_proof_set_after_subproof_with_number:n {
7842   \str_if_empty:NTF \l_stex_key_numname_str {
7843     \str_if_empty:NTF \l_stex_key_name_str \use_none:n \__stex_proof_set_after_subproof_wit
7844   }\__stex_proof_set_after_subproof_with_number_i:n
7845 }
7846 \cs_new_protected:Nn \__stex_proof_set_after_subproof_i:n {
7847   \tl_gset:cn{subproof_arg_ \int_use:N \currentgrouplevel}{#{1}{}}
7848
7849   \__stex_proof_set_aftergroup: % \bool_if:NTF \l__stex_proof_in_spfblock_bool \__stex_proo
7850 }
7851 \cs_new_protected:Nn \__stex_proof_set_after_subproof_with_number_i:n {
7852   \__stex_proof_number_as_string:N \l__stex_proof_counter_str
7853   \tl_gset:ce{subproof_arg_ \int_use:N \currentgrouplevel}{\exp_not:n{#1}}{\l__stex_proof_
7854   \__stex_proof_set_aftergroup:
7855 }
7856 \cs_new_protected:Nn \__stex_proof_set_aftergroup: {
7857   \aftergroup \__stex_proof_do_after_subproof:N
7858   \expandafter \aftergroup \csname subproof_arg_ \int_use:N \currentgrouplevel\endcsname
7859 }
7860
7861 \stex_new_stylable_env:nnnnnn{subproof}{s O{} m}{
7862   \stex_keys_set:nn{subproof}{#2}
7863   \stex_do_for_list:
7864   \stex_if_do_html:T {
7865     \__stex_proof_html_env_subproof:
7866   }
7867   \seq_map_inline:Nn \l_stex_fors_seq {

```



```

7868 \stex_debug:nn{definiens}{Adding-definiens-to-##1}
7869 \stex_if_do_html:TF{
7870   \STEXinvisible{\hbox{\_stex_add_definiens:nn {##1}{proven}}}}
7871 }{
7872   \_stex_add_definiens:nn {##1}{\STEXinvisible{proven}}
7873 }
7874 }
7875
7876 \IfBooleanTF #1 {
7877   \bool_set_false:N \l__stex_proof_in_spfblock_bool
7878   \__stex_proof_set_after_subproof:n{#2}
7879   \stex_if_do_html:F \stex_style_apply:
7880   \str_if_empty:NF \l_stex_key_id_str {
7881     \__stex_proof_number_as_string:N \@currentlabel
7882     %\str_set:Ne \@currentHref{subproof.\@currentlabel}
7883     %\_stex_do_id:
7884   }
7885
7886
7887   \stex_annotate:nn{data-ftml-title={}}{#3}
7888 }{
7889   \bool_if:NTF \l__stex_proof_in_spfblock_bool {
7890
7891     \__stex_proof_set_after_subproof_with_number:n{#2}
7892
7893     %\str_if_empty:NF \l_stex_key_id_str {
7894       %\_stex_proof_number_as_string:N \@currentlabel
7895       %\str_set:Ne \@currentHref{subproof.\@currentlabel}
7896       %\_stex_do_id:
7897     %}
7898     \use:c{stex_annotate_env}{data-ftml-title={}} \__stex_proof_start_list:n{\__stex_proo
7899       \__stex_proof_add_counter:
7900       \stex_if_do_html:F \stex_style_apply:
7901   }{
7902     \noindent \stex_annotate:nn{data-ftml-title={}}{#3} \par
7903     \stex_if_do_html:F \stex_style_apply:
7904
7905     \__stex_proof_set_after_subproof:n{#2}
7906
7907     %\_stex_do_id:
7908   }
7909 }
7910 \stex_if_do_html:TF{
7911   \use:c{stex_annotate_env}{data-ftml-proofbody={}}
7912   \_stex_annotate_force_break:n{
7913 }{\bool_if:NT \l_stex_key_hide_bool{\setbox0\vbox\bgroup}}
7914 \__stex_proof_start_comment:
7915 }{
7916   \bool_if:NT\l__stex_proof_in_spfblock_bool {
7917     \__stex_proof_remove_counter:
7918     \__stex_proof_inc_counter:
7919   }
7920
7921   \stex_if_do_html:TF{

```

```

7922 \use:c{endstex_annotate_env} %proofbody
7923 \bool_if:NT\l__stex_proof_in_spfblock_bool \__stex_proof_end_list:
7924 \use:c{endstex_annotate_env} % subproof
7925 }{
7926 \stex_style_apply:
7927 \bool_if:NT \l__stex_proof_in_spfblock_bool \__stex_proof_end_list:
7928 }
7929 }{}{}{}
7930
7931 \AddToHook{env/subproof/before}{
7932 \__stex_proof_end_comment:
7933 }
7934
7935 \AddToHook{env/subproof/after}{
7936 \__stex_proof_start_comment:
7937 }
7938
7939 \AddToHook{env/subproof/end}{
7940 \__stex_proof_end_comment:
7941 \stex_if_do_html:F{\bool_if:NT \l_stex_key_hide_bool\egroup}
7942 }
7943
7944 \stex_deactivate_macro:Nn \subproof {sproof~environments}
7945

```

spfsketchenv (env.) TODO:

```

7946 \newenvironment{spfsketchenv}{}{}

```

\spfsketch

```

7947 \stex_new_stylable_cmd:nnnn{spfsketch}{0}{ m}{\par
7948 \begin{spfsketchenv}
7949 \stex_keys_set:nn{spftop}{#1}
7950 \_stex_do_for_list:
7951 \_stex_do_id:
7952 \par
7953 \exp_args:Nne \use:c{stex_annotate_env}{
7954 data-ftml-proofsketch={
7955 \seq_if_empty:NF \l_stex_fors_seq {
7956 \seq_map_function:NN \l_stex_fors_seq \stex_use_symbol_uri:N
7957 }
7958 }
7959 }
7960 \stex_style_apply:
7961 #2\par
7962 \use:c{endstex_annotate_env}
7963 \end{spfsketchenv}
7964 }{
7965 \noindent\emph{\spfsketchenvautorefname :}~
7966 }

```

(End of definition for \spfsketch. This function is documented on page 158.)

sfunction

Keys for proof step macros:

```

7967 \stex_keys_define:nnnn { spfsteps } {

```

```

7968 %\str_clear:N \l_stex_key_name_str
7969 }{
7970 %name .str_set_x:N = \l_stex_key_name_str
7971 }{spfcommon,spfnamed}

Common setup for all the proof step macros:

7972 \cs_new_protected:Nn \__stex_proof_make_step_macro:Nnnnn {
7973   \NewDocumentCommand #1 {s O{}} +m {
7974     \__stex_proof_end_comment:
7975     \stex_keys_set:nn{spfsteps}{##2}
7976     %\str_if_empty:NF \l_stex_key_name_str {
7977       % \exp_args:Nne\use:nn{\vardef}{\v\l_stex_key_name_str}[name=\l_stex_key_name_str]{\l_
7978       %}
7979     \__stex_proof_do_spf_name:
7980
7981     \spfstepenv
7982     \str_if_empty:NF \l_stex_key_id_str {
7983       %\__stex_proof_number_as_string:N \@currentlabel
7984       %\str_set:Ne \@currentHref{spfstep.\@currentlabel}
7985       %\__stex_do_id:
7986     }
7987
7988     \bool_if:NTF \l__stex_proof_in_spfblock_bool {
7989       \IfBooleanTF ##1 {
7990         \__stex_proof_step_html_i:nn{#2}{##3}
7991       }{
7992         \__stex_proof_step_html:nn{#2}{\__stex_proof_start_list:n{#3} ##3 \__stex_proof_end
7993         #5
7994       }
7995       \endspfstepenv
7996       \__stex_proof_start_comment:
7997     }{
7998       \__stex_proof_step_html_i:nn{#2}{##3}
7999       \endspfstepenv
8000     }
8001   }
8002   \stex_deactivate_macro:Nn #1 {sproof~environments}
8003 }

8004
8005 \cs_new_protected:Nn \__stex_proof_step_html:nn {
8006   \str_if_empty:NF \l_stex_key_numname_str {
8007     \__stex_proof_number_as_string:N \l__stex_proof_counter_str
8008   }
8009   \stex_if_do_html:TF{
8010     %\group_begin:
8011     \exp_args:Nne \use:c{stex_annotate_env}{
8012       data-ftml-spf#1={
8013         \seq_if_empty:NF \l_stex_fors_seq {
8014           \seq_map_function:NN \l_stex_fors_seq \stex_use_symbol_uri:N
8015         }
8016       }
8017     \str_if_empty:NTF \l_stex_key_numname_str {
8018       \str_if_empty:NF \l_stex_key_name_str {,
8019         data-ftml-stepname={\l_stex_key_name_str}
8020       }

```

```

8021     }{
8022     , data-ftml-stepname={\l__stex_proof_counter_str}
8023     }
8024   }
8025   \__stex_proof_html:
8026   #2
8027   \par
8028   \use:c{endstex_annotate_env}
8029   %\group_end:
8030   }{ #2 }
8031 }
8032 \cs_new_protected:Nn \__stex_proof_step_html_i:nn {
8033   \stex_if_do_html:TF{
8034     \noindent
8035     %\group_begin:
8036     \exp_args:Ne \stex_annotate:nn{
8037       data-ftml-spf#1={
8038         \seq_if_empty:NF \l_stex_fors_seq {
8039           \seq_map_function:NN \l_stex_fors_seq \stex_use_symbol_uri:N
8040         }
8041       }
8042       \str_if_empty:NTF \l_stex_key_numname_str {
8043         \str_if_empty:NF \l_stex_key_name_str {,
8044           data-ftml-stepname={\l_stex_key_name_str}
8045         }
8046       }{
8047         , data-ftml-stepname={\l__stex_proof_counter_str}
8048       }
8049     }{
8050       \__stex_proof_html:
8051       #2
8052     }
8053     %\group_end:
8054   }{ #2 }
8055 }

```

spfstepenv (*env*.) TODO:

```

8056 \newenvironment{spfstepenv}{
8057   \str_set_eq:NN \spfstepenvautorefname \spfstepautorefname
8058 }{}

```

spfblock (*env*.)

```

8059 \NewDocumentEnvironment{spfblock}{}{
8060   \bool_set_false:N \l__stex_proof_in_spfblock_bool
8061 }{
8062   \aftergroup\__stex_proof_start_comment:
8063 }
8064
8065 \stex_deactivate_macro:Nn \spfblock {sproof~environments}
8066 \AddToHook{env/spfblock/before}{
8067   \__stex_proof_end_comment:
8068 }

```

\spfstep

```

8069 \__stex_proof_make_step_macro:Nnnnn \spfststep {step} {\__stex_proof_insert_number:\__stex_pr
(End of definition for \spfststep. This function is documented on page 158.)
sfunction

\assumption
8070 \__stex_proof_make_step_macro:Nnnnn \assumption {assumption} {\__stex_proof_insert_number:\
(End of definition for \assumption. This function is documented on page 158.)
sfunction

\conclude
8071 \__stex_proof_make_step_macro:Nnnnn \conclude {conclusion} {\$ \Rightarrow$} {} {}
(End of definition for \conclude. This function is documented on page 158.)
sfunction

\eqstep
8072 \NewDocumentCommand \eqstep {s m}{
8073   \__stex_proof_end_comment:
8074   \spfststepenv
8075
8076   \bool_if:NTF \l__stex_proof_in_spfblock_bool {
8077     \IfBooleanTF #1 {
8078       \__stex_proof_step_html_i:nn{eqstep}{\$= #2$}
8079     }{
8080       \__stex_proof_step_html:nn{eqstep}{\__stex_proof_start_list:n{\$=$} \$#2$ \__stex_proof
8081     }
8082     \endspfststepenv
8083     \__stex_proof_start_comment:
8084   }{
8085     \__stex_proof_step_html_i:nn{eqstep}{\$= #2$}
8086     \endspfststepenv
8087   }
8088 }
8089 \stex_deactivate_macro:Nn \eqstep {sproof~environments}
(End of definition for \eqstep. This function is documented on page 158.)
sfunction

\yield
8090 \NewDocumentCommand \yield {+m}{
8091   \stex_annotate:nn{data-ftml-proofterm={}}{ #1 }
8092 }
8093 \stex_deactivate_macro:Nn \yield {sproof~environments}
(End of definition for \yield. This function is documented on page 158.)
sfunction

\spfjust
8094 \newcommand\spfjust[1]{
8095   \stex_annotate:nn{data-ftml-spfjust={}}{ #1 }
8096 }
8097 \stex_deactivate_macro:Nn \spfjust {sproof~environments}

```

(End of definition for `\spfjust`. This function is documented on page 158.)

sfunction

`\spfby`

```
8098 \newcommand\spfby[1]{
8099   \stex_annotate:nn{data-ftml-proofmethod={}}{ #1 }
8100 }
8101 \stex_deactivate_macro:Nn \spfby {sproof~environments}
```

(End of definition for `\spfby`. This function is documented on page 158.)

sfunction

`\spfarg`

```
8102 \newcommand\spfarg[2][]{
8103   \stex_annotate:nn{data-ftml-spfarg={#1}}{ #2 }
8104 }
8105 \stex_deactivate_macro:Nn \spfarg {sproof~environments}
```

(End of definition for `\spfarg`. This function is documented on page 159.)

sfunction

`\sproofend`

```
8106 \tl_set:Nn \__stex_proof_proof_box_tl {
8107   \ltx@ifpackageloaded{amssymb}{\square$}{
8108     \hbox{\vrule\vbox{\hrule width 6 pt\vskip 6pt\hrule}\vrule}
8109   }
8110 }
8111
8112 \tl_set:Nn \sproofend {
8113   \tl_if_empty:NF \l_stex_key_proofend_tl {
8114     \hfil\null\nobreak\hfill\l_stex_key_proofend_tl\par\smallskip
8115   }
8116 }
```

(End of definition for `\sproofend`. This function is documented on page 159.)

sfunction

`\stexcommentfont`

```
8117 \cs_new_protected:Npn \stexcommentfont {
8118   \small\itshape
8119 }
```

(End of definition for `\stexcommentfont`. This function is documented on page 159.)

sfunction

Chapter 37

Metatheory

```
8120 <@@=stex_meta>

      Setup:
8121 \group_begin:
8122 \cs_set:Npn \__stex_modules_persist_module: {}
8123 \cs_set:Npn \stex_check_term:nn #1 #2 {}
8124 \cs_set:Npn \stex_sref_do_aux:n #1 { #1 }
8125 \bool_set_false:N \c_stex_check_terms_bool
8126 \bool_set_false:N \l__stex_annotate_do_output_bool
8127 \stex_set_meta_archive:
8128 \tl_set:Nx \l_stex_current_document_uri {
8129   {\tl_to_str:n{FTML/meta}}{}
8130   {\tl_to_str:n{Metatheory}}{\tl_to_str:n{en}}{}
8131 }
8132 \stex_module_setup_top_nosig:n{Metatheory}
8133 \g_stex_every_module_tl

8134
8135 \symdef{typeof}[name=type~of,args=1]{\maincomp{\mathrm{tp}}\dobrackets{#1}}
8136
8137 \symdef{commented}[args=2]{#2 \; (#1)}
8138 \notation{commented}[inv]{#2}
8139
8140 \symdef{of~type}[args=ii,invisible]{#1}
8141 \notation{of~type}[colon]{#1 \mathbin{\comp{:}} #2}
8142
8143 \symdef{apply}[args=ia,prec=0;\infpref x\infpref]{#1\mathopen{\comp{}} #2 \mathclose{\comp{}}}
8144 \notation{apply}[lambda]{#1\; \argsep{#2}{\;}}
8145 \notation{apply}[infixop]{\argsep{#2}{\mathbin{#1}}\STEXinvisible{#1}}
8146 \notation{apply}[infixrel]{\argsep{#2}{\mathrel{#1}}\STEXinvisible{#1}}
8147
8148 \symdef{autoprove}[name=prove automatically]{\mathrm{automatically\ proved}}
8149
8150 % structures
8151 \symdef{module~type}[args=i,op=\mathtt{MOD}]
8152   {\mathopen{\comp{\mathtt{MOD}}}{}#1\mathclose{\comp{}}}
8153 \symdef{module~type~merge}[args=a,op=\oplus]
8154   {\argsep{#1}{\mathbin{\comp{\oplus}}}}
8155 \symdef{anonymous~record}[args=a]
```

```

8156   {\mathopen{\comp{[[]}\#1\mathclose{\comp{[]}}}}
8157 \symdef{record~field}[args=2]{\#1\comp{.}\#2}
8158 \symdecl*{record~type}
8159
8160 \symdecl{mathstruct}[name=mathematical~structure,args=a] % TODO
8161 \notation{mathstruct}[angle,prec=nobrackets]
8162   {\mathopen{\comp{\langle} \#1 \mathclose{\comp{\rangle}}}
8163 \notation{mathstruct}[parens,prec=nobrackets]
8164   {\mathopen{\comp{() \#1 \mathclose{\comp{}}}}
8165
8166 % sequences
8167 \symdef{ellipses}[ldots]{\ldots}
8168 \symdef{sequence~expression}[comma,args=a]{\#1}
8169 \symdef{sequence~type}[args=1]{\#1^{\comp{ast}}}
8170 \symdef{ranged~sequence~type}[args=ia]{\#1\c_math_subscript_token{\dobrackets{\#2}}}
8171 \symdef{sequence~range}[inv,args=a]{\#1}
8172 \symdef{sequence~map}[args=ai]{
8173   \comp{\mathrm{map}}\mathopen{\comp{()}\#1\mathpunct{\comp{,}}}
8174   \#2\mathclose{\comp{()}}
8175 }
8176 \symdef{fold~right}[args=aibbi]{
8177   \maincomp{\mathrm{fold}}\dobrackets{\#2\mathpunct{\comp{;}}\#1\mathpunct{\comp{;}}
8178     \dobrackets{\#3,\#4} \mathrel{\comp{\mapsto}} \#5
8179 }
8180 }
8181
8182 \symdef{fold}[args=abbi]{
8183   \maincomp{\mathrm{fold}}\dobrackets{\#1\mathpunct{\comp{;}}
8184     \dobrackets{\#2,\#3} \mathrel{\comp{\mapsto}} \#4
8185 }
8186 }
8187
8188 \symdecl{bind}[args=Bi,assoc=pre]
8189 \notation{bind}[defun,prec=nobrackets,op=(\cdot)\;\to\;\cdot]
8190   {\mathopen{\comp{() \#1 \mathclose{\comp{()}\mathbin{\comp{\to}}}} \#2}
8191 \notation{bind}[forall]{\comp{\forall} \#1.\;\#2}
8192 \notation{bind}[Pi]{\mathop{\comp{\prod}}\c_math_subscript_token{\#1}\#2}
8193
8194 \symdef{implicit~bind}[args=Bi,assoc=pre]{\mathopen{\comp{\{} \#1 \mathclose{\comp{\}}\c_math_
8195 \symdef{apply~implicits}[args=ia,prec=nobrackets]{
8196   \underbrace{\#1}_{\#2}
8197 }
8198
8199 \symdecl{arrow}[args=ai,assoc=pre]
8200 \notation{arrow}[single]{
8201   \argsep{\#1}{\mathbin{\comp{\times}}}
8202   \mathrel{\maincomp{\to}} \#2
8203 }
8204 \notation{arrow}[double]{
8205   \argsep{\#1}{\mathbin{\comp{\times}}}
8206   \mathrel{\maincomp{\Rrightarrow}} \#2
8207 }
8208
8209 \symdef{intsecttp}[intersection type,args=a,sqcap,prec=0;-10]{ \argsep{\#1}{\maincomp{\mathr

```



```

8210
8211 \symdef{bindin}[args=Bi]{ \mathop{\maincomp{\amalg}}_{\#1}\#2 }
8212
8213 \symdecl*{integer~literal}
8214 \notation{integer~literal}{\mathbb Z}
8215
8216 \symdecl*{ordinal}
8217 \notation{ordinal}{\mathtt{Ord}}
8218
8219 % propositions
8220 \symdef{prop}[name=proposition]{\mathtt{Prop}}
8221 \symdef{judgment~holds}[args=i,role=judgment]{\comp\vdash\;#1}
8222
8223 % any object
8224 \symdef{object}{\mathtt{Obj}}
8225
8226 \symdef{letin}[name=let in,args=Bi,role=let]{\maincomp{\mathrm{let}}\ #1\ \comp{\mathrm{in}}}
8227
8228 % TODO DELETE
8229 \symdef{aseqdots}[args=a,prec=nobrackets]
8230   {\#1\comp{\,\ldots}}\%{\##1\comp,\##2}
8231 \symdef{aseqfromto}[args=ai,prec=nobrackets]
8232   {\#1\comp{\,\ldots,}\#2}\%{\##1\comp,\##2}
8233 \symdef{aseqfromtovia}[args=aai,prec=nobrackets]
8234   {\#1\comp{\,\ldots,}\#2\comp{\,\ldots,}\#3}\%{\##1\comp,\##2}

Closing:
8235 \global \let \l_stex_metatheory_uri \l_stex_current_module_uri
8236 \global \let \c_stex_default_metatheory \l_stex_metatheory_uri
8237 \stex_close_module:
8238 \group_end:

```

Chapter 38

Others

```
8239 <@@=stex_others>
8240
8241 \cs_new_protected:Npn \MSC #1 {}
8242
8243 \_stex_persist_read_now:
8244 \cs_new_protected:Nn \__stex_others_newlabel:n {
8245   \tl_gset:cn{__stex_sref_aux_sym_ \tl_to_str:n{#1}}{X}
8246   \exp_args:Ne\__stex_others_old_newlabel:{\tl_to_str:n{#1}}
8247 }
8248 \AtBeginDocument{
8249   \iow_now:Nn \@auxout {
8250     \ExplSyntaxOn
8251     \let\__stex_others_old_newlabel:\newlabel
8252     \let\newlabel\__stex_others_newlabel:n
8253     \ExplSyntaxOff
8254   }
8255 }

Inference Rules
8256 \NewDocumentCommand \FTMLrule {m m}{
8257   \tl_if_empty:nTF{#2}{
8258     \seq_clear:N \l_tmpa_seq
8259   }{
8260     \seq_set_split:Nnn \l_tmpa_seq , {#2}
8261   }
8262   \int_zero:N \l_tmpa_int
8263   \stex_annotate_invisible:n{\hbox{
8264     $
8265     \stex_annotate:nn{data-ftml-inferencerule={#1}}{
8266       \_stex_annotate_force_break:n{~
8267         \seq_if_empty:NF \l_tmpa_seq {
8268           \seq_map_inline:Nn \l_tmpa_seq {
8269             \int_incr:N \l_tmpa_int
8270             \stex_annotate:nn{
8271               data-ftml-argmode=i,
8272               data-ftml-arg={\int_use:N \l_tmpa_int}
8273             }{ ##1 }
8274           }
8275         }
8276       }
8277     }
8278   }
8279 }
```

```

8276         ~}
8277     }$
8278 }}
8279 }
8280 \stex_deactivate_macro:Nn \FTMLrule {module~environments}
8281 \stex_every_module:n{\stex_reactivate_macro:N \FTMLrule}

```

Chapter 39

Additional Packages

39.1 Implementation: The notesslides Package

39.1.1 Class and Package Options

We define some Package Options and switches for the `notesslides` class and activate them by passing them on to `beamer.cls` and `omdoc.cls` and the `notesslides` package. We pass the `nontheorem` option to the `statements` package when we are not in notes mode, since the `beamer` package has its own (overlay-aware) theorem environments.

```
8282 (*cls)
8283 (@@=notesslides)
8284 \ProvidesExplClass{notesslides}{2026/06/24}{4.1.0}{notesslides Class}
8285 \RequirePackage{13keys2e}
8286
8287 \str_const:Nn \c__notesslides_class_str {article}
8288
8289 \keys_define:nn{notesslides / cls}{
8290   class .str_set_x:N = \c__notesslides_class_str,
8291   notes .bool_set:N = \c_notesslides_notes_bool ,
8292   slides .code:n      = { \bool_set_false:N \c_notesslides_notes_bool },
8293   %docopt .str_set_x:N = \c__notesslides_docopt_str,
8294   unknown .code:n      = {
8295     \PassOptionsToClass{\CurrentOption}{beamer}
8296     \PassOptionsToClass{\CurrentOption}{\c__notesslides_class_str}
8297     \PassOptionsToPackage{\CurrentOption}{notesslides}
8298     \PassOptionsToPackage{\CurrentOption}{stex}
8299   }
8300 }
8301 \ProcessKeysOptions{ notesslides / cls }
8302
8303 \RequirePackage{stex}
8304 \stex_if_html_backend:T {
8305   \bool_set_true:N \c_notesslides_notes_bool
8306 }
8307
8308 \bool_if:NTF \c_notesslides_notes_bool {
8309   \PassOptionsToPackage{notes=true}{notesslides}
8310   \message{notesslides.cls:~Formatting~document~in~notes~mode}
```

```

8311 }{
8312   \PassOptionsToPackage{notes=false}{notesslides}
8313   \message{notesslides.cls:~Formatting~document~in~slides~mode}
8314 }
8315
8316 \bool_if:NTF \c_notesslides_notes_bool {
8317   \LoadClass{\c_notesslides_class_str}
8318 }{
8319   \LoadClass[10pt,notheorems,xcolor={dvipsnames,svgnames}]{beamer}
8320   %\newcounter{Item}
8321   %\newcounter{paragraph}
8322   %\newcounter{subparagraph}
8323   %\newcounter{Hfootnote}
8324 }
8325 \RequirePackage{notesslides}
8326 </cls>

```

now we do the same for the notesslides package.

```

8327 *package}
8328 \ProvidesExplPackage{notesslides}{2026/06/24}{4.1.0}{notesslides Package}
8329 \RequirePackage{l3keys2e}
8330
8331 \keys_define:nn{notesslides / pkg}{
8332   notes .bool_set:N = \c_notesslides_notes_bool ,
8333   slides .code:n = { \bool_set_false:N \c_notesslides_notes_bool },
8334   sectocframes .bool_set:N = \c_notesslides_sectocframes_bool ,
8335   topsect .str_set_x:N = \c_notesslides_topsect_str,
8336   unknown .code:n = {
8337     \PassOptionsToPackage{\CurrentOption}{stex}
8338     \PassOptionsToPackage{\CurrentOption}{tikzinput}
8339   }
8340 }
8341 \ProcessKeysOptions{ notesslides / pkg }
8342
8343 \RequirePackage{stex}
8344 \stex_if_html_backend:T {
8345   \bool_set_true:N \c_notesslides_notes_bool
8346 }
8347
8348 \cs_set:Npn \sectiontitleemph #1 {
8349   \textbf{\Large #1}
8350 }
8351
8352 \newif\ifnotes
8353 \bool_if:NTF \c_notesslides_notes_bool {
8354   \notesttrue
8355   \PassOptionsToPackage{dvipsnames,svgnames}{xcolor}
8356   \RequirePackage[noamsthm,hyperref]{beamerarticle}
8357   \RequirePackage{mdframed}
8358   \str_if_empty:NTF \c_notesslides_topsect_str{
8359     %\setsectionlevel{section}
8360   } {
8361     \exp_args:No \setsectionlevel \c_notesslides_topsect_str
8362   }

```

```

8363 }{
8364   \notesfalse
8365
8366   \cs_new_protected:Nn \__notesslides_do_sectocframes: {
8367     \cs_set_protected:Nn \__notesslides_do_label:n {
8368       \str_case:nnF{##1}{
8369         {part} {
8370           \tl_set:Nx\l__notesslides_num{\thepart}
8371           \tl_set:cx{@ @ label}{
8372             \cs_if_exist:NTF\parttitlename{\exp_not:N\parttitlename}{\exp_not:N\partname}{\exp_not:N\partname}
8373           }
8374         {chapter} {
8375           \tl_set:Nx\l__notesslides_num{\thechapter}
8376           \tl_set:cx{@ @ label}{
8377             \cs_if_exist:NTF\chaptertitlename{\exp_not:N\chaptertitlename}{\exp_not:N\chaptername}{\exp_not:N\chaptername}
8378           }
8379         {section} {
8380           \tl_set:Nx\l__notesslides_num{\cs_if_exist:NT\thechapter{\thechapter.}\thesection}
8381           \tl_set:cx{@ @ label}{\l__notesslides_num\quad}
8382         }
8383         {subsection} {
8384           \tl_set:Nx\l__notesslides_num{\cs_if_exist:NT\thechapter{\thechapter.}\thesection}
8385           \tl_set:cx{@ @ label}{\l__notesslides_num\quad}
8386         }
8387         {subsubsection} {
8388           \tl_set:Nx\l__notesslides_num{\cs_if_exist:NT\thechapter{\thechapter.}\thesection}
8389           \tl_set:cx{@ @ label}{\l__notesslides_num\quad}
8390         }
8391         {paragraph} {
8392           \tl_set:Nx\l__notesslides_num{\cs_if_exist:NT\thechapter{\thechapter.}\thesection}
8393           \tl_set:cx{@ @ label}{\l__notesslides_num\quad}
8394         }
8395       }{
8396         \tl_set:Nx\l__notesslides_num{\cs_if_exist:NT\thechapter{\thechapter.}\thesection.\l__notesslides_num}
8397         \tl_set:cx{@ @ label}{\l__notesslides_num\quad}
8398       }
8399     }
8400     \cs_set_protected:Nn \_sfragment_do_level:nn {
8401       \tl_if_exist:cT{c@##1}{\stepcounter{##1}}
8402       \addcontentsline{toc}{##1}{\protect\numberline{\use:c{the##1}}{##2}}
8403       \__notesslides_do_label:n{##1}
8404       \pdfbookmark[\int_use:N \l_stex_current_section_level_int]{\l__notesslides_num\ ##2}{\l__notesslides_num\ ##2}
8405       \begin{frame}[noframenumbering]
8406         \vfill\centering
8407         \sectiontitleemph{
8408           \use:c{@ @ label} ##2
8409         }
8410       \end{frame}
8411       \int_incr:N \l_stex_current_section_level_int
8412       \str_set:Nn \l_stex_current_section_level_str{##1}
8413     }
8414   }
8415
8416   \cs_set_protected:Nn \_stex_titlefragment: {

```

```

8417     \begin{frame}[noframenumbering]\maketitle\end{frame}
8418     \let\maketitle\relax
8419 }
8420
8421 \AtBeginDocument{
8422     \str_if_empty:NTF \c_notesslides_topsect_str {
8423         \setsectionlevel{section}
8424     } {
8425         \exp_args:No \setsectionlevel \c_notesslides_topsect_str
8426         \exp_args:No \str_if_eq:nnTF \c_notesslides_topsect_str {chapter} {
8427             \__notesslides_define_chapter:
8428         }{
8429             \exp_args:No \str_if_eq:nnT \c_notesslides_topsect_str {part} {
8430                 \__notesslides_define_chapter:
8431                 \__notesslides_define_part:
8432             }
8433         }
8434     }
8435 }
8436
8437 \bool_if:NT \c_notesslides_sectocframes_bool {
8438     \__notesslides_do_sectocframes:
8439 }
8440 }
8441
8442 \cs_new_protected:Nn \__notesslides_define_chapter: {
8443     \cs_if_exist:NF \chaptername {
8444         \cs_set_protected:Npn \chaptername {Chapter}
8445     }
8446     \cs_if_exist:NF \chapter {
8447         \cs_set_protected:Npn \chapter {INVALID}
8448     }
8449     \cs_if_exist:NF \c@chapter {
8450         \newcounter{chapter}\counterwithin*{section}{chapter}
8451     }
8452 }
8453
8454 \cs_new_protected:Nn \__notesslides_define_part: {
8455     \cs_if_exist:NF \partname {
8456         \cs_set_protected:Npn \partname {Part}
8457     }
8458     \cs_if_exist:NF \part {
8459         \cs_set_protected:Npn \part {INVALID}
8460     }
8461     \cs_if_exist:NF \c@part {
8462         \newcounter{part}\counterwithin*{chapter}{part}
8463     }
8464 }

```

\prematuarestop We initialize \afterprematuarestop, and provide \prematuarestop@endsfragment which looks up \sfragment@level and recursively ends enough {sfragment}s.

```

8465 \def \c__notesslides_document_str{document}
8466 \newcommand\afterprematuarestop{}
8467 \def\prematuarestop@endsfragment{

```

```

8468 \unless\ifx\@currenvir\c__notesslides_document_str
8469 \expandafter\expandafter\expandafter\end\expandafter\expandafter\expandafter{\expandafter
8470 \expandafter\prematurestop@endsfragment
8471 \fi
8472 }
8473 \providecommand\prematurestop{
8474 \stex_if_html_backend:F{
8475 \message{Stopping~sTeX~processing~prematurely}
8476 \prematurestop@endsfragment
8477 \afterprematurestop
8478 \end{document}}
8479 }
8480 }

```

(End of definition for \prematurestop. This function is documented on page 111.)

39.1.2 Notes and Slides

For the notes case, we also provide the \usetheme macro that would otherwise come from the the `beamer` class.

```

8481 \bool_if:NT \c_notesslides_notes_bool {
8482 \renewcommand\usetheme[2][\usepackage[#1]{beamertheme#2}]
8483 }
8484 \NewDocumentCommand \libusetheme {0} m {
8485 \libusepackage[#1]{beamertheme#2}
8486 }

```

We define the sizes of slides in the notes. Somehow, we cannot get by with the same here.

```

8487 \newlength{\slidewidth}\setlength{\slidewidth}{13.5cm}
8488 \newlength{\slideheight}\setlength{\slideheight}{9cm}

```

We first set up the slide boxes in `notes` mode. We set up sizes and provide a box register for the frames and a counter for the slides.

```

8489 \ifnotes
8490
8491 \newlength{\slideframewidth}
8492 \setlength{\slideframewidth}{1.5pt}

```

`frame (env.)` We first define the keys.

```

8493 \cs_new_protected:Nn \__notesslides_do_yes_param:Nn {
8494 \exp_args:Nx \str_if_eq:nnTF { \str_uppercase:n{ #2 } }{ yes }{
8495 \bool_set_true:N #1
8496 }{
8497 \bool_set_false:N #1
8498 }
8499 }
8500
8501 \stex_keys_define:nnnn{notesslides / frame}{
8502 \str_clear:N \l__notesslides_frame_label_str
8503 \bool_set_true:N \l__notesslides_frame_allowframebreaks_bool
8504 \bool_set_true:N \l__notesslides_frame_allowdisplaybreaks_bool
8505 \bool_set_true:N \l__notesslides_frame_fragile_bool
8506 \bool_set_true:N \l__notesslides_frame_shrink_bool

```



```

8507 \bool_set_true:N \l__notesslides_frame_squeeze_bool
8508 \bool_set_true:N \l__notesslides_frame_t_bool
8509 }{
8510   label .str_set_x:N = \l__notesslides_frame_label_str,
8511   allowframebreaks .code:n = {
8512     \__notesslides_do_yes_param:Nn \l__notesslides_frame_allowframebreaks_bool { #1 }
8513   },
8514   allowdisplaybreaks .code:n = {
8515     \__notesslides_do_yes_param:Nn \l__notesslides_frame_allowdisplaybreaks_bool { #1 }
8516   },
8517   fragile .code:n = {
8518     \__notesslides_do_yes_param:Nn \l__notesslides_frame_fragile_bool { #1 }
8519   },
8520   shrink .code:n = {
8521     \__notesslides_do_yes_param:Nn \l__notesslides_frame_shrink_bool { #1 }
8522   },
8523   squeeze .code:n = {
8524     \__notesslides_do_yes_param:Nn \l__notesslides_frame_squeeze_bool { #1 }
8525   },
8526   t .code:n = {
8527     \__notesslides_do_yes_param:Nn \l__notesslides_frame_t_bool { #1 }
8528   },
8529   unknown .code:n = {}
8530 }{}

```

We redefine the `itemize` environment so that it looks more like the one in `beamer`.

```

8531 \cs_new_protected:Nn \__notesslides_setup_itemize: {
8532   \def\itemize@level{outer}
8533   \def\itemize@outer{outer}
8534   \def\itemize@inner{inner}
8535   %\newcommand\metakeys@show@keys[2]{\marginnote{\scriptsize ##2}}
8536   \renewenvironment{itemize}{
8537     \ifx\itemize@level\itemize@outer
8538       \def\itemize@label{$\rhd$}
8539     \fi
8540     \ifx\itemize@level\itemize@inner
8541       \def\itemize@label{$\scriptstyle\rhd$}
8542     \fi
8543     \begin{list}
8544       {\itemize@label}
8545       {\setlength{\labelsep}{.3em}
8546        \setlength{\labelwidth}{.5em}
8547        \setlength{\leftmargin}{1.5em}
8548       }
8549     \edef\itemize@level{\itemize@inner}
8550   }{
8551     \end{list}
8552   }
8553 }

```

We create the box with the `mdframed` environment from the `equinymous` package.

```

8554 \stex_if_html_backend:TF {
8555   %\stex_css_literal:n{
8556   % .stex-frame {

```

```

8557 % background-color:#fff;
8558 % margin:5px ~ 0;
8559 % border:2px ~ solid ~ #000;
8560 % border-radius:5px;
8561 % padding:5px;
8562 % padding-right:10px;
8563 % width: var(--rustex-curr-width);
8564 % display:block;
8565 % }
8566 %}

8567
8568 \cs_new_protected:Nn \__notesslides_frame_box_begin: {
8569 \vbox\bgroup
8570 \begin{stex_annotate_env}{data-ftml-slide={}}%{class:stex-frame={}}
8571 \mdf@patchamsthm\notesslidesfont
8572 }
8573 \cs_new_protected:Nn \__notesslides_frame_box_end: {
8574 %^A \notesslides@slidelabel
8575 \medskip\par\hbox to \textwidth{\tiny\notesslidesfooter}
8576 \end{stex_annotate_env}\egroup
8577 }
8578 }{
8579 \cs_new_protected:Nn \__notesslides_frame_box_begin: {
8580 \begin{mdframed}[
8581 linewidth=\slideframewidth,
8582 skipabove=1ex,
8583 skipbelow=1ex,
8584 userdefinedwidth=\slidewidth,
8585 align=center,
8586 %usetwoside=false
8587 ]\notesslidesfont
8588 }
8589 \cs_new_protected:Nn \__notesslides_frame_box_end: {
8590 \medskip\par\noindent\tiny\notesslidesfooter%^A\notesslides@slidelabel
8591 \end{mdframed}
8592 }
8593 }

```

We define the environment, read them, and construct the slide number and label.

```

8594 \renewenvironment{frame}[1][]{
8595 \stex_keys_set:nn{notesslides / frame}{#1}
8596 \stepcounter{framenum}
8597 \renewcommand\newpage{\addtocounter{framenum}{1}}
8598 \def\@currentlabel{\theframenum}
8599 \str_if_empty:NF \l__notesslides_frame_label_str {
8600 \label{\l__notesslides_frame_label_str}
8601 }
8602 \__notesslides_setup_itemize:
8603 \__notesslides_frame_box_begin:
8604 }{
8605 \__notesslides_frame_box_end:
8606 }

```

Now, we need to redefine the frametitle (we are still in course notes mode).


```

8653 %      \rustex_if:T {\par
8654 %      \rustex_direct_HTML:n{</td><td>}
8655 %      }
8656 %  }
8657 % }
8658 \end{lrbox}\usebox\columnbox
8659 }
8660 \fi

```

39.1.3 Environment and Macro Patches

The `note` environment is used to leave out text in the `slides` mode. It does not have a counterpart in OMDoc. So for course notes, we define the `note` environment to be a no-operation otherwise we declare the `note` environment to produce no output.

```

8661 \cs_if_exist:cTF{note}{
8662   \bool_if:NTF \c_notesslides_notes_bool {
8663     \renewenvironment{note}{\ignorespaces}{}
8664   }{
8665     \renewenvironment{note}{\setbox \l_tmpa_box\vbox\bgroup}{\egroup}
8666   }
8667 }{
8668   \bool_if:NTF \c_notesslides_notes_bool {
8669     \newenvironment{note}{\ignorespaces}{}
8670   }{
8671     \newenvironment{note}{\setbox \l_tmpa_box\vbox\bgroup}{\egroup}
8672   }
8673 }

```

For other environments we introduce variants prefixed with `n`, which are excluded in `slides` mode.

```

8674 \cs_new_protected:Nn \__notesslides_notes_env:nnnn {
8675   \bool_if:NTF \c_notesslides_notes_bool {
8676     \newenvironment{#1}#2{#3}{#4}
8677   }{
8678     \newenvironment{#1}#2{
8679       \cs_set:Npn \__notesslides_eat: ####1 \end ####2 {
8680         \str_if_eq:nnTF{#1}{####2}{
8681           \end{#1}
8682         }{
8683           \__notesslides_eat:
8684         }
8685       }
8686       \__notesslides_eat:
8687       %\setbox\l_tmpa_box\vbox\bgroup#3
8688     }{
8689       %#4\egroup
8690     }
8691   }
8692 }
8693
8694 \__notesslides_notes_env:nnnn{nparagraph}{[1] []}{\begin{sparagraph}[#1]}{\end{sparagraph}}
8695 \__notesslides_notes_env:nnnn{nfragment}{[2] []}{\begin{sfragment}[#1]{#2}}{\end{sfragment}}
8696 \__notesslides_notes_env:nnnn{ndefinition}{[1] []}{\begin{sdefinition}[#1]}{\end{sdefinition}}

```

```

8697 \_notesslides\_notes\_env:nnnn{nassertion}{[1] []}{\begin{sassertion}[#1]}{\end{sassertion}}
8698 \_notesslides\_notes\_env:nnnn{nproof}{[2] []}{\begin{sproof}[#1]{#2}}{\end{sproof}}
8699 \_notesslides\_notes\_env:nnnn{nexample}{[1] []}{\begin{sexample}[#1]}{\end{sexample}}
8700
8701 \RequirePackage{graphicx}
8702
8703 \NewDocumentCommand\frameimage{s O{} m}{
8704   \IfBooleanTF #1 {
8705     \begin{frame}[plain]
8706   }{
8707     \begin{frame}
8708   }
8709   \bool_if:NTF \c_notesslides\_notes\_bool {
8710     \slidewidth=\dimexpr\slidewidth-(2\slideframewidth)\relax
8711   }{
8712     \slidewidth=\textwidth\relax
8713   }
8714   \def\Gin@ewidth{}\setkeys{Gin}{#2}
8715   \tl_if_empty:NTF \Gin@ewidth {
8716     \mhgraphics[width=\slidewidth,#2]{#3}
8717   }{
8718     \mhgraphics[#2]{#3}
8719   }
8720   \end{frame}
8721 }

```

hacking inputref:

`\inputref*`

```

8722 \cs_set_eq:NN\_notesslides\_inputref:\inputref
8723 \cs_set_protected:Npn\inputref{\@ifstar\ninputref\_notesslides\_inputref:}
8724 \bool_if:NTF \c_notesslides\_notes\_bool {
8725   \newcommand\ninputref[2] []{
8726     \_notesslides\_inputref:[#1]{#2}
8727   }
8728 }{
8729   \newcommand\ninputref[2] []{}
8730 }

```

(End of definition for \inputref. This function is documented on page 110.)*

39.1.4 Styling Across Notes/Slides

```

8731 \def\notesslides\titleemph#1{
8732   {\Large\bf\sffamily#1}
8733   \vskip0.1\baselineskip
8734   \leaders\vrule width \textwidth
8735   \vskip0.4pt%
8736   \nointerlineskip
8737 }
8738
8739 \def\notesslides\footer{}
8740
8741 \let\notesslides\font\sffamily

```

39.1.5 Beamer Compatibility

All of this should be removed and made part of a template

```
8742
8743 \stex_if_do_html:TF{
8744   \NewDocumentEnvironment{slideshow}{}{
8745     \begin{stex_annotate_env}{data-ftml-slideshow={}}
8746     \cs_set_protected:Npn \nextslide ##1 {
8747       \begin{stex_annotate_env}{data-ftml-slideshow-slide={}} ##1
8748       \end{stex_annotate_env}
8749     }
8750     \cs_set_protected:Npn \lastslide ##1 {
8751       \begin{stex_annotate_env}{data-ftml-slideshow-slide={}} ##1
8752       \end{stex_annotate_env}
8753     }
8754   }{
8755     \end{stex_annotate_env}
8756   }
8757 }{
8758   \int_new:N \l__notesslides_slideshow_counter_int
8759   \NewDocumentEnvironment{slideshow}{}{
8760     \int_zero:N \l__notesslides_slideshow_counter_int
8761     \cs_set_protected:Npn \nextslide ##1 {
8762       \int_incr:N \l__notesslides_slideshow_counter_int
8763       \only<\int_use:N \l__notesslides_slideshow_counter_int>{##1}
8764     }
8765     \cs_set_protected:Npn \lastslide ##1 {
8766       \int_incr:N \l__notesslides_slideshow_counter_int
8767       \only<\int_use:N \l__notesslides_slideshow_counter_int ->{##1}
8768     }
8769   }{
8770
8771   }
8772 }
8773
8774 \bool_if:NT \c_notesslides_notes_bool {
8775   \def\author{\@dblarg\ns@author}
8776   \long\def\ns@author[#1]#2{%
8777     \tl_if_empty:nTF{#1}{
8778       \def\beamer@shortauthor{#2}
8779     }{
8780       \def\beamer@shortauthor{#1}
8781     }
8782     \def\@author{#2}
8783   }
8784   \def\title{\@dblarg\ns@title}
8785   \long\def\ns@title[#1]#2{%
8786     \tl_if_empty:nTF{#1}{
8787       \def\beamer@shorttitle{#2}
8788     }{
8789       \def\beamer@shorttitle{#1}
8790     }
8791     \def\@title{#2}
8792     \stexdoctitle{#2}
```

```

8793 }
8794 \def\insertshortauthor{
8795   \hbox\bgroup\def\{\}\cs_if_exist:NT\beamer@shortauthor\beamer@shortauthor\egroup
8796 }
8797 \def\insertshorttitle{
8798   \hbox\bgroup\def\{\}\cs_if_exist:NT\beamer@shorttitle\beamer@shorttitle\egroup
8799 }
8800 \stex_if_html_backend:TF{
8801   \def\insertframenumber{\stex_annotate:nn{data-ftml-slide-number={}}{}}
8802 }{
8803   \def\insertframenumber{\@arabic\c@framenumber}
8804 }
8805 \def\insertshortdate{\today}
8806 }

```

39.1.6 TODO Excursions

`\excursion` The excursion macros are very simple, we define a new internal macro `\excursionref` and use it in `\excursion`, which is just an `\inputref` that checks if the new macro is defined before formatting the file in the argument.

```

8807 \gdef\printexcursions{
8808
8809
8810
8811 \stex_if_html_backend:TF{
8812   \newcommand\excursionref[4][\% repos, label, path, text
8813     \begin{sparagraph}[title=Excursion]
8814       #4~
8815       \stex_in_archive:nn{#1}{
8816         \stex_document_uri_from_archive_file:Nn \l__notesslides_uri {#3}
8817         \exp_args:Ne\href{\stex_use_document_uri:N\l__notesslides_uri}{here}
8818       }
8819     \end{sparagraph}
8820   }
8821   \def\excursion{\excursionref}
8822 }{
8823   \newcommand\excursionref[4][\% repos, label, path, text
8824     \bool_if:NT \c_notesslides_notes_bool {
8825       \begin{sparagraph}[title=Excursion]
8826         #4~\sref[fallback=the appendix]{#2}.
8827       \end{sparagraph}
8828     }
8829   }
8830   \newcommand\activate@excursion[2][\%
8831     \tl_gput_right:Nn\printexcursions{\inputref[#1]{#2}}
8832   }
8833   \newcommand\excursion[4][\% repos, label, path, text
8834     \bool_if:NT \c_notesslides_notes_bool {
8835       \activate@excursion[#1]{#3}
8836       \excursionref[#1]{#2}{#3}{#4}
8837     }
8838   }
8839 }

```

8840 }

(End of definition for \excursion. This function is documented on page 112.)

\excursiongroup

```

8841 \keys_define:nn{notesslides / excursiongroup }{
8842   id          .str_set_x:N = \l__notesslides_excursion_id_str,
8843   intro       .tl_set:N    = \l__notesslides_excursion_intro_tl,
8844   archive     .str_set_x:N = \l__notesslides_excursion_mhrepos_str
8845 }
8846 \cs_new_protected:Nn \__notesslides_excursion_args:n {
8847   \tl_clear:N \l__notesslides_excursion_intro_tl
8848   \str_clear:N \l__notesslides_excursion_id_str
8849   \str_clear:N \l__notesslides_excursion_mhrepos_str
8850   \keys_set:nn {notesslides / excursiongroup }{ #1 }
8851 }
8852
8853
8854 \stex_if_html_backend:TF{
8855   \newcommand\excursiongroup[1][]{ }
8856 }{
8857   \newcommand\excursiongroup[1][]{
8858     \__notesslides_excursion_args:n{ #1 }
8859     \tl_if_empty:NF\printexcursions
8860     {\IfInputref}{\begin{note}
8861       \begin{sfragment}{Excursions}% TODO pass on id
8862       \ifdefempty\l__notesslides_excursion_intro_tl{\{
8863         \exp_args:NNe \use:nn \inputref{[\l__notesslides_excursion_mhrepos_str]{
8864           \l__notesslides_excursion_intro_tl
8865         }}
8866       }
8867       \printexcursions%
8868       \end{sfragment}
8869       \end{note}}}
8870   }
8871 }
8872
8873 \ifcsname beameritemnestingprefix\endcsname\else\def\beameritemnestingprefix{}\fi

```

(End of definition for \excursiongroup. This function is documented on page 112.)

```

8874 \prop_new:N \g__notesslides_variables_prop
8875 \cs_set_protected:Npn \setSGvar #1 #2 {
8876   \prop_gput:Nnn \g__notesslides_variables_prop {#1}{#2}
8877 }
8878 \cs_set_protected:Npn \useSGvar #1 {
8879   \prop_item:Nn \g__notesslides_variables_prop {#1}
8880 }
8881 \cs_set_protected:Npn \ifSGvar #1 #2 #3 {
8882   \prop_get:NnNF \g__notesslides_variables_prop {#1} \l__notesslides_tmp {
8883     \PackageError{document-structure}
8884     {The sTeX Global variable #1 is undefined}
8885     {set it with \protect\setSGvar}\TODO better error
8886   }
8887   \tl_if_eq:NnT \l__notesslides_tmp {#2}{ #3 }

```



```

8888 }
8889
8890
8891 </package>

```

39.2 Implementation: The problem Package

39.2.1 Package Options

The first step is to declare (a few) package options that handle whether certain information is printed or not. They all come with their own conditionals that are set by the options.

```

8892 <*package>
8893 <@@=problems>
8894 \ProvidesExplPackage{problem}{2026/06/24}{4.1.0}{Semantic Markup for Problems}
8895 \RequirePackage{l3keys2e}
8896
8897 \keys_define:nn { problem / pkg }{
8898   notes      .default:n      = { true },
8899   notes      .bool_set:N     = \c__problems_notes_bool,
8900   gnotes     .default:n      = { true },
8901   gnotes     .bool_set:N     = \c__problems_gnotes_bool,
8902   hints      .default:n      = { true },
8903   hints      .bool_set:N     = \c__problems_hints_bool,
8904   solutions  .default:n      = { true },
8905   solutions  .bool_set:N     = \c__problems_solutions_bool,
8906   pts        .default:n      = { true },
8907   pts        .bool_set:N     = \c__problems_pts_bool,
8908   min        .default:n      = { true },
8909   min        .bool_set:N     = \c__problems_min_bool,
8910   %boxed     .default:n      = { true },
8911   %boxed     .bool_set:N     = \c__problems_boxed_bool,
8912   test       .default:n      = { true },
8913   test       .bool_set:N     = \c__problems_test_bool,
8914   unknown    .code:n         = {
8915     \PassOptionsToPackage{\CurrentOption}{stex}
8916   }
8917 }
8918 \newif\ifsolutions
8919
8920 \ProcessKeysOptions{ problem / pkg }
8921 \bool_if:NTF \c__problems_solutions_bool {
8922   \solutionstrue
8923 }{
8924   \solutionsfalse
8925 }
8926 \newif\ifintest
8927 \bool_if:NTF \c__problems_test_bool {
8928   \intesttrue
8929 }{
8930   \intestfalse
8931 }
8932

```

```

8933 \RequirePackage{stex}
8934
8935 \ifintest
8936   \AtBeginDocument{
8937     \def\symrefemph#1{#1}
8938     \def\compemph#1{#1}
8939     \def\varemp#1{#1}
8940   }
8941 \fi
8942
8943 \newcommand\precondition[2]{
8944   \stex_get_symbol:n{#2}
8945   \tl_if_empty:NF \l_stex_get_symbol_uri {
8946     \str_case:nnTF {#1}{
8947       {remember}{}
8948       {understand}{}
8949       {analyze}{}
8950       {evaluate}{}
8951       {apply}{}
8952       {create}{}
8953     }{
8954       \ifstexhtml\else\bool_if:NT\c_stex_metadata_bool {
8955         \marginpar{\tiny \textbf{Precondition:}}~#1~
8956         \exp_args:Ne\symrefemph@uri{\stex_symbol_uri_name:N \l_stex_get_symbol_uri}{\l_stex
8957       }
8958     }\fi
8959     \stex_annotate_invisible:nn{
8960       data-ftml-preconditionsymbol={\stex_use_symbol_uri:N \l_stex_get_symbol_uri},
8961       data-ftml-preconditiondimension={#1}
8962     }{}
8963   }{\errmessage{Unknown~cognitive~dimension~#1}}
8964 }
8965 }
8966 \newcommand\objective[2]{
8967   \stex_get_symbol:n{#2}
8968   \tl_if_empty:NF \l_stex_get_symbol_uri {
8969     \str_case:nnTF {#1}{
8970       {remember}{}
8971       {understand}{}
8972       {analyze}{}
8973       {evaluate}{}
8974       {apply}{}
8975       {create}{}
8976     }{
8977       \ifstexhtml\else\bool_if:NT\c_stex_metadata_bool {
8978         \marginpar{\tiny \textbf{Objective:}}~#1~
8979         \exp_args:Ne\symrefemph@uri{\stex_symbol_uri_name:N \l_stex_get_symbol_uri}{\l_stex
8980       }
8981     }\fi
8982     \stex_annotate_invisible:nn{
8983       data-ftml-objectivesymbol={\stex_use_symbol_uri:N \l_stex_get_symbol_uri},
8984       data-ftml-objectivedimension={#1}
8985     }{}
8986   }{\errmessage{Unknown~cognitive~dimension~#1}}

```

```

8987 }
8988 }

```

\problem@kw@* For multilinguality, we define internal macros for keywords that can be specialized in *.ldf files.

```

8989 \AddToHook{begindocument}{
8990   \ExplSyntaxOn\makeatletter
8991   \input{problem-english.ldf}
8992   \ltx@ifpackageloaded{babel}{
8993     \clist_set:Nx \l_tmpa_clist {\exp_args:No \tl_to_str:n \bbl@loaded}
8994     \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{ngerman}}{
8995       \input{problem-ngerman.ldf}
8996     }
8997     \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{finnish}}{
8998       \input{problem-finnish.ldf}
8999     }
9000     \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{french}}{
9001       \input{problem-french.ldf}
9002     }
9003     \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{russian}}{
9004       \input{problem-russian.ldf}
9005     }
9006   }{}
9007   \makeatother\ExplSyntaxOff
9008 }

```

(End of definition for \problem@kw@*. This function is documented on page ??.)

39.2.2 Problems and Solutions

We now prepare the KeyVal support for problems. The key macros just set appropriate internal macros.

```

9009 \bool_new:N \l_stex_key_autogradable_bool
9010 \stex_keys_define:nnnn{ problem }{
9011   \tl_clear:N \l_stex_key_pts_tl
9012   \tl_set:Nn \l_stex_key_min_tl 0
9013   \str_clear:N \l_stex_key_name_str
9014   \str_clear:N \l_stex_key_mhrepos_str
9015   \bool_set_false:N \l_stex_key_autogradable_bool
9016 }{
9017   pts .tl_set:N = \l_stex_key_pts_tl,
9018   min .tl_set:N = \l_stex_key_min_tl,
9019   name .str_set:N = \l_stex_key_name_str,
9020   autogradable .bool_set:N = \l_stex_key_autogradable_bool,
9021   archive .code:n = {},
9022   %archive .str_set:N = \l_stex_key_mhrepos_str,
9023   %creators .code:n = {}
9024   %imports .tl_set:N = \l__problems_prob_imports_tl,
9025   %refnum .int_set:N = \l__problems_prob_refnum_int,
9026 }{id,title,style,uses}

```

Then we set up a counter for problems.

\numberproblemsin

```

9027 \newcounter{sproblem}[section]
9028 \newcommand\numberproblemsin[1]{
9029   \@addtoreset{sproblem}{#1}
9030   \def\thesproblem{\arabic{#1}.\arabic{sproblem}}
9031 }
9032 \numberproblemsin{section}
9033 %\def\theplainsproblem{\arabic{sproblem}}
9034 %\def\thesproblem{\thesection.\theplainsproblem}

```

(End of definition for \numberproblemsin. This function is documented on page ??.)

sproblem (env.)

```

9035 \cs_new:Nn \__problems_activate_macros: {
9036   \stex_reactivate_macro:N \solution
9037   \stex_reactivate_macro:N \mcb
9038   \stex_reactivate_macro:N \scb
9039   \stex_reactivate_macro:N \fillinsol
9040   \stex_reactivate_macro:N \hint
9041   \stex_reactivate_macro:N \exnote
9042   \stex_reactivate_macro:N \gnote
9043 }
9044
9045 \fp_new:N \g_problem_total_pts_fp
9046 \fp_new:N \g_problem_total_min_fp
9047
9048 \bool_new:N \l__problems_in_problem_bool
9049 \bool_new:N \l__problems_has_pts_bool
9050 \bool_new:N \l__problems_has_min_bool
9051 \bool_set_false:N \l__problems_in_problem_bool
9052 \stex_new_stylable_env:nnnnnnn {problem} {0{}}{
9053   \bool_if:NT \l__problems_in_problem_bool {
9054     \msg_error:nn{stex}{error/nestedproblem}
9055   }
9056   \cs_if_exist:NTF \l_problem_inputproblem_keys_tl {
9057     \tl_put_left:Nn \l_problem_inputproblem_keys_tl {#1,}
9058     \exp_args:Nno \stex_keys_set:nn{problem}{
9059       \l_problem_inputproblem_keys_tl
9060     }
9061   }{
9062     \stex_keys_set:nn{problem}{#1}
9063   }
9064   \refstepcounter{sproblem}
9065
9066   \stex_if_do_html:T {
9067     \str_if_empty:NT \l_stex_key_name_str {
9068       \stex_file_split_off_lang:NN \l__problems_path_seq \g_stex_current_file
9069       \seq_get_right:NN \l__problems_path_seq \l_stex_key_name_str
9070     }
9071     \exp_args:Nne \begin{stex_annotate_env} {
9072       data-ftml-problem={\l_stex_key_name_str},
9073       %data-ftml-language={ \l_stex_current_language_str},
9074       data-ftml-autogradable={\bool_if:NTF \l_stex_key_autogradable_bool {true}{false}}
9075       \tl_if_empty:NF \l_stex_key_pts_tl{,data-ftml-problempoints={\l_stex_key_pts_tl}}

```

```

9076     }
9077     \noindent \stex_annotate:nn{data-ftml-title={}}{\_stex_annotate_force_break:n{\l_stex_k
9078     \tl_if_empty:NF \l_stex_key_title_tl {
9079         \exp_args:No \stexdoctitle \l_stex_key_title_tl
9080     }
9081     \_stex_annotate_force_break:n{}}
9082 }
9083 \tl_set_eq:NN \thistitle \l_stex_key_title_tl
9084 \tl_if_empty:NT \l_stex_key_pts_tl { \tl_set:Nn \l_stex_key_pts_tl{0} }
9085
9086
9087 \bool_set_true:N \l__problems_in_problem_bool
9088 \tl_set_eq:NN \l__problems_pts_tl \l_stex_key_pts_tl
9089 \tl_set_eq:NN \l__problems_min_tl \l_stex_key_min_tl
9090 \tl_if_eq:NnTF \l__problems_pts_tl {0}
9091     {\bool_set_false:N \l__problems_has_pts_bool}
9092     {\bool_set_true:N \l__problems_has_pts_bool}
9093 \tl_if_eq:NnTF \l__problems_min_tl {0}
9094     {\bool_set_false:N \l__problems_has_min_bool}
9095     {\bool_set_true:N \l__problems_has_min_bool}
9096 \int_gzero:N \g__problems_subproblem_int
9097
9098 \stex_if_do_html:F\stex_style_apply:
9099 \_stex_do_id:
9100 \__problems_activate_macros:
9101 }{
9102 \fp_gadd:Nn \g_problem_total_pts_fp { \l__problems_pts_tl}
9103 \fp_gadd:Nn \g_problem_total_min_fp { \l__problems_min_tl}
9104 \__problems_record_problem:
9105 \stex_if_do_html:F\stex_style_apply:
9106 \stex_if_do_html:T{ \end{stex_annotate_env} }
9107 }{
9108 \par\noindent\problemheader
9109 \stex_if_do_html:F{
9110     \bool_if:NT \c__problems_pts_bool {
9111         \tl_if_eq:NnF \l__problems_pts_tl {0}{
9112             \marginpar{\l__problems_pts_tl}{~\problem@kw@pts\smallskip}
9113         }
9114     }
9115     \bool_if:NT \c__problems_min_bool {
9116         \tl_if_eq:NnF \l__problems_min_tl {0} {
9117             \marginpar{\l__problems_min_tl}{~\problem@kw@minutes\smallskip}
9118         }
9119     }
9120 }
9121 \par
9122 \stex_ignore_spaces_and_pars:
9123 }{
9124 \par\bigskip
9125 % \bool_if:NT \c__problems_test_bool \pagebreak
9126 }{s}
9127
9128 \tl_set:Nn \problemheader {
9129     \stex_if_do_html:TF{

```

```

9130 \tl_if_empty:NF \thistitle {
9131 \stex_annotate:nn{data-ftml-title={}}{\textbf{\thistitle}}
9132 }
9133 }{
9134 \textbf{\sproblemautorefname{~}\thesproblem
9135 \tl_if_empty:NF \thistitle {
9136 {~}(\thistitle)
9137 }
9138 }
9139 }
9140 }
9141
9142 \cs_new_protected:Nn \__problems_record_problem: {
9143 \exp_args:Nne \iow_now:Nn \@auxout {
9144 \problem@restore {\thesproblem}{\l__problems_pts_tl}{\l__problems_min_tl}
9145 }
9146 }
9147
9148 \cs_new_protected:Npn \problem@restore #1 #2 #3 {}

```

subproblem (env.)

```

9149 \int_new:N \g__problems_subproblem_int
9150
9151 \stex_new_stylable_env:nnnnnn {subproblem} {0}{}{
9152 \stex_keys_set:nn{problem}{#1}
9153 \bool_if:NF \l__problems_in_problem_bool{
9154 \ifstexhtml\else
9155 \par{\bfseries WARNING~subproblem~to~be~used~in~some~problem}\par
9156 \fi
9157 \__problems_activate_macros:
9158 \bool_set_true:N \l__problems_in_problem_bool
9159 \tl_set:Nn \l__problems_pts_tl{0}
9160 \tl_set:Nn \l__problems_min_tl{0}
9161 }
9162 \str_if_empty:NT \l_stex_key_name_str {
9163 \stex_file_split_off_lang:NN \l__problems_path_seq \g_stex_current_file
9164 \seq_get_right:NN \l__problems_path_seq \l_stex_key_name_str
9165 }
9166 \stex_if_do_html:T {
9167 \str_if_empty:NT \l_stex_key_name_str {
9168 \stex_file_split_off_lang:NN \l__problems_path_seq \g_stex_current_file
9169 \seq_get_right:NN \l__problems_path_seq \l_stex_key_name_str
9170 }
9171 \exp_args:Nne \begin{stex_annotate_env} {
9172 data-ftml-subproblem={\l_stex_key_name_str},
9173 %data-ftml-language={ \l_stex_current_language_str},
9174 data-ftml-autogradable={\bool_if:NTF \l_stex_key_autogradable_bool {true}{false}}
9175 \tl_if_empty:NF \l_stex_key_pts_tl{,data-ftml-problempoints={\l_stex_key_pts_tl}}
9176 }
9177 \noindent \stex_annotate:nn{data-ftml-title={}}{\stex_annotate_force_break:n{\l_stex_k
9178 \tl_if_empty:NF \l_stex_key_title_tl {
9179 \exp_args:No \stexdoctitle \l_stex_key_title_tl
9180 }
9181 \tl_if_empty:NF \l_stex_key_title_tl {

```

```

9182     \exp_args:No \stexdoctitle \l_stex_key_title_tl
9183   }
9184   \_stex_annotate_force_break:n{}
9185 }
9186 \tl_if_empty:NT \l_stex_key_pts_tl { \tl_set:Nn \l_stex_key_pts_tl{0} }
9187 \int_gincr:N \g__problems_subproblem_int
9188 \bool_if:NF \l__problems_has_pts_bool {
9189   \tl_gset:Ne \l__problems_pts_tl {\fp_to_decimal:n {\l__problems_pts_tl + \l_stex_key_pt
9190 }
9191 \bool_if:NF \l__problems_has_min_bool {
9192   \tl_gset:Ne \l__problems_min_tl {\fp_to_decimal:n {\l__problems_min_tl + \l_stex_key_mi
9193 }
9194 \stex_if_smsmode:F {\stex_if_do_html:F \stex_style_apply: }
9195 }{
9196   \stex_if_do_html:TF{ \end{stex_annotate_env} }{\stex_if_smsmode:F\stex_style_apply:}
9197 }{
9198   \begin{list}{}{
9199     \setlength\topsep{0pt}
9200     \setlength\parsep{0pt}
9201     \setlength\rightmargin{0pt}
9202   }\item[\int_use:N \g__problems_subproblem_int .]
9203   \bool_if:NT \c__problems_pts_bool {
9204     \bool_if:NF \l__problems_has_pts_bool {
9205       \marginpar{\smallskip\l_stex_key_pts_tl{}}~\problem@kw@pts}
9206     }
9207   }
9208   \bool_if:NT \c__problems_min_bool {
9209     \bool_if:NF \l__problems_has_min_bool{
9210       \marginpar{\smallskip\l_stex_key_min_tl{}}~\problem@kw@minutes}
9211     }
9212   }
9213   \ignorespaces
9214 }{
9215   \end{list}
9216 }{}

```

\includeproblem

```

9217 \stex_keys_define:nnnn{ includeproblem }{
9218   \str_clear:N \l_stex_key_mhrepos_str
9219 }{
9220   archive .str_set:N      = \l_stex_key_mhrepos_str,
9221   unknown .code:n = {}
9222 }{}
9223
9224 \NewDocumentCommand\includeproblem{0{ } m}{
9225   \group_begin:
9226   \tl_set:Nn \l_problem_inputproblem_keys_tl {#1}
9227   \stex_keys_set:nn{includeproblem}{#1}
9228   \exp_args:Nno \use:nn{\inputref[]\l_stex_key_mhrepos_str}{#2}
9229   \group_end:
9230 }
9231

```

(End of definition for \includeproblem. This function is documented on page 117.)

`solution (env.)`

```

9232 \int_new:N \g_problem_id_counter
9233 \dim_new:N \l_stex_key_testspace_dim
9234 \stex_keys_define:nnnn{ solution }{
9235   \str_clear:N \l_stex_key_answerclass_str
9236   \dim_zero:N \l_stex_key_testspace_dim
9237 }{
9238   testspace .dim_set:N = \l_stex_key_testspace_dim,
9239   answerclass .str_set:N = \l_stex_key_answerclass_str
9240 }{id,title,style}
9241
9242 \cs_new_protected:Nn \__problems_solution_start:n {
9243   \stex_keys_set:nn{ solution }{#1}
9244   \str_if_empty:NT \l_stex_key_id_str {
9245     \int_gincr:N \g_problem_id_counter
9246     \str_set:Nx \l_stex_key_id_str {
9247       SOLUTION_\int_use:N \g_problem_id_counter
9248     }
9249   }
9250   \stex_if_do_html:T{
9251     \begin{stex_annotate_env}{
9252       data-ftml-solution=\l_stex_key_id_str,
9253       data-ftml-answerclass={\l_stex_key_answerclass_str}
9254     }
9255   }
9256 }
9257
9258 \stex_new_stylable_env:nnnnnnn { solution }{ 0{ } }{
9259   \stex_if_do_html:TF{
9260     \__problems_solution_start:n{#1}
9261   }{
9262     \ifsolutions
9263       \__problems_solution_start:n{#1}
9264       \stex_style_apply:
9265     \else
9266       \stex_keys_set:nn{ solution }{#1}
9267       \testspace{\l_stex_key_testspace_dim}
9268       \setbox\l_tmpa_box\vbox\bgroup
9269     \fi
9270   }
9271 }{
9272   \stex_if_do_html:TF{
9273     \end{stex_annotate_env}
9274   }{
9275     \ifsolutions
9276       \stex_style_apply:
9277       \stex_if_do_html:T{
9278         \end{stex_annotate_env}
9279       }
9280     \else
9281       \egroup
9282     \fi
9283   }
9284 }{

```



```

9285 \par\smallskip\rule[.3em]{\linewidth}{0.4pt}\newline\smallskip
9286 \noindent\emph{\problem@kw@solution\tl_if_empty:NF \l_stex_key_title_tl{
9287   {~}\l_stex_key_title_tl \str_if_empty:NF \l_stex_key_answerclass_str {
9288     (Answer Class: \l_stex_key_answerclass_str)
9289   }
9290 } :~}
9291 }{
9292 \par\rule[.3em]{\linewidth}{0.4pt}\newline
9293 }{ }
9294
9295 \stex_deactivate_macro:Nn \solution {sproblem-environments}

```

`\startsolutions`
`\stopsolutions`

```

9296 \cs_new_protected:Npn \startsolutions{
9297   \global\solutionstrue
9298 }
9299 \cs_new_protected:Npn \stopsolutions{
9300   \global\solutionsfalse
9301 }

```

(End of definition for `\startsolutions` and `\stopsolutions`. These functions are documented on page 114.)

`hint (env.)`

```

9302
9303 \stex_keys_define:nnnn{ problemenv }{{}}{id,title,style}
9304
9305 \cs_new_protected:Nn \__problems_hint_start:n {
9306   \stex_keys_set:nn{ problemenv }{#1}
9307   \str_if_empty:NT \l_stex_key_id_str {
9308     \int_gincr:N \g_problem_id_counter
9309     \str_set:Nx \l_stex_key_id_str {
9310       HINT_\int_use:N \g_problem_id_counter
9311     }
9312   }
9313   \stex_if_do_html:TF{
9314     \begin{stex_annotate_env}{
9315       data-ftml-problemhint=\l_stex_key_id_str
9316     }
9317   }
9318 }
9319
9320 \stex_new_stylable_env:nnnnnnn { hint }{ 0{ } }{
9321   \stex_if_do_html:TF{
9322     \__problems_hint_start:n{#1}
9323   }{
9324     \bool_if:NTF \c__problems_hints_bool {
9325       \__problems_hint_start:n{#1}
9326       \stex_style_apply:
9327     }{
9328       \setbox\l_tmpa_box\vbox\bgroup
9329     }
9330   }
9331 }{

```

```

9332 \stex_if_do_html:TF{
9333   \end{stex_annotate_env}
9334 }{
9335   \bool_if:NTF \c__problems_hints_bool {
9336     \stex_style_apply:
9337     \stex_if_do_html:T{
9338       \end{stex_annotate_env}
9339     }
9340   }{
9341     \egroup
9342   }
9343 }
9344 }{
9345   \par\smallskip\rule[.3em]{\linewidth}{0.4pt}\newline\smallskip
9346   \noindent\emph{\problem@kw@hint\tl_if_empty:NF \l_stex_key_title_tl{
9347     {~}\l_stex_key_title_tl
9348   } :~}
9349 }{
9350   \par\rule[.3em]{\linewidth}{0.4pt}\newline
9351 }{}
9352 \stex_deactivate_macro:Nn \hint {sproblem-environments}

```

exnote (env.)

```

9353 \cs_new_protected:Nn \__problems_exnote_start:n {
9354   \stex_keys_set:nn{ problemenv }{#1}
9355   \str_if_empty:NT \l_stex_key_id_str {
9356     \int_gincr:N \g_problem_id_counter
9357     \str_set:Nx \l_stex_key_id_str {
9358       EXNOTE_\int_use:N \g_problem_id_counter
9359     }
9360   }
9361   \stex_if_do_html:T{
9362     \begin{stex_annotate_env}{
9363       data-ftml-problemnote=\l_stex_key_id_str
9364     }
9365   }
9366 }
9367
9368 \stex_new_stylable_env:nnnnnn { exnote }{ 0{} }{
9369   \stex_if_do_html:TF{
9370     \__problems_exnote_start:n{#1}
9371   }{
9372     \bool_if:NTF \c__problems_notes_bool {
9373       \__problems_exnote_start:n{#1}
9374       \stex_style_apply:
9375     }{
9376       \setbox\l_tmpa_box\vbox\bgroup
9377     }
9378   }
9379 }{
9380   \stex_if_do_html:TF{
9381     \end{stex_annotate_env}
9382   }{
9383     \bool_if:NTF \c__problems_notes_bool {

```

```

9384     \stex_style_apply:
9385     \stex_if_do_html:T{
9386         \end{stex_annotate_env}
9387     }
9388 }{
9389     \egroup
9390 }
9391 }
9392 }{
9393     \par\smallskip\rule[.3em]{\linewidth}{0.4pt}\newline\smallskip
9394     \noindent\emph{\problem@kw@note\tl_if_empty:NF \l_stex_key_title_tl{
9395         {~}\l_stex_key_title_tl
9396     } :~}
9397 }{
9398     \par\rule[.3em]{\linewidth}{0.4pt}\newline
9399 }{}
9400 \stex_deactivate_macro:Nn \exnote {sproblem~environments}

```

gnote (env.)

```

9401 \int_new:N \l__problems_anscls_int
9402
9403 \cs_new_protected:Nn \__problems_gnote_start:n {
9404     \stex_keys_set:nn{ problemenv }{#1}
9405     \str_if_empty:NT \l_stex_key_id_str {
9406         \int_gincr:N \g_problem_id_counter
9407         \str_set:Nx \l_stex_key_id_str {
9408             GNOTE_\int_use:N \g_problem_id_counter
9409         }
9410     }
9411
9412     \stex_if_do_html:TF{
9413         \begin{stex_annotate_env}{
9414             data-ftml-problemgnote=\l_stex_key_id_str
9415         }
9416     }
9417     \stex_style_apply:
9418 }
9419
9420 \stex_new_stylable_env:nnnnnn { gnote }{ 0{} }{
9421     \stex_if_do_html:TF{
9422         \__problems_gnote_start:n{#1}
9423     }{
9424         \bool_if:NTF \c__problems_gnotes_bool {
9425             \__problems_gnote_start:n{#1}
9426         }{
9427             \setbox\l_tmpa_box\vbox\bgroup
9428         }
9429     }
9430     \stex_reactivate_macro:N \anscls
9431 }{
9432     \stex_if_do_html:TF{
9433         \end{stex_annotate_env}
9434     }{
9435         \bool_if:NTF \c__problems_gnotes_bool {

```

```

9436     \stex_style_apply:
9437     \stex_if_do_html:T{
9438         \end{stex_annotate_env}
9439     }
9440 }{
9441     \egroup
9442 }
9443 }
9444 }{
9445     \par\smallskip\rule[.3em]{\linewidth}{0.4pt}\newline\smallskip
9446     \noindent\emph{\problem@kw@grading\str_if_empty:NF \l_stex_key_title_tl{
9447         {~}\l_stex_key_title_tl
9448     } :~}
9449 }{
9450     \par\rule[.3em]{\linewidth}{0.4pt}\newline
9451 }{}
9452 \stex_deactivate_macro:Nn \gnote {sproblem-environments}
9453
9454
9455 \stex_keys_define:nnnn{ anscls }{
9456     \str_clear:N \l_stex_key_pts_str
9457     \tl_clear:N \l_stex_key_feedback_tl
9458 }{
9459     pts      .str_set:N = \l_stex_key_pts_str,
9460     feedback .tl_set:N = \l_stex_key_feedback_tl,
9461     update   .code:n    = {}
9462 }{id}
9463 \newcommand \anscls [2] [] {
9464     \stex_keys_set:nn{ anscls }{#1}
9465     \str_if_empty:NT \l_stex_key_id_str {
9466         \int_incr:N \l__problems_anscls_int
9467         \str_set:Nx \l_stex_key_id_str {
9468             AC\int_use:N \l__problems_anscls_int
9469         }
9470     }
9471     \stex_if_do_html:TF{
9472         \begin{list}{}{
9473             \setlength\topsep{0pt}
9474             \setlength\parsep{0pt}
9475             \setlength\rightmargin{0pt}
9476         }\item[] \exp_args:Ne \stex_annotate:nn{
9477             data-ftml-answerclass={\l_stex_key_id_str}
9478             \str_if_empty:NF \l_stex_key_pts_str{
9479                 ,data-ftml-answerclass-pts={\l_stex_key_pts_str}
9480             }
9481         }{
9482             #2~
9483             \tl_if_empty:NF \l_stex_key_feedback_tl{
9484                 \stex_annotate:nn{
9485                     data-ftml-answerclass-feedback={}
9486                 }{\l_stex_key_feedback_tl}
9487             }
9488         }
9489     } \end{list}

```

```

9490 }{
9491   \begin{list}{}{
9492     \setlength\topsep{0pt}
9493     \setlength\parsep{0pt}
9494     \setlength\rightmargin{0pt}
9495   }\item[\l_stex_key_id_str] #2
9496     \str_if_empty:NF \l_stex_key_pts_str {\par
9497       ~ \problem@kw@points :~\l_stex_key_pts_str
9498     }
9499     \tl_if_empty:NF \l_stex_key_feedback_tl {\par
9500       ~ \problem@kw@feedback :~\l_stex_key_feedback_tl
9501     }
9502   \end{list}
9503 }
9504 }
9505 \stex_deactivate_macro:Nn \anscls {gnote-environments}

```

The margin pars are reader-visible, so we need to translate

```

9506 \def\pts#1{
9507   \bool_if:NT \c__problems_pts_bool {
9508     \stex_annotate:nn{data-ftml-problempoints={#1}}{\marginpar{#1~\problem@kw@pts}}
9509   }\hbox_unpack:N\c_empty_box
9510 }
9511 \def\min#1{
9512   \bool_if:NT \c__problems_min_bool {
9513     \stex_annotate:nn{data-ftml-problemminutes={}}{\marginpar{#1~\problem@kw@minutes}}
9514   }\hbox_unpack:N\c_empty_box
9515 }

```

mcb (*env.*)

```

9516 \stex_new_stylable_env:nnnnnnn{mcb}{0}{0}{
9517   \stex_keys_set:nn{style}{#1}
9518   \cs_set:Nn \__problems_mccline:n{
9519     \begin{list}{}{
9520       \setlength\topsep{0pt}
9521       \setlength\parsep{0pt}
9522       \setlength\rightmargin{0pt}
9523     }\item[\problem_mcc_box_tl] ##1 \end{list}
9524   }
9525
9526   \stex_if_do_html:T{
9527     \tl_set:Nn\problem_mcc_box_tl{}
9528     \__problems_maybe_inline:n{
9529       \exp_args:Nne \begin{stex_annotate_env}{
9530         data-ftml-multiple-choice-block={}
9531         \clist_if_empty:NF \l_stex_key_style_clist {,
9532           data-ftml-styles={\l_stex_key_style_clist}
9533         }
9534       }
9535     }
9536     \clist_if_in:NnTF \l_stex_key_style_clist {inline} {
9537       \cs_set:Nn \__problems_mccline:n {##1}
9538     }{
9539       \clist_if_in:NnT \l_stex_key_style_clist {dropdown} {

```

```

9540     \cs_set:Nn \__problems_mccline:n {##1}
9541   }
9542 }
9543 }
9544 \stex_deactivate_macro:Nn \mcb {sproblem~environments}
9545 \stex_deactivate_macro:Nn \scb {sproblem~environments}
9546 \stex_deactivate_macro:Nn \solution {sproblem~environments}
9547 \stex_deactivate_macro:Nn \hint {sproblem~environments}
9548 \stex_deactivate_macro:Nn \exnote {sproblem~environments}
9549 \stex_deactivate_macro:Nn \gnote {sproblem~environments}
9550 \stex_reactivate_macro:N \mcc
9551 \stex_if_do_html:F \stex_style_apply:
9552 }{
9553   \stex_if_do_html:TF{
9554     \__problems_maybe_inline:n{\end{stex_annotate_env}}
9555   }\stex_style_apply:
9556 }{\par}{\{}}
9557
9558 \stexstylemcb[inline]{
9559   {\Large[]\cs_set:Nn \__problems_mccline:n{\problem_mcc_box_tl{~} #1}
9560 }{\{ \Large[]}}
9561
9562 \stexstylemcb[dropdown]{
9563   {\Large[]\cs_set:Nn \__problems_mccline:n{\problem_mcc_box_tl{~} #1}
9564 }{\{ \Large[]}}
9565
9566 \stex_deactivate_macro:Nn \mcb {sproblem~environments}
we define the keys for the mcc macro
9567 \cs_new_protected:Nn \__problems_do_yes_param:Nn {
9568   \exp_args:Nx \str_if_eq:nnTF { \str_lowercase:n{ #2 } }{ yes }{
9569     \bool_set_true:N #1
9570   }{
9571     \bool_set_false:N #1
9572   }
9573 }
9574 \stex_keys_define:nnnn{mcc}{
9575   \tl_clear:N \l_stex_key_feedback_tl
9576   \bool_set_false:N \l_stex_key_T_bool
9577   \tl_clear:N \l_stex_key_Ttext_tl
9578   \tl_clear:N \l_stex_key_Ftext_tl
9579 }{
9580   feedback .tl_set:N      = \l_stex_key_feedback_tl ,
9581   T         .code:n       = {\bool_set_true:N \l_stex_key_T_bool} ,
9582   F         .code:n       = {\bool_set_false:N \l_stex_key_T_bool} ,
9583   Ttext     .tl_set:N     = \l_stex_key_Ttext_tl ,
9584   Ftext     .tl_set:N     = \l_stex_key_Ftext_tl ,
9585 }{id}
9586
\mcc
9587
9588 \cs_set:Nn \__problems_maybe_newline: { \\\ }
9589

```

```

9590 \stex_if_html_backend:TF{
9591   \tl_set:Nn \problem_mcc_box_tl {
9592     \ltx@ifpackageloaded{amssymb}{\square$}{
9593       \hbox{\vrule\vbox{\hrule width 6 pt\vskip 6pt\hrule}\vrule}
9594     }
9595   }
9596
9597   \newcommand\mcc[2][ ]{
9598     \stex_keys_set:nn{mcc}{#1}
9599     \__problems_mccline:n{
9600       \stex_if_do_html:T{
9601         \stex_annotate:nn{data-ftml-problem-choice={
9602           \bool_if:NTF \l_stex_key_T_bool {true}{false}
9603         }}{
9604           #2~
9605           \stex_annotate:nn{data-ftml-problem-choice-verdict={}}{
9606             \bool_if:NTF \l_stex_key_T_bool {
9607               \tl_if_empty:NTF \l_stex_key_Ttext_tl \problem@kw@correct \l_stex_key_Ttext_tl
9608             }{
9609               \tl_if_empty:NTF \l_stex_key_Ftext_tl \problem@kw@wrong \l_stex_key_Ftext_tl
9610             }
9611           }
9612           \tl_if_empty:NF \l_stex_key_feedback_tl {
9613             \stex_annotate:nn{data-ftml-problem-choice-feedback={}}{
9614               \l_stex_key_feedback_tl
9615             }
9616           }
9617         }
9618       }
9619     }
9620   }
9621 }{
9622   \tl_set:Nn \problem_mcc_box_default_tl {
9623     \ltx@ifpackageloaded{amssymb}{\square$}{
9624       \hbox{\vrule\vbox{\hrule width 6 pt\vskip 6pt\hrule}\vrule}
9625     }
9626   }
9627   \tl_set:Nn \problem_mcc_box_tl {
9628     \ifsolutions
9629       \bool_if:NTF \l_stex_key_T_bool {
9630         \makebox[0pt][l]{\problem_mcc_box_default_tl}
9631         \hspace{0.1em}$\cs_if_exist:NTF\checkmark{
9632           \raisebox{.15ex}{\checkmark}
9633         } X$
9634       }{\problem_mcc_box_default_tl}
9635     \else
9636       \problem_mcc_box_default_tl
9637     \fi
9638   }
9639   \newcommand\mcc[2][ ]{
9640     \stex_keys_set:nn{mcc}{#1}
9641     \ifsolutions
9642       \tl_set:Nx \l_tmpa_tl{
9643         \bool_if:NTF \l_stex_key_T_bool {

```

```

9644         \tl_if_empty:NF \l_stex_key_Ttext_tl {\exp_not:N \l_stex_key_Ttext_tl}
9645     }{
9646         \tl_if_empty:NF \l_stex_key_Ftext_tl {\exp_not:N \l_stex_key_Ftext_tl}
9647     }
9648     \tl_if_empty:NF \l_stex_key_feedback_tl {
9649         \exp_not:n{\emph{\l_stex_key_feedback_tl}}
9650     }
9651 }
9652 \fi
9653
9654 \__problems_mccline:n{ #2
9655     \ifsolutions
9656         \tl_if_empty:NF \l_tmpa_tl { \footnote{\l_tmpa_tl} }
9657     \fi
9658 }
9659 }
9660 }
9661 \stex_deactivate_macro:Nn \mcc {mcb~environments}

```

(End of definition for \mcc. This function is documented on page 115.)

scb (env.)

```

9662
9663 \cs_new_protected:Nn \__problems_maybe_inline:n {
9664     \clist_if_in:NnTF \l_stex_key_style_clist {inline} {
9665         \let\__problems_oldpar:\stex_par:
9666         \cs_set:Nn\stex_par:{~}
9667         \cs_set:Nn\__problems_maybe_newline:{}
9668         #1
9669         \let\stex_par:\__problems_oldpar:
9670     }{
9671         \clist_if_in:NnTF \l_stex_key_style_clist {dropdown} {
9672             \let\__problems_oldpar:\stex_par:
9673             \cs_set:Nn\stex_par:{~}
9674             \cs_set:Nn\__problems_maybe_newline:{}
9675             #1
9676             \let\stex_par:\__problems_oldpar:
9677         }{\par #1}
9678     }
9679 }
9680
9681 \stex_new_stylable_env:nnnnnn{scb}{0}{}{
9682     \stex_keys_set:nn{style}{#1}
9683     \cs_set:Nn \__problems_sccline:n{
9684         \begin{list}{}{
9685             \setlength\topsep{0pt}
9686             \setlength\parsep{0pt}
9687             \setlength\rightmargin{0pt}
9688         }\item[\problem_scc_box_tl] ##1 \end{list}
9689     }
9690
9691 \stex_if_do_html:T{
9692     \__problems_maybe_inline:n{
9693         \exp_args:Ne\stex_annotate_env{

```



```

9694     data-ftml-single-choice-block={}
9695     \clist_if_empty:NF \l_stex_key_style_clist {,
9696       data-ftml-styles={\l_stex_key_style_clist}
9697     }
9698   }
9699 }
9700 \tl_set:Nn\problem_scc_box_tl{}
9701 \clist_if_in:NnTF \l_stex_key_style_clist {inline} {
9702   \cs_set:Nn \__problems_sccline:n {##1}
9703 }{
9704   \clist_if_in:NnT \l_stex_key_style_clist {dropdown} {
9705     \cs_set:Nn \__problems_sccline:n {##1}
9706   }
9707 }
9708 }
9709 \stex_deactivate_macro:Nn \mcb {sproblem~environments}
9710 \stex_deactivate_macro:Nn \scb {sproblem~environments}
9711 \stex_deactivate_macro:Nn \solution {sproblem~environments}
9712 \stex_deactivate_macro:Nn \hint {sproblem~environments}
9713 \stex_deactivate_macro:Nn \exnote {sproblem~environments}
9714 \stex_deactivate_macro:Nn \gnote {sproblem~environments}
9715 \stex_reactivate_macro:N \scc
9716 \stex_if_do_html:F \stex_style_apply:
9717 }{
9718   \stex_if_do_html:TF{
9719     \__problems_maybe_inline:n { \endstex_annotate_env }
9720   }\stex_style_apply:
9721 }{\par}{\par}
9722
9723 \stexstylescb[inline]{
9724   {\Large[]\cs_set:Nn \__problems_sccline:n{\problem_scc_box_tl{~} #1}
9725   }{\{ \Large[]}}
9726
9727 \stexstylescb[dropdown]{
9728   {\Large[]\cs_set:Nn \__problems_sccline:n{\problem_scc_box_tl{~} #1}
9729   }{\{ \Large[]}}
9730
9731 \stex_deactivate_macro:Nn \scb {sproblem~environments}

```

\scc

```

9732
9733 \stex_if_html_backend:TF{
9734   \tl_set:Nn\problem_scc_box_tl{${\bigcirc}}
9735   \newcommand\scc[2][]{
9736     \stex_keys_set:nn{mcc}{#1}
9737     \__problems_sccline:n{
9738       \stex_if_do_html:T{
9739         \stex_annotate:nn{data-ftml-problem-choice={
9740           \bool_if:NTF \l_stex_key_T_bool {true}{false}
9741         }}{
9742           #2~
9743           \stex_annotate:nn{data-ftml-problem-choice-verdict={}}{
9744             \bool_if:NTF \l_stex_key_T_bool {
9745               \tl_if_empty:NTF \l_stex_key_Ttext_tl \problem@kw@correct \l_stex_key_Ttext_t

```

```

9746         }{
9747         \tl_if_empty:NTF \l_stex_key_Ftext_tl \problem@kw@wrong \l_stex_key_Ftext_tl
9748         }
9749     }
9750     \tl_if_empty:NF \l_stex_key_feedback_tl {
9751         \stex_annotate:nn{data-ftml-problem-choice-feedback={}}{
9752             \l_stex_key_feedback_tl
9753         }
9754     }
9755 }
9756 }
9757 }
9758 }
9759 }{
9760 \tl_set:Nn\problem_scc_box_default_tl{${\bigcirc$}
9761 \tl_set:Nn\problem_scc_box_tl{
9762     \ifsolutions
9763     \bool_if:NTF \l_stex_key_T_bool {
9764         \makebox[Opt][l]{\problem_scc_box_default_tl}
9765         \hspace{0.1em}${\cs_if_exist:NTF\checkmark{
9766             \raisebox{.15ex}{\checkmark}
9767         }} X$
9768     }{\problem_scc_box_default_tl}
9769     \else
9770     \problem_scc_box_default_tl
9771     \fi
9772 }
9773 \newcommand\scc[2][]{
9774     \stex_keys_set:nn{mcc}{#1}
9775     \ifsolutions
9776     \tl_set:Nx \l_tmpa_tl{
9777         \bool_if:NTF \l_stex_key_T_bool {
9778             \tl_if_empty:NF \l_stex_key_Ttext_tl {\exp_not:N \l_stex_key_Ttext_tl}
9779         }{
9780             \tl_if_empty:NF \l_stex_key_Ftext_tl {\exp_not:N \l_stex_key_Ftext_tl}
9781         }
9782         \tl_if_empty:NF \l_stex_key_feedback_tl {
9783             \exp_not:n{ \\emph{\l_stex_key_feedback_tl} }
9784         }
9785     }
9786     \fi
9787
9788     \__problems_sccline:n{ #2
9789     \ifsolutions
9790     \tl_if_empty:NF \l_tmpa_tl { \footnote{\l_tmpa_tl} }
9791     \fi
9792 }
9793 }
9794 }
9795
9796 \stex_deactivate_macro:Nn \scc {scb-environments}
9797
9798
9799 \newcommand\yesTnoF{

```

```

9800 \begin{scb}[style=inline]
9801 \scc[T]{yes}~\scc[F]{no}
9802 \end{scb}
9803 }
9804 \newcommand\yesFnoT{
9805 \begin{scb}[style=inline]
9806 \scc[F]{yes}~\scc[T]{no}
9807 \end{scb}
9808 }
9809 \newcommand\trueTfalseF{
9810 \begin{scb}[style=inline]
9811 \scc[T]{true}~\scc[F]{false}
9812 \end{scb}
9813 }
9814 \newcommand\trueFfalseT{
9815 \begin{scb}[style=inline]
9816 \scc[F]{true}~\scc[T]{false}
9817 \end{scb}
9818 }

```

(End of definition for \scc. This function is documented on page ??.)

\fillinsol

```

9819 \stex_keys_define:nnnn{fillinsol}{
9820 \tl_clear:N \l__problems_fillin_solution_tl
9821 \dim_zero:N \l_stex_key_testspace_dim
9822 }{
9823 \testspace .dim_set:N = \l_stex_key_testspace_dim,
9824 \exact .code:n = {\__problems_parse_fillin_arg:nnnn{exact}#1 },
9825 \numrange .code:n = {\__problems_parse_fillin_arg:nnnn{numrange}#1 },
9826 \regex .code:n = {\__problems_parse_fillin_arg:nnnn{regex}#1 }
9827 }{}
9828
9829 \tl_const:Nn \c__problems_true_tl{T}
9830 \tl_const:Nn \c__problems_false_tl{F}
9831
9832 \msg_set:nnn{problem}{error/neither-true-nor-false}{
9833 \Fillinsol~verdict~#1~is~neither~T~nor~F!
9834 }
9835
9836 \stex_if_html_backend:TF{
9837 \cs_new:Nn \__problems_parse_fillin_arg:nnnn {
9838 \tl_set:Nn \l_tmpa_tl{#3}
9839 \tl_if_eq:NNF \l_tmpa_tl\c__problems_true_tl{
9840 \tl_if_eq:NNF \l_tmpa_tl\c__problems_false_tl {
9841 \msg_error:nnn{problem}{error/neither-true-nor-false}{#3}
9842 }
9843 }
9844 \tl_put_right:Ne \l__problems_fillin_solution_tl {
9845 \stex_annotate:nn{
9846 data-ftml-fillin-case={#1},
9847 data-ftml-fillin-case-value={\tl_to_str:n{#2}},
9848 data-ftml-fillin-case-verdict={
9849 \tl_if_eq:NNTF\l_tmpa_tl\c__problems_true_tl{true}{false}

```

```

9850     },
9851     }{\_stex_annotate_force_break:n{\exp_not:n{#4}}}}
9852   }
9853 }
9854 }{
9855   \cs_new:Nn \_problems_parse_fillin_arg:nnnn {
9856     \tl_put_right:Nn \l_problems_fillin_solution_tl {
9857       #1 & \tl_to_str:n{#2} & #3 & #4 \crr
9858     }
9859   }
9860 }
9861
9862
9863 \newcommand\fillinsol[2][{}{
9864   \stex_if_do_html:F \quad
9865   \mode_if_math:TF{
9866     \hbox{\_problems_fillinsol:nn{#1}{#2}}
9867   }{
9868     \_problems_fillinsol:nn{#1}{#2}
9869   }
9870   \stex_if_do_html:F \quad
9871 }
9872
9873 \stex_if_html_backend:TF{
9874   \cs_new_protected:Nn \_problems_fillinsol:nn {
9875     \stex_keys_set:nn{fillinsol}{#1}
9876     \tl_if_empty:nF{#2}{
9877       \_problems_parse_fillin_arg:nnnn{exact}{#2}{T}{}
9878     }
9879     \hbox_set:Nn \l_tmpa_box{\fbox{\huge{\texttt{\tl_to_str:n{#2}}}\hskip\l_stex_key_testsp
9880     \exp_args:Ne \stex_annotate:nn{
9881       data-ftml-fillinsol={},
9882       data-ftml-fillinsol-width={\dim_to_decimal:n{1.5 \box_wd:N \l_tmpa_box }}
9883     }{
9884       \_stex_annotate_force_break:n{
9885         \l_problems_fillin_solution_tl
9886         #2
9887       }
9888     }
9889   }
9890 }{
9891   \cs_new_protected:Nn \_problems_fillinsol:nn {
9892     \stex_keys_set:nn{fillinsol}{#1}
9893     \ifsolutions
9894
9895     \cs_if_exist:NT\textcolor{\textcolor{red}}{\fbox{\texttt{\tl_to_str:n{#2}}}}
9896     \tl_if_empty:NF \l_problems_fillin_solution_tl {
9897       \footnote{
9898         \halign{ ~~~~\hfil & ~~~~\hfil & ~~~~\hfil & ~~~~\hfil \cr
9899           \textbf{type}&\textbf{case}&\textbf{verdict}&\textbf{feedback}\cr
9900           \tl_if_empty:nF{#2}{
9901             exact & #2 & T & \cr
9902           }
9903           \l_problems_fillin_solution_tl

```

```

9904     }
9905   }
9906 }
9907 \else
9908   \fbox{\dim_compare:nNnTF\l_stex_key_testspace_dim={0pt}{
9909     \phantom{\huge\tl_to_str:n{#2}}}}
9910   }{
9911     \hspace{\l_stex_key_testspace_dim}\vphantom{\huge{A \tl_to_str:n{#2}}}}
9912   }}
9913 \fi
9914 }
9915 }
9916
9917 \stex_deactivate_macro:Nn \fillinsol {problem-environments}

```

(End of definition for `\fillinsol`. This function is documented on page 117.)

`\testemptypage`

```

9918 \newcommand\testemptypage[1][\%
9919 \bool_if:NT \c__problems_test_bool {\vfill\begin{center}\hwexam@kw@testemptypage\end{center}}
9920 }

```

(End of definition for `\testemptypage`. This function is documented on page ??.)

`\testspace`

```

9921 \newcommand\testspace[1]{\bool_if:NT \c__problems_test_bool {\vspace*{#1}}}
9922 \newcommand\testsmallspace{\testspace{1cm}}
9923 \newcommand\testmedspace{\testspace{2cm}}
9924 \newcommand\testbigspace{\testspace{3cm}}

```

(End of definition for `\testspace`. This function is documented on page ??.)

`\testnewpage`

```

9925 \newcommand\testnewpage{\bool_if:NT \c__problems_test_bool {\newpage}}

```

(End of definition for `\testnewpage`. This function is documented on page ??.)

`\testnewpage`

```

9926 \newcommand{\testnewpageInProblem}%
9927 {\ifintest\vfill\begin{center}\emph{continued-on-next-page}\end{center}\testnewpage\fi}%

```

(End of definition for `\testnewpage`. This function is documented on page ??.)

```

9928 \end{package}

```

39.3 Implementation: The hwexam Package

39.3.1 Package Options

The first step is to declare (a few) package options that handle whether certain information is printed or not. Some come with their own conditionals that are set by the options, the rest is just passed on to the `problems` package.

```

9929 \*package
9930 \ProvidesExplPackage{hwexam}{2026/06/24}{4.1.0}{homework assignments and exams}

```

```

9931 \RequirePackage{l3keys2e}
9932
9933 \keys_define:nn {hwexam / pkg}{
9934   multiple .default:n = { false },
9935   multiple .bool_set:N = \c_hwexam_multiple_bool,
9936   qrcode .default:n = { false },
9937   qrcode .bool_set:N = \c_hwexam_qrcode_bool,
9938   unknown .code:n = {
9939     \PassOptionsToPackage{\CurrentOption}{problem}
9940   }
9941 }
9942 \ProcessKeysOptions{ hwexam /pkg }
9943 \RequirePackage{problem}

```

`\hwexam_kw_*` For multilinguality, we define internal macros for keywords that can be specialized in `*.ldf` files.

```

9944 \AddToHook{begindocument}{
9945   \ExplSyntaxOn\makeatletter
9946   \input{hwexam-english.ldf}
9947   \ltx@ifpackageloaded{babel}{
9948     \clist_set:Nx \l_tmpa_clist {\exp_args:No \tl_to_str:n \bbl@loaded}
9949     \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{ngerman}}{
9950       \input{hwexam-ngerman.ldf}
9951     }
9952     \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{finnish}}{
9953       \input{hwexam-finnish.ldf}
9954     }
9955     \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{french}}{
9956       \input{hwexam-french.ldf}
9957     }
9958     \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{russian}}{
9959       \input{hwexam-russian.ldf}
9960     }
9961   }{}
9962   \makeatother\ExplSyntaxOff
9963 }

```

(End of definition for `\hwexam_kw_*`. This function is documented on page ??.)

39.3.2 QR Codes

```

9964 \group_begin:
9965   \escapechar=-1
9966   \xdef\__qr_backslash{\string\}
9967 \group_end:
9968
9969 \bool_if:NT \c_hwexam_qrcode_bool {
9970   \RequirePackage{qrcode}
9971   \RequirePackage{marginnote}
9972   \str_new:N \g__qr_json_str
9973   \bool_new:N \g__qr_in_problems_json_bool
9974   \bool_set_false:N \g__qr_in_problems_json_bool
9975   \bool_new:N \g__qr_in_subproblems_json_bool
9976   \bool_set_false:N \g__qr_in_subproblems_json_bool

```

```

9977 \bool_new:N \g_@@_qr_in_anscls_json_bool
9978 \bool_set_false:N \g_@@_qr_in_anscls_json_bool
9979 \bool_if:NTF \c__problems_gnotes_bool {
9980   \gdef \qrjson { \str_gput_right:Nx \g_@@_qr_json_str}
9981 }{
9982   \gdef \qrjson #1 {}
9983 }
9984 \qrjson{[]}
9985 \AtEndDocument{\qrjson{}}
9986 \def\__qr_escape_char:n #1 {
9987   #1\exp_args:Nno\use:nn{\if_charcode:w#1}\__qr_backslash#1\fi
9988 }
9989
9990 \gdef\__qr_escape:n #1{\exp_args:Ne\str_map_function:nN{\tl_to_str:n{#1}}\__qr_escape_
9991 \gdef \__qr_escape:o #1{\exp_args:No\__qr_escape:n#1}
9992
9993 \def\qrschema{T0:D0:\examnumber}
9994
9995 \bool_if:NTF \c__problems_test_bool {
9996   \def\doproblemqr{
9997     \ifstexhtml\else{
9998       \reversemarginpar\marginnote{\qrcline[height=1.5cm]{\qrschema}}
9999       \normalmarginpar
10000     }\fi
10001   }
10002   \def\insertexamnumber{
10003     \ifstexhtml\else
10004       \tl_if_exist:NTF \examnumber {
10005         {\Large\bfseries ID:~\examnumber}
10006       }{
10007         {\color{red}\Large\bfseries!!!~ WARNING:~NO~{\string\examnumber}~SET~!!!}\gdef\examnum
10008       }
10009       \global\def\insertexamnumber{}
10010     \fi
10011   }
10012 }{
10013   \def\doproblemqr{}
10014   \def\insertexamnumber{}
10015 }
10016
10017 \stexstyleproblem[noqr]{
10018   \par\noindent\problemheader \xdef\grid{\thesproblem}
10019   \bool_if:NT \c__problems_pts_bool {
10020     \tl_if_eq:NnF \l__problems_pts_tl {0}{
10021       \marginpar{\l__problems_pts_tl}{~\problem@kw@points\smallskip}
10022     }
10023   }
10024   \bool_if:NT \c__problems_min_bool {
10025     \tl_if_eq:NnF \l__problems_min_tl {0}{
10026       \marginpar{\l__problems_min_tl}{~\problem@kw@minutes\smallskip}
10027     }
10028   }
10029
10030   \bool_if:NTF \g_@@_qr_in_problems_json_bool {

```

```

10031 \qrjson{,}
10032 }{
10033 \bool_gset_true:N \g_@@_qr_in_problems_json_bool
10034 }
10035
10036 \qrjson {
10037 \c_left_brace_str
10038 "id": "\_@@_qr_escape:o\l_stex_key_id_str", "title": "\_@@_qr_escape:o\l_stex_key_title_tl
10039 "number": "\thesproblem"
10040 }
10041
10042 \tl_if_eq:NnF \l__problems_pts_tl {0}{
10043 \qrjson {
10044 , "pts": \l__problems_pts_tl
10045 }
10046 }
10047 \par
10048 \stex_ignore_spaces_and_pars:
10049 }{
10050 \bool_if:NT \g_@@_qr_in_subproblems_json_bool {
10051 \qrjson {}}
10052 }
10053 \bool_gset_false:N \g_@@_qr_in_subproblems_json_bool
10054 \qrjson {\c_right_brace_str}
10055 \par\bigskip
10056 }
10057
10058 \stexstyleproblem{
10059 \par\noindent\problemheader \xdef\grid{\thesproblem}
10060 \doproblemqr
10061 \bool_if:NT \c__problems_pts_bool {
10062 \tl_if_eq:NnF \l__problems_pts_tl {0}{
10063 \marginpar{\l__problems_pts_tl}{~\problem@kw@points\smallskip}
10064 }
10065 }
10066 \bool_if:NT \c__problems_min_bool {
10067 \tl_if_eq:NnF \l__problems_min_tl {0} {
10068 \marginpar{\l__problems_min_tl}{~\problem@kw@minutes\smallskip}
10069 }
10070 }
10071
10072 \bool_if:NTF \g_@@_qr_in_problems_json_bool {
10073 \qrjson{,}
10074 }{
10075 \bool_gset_true:N \g_@@_qr_in_problems_json_bool
10076 }
10077
10078 \qrjson {
10079 \c_left_brace_str
10080 "id": "\_@@_qr_escape:o\l_stex_key_id_str", "title": "\_@@_qr_escape:o\l_stex_key_title_tl
10081 "number": "\thesproblem"
10082 }
10083
10084 \tl_if_eq:NnF \l__problems_pts_tl {0}{

```



```

10085 \qrjson {
10086   , "pts": \l__problems_pts_tl
10087 }
10088 }
10089 \par
10090 \stex_ignore_spaces_and_pars:
10091 }{
10092   \bool_if:NT \g_@@_qr_in_subproblems_json_bool {
10093     \qrjson {}
10094   }
10095   \bool_gset_false:N \g_@@_qr_in_subproblems_json_bool
10096   \qrjson {\c_right_brace_str}
10097   \par\bigskip
10098 }
10099
10100 \stexstylesubproblem[noqr]{
10101   \begin{list}{}{
10102     \setlength\topsep{0pt}
10103     \setlength\parsep{0pt}
10104     \setlength\rightmargin{0pt}
10105   } \item[\int_use:N \g__problems_subproblem_int .]
10106   \xdef\grid{\thesproblem.\int_use:N \g__problems_subproblem_int}
10107   \bool_if:NT \c__problems_pts_bool {
10108     \bool_if:NF \l__problems_has_pts_bool {
10109       \marginpar{\smallskip\l_stex_key_pts_tl}{~\problem@kw@points}
10110     }
10111   }
10112   \bool_if:NT \c__problems_min_bool {
10113     \bool_if:NF \l__problems_has_min_bool{
10114       \marginpar{\smallskip\l_stex_key_min_tl}{~\problem@kw@minutes}
10115     }
10116   }
10117   \bool_if:NTF \g_@@_qr_in_subproblems_json_bool {
10118     \qrjson {,}
10119   }{
10120     \qrjson {
10121       , "subproblems": [
10122       ]
10123       \bool_gset_true:N \g_@@_qr_in_subproblems_json_bool
10124     }
10125     \qrjson {
10126       \c_left_brace_str
10127       "id": "\_@@_qr_escape:o\l_stex_key_id_str", "title": "\_@@_qr_escape:o\l_stex_key_title_tl
10128       "number": "\thesproblem.\int_use:N \g__problems_subproblem_int"
10129     }
10130     \bool_if:NF \l__problems_has_pts_bool {
10131       \qrjson {
10132         , "pts": \l_stex_key_pts_tl
10133       }
10134     }
10135   }{
10136     \qrjson {\c_right_brace_str}
10137   \end{list}
10138 }

```

```

10139
10140 \stexstylesubproblem{
10141   \begin{list}{}{
10142     \setlength\topsep{0pt}
10143     \setlength\parsep{0pt}
10144     \setlength\rightmargin{0pt}
10145   }\item[\int_use:N \g__problems_subproblem_int .]
10146   \xdef\grid{\thesproblem.\int_use:N \g__problems_subproblem_int}
10147   \doproblemqr
10148   \bool_if:NT \c__problems_pts_bool {
10149     \bool_if:NF \l__problems_has_pts_bool {
10150       \marginpar{\smallskip\l_stex_key_pts_tl}{~\problem@kw@points}
10151     }
10152   }
10153   \bool_if:NT \c__problems_min_bool {
10154     \bool_if:NF \l__problems_has_min_bool{
10155       \marginpar{\smallskip\l_stex_key_min_tl}{~\problem@kw@minutes}
10156     }
10157   }
10158   \bool_if:NTF \g_@@_qr_in_subproblems_json_bool {
10159     \qrjson {,}
10160   }{
10161     \qrjson {
10162       , "subproblems": [
10163     }
10164     \bool_gset_true:N \g_@@_qr_in_subproblems_json_bool
10165   }
10166   \qrjson {
10167     \c_left_brace_str
10168     "id": "\_@@_qr_escape:o\l_stex_key_id_str", "title": "\_@@_qr_escape:o\l_stex_key_title_tl"
10169     "number": "\thesproblem.\int_use:N \g__problems_subproblem_int"
10170   }
10171   \bool_if:NF \l__problems_has_pts_bool {
10172     \qrjson {
10173       , "pts": \l_stex_key_pts_tl
10174     }
10175   }
10176 }{
10177   \qrjson {\c_right_brace_str}
10178   \end{list}
10179 }
10180
10181 \stexstylegnote{
10182   \par\smallskip\rule[.3em]{\linewidth}{0.4pt}\newline\smallskip
10183   \noindent\emph{\problem@kw@grading\str_if_empty:NF \l_stex_key_title_tl{
10184     {~}\l_stex_key_title_tl
10185   } :~}
10186   \qrjson {
10187     , "gnote": \c_left_brace_str
10188     "id": "\_@@_qr_escape:o\l_stex_key_id_str", "title": "\_@@_qr_escape:o\l_stex_key_title_tl"
10189     "anscls": [
10190   }
10191 }{
10192   \qrjson {

```

```

10193   ]\c_right_brace_str
10194 }
10195 \par\rule[.3em]{\linewidth}{0.4pt}\newline
10196 }
10197
10198 \renewcommand \anscls [2] [] {
10199   \stex_keys_set:nn{ anscls }{#1}
10200   \str_if_empty:NT \l_stex_key_id_str {
10201     \int_incr:N \l__problems_anscls_int
10202     \str_set:Nx \l_stex_key_id_str {
10203       AC\int_use:N \l__problems_anscls_int
10204     }
10205   }
10206   \bool_if:NTF \g_@@_qr_in_anscls_json_bool {
10207     \qrjson{,}
10208   }{
10209     \bool_set_true:N \g_@@_qr_in_anscls_json_bool
10210   }
10211   \qrjson{
10212     \c_left_brace_str
10213     "id": "\_@@_qr_escape:o\l_stex_key_id_str", "description": "\_@@_qr_escape:n{#2}",
10214     "feedback": "\_@@_qr_escape:o\l_stex_key_feedback_tl",
10215     "pts": "\l_stex_key_pts_str"
10216     \c_right_brace_str
10217   }
10218   \begin{list}{}{
10219     \setlength\topsep{Opt}
10220     \setlength\parsep{Opt}
10221     \setlength\rightmargin{Opt}
10222   }\item[\l_stex_key_id_str]
10223     \stex_if_do_html:TF{
10224       \exp_args:Ne \stex_annotate:nn{
10225         data-ftml-answerclass={\l_stex_key_id_str}
10226         \str_if_empty:NF \l_stex_key_pts_str{
10227           ,data-ftml-answerclass-pts={\l_stex_key_pts_str}
10228         }
10229       }{
10230         #2
10231         \tl_if_empty:NF \l_stex_key_feedback_tl{
10232           \stex_annotate_invisible:nn{
10233             data-ftml-answerclass-feedback={true}
10234           }{\l_stex_key_feedback_tl}
10235         }
10236       }
10237     }{#2}
10238     \str_if_empty:NF \l_stex_key_pts_str {\par
10239       ~ \problem@kw@points :~\l_stex_key_pts_str
10240     }
10241     \str_if_empty:NF \l_stex_key_feedback_tl {\par
10242       ~ \problem@kw@feedback :~\l_stex_key_feedback_tl
10243     }
10244   \end{list}
10245 }
10246 \stex_deactivate_macro:Nn \anscls {gnote-environments}

```

```

10247
10248 \AtEndDocument{
10249   \message{^^J^^J\g_@@_qr_json_str^^J^^J}
10250   \bool_if:NT \c__problems_gnotes_bool {
10251     \iow_new:N \c_@@_qr_json_iow
10252     \iow_open:Nn \c_@@_qr_json_iow {\jobname-vollkorn.json}
10253     \iow_now:Nx \c_@@_qr_json_iow {\g_@@_qr_json_str}
10254     \iow_close:N \c_@@_qr_json_iow
10255   }
10256 }
10257
10258 \renewcommand\testemptypage[1][\%
10259   \bool_if:NT \c__problems_test_bool {\
10260     \xdef\grid{P\thepage}
10261     \doprobblemqr
10262     \vfill\begin{center}\hwexam@kw@testemptypage\end{center}\eject
10263   }
10264 }
10265 }

```

39.3.3 Assignments

Then we set up a counter for problems and make the problem counter inherited from `problem.sty` depend on it. Furthermore, we specialize the `\prob@label` macro to take the assignment counter into account.

`assignment (env.)`

```

10266 \stex_keys_define:nnnn{ assignment }{
10267   \tl_clear:N \l_stex_key_number_tl
10268   \tl_clear:N \l_stex_key_given_tl
10269   \tl_clear:N \l_stex_key_due_tl
10270 }{
10271   number .tl_set:N      = \l_stex_key_number_tl,
10272   given   .tl_set:N      = \l_stex_key_given_tl,
10273   due     .tl_set:N      = \l_stex_key_due_tl,
10274   unknown .code:n = {}
10275 }{id,title,style}
10276
10277 \newcounter{assignment}
10278 \stex_new_stylable_env:nnnnnn {assignment}{0}{\{
10279   \cs_if_exist:NTF \l_hwexam_includeassignment_keys_tl {
10280     \tl_put_left:Nn \l_hwexam_includeassignment_keys_tl {\#1,}
10281     \exp_args:Nno \stex_keys_set:nn{assignment}{
10282       \l_hwexam_includeassignment_keys_tl
10283     }
10284   }{
10285     \stex_keys_set:nn{assignment}{\#1}
10286   }
10287   \tl_if_empty:NF \l_stex_key_number_tl {
10288     \global\setcounter{assignment}{\int_eval:n{\l_stex_key_number_tl-1}}
10289   }
10290   \global\refstepcounter{assignment}
10291   \setcounter{sproblem}{0}
10292   \def\thesproblem{\theassignment.\arabic{sproblem}}

```

```

10293 \stex_if_do_html:T{
10294   \tl_if_empty:NF \l_stex_key_title_tl {
10295     \stexdoctitle \l_stex_key_title_tl
10296   }
10297 }
10298 \stex_style_apply:
10299 \_stex_do_id:
10300 }{
10301 \stex_style_apply:
10302 }{
10303 \par\begin{center}
10304 \textbf{\Large\assignmentautorefname~\theassignment
10305   \tl_if_empty:NF \l_stex_key_title_tl {
10306     {~}---\l_stex_key_title_tl
10307   }
10308 }\par\smallskip
10309 \textbf{
10310   \tl_if_empty:NF \l_stex_key_given_tl {
10311     \hwexam@kw@given :~\l_stex_key_given_tl\quad
10312   }
10313   \tl_if_empty:NF \l_stex_key_due_tl {
10314     \hwexam@kw@due :~\l_stex_key_due_tl\quad
10315   }
10316 }
10317 \end{center}
10318 \par\bigskip
10319 }{
10320 \par\pagebreak
10321 }{}

```

`\includeassignment`

```

10322 \NewDocumentCommand\includeassignment{0{} m}{
10323   \group_begin:
10324   \tl_set:Nn \l_hwexam_includeassignment_keys_tl {#1}
10325   \stex_keys_set:nn{includeproblem}{#1}
10326   \exp_args:Nno \use:nn{\inputref[]\l_stex_key_mhrepos_str}{#2}
10327   \group_end:
10328 }

```

(End of definition for \includeassignment. This function is documented on page 76.)

Restoring information about problems:

```

10329 \prop_new:N \c_@@_problems_prop
10330 \tl_set:Nn \c_@@_total_mins_tl {0pt}
10331 \tl_set:Nn \c_@@_total_pts_tl {0pt}
10332 \int_new:N \c_@@_total_problems_int
10333 \cs_set_protected:Npn \problem@restore #1 #2 #3 {
10334   \int_gincr:N \c_@@_total_problems_int
10335   \prop_gput:Nnn \c_@@_problems_prop {#1}{#2}{#3}
10336   \tl_gset:Ne \c_@@_total_pts_tl { \dim_eval:n { \c_@@_total_pts_tl + #2pt }}
10337   \tl_gset:Ne \c_@@_total_mins_tl { \dim_eval:n { \c_@@_total_mins_tl + #2pt }}
10338 }

```

`\correction@table` This macro generates the correction table

```

10339 \newcommand\correction@table{

```

```

10340 \int_compare:nNnT \c_@@_total_problems_int = 0 {
10341   \int_incr:N \c_@@_total_problems_int
10342   \prop_put:Nnn \c_@@_problems_prop {~}{~}{~}}
10343 }
10344 \tl_clear:N \l_tmpa_tl
10345 \tl_clear:N \l_tmpb_tl
10346 \tl_clear:N \l_tmpc_tl
10347 \prop_map_inline:Nn \c_@@_problems_prop {
10348   \tl_put_right:Nn \l_tmpa_tl { ##1 & }
10349   \tl_put_right:Nx \l_tmpb_tl { \use_i:nn ##2 & }
10350   \tl_put_right:Nn \l_tmpc_tl { & }
10351 }
10352 \resizebox{\textwidth}{!}{%
10353 \exp_args:Nne \begin{tabular}{|l|*{\int_use:N \c_@@_total_problems_int}{c|}c|l|}\hline
10354 &\exp_args:Ne \multicolumn{\int_eval:n{ \c_@@_total_problems_int + 1}}{c|}
10355 {\footnotesize\hwexam@kw@forgrading} &\\ \hline
10356 \hwexam@kw@probs & \l_tmpa_tl \hwexam@kw@sum & \hwexam@kw@grade\\ \hline
10357 \hwexam@kw@pts & \l_tmpb_tl \dim_to_decimal:n{\c_@@_total_pts_tl} & \\ \hline
10358 \hwexam@kw@reached & \l_tmpc_tl & \\ [.7cm] \hline
10359 \end{tabular}}

```

(End of definition for \correction@table. This function is documented on page ??.)

\testheading

```

10360 \def\hwexamheader{\input{hwexam-default.header}}
10361
10362 \def\hwexamminutes{
10363   \tl_if_empty:NTF \hwexam@duration {
10364     {\hwexam@min}~\hwexam@minutes@kw
10365   }{
10366     \hwexam@duration
10367   }
10368 }
10369
10370 \stex_keys_define:nnnn{ hwexam / testheading }{
10371   \tl_clear:N \hwexam@min
10372   \tl_clear:N \hwexam@duration
10373   \tl_clear:N \hwexam@reqpts
10374   \tl_clear:N \hwexam@tools
10375 }{
10376   min .tl_set:N = \hwexam@min,
10377   duration .tl_set:N = \hwexam@duration,
10378   reqpts .tl_set:N = \hwexam@reqpts,
10379   tools .tl_set:N = \hwexam@tools
10380 }{}
10381
10382 \newenvironment{testheading}[1][]{
10383   \stex_keys_set:nn { hwexam / testheading}{#1}
10384
10385   \tl_set:Nx \hwexam@totalpts {\dim_to_decimal:n \c_@@_total_pts_tl}
10386   \tl_set:Nx \hwexam@totalmin {\dim_to_decimal:n \c_@@_total_mins_tl}
10387   \tl_set:Nx \hwexam@checktime {\dim_to_decimal:n { \hwexam@min pt - \hwexam@totalmin pt }}
10388
10389   \newif\if@bonuspoints

```

```

10390 \tl_if_empty:NTF \hwexam@reqpts {
10391   \@bonuspointsfalse
10392 }{
10393   \tl_set:Nx \hwexam@bonuspts {
10394     \dim_to_decimal:n{\hwexam@totalpts pt - \hwexam@reqpts pt}
10395   }
10396   \@bonuspointstrue
10397 }
10398
10399 \makeatletter\hwexamheader\makeatother
10400 }{
10401   \newpage
10402 }

```

(End of definition for \testheading. This function is documented on page ??.)

```

examdata
quizdata
homeworkdata
10403 \bool_new:N \g_@@_allow_data_bool
10404 \bool_gset_true:N \g_@@_allow_data_bool
10405 \AtBeginDocument{
10406   \bool_gset_false:N \g_@@_allow_data_bool
10407 }
10408
10409 \msg_set:nnn{stex}{hwexam/doubledata}{
10410   Exam/Homework/Quiz~Data~already~set
10411 }
10412
10413 \msg_set:nnn{stex}{hwexam/nodate}{
10414   Exam/Homework/Quiz~Data~missing~date~value
10415 }
10416
10417 \msg_set:nnn{stex}{hwexam/nocourse}{
10418   Exam/Homework/Quiz~Data~missing~course~value
10419 }
10420
10421 \stex_keys_define:nnnn{ hwexam / commondata }{
10422   % https://docs.rs/dateparser/latest/dateparser/#accepted-date-formats
10423   \str_clear:N \l_@@_key_exam_date_str
10424   \str_clear:N \l_@@_key_exam_course_id_str
10425   \str_clear:N \l_@@_key_exam_term_str
10426   \int_set:Nn \l_@@_key_exam_num_int {1}
10427 }{
10428   date      .str_set:N = \l_@@_key_exam_date_str,
10429   course    .str_set:N = \l_@@_key_exam_course_id_str,
10430   term      .str_set:N = \l_@@_key_exam_term_str,
10431   num       .int_set:N = \l_@@_key_exam_num_int
10432 }{}
10433
10434 \stex_keys_define:nnnn{ hwexam / examdata }{
10435   \bool_set_false:N \l_@@_key_exam_retake_bool
10436 }{
10437   retake    .bool_set:N = \l_@@_key_exam_retake_bool
10438 }{hwexam/commondata}
10439

```

```

10440 \cs_set_protected:Nn \_@@_do_metadata:nnnn {
10441   \bool_if:NF \g_@@_allow_data_bool {
10442     \msg_fatal:nn{stex}{hwexam/doubledata}
10443   }
10444   \bool_gset_false:N \g_@@_allow_data_bool
10445   \stex_keys_set:nn { hwexam / #1}{#2}
10446   \str_set:Nn \g_stex_document_kind_str{#4}
10447   \str_if_empty:NT\l_@@_key_exam_date_str{
10448     \str_set:Ne \l_@@_key_exam_date_str \today
10449     \%msg_fatal:nn{stex}{hwexam/nodate}
10450   }
10451   \str_if_empty:NT\l_@@_key_exam_course_id_str{
10452     \msg_fatal:nn{stex}{hwexam/nocourse}
10453   }
10454   \cs_if_exist:NT \date {
10455     \exp_args:No \date \l_@@_key_exam_date_str
10456   }
10457
10458   \tl_set:Ne \g_stex_document_kind_parameters_tl{
10459     ,data-ftml-document-kind-date={\l_@@_key_exam_date_str}
10460     ,data-ftml-document-kind-num=\int_use:N \l_@@_key_exam_num_int
10461     ,data-ftml-document-kind-course=\l_@@_key_exam_course_id_str
10462     \str_if_empty:NF \l_@@_key_exam_term_str{
10463       ,data-ftml-document-kind-term=\l_@@_key_exam_term_str
10464     }
10465     #3
10466   }
10467 }
10468
10469 \newcommand\examdata[1]{
10470   \_@@_do_metadata:nnnn{examdata}{#1}{
10471     ,data-ftml-document-kind-retake=\bool_if:NTF \l_@@_key_exam_retake_bool{true}{false}
10472   }{exam}
10473 }
10474
10475 \newcommand\homeworkdata[1]{
10476   \_@@_do_metadata:nnnn{commondata}{#1}{}{homework}
10477 }
10478
10479 \newcommand\quizdata[1]{
10480   \_@@_do_metadata:nnnn{commondata}{#1}{}{quiz}
10481 }
10482

```

(End of definition for examdata, quizdata, and homeworkdata. These functions are documented on page ??.)

```

10483 \endpackage

```

39.3.4 Leftovers

at some point, we may want to reactivate the logos font, then we use

```

here we define the logos that characterize the assignment
\font\bierfont=../assignments/bierglas

```



```

\font\denkerfont=../assignments/denker
\font\uhrfont=../assignments/uhr
\font\warnschildfont=../assignments/achtung

\newcommand\bierglas{{\bierfont\char65}}
\newcommand\denker{{\denkerfont\char65}}
\newcommand\uhr{{\uhrfont\char65}}
\newcommand\warnschild{{\warnschildfont\char 65}}
\newcommand\hardA{\warnschild}
\newcommand\longA{\uhr}
\newcommand\thinkA{\denker}
\newcommand\discussA{\bierglas}

```

39.4 Tikzinput Implementation

```

10484 <@@=tikzinput>
10485 <*package>
10486
10487 %%%%%%%%%%%%%% tikzinput.dtx %%%%%%%%%%%%%%
10488
10489 \ProvidesExplPackage{tikzinput}{2025/11/11}{4.0.0}{tikzinput package}
10490 \RequirePackage{l3keys2e}
10491
10492 \keys_define:nn { tikzinput } {
10493   image .bool_set:N = \c_tikzinput_image_bool,
10494   image .default:n = false ,
10495   unknown .code:n = {}
10496 }
10497
10498 \ProcessKeysOptions { tikzinput }
10499
10500 \bool_if:NTF \c_tikzinput_image_bool {
10501   \RequirePackage{graphicx}
10502
10503   \providecommand\usetikzlibrary[]{}
10504   \newcommand\tikzinput[2] [] {\includegraphics[#1]{#2}}
10505 }{
10506   \RequirePackage{tikz}
10507   \RequirePackage{standalone}
10508
10509   \newcommand \tikzinput [2] [] {
10510     \setkeys{Gin}{#1}
10511     \ifx \Gin@ewidth \Gin@exclamation
10512       \ifx \Gin@eheight \Gin@exclamation
10513         \input { #2 }
10514       \else
10515         \resizebox{!}{ \Gin@eheight }{
10516           \input { #2 }
10517         }
10518       \fi
10519     \else

```

```

10520     \ifx \Gin@eheight \Gin@exclamation
10521         \resizebox{ \Gin@ewidth }{!}{
10522             \input { #2 }
10523         }
10524     \else
10525         \resizebox{ \Gin@ewidth }{ \Gin@eheight }{
10526             \input { #2 }
10527         }
10528     \fi
10529 \fi
10530 }
10531 }
10532
10533 \newcommand \ctikzinput [2] [] {
10534     \begin{center}
10535         \tikzinput [#1] {#2}
10536     \end{center}
10537 }
10538 \end{package}

```

Index

The italic numbers denote the pages where the corresponding entry is described, numbers underlined point to the definition, all others indicate the places where it is used.

Symbols	
\!	6682
\\$	2157, 2160, 2164
* commands:	
*:n	145
\,	6682
\:	617
\;	8137, 8144, 8189, 8191, 8221
@@ commands:	
\g_@@_allow_data_bool	10403, 10404, 10406, 10441, 10444
_@@_do_metadata:nnnn	10440, 10470, 10476, 10480
\l_@@_key_exam_course_id_str	10424, 10429, 10451, 10461
\l_@@_key_exam_date_str	10423, 10428, 10447, 10448, 10455, 10459
\l_@@_key_exam_num_int	10426, 10431, 10460
\l_@@_key_exam_retake_bool	10435, 10437, 10471
\l_@@_key_exam_term_str	10425, 10430, 10462, 10463
\c_@@_problems_prop	10329, 10335, 10342, 10347
_@@_qr_escape:n	9990, 9991, 10038, 10080, 10127, 10168, 10188, 10213, 10214
_@@_qr_escape_char:n	9986, 9990
\g_@@_qr_in_anscls_json_bool	9977, 9978, 10206, 10209
\g_@@_qr_in_problems_json_bool	9973, 9974, 10030, 10033, 10072, 10075
\g_@@_qr_in_subproblems_json_-bool	9975, 9976, 10050, 10053, 10092, 10095, 10117, 10123, 10158, 10164
\c_@@_qr_json_iow	10251, 10252, 10253, 10254
\g_@@_qr_json_str	9972, 9980, 10249, 10253
\c_@@_total_mins_tl	10330, 10337, 10386
\c_@@_total_problems_int	10332, 10334, 10340, 10341, 10353, 10354
c@@_total_pts_tl	10331, 10336, 10357, 10385
\\	88, 115, 842, 1464, 8372, 8377, 8795, 8798, 9588, 9649, 9783, 9966, 10355, 10356, 10357, 10358
\{	8194
\}	8194
\sqcup	19, 79, 619, 756, 8148, 8226, 8404, 9919, 10259
_	618
_@@_qr_backslash	9966, 9987
_comp	3877, 4635, 5922, 6137, 6289, 7034
_customthiscomp	4995, 4996, 5008, 5013, 5016, 5021
_defcomp	5955, 6121
_thiscomp	4764, 4765, 5926
_varcomp	4322, 4481, 5252, 5273, 5823, 5944, 6052, 6066, 6080
\l	2156, 2159, 2163
A	
\activateexcursion	64, 112
\addbibresource	82, 141, 2173
\addcontentsline	8402
\addmhbibresource	82, 141, 2166
\addtocounter	8597
\AddToHook	1793, 7791, 7931, 7935, 7939, 8066, 8989, 9944
\aftergroup	125, 126, 2922, 2944, 2993, 4598, 7857, 7858, 8062
\afterprematurestop	63, 111, 8466, 8477
\amalg	8211
\anscls	9430, 9463, 9505, 10198, 10246
\apply	90
\arabic	9030, 9033, 10292
\arg	40, 95, 5373
\argarraymap	94, 154, 6369, 6924
\argmap	94, 154, 296, 6368, 6685, 6900
\argsep	36, 94, 154, 5756, 6367, 6680, 6884, 8144, 8145, 8146, 8154, 8201, 8205, 8209
\assign	58, 146, 3302, 3561
assignment (env.)	76, 119, 10266
\assignmentautorefname	10304
\assignMorphism	146, 3303, 3666
\assumption	158, 7687, 8070

`\ast` 8169
`\AtBeginDocument` 69, 77,
 806, 814, 1483, 1510, 1691, 1716,
 1797, 2940, 8248, 8421, 8936, 10405
`\AtEndDocument` ... 627, 1798, 9985, 10248
`\AtEndOfPackageFile` 2209, 2225, 2240
`\author` 8775
`\autoref` 86, 1862, 1878

B

`\backmatter` 210, 1758, 1759, 1761
`\baselineskip` 8733
`\beameritemnestingprefix` 8873
`\begin` 98, 128, 139, 142–144,
 157, 1470, 1582, 1645, 2068, 2223,
 2238, 2258, 2602, 2760, 2778, 2793,
 3057, 6947, 7284, 7948, 8405, 8417,
 8543, 8570, 8580, 8619, 8640, 8648,
 8694, 8695, 8696, 8697, 8698, 8699,
 8705, 8707, 8745, 8747, 8751, 8813,
 8825, 8860, 8861, 9071, 9171, 9198,
 9251, 9314, 9362, 9413, 9472, 9491,
 9519, 9529, 9684, 9800, 9805, 9810,
 9815, 9919, 9927, 10101, 10141,
 10218, 10262, 10303, 10353, 10534
`\begingroup` 46, 2076, 2581
`\bf` 8732
`\bfseries` 9155, 10005, 10007
`\bgroup` .. 1899, 1980, 4365, 4538, 7700,
 7824, 7913, 8569, 8665, 8671, 8687,
 8795, 8798, 9268, 9328, 9376, 9427
`\bigcirc` 9734, 9760
`\bigskip` 9124, 10055, 10097, 10318
`blindfragment (env.)` 84, 139, 1628
 bool commands:
 `\bool_gset_eq:NN` 3113, 7833
 `\bool_gset_false:N`
 10053, 10095, 10406, 10444
 `\bool_gset_true:N`
 .. 10033, 10075, 10123, 10164, 10404
 `\bool_if:NTF` 473, 581, 600,
 601, 607, 696, 724, 978, 987, 989,
 1083, 1626, 1743, 2383, 2435, 2663,
 2978, 3174, 3314, 3346, 3710, 4380,
 4605, 5247, 5271, 5552, 5562, 5568,
 5578, 5582, 5602, 5616, 5620, 5634,
 5663, 5670, 5684, 6126, 6149, 6207,
 6537, 6963, 6969, 7571, 7700, 7710,
 7726, 7759, 7765, 7793, 7849, 7889,
 7913, 7916, 7923, 7927, 7941, 7988,
 8076, 8308, 8316, 8353, 8437, 8481,
 8662, 8668, 8675, 8709, 8724, 8774,
 8824, 8834, 8921, 8927, 8954, 8977,
 9053, 9074, 9110, 9115, 9125, 9153,

 9174, 9188, 9191, 9203, 9204, 9208,
 9209, 9324, 9335, 9372, 9383, 9424,
 9435, 9507, 9512, 9602, 9606, 9629,
 9643, 9740, 9744, 9763, 9777, 9919,
 9921, 9925, 9969, 9979, 9995, 10019,
 10024, 10030, 10050, 10061, 10066,
 10072, 10092, 10107, 10108, 10112,
 10113, 10117, 10130, 10148, 10149,
 10153, 10154, 10158, 10171, 10206,
 10250, 10259, 10441, 10471, 10500
`\bool_if_exist:NTF` 339, 353
`\bool_lazy_any:NTF` 4211, 4280
`\bool_lazy_any_p:n` 285
`\bool_new:N` 27, 685, 1312,
 1606, 2374, 2660, 3020, 4299, 4626,
 5549, 5963, 6957, 7179, 9009, 9048,
 9049, 9050, 9973, 9975, 9977, 10403
`\bool_not_p:n` 284, 288
`\bool_set_false:N` 455,
 582, 693, 701, 975, 998, 1314, 1621,
 2388, 3027, 3327, 3683, 4629, 5979,
 6277, 6525, 6816, 6964, 7192, 7534,
 7539, 7549, 7554, 7578, 7664, 7799,
 7877, 8060, 8125, 8126, 8292, 8333,
 8497, 9015, 9051, 9091, 9094, 9571,
 9576, 9582, 9974, 9976, 9978, 10435
`\bool_set_true:N` 341, 355,
 463, 571, 577, 690, 705, 972, 1352,
 1611, 2386, 2703, 3023, 3311, 3695,
 3705, 4028, 4385, 4627, 4696, 4966,
 4998, 5023, 5065, 5066, 5358, 5367,
 5482, 5583, 5589, 5607, 5625, 5658,
 5781, 5871, 5904, 5929, 5930, 5968,
 6204, 6532, 6981, 7182, 7190, 7586,
 7596, 7777, 8305, 8345, 8495, 8503,
 8504, 8505, 8506, 8507, 8508, 9087,
 9092, 9095, 9158, 9569, 9581, 10209
`\bool_while_do:Nn` 973
`\bool_while_do:nn`
 283, 2189, 7601, 7613, 7625, 7642, 7654
`\l_tmpa_bool` 3683,
 3695, 3705, 3710, 6525, 6532, 6537
 box commands:
 `\box_wd:N` 9882
 `\c_empty_box`
 1530, 1540, 3903, 9509, 9514
 `\l_tmpa_box`
 ... 2702, 6359, 8665, 8671, 8687,
 9268, 9328, 9376, 9427, 9879, 9882
 boxed 113

C

`\catcode` 19, 20, 46,
 79, 88, 617, 618, 619, 756, 757, 842,

1981, 2155, 2156, 2157, 2158, 2159,	8152, 8154, 8156, 8157, 8162, 8164,
2160, 2162, 2163, 2164, 2494, 2495	8169, 8173, 8174, 8177, 8178, 8183,
\cdot 8189	8184, 8190, 8191, 8192, 8194, 8201,
\centering 8406, 8620	8205, 8221, 8226, 8230, 8232, 8234
\chapter 26, 84, 8446, 8447	\comp-macro 151
\chaptername 8377, 8443, 8444	\compemph 105, 151, <u>5914</u> , 8938
\chaptertitlename 8377	\conclude 158, 7686, <u>8071</u>
\checkmark 9631, 9632, 9765, 9766	\conclusion .. 44, 101, 103, 157, 7423, <u>7497</u>
\child 125	\copymod 55–57, 145, 3489, 3491, 3493
\circ 47	copymodule (env.) 145, <u>3429</u>
\clearpage 1752, 1765, 1776, 1787	\copymodule 3464, 3466
clist commands:	\counterwithin 8450, 8462
\clist_clear:N ... 125, 399, 3751,	\cr 9898, 9899, 9901
4697, 4847, 4947, 6258, 7254, 7259	\crrcr 9857
\clist_count:N 4826, 4837	cs commands:
\clist_count:n 4930, 6932	\cs:w 483, 486, 517, 520,
\clist_get:NN 508, 546	620, 2439, 2440, 2441, 2447, 2451,
\clist_if_empty:NTF 454, 504, 542, 4044, 4825,	2457, 2716, 7133, 7135, 7313, 7327
4872, 5192, 6456, 7339, 9531, 9695	\cs_argument_spec:N 4163
\clist_if_in:NnTF 104, 107, 131, 7434, 8994, 8997,	\cs_end: 483, 486, 517, 520,
9000, 9003, 9536, 9539, 9664, 9671,	620, 2439, 2440, 2441, 2449, 2453,
9701, 9704, 9949, 9952, 9955, 9958	2457, 2716, 7133, 7135, 7313, 7327
\clist_if_in:nntf 4956	\cs_generate_from_arg_count:NNnn
\clist_item:Nn 5143, 6465	 4037,
\clist_item:nn 6943	4394, 4553, 4730, 5491, 5527, 6812
\clist_map_function:NN ... 4698, 4875	\cs_generate_variant:Nn 56,
\clist_map_function:nN 3479, 3545, 4129, 6632	146, 176, 248, 331, 732, 1330, 2533,
\clist_map_inline:Nn 126, 135, 457, 3214, 4046, 4403,	2539, 3017, 4603, 4625, 5486, 6515
4562, 5193, 7057, 7092, 7237, 7272	\cs_if_eq:NNTF .. 65, 73, 802, 810,
\clist_map_inline:nn 363, 412, 5707, 5810, 5839, 7055, 7489	1200, 1205, 1688, 2746, 2760, 2763,
\clist_pop:NN 6467	2778, 2781, 4165, 5740, 5742, 6895
\clist_put_right:Nn 127, 4711, 4715, 4854, 4974, 4976	\cs_if_exist:NTF 675, 1563, 1567, 1610, 1614, 1630,
\clist_set:Nn 42, 123, 4467, 8993, 9948	1633, 1662, 1666, 1700, 1706, 1717,
\clist_set_eq:NN 121, 7289	1745, 1758, 1773, 1784, 1832, 1833,
\l_tmpa_clist 8993, 8994, 8997, 9000,	1861, 1919, 1960, 2034, 2035, 2037,
9003, 9948, 9949, 9952, 9955, 9958	2132, 3905, 4160, 4667, 5222, 5225,
\clstinptmhlisting 85, <u>141</u> , <u>2225</u>	5233, 5236, 6544, 6549, 6581, 6587,
\cmhgraphics 85, <u>141</u> , <u>2209</u>	6611, 6613, 6653, 6659, 8372, 8377,
\cmhtikzinput 121, <u>141</u> , <u>2240</u>	8380, 8384, 8388, 8392, 8396, 8443,
\color 10007	8446, 8449, 8455, 8458, 8461, 8622,
\columnbox 8638, 8640, 8658	8630, 8642, 8652, 8661, 8795, 8798,
\comp 31–34, 40, 92, 95, 105, 151, 3877, 4322,	9056, 9631, 9765, 9895, 10279, 10454
4481, 4646, 4647, 4765, 4813, 4821,	\cs_meaning:N . 2438, 4071, 6518, 6667
5004, 5005, 5015, 5689, 5823, <u>5914</u> ,	\cs_new:Nn 26, 34, 205, 306, 649, 763, 771,
5966, 5967, 6052, 6066, 6080, 6121,	888, 895, 1098, 1105, 1117, 1120,
6137, 6139, 6289, 6666, 6760, 6766,	1123, 1126, 1137, 1140, 1143, 1146,
6768, 6982, 6987, 7034, 8141, 8143,	1149, 1155, 1161, 1238, 1241, 1244,
	1251, 1254, 1257, 1260, 1264, 1285,
	1288, 1299, 1307, 1407, 1410, 1413,
	1421, 1424, 1427, 1430, 1433, 1436,
	1440, 1443, 1446, 1964, 1976, 2261,
	2264, 2267, 2270, 2273, 2276, 2279,

2282, 2285, 2465, 2470, 2475, 3142,
 3146, 3941, 4104, 4740, 4883, 4890,
 4893, 4903, 5402, 5499, 5686, 6004,
 6029, 6470, 6481, 6495, 6498, 6554,
 6567, 6717, 6720, 6724, 6771, 7161,
 7330, 7344, 7485, 9035, 9837, 9855
 \cs_new:Npn 726,
 727, 735, 736, 881, 884, 891, 1006,
 1152, 1158, 1291, 1972, 2001, 3290,
 5735, 6008, 6168, 6560, 6573, 7217
 \cs_new_nopar:Nn 359, 374
 \cs_new_protected:Nn
 3, 4, 42, 45, 53, 67,
 73, 77, 87, 88, 97, 98, 103, 113, 141,
 153, 169, 177, 192, 208, 215, 245,
 249, 313, 324, 333, 346, 482, 495,
 503, 516, 541, 556, 599, 611, 625,
 630, 654, 699, 729, 740, 741, 779,
 825, 834, 898, 920, 928, 944, 957,
 968, 985, 995, 1010, 1164, 1170,
 1188, 1199, 1218, 1271, 1275, 1313,
 1332, 1359, 1372, 1384, 1392, 1495,
 1504, 1579, 1587, 1591, 1604, 1628,
 1643, 1698, 1744, 1811, 1830, 1850,
 1860, 1871, 1898, 1908, 1917, 1945,
 1978, 1990, 2025, 2051, 2065, 2075,
 2142, 2151, 2177, 2313, 2350, 2359,
 2376, 2382, 2391, 2406, 2418, 2434,
 2444, 2482, 2501, 2505, 2513, 2527,
 2530, 2534, 2552, 2556, 2590, 2600,
 2613, 2623, 2628, 2633, 2647, 2653,
 2668, 2700, 2711, 2720, 2731, 2738,
 2745, 2751, 2773, 2790, 2798, 2807,
 2816, 2825, 2848, 2863, 2878, 2905,
 2924, 2950, 2956, 2976, 2997, 3012,
 3031, 3069, 3078, 3088, 3099, 3107,
 3120, 3150, 3171, 3196, 3204, 3212,
 3222, 3247, 3262, 3267, 3272, 3278,
 3282, 3309, 3332, 3349, 3369, 3373,
 3383, 3403, 3405, 3410, 3453, 3520,
 3573, 3589, 3617, 3629, 3648, 3717,
 3780, 3811, 3900, 3908, 3921, 3935,
 3945, 3956, 4007, 4054, 4070, 4077,
 4083, 4107, 4124, 4133, 4145, 4152,
 4182, 4197, 4210, 4221, 4223, 4235,
 4276, 4300, 4337, 4350, 4363, 4419,
 4427, 4433, 4443, 4495, 4509, 4522,
 4585, 4604, 4633, 4642, 4660, 4664,
 4669, 4688, 4709, 4728, 4742, 4758,
 4763, 4769, 4823, 4846, 4853, 4857,
 4921, 4946, 4972, 4994, 5012, 5020,
 5030, 5047, 5058, 5078, 5088, 5131,
 5139, 5163, 5177, 5191, 5199, 5208,
 5218, 5246, 5279, 5298, 5309, 5356,
 5364, 5418, 5479, 5487, 5503, 5509,
 5546, 5551, 5562, 5567, 5578, 5581,
 5598, 5601, 5616, 5619, 5634, 5638,
 5648, 5651, 5662, 5668, 5684, 5696,
 5700, 5760, 5771, 5803, 5835, 5888,
 5965, 6124, 6145, 6218, 6226, 6229,
 6249, 6310, 6342, 6391, 6403, 6430,
 6450, 6455, 6487, 6502, 6517, 6524,
 6542, 6579, 6593, 6627, 6636, 6647,
 6664, 6673, 6739, 6749, 6754, 6775,
 6794, 6807, 6842, 6847, 6853, 6894,
 6962, 7029, 7049, 7138, 7153, 7165,
 7171, 7198, 7235, 7278, 7312, 7348,
 7362, 7381, 7391, 7397, 7449, 7464,
 7477, 7546, 7560, 7570, 7610, 7666,
 7704, 7717, 7740, 7758, 7764, 7808,
 7817, 7823, 7829, 7846, 7851, 7856,
 7972, 8005, 8032, 8244, 8366, 8442,
 8454, 8493, 8531, 8568, 8573, 8579,
 8589, 8674, 8846, 9142, 9242, 9305,
 9353, 9403, 9567, 9663, 9874, 9891
 \cs_new_protected:Npn
 16, 148, 303, 327,
 422, 587, 709, 713, 716, 720, 721,
 753, 1267, 1400, 1404, 1450, 1456,
 1491, 1561, 1660, 1676, 2003, 2033,
 2346, 2586, 2722, 2830, 2896, 3022,
 3026, 3182, 3192, 3254, 3294, 3395,
 3637, 4158, 4178, 4572, 4622, 4675,
 4695, 4750, 4755, 4790, 4812, 4817,
 4910, 4929, 4942, 5302, 5334, 5396,
 5714, 5892, 5916, 5920, 5922, 5926,
 5938, 5942, 5944, 5947, 5951, 5955,
 5958, 5962, 6609, 6652, 6658, 6779,
 6786, 6834, 6884, 6900, 6925, 6979,
 6991, 7001, 7181, 7203, 7218, 7270,
 7599, 7623, 7640, 7652, 7770, 7836,
 7841, 8117, 8241, 9148, 9296, 9299
 \cs_set:Nn 9518, 9537, 9540,
 9559, 9563, 9588, 9666, 9667, 9673,
 9674, 9683, 9702, 9705, 9724, 9728
 \cs_set:Npe 4087
 \cs_set:Npn 487,
 524, 530, 614, 615, 900, 913, 919,
 2678, 3312, 3326, 3439, 3444, 3505,
 3510, 4038, 4109, 4111, 4114, 4127,
 4184, 4395, 4445, 4554, 4730, 4949,
 4955, 5000, 5024, 5492, 5528, 5837,
 5894, 6362, 6363, 6367, 6368, 6369,
 6630, 6666, 6686, 6797, 6812, 6906,
 6931, 8122, 8123, 8124, 8348, 8679
 \cs_set_eq:NN 54, 74, 315, 319, 1512,
 1518, 1649, 1900, 1982, 3305, 3306,
 3307, 4161, 4636, 4638, 4691, 5253,

5255, 5373, 5376, 5404, 5405, 5565,
5694, 5805, 5808, 6588, 6600, 6603,
6654, 6660, 6810, 6863, 6876, 8722
`\cs_set_protected:Nn`
..... 1040, 8367, 8400, 8416, 10440
`\cs_set_protected:Npn`
..... 69, 1528, 1538, 3901, 3902,
5021, 5914, 6680, 6685, 6691, 8444,
8447, 8456, 8459, 8723, 8746, 8750,
8761, 8765, 8875, 8878, 8881, 10333
`\cs_set_protected:Npx` 4996
`\cs_undefine:N` 351, 1071, 1834
`\l_tmpa_cs` 5837,
5855, 5864, 5894, 5899, 6362, 6372
`\csname` ... 349, 663, 2153, 2154, 2155,
2156, 2157, 2162, 2163, 2164, 2567,
3470, 3472, 3536, 3538, 7572, 7858
`\ctikzinput` 121, 10533
`\currentgrouplevel` 325,
328, 334, 335, 337, 339, 341, 347,
349, 351, 353, 355, 7847, 7853, 7858
`\CurrentOption` 10, 8295, 8296,
8297, 8298, 8337, 8338, 8915, 9939
`\Currentsectionlevel` 84, 139, 1528
`\currentsectionlevel` 84, 139, 1528

D

`\date` 10454, 10455
`\DeclareOption` 10
`\def` 93, 669,
673, 676, 678, 1493, 2490, 2516,
3820, 3877, 4322, 4329, 4481, 4488,
4646, 4647, 5003, 5008, 5016, 5025,
5823, 5937, 5967, 6027, 6044, 6052,
6066, 6080, 6106, 6116, 6121, 6137,
6289, 6314, 6334, 7034, 8465, 8467,
8532, 8533, 8534, 8538, 8541, 8598,
8714, 8731, 8739, 8775, 8776, 8778,
8780, 8782, 8784, 8785, 8787, 8789,
8791, 8794, 8795, 8797, 8798, 8801,
8803, 8805, 8821, 8873, 8937, 8938,
8939, 9030, 9033, 9034, 9506, 9511,
9986, 9993, 9996, 10002, 10009,
10013, 10014, 10292, 10360, 10362
`\defemph` 105, 151, 5947
`\Definame` 102, 152, 6084, 7406, 7428
`\definame` 23, 33, 39, 101,
102, 105, 152, 305, 6084, 7405, 7427
`\Definames` 6116
`\definames` 6106
`\definiendum` 23, 33,
39, 101, 102, 105, 152, 6084, 7403, 7425
`\definiens` 41,
50, 58, 101–103, 7408, 7442, 7462, 7463

`\definiens` 157, 7442
`\defnotation`
... 33, 39, 102, 152, 6084, 7404, 7426
`\detokenize` 8994, 8997,
9000, 9003, 9949, 9952, 9955, 9958
dim commands:
`\dim_compare:nNnTF` 9908
`\dim_eval:n` 10336, 10337
`\dim_new:N` 9233
`\dim_to_decimal:n` 9882,
10357, 10385, 10386, 10387, 10394
`\dim_zero:N` 9236, 9821
`\dimexpr` 8710
do commands:
`\_do_comp:nNn` 151
`\_do_comp:nnNn` 151, 5923,
5945, 5956, 5963, 5965, 6161, 6692
`\dobracket` 94
`\dobrackets` ... 155, 6970, 6979, 7002,
8135, 8170, 8177, 8178, 8183, 8184
`\document-URI` 125, 144
`\doproblemqr`
... 9996, 10013, 10060, 10147, 10261
`\dowithbrackets` 155, 7001
`\due` 76, 119
`\duration` 77, 119

E

`\edef` 2155, 2156, 2157, 4765, 8549
`\egroup` .. 1903, 1985, 4410, 4569, 7793,
7826, 7941, 8576, 8665, 8671, 8689,
8795, 8798, 9281, 9341, 9389, 9441
`\eject` 9919, 10262
`\ellipses`
98, 4710, 4712, 5811, 5849, 6910, 6911
`\else` 662, 674, 1469, 2069, 2091,
2106, 2734, 2741, 5952, 8873, 8954,
8977, 9154, 9265, 9280, 9635, 9769,
9907, 9997, 10003, 10514, 10519, 10524
`\emph` 17, 18, 104, 1471,
5962, 7786, 7803, 7965, 9286, 9346,
9394, 9446, 9649, 9783, 9927, 10183
`\end` 128, 144,
1474, 1588, 1657, 2068, 2223, 2238,
2258, 2617, 2763, 2781, 2801, 3115,
6949, 7304, 7963, 8410, 8417, 8469,
8478, 8551, 8576, 8591, 8636, 8650,
8658, 8679, 8681, 8694, 8695, 8696,
8697, 8698, 8699, 8720, 8748, 8752,
8755, 8819, 8827, 8868, 8869, 9106,
9196, 9215, 9273, 9278, 9333, 9338,
9381, 9386, 9433, 9438, 9489, 9502,
9523, 9554, 9688, 9802, 9807, 9812,

9817, 9919, 9927, 10137, 10178,	<code>\envname</code> 144
10244, 10262, 10317, 10359, 10536	<code>\eq</code> 31
<code>\endbackmatter</code> 1760, 1762	<code>\eqstep</code> 158, 7688, 8072
<code>\endcsname</code> 349,	<code>\equal</code> 30
662, 663, 2153, 2154, 2155, 2156,	<code>\errmessage</code> 6538, 8963, 8986
2157, 2162, 2163, 2164, 2567, 3470,	<code>\escapechar</code> 88, 842, 9965
3472, 3536, 3538, 7572, 7858, 8873	<code>\everyeof</code> 2712
<code>\endfrontmatter</code> 1747, 1748	<code>\examdata</code> 10469
<code>\endgroup</code> 48, 50, 2083, 2583	<code>examdata</code> 10403
<code>\endinput</code> 1912, 1993	<code>\examnumber</code> 9993, 10004, 10005, 10007
<code>\endlist</code> 7819	<code>\excursion</code> 64, 112, 8807
<code>\endspfststepenv</code> 7995, 7999, 8082, 8086	<code>\excursiongroup</code> 64, 112, 8841
endstex commands:	<code>\excursionref</code> 64, 112, 8812, 8821, 8823, 8836
<code>\endstex_annotate_env</code> 2344, 9719	<code>exnote (env.)</code> 1, 9353
<code>\ensuremath</code> 5584, 5590, 5905	<code>\exnote</code> 9041, 9400, 9548, 9713
environments:	exp commands:
<code>assignment</code> 76, 119, 10266	<code>\exp_after:wN</code>
<code>blindfragment</code> 84, 139, 1628	 158, 483, 486, 517, 520, 882,
<code>copymodule</code> 145, 3429	885, 892, 1006, 1030, 1121, 1138,
<code>exnote</code> 1, 9353	1141, 1144, 1147, 1150, 1153, 1159,
<code>extstructure</code> 98, 156, 7053	1239, 1252, 1255, 1261, 1268, 1293,
<code>extstructure*</code> 98	1295, 1408, 1428, 1431, 1434, 1444,
<code>frame</code> 1, 1, 8493	1543, 1958, 2265, 2271, 2277, 2283,
<code>gnote</code> 1, 9401	2377, 2378, 2439, 2440, 2441, 2447,
<code>hint</code> 1, 9302	2451, 2455, 2456, 2516, 2714, 2733,
<code>interpretmodule</code> 145, 3494	2734, 2735, 2740, 2741, 2742, 3165,
<code>mathstructure</code> 98, 156, 7018	3232, 3233, 3234, 3470, 3472, 3536,
<code>mcb</code> 1, 9516	3538, 3583, 3584, 3585, 3621, 4041,
<code>nassertion</code> 1, 1	4398, 4557, 5080, 5153, 5154, 5159,
<code>ndefinition</code> 1, 1	5317, 5327, 5329, 5372, 5505, 5512,
<code>nexample</code> 1, 1	5513, 5514, 5522, 5523, 5524, 5537,
<code>note</code> 1, 1	5719, 5721, 5826, 5827, 5828, 5841,
<code>nparagraph</code> 1, 1, 1, 1	6076, 6132, 6156, 6556, 6569, 6914,
<code>nsproof</code> 1, 1	6942, 7132, 7135, 7207, 7313, 7327
<code>problem</code> 1	<code>\exp_args:Ne</code>
<code>sassertion</code> 101, 157, 7415	 58, 63, 69, 77, 81, 157, 265,
<code>scb</code> 9662	266, 274, 806, 814, 818, 903, 962,
<code>sdefinition</code> 101, 157, 7410	1451, 1814, 1835, 1837, 1854, 2030,
<code>sexample</code> 101, 157, 7430	2055, 2314, 2723, 2841, 3156, 3164,
<code>sfragment</code> 84, 139, 1547	3165, 3215, 3651, 3863, 3932, 4009,
<code>smodule</code> 88, 142, 2289	4098, 4163, 4165, 4242, 4365, 4523,
<code>solution</code> 1, 9232	4764, 4791, 4795, 4874, 4885, 4956,
<code>sparagraph</code> 101, 157, 7433	4995, 5013, 5143, 5421, 5639, 5740,
<code>spfblock</code> 158, 8059	5742, 5750, 5756, 6020, 6037, 6160,
<code>spfsketchenv</code> 158, 7946	6178, 6188, 6529, 6895, 7125, 7185,
<code>spfststepenv</code> 158, 8056	7273, 7319, 7400, 7419, 7471, 7479,
<code>sproblem</code> 9035	7481, 8036, 8246, 8817, 8956, 8979,
<code>sproof</code> 103, 158, 7664	9476, 9693, 9880, 9990, 10224, 10354
<code>stex_annotate_env</code> 128, 709	<code>\exp_args:NNe</code> 89, 474, 477,
<code>stex_env_node</code> 128, 709	879, 889, 896, 1902, 1984, 2413,
<code>subproblem</code> 9149	2517, 3080, 3594, 4204, 4249, 4261,
<code>subproof</code> 158, 7821	4976, 5092, 5106, 5201, 5220, 5453,
<code>testheading</code> 77, 119	5456, 5726, 5777, 5867, 6040, 6113,
<code>titlefragment</code> 139, 1606	6376, 6472, 6483, 7147, 8863, 9143

<code>\exp_args:Nne</code>	304,	<code>\exp_not:N</code>	
1065, 1342, 1484, 1596, 1815, 2124,		94, 257, 308, 310, 913, 1080, 1081,	
2145, 2602, 2881, 3013, 3042, 3257,		1086, 1087, 1118, 1127, 1128, 1133,	
3821, 3854, 3922, 4097, 4330, 4729,		1245, 1246, 1414, 1415, 1820, 1904,	
5015, 5032, 5080, 5147, 5149, 5202,		1986, 2461, 2712, 2732, 2740, 3043,	
5490, 5526, 5536, 6315, 6335, 6503,		3658, 4587, 4588, 4591, 4595, 4599,	
6668, 6698, 7040, 7140, 7143, 7146,		4656, 4732, 4735, 4829, 4905, 4979,	
7284, 7370, 7479, 7481, 7515, 7551,		4985, 5005, 5038, 5067, 5070, 5094,	
7556, 7563, 7566, 7705, 7721, 7953,		5100, 5109, 5114, 5123, 5158, 5381,	
7977, 8011, 9071, 9171, 9529, 10353		5387, 5500, 5778, 5783, 5862, 5868,	
<code>\exp_args:NNne</code>	4251, 6776	5872, 5874, 5875, 5897, 6344, 6751,	
<code>\exp_args:NNne</code> 4338, 4496, 5432, 5438		8372, 8377, 9644, 9646, 9778, 9780	
<code>\exp_args:NNno</code>		<code>\exp_not:n</code>	257,
.	1017, 2864, 3604, 4338, 4496	308, 914, 1596, 1823, 1824, 1930,	
<code>\exp_args:NNno</code>	143	1936, 1966, 1968, 1986, 2455, 2466,	
<code>\exp_args:Nnno</code>	3175, 3177, 5906	2521, 3882, 3929, 4090, 4344, 4345,	
<code>\exp_args:NNNx</code>	1017	4346, 4356, 4357, 4358, 4502, 4503,	
<code>\exp_args:NNo</code>		4504, 4515, 4516, 4517, 4589, 4592,	
.	13, 42, 63, 83, 104, 107, 127,	4596, 4598, 4600, 4740, 4830, 4868,	
131, 156, 222, 348, 750, 800, 1020,		4899, 4904, 4906, 4978, 4980, 4982,	
1342, 1875, 2193, 2204, 2409, 2881,		4986, 4989, 4999, 5068, 5071, 5095,	
2913, 3601, 3607, 3611, 5229, 5230,		5097, 5101, 5103, 5108, 5112, 5121,	
5240, 5241, 5854, 6423, 6436, 6824		5158, 5513, 5529, 5534, 5539, 5541,	
<code>\exp_args:Nno</code>		5763, 5784, 5821, 5826, 5864, 5876,	
.	335, 337, 372, 1910, 1940, 2491,	5877, 5899, 6345, 6385, 6386, 6745,	
3159, 4036, 4204, 4249, 4261, 4393,		6914, 6942, 7853, 9649, 9783, 9851	
4542, 4552, 4574, 4752, 4756, 4965,		<code>\expandafter</code>	48,
7830, 9058, 9228, 9987, 10281, 10326		663, 2153, 2154, 2155, 2156, 2157,	
<code>\exp_args:NNx</code>		2567, 5952, 7825, 7858, 8469, 8470	
.	90, 93, 426, 439, 8994, 8997,	<code>\ExplSyntaxOff</code> 2591, 2639, 8253, 9007, 9962	
9000, 9003, 9949, 9952, 9955, 9958		<code>\ExplSyntaxOn</code>	
<code>\exp_args:No</code>	48, 257, 308, 348,		143, 2587, 2638, 8250, 8990, 9945
570, 576, 657, 668, 785, 914, 1029,		<code>\extref</code>	87, 2023
1521, 1596, 1615, 1874, 1882, 1883,		<code>extstructure</code> (env.)	98, 156, 7053
1909, 1920, 1923, 1928, 1930, 1936,		<code>\extstructure</code>	7082, 7084
1937, 1938, 1947, 1960, 1968, 1986,		<code>extstructure*</code> (env.)	98
1991, 2018, 2019, 2254, 2330, 2363,			
2423, 2521, 2553, 2839, 2843, 3075,			
3079, 3162, 3242, 3375, 3396, 3411,			
3419, 3618, 3624, 3625, 3668, 3850,			
3913, 3929, 4089, 4094, 4202, 4251,			
4344, 4345, 4346, 4356, 4357, 4358,			
4502, 4503, 4504, 4515, 4516, 4517,			
4589, 4592, 4596, 4623, 4716, 4830,			
4834, 4841, 4868, 4904, 4906, 4978,			
4980, 4986, 5068, 5095, 5101, 5103,			
5108, 5112, 5121, 5144, 5158, 5219,			
5430, 5458, 5529, 5534, 5539, 5541,			
5740, 5742, 5784, 5864, 5877, 5899,			
6203, 6345, 6745, 6932, 7037, 7045,			
7101, 7104, 7126, 7186, 7206, 7377,			
7459, 8361, 8425, 8426, 8429, 8993,			
9079, 9179, 9182, 9948, 9991, 10455			
<code>\exp_args:Nx</code>	8494, 9568		

F

<code>\fbox</code>	9879, 9895, 9908
<code>\fi</code>	665, 680, 1475, 2071,
2095, 2106, 2735, 2742, 4634, 4665,	
5251, 5952, 6125, 6146, 8471, 8539,	
8542, 8660, 8873, 8941, 8958, 8981,	
9156, 9269, 9282, 9637, 9652, 9657,	
9771, 9786, 9791, 9913, 9927, 9987,	
10000, 10010, 10518, 10528, 10529	
<code>fiboxed</code>	60, 108
file commands:	
<code>\file_if_exist:nTF</code>	612, 631, 997
<code>\fillinsol</code>	117, 9039, 9819
<code>\fn</code>	47
<code>\foldexpr</code>	152, 6175
<code>\foo</code>	16, 47, 90
<code>\foiname</code>	16

\footnote	9656, 9790, 9897	\hbox	1506, 2927, 3000, 3564, 3579, 3874,
\footnotesize	10355		3904, 4009, 4365, 4538, 6674, 7066,
\foral	40		7244, 7677, 7744, 7870, 8108, 8263,
\forall	40, 8191		8575, 8795, 8798, 9593, 9624, 9866
fp commands:		hbox commands:	
\fp_gadd:Nn	9102, 9103	\hbox_set:Nn	6359, 9879
\fp_new:N	9045, 9046	\hbox_unpack:N	
\fp_to_decimal:n	9189, 9192		1530, 1540, 3903, 9509, 9514
frame (env.)	1, 1, 8493	\HCode	675
\frameimage	62, 111, 8703	\hfil	8114, 9898
frameimages	60, 108	\hfill	8114
\frametitle	8607	hint (env.)	1, 9302
\frontmatter	210, 1745, 1746, 1749	\hint	9040, 9352, 9547, 9712
\FTMLrule	8256, 8280, 8281	hints	76, 113, 118
\fun	39	\hline ..	10353, 10355, 10356, 10357, 10358
\funspace	34	\homeworkdata	10475
		homeworkdata	10403
G		\href	1960, 2037, 8817
\gdef	1497, 1514, 1519,	\hrule	8108, 9593, 9624
	8807, 9980, 9982, 9990, 9991, 10007	\hskip	9879
\given	76, 119	\hspace	9631, 9765, 9911
\global	1493, 2154, 8235,	\HTML	16, 25
	8236, 9297, 9300, 10009, 10288, 10290	\huge	9879, 9909, 9911
gnote (env.)	1, 9401	hwexam commands:	
\gnote	9042, 9452, 9549, 9714	\l_hwexam_includeassignment_	
gnotes	76, 113, 118	keys_tl ..	10279, 10280, 10282, 10324
group commands:		\hwexam_kw_*	9944
\group_begin:		\c_hwexam_multiple_bool	9935
	18, 78, 87, 613, 755, 841,	\c_hwexam_qrcode_bool ...	9937, 9969
	2493, 2609, 2673, 2682, 2690, 2931,	\hwexamheader	10360, 10399
	2967, 3004, 3040, 3063, 3781, 3812,	\hwexamminutes	10362
	3886, 4325, 4484, 4606, 4717, 4721,	\hyperlink	2035
	4997, 5022, 5140, 5248, 5263, 5481,	I	
	5524, 5652, 5697, 5772, 5820, 5836,	\if	2733, 2740
	5928, 5998, 6015, 6032, 6046, 6058,	if commands:	
	6072, 6147, 6220, 6243, 6298, 6330,	\if_charcode:w	9987
	6980, 7314, 7443, 7498, 7761, 7810,	\IfBooleanTF ..	3477, 3543, 3766, 5409,
	8010, 8035, 8121, 9225, 9964, 10323		6292, 6327, 7876, 7989, 8077, 8704
\group_end:	82, 85,	\ifcsname	662, 8873
	93, 621, 822, 845, 2507, 2618, 2686,	\ifdefempty	8862
	2693, 2696, 2936, 2973, 3009, 3042,	\IfExists	6, 21, 1883, 1947,
	3116, 3789, 3826, 3891, 4335, 4492,		2052, 2192, 2203, 2884, 2889, 2898
	4667, 4680, 4706, 4718, 4722, 4752,	\IfInputref	27, 85, 140, 2099, 8860
	4913, 4969, 5006, 5027, 5033, 5106,	\ifinputref ..	27, 85, 140, 2089, 2098, 2106
	5159, 5277, 5314, 5325, 5341, 5352,	\ifintest	8926, 8935, 9927
	5360, 5393, 5484, 5505, 5537, 5542,	\ifmmode	5952
	5660, 5697, 5777, 5828, 5867, 5932,	\ifnotes	61, 109, 8352, 8489
	6143, 6165, 6222, 6243, 6301, 6340,	\ifnum	2066
	6986, 7323, 7445, 7509, 7767, 7819,	\IfSGvar	63, 112, 8881
	8029, 8053, 8238, 9229, 9967, 10327	\ifsolutions ..	114, 8918, 9262, 9275, 9628,
\group_insert_after:N	340, 354		9641, 9655, 9762, 9775, 9789, 9893
H		\ifstexhtml	27, 83, 128, 685,
\halign	9898		1469, 8954, 8977, 9154, 9997, 10003

<code>\ifvmode</code>	4634, 4665, 5251, 6125, 6146	1638, 1663, 1667, 1703, 1709, 3998,
<code>\ifx</code>	2153,	3999, 4000, 4001, 5715, 6854, 6937,
	8468, 8537, 8540, 10511, 10512, 10520	7607, 7619, 7630, 7647, 7659, 8269,
<code>\ignorespaces</code>	2922, 2944,	8411, 8762, 8766, 9466, 10201, 10341
	2993, 4335, 4492, 8663, 8669, 9213	<code>\int_new:N</code>
<code>image</code>	120	1996, 3954, 4150, 5362, 5699, 6924,
<code>\importmodule</code>	35, 43, 48, 96,	6954, 8758, 9149, 9232, 9401, 10332
	97, 124, 128, 143, 145, 222, 2941, 3299	<code>\int_set:Nn</code> 1678, 1679, 1680, 1681,
<code>\includeassignment</code>	76, 10322	1682, 1683, 1685, 3386, 3869, 3963,
<code>\includegraphics</code> . . .	85, 141, 2221, 10504	3967, 3971, 3975, 3979, 3983, 3987,
<code>\includeproblem</code>	75, 117, 9217	3991, 3995, 4186, 4226, 4287, 4315,
<code>\indent</code>	4634, 4665, 5251, 6125, 6146	4447, 5657, 6827, 6932, 6939, 6961,
<code>\infpref</code>	92, 93, 154, 6423, 6958, 8143	7600, 7612, 7624, 7641, 7653, 10426
<code>\inline</code>	157	<code>\int_set_eq:NN</code>
<code>\inline*</code>	102	325
<code>\inlineass</code>	44, 48, 51, 102, 157	<code>\int_step_function:nN</code> . . . 4733, 5493
<code>\inlinedef</code>	44, 102, 157	<code>\int_step_inline:nn</code>
<code>\inlineex</code>	102, 157	 3947, 6415, 6422, 6442, 6762
<code>\input</code> . . .	26, 82, 125, 143, 182, 44, 246,	<code>\int_use:N</code> . 1582, 1645, 1689, 1692,
	684, 2116, 8991, 8995, 8998, 9001,	1713, 3925, 4342, 4354, 4500, 4513,
	9004, 9946, 9950, 9953, 9956, 9959,	4576, 5385, 5421, 5762, 5780, 5870,
	10360, 10513, 10516, 10522, 10526	6491, 6596, 6827, 6958, 6959, 7372,
<code>\inputassignment</code>	119	7847, 7853, 7858, 8272, 8404, 8763,
<code>\inputref</code>	26,	8767, 9202, 9247, 9310, 9358, 9408,
	27, 84, 85, 104, 124, 140, 2041,	9468, 10105, 10106, 10128, 10145,
	8722, 8723, 8831, 8863, 9228, 10326	10146, 10169, 10203, 10353, 10460
<code>\inputref*</code>	61, 110, 8722	<code>\int_zero:N</code>
<code>\inputreffalse</code>	2094, 2098	3958, 5706, 6809, 6935, 8262, 8760
<code>\inputreftrue</code>	2077, 2092	<code>\c_max_int</code>
<code>\insertexamnumber</code> . .	10002, 10009, 10014	6958, 6959
<code>\insertframenumber</code>	8801, 8803	<code>\l_tmpa_int</code>
<code>\insertshortauthor</code>	8794	6809, 6812, 6827, 6854, 6935, 6937,
<code>\insertshortdate</code>	8805	6938, 6939, 6943, 7600, 7603, 7606,
<code>\insertshorttitle</code>	8797	7607, 7612, 7615, 7618, 7619, 7624,
<code>\inset</code>	39	7627, 7630, 7632, 7633, 7635, 7636,
		7641, 7644, 7647, 7649, 7653, 7656,
		7659, 7661, 7662, 8262, 8269, 8272
int commands:		intarray commands:
<code>\int_case:nn</code>	1629	<code>\intarray_gset:Nnn</code>
<code>\int_case:nnTF</code>	1562, 1661, 1699, 1718	 7635, 7649, 7662, 7668
<code>\int_compare:nNnTF</code>		<code>\intarray_gzero:N</code>
	328, 1599, 1637, 2122,	7667
	2144, 3595, 4826, 4837, 4930, 5343,	<code>\intarray_item:Nn</code> 7603, 7606,
	5739, 5749, 5755, 5814, 6350, 6384,	7615, 7618, 7627, 7636, 7644, 7656
	6938, 6968, 7345, 7383, 7632, 10340	<code>\intarray_new:Nn</code>
<code>\int_compare_p:nNn</code>		7597
	7602, 7614, 7626, 7643, 7655	<code>\interpretmod</code>
<code>\int_decr:N</code>	7633, 7661	145, 3556, 3558, 3560
<code>\int_eval:n</code>	347, 349, 351,	<code>interpretmodule (env.)</code>
	5435, 5441, 6967, 7173, 10288, 10354	145, 3494
<code>\int_gincr:N</code>	5420,	<code>\interpretmodule</code>
	9187, 9245, 9308, 9356, 9406, 10334	3530, 3532
<code>\int_gset:Nn</code>	5385	<code>\intestfalse</code>
<code>\int_gzero:N</code>	5370, 9096	8930
<code>\int_incr:N</code>		<code>\intesttrue</code>
	1564, 1568, 1600, 1631, 1634,	8928
		ior commands:
		<code>\ior_close:N</code>
		638, 644, 846, 1028
		<code>\ior_map_inline:Nn</code>
		1016
		<code>\ior_new:N</code>
		1002
		<code>\ior_open:Nn</code>
		632, 640, 1011
		<code>\ior_open:NnTF</code>
		840
		<code>\ior_str_get:NN</code>
		843

\ior_str_map_inline:Nn	 634, 641	\libusepackage	... 82, 83, 140, 2120, 8485
\g_tmpa_ior	 632,	\libusetHEME	 61, 8484
634, 638, 640, 641, 644, 840, 843, 846		\libusetikzlibrary	 82, 121, 140, 2131
iow commands:		\linewidth	... 9285, 9292, 9345, 9350,
\iow_close:N	... 627, 637, 1798, 10254	9393, 9398, 9445, 9450, 10182, 10195	
\iow_new:N	 585, 1796, 10251	\list	 7810
\iow_now:Nn	 635,	\LoadClass	 16, 8317, 8319
642, 730, 1816, 1819, 8249, 9143, 10253		\loadsms	 127, 586, 587
\iow_open:Nn	... 626, 633, 1797, 10252	\long	 8776, 8785
\g_tmpa_iow	 633, 635, 637	\lstinputlisting	 85, 141, 2237
\isassociative	 43	\lstinputmhlising	 85, 141, 2225
\iscommutative	 43		
\item	 7814, 9202, 9476,	M	
9495, 9523, 9688, 10105, 10145, 10222		\macro	 123, 124, 126, 135, 144
\itshape	 8118	\macroname	 31, 127
J		\magma	 46
\jobname	 79, 235, 604, 626,	\maincomp	 31–34, 47, 92, 100,
631, 632, 633, 640, 645, 1797, 10252		105, 4417, 4647, 4666, 4764, 4765,	
\join	 56	4995, 5003, 5008, 5013, 5016, 5025,	
K		5359, 5937, 6344, 6353, 6363, 6364,	
keys commands:		6385, 6691, 6751, 7191, 8135, 8177,	
\l_keys_choice_tl	 3740	8183, 8202, 8206, 8209, 8211, 8226	
\keys_define:n	 29, 372,	\mainmatter	 1773, 1774, 1784, 1785
8289, 8331, 8841, 8897, 9933, 10492		\makeatletter	 2636, 8990, 9945, 10399
\l_keys_key_str	6266, 6269, 7011, 7014	\makeatother	 2637, 9007, 9962, 10399
\l_keys_key_tl	... 13, 750, 6267, 7012	\makebox	 9630, 9764
\keys_set:n	 126, 376, 8850	\maketitle	
keyval commands:		139, 1518, 1519, 1610, 1619, 8417, 8418	
\keyval_parse:NNn	 4849	\mapsto	 8178, 8184
L		\marginnote	 8535, 9998
\label	 86, 1814, 1835, 1837, 8600	\marginpar	 152, 6219,
\labelsep	 8545	8955, 8978, 9112, 9117, 9205, 9210,	
\labelwidth	 8546	9508, 9513, 10021, 10026, 10063,	
\langle	 8162	10068, 10109, 10114, 10150, 10155	
\Large	 8349,	\mathbb	 8214
8732, 9559, 9560, 9563, 9564, 9724,		\mathbin	
9725, 9728, 9729, 10005, 10007, 10304		32, 8141, 8145, 8154, 8190, 8201, 8205	
\lastslide	 8750, 8765	\mathcal	 5862, 5864, 5897, 5899
\LaTeX	 23	\mathclose	 32, 6987, 8143, 8152,
\latex	 23	8156, 8162, 8164, 8174, 8190, 8194	
\ldots	 8167, 8230, 8232, 8234	\mathhub	 80, 132, 32, 837
\leaders	 8734	\mathop	 32, 8192, 8211
\leavevmode	 8620	\mathopen	 32, 6982, 8143, 8152,
\leftmargin	 8547	8156, 8162, 8164, 8173, 8190, 8194	
\let	 1746, 1747, 1748, 1749,	\mathord	 32
1759, 1760, 1761, 1762, 2078, 2154,		\mathpunct	32, 5689, 6766, 8173, 8177, 8183
2675, 2683, 2691, 2713, 2718, 2987,		\mathrel	 31,
3904, 4645, 4666, 6003, 6026, 6043,		32, 8146, 8178, 8184, 8202, 8206, 8209	
6138, 7197, 8235, 8236, 8251, 8252,		\mathrm	 6751,
8418, 8741, 9665, 9669, 9672, 9676		8135, 8148, 8173, 8177, 8183, 8226	
\libinput	 19, 82, 140, 2109, 2179	mathstructure (env.)	 98, 156, 7018
		\mathstructure	 7027, 7028
		\mathtt	 8151, 8152, 8217, 8220, 8224
		mcb (env.)	 1, 9516

\mcb	9037, 9544, 9566, 9709	\neginfprec	37, 92, 93, 154, 4678, 5337, 5348,
\mcc	70, 115, 9550, <u>9587</u>		6393, 6396, 6405, 6408, 6421, <u>6958</u>
\meaning	1956,	\newcommand	483, 517, 520,
	2567, 3122, 3125, 3128, 4677, 5157,		2011, 2100, 2105, 2109, 2120, 2131,
	5520, 5532, 5533, 6389, 6694, 6849		2166, 2232, 2238, 2248, 2258, 8094,
\medskip	2083, 8575, 8590, 8614		8098, 8102, 8466, 8535, 8616, 8725,
\meet	56		8729, 8812, 8823, 8830, 8833, 8855,
\message	28,		8857, 8943, 8966, 9028, 9463, 9597,
	1864, 2739, 8310, 8313, 8475, 10249		9639, 9735, 9773, 9799, 9804, 9809,
\mhframeimage	62, 111		9814, 9863, 9918, 9921, 9922, 9923,
\mhgraphics	85, 141, <u>2209</u> , 8716, 8718		9924, 9925, 9926, 10339, 10469,
\mhinput	85, 140, <u>2086</u>		10475, 10479, 10504, 10509, 10533
\mhtikzinput	121, 141, <u>2240</u>	\newcounter	8320, 8321,
\min	77, 119		8322, 8323, 8450, 8462, 9027, 10277
\min	9511	\NewDocumentCommand ..	127, 486, 1468,
min	76, 113, 118		1829, 1840, 2023, 2028, 2041, 2086,
\mmlarg	129, <u>712</u>		2575, 4413, 4416, 5262, 5408, 5997,
\mmlintent	129, <u>712</u>		6014, 6031, 6045, 6057, 6071, 6094,
\mname	89, 98		6100, 6110, 6120, 6176, 6187, 7313,
mode commands:			7442, 7488, 7497, 7512, 7973, 8072,
\mode_if_math:TF			8090, 8256, 8484, 8703, 9224, 10322
..	3579, 3904, 3905, 4662, 4667, 9865	\NewDocumentEnironment	127
\mode_if_vertical:TF	1530, 1540, 3903	\NewDocumentEnvironment	
\mode_leave_vertical:	3236		523, 1553, 1608, 1652, 8059, 8744, 8759
\MSC	8241	\newenvironment	1770, 1781, 7946,
msg commands:			8056, 8669, 8671, 8676, 8678, 10382
\msg_error:nn		\newif 663, 687, 2098, 8352, 8918, 8926, 10389	
..	3047, 3228, 5664, 7455, 7504, 9054	\newlabel	8251, 8252
\msg_error:nnn		\newlength	8487, 8488, 8491
.....	58, 469, 795, 2133, 2179,	\newline	9285, 9292, 9345, 9350,
	2563, 2810, 2915, 3681, 3711, 4154,		9393, 9398, 9445, 9450, 10182, 10195
	4423, 5127, 5205, 6897, 7200, 9841	\newpage	8597, 9925, 10401
\msg_error:nnnn ..	70, 2114, 2171,	\newsavebox	8638
	3285, 3361, 4003, 4614, 5259, 5275,	\newseq	148, <u>4416</u>
	5446, 5473, 6141, 6163, 6622, 7167	\newvar	148, <u>4413</u>
\msg_fatal:nn	947, 10442, 10449, 10452	nexample (env.)	1, 1
\msg_fatal:nnnn	951, 2128, 2147	\nextslide	8746, 8761
\msg_none:nn	117	\ninputref	8723, 8725, 8729
\msg_redirect_module:nnn	132	\nobreak	8114
\msg_redirect_name:nnn	136	\noexpand	2733
\msg_set:nnn		\noindent	1583, 2054, 2067, 7291,
.....	114, 9832, 10409, 10413, 10417		7695, 7902, 7965, 8034, 8590, 8610,
\msg_warning:nn	859		8618, 8636, 9077, 9108, 9177, 9286,
\msg_warning:nnnn	651		9346, 9394, 9446, 10018, 10059, 10183
\mult	37, 93, 94	\nointerlineskip	8736
\multicolumn	10354	\normalmarginpar	9999
\multiple	76, 118	\notation	31, 32, 37, 90, 92, 97,
			143, 153, 3298, 6282, 8138, 8141,
			8144, 8145, 8146, 8161, 8163, 8189,
			8191, 8192, 8200, 8204, 8214, 8217
N		note (env.)	1, 1
nassertion (env.)	1, 1	notes	60, 76, 108, 113, 118
\Nat	34	\notesfalse	8364
ndefinition (env.)	1, 1		

notesslides commands:	
\c_notesslides_notes_bool	_notesslides_notes_env:nnnn ...
8291, 8292, 8305, 8308, 8316, 8332,	8674,
8333, 8345, 8353, 8481, 8662, 8668,	8694, 8695, 8696, 8697, 8698, 8699
8675, 8709, 8724, 8774, 8824, 8834	\l_notesslides_num
\c_notesslides_sectocframes_bool	8370, 8372, 8375,
..... 8334, 8437	8377, 8380, 8381, 8384, 8385, 8388,
\c_notesslides_topsect_str 8335,	8389, 8392, 8393, 8396, 8397, 8404
8358, 8361, 8422, 8425, 8426, 8429	_notesslides_setup_itemize: ...
notesslides internal commands:	 8531, 8602
\c_notesslides_class_str	\l_notesslides_slideshow_-
..... 8287, 8290, 8296, 8317	counter_int
_notesslides_define_chapter: ..	.. 8758, 8760, 8762, 8763, 8766, 8767
..... 8427, 8430, 8442	\l_notesslides_tmp 8882, 8887
_notesslides_define_part:	\l_notesslides_uri 8816, 8817
..... 8431, 8454	\g_notesslides_variables_prop ..
_notesslides_do_label:n 8367, 8403	 8874, 8876, 8879, 8882
_notesslides_do_sectocframes: .	\notesslidesfont 8571, 8587, 8741
..... 8366, 8438	\notesslidesfooter 8575, 8590, 8739
_notesslides_do_yes_param:Nn ..	\notesslides_title_emph .. 8610, 8612, 8731
..... 8493,	\notesttrue 8354
8512, 8515, 8518, 8521, 8524, 8527	nparagraph (env.) 1, 1, 1, 1
\c_notesslides_docopt_str ... 8293	nsproof (env.) 1, 1
\c_notesslides_document_str ...	\null 8114
..... 8465, 8468	\number 76, 119
_notesslides_eat: . 8679, 8683, 8686	\numberline 8402
_notesslides_excursion_args:n .	\numberproblemsin 9027
..... 8846, 8858	
\l_notesslides_excursion_id_str	O
..... 8842, 8848	\objective 8966
\l_notesslides_excursion_intro_-	\OMDoc 25
tl 8843, 8847, 8862, 8864	\omdoc 16
\l_notesslides_excursion_-	\only 8763, 8767
mhrepos_str 8844, 8849, 8863	\oplus 8153, 8154
\l_notesslides_frame_allowdisplaybreaks_-	P
bool 8504, 8515	\PackageError 8883
\l_notesslides_frame_allowframebreaks	\pagebreak 9125, 10320
bool 8503, 8512	\pagenumbering ... 1754, 1767, 1778, 1789
_notesslides_frame_box_begin: .	\par .. 47, 1581, 1585, 2067, 7666, 7695,
..... 8568, 8579, 8603	7760, 7766, 7809, 7818, 7898, 7902,
_notesslides_frame_box_end: ...	7947, 7952, 7961, 8027, 8114, 8575,
..... 8573, 8589, 8605	8590, 8610, 8618, 8623, 8631, 8636,
\l_notesslides_frame_fragile_-	8643, 8653, 9108, 9121, 9124, 9155,
bool 8505, 8518	9285, 9292, 9345, 9350, 9393, 9398,
\l_notesslides_frame_label_str .	9445, 9450, 9496, 9499, 9556, 9677,
..... 8502, 8510, 8599, 8600	9721, 10018, 10047, 10055, 10059,
\l_notesslides_frame_shrink_-	10089, 10097, 10182, 10195, 10238,
bool 8506, 8521	10241, 10303, 10308, 10318, 10320
\l_notesslides_frame_squeeze_-	\paragraph 26, 84
bool 8507, 8524	\parent 125
\l_notesslides_frame_t_bool ...	\parsep 7812, 9200, 9474,
..... 8508, 8527	9493, 9521, 9686, 10103, 10143, 10220
_notesslides_inputref:	\part 26, 84, 8458, 8459
..... 8722, 8723, 8726	\partname 8372, 8455, 8456

\parttitlename	8372	\g_problem_total_pts_fp ..	9045, 9102
\PassOptionsToClass	8295, 8296	\problemheader ..	9108, 9128, 10018, 10059
\PassOptionsToPackage		problems internal commands:	
..... 10, 8297, 8298, 8309,		__problems_activate_macros: ...	
8312, 8337, 8338, 8355, 8915, 9939		 9035, 9100, 9157	
\pause	8616	\l__problems_anscls_int	
\PDF	16, 25	 9401, 9466, 9468, 10201, 10203	
\pdfbookmark	8404	\c__problems_boxed_bool	8911
peek commands:		__problems_do_yes_param:Nn ..	9567
\peek_charcode:NTF		__problems_exnote_start:n	
1393, 3630, 4671, 4684, 4744, 4746,		 9353, 9370, 9373	
4773, 4779, 4922, 4933, 5285, 5292		\c__problems_false_tl ...	9830, 9840
\peek_charcode_remove:NTF		\l__problems_fillin_solution_tl .	
... 4670, 4743, 4778, 5031, 5281,		.. 9820, 9844, 9856, 9885, 9896, 9903	
5283, 5290, 5300, 5303, 5504, 5547		__problems_fillinsol:n	
\phantom	9909	 9866, 9868, 9874, 9891	
\plus	35–38, 92–94	__problems_gnote_start:n	
\precondition	8943	 9403, 9422, 9425	
\prematurestop	63, 111, 8465	\c__problems_gnotes_bool	
\premise	103, 157, 7424, 7512	 8901, 9424, 9435, 9979, 10250	
prg commands:		\l__problems_has_min_bool .	9050,
\prg_new_conditional:Nnn		9094, 9095, 9191, 9209, 10113, 10154	
. 57, 62, 263, 272, 695, 2540, 2544,		\l__problems_has_pts_bool	
2548, 2662, 4093, 4096, 4271, 5452,		 9049, 9091, 9092,	
5738, 5748, 5754, 6197, 6208, 6212		9188, 9204, 10108, 10130, 10149, 10171	
\prg_new_protected_conditional:Nnn		__problems_hint_start:n	
..... 279		 9305, 9322, 9325	
\prg_return_false:	60,	\c__problems_hints_bool	
65, 264, 267, 273, 275, 295, 299,		 8903, 9324, 9335	
697, 2542, 2546, 2550, 2663, 4094,		\l__problems_in_problem_bool ...	
4105, 5470, 5474, 5744, 5745, 5746,		.. 9048, 9051, 9053, 9087, 9153, 9158	
5751, 5752, 5757, 5758, 6198, 6213		__problems_maybe_inline:n	
\prg_return_true:		 9528, 9554, 9663, 9692, 9719	
..... 60, 65, 265, 267, 275, 299,		__problems_maybe_newline:	
697, 2542, 2546, 2550, 2663, 4094,		 9588, 9667, 9674	
4105, 5468, 5744, 5751, 5757, 6209		__problems_mccline:n	9518,
\printexcursion	64, 112	9537, 9540, 9559, 9563, 9599, 9654	
\printexcursions ..	8807, 8831, 8859, 8867	\c__problems_min_bool ..	8909, 9115,
problem (env.)	1	9208, 9512, 10024, 10066, 10112, 10153	
problem commands:		\l__problems_min_tl	
\g_problem_id_counter		9089, 9093, 9103, 9116, 9117, 9144,	
..... 9232, 9245, 9247,		9160, 9192, 10025, 10026, 10067, 10068	
9308, 9310, 9356, 9358, 9406, 9408		\c__problems_notes_bool	
\l_problem_inputproblem_keys_tl .		 8899, 9372, 9383	
..... 9056, 9057, 9059, 9226		__problems_oldpar:	
\problem_mcc_box_default_tl		 9665, 9669, 9672, 9676	
..... 9622, 9630, 9634, 9636		__problems_parse_fillin_-	
\problem_mcc_box_tl		arg:nnnn	
.. 9523, 9527, 9559, 9563, 9591, 9627		.. 9824, 9825, 9826, 9837, 9855, 9877	
\problem_scc_box_default_tl		\l__problems_path_seq	
..... 9760, 9764, 9768, 9770		.. 9068, 9069, 9163, 9164, 9168, 9169	
\problem_scc_box_tl		\l__problems_prob_imports_tl .	9024
.. 9688, 9700, 9724, 9728, 9734, 9761		\l__problems_prob_refnum_int .	9025
\g_problem_total_min_fp ..	9046, 9103		

<code>\reversemarginpar</code>	9998	<code>\seq_map_function:NN</code> 182, 186, 2116,	
<code>\rhd</code>	8538, 8541	2173, 3139, 4871, 7956, 8014, 8039	
<code>\Rightarrow</code>	8071, 8206	<code>\seq_map_indexed_function:NN</code> ...	
<code>\rightmargin</code>	7813, 9201, 9475,		7334, 7708, 7724
	9494, 9522, 9687, 10104, 10144, 10221	<code>\seq_map_inline:Nn</code>	
<code>\rule</code>	9285, 9292, 9345, 9350,		866, 958, 1060, 1064,
	9393, 9398, 9445, 9450, 10182, 10195		1225, 2184, 3131, 3152, 3184, 3273,
rustex commands:			3353, 3357, 3414, 3422, 4117, 4240,
<code>\rustex_direct_HTML:n</code>			5848, 6474, 6909, 6936, 7064, 7106,
	8624, 8632, 8644, 8654		7242, 7399, 7418, 7674, 7867, 8268
<code>\rustex_if:TF</code>	8622, 8623,	<code>\seq_new:N</code>	
	8630, 8631, 8642, 8643, 8652, 8653		24, 761, 2288, 2631, 2651, 2666
<code>\rustexBREAK</code>	2640	<code>\seq_pop:NN</code>	179
		<code>\seq_pop_left:NN</code>	
			163, 292, 293, 1064, 1226,
			1228, 2197, 3597, 3599, 3606, 3610
		<code>\seq_pop_left:NNTF</code>	
			1019, 6433, 6435, 6443
		<code>\seq_pop_right:NN</code>	
		.	197, 210, 212, 217, 219, 221, 979,
			1190, 1192, 1204, 1277, 1317, 1320,
			2185, 2865, 4200, 4237, 4862, 6777
		<code>\seq_push:Nn</code> 185, 862, 2674, 3092, 3658	
		<code>\seq_put_left:Nn</code>	
			164, 183, 2432, 4135, 5165, 5179, 6638
		<code>\seq_put_right:Nn</code> 200, 213, 223, 226,	
			235, 467, 873, 1209, 2194, 2198,
			2205, 2372, 2559, 3080, 3101, 3216,
			3233, 3335, 3584, 5201, 5761, 5778,
			5812, 5819, 5844, 5850, 5854, 5868,
			6416, 6423, 6444, 6446, 6763, 6911,
			6913, 7059, 7094, 7239, 7274, 7364
		<code>\seq_reverse:N</code>	4863
		<code>\seq_set_eq:NN</code>	
			188, 209, 216, 234, 280,
			281, 1063, 1189, 1224, 5859, 6918, 6946
		<code>\seq_set_split:Nnn</code>	
			143, 211, 218, 865, 970, 971, 1018,
			1061, 1191, 1223, 1276, 1316, 2182,
			2183, 2864, 3594, 3604, 4199, 4236,
			4861, 6432, 6436, 6472, 6776, 8260
		<code>\seq_use:Nn</code>	206,
			213, 226, 472, 475, 478, 879, 952,
			1021, 1215, 1282, 2112, 2169, 2187,
			2866, 3602, 3608, 3612, 4205, 4238,
			5388, 5688, 6678, 6766, 6888, 6919
		<code>\l_tmpa_seq</code>	
			456, 467, 472, 475, 478, 860,
			862, 865, 866, 970, 974, 1061, 1062,
			1064, 1191, 1192, 1193, 1204, 1209,
			1215, 1223, 1225, 1316, 1317, 1319,
			1320, 1373, 1378, 2864, 2865, 2866,
			4236, 4237, 4238, 4859, 4861, 4862,
			4863, 4871, 6472, 6474, 6761, 6763,

S

<code>sassertion (env.)</code>	101, 157, 7415
<code>scb (env.)</code>	9662
<code>\scb</code>	9038, 9545, 9710, 9731
<code>\scc</code>	9715, 9732
<code>\scriptsize</code>	8535
<code>\scriptstyle</code>	8541
<code>sdefinition (env.)</code>	101, 157, 7410
<code>\section</code>	26, 84, 139
<code>\sectiontitleemph</code>	8348, 8407
<code>sectocframes</code>	60, 108
seq commands:	
<code>\seq_clear:N</code>	142, 180, 456,
	861, 864, 2079, 2181, 2676, 3070,
	3071, 3072, 3073, 3089, 3213, 3310,
	4125, 4859, 5133, 5134, 5142, 5705,
	5773, 5838, 5847, 6348, 6628, 6761,
	6908, 7054, 7089, 7236, 7271, 8258
<code>\seq_count:N</code> ..	2122, 2144, 3595, 7383
<code>\seq_gclear:N</code>	2669, 2670, 5369
<code>\seq_get_right:NN</code> ...	9069, 9164, 9169
<code>\seq_gpush:Nn</code>	64, 801
<code>\seq_gput_right:Nn</code> ..	2634, 2654, 5398
<code>\seq_gset_eq:NN</code>	
	 244, 252, 3109, 3110, 3111, 3112
<code>\seq_gset_split:Nnn</code>	5387
<code>\seq_if_empty:NNTF</code>	178,
	196, 264, 273, 299, 930, 946, 1193,
	1278, 1319, 2113, 2170, 4201, 7333,
	7707, 7723, 7955, 8013, 8038, 8267
<code>\seq_if_empty_p:N</code>	286, 287, 2189
<code>\seq_if_exist:NNTF</code>	5386
<code>\seq_if_in:NnNTF</code> 63, 800, 2193, 2204,	
	2557, 2672, 2791, 2799, 3091, 3284,
	3334, 4134, 4973, 5092, 5220, 6637
<code>\seq_item:Nn</code>	265, 266,
	274, 2123, 2145, 5454, 6458, 6464, 7384
<code>\seq_map_break:</code>	960, 1092, 4112
<code>\seq_map_break:n</code>	3198,
	3206, 3384, 3412, 3420, 4115, 4224

6766, 6776, 6777, 6908, 6911, 6913,
 6918, 6946, 8258, 8260, 8267, 8268
 \l_tmpb_seq 1063,
 1064, 1065, 1224, 1226, 1228, 1229
 \seqmap 98, 151, 296, 5750, 5892
 \setbox 144, 1899, 1980, 7700, 7913, 8665,
 8671, 8687, 9268, 9328, 9376, 9427
 \setcounter 10288, 10291
 \setkeys ... 2220, 2236, 2253, 8714, 10510
 \setlength 7811, 7812, 7813, 8487, 8488,
 8492, 8545, 8546, 8547, 9199, 9200,
 9201, 9473, 9474, 9475, 9492, 9493,
 9494, 9520, 9521, 9522, 9685, 9686,
 9687, 10102, 10103, 10104, 10142,
 10143, 10144, 10219, 10220, 10221
 \setlicensing 62, 110
 \setmetatheory 143, 2575, 2642
 \setnotation 32, 93, 153, 6593
 \setsectionlevel 26,
 84, 139, 1676, 8359, 8361, 8423, 8425
 \setSGvar 63, 112, 8875, 8885
 \setslidelogo 62, 110
 \setsource 62, 110
 sexample (env.) 101, 157, 7430
 \sf 8732
 \sffamily 8741
 sfragment (env.) 84, 139, 1547
 sfragment commands:
 _sfragment_do_level:nn
 1563, 1567, 1571, 1572,
 1573, 1574, 1575, 1579, 1591, 8400
 _sfragment_end: 1558, 1587, 1604, 1626
 \shorttitle 1614, 1615
 \skipfragment 84, 139, 1660
 \slideframewidth . 8491, 8492, 8581, 8710
 \slideheight 8488
 slides 60, 108
 \slidewidth
 .. 8487, 8584, 8620, 8710, 8712, 8716
 \smacro 94, 95
 \small 8114, 9112, 9117,
 9205, 9210, 9285, 9345, 9393, 9445,
 10021, 10026, 10063, 10068, 10109,
 10114, 10150, 10155, 10182, 10308
 smodule (env.) 88, 142, 2289
 \smodule 3301
 \Sn 19, 91, 6043
 \sn 19, 23, 91, 152, 1463, 6004
 \Sn\Sns 152, 6004
 \Sns 19, 91, 6044
 \sns 19, 91, 152, 6004
 \solution 68
 solution (env.) 1, 9232
 \solution 9036, 9295, 9546, 9711
 solutions 76, 113, 118
 \solutionsfalse 8924, 9300
 \solutionstrue 8922, 9297
 \source 124
 sparagraph (env.) 101, 157, 7433
 \spfarg 159, 7694, 8102
 spfblock (env.) 158, 8059
 \spfblock 7691, 8065
 \spfby 158, 7693, 8098
 \spfescape 7690, 7770, 7773
 \spfjust 158, 7692, 8094
 \spfsketch 158, 7947
 spfsketchenv (env.) 158, 7946
 \spfsketchenvautorefname 7965
 \spfstep 158, 7685, 8069
 \spfstepautorefname 7821, 8057
 spfststepenv (env.) 158, 8056
 \spfstepenv 7981, 8074
 \spfstepenvautorefname 8057
 sproblem (env.) 9035
 \sproblemautorefname 9134
 sproof (env.) 103, 158, 7664
 \sproofautorefname 7786
 \sproofend 159, 7788, 7805, 8106
 \sqcap 8209
 \square 8107, 9592, 9623
 \sr 19, 23, 90, 152, 5997
 \sref 86, 87, 101, 1840, 8826
 \sreflabel 86, 88, 1811
 \srefsetin 86, 2011
 \srefsym 91, 2028
 \srefsymuri 91, 2030, 2033
 \startsolutions 114, 9296
 \stepcounter 1662, 1666, 1670,
 1671, 1672, 1673, 1674, 8401, 8596
 \TeX 14, 16, 83
 \stex 16, 23, 83
 stex commands:
 _l_stex_aB_args_seq 295,
 5688, 5705, 5761, 5773, 5778, 5812,
 5819, 5838, 5844, 5848, 5859, 5868,
 6888, 6909, 6918, 6919, 6936, 6946
 _stex_activate_module:N
 143, 2398, 2416, 2552, 2552, 2813, 2820
 _stex_activate_module:n 143, 2384,
 2387, 2552, 2553, 2556, 2958, 5143,
 7072, 7114, 7116, 7126, 7141, 7247
 _stex_activate_notations: 2566, 6517
 _stex_activate_symbols: . 2565, 4070
 _stex_add_definiens:nn
 157, 3306, 7459,
 7464, 7464, 7677, 7679, 7870, 7872

`\stex_add_morphism:nnnn` 145, 2960,
3012, 3012, 3017, 3135, 7069, 7111
`\stex_add_notation:nnnnn`
153, 3164, 6488, 6495, 6502, 6515, 6595
`\stex_add_symbol:nnnnnnN` .. 147, 4054
`\stex_add_symbol:nnnnnnnN`
..... 148, 3188,
3193, 3854, 3922, 4054, 7040, 7370
`\l_stex_all_module_seq` 5142
`\l_stex_all_modules_seq`
..... 142, 2079, 2288, 2372,
2432, 2557, 2559, 2676, 4117, 4240
`\l_stex_allow_semantic_bool`
..... 151, 4605, 4626,
4627, 4629, 4696, 4966, 4998, 5023,
5065, 5247, 5271, 5358, 5367, 5482,
5583, 5589, 5607, 5625, 5658, 5781,
5871, 5904, 5929, 6126, 6149, 7182
`\stex_annotate:nn`
..... 128, 129, 709, 714, 717,
1531, 1541, 2101, 2102, 3236, 3238,
3579, 4009, 4030, 4033, 4040, 4045,
4047, 4387, 4390, 4397, 4401, 4404,
4541, 4546, 4549, 4556, 4560, 4563,
4699, 4885, 4894, 4899, 5118, 5639,
5672, 5787, 5970, 6177, 6180, 6674,
6681, 6687, 6688, 6697, 6708, 6718,
7292, 7319, 7466, 7506, 7517, 7695,
7746, 7749, 7752, 7887, 7902, 8036,
8091, 8095, 8099, 8103, 8265, 8270,
8610, 8801, 9077, 9131, 9177, 9476,
9484, 9508, 9513, 9601, 9605, 9613,
9739, 9743, 9751, 9845, 9880, 10224
`\stex_annotate_env` 2314, 9693
`\stex_annotate_force_break:n` ...
. 129, 712, 1584, 1646, 3237, 3578,
4027, 4045, 4384, 4402, 4539, 4561,
4898, 5034, 5112, 5644, 5671, 5675,
5687, 5792, 6674, 7292, 7320, 7467,
7507, 7695, 7698, 7741, 7912, 8266,
9077, 9081, 9177, 9184, 9851, 9884
`\stex_annotate_invisible:n` . 129,
709, 710, 1656, 2322, 2608, 3062,
3564, 4008, 5410, 6674, 7744, 8263
`\stex_annotate_invisible:nn`
..... 129, 709,
1506, 1701, 1707, 1713, 2927, 2963,
3000, 3577, 3651, 3691, 3701, 4365,
4523, 5588, 5606, 5624, 5903, 7066,
7108, 7244, 7492, 8959, 8982, 10232
`\stex_apply_patch_begin:n`
..... 525, 541, 3440, 3506
`\stex_apply_patch_end:n`
..... 531, 556, 3445, 3511
`\stex_archive_base:N`
..... 132, 884, 884, 1079
`\stex_archive_base:n`
132, 884, 888, 1118, 1127, 1245, 1414
`\stex_archive_id:N` 132, 881,
881, 903, 1072, 1076, 1079, 1080,
1085, 1086, 1174, 1362, 2014, 2183
`\stex_archive_path:N`
..... 132, 891, 891, 1100, 1222, 2182
`\stex_archive_path:n`
..... 132, 891, 895, 1110
`\stex_args_end:` 5372,
5396, 5399, 6133, 6157, 6168, 6172
`\stex_assign_do:n`
..... 3251, 3564, 3566, 3573, 3625
`\l_stex_assoc_args_count`
..... 3954, 3958, 4000, 4001
`\l_stex_brackets_dones_bool`
..... 6816, 6957, 6963, 6964, 6981
`\stex_check_term:nn`
152, 2678, 3575, 6217, 6218, 6226,
6232, 6236, 6240, 6245, 6665, 8123
`\stex_check_terms:` ... 152, 3853,
3917, 4310, 4475, 6228, 6229, 6249
`\c_stex_check_terms_bool`
..... 36, 6204, 6207, 8125
`\stex_close_module:`
... 142, 2342, 2434, 2434, 2615, 8237
`\stex_comma_sep_uri:nn`
..... 7334, 7344, 7708, 7724
`\stex_css_link:n` 129, 712
`\stex_css_literal:n`
..... 129, 81, 712, 818, 1722, 8555
`\l_stex_current*` 149, 150
`\l_stex_current_archive`
.... 132, 133, 902, 903, 912, 923,
1060, 1099, 1100, 1172, 1174, 1221,
1222, 1360, 1362, 2178, 2182, 2183
`\l_stex_current_args_tl`
.. 4649, 5371, 5372, 5375, 6741, 6756
`\l_stex_current_arity_str`
..... 4648, 4716, 4731,
4733, 5343, 5492, 5493, 5528, 6762
`\l_stex_current_def_uri`
3225, 3227, 3230, 3234, 3236, 3242,
7355, 7377, 7378, 7384, 7385, 7452,
7454, 7457, 7459, 7501, 7503, 7506
`\l_stex_current_display_tl`
..... 149, 4609,
5250, 5266, 5269, 5655, 5775, 6019,
6023, 6036, 6040, 6051, 6065, 6079,
6103, 6113, 6151, 6361, 6776, 7189
`\l_stex_current_doc_uri` 7337

<code>\l_stex_current_document_uri</code> . . .	<code>\l_stex_current_symbol_arity_int</code>
. 125, 135, 144, 253, 254,	 149, 150, 272
259, 1159, 1221, 1295, 1315, 1334,	<code>\l_stex_current_symbol_invoke_cs</code> 149
1338, 1343, 1345, 1452, 2410, 8128	<code>\l_stex_current_symbol_name_str</code> .
<code>\l_stex_current_domain_str</code> . . . 3719	 6129, 6130, 6131, 6132
<code>\l_stex_current_domain_uri</code>	<code>\l_stex_current_symbol_return_tl</code>
. . . 3019, 3032, 3036, 3038, 3043,	 149, 150, 272
3051, 3059, 3075, 3079, 3108, 3137,	<code>\l_stex_current_symbol_type_tl</code> . .
3162, 3457, 3482, 3524, 3548, 3668	 149, 150, 272
<code>\g_stex_current_file</code>	<code>\l_stex_current_symbol_uri</code>
. . 125, 244, 252, 257, 1102, 1112,	 149, 150,
1374, 2407, 2881, 9068, 9163, 9168	272, 294, 3782, 3795, 3796, 3801,
<code>\l_stex_current_full_tl</code> 149,	3813, 3849, 3864, 3873, 3878, 3881,
4608, 4617, 4656, 4661, 4676, 4689,	3887, 3912, 3933, 4607, 4608, 4610,
4701, 4829, 4912, 5038, 5070, 5114,	5653, 6127, 6128, 6129, 6982, 6987
5249, 5259, 5265, 5268, 5275, 5310,	<code>\l_stex_current_term_tl</code> 4631, 4652,
5311, 5322, 5335, 5336, 5365, 5446,	4905, 4911, 5109, 5272, 5554, 5570,
5473, 5641, 5654, 5774, 5824, 5872,	5584, 5590, 5605, 5608, 5623, 5626,
5873, 5972, 5975, 6018, 6035, 6050,	5656, 5782, 5783, 5784, 5874, 6136
6064, 6078, 6128, 6141, 6150, 6160,	<code>\stex_current_this:</code> . 4645, 7181, 7197
6161, 6163, 6360, 6696, 6707, 7184	<code>\l_stex_current_this_tl</code>
<code>\l_stex_current_language_str</code> . . .	 7006, 7009, 7183, 7191
. 125, 127, 254, 258, 453,	<code>\l_stex_current_type_tl</code>
458, 459, 1195, 1201, 1206, 1235,	 4651, 4698, 4771, 4826,
2316, 2850, 2852, 2868, 9073, 9173	4834, 4837, 4841, 5143, 5144, 5219
<code>\l_stex_current_module_str</code>	<code>\stex_deactivate_macro:Nn</code>
. 2614, 3684, 3693, 3703	 123, 67, 67, 2598, 2939,
<code>\l_stex_current_module_uri</code>	2946, 2994, 3295, 3296, 3297, 3298,
142, 1293, 1422, 2080, 2288, 2315,	3299, 3300, 3301, 3464, 3470, 3489,
2336, 2371, 2403, 2420, 2431, 2437,	3530, 3536, 3556, 3571, 3646, 3715,
2438, 2439, 2440, 2441, 2446, 2448,	3776, 3807, 3896, 6098, 6107, 6117,
2452, 2457, 2459, 2535, 2536, 2541,	6123, 6306, 7027, 7082, 7132, 7462,
2561, 2562, 2567, 2568, 2570, 2592,	7496, 7511, 7519, 7773, 7944, 8002,
2677, 3013, 4055, 4062, 4071, 4072,	8065, 8089, 8093, 8097, 8101, 8105,
4084, 4089, 6506, 6508, 6518, 6519,	8280, 9295, 9352, 9400, 9452, 9505,
6527, 7121, 7365, 7377, 7471, 8235	9544, 9545, 9546, 9547, 9548, 9549,
<code>\l_stex_current_ns_uri</code> 2302	9566, 9661, 9709, 9710, 9711, 9712,
<code>\l_stex_current_redo_tl</code>	9713, 9714, 9731, 9796, 9917, 10246
4632, 4637, 4644, 4654, 4655, 4658,	<code>\stex_debug:nn</code> 124, 103, 103,
4830, 4868, 4904, 5060, 5254, 5524	133, 137, 166, 173, 237, 472, 547,
<code>\l_stex_current_return_tl</code>	616, 847, 852, 880, 922, 931, 949,
. . 4650, 4666, 5312, 5324, 5344, 5529	996, 1033, 1067, 1075, 1096, 1166,
<code>\l_stex_current_section_level_-</code>	1237, 1315, 1353, 1366, 1381, 1390,
int 139, 1526, 1562,	1498, 1851, 1876, 1878, 1887, 1891,
1564, 1568, 1582, 1599, 1600, 1629,	1930, 1936, 1950, 1953, 1956, 1979,
1631, 1634, 1637, 1638, 1645, 1661,	2081, 2112, 2169, 2187, 2191, 2202,
1663, 1667, 1678, 1679, 1680, 1681,	2355, 2362, 2364, 2392, 2395, 2397,
1682, 1683, 1685, 1689, 1692, 1699,	2400, 2412, 2420, 2422, 2424, 2436,
1703, 1709, 1713, 1718, 8404, 8411	2484, 2536, 2558, 2567, 2572, 2576,
<code>\l_stex_current_section_level_-</code>	2578, 2679, 2732, 2753, 2757, 2775,
str 139, 1526, 1533, 1543, 1602, 8412	2792, 2800, 2882, 2897, 2899, 2925,
<code>\l_stex_current_symbol_args_tl</code> . .	2998, 3033, 3051, 3093, 3121, 3158,
. 149, 150, 272	3173, 3230, 3249, 3350, 3406, 3574,
	3593, 3596, 3600, 3620, 3624, 3649,

3669, 3685, 3689, 3699, 3720, 3722,
 3795, 4055, 4071, 4084, 4198, 4239,
 4241, 4260, 4277, 4279, 4316, 4444,
 4661, 4677, 4770, 5014, 5141, 5157,
 5280, 5299, 5311, 5313, 5316, 5322,
 5335, 5357, 5365, 5480, 5520, 5531,
 5701, 5716, 5718, 5725, 5728, 6230,
 6234, 6238, 6242, 6389, 6420, 6431,
 6503, 6518, 6526, 6528, 6531, 6667,
 6694, 6795, 6808, 6849, 6967, 7139,
 7221, 7224, 7228, 7378, 7382, 7385,
 7387, 7457, 7478, 7514, 7675, 7868
 \c_stex_debug_clist 30,
 42, 104, 107, 121, 125, 127, 131, 135
 \c_stex_default_metatheory
 2078, 2675, 8236
 \l_stex_default_notation
 ... 4691, 5325, 5330, 5350, 5808,
 6744, 6745, 6751, 6760, 6765, 6768
 \stex_definiens_impl:nn
 3307, 7444, 7449
 \stex_do_default_notation:
 ... 154, 4690, 5323, 5807, 6739, 6739
 \stex_do_default_notation_op: ...
 154, 5347, 6739, 6740, 6749
 \stex_do_deprecation:n 649, 649, 3936
 \stex_do_for_list: ... 157, 3305,
 7270, 7270, 7354, 7670, 7863, 7950
 \stex_do_id: 654, 654,
 1556, 1623, 7299, 7317, 7683, 7883,
 7896, 7907, 7951, 7985, 9099, 10299
 \stex_do_up_to_module:n
 142, 2530, 2530,
 2533, 2537, 4065, 6510, 7125, 7172
 \g_stex_document_kind_parameters_
 tl 1480, 1486, 10458
 \g_stex_document_kind_str
 14, 1479, 1481, 1485, 10446
 \stex_document_uri:n 134
 \stex_document_uri_archive:N ...
 134, 1137, 1137, 1338
 \stex_document_uri_element:N ...
 135, 1149, 1149
 \stex_document_uri_from_archive_
 file:Nn 135,
 1164, 1164, 1875, 2043, 2088, 8816
 \stex_document_uri_language:N ...
 135, 254, 1146, 1146, 1235
 \stex_document_uri_name:N
 135, 1143, 1143, 1343, 1345
 \stex_document_uri_path:N
 135, 1140, 1140, 1334
 \stex_document_uri_with_language:Nn
 135, 1152, 1152, 2409
 \stex_eat_exclamation_point: ...
 150, 3942, 5314, 5325, 5546, 5546, 5547
 \stex_end: 414, 422
 \stex_end: 7207, 7217
 \stex_every_module:n
 142, 2526, 2527,
 2599, 2947, 2995, 3465, 3471, 3490,
 3531, 3537, 3557, 3777, 3808, 3897,
 6307, 7028, 7083, 7134, 7178, 8281
 \g_stex_every_module_tl
 2353, 2526, 2528, 8133
 \l_stex_every_symbol_tl 4586, 4588,
 4589, 4595, 4596, 4599, 4628, 4654
 \stex_execute_in_module:n
 142, 2377, 2534, 2534,
 2539, 2957, 3880, 7071, 7100, 7113
 \l_stex_feature_name_str .. 3054,
 3136, 3172, 3173, 3179, 3362, 3661
 \stex_file_in_smsmode:Nn
 144, 2413, 2665, 2668, 2913
 \stex_file_resolve:Nn 124, 147, 153,
 169, 176, 233, 243, 251, 867, 869, 1373
 \stex_file_set:Nn
 124, 141, 141, 146, 160, 171, 860, 1165
 \stex_file_split_off_ext:NN
 124, 208, 208
 \stex_file_split_off_lang:NN ...
 124, 215, 215, 2407, 9068, 9163, 9168
 \stex_file_use:N ... 124, 166, 173,
 205, 205, 237, 840, 860, 862, 870,
 874, 962, 996, 997, 999, 1012, 1065,
 1102, 1112, 1166, 1181, 1374, 1378,
 1799, 2190, 2201, 2412, 2414, 2881
 \l_stex_fors_seq
 157, 3213, 3216, 7271, 7274, 7333,
 7334, 7364, 7383, 7384, 7399, 7418,
 7674, 7707, 7708, 7723, 7724, 7867,
 7955, 7956, 8013, 8014, 8038, 8039
 \stex_get_env:Nn
 124, 86, 87, 98, 119, 229,
 231, 239, 241, 568, 574, 667, 838, 6201
 \stex_get_in_morphism:n 145, 3215,
 3224, 3349, 3349, 3562, 3618, 3641
 \stex_get_mathstructure:n
 156, 3035, 7056, 7090, 7198, 7198, 7227
 \stex_get_mathstructure_maybe:nTF
 7199, 7203, 7220
 \l_stex_get_structure_module_uri
 156, 3036,
 7098, 7101, 7204, 7207, 7221, 7228
 \l_stex_get_svariable_str 6154
 \stex_get_symbol:n
 148, 2029, 4150, 4152,

4573, 6000, 6017, 6034, 6148, 6284,
 6610, 7273, 7451, 7500, 8944, 8967
 \stex_get_symbol:nTF ... 148, 1462,
 4150, 4153, 4158, 4178, 4179, 7205
 \l_stex_get_symbol_args_tl
 148, 3387, 3926,
 3946, 3948, 3949, 4187, 4227, 4288,
 4343, 4355, 4448, 4501, 4514, 4577,
 5375, 6555, 6557, 6694, 6756, 7373
 \l_stex_get_symbol_arity_int ...
 148, 3386, 3869, 3925, 3947,
 3957, 3963, 3967, 3971, 3975, 3979,
 3983, 3987, 3991, 3995, 3998, 3999,
 4000, 4001, 4038, 4150, 4186, 4226,
 4287, 4315, 4342, 4354, 4395, 4447,
 4500, 4513, 4554, 4576, 6350, 6384,
 6415, 6422, 6442, 6491, 6596, 7372
 \l_stex_get_symbol_def_tl
 148, 3263,
 3388, 4188, 4228, 4289, 4449, 4578
 \l_stex_get_symbol_invoke_cs ...
 148, 3391,
 4191, 4231, 4292, 4452, 4581, 7206
 \l_stex_get_symbol_macro_str . 3385
 \l_stex_get_symbol_name_str .. 3256
 \l_stex_get_symbol_return_tl ...
 148, 3390, 4190,
 4230, 4291, 4317, 4451, 4466, 4580
 \l_stex_get_symbol_type_tl
 148, 3389, 4189, 4229, 4290,
 4450, 4579, 7057, 7092, 7207, 7237
 \l_stex_get_symbol_uri . 148, 1463,
 2030, 3217, 3225, 3248, 3255, 3351,
 3356, 3360, 3376, 3397, 3574, 3577,
 3585, 3649, 3652, 3659, 3661, 3796,
 3873, 4159, 4175, 4185, 4225, 4286,
 4575, 6018, 6020, 6035, 6037, 6127,
 6150, 6152, 6290, 6294, 6312, 6313,
 6489, 6600, 6603, 6611, 6612, 6613,
 6614, 6619, 6623, 7275, 7452, 7501,
 8945, 8956, 8960, 8968, 8979, 8983
 \stex_get_var:n 149, 4419,
 4419, 6048, 6060, 6074, 6322, 7490
 \l_stex_get_variable_str
 ... 4420, 4422, 4446, 6050, 6051,
 6062, 6064, 6065, 6076, 6078, 6079,
 6155, 6156, 6323, 6328, 6331, 7492
 \stex_has_definiens:N 4093
 \stex_has_definiens:n 4096
 \stex_has_definiens:NTF ... 147, 4093
 \stex_has_definiens:nTF
 147, 4093, 4094
 \stex_has_definiens_p:N ... 147, 4093
 \stex_has_definiens_p:n ... 147, 4093
 \c_stex_home_file .. 125, 238, 840, 860
 \stex_html_literal:n
 ... 27, 35, 764, 772, 1689, 1692, 2055
 \stex_html_node:nnn
 128, 129, 709, 1484, 1583
 \stex_if_check_terms: 6197, 6208, 6212
 \stex_if_check_terms:TF
 152, 3798, 4310, 4319, 4475,
 4478, 6195, 6217, 6228, 6286, 6325
 \stex_if_check_terms_p: ... 152, 6195
 \stex_if_do_html: 695
 \stex_if_do_html:TF 128, 695, 704,
 1505, 1529, 1539, 1687, 2334, 2344,
 2601, 2616, 2926, 2962, 2999, 3056,
 3114, 3231, 3563, 3576, 3650, 3800,
 3865, 3876, 3919, 4313, 4321, 4364,
 4474, 4480, 5669, 6288, 7065, 7107,
 7243, 7376, 7465, 7491, 7671, 7676,
 7682, 7696, 7781, 7793, 7801, 7864,
 7869, 7879, 7900, 7903, 7910, 7921,
 7941, 8009, 8033, 8743, 9066, 9098,
 9105, 9106, 9109, 9129, 9166, 9194,
 9196, 9250, 9259, 9272, 9277, 9313,
 9321, 9332, 9337, 9361, 9369, 9380,
 9385, 9412, 9421, 9432, 9437, 9471,
 9526, 9551, 9553, 9600, 9691, 9716,
 9718, 9738, 9864, 9870, 10223, 10293
 \stex_if_do_html_p: 128, 695
 \stex_if_file_absolute:N ... 263, 272
 \stex_if_file_absolute:NTF
 125, 262, 868
 \stex_if_file_absolute_p:N . 125, 262
 \stex_if_file_starts_with:NN 125, 279
 \stex_if_file_starts_with:NNTF ..
 125, 279, 1062
 \stex_if_html_backend 518
 \stex_if_html_backend:TF ... 128,
 179, 2, 529, 685, 688, 712, 725,
 739, 1482, 1578, 1642, 1655, 1697,
 2048, 2099, 2312, 4634, 4665, 5251,
 5550, 5566, 5580, 5600, 5618, 5637,
 5667, 5969, 6125, 6146, 6175, 6196,
 7290, 7303, 8304, 8344, 8474, 8554,
 8609, 8621, 8629, 8641, 8651, 8800,
 8811, 8854, 9590, 9733, 9836, 9873
 \stex_if_html_backend_p: ... 128, 685
 \stex_if_in_module: 2540
 \stex_if_in_module:TF
 143, 1292, 2347, 2534, 2540, 6594, 7470
 \stex_if_in_module_p: 143, 2540
 \stex_if_module_exists:N 2544
 \stex_if_module_exists:n 2548
 \stex_if_module_exists:NTF
 143, 1350, 2363,

2396, 2423, 2544, 2808, 2817, 2914
 \stex_if_module_exists:nTF
 143, 2544, 7058, 7093, 7238
 \stex_if_module_exists_p:N 143, 2544
 \stex_if_module_exists_p:n 143, 2544
 \stex_if_smsmode: 2662
 \stex_if_smsmode:TF
 . 144, 655, 1496, 1812, 1831, 2335,
 2343, 2660, 2880, 2930, 2966, 3003,
 3243, 3432, 3447, 3455, 3480, 3498,
 3514, 3522, 3546, 3772, 3803, 3885,
 6297, 7219, 7283, 7302, 7318, 7324,
 7398, 7417, 7431, 7469, 9194, 9196
 \stex_if_smsmode_p: 144, 2660
 \stex_ignore_spaces_and_pars: ...
 123, 45, 45, 48,
 2591, 2593, 7324, 9122, 10048, 10090
 \l_stex_import_uri 2577,
 2578, 2580, 2919, 2920, 2928, 2932,
 2933, 2951, 2952, 2958, 2961, 2964,
 2969, 2970, 2977, 2980, 2990, 2991,
 3001, 3005, 3006, 3039, 3041, 3043
 \stex_in_archive:nn
 132, 898, 898, 1874, 1937,
 2018, 2042, 2087, 2110, 2138, 2167,
 2211, 2227, 2242, 2254, 2839, 8815
 \g_stex_in_comp_bool
 5663, 5963, 5968, 5979
 \stex_in_invisible_html_bool ...
 4028, 4385, 5549, 5582
 \l_stex_in_meta_bool
 2374, 2383, 2386, 2388
 \l_stex_in_this_bool
 5066, 5552, 5562,
 5568, 5578, 5602, 5616, 5620, 5634,
 5670, 5684, 5930, 7179, 7190, 7192
 \l_stex_inpararray_bool 6969
 \stex_input_with_hooks:Nn .. 125,
 245, 245, 248, 2062, 2082, 2090, 2093
 \stex_invoke_sequence:
 ... 150, 4505, 4518, 4669, 4669, 5743
 \stex_invoke_structure:
 150, 4758, 4758, 7042, 7206
 \stex_invoke_symbol:
 149, 3930, 4347, 4359, 4660, 4660, 7374
 \stex_invoke_symbol:NnnnnnN ...
 149, 4574, 4604, 4622, 4625
 \stex_invoke_symbol:nnnnnnN ...
 149, 4088, 4166, 4183, 4184,
 4604, 4604, 4623, 5168, 5182, 7156
 \stex_invoke_text_symbol:
 149, 3861, 4664, 4664
 \stex_invoke_variable:nnnnnn ...
 150, 5246
 \stex_invoke_variable:nnnnnnN ..
 4352, 4511, 5246, 5741
 \stex_is_sequentialized:n
 5696, 5779, 5852, 5869
 \stex_iterate_break:
 148, 4107, 4108, 4111
 \stex_iterate_break:n
 148, 3326, 3690,
 3700, 3718, 4107, 4108, 4114, 4285
 \stex_iterate_morphisms:nn
 145, 3309, 3309, 3668, 3684
 \stex_iterate_notations:nn
 154, 3162, 5219, 6627, 6627
 \stex_iterate_symbols:n
 148, 4107, 4107, 4278
 \stex_iterate_symbols:nn
 ... 148, 3079, 3721, 4124, 4124, 5144
 \l_stex_key_* 147
 \l_stex_key_answerclass_str
 9235, 9239, 9253, 9287, 9288
 \l_stex_key_archive_str
 379, 382, 1874, 1937
 \l_stex_key_args_str
 3732, 3737, 3746, 3786,
 3817, 3950, 3959, 3964, 3968, 3972,
 3976, 3980, 3984, 3988, 3992, 3996,
 4011, 4367, 4461, 4462, 4525, 7255
 \l_stex_key_argtypes_clist
 3751, 3755, 4044, 4046, 4400, 4403,
 4559, 4562, 6243, 6244, 6245, 7259
 \l_stex_key_assoc_str 3735, 3740,
 4018, 4019, 4371, 4372, 4529, 4530
 \l_stex_key_autogradable_bool ...
 9009, 9015, 9020, 9074, 9174
 \l_stex_key_continues_tl . 7579, 7589
 \l_stex_key_def_tl 3748, 3758, 3785,
 3816, 3831, 3835, 3858, 3927, 4032,
 4033, 4344, 4356, 4389, 4390, 4502,
 4515, 4548, 4549, 6235, 6236, 7257
 \stex_key_def_tl 152
 \l_stex_key_deprecate_str
 405, 407, 650, 651
 \l_stex_key_due_tl
 10269, 10273, 10313, 10314
 \l_stex_key_fallback_tl
 1803, 1856, 1888, 1954
 \l_stex_key_feedback_tl
 9457, 9460, 9483, 9486,
 9499, 9500, 9575, 9580, 9612, 9614,
 9648, 9649, 9750, 9752, 9782, 9783,
 10214, 10231, 10234, 10241, 10242
 \l_stex_key_file_str
 380, 383, 1843, 1875, 1882, 1938, 2019

<code>\l_stex_key_for_clist</code>	6364, 6493, 6585, 6586, 6589, 6598,
157, 3214, 7254, 7262, 7272, 7523, 7528	6602, 6604, 6614, 6616, 6705, 6709
<code>\l_stex_key_for_str</code>	7289
<code>\l_stex_key_from_tl</code>	7581, 7591
<code>\l_stex_key_Ftext_tl</code>	
.. 9578, 9584, 9609, 9646, 9747, 9780	
<code>\l_stex_key_functions_tl</code> .	7580, 7590
<code>\l_stex_key_given_tl</code>	
..... 10268, 10272, 10310, 10311	
<code>\l_stex_key_hide_bool</code>	7578, 7586,
7700, 7710, 7726, 7793, 7913, 7941	
<code>\l_stex_key_id_str</code> .	387, 389, 656,
657, 7336, 7337, 7392, 7394, 7880,	
7893, 7982, 9244, 9246, 9252, 9307,	
9309, 9315, 9355, 9357, 9363, 9405,	
9407, 9414, 9465, 9467, 9477, 9495,	
10038, 10080, 10127, 10168, 10188,	
10200, 10202, 10213, 10222, 10225	
<code>\l_stex_key_intent_args_clist</code> ...	
..... 6258, 6264, 6456, 6465, 6467	
<code>\l_stex_key_intent_str</code>	
..... 3871, 6257, 6263, 6381, 6459	
<code>\l_stex_key_just_tl</code>	
..... 7526, 7531, 7742, 7748, 7749	
<code>\l_stex_key_macroname_str</code>	
.. 7253, 7263, 7349, 7351, 7367, 7371	
<code>\l_stex_key_method_tl</code>	
..... 7525, 7530, 7742, 7751, 7752	
<code>\l_stex_key_mhrepos_str</code>	
. 9014, 9022, 9218, 9220, 9228, 10326	
<code>\l_stex_key_min_tl</code>	9012,
9018, 9089, 9192, 9210, 10114, 10155	
<code>\l_stex_key_name_str</code>	3745,
3753, 3783, 3814, 3829, 3833, 3843,	
3844, 3846, 3850, 3856, 3888, 3909,	
3910, 3913, 3924, 3936, 4010, 4305,	
4306, 4316, 4323, 4326, 4339, 4341,	
4353, 4366, 4458, 4459, 4482, 4485,	
4497, 4499, 4512, 4524, 6323, 6580,	
6581, 6582, 6586, 6587, 6588, 7252,	
7261, 7288, 7350, 7351, 7357, 7365,	
7371, 7377, 7537, 7541, 7548, 7551,	
7572, 7730, 7731, 7831, 7838, 7843,	
7968, 7970, 7976, 7977, 8018, 8019,	
8043, 8044, 9013, 9019, 9067, 9069,	
9072, 9162, 9164, 9167, 9169, 9172	
<code>\l_stex_key_number_tl</code>	
..... 10267, 10271, 10287, 10288	
<code>\l_stex_key_numname_str</code>	
7538, 7542, 7547, 7556, 7718, 7729,	
7832, 7837, 7842, 8006, 8017, 8042	
<code>\l_stex_key_op_tl</code>	
... 3870, 4468, 4470, 6256, 6262,	
6343, 6344, 6345, 6352, 6353, 6358,	
6364, 6493, 6585, 6586, 6589, 6598,	
6602, 6604, 6614, 6616, 6705, 6709	
<code>\l_stex_key_post_tl</code>	
... 1802, 1862, 1867, 5984, 5988,	
6023, 6040, 6062, 6076, 6103, 6113	
<code>\l_stex_key_pre_tl</code>	
... 1801, 1862, 1867, 5983, 5987,	
6023, 6040, 6062, 6076, 6103, 6113	
<code>\l_stex_key_prec_str</code>	3872,
6255, 6261, 6392, 6395, 6398, 6404,	
6407, 6410, 6413, 6419, 6431, 6432	
<code>\l_stex_key_proofend_tl</code>	
..... 7577, 7583, 8113, 8114	
<code>\l_stex_key_pts_str</code>	
9456, 9459, 9478, 9479, 9496, 9497,	
10215, 10226, 10227, 10238, 10239	
<code>\l_stex_key_pts_tl</code>	9011,
9017, 9075, 9084, 9088, 9175, 9186,	
9189, 9205, 10109, 10132, 10150, 10173	
<code>\l_stex_key_reorder_str</code>	3734, 3738,
4021, 4022, 4377, 4378, 4535, 4536	
<code>\l_stex_key_return_tl</code>	
3749, 3754, 3929, 4035, 4038, 4317,	
4346, 4358, 4392, 4395, 4466, 4504,	
4517, 4551, 4554, 6239, 6240, 7258	
<code>\stex_key_return_tl</code>	152
<code>\l_stex_key_role_str</code>	
..... 3733, 3741, 3848, 4024,	
4025, 4374, 4375, 4532, 4533, 7368	
<code>\l_stex_key_root_tl</code>	
..... 5985, 5989, 5993, 5995	
<code>\l_stex_key_short_tl</code>	
.. 1548, 1550, 1593, 1596, 1613, 1615	
<code>\l_stex_key_showname_bool</code> .	7534,
7539, 7543, 7549, 7554, 7571, 7833	
<code>\l_stex_key_sig_str</code> ..	2292, 2307,
2317, 2351, 2392, 2410, 2412, 2414	
<code>\l_stex_key_style_clist</code>	
399, 401, 504, 508, 542, 546, 7339,	
7340, 7434, 9531, 9532, 9536, 9539,	
9664, 9671, 9695, 9696, 9701, 9704	
<code>\l_stex_key_T_bool</code>	
... 9576, 9581, 9582, 9602, 9606,	
9629, 9643, 9740, 9744, 9763, 9777	
<code>\l_stex_key_term_tl</code>	
..... 7524, 7529, 7742, 7745, 7746	
<code>\l_stex_key_testspace_dim</code>	
..... 9233, 9236, 9238,	
9267, 9821, 9823, 9879, 9908, 9911	
<code>\l_stex_key_title_str</code>	7264
<code>\l_stex_key_title_tl</code>	
..... 393, 395, 1925, 1930,	
1967, 1968, 2328, 7287, 7293, 7294,	
9077, 9078, 9079, 9083, 9177, 9178,	

9179, 9181, 9182, 9286, 9287, 9346,
 9347, 9394, 9395, 9446, 9447, 10038,
 10080, 10127, 10168, 10183, 10184,
 10188, 10294, 10295, 10305, 10306
 \l_stex_key_Ttext_tl
 .. 9577, 9583, 9607, 9644, 9745, 9778
 \l_stex_key_type_tl
 3747, 3757, 3784, 3815, 3830, 3834,
 4029, 4030, 4345, 4357, 4386, 4387,
 4545, 4546, 6231, 6232, 7256, 7265
 \stex_key_type_tl 152
 \l_stex_key_variant_str
 3819, 4328, 4487,
 6254, 6260, 6267, 6269, 6311, 6333,
 6379, 6490, 6580, 6586, 6676, 6709
 \l_stex_key_wikidata_str
 3750, 3756, 4015, 4016
 \l_stex_keys_cls_id 6, 13, 36, 43, 44,
 46, 81, 743, 750, 773, 780, 781, 783, 818
 \l_stex_keys_counter_id
 7, 10, 28, 37, 61, 62, 63, 64,
 744, 747, 765, 774, 798, 799, 800, 801
 \l_stex_keys_counter_parent
 8, 11, 29, 48, 50, 51, 52, 53,
 54, 55, 56, 58, 745, 748, 766, 785,
 787, 788, 789, 790, 791, 792, 793, 795
 \stex_keys_define:nnnn 126,
 5, 359, 359, 378, 386, 392, 398,
 404, 410, 742, 1547, 1800, 1810,
 2291, 3731, 3744, 3828, 5982, 5992,
 6084, 6089, 6253, 6274, 6276, 7005,
 7251, 7522, 7536, 7576, 7594, 7967,
 8501, 9010, 9217, 9234, 9303, 9455,
 9574, 9819, 10266, 10370, 10421, 10434
 \stex_keys_set:nn
 126, 17, 374, 374, 754, 1554, 1609,
 1841, 1940, 2327, 3765, 3792, 3840,
 3841, 4303, 4456, 5999, 6016, 6033,
 6047, 6059, 6073, 6095, 6101, 6111,
 6283, 6321, 7030, 7280, 7315, 7669,
 7830, 7862, 7949, 7975, 8595, 9058,
 9062, 9152, 9227, 9243, 9266, 9306,
 9354, 9404, 9464, 9517, 9598, 9640,
 9682, 9736, 9774, 9875, 9892, 10199,
 10281, 10285, 10325, 10383, 10445
 \stex_kpsewhich:Nn 123, 77, 77, 89, 99
 \c_stex_language_abbrevs_prop ...
 127, 426
 \c_stex_languages_clist . 31, 454, 457
 \c_stex_languages_prop
 127, 222, 426, 465, 1203, 1208
 \g_stex_last_feature_str 2614
 \l_stex_macroname_str
 147, 2604, 2605, 3762, 3767, 3769,
 3793, 3842, 3855, 3923, 4012, 4013,
 4304, 4340, 4351, 4368, 4369, 4457,
 4498, 4510, 4526, 4527, 7043, 7367
 \c_stex_main_archive 133, 1060, 2014
 \c_stex_main_document_uri
 135, 1221, 1877
 \c_stex_main_file
 124, 228, 244, 1224, 1231
 _stex_map_args:N
 153, 4039, 4396, 4555, 5377,
 6452, 6478, 6554, 6554, 6701, 6758
 _stex_map_notation_args:N
 153, 6554, 6567, 6721
 \c_stex_mathhub<id>_archive .. 132
 \c_stex_mathhub_file 962
 \c_stex_mathhub_files
 ... 132, 837, 930, 946, 952, 958, 1060
 \c_stex_mathhub_main_archive ...
 1070, 1071
 _stex_maybe_brackets:nn
 154, 4678,
 5337, 5348, 6798, 6814, 6962, 6962
 \c_stex_metadata_bool 37, 8954, 8977
 \stex_metagroup_do_in:n
 125, 126, 327,
 327, 331, 348, 2531, 3232, 3583, 3657
 \stex_metagroup_new:
 126, 323, 324, 2360, 2393, 2419, 3066
 \l_stex_metatheory_uri 2078, 2289,
 2296, 2298, 2318, 2319, 2354, 2355,
 2378, 2580, 2582, 2675, 8235, 8236
 _stex_module_code_macro:N
 2282, 2366, 2426, 2438, 2457, 2491,
 2535, 2545, 2562, 2567, 2568, 2592
 _stex_module_code_macro:n
 2285, 2549, 7140
 \stex_module_setup:n
 142, 2332, 2346, 2346, 2610
 _stex_module_setup_top_nosig:n .
 2352, 2359, 8132
 \stex_module_setup_top_nosig:n ..
 142, 2359
 \stex_module_uri:n 135
 \stex_module_uri_archive:N
 135, 1251, 1251, 2832
 \stex_module_uri_as_qm:n
 136, 1285, 1285, 4241, 4242
 \stex_module_uri_name:N 136, 1260,
 1260, 2315, 2834, 2933, 2970, 3006
 \stex_module_uri_name:n
 136, 1260, 1264, 4249, 4252
 \stex_module_uri_path:N
 135, 1254, 1254, 2833

<code>\stex_module_uri_path:n</code>	<code>_stex_notation_check:</code> . 154, 3798,
..... 135, <u>1254</u> , 1257, 4261	4319, 4478, 6286, 6325, <u>6664</u> , 6664
<code>\stex_module_uri_split_name:NNN</code> .	<code>\l_stex_notation_cs</code>
..... 136, <u>1267</u> , 1267, 7098	 154, 4677, 4680, 4691,
<code>\stex_module_uri_split_name:NNn</code> .	4718, 4722, 4730, 4737, 4752, 4913,
..... 136, <u>1267</u> , 1271	5314, 5318, 5339, 5805, 6654, 6660
<code>\l_stex_morphism_assigns_seq</code> 3073,	<code>_stex_notation_do_html:n</code>
3111, 3128, 3184, 3233, 3284, 3584	 154, 3801,
<code>\l_stex_morphism_morphisms_seq</code> ..	3878, 4323, 4482, 6290, <u>6673</u> , 6673
..... 3072, 3101, 3112	<code>\l_stex_notation_downprec</code>
<code>\l_stex_morphism_renames_seq</code> 3071,	 5657, 6954, 6961, 6967, 6968
3110, 3125, 3139, 3152, 3357, 3658	<code>\l_stex_notation_macrocode_cs</code> ...
<code>\l_stex_morphism_symbols_prop</code> ...	 153, 3821, 4330, 4489, 6315,
..... 3258, 3724	6335, 6377, 6389, 6492, 6580, 6583,
<code>\l_stex_morphism_symbols_seq</code> ...	6597, 6601, 6612, 6667, 6668, 6699
..... 3070, 3080, 3109,	<code>_stex_notation_make_args:</code>
3122, 3131, 3273, 3353, 3414, 3422	 154, 3822,
<code>_stex_morphisms_macro:N</code> .. 2270,	4331, 6316, 6336, 6669, <u>6720</u> , 6720
2367, 2427, 2439, 2448, 2485, 3013	<code>\stex_notation_parse:n</code>
<code>_stex_morphisms_macro:n</code>	 153, 3797, 3874, 4318,
.. 2273, 3090, 3336, 4136, 6642, 7143	4469, 4472, 6285, 6324, <u>6342</u> , 6342
<code>\stex_new_document_element_uri:n</code>	<code>_stex_notation_set_default:n</code> ...
..... 135, <u>1158</u> , 1158, 1813	... 153, 6293, 6328, <u>6593</u> , 6593, 6618
<code>\stex_new_module_uri:n</code>	<code>_stex_notations_macro:N</code>
..... 136, <u>1291</u> , 1291,	... 2276, 2369, 2429, 2441, 2459,
2361, 2394, 2421, 7037, 7101, 7126	2492, 2518, 6508, 6518, 6519, 6527
<code>\stex_new_statement:nnn</code>	<code>_stex_notations_macro:n</code> . 2279, 6639
157, <u>7278</u> , 7278, 7410, 7415, 7430, 7433	<code>\stex_par:</code>
<code>\stex_new_stylable_cmd:nnnn</code> 127,	.. 9665, 9666, 9669, 9672, 9673, 9676
482, 482, 2918, 2941, 2989, 3476,	<code>\stex_persist:n</code>
3542, 3561, 3640, 3666, 3764, 3791,	184, <u>586</u> , 614, 615, 726, 727, 729,
3839, 4302, 4455, 6282, 6320, 7947	732, 735, 736, 1035, 1078, 1085, 2445
<code>\stex_new_stylable_env:nnnnnnn</code> ..	<code>\c_stex_persist_force_bool</code> .. 35, 581
.. 127, <u>516</u> , 516, 2326, 3429, 3438,	<code>\c_stex_persist_mode_bool</code>
3494, 3504, 7018, 7053, 7087, 7279,	 33, 571, 582, 600
7775, 7797, 7861, 9052, 9151, 9258,	<code>_stex_persist_read_now:</code>
9320, 9368, 9420, 9516, 9681, 10278	 599, 1084, 8243
<code>\stex_new_symbol_uri:n</code>	<code>\c_stex_persist_write_mode_bool</code> .
136, <u>1421</u> , 1421, 3165, 3850, 3913, 7186	... 34, 577, 601, 607, 724, 1083, 2435
<code>\stex_new_symbol_uri:nn</code>	<code>\stex_pseudogroup:nn</code> 125, 126, 250,
..... 136, <u>1424</u> , 1424, 3081,	<u>303</u> , 303, 700, 899, 2560, 5366, 6992
4089, 4225, 4286, 5151, 7156, 7377	<code>\stex_pseudogroup_restore:N</code>
<code>_stex_next_symbol:n</code>	 126, 258,
149, <u>4583</u> , 4585, 4603, 4977, 5107, 6121	259, <u>306</u> , 306, 912, 2570, 6997, 6998
<code>\c_stex_no_archive</code>	<code>\stex_pseudogroup_with:nn</code>
133, <u>1051</u> , 1177, 1377, 2835	 126, <u>313</u> , 313, 4108, 4126,
<code>\c_stex_no_archive_str</code>	4183, 5966, 6629, 6886, 6902, 6927
..... 133, <u>1051</u> , 1177, 1377, 2835	<code>\c_stex_pwd_file</code>
<code>\c_stex_no_archive_uri</code> ... 1053, 1059	 124, 228, 870, 1062, 1063, 1799
<code>\c_stex_no_frontmatter_bool</code> 39, 1743	<code>\stex_reactivate_macro:N</code>
<code>_stex_notation_add:</code>	 123, <u>67</u> , 73, 2599, 2940,
... 153, 3799, 3875, 6287, <u>6487</u> , 6487	2947, 2986, 2995, 3302, 3303, 3304,
<code>\l_stex_notation_args_tl</code>	3466, 3472, 3491, 3532, 3538, 3558,
..... 6349, 6382,	3777, 3808, 3897, 6307, 7028, 7084,
6451, 6457, 6463, 6568, 6570, 6667	

7135, 7403, 7404, 7405, 7406, 7407,
 7408, 7422, 7423, 7424, 7425, 7426,
 7427, 7428, 7684, 7685, 7686, 7687,
 7688, 7689, 7690, 7691, 7692, 7693,
 7694, 8281, 9036, 9037, 9038, 9039,
 9040, 9041, 9042, 9430, 9550, 9715
`\stex_ref_new_doc_target:n`
 657, 1811, 1811, 1829
`\_stex_ref_new_id:n` 7393
`\stex_ref_new_sym_target:n`
 1830, 1830, 6160, 7400, 7419
`\stex_ref_new_symbol:n`
 2025, 2488, 3863, 3932
`\l_stex_relative_import_bool` ...
 1312, 1314, 1352, 2978
`\_stex_renamedec1_do:nn`
 3638, 3642, 3648
`\stex_require_module:N` . 145, 2582,
2807, 2807, 2920, 2952, 2991, 3041
`\stex_require_module_noerr:N` ...
 145, 2807, 2816, 2827
`\stex_require_module_noerr:n` ...
 145, 2807, 2825, 2980
`\_stex_return_args:nn`
 3941, 4039, 4396, 4555
`\l_stex_return_notation_tl`
 4643, 4813, 4821, 4985,
 4986, 5074, 5100, 5101, 5511, 5517
`\stex_set_language:n` 2305
`\_stex_set_meta_archive:` .. 928, 8127
`\stex_sms_allow:N` 144, 2621,
 2623, 2636, 2637, 2638, 2639, 2640
`\stex_sms_allow_env:n`
144, 2631, 2633, 2641, 3468, 3474,
 3534, 3540, 7026, 7081, 7131, 7307
`\stex_sms_allow_escape:N`
144, 2626, 2628, 2642, 2643, 2948,
 3493, 3560, 3570, 3645, 3714, 3778,
 3809, 3898, 6308, 7233, 7327, 7463
`\stex_sms_allow_import:Nn`
 144, 2644, 2647, 2985
`\stex_sms_allow_import_*` 144
`\stex_sms_allow_import_env:nn` ...
 144, 2651, 2653, 2658
`\g_stex_sms_import_code`
 144, 2659, 2671, 2688, 2979
`\stex_smsmode_do:` 144, 2340, 2584,
 2595, 2713, 2715, 2718, 2718, 2943,
 3460, 3486, 3527, 3553, 3568, 3643,
 3773, 3804, 3893, 6303, 7020, 7075,
 7119, 7231, 7300, 7324, 7325, 7446
`\stex_source_path:n`
 133, 1098, 1098, 1107, 1882,
 1938, 2019, 2052, 2062, 2081, 2082,
 2090, 2093, 2218, 2234, 2251, 2841
`\stex_source_path:nn`
 ... 133, 1098, 1105, 2213, 2229, 2244
`\_stex_sref_do_aux:n` 8124
`\stex_str_if_ends_with:nn` .. 123, 62
`\stex_str_if_ends_with:nnTF`
 123, 62,
 588, 591, 3675, 3698, 4242, 4249, 4261
`\stex_str_if_ends_with_p:nn`
 123, 62, 4214, 4283
`\stex_str_if_starts_with:nn` 123, 57
`\stex_str_if_starts_with:nnTF` ...
 . 123, 57, 413, 1029, 1910, 4791, 4795
`\stex_str_if_starts_with_p:nn` ...
 123, 57
`\l_stex_struct_this_tl`
 4777, 4979, 4980, 4983, 5061, 5064,
 5073, 5094, 5095, 5098, 5112, 5931
`\stex_structural_feature_-`
`module:nn` ... 143, 2600, 2600, 7045
`\stex_structural_feature_module_-`
`end:` 143, 2600, 2613, 7022, 7077, 7124
`\stex_structural_feature_-`
`morphism:nnnnn` 145,
3019, 3026, 3430, 3478, 3495, 3544
`\stex_structural_feature_-`
`morphism_check_total:`
 3272, 3497, 3513, 3551
`\stex_structural_feature_-`
`morphism_end:` . 145, 3019, 3107,
 3435, 3450, 3485, 3501, 3517, 3552
`\stex_structural_feature_-`
`morphism_with_macros:nnnnn` ..
145, 3019, 3022, 3442, 3477, 3508, 3543
`\stex_style_apply:`
 127, 3, 482, 487, 524,
 530, 740, 2338, 2343, 2935, 2972,
 3008, 3433, 3439, 3444, 3448, 3458,
 3483, 3499, 3505, 3510, 3515, 3525,
 3549, 3788, 3825, 3890, 4334, 4491,
 6300, 6339, 7297, 7303, 7682, 7784,
 7801, 7879, 7900, 7903, 7926, 7960,
 9098, 9105, 9194, 9196, 9264, 9276,
 9326, 9336, 9374, 9384, 9417, 9436,
 9551, 9555, 9716, 9720, 10298, 10301
`\stex_suppress_html:n`
 128, 699, 699, 2701, 6372
`\_stex_symbol_macro:N`
 ... 2264, 2368, 2428, 2440, 2452,
 2486, 2487, 4062, 4071, 4072, 7121
`\_stex_symbol_macro:n`
 ... 2267, 4098, 4118, 4139, 4243,
 4256, 4262, 7148, 7154, 7479, 7481

<code>\stex_symbol_uri:n</code>	136	<code>\stex_uri_resolve:Nn</code>	2302
<code>\stex_symbol_uri_archive:N</code>		<code>\stex_uri_use:N</code>	7337
.....	136, 1427, 1427	<code>\stex_use_archive_uri:n</code>	
<code>\stex_symbol_uri_module:N</code>			134, 1117, 1117
.....	137, 1433, 1433	<code>\stex_use_document_uri:N</code>	
<code>\stex_symbol_uri_module:n</code>			134, 1120, 1120, 1237, 1315,
137, 1433, 1436, 4098, 7471, 7479, 7481		1452, 1814, 1815, 1817, 1822, 1876,	
<code>\stex_symbol_uri_name:N</code>		1881, 2057, 2081, 2412, 2679, 8817	
137, 1443, 1443, 3661, 4610, 6020,		<code>\stex_use_document_uri:n</code>	
6037, 6129, 6152, 6312, 8956, 8979			134, 1120, 1123
<code>\stex_symbol_uri_name:n</code>		<code>\stex_use_module_uri:N</code>	135,
137, 1443, 1446, 3156, 4100, 7480, 7481		1238, 1238, 1353, 1366, 1381, 1390,	
<code>\stex_symbol_uri_path:N</code>		2319, 2336, 2355, 2362, 2395, 2420,	
.....	136, 1430, 1430	2422, 2437, 2484, 2536, 2578, 2810,	
<code>\stex_symdecl_do:</code>		2915, 2928, 2932, 2964, 2969, 3001,	
.....	147, 257, 333, 3852,	3005, 3051, 3059, 3457, 3482, 3524,	
3916, 3935, 3935, 4309, 4464, 7369		3548, 4055, 4084, 6506, 7221, 7228	
<code>_stex_symdecl_html:</code>		<code>\stex_use_module_uri:n</code>	135,
... 147, 3866, 3919, 4007, 4007, 7376		1238, 1241, 2558, 2563, 2572, 3014,	
<code>_stex_term_arg:nnn</code>		4885, 7067, 7109, 7139, 7143, 7245	
... 151, 5483, 5659, 5662, 5668, 5684		<code>\stex_use_notation:nn</code>	154
<code>_stex_term_arg:nnnnn</code>		<code>\stex_use_notation:nnTF</code>	
.....	151, 5651, 5651, 5703,	154, 4689, 4912, 5310, 5804, 6652, 6652	
5762, 5780, 5870, 6735, 6858, 6866		<code>\stex_use_op_notation:nnTF</code>	
<code>_stex_term_arg_aB:nnnnn</code>			154, 4676, 5336, 6652, 6658
... 151, 5686, 5700, 6871, 6879, 6896		<code>\c_stex_use_sref_bool</code>	27
<code>_stex_term_do_aB_clist:</code> ...	295,	<code>\stex_use_symbol_uri:N</code>	
5694, 5710, 5764, 5793, 5822, 5880,		136, 1407, 1407, 2030, 3230, 3236,	
6886, 6887, 6903, 6907, 6928, 6933		3574, 3577, 3649, 3652, 3782, 3795,	
<code>_stex_term_oma:nn</code>		3801, 3813, 3864, 3878, 3881, 3887,	
... 150, 5376, 5600, 5601, 5616, 6810		3933, 4608, 6018, 6035, 6128, 6150,	
<code>_stex_term_oma_or_omb:nn</code>		6290, 6313, 6600, 6603, 6611, 6612,	
.....	6810, 6815, 6863, 6876	6613, 6614, 6623, 7378, 7385, 7457,	
<code>_stex_term_omb:nn</code>	150, 5404,	7506, 7956, 8014, 8039, 8960, 8983	
5405, 5618, 5619, 5634, 6863, 6876		<code>\stex_use_symbol_uri:n</code>	
<code>_stex_term_oms:nn</code>			136, 1407, 1410, 2519, 3285,
....	150, 4636, 4638, 5550, 5551,	4615, 5222, 5223, 5225, 5226, 5233,	
5562, 5565, 6001, 6024, 6041, 6709		5234, 5236, 5237, 6509, 6526, 6529,	
<code>_stex_term_oms_or_omv:nn</code>		6543, 6544, 6545, 6548, 6549, 6550,	
.....	4636, 4638, 4666,	7185, 7346, 7400, 7419, 7466, 7478	
4679, 4831, 4869, 4908, 4964, 5253,		<code>_stex_var_notation_macro:</code>	
5255, 5338, 5349, 5359, 5565, 6799		... 153, 4320, 4479, 6326, 6579, 6579	
<code>_stex_term_omv:nn</code>		<code>_stex_variable:nnnnnnN</code> .	4300, 4445
.....	150, 5253, 5255, 5273,	<code>\l_stex_variables_prop</code>	
5566, 5567, 5578, 6053, 6067, 6081			148, 4298, 4339, 4428, 4497
<code>_stex_titlefragment:</code>	8416	<code>\stex_with_file_hooks:Nnn</code>	
<code>\stex_undefine:N</code>			125, 246, 249, 249, 2680
. 123, 53, 53, 56, 310, 320, 2080, 2677		stex internal commands:	
<code>\stex_uri_from_archive_file:Nn</code> .	135	<code>\l__stex_annotate_do_output_bool</code>	
<code>\stex_uri_from_pair:Nn</code>		... 685, 690, 693, 696, 701, 705, 8126	
.....	136, 1392, 1392, 2298	<code>__stex_annotate_env_str</code> ...	667, 668
<code>\stex_uri_from_pair:Nnn</code>		<code>__stex_check_env_str</code>	6201, 6202, 6203
136, 1312, 1313, 1330, 1401, 1405,		<code>__stex_comps_do_defref:nn</code>	
2577, 2919, 2951, 2977, 2990, 3039			6096, 6102, 6112, 6145

_stex_comps_do_ref:nNn .. 6001,	_stex_expr_arg_inner:nn
6022, 6039, 6049, 6061, 6075, 6124	 5411, 5414, 5418, 5424
_stex_comps_slash:w	\l_stex_expr_assigned_seq
..... 6132, 6156, 6168, 6172	 4973, 5092, 5134, 5201
_stex_comps_split_slash:n	_stex_expr_assoc_make_seq:nnn .
..... 6004, 6020, 6037	 5776, 5803, 5889
_stex_comps_split_slash_i:nw ..	_stex_expr_assoc_seq:nnnnnnn ..
..... 6005, 6008, 6010	 5720, 5771
_stex_comps_uppercase:n	_stex_expr_check:n
..... 6029, 6040, 6076, 6113	 5452
_stex_debug:nn	_stex_expr_check:nTF ... 5421, 5427
\l_stex_debug_cl	_stex_expr_check_b:nn .. 5377, 5402
..... 121, 123, 126	_stex_expr_check_comp:
_stex_debug_env_str . 119, 120, 123	... 5553, 5562, 5569, 5578, 5603,
_stex_doc_check_topsect:	5616, 5621, 5634, 5662, 5676, 5684
..... 1698, 1704, 1710, 1716	\l_stex_expr_clist .. 4697, 4704,
_stex_doc_do_section:n	4711, 4715, 4947, 4968, 4974, 4976
..... 1555, 1561, 1565, 1569, 1622	\l_stex_expr_comp_cs
\g_stex_doc_ftml_link_text_tl ..	 5000, 5005, 5024, 5026
..... 1457, 1460, 1472	\l_stex_expr_count_int
_stex_doc_maketitle: ... 1518, 1523	.. 5699, 5706, 5715, 5762, 5780, 5870
_stex_doc_orig_backmatter	\l_stex_expr_cs
..... 1759, 1764, 1782	.. 4949, 4955, 4962, 5805, 5808, 5828
_stex_doc_orig_endbackmatter 1760	_stex_expr_current_type:
_stex_doc_orig_endfrontmatter .	 4903, 5039, 5115
..... 1747, 1756	\l_stex_expr_current_type_tl ...
_stex_doc_orig_frontmatter ...	 4828, 4865, 4906, 4911
..... 1746, 1751, 1771	_stex_expr_custom:n 5290, 5300, 5364
_stex_doc_set_title:n .. 1495, 1512	\l_stex_expr_customs_prop
_stex_doc_skip_section:	 5368, 5380, 5381,
..... 1643, 1649, 1653	5382, 5397, 5432, 5438, 5453, 5456
_stex_doc_skip_section_i:	\l_stex_expr_customs_seq
..... 1628, 1631, 1634, 1644, 1649	.. 5369, 5386, 5387, 5388, 5398, 5454
_stex_doc_title_html:	_stex_expr_do_aB_clist:
..... 1500, 1504, 1515	 5686, 5694, 5764, 5822
\g_stex_doc_title_tl	_stex_expr_do_ab_next:nn
..... 1477, 1492, 1499, 1506, 1511	 5376, 5378, 5404, 5405
\l_stex_doc_titlefragment_bool .	_stex_expr_do_all:w ... 4746, 4755
..... 1606, 1611, 1621, 1626	_stex_expr_do_assign:nn 4849, 4853
_stex_doc_x_matter: ... 1744, 1793	_stex_expr_do_assign_list:n ...
_stex_expr_aB_arg:nnnnn 5708, 5714	 4824, 4846
_stex_expr_aB_simple_arg:nnnnn	_stex_expr_do_decl:nnnnnnnnn ..
..... 5729, 5760	 5149, 5177
_stex_expr_add_prop_arg:nnw ...	_stex_expr_do_decl_nomacro:nnnnnnnnn
..... 5372, 5396, 5399	 5147, 5163
\l_stex_expr_after_tl	_stex_expr_do_first_arg:n
.. 5153, 5157, 5158, 5159, 5230, 5241	 4733, 4740
_stex_expr_annotate:nnn	_stex_expr_do_first_next:
..... 5555, 5571,	 4735, 4742
5587, 5604, 5622, 5638, 5648, 5902	_stex_expr_do_headterm:nn
_stex_expr_arg:n	 5557, 5573, 5581, 5598, 5879
\l_stex_expr_arg_counter_int ...	_stex_expr_do_one:w ... 4744, 4750
..... 5362, 5370, 5385, 5420, 5421	_stex_expr_do_seqmap:nnnnnn ..
_stex_expr_arg_do:nnn	 5726, 5835
..... 5422, 5428, 5444, 5479, 5486	

__stex_expr_end:	__stex_expr_op_custom:n
.. 4792, 4796, 4812, 4817, 5722, 5735	 5283, 5300, 5356
\l__stex_expr_field_name_str	__stex_expr_op_notation:w
..... 5079, 5083, 5084, 5118	 5285, 5286, 5304, 5334
\l__stex_expr_fields_clist 4825,	__stex_expr_present: .. 4921, 5052
4847, 4854, 4872, 4875, 5192, 5193	__stex_expr_present:nn
\l__stex_expr_first_args_tl	 4925, 4931, 4936, 4943, 4946
..... 4732, 4752, 4756	__stex_expr_present_entry:nn ...
__stex_expr_full_notation:n ...	 4951, 4957, 4972
..... 5306, 5309	__stex_expr_present_i:w
__stex_expr_get_field_name:n ...	 4917, 4923, 4929
..... 5078, 5090	__stex_expr_present_ii:nw 4934, 4942
__stex_expr_get_index_notation:n	\l__stex_expr_prop
..... 4683, 4688, 4751	5081, 5089, 5124, 5132, 5154, 5164,
__stex_expr_gobble:nnnnnnnn ...	5166, 5178, 5180, 5200, 5203, 5209
..... 5722, 5735	__stex_expr_prop_do_decls:
\l__stex_expr_iarg_tl 5815, 5817, 5828	 5136, 5139
__stex_expr_invoke:nnnnN 4612, 4633	__stex_expr_prop_do_notations: ..
__stex_expr_invoke_field:n	 5156, 5218
..... 5054, 5088	__stex_expr_range:w 4684, 4695
__stex_expr_invoke_maybe_-	\l__stex_expr_redo_tl
field:nn	.. 5108, 5135, 5158, 5212, 5229, 5240
5043, 5047	\l__stex_expr_reset_tl
__stex_expr_invoke_this:n 4785, 5030	 4583, 4587, 4591, 4592, 4598
__stex_expr_is_argsep:n	__stex_expr_ret_cs:
5754	 5491, 5496, 5527, 5532, 5537
__stex_expr_is_seqmap:n	__stex_expr_return: 5506, 5509
5748	__stex_expr_return_arg:n 5493, 5499
__stex_expr_is_seqmap:nTF ... 5724	\l__stex_expr_return_args_tl ...
__stex_expr_is_varseq:n	.. 5489, 5500, 5505, 5514, 5534, 5539
5738	__stex_expr_return_next: 5494, 5503
__stex_expr_is_varseq:nTF 5717, 5840	\l__stex_expr_return_this_tl ...
__stex_expr_make_mod:n .. 4871, 4883	 5488, 5505,
__stex_expr_make_oml:n .. 4875, 4890	5510, 5514, 5520, 5523, 5533, 5541
__stex_expr_make_oml:nn . 4891, 4893	\l__stex_expr_seq
__stex_expr_make_prop:	 5133, 5165, 5179, 5220
..... 4915, 5048, 5131	__stex_expr_seq_arg:n ... 4698, 4709
__stex_expr_make_prop_assign: ..	__stex_expr_seq_first: .. 4672, 4728
..... 4916, 5051, 5191	__stex_expr_seq_op:w ... 4671, 4675
__stex_expr_make_prop_assign:nn	\l__stex_expr_set_comp_tl
..... 5194, 5199	 4759, 4814, 4818, 4978, 5103
__stex_expr_make_prop_assign_-	__stex_expr_set_custom_comp:n ..
replace:nnnn	 4819, 4994, 5012
5202, 5208	__stex_expr_set_custom_i:nn ...
__stex_expr_make_type:n	 5015, 5020
..... 4834, 4839, 4841, 4857	__stex_expr_set_customcomp: ...
__stex_expr_math:	 4792, 4817
4662, 5279	__stex_expr_set_this:n .. 5049, 5058
__stex_expr_maybe_notation:w ...	__stex_expr_set_thiscomp: 4759, 4763
..... 4780, 4782, 4910	__stex_expr_set_thisnotation: ..
__stex_expr_maybe_return:n	 4796, 4812
..... 5317, 5328, 5487	__stex_expr_setup:nnnnn
__stex_expr_merge:nw ... 4774, 4790	 4635, 4642, 5252
\l__stex_expr_more_nextsymbol_tl	
..... 5091, 5093, 5121	
__stex_expr_notation:w	
..... 4756, 5292, 5293, 5302, 5345	
\l__stex_expr_old_seq	
..... 5847, 5850, 5854, 5859	

```

\__stex_expr_struct_top:n . 4760,
    4769, 4793, 4797, 4800, 4803, 4805
\__stex_expr_struct_type:n 4776, 4823
\__stex_expr_text: . . . . . 4662, 5298
\__stex_expr_varseq_in_map:nnnnnnnn
    . . . . . 5842, 5888
\__stex_groups_do: . . . . 340, 346, 354
\__stex_groups_do_in:n . . . . 329, 333
\l__stex_groups_lvl_int 323, 325, 328
\l__stex_import_archive_str . . . .
    . . . . . 2832, 2835, 2839, 2907
\__stex_import_check_file:nnn . . . .
    . . . . . 2851,
    2852, 2853, 2867, 2868, 2869, 2896
\l__stex_import_doc_uri . . 2906, 2913
\l__stex_import_file_str . . . . .
    . . . . . 2819, 2849,
    2879, 2881, 2884, 2889, 2900, 2913
\__stex_import_import_module:nn .
    . . . . . 2942, 2950, 2987
\__stex_import_import_module_i:n
    . . . . . 2953, 2956
\__stex_import_import_module_-
    presms:nn . . . . . 2976, 2987
\l__stex_import_language_str . . . .
    . . . . . 2850, 2854, 2870, 2910
\__stex_import_load_check:n . . . .
    . . . . . 2837, 2843, 2848
\__stex_import_load_check_i:n . . . .
    . . . . . 2856, 2863
\__stex_import_load_check_ii: . . . .
    . . . . . 2860, 2878
\__stex_import_load_file: . . . . .
    . . . . . 2885, 2890, 2905
\__stex_import_load_module:NTF . . . .
    . . . . . 2809, 2818, 2830
\l__stex_import_name_str . . . . .
    . . 2834, 2851, 2852, 2853, 2865, 2909
\l__stex_import_path_str . . . . .
    . . 2833, 2836, 2841, 2864, 2866, 2908
\l__stex_import_pre_str . . . . .
    . . . . . 2836, 2840, 2897, 2898, 2900
\__stex_import_requiremodule: . . . .
    . . . . . 2992, 2997
\l__stex_import_uri . . . . .
    . . . . . 2826, 2827, 2831, 2914, 2915
\__stex_import_usemodule: 2921, 2924
\l__stex_inputs_gin_repo_str . . . .
    . . . . . 2246, 2249, 2254
\l__stex_inputs_id_seq . . . . .
    . . . . . 2183, 2184, 2189, 2197
\l__stex_inputs_libinput_files_-
    seq . . . . . 2112, 2113, 2116,
    2122, 2123, 2144, 2145, 2169, 2170,
    2173, 2181, 2193, 2194, 2204, 2205
\l__stex_inputs_path_seq . . . . .
    . . 2182, 2185, 2187, 2190, 2198, 2201
\l__stex_inputs_path_str . . . . .
    2190, 2191, 2192, 2193, 2194, 2197,
    2198, 2201, 2202, 2203, 2204, 2205
\__stex_inputs_ref:n 2044, 2065, 2075
\__stex_inputs_ref_i:n . . . . .
    . . . . . 2051, 2067, 2070
\c__stex_inputs_ref_post_str . . . .
    . . . . . 2050, 2058
\c__stex_inputs_ref_pre_str . . . .
    . . . . . 2049, 2056
\l__stex_inputs_tmp_str . . 2123, 2125
\__stex_inputs_up_archive:nn . . . .
    . . 2111, 2121, 2143, 2168, 2177, 2177
\l__stex_inputs_uri . . 2043, 2057,
    2062, 2081, 2082, 2088, 2090, 2093
\__stex_inputs_usetikzlibrary:n .
    . . . . . 2136, 2138, 2142
\__stex_inputs_usetikzlibrary_-
    i:nn . . . . . 2145, 2151
\__stex_keys_split_at_bracket:w .
    . . . . . 414, 422
\l__stex_keys_tl . . . . .
    . . . . . 364, 365, 366, 368, 371, 372
\l__stex_lang_turkish_bool . . . . .
    . . . . . 455, 463, 473
\__stex_mathhub: . . . . . 1006, 1030
\l__stex_mathhub_base_str . . . . .
    . . 1014, 1024, 1029, 1030, 1032, 1036
\l__stex_mathhub_bool . . . . .
    . . . . 972, 973, 975, 978, 987, 989, 998
\__stex_mathhub_check_manifest: .
    . . . . . 977, 985
\__stex_mathhub_check_manifest:n
    . . . . . 986, 988, 990, 995
\l__stex_mathhub_cs . . . . .
    . . . . 900, 903, 905, 909, 913, 914, 919
\__stex_mathhub_do_manifest:nn . .
    . . . . . 932, 950, 957, 1041
\l__stex_mathhub_file_str 1012, 1032
\__stex_mathhub_find_manifest:n 962
\__stex_mathhub_find_manifest:nn
    . . . . . 959, 968, 1065
\l__stex_mathhub_id_str . . . . .
    . . . . . 1013, 1023, 1032, 1036
\l__stex_mathhub_key 1019, 1020, 1022
\c__stex_mathhub_manifest_ior . . . .
    . . . . . 1002, 1011, 1016, 1028
\l__stex_mathhub_manifest_str . . . .
    . . . . . 960, 963, 969, 999, 1011, 1066

```

<code>__stex_mathhub_parse_manifest:n</code>	2372, 2394, 2395, 2396, 2398, 2403,
.....	964, 1010, 1069
<code>__stex_mathhub_replace_https:w</code>	2416, 2421, 2422, 2423, 2426, 2427,
.....	2428, 2429, 2431, 2432, 2483, 2484,
	2485, 2486, 2487, 2491, 2492, 2518
<code>__stex_mathhub_require:n</code>	921, 944
<code>__stex_mathhub_restore:nn</code>	
.....	1036, 1040, 1079
<code>\l__stex_mathhub_seq</code>	971, 974, 979,
	996, 997, 999, 1012, 1018, 1019, 1021
<code>__stex_mathhub_set_current:n</code>	
.....	908, 920
<code>\l__stex_mathhub_str</code>	
.....	838, 839, 843, 844,
	849, 855, 858, 865, 867, 868, 869, 874
<code>\l__stex_mathhub_tl</code>	
	979
<code>\l__stex_mathhub_val</code>	1021, 1023, 1024
<code>__stex_modules_export:n</code>	2588, 2590
<code>__stex_modules_html_annots:</code>	
.....	2313, 2334
<code>__stex_modules_load_meta:</code>	
.....	2356, 2374, 2376
<code>__stex_modules_load_meta_i:n</code>	
.....	2378, 2382
<code>__stex_modules_load_sig:</code>	2401, 2406
<code>__stex_modules_macro_short:nnn</code>	
.....	2261, 2265, 2268,
	2271, 2274, 2277, 2280, 2283, 2286
<code>__stex_modules_nested:n</code>	
.....	2347, 2418, 2418
<code>__stex_modules_persist_module:</code>	
.....	2435, 2444, 8122
<code>__stex_modules_persist_noti-</code>	
<code>i:nn</code>	
.....	2460, 2465
<code>__stex_modules_restore_module:nnnn</code>	
.....	2446, 2482
<code>__stex_modules_restore_noti:n</code>	
.....	2496, 2501
<code>__stex_modules_restore_noti:n</code>	
.....	2502, 2505, 2524
<code>__stex_modules_restore_noti-</code>	
<code>ii:nnnnn</code>	
.....	2509, 2513
<code>\l__stex_modules_sig</code>	2408, 2412, 2413
<code>\l__stex_modules_sigfile</code>	
.....	2407, 2412, 2414
<code>__stex_modules_stringify-</code>	
<code>uri:nnn</code>	
.....	2470, 2483
<code>__stex_modules_stringify-</code>	
<code>uri:nnnn</code>	
.....	2475, 2520
<code>\l__stex_modules_tl</code>	2514, 2516, 2521
<code>__stex_modules_top:n</code>	
.....	2347, 2350
<code>__stex_modules_top_sig:n</code>	
.....	2352, 2391, 2391
<code>\l__stex_modules_uri</code>	2361, 2362,
	2363, 2366, 2367, 2368, 2369, 2371,
	2372, 2394, 2395, 2396, 2398, 2403,
	2416, 2421, 2422, 2423, 2426, 2427,
	2428, 2429, 2431, 2432, 2483, 2484,
	2485, 2486, 2487, 2491, 2492, 2518
<code>__stex_morphisms_add_definiens:nn</code>	
.....	3242, 3247, 3306
<code>__stex_morphisms_add_symbol:nnnnnnnnN</code>	
.....	3159, 3175, 3177, 3182
<code>__stex_morphisms_add_symbol_-</code>	
<code>i:nnnnnnnN</code>	
.....	3189, 3192
<code>\l__stex_morphisms_ass_tl</code>	
..	3592, 3608, 3612, 3623, 3624, 3625
<code>__stex_morphisms_begin_copy:Nnnn</code>	
.....	3430, 3442, 3453
<code>__stex_morphisms_begin_interpret:Nnnn</code>	
.....	3495, 3508, 3520
<code>__stex_morphisms_break:</code>	3250, 3254
<code>__stex_morphisms_check_name:nnnn</code>	
.....	3370, 3373
<code>\l__stex_morphisms_continue_bool</code>	
.....	3311, 3314, 3327, 3346
<code>__stex_morphisms_cs:nnnn</code>	3312, 3337
<code>__stex_morphisms_definiens_-</code>	
<code>impl:nn</code>	
.....	3222, 3307
<code>__stex_morphisms_do:n</code>	
.....	3075, 3088, 3103
<code>__stex_morphisms_do_decls:</code>	
.....	3074, 3078
<code>__stex_morphisms_do_elaboration:</code>	
.....	3117, 3120
<code>__stex_morphisms_do_for_list:</code>	
.....	3212, 3305
<code>__stex_morphisms_do_morph:nnnn</code>	
.....	3094, 3099
<code>__stex_morphisms_do_morph_-</code>	
<code>assign:nnn</code>	
.....	3673, 3676, 3717
<code>__stex_morphisms_do_parsed_-</code>	
<code>assign:</code>	
.....	3614, 3617
<code>__stex_morphisms_do_parsed_-</code>	
<code>newname:</code>	
.....	3621, 3629
<code>__stex_morphisms_do_parsed_-</code>	
<code>newname:w</code>	
.....	3631, 3633, 3637
<code>\l__stex_morphisms_dones_seq</code>	
.....	3089, 3091, 3092
<code>__stex_morphisms_elab:nn</code>	3132, 3150
<code>__stex_morphisms_elab_check_-</code>	
<code>assign:nnnnn</code>	
.....	3185, 3196
<code>__stex_morphisms_elab_check_-</code>	
<code>rename:nnn</code>	
.....	3153, 3204
<code>__stex_morphisms_elab_i:nnn</code>	
.....	3156, 3171
<code>__stex_morphisms_end:</code>	3621, 3637
<code>\l__stex_morphisms_feature_str</code>	
.....	3053, 3138

<code>__stex_morphisms_get_check:nn</code> ..	<code>__stex_morphisms_total_check:nn</code>
..... 3354, 3369, 3415, 3423	 3274, 3278
<code>\l__stex_morphisms_get_str</code>	<code>__stex_morphisms_total_check:nnnnnnnN</code>
..... 3352, 3375,	 3279, 3282
3396, 3406, 3411, 3413, 3419, 3421	<code>\l__stex_morphisms_with_macros_-</code>
<code>__stex_morphisms_gobble:nnnnnnN</code>	bool ... 3020, 3023, 3027, 3113, 3174
..... 3399, 3403	<code>__stex_notations_activate_-</code>
<code>__stex_morphisms_iterate:nn</code> ...	not:nn 6512, 6524
..... 3317, 3321, 3330, 3332	<code>__stex_notations_add:nnnnn</code>
<code>__stex_morphisms_macro:nnnnnnnN</code>	 6843, 6848, 6853
..... 3379, 3395	<code>__stex_notations_add_last:nnnnn</code>
<code>\l__stex_morphisms_mods_seq</code>	 6836, 6847
..... 3310, 3334, 3335	<code>__stex_notations_add_missing_-</code>
<code>__stex_morphisms_morphism:nnnnn</code>	args:nn 6478, 6481
..... 3024, 3028, 3031	<code>__stex_notations_add_next:nnnnnnn</code>
<code>\l__stex_morphisms_morphism_dom_-</code>	 6838, 6842
str ... 3667, 3680, 3692, 3702, 3719	<code>\l__stex_notations_after_tl</code>
<code>\l__stex_morphisms_name_str</code>	 6823, 6850
..... 3590, 3599, 3600, 3601,	<code>__stex_notations_args_end:</code>
3605, 3606, 3607, 3618, 3620, 3624	 6557, 6560, 6563,
<code>\l__stex_morphisms_newname_str</code> ..	6570, 6573, 6576, 6831, 6834, 6844
.. 3591, 3610, 3611, 3619, 3620, 3621	<code>\l__stex_notations_args_tl</code>
<code>\l__stex_morphisms_next_tl</code>	 6742, 6744, 6757
..... 3597, 3602, 3604	<code>__stex_notations_augment_arg:nn</code>
<code>__stex_morphisms_parse_assign:n</code>	 6758, 6771
..... 3479, 3545, 3589	<code>\l__stex_notations_bind_bool</code> ...
<code>__stex_morphisms_reactivate:</code> ...	 6277, 6279
..... 3065, 3294	<code>__stex_notations_check_aB_-</code>
<code>__stex_morphisms_rename:n</code> 3139, 3142	arg:Nn 6885, 6894, 6901, 6926
<code>__stex_morphisms_rename:nn</code>	<code>\l__stex_notations_clist_count_-</code>
..... 3143, 3146	int 6924, 6932, 6938
<code>__stex_morphisms_rename_all:</code> . 3267	<code>\l__stex_notations_code_tl</code>
<code>__stex_morphisms_renamed_-</code>	 6781, 6796, 6811, 6825,
check:nn 3358, 3405	6826, 6849, 6857, 6865, 6870, 6878
<code>__stex_morphisms_renamed_-</code>	<code>__stex_notations_complex:nnnnnnn</code>
check:nnnnnn 3407, 3410	 6790, 6807
<code>\l__stex_morphisms_seq</code>	<code>__stex_notations_const_precs:</code> ..
..... 3594, 3595, 3597, 3599,	 6351, 6391
3602, 3604, 3606, 3608, 3610, 3612	<code>\l__stex_notations_cs</code>
<code>__stex_morphisms_set:nnnnnnN</code> ...	 6797, 6802, 6812, 6821
..... 3377, 3383, 3398	<code>__stex_notations_default_args:</code> .
<code>__stex_morphisms_set_definiens_-</code>	 6743, 6754
macros: 3250, 3254	<code>__stex_notations_do_argname:nn</code> .
<code>__stex_morphisms_set_definiens_-</code>	 6452, 6455
macros_i:nnnnnnn 3257, 3262	<code>__stex_notations_do_argnames:</code> ..
<code>__stex_morphisms_setup:</code> . 3064, 3069	 6427, 6450
<code>__stex_morphisms_split_qm:w</code> . 3290	<code>__stex_notations_do_missing_-</code>
<code>\l__stex_morphisms_tmp</code> 3151,	args:n 6357, 6470
3155, 3158, 3159, 3165, 3172, 3207	<code>__stex_notations_fun_precs:</code> ...
<code>\l__stex_morphisms_tmp_b</code>	 6356, 6403
..... 3183, 3187, 3199	<code>__stex_notations_it_not_-</code>
<code>\l__stex_morphisms_todo_tl</code>	check:nnnn 6643, 6647
..... 3316, 3320, 3333, 3346	<code>__stex_notations_it_not_i:n</code> ...
	 6632, 6636, 6649

<code>\l__stex_notations_left_bracket_-</code>	<code>\l__stex_notations_str</code>
<code>str</code> 6955, 6983, 6993, 6997	.. 6433, 6434, 6435, 6437, 6443, 6444
<code>__stex_notations_macro:nn</code>	<code>__stex_notations_styledefs:</code> ...
6495, 6543, 6544, 6545, 6580, 6581,	 6299, 6310
6582, 6600, 6611, 6612, 6653, 6654	<code>__stex_others_newlabel:n</code> 8244, 8252
<code>__stex_notations_make_arg:nnnn</code> .	<code>__stex_others_old_newlabel:</code> ...
..... 6721, 6724	 8246, 8251
<code>__stex_notations_make_arg_-</code>	<code>__stex_path:</code> 148, 158
<code>html:nn</code> 6701, 6717	<code>\l__stex_path_a_seq</code> ... 280, 286, 292
<code>__stex_notations_make_name:</code> ...	<code>\l__stex_path_a_tl</code> 292, 294
..... 6750, 6755, 6775	<code>\l__stex_path_b_seq</code> 281, 287, 293, 299
<code>__stex_notations_map_args_i:w</code> ..	<code>\l__stex_path_b_tl</code> 293, 294
..... 6556, 6560, 6563	<code>\l__stex_path_can_seq</code>
<code>__stex_notations_map_args_ii:w</code> .	 180, 183, 188, 196, 197, 200
..... 6569, 6573, 6576	<code>\l__stex_path_can_str</code>
<code>__stex_notations_map_cs:</code>	 179, 181, 185, 197
..... 6686, 6688,	<code>__stex_path_canonicalize:N</code>
6904, 6906, 6914, 6929, 6931, 6942	 161, 172, 177
<code>\l__stex_notations_missing_str</code> ..	<code>__stex_path_dodots:n</code> . 182, 186, 192
..... 6471, 6472, 6473, 6475, 6483	<code>\l__stex_path_return_tl</code>
<code>\l__stex_notations_missing_tl</code> ...	 282, 288, 295, 298, 300
..... 6386, 6477, 6484	<code>\l__stex_path_seq</code> 211, 212,
<code>\l__stex_notations_mods_seq</code>	213, 218, 219, 221, 223, 226, 251, 252
..... 6628, 6637, 6638	<code>\l__stex_path_str</code> 149, 150, 151, 154,
<code>\l__stex_notations_name_str</code>	156, 157, 158, 160, 163, 170, 171,
..... 6751, 6777	210, 211, 212, 217, 218, 219, 221,
<code>__stex_notations_not_cs:nnnnn</code> ..	222, 223, 229, 231, 233, 239, 241, 243
..... 6629, 6630, 6640	<code>\l__stex_path_win_drive</code>
<code>__stex_notations_op_macro:nn</code> ...	 150, 155, 162, 164
6498, 6548, 6549, 6550, 6586, 6587,	<code>__stex_path_win_take:w</code> 148, 158
6588, 6603, 6613, 6614, 6659, 6660	<code>__stex_persist_env_str</code>
<code>\l__stex_notations_opprec_tl</code> 6380,	 568, 569, 570, 574, 575, 576
6393, 6396, 6398, 6405, 6408, 6410,	<code>__stex_persist_load_file:n</code>
6414, 6421, 6434, 6440, 6446, 6677	 589, 592, 594, 604, 611, 645
<code>__stex_notations_parse_args:nnnnw</code>	<code>__stex_persist_read_and_write:</code> .
..... 6831, 6834, 6844	 602, 630
<code>__stex_notations_parse_precs:</code> ..	<code>\c__stex_persist_sms_iow</code>
..... 6425, 6430	 585, 626, 627, 642, 730
<code>\l__stex_notations_pre_tl</code>	<code>__stex_persist_write_only:</code>
..... 6810, 6825, 6862, 6875	 607, 625, 639, 646
<code>\l__stex_notations_precs_seq</code> ...	<code>__stex_proof_add_counter:</code> 7640, 7899
..... 6348, 6416,	<code>__stex_proof_begin_proof:nn</code> ...
6423, 6444, 6446, 6458, 6464, 6678	 7666, 7776, 7798
<code>__stex_notations_process:nnnnnn</code>	<code>\l__stex_proof_counter_intarray</code> .
..... 6780, 6786	 7597, 7603,
<code>\l__stex_notations_right_-</code>	7606, 7615, 7618, 7627, 7635, 7636,
<code>bracket_str</code> . 6956, 6988, 6994, 6998	7644, 7649, 7656, 7662, 7667, 7668
<code>\l__stex_notations_seq</code>	<code>\l__stex_proof_counter_str</code>
..... 6432, 6433, 6435, 6436, 6443	... 7555, 7556, 7566, 7719, 7734,
<code>__stex_notations_set_macro:nnnnn</code>	7834, 7852, 7853, 8007, 8022, 8047
..... 6511, 6520, 6533, 6542	<code>__stex_proof_do_after_subproof:N</code>
<code>__stex_notations_simple:nnnnn</code> ..	 7823, 7857
..... 6788, 6794	<code>__stex_proof_do_after_subproof_-</code>
	<code>i:nn</code> 7825, 7829

```

\__stex_proof_do_spf_name: 7546, 7979
\__stex_proof_do_spf_name_i: ...
..... 7560, 7827
\__stex_proof_do_spf_varname: ...
..... 7570, 7898, 8069, 8070
\__stex_proof_end_comment: .....
..... 7764, 7771,
7792, 7932, 7940, 7974, 8067, 8073
\__stex_proof_end_list: .....
.. 7782, 7817, 7923, 7927, 7992, 8080
\__stex_proof_html: .....
..... 7714, 7737, 7740, 8025, 8050
\__stex_proof_html_env_proof: ...
..... 7672, 7704
\__stex_proof_html_env_subproof:
..... 7717, 7865
\l__stex_proof_in_spfblock_bool .
..... 7664, 7759,
7765, 7777, 7799, 7849, 7877, 7889,
7916, 7923, 7927, 7988, 8060, 8076
\__stex_proof_inc_counter: .....
..... 7623, 7918, 8069, 8070
\l__stex_proof_inc_counter_bool 7596
\__stex_proof_insert_number: ...
..... 7599, 7898, 8069, 8070
\l__stex_proof_key_name_str ....
..... 7562, 7563, 7831
\l__stex_proof_key_numname_str ..
..... 7561, 7566, 7832
\l__stex_proof_key_showname_bool
..... 7833
\__stex_proof_make_step_macro:Nnnnn
..... 7972, 8069, 8070, 8071
\__stex_proof_number_as_string:N
..... 7555, 7610,
7719, 7852, 7881, 7894, 7983, 8007
\__stex_proof_proof_box_tl 7577, 8106
\__stex_proof_remove_counter: ...
..... 7652, 7917
\__stex_proof_set_after_subproof:n
..... 7836, 7878, 7905
\__stex_proof_set_after_subproof_-
i:n ..... 7838, 7839, 7846
\__stex_proof_set_after_subproof_-
with_number:n ..... 7841, 7891
\__stex_proof_set_after_subproof_-
with_number_i:n . 7843, 7844, 7851
\__stex_proof_set_aftergroup: ...
..... 7849, 7854, 7856
\__stex_proof_set_aftergroup_-
double: ..... 7849
\__stex_proof_start_comment: ...
..... 7758, 7771,
7779, 7914, 7936, 7996, 8062, 8083
\__stex_proof_start_list:n .....
..... 7778, 7808, 7898, 7992, 8080
\__stex_proof_step_html:nn .....
..... 7992, 8005, 8080
\__stex_proof_step_html_i:nn ...
..... 7990, 7998, 8032, 8078, 8085
\__stex_refs_check_i:nnnn 1900, 1908
\__stex_refs_check_in:nnnn 1982, 1990
\l__stex_refs_default_archive ...
..... 2009, 2014, 2016, 2018
\l__stex_refs_default_file 1922,
1926, 1927, 1930, 2008, 2012, 2019
\l__stex_refs_default_title ....
..... 1925, 2007, 2021
\l__stex_refs_file 1882, 1883, 1901,
1926, 1938, 1947, 1950, 1979, 1983
\__stex_refs_find: ..... 1884, 1898
\__stex_refs_find_in: ... 1948, 1978
\l__stex_refs_in 1873, 1918, 1936, 1940
\__stex_refs_in: .... 1931, 1941, 1945
\__stex_refs_in_text:nn .. 1958, 1964
\__stex_refs_in_text:w ... 1965, 1972
\c__stex_refs_iow .....
..... 1796, 1797, 1798, 1819
\c__stex_refs_iow_str ... 1799, 1927
\l__stex_refs_key ..... 1872, 1909
\__stex_refs_local:n 1844, 1850, 1879
\__stex_refs_local_ref:n .....
..... 1854, 1860, 1920, 1923, 1928
\__stex_refs_maybe_in: ... 1892, 1917
\l__stex_refs_new_tl .....
..... 1813, 1814, 1815, 1817, 1822
\__stex_refs_remote:nn ... 1846, 1871
\__stex_refs_stop: ..... 1965, 1972
\l__stex_refs_str ..... 7394
\l__stex_refs_tmp ..... 1842,
1886, 1890, 1904, 1911, 1946, 1952,
1956, 1957, 1958, 1960, 1986, 1992
\l__stex_refs_unnamed_counter_-
int ..... 1996
\__stex_refs_uppercase:n . 1973, 1976
\l__stex_refs_uri .... 1875, 1876,
1877, 1881, 1890, 1891, 1910, 1919,
1920, 1923, 1928, 1960, 1979, 1991
\__stex_seqs_add: ..... 4476, 4495
\l__stex_seqs_args_tl ... 4555, 4557
\l__stex_seqs_cs ..... 4553, 4557
\__stex_seqs_html: ..... 4474, 4522
\__stex_seqs_macro: ..... 4477, 4509
\l__stex_seqs_range_clist .....
..... 4467, 4503, 4516, 4542
\g__stex_smsmode_allowed_escape_-
tl ..... 2626, 2629, 2756

```

\g__stex_smsmode_allowed_import_- env_seq 2651, 2654, 2779, 2782	__stex_statements_setup_named:n 7358, 7362
\g__stex_smsmode_allowed_import_- tl 2644, 2648, 2774	__stex_statements_setup_noname: 7357, 7381
\g__stex_smsmode_allowed_tl 2621, 2624, 2752	__stex_structures_begin:nn 7019, 7029, 7063, 7104
\g__stex_smsmode_allowedenvs_seq 2631, 2634, 2761, 2764	__stex_structures_check_- def:nnnnnnnn 7122, 7165
\g__stex_smsmode_bool 2660, 2663, 2703	__stex_structures_do externals: 7023, 7049, 7078, 7128
__stex_smsmode_check_begin:Nn 2761, 2779, 2790	__stex_structures_do_usestructure: 7222, 7229, 7235
__stex_smsmode_check_cs:N 2725, 2731	\l__stex_structures_exstruct_- name_str . . . 7101, 7104, 7126, 7175
__stex_smsmode_check_cs:NNn 2724, 2738	__stex_structures_extend_- i:nnnnnnnnNn 7147, 7161
__stex_smsmode_check_end:Nn 2764, 2782, 2798	__stex_structures_extend_- ii:nnnn 7146, 7153
\l__stex_smsmode_cycles_seq 2666, 2672, 2674	__stex_structures_extend_- structure:nnnn 7101, 7138
__stex_smsmode_do:w 2720, 2722, 2724, 2727, 2735, 2754, 2766, 2784, 2795, 2801, 2803	\l__stex_structures_extname_- count 7173, 7175, 7178
__stex_smsmode_do_aux:N 2724, 2734, 2745	__stex_structures_get:w . 7207, 7217
__stex_smsmode_do_aux_curr:N 2683, 2691, 2747	\l__stex_structures_imports_seq 7054, 7059, 7064, 7089, 7094, 7106, 7236, 7239, 7242
__stex_smsmode_do_aux_imports:N 2683, 2773	\l__stex_structures_last_name_- str 7098, 7101
__stex_smsmode_do_aux_normal:N 2691, 2751	\l__stex_structures_name_str 7007, 7012, 7014, 7031, 7032, 7037, 7041, 7046, 7186, 7189
\g__stex_smsmode_import_setup_tl 2645, 2649, 2655, 2684	__stex_structures_new_extstruct_- name: 7088, 7171
\l__stex_smsmode_importmodules_- seq 2669	\l__stex_structures_parent_uri 7098, 7101
__stex_smsmode_in_smsmode:n 2681, 2700	\l__stex_structures_replace_- this_tl 7050
\l__stex_smsmode_sigmodules_seq 2670	\l__stex_structures_struct_uri 7036, 7041
__stex_smsmode_smsmode_do: 2713, 2720	__stex_styles_apply_patch:n 488, 503
__stex_smsmode_start_smsmode:n 2685, 2692, 2711	\g__stex_styles_counters_seq 24, 63, 64, 761, 800, 801
__stex_statements_add_definiens:n 7472, 7477	__stex_styles_css_patch:nnnn 16, 97, 521, 753, 834
__stex_statements_definiens_- inner:nnnnnnN 7481, 7485	\l__stex_styles_first_str 60, 62, 81, 797, 799, 818
__stex_statements_force_id: 7363, 7391, 7411, 7435	__stex_styles_patch:nnn . . . 484, 495
__stex_statements_html_keyvals:nn 7285, 7319, 7330	__stex_styles_patch:nnnn 4, 88, 518, 741, 825
__stex_statements_make_macro:nnn 7309, 7312	__stex_styles_patch_html: 34, 74, 77, 771, 811, 814
__stex_statements_setup:n 7348, 7348, 7412, 7416, 7431, 7436, 7439	__stex_styles_patch_html_c: 26, 66, 69, 763, 803, 806
__stex_statements_setup_def: 7397, 7397, 7413, 7437	

```

\__stex_styles_patch_i:nnn .....
..... 21, 42, 758, 779
\__stex_syms_activate_i:nnnnnnnn
..... 4066, 4079, 4083
\__stex_syms_add_decl: ... 3918, 3921
\l__stex_syms_args_tl ... 4039, 4041
\l__stex_syms_cs .....
.. 4037, 4041, 4161, 4163, 4165, 4193
\__stex_syms_def_style: .. 3803, 3811
\__stex_syms_do_args: ... 3938, 3945
\__stex_syms_get_from_one_-
string:n ..... 4202, 4276
\__stex_syms_get_symbol_from_cs:
..... 4167, 4182
\__stex_syms_get_symbol_from_-
modules:nn ..... 4204, 4235
\__stex_syms_get_symbol_from_-
string:n ... 4168, 4170, 4173, 4197
\__stex_syms_has_definiens:nnnnnnnN
..... 4097, 4104
\__stex_syms_it_decl_check:nnnn .
..... 4137, 4145
\__stex_syms_it_decl_i:n .....
..... 4129, 4133, 4147
\__stex_syms_maybe_activate:nnnnnnnn
..... 4073, 4077
\l__stex_syms_mods_seq .....
..... 4125, 4134, 4135
\l__stex_syms_name ..... 4200, 4206
\l__stex_syms_name_str .....
..... 4237, 4249, 4251
\__stex_syms_parse_arity: 3937, 3956
\l__stex_syms_path_str .....
..... 4238, 4248, 4255, 4260, 4261
\l__stex_syms_seq .....
..... 4199, 4200, 4201, 4205
\__stex_syms_set_textsymdecl_-
macro:nnn ..... 3881, 3900
\__stex_syms_style: ..... 3772, 3780
\__stex_syms_sym_cs:nnnnnnnnN ...
.. 4108, 4109, 4119, 4126, 4127, 4140
\__stex_syms_sym_from_str_i:nnnn
..... 4210, 4244, 4257, 4263
\__stex_syms_sym_i_finish:nnnnnnnN
..... 4216, 4223
\__stex_syms_sym_i_gobble:nnnnnn
..... 4218, 4221
\__stex_syms_top:n .....
..... 3771, 3794, 3908, 3908
\__stex_syms_uri_match:n ..... 4271
\__stex_uris_absolute:N .....
..... 1324, 1355, 1359
\l__stex_uris_archive 1173, 1177, 1180
\__stex_uris_as_qm:nnn ... 1286, 1288

\__stex_uris_doc:nnnnn .....
..... 1121, 1124, 1126
\__stex_uris_doc_from_archive_-
file:NN .... 1167, 1170, 1229, 1231
\__stex_uris_end: ... 1397, 1400, 1404
\l__stex_uris_file .. 1165, 1166, 1167
\__stex_uris_first_three:nnnn ...
..... 1434, 1437, 1440
\l__stex_uris_lang_str .... 1183,
1192, 1194, 1195, 1200, 1201, 1203,
1204, 1205, 1206, 1208, 1209, 1210
\__stex_uris_maybe_relative:N ...
..... 1324, 1332
\__stex_uris_module:nnn .....
..... 1239, 1242, 1244
\l__stex_uris_name_str .....
... 1182, 1190, 1191, 1194, 1215,
1317, 1342, 1348, 1364, 1379, 1388
\__stex_uris_new_mod:nnnnnnn ....
..... 1295, 1299
\__stex_uris_new_nested_mod:nnnnn
..... 1293, 1307
\__stex_uris_pair_in_archive:Nn .
..... 1327, 1384
\__stex_uris_pair_no_archive:N ..
..... 1368, 1372
\__stex_uris_parent:NNnnn .....
..... 1268, 1272, 1275
\__stex_uris_parse_i:Nw .. 1394, 1400
\__stex_uris_parse_ii:Nw . 1396, 1404
\l__stex_uris_path_seq .....
..... 1181, 1189, 1190
\l__stex_uris_path_str .....
.. 1318, 1320, 1323, 1363, 1374, 1387
\__stex_uris_split_file:N 1171, 1188
\__stex_uris_split_file_i: 1196, 1199
\__stex_uris_split_file_ii: .. 1218
\__stex_uris_symbol:nnnn .....
..... 1408, 1411, 1413
\__stex_uris_with_elem:nnnnnnn ...
..... 1159, 1161
\__stex_uris_with_language:nnnnnnn
..... 1153, 1155
\__stex_vars_add: ..... 4311, 4337
\l__stex_vars_args_tl ... 4396, 4398
\l__stex_vars_bind_bool .. 4299, 4380
\__stex_vars_check_var:nnnnnnnnN
..... 4429, 4433
\l__stex_vars_cs ..... 4394, 4398
\__stex_vars_get_var:n ... 4421, 4427
\__stex_vars_html: ..... 4313, 4363
\__stex_vars_macro: ..... 4312, 4350
\__stex_vars_set_vars:nnnnnnnN ...
..... 4435, 4438, 4443

```

<code>\stex_annotate_env (env.)</code>	128, 709	<code>\stexstylevarnotation</code>	106
<code>\stex_env_node (env.)</code>	128, 709	<code>\stexstylevarseq</code>	106
<code>\stexcommentfont</code>	159, 7761, 8117	<code>\stopsolutions</code>	114, 9296
<code>\stexdoctitle</code>	138, 1477, 1580, 1592, 1618, 2330, 8608, 8792, 9079, 9179, 9182, 10295	str commands:	
<code>\STEXexport</code>	43, 88, 89, 143, 2586, 2643	<code>\c_ampersand_str</code>	1129, 1131, 1132, 1134, 1247, 1249, 1416, 1418, 1419
<code>\STEXftmlink</code>	138, 1450	<code>\c_backslash_str</code>	156, 1865
<code>\stexhtmlfalse</code>	692	<code>\c_colon_str</code>	1018
<code>\stexhtmltrue</code>	689	<code>\c_dollar_str</code>	6465
<code>\STEXInternalNewSRefLabel</code>	1815, 1817, 2003	<code>\c_hash_str</code>	6472, 6483, 6504
<code>\STEXInternalNotation</code>	154, 6378, 6744, 6779	<code>\c_left_brace_str</code>	10037, 10079, 10126, 10167, 10187, 10212
<code>\STEXInternalSRefLabel</code>	1820, 1900, 1982, 2001	<code>\c_percent_str</code>	89, 90, 239
<code>\STEXinvisible</code>	83, 129, 709, 6484, 7677, 7679, 7870, 7872, 8145, 8146	<code>\c_right_brace_str</code>	10054, 10096, 10136, 10177, 10193, 10216
<code>\STEXlinkftml</code>	138, 1450	<code>\str_case:Nn</code>	1022
<code>\STEXRestoreNotsEnd</code>	2461, 2499, 2506	<code>\str_case:nn</code>	5403, 6855
<code>\STEXsetlinkftml</code>	138, 1450	<code>\str_case:nnTF</code>	48, 785, 1677, 3960, 5430, 5458, 6725, 8368, 8946, 8969
<code>\stexstyle<environmentname></code>	127	<code>\str_clear:N</code>	6, 7, 8, 91, 155, 379, 380, 387, 405, 743, 744, 745, 849, 969, 1318, 2249, 2292, 2849, 3590, 3591, 3667, 3732, 3733, 3734, 3735, 3745, 3746, 3750, 3767, 3829, 3871, 3872, 4420, 6254, 6255, 6256, 6257, 6473, 7007, 7252, 7253, 7255, 7537, 7538, 7611, 7968, 8502, 8848, 8849, 9013, 9014, 9218, 9235, 9456, 10423, 10424, 10425
<code>\stexstyle<macroname></code>	127	<code>\str_const:Nn</code>	1051, 8287
<code>\stexstyleassertion</code>	106	<code>\str_count:n</code>	59, 64
<code>\stexstyleassign</code>	106	<code>\str_gput_right:Nn</code>	9980
<code>\stexstyleassignMorphism</code>	106	<code>\str_gset:Nn</code>	7834
<code>\stexstylecopymod</code>	106	<code>\str_gset_eq:NN</code>	844, 7831, 7832
<code>\stexstylecopymodule</code>	106	<code>\str_if_empty:NTF</code>	43, 61, 120, 162, 181, 458, 569, 575, 650, 656, 780, 798, 837, 839, 858, 960, 963, 1066, 1323, 1345, 1843, 2317, 2351, 2604, 2819, 2879, 3605, 3619, 3680, 3843, 3909, 4012, 4015, 4018, 4021, 4024, 4248, 4255, 4305, 4368, 4371, 4374, 4377, 4422, 4458, 4461, 4526, 4529, 4532, 4535, 5083, 6202, 6266, 6392, 6404, 6413, 6459, 7011, 7031, 7336, 7349, 7350, 7357, 7392, 7547, 7548, 7561, 7562, 7718, 7729, 7730, 7837, 7838, 7842, 7843, 7880, 7893, 7976, 7982, 8006, 8017, 8018, 8042, 8043, 8358, 8422, 8599, 9067, 9162, 9167, 9244, 9287, 9307, 9355, 9405, 9446, 9465, 9478, 9496, 10183, 10200, 10226, 10238, 10241, 10447, 10451, 10462
<code>\stexstyleextstructure</code>	106		
<code>\stexstylegnote</code>	10181		
<code>\stexstyleimportmodule</code>	106		
<code>\stexstyleinterpretmod</code>	106		
<code>\stexstyleinterpretmodule</code>	106		
<code>\stexstylemathstructure</code>	106		
<code>\stexstylemcb</code>	9558, 9562		
<code>\stexstyleMMTinclude</code>	106		
<code>\stexstylemodule</code>	106		
<code>\stexstylenotation</code>	106		
<code>\stexstyleparagraph</code>	106, 127		
<code>\stexstyleproblem</code>	10017, 10058		
<code>\stexstyleproof</code>	106		
<code>\stexstylerealization</code>	106		
<code>\stexstylerealize</code>	106		
<code>\stexstylerenamedekl</code>	106		
<code>\stexstylerequiremodule</code>	106		
<code>\stexstylescb</code>	9723, 9727		
<code>\stexstylespfsketch</code>	106		
<code>\stexstylesubproblem</code>	10100, 10140		
<code>\stexstylesubproof</code>	106		
<code>\stexstylesymdecl</code>	106		
<code>\stexstylesymdef</code>	106		
<code>\stexstyletextsymbdecl</code>	106		
<code>\stexstyleusemodule</code>	106		
<code>\stexstylevardef</code>	106		

<code>\str_if_empty:nTF</code>	1195, 1201, 1206, 1890, 1926, 2836,
... 89, 142, 193, 496, 826, 3654, 4078	2850, 3054, 3796, 3819, 4328, 4487,
<code>\str_if_eq:NNTF</code>	6269, 6311, 6323, 6333, 7014, 7189,
<code>\str_if_eq:nnTF</code>	7288, 7351, 7367, 7394, 7821, 8057
63, 90, 157, 194, 195, 266, 462,	<code>\str_uppercase:n</code>
570, 576, 668, 1034, 1301, 1342,	1543, 8494
1909, 1991, 2881, 3163, 3315, 3375,	<code>\l_tmpa_str</code>
3396, 3411, 3419, 3672, 3688, 4251,	461, 465, 466, 467, 469, 1004, 1006,
4272, 4434, 4437, 4716, 6178, 6188,	1222, 1223, 3049, 3051, 3054, 3058
6203, 6395, 6407, 6419, 6529, 7471,	<code>\string</code>
7585, 8426, 8429, 8494, 8680, 9568	1878, 9966, 10007
<code>\str_if_eq_p:nn</code> 4212, 4213, 4281, 4282	<code>\subparagraph</code>
<code>\str_if_in:NnTF</code> 6130, 6154, 6483	26, 84
<code>\str_item:Nn</code>	<code>subproblem (env.)</code>
157, 3950	9149
<code>\str_item:nn</code>	<code>subproof (env.)</code>
266	158, 7821
<code>\str_lowercase:n</code>	<code>\subproof</code>
9568	7684, 7944
<code>\str_map_break:</code>	<code>\subproofautorefname</code>
3961	7821
<code>\str_map_break:n</code> . 3962, 3966, 3970,	<code>\subsection</code>
3974, 3978, 3982, 3986, 3990, 3994	26, 84, 139
<code>\str_map_function:nN</code>	<code>\subsubsection</code>
9990	26, 84
<code>\str_map_inline:Nn</code>	<code>\svar</code>
3959	97,
<code>\str_new:N</code>	150, 3942, 5262, 5862, 5864, 5897, 5899
... 453, 1479, 2008, 2009, 3762, 9972	<code>\symbol</code>
<code>\str_put_left:Nn</code>	91
62, 799	<code>\symdecl</code> 22, 30, 31, 36, 41, 89–91, 101,
<code>\str_put_right:Nn</code>	106, 143, 147, 149, 257, 3295, 3762,
7618	8158, 8160, 8188, 8199, 8213, 8216
<code>\str_range:Nnn</code>	<code>\symdef</code>
2125	31–
<code>\str_range:nnn</code>	33, 37, 90, 97, 143, 147, 149, 257,
59, 64	3297, 3791, 8135, 8137, 8140, 8143,
<code>\str_replace_all:Nnn</code>	8148, 8151, 8153, 8155, 8157, 8167,
156	8168, 8169, 8170, 8171, 8172, 8176,
<code>\str_set:Nn</code>	8182, 8194, 8195, 8209, 8211, 8220,
13, 14, 44, 46,	8221, 8224, 8226, 8229, 8231, 8233
50, 51, 52, 53, 54, 55, 56, 60, 83, 94,	<code>\Symname</code>
149, 150, 151, 154, 170, 254, 459,	91, 102, 152, 6004
461, 750, 781, 783, 787, 788, 789,	<code>\symname</code>
790, 791, 792, 793, 797, 879, 999,	18,
1004, 1012, 1013, 1014, 1020, 1021,	19, 23, 56, 91, 92, 102, 104, 152, 6004
1030, 1173, 1210, 1215, 1222, 1235,	<code>\symref</code>
1481, 1527, 1799, 1872, 1881, 1882,	18, 19,
1911, 1938, 2012, 2014, 2016, 2019,	23, 42, 90, 91, 95, 102, 104, 152, 5997
2021, 2049, 2050, 2123, 2190, 2201,	<code>\symrefemph</code> 104, 105, 151, 5958, 8937
2212, 2217, 2228, 2233, 2243, 2246,	<code>\symuse</code> .. 42, 91, 149, 4542, 4572, 4866,
2250, 2336, 2337, 2840, 2854, 2866,	4873, 4884, 4967, 5033, 5110, 5111,
2870, 2900, 2932, 2933, 2969, 2970,	5861, 5875, 5893, 5896, 5906, 5910
3005, 3006, 3049, 3053, 3255, 3256,	sys commands:
3352, 3385, 3413, 3421, 3601, 3607,	<code>\sys_get_shell:nnN</code>
3611, 3719, 3740, 3769, 3793, 3842,	80
3844, 3846, 3848, 3910, 3964, 3968,	<code>\sys_if_platform_windows:TF</code>
3972, 3976, 3980, 3984, 3988, 3992,	86, 147, 228, 238, 262
3996, 4238, 4304, 4306, 4446, 4457,	
4459, 4462, 4648, 5079, 5084, 6129,	
6131, 6155, 6267, 6312, 6471, 7012,	
7032, 7043, 7173, 7175, 7178, 7368,	
7882, 7895, 7984, 8412, 9246, 9309,	
9357, 9407, 9467, 10202, 10446, 10448	
<code>\str_set_eq:NN</code>	
.... 855, 1023, 1024, 1177, 1194,	

	T
<code>\target</code>	124
<code>\test</code>	76, 118
<code>test</code>	113
<code>\testbigspace</code>	9924
<code>\testemptypage</code>	75, 118
<code>\testemptypage</code>	9918, 10258
<code>testheading (env.)</code>	77, 119
<code>\testheading</code>	10360
<code>\testmedspace</code>	9923
<code>\testnewpage</code>	75, 118

<code>\testnewpage</code>	9925, 9926	<code>\hwexam@kw@forgrading</code>	10355
<code>\testnewpageInProblem</code>	9926	<code>\hwexam@kw@given</code>	10311
<code>\testsmallspace</code> . 75, 75, 75, 118, 118, 118		<code>\hwexam@kw@grade</code>	10356
<code>\testsmallspace</code>	9922	<code>\hwexam@kw@probs</code>	10356
<code>\testspace</code>	75, 118	<code>\hwexam@kw@pts</code>	10357
<code>\testspace</code>	9267, 9921	<code>\hwexam@kw@reached</code>	10358
<code>\TeX</code>	23	<code>\hwexam@kw@sum</code>	10356
<code>\tex</code>	23	<code>\hwexam@kw@testemptypage</code>	9919, 10262
TeX and L ^A T _E X 2 _ε commands:		<code>\hwexam@min</code>	10364, 10371, 10376, 10387
<code>\@</code>	2155, 2158, 2162	<code>\hwexam@minutes@kw</code>	10364
<code>\@addtoreset</code>	9029	<code>\hwexam@reqpts</code>	
<code>\@arabic</code>	8803		10373, 10378, 10390, 10394
<code>\@author</code>	8782	<code>\hwexam@tools</code>	10374, 10379
<code>\@auxout</code>	1816, 8249, 9143	<code>\hwexam@totalmin</code>	10386, 10387
<code>\@bonuspointsfalse</code>	10391	<code>\hwexam@totalpts</code>	10385, 10394
<code>\@bonuspointstrue</code>	10396	<code>\if@bonuspoints</code>	10389
<code>\@currentHref</code> . 1823, 7882, 7895, 7984		<code>\if@rustex</code>	672
<code>\@currentlabel</code>	7881,	<code>\itemize@inner</code>	8534, 8540, 8549
7882, 7894, 7895, 7983, 7984, 8598		<code>\itemize@label</code>	8538, 8541, 8544
<code>\@currentlabelname</code>	1824	<code>\itemize@level</code>	8532, 8537, 8540, 8549
<code>\@currenvir</code>	8468, 8469	<code>\itemize@outer</code>	8533, 8537
<code>\@dblarg</code>	8775, 8784	<code>\lst@mhrepos</code>	2228, 2233, 2237
<code>\@gobble</code>	48	<code>\ltx@ifpackageloaded</code> . 2209, 2225,	
<code>\@ifnextchar</code>	47	2240, 8107, 8992, 9592, 9623, 9947	
<code>\@ifstar</code>	8723	<code>\mdf@patchamsthm</code>	8571
<code>\@listdepth</code>	2066	<code>\metakeys@show@keys</code>	8535
<code>\@mainmatterfalse</code>	1753, 1766	<code>\notesslides@slidelabel</code> ..	8574, 8590
<code>\@mainmattertrue</code>	1777, 1788	<code>\ns@author</code>	8775, 8776
<code>\@notprerr</code>	1688	<code>\ns@title</code>	8784, 8785
<code>\@onlypreamble</code>	1688	<code>\pgf@temp</code>	2152, 2153, 2154
<code>\@rustexfalse</code>	664	<code>\pgfkeys@spdef</code>	2152
<code>\@title</code>	1520, 1521, 8791	<code>\pgfutil@empty</code>	2154
<code>\activate@excursion</code>	8830, 8835	<code>\pgfutil@InputIfFileExists</code> ...	2161
<code>\bbl@loaded</code>	8993, 9948	<code>\prematurestop@endsofragment</code>	
<code>\beamer@shortauthor</code> . 8778, 8780, 8795			8467, 8470, 8476
<code>\beamer@shorttitle</code> .. 8787, 8789, 8798		<code>\problem@kw*</code>	8989
<code>\c@chapter</code>	8449	<code>\problem@kw@correct</code>	9607, 9745
<code>\c@framenummer</code>	8803	<code>\problem@kw@feedback</code> ...	9500, 10242
<code>\c@part</code>	8461	<code>\problem@kw@grading</code>	9446, 10183
<code>\compemph@uri</code> 105, 151, 5914, 6138, 6692		<code>\problem@kw@hint</code>	9346
<code>\correction@table</code>	10339	<code>\problem@kw@minutes</code>	9117,
<code>\defemph@uri</code> 105, 151, 5947, 6161		9210, 9513, 10026, 10068, 10114, 10155	
<code>\define@key</code>	2210, 2226, 2241	<code>\problem@kw@note</code>	9394
<code>\Gin@eheight</code> 10512, 10515, 10520, 10525		<code>\problem@kw@points</code>	9497,
<code>\Gin@ewidth</code>		10021, 10063, 10109, 10150, 10239	
.... 8714, 8715, 10511, 10521, 10525		<code>\problem@kw@pts</code>	9112, 9205, 9508
<code>\Gin@exclamation</code> . 10511, 10512, 10520		<code>\problem@kw@solution</code>	9286
<code>\Gin@mhrepos</code>		<code>\problem@kw@wrong</code>	9609, 9747
.. 2212, 2217, 2221, 2243, 2250, 2255		<code>\problem@restore</code> ...	9144, 9148, 10333
<code>\hwexam@bonuspts</code>	10393	<code>\stex@backend</code>	128, 662
<code>\hwexam@checktime</code>	10387	<code>\symrefemph@uri</code>	104, 105,
<code>\hwexam@duration</code>		151, 5958, 6001, 6024, 6041, 8956, 8979	
..... 10363, 10366, 10372, 10377		<code>\varemph@uri</code>	
<code>\hwexam@kw@due</code>	10314	 105, 151, 5938, 6049, 6063, 6077	

tex commands:	<code>\tikzinput</code>
<code>\tex_undefined:D</code>	121, 141, 2255, 10504, 10509, 10535
<code>\texname</code>	tikzinput commands:
<code>\text</code>	<code>\c_tikzinput_image_bool</code>
<code>\textbf</code>	38, 10493, 10500
8978, 9131, 9134, 9899, 10304, 10309	<code>\times</code>
<code>\textcolor</code>	8201, 8205
9895	<code>\tiny</code> ...
<code>\textsymdecl</code> .	152, 6219, 8575, 8590, 8955, 8978
23, 90, 147, 149, 3296, 3828	<code>\title</code>
<code>\texttt</code>	76, 119
9879, 9895	<code>\title</code>
<code>\textwarning</code>	1612, 8784
63, 111	<code>titlefragment (env.)</code>
<code>\textwidth</code>	139, 1606
8575, 8712, 8734, 10352	tl commands:
<code>\the</code>	<code>\tl_clear:N</code>
334, 335, 337,	..
339, 341, 353, 355, 2155, 2156, 2157	282, 393, 505, 543, 1280, 1548,
<code>\theassignment</code>	1801, 1802, 1803, 1842, 1946, 2296,
10292, 10304	2671, 2934, 2971, 3007, 3032, 3151,
<code>\thechapter</code> 1567, 1633, 1666, 1706, 1717,	3183, 3333, 3351, 3592, 3747, 3748,
8375, 8380, 8384, 8388, 8392, 8396	3749, 3787, 3818, 3830, 3831, 3870,
<code>\theframenumber</code>	3889, 3946, 4159, 4327, 4470, 4486,
8598	4643, 4652, 5061, 5091, 5135, 5272,
<code>\thepage</code>	5344, 5489, 5653, 5654, 5655, 5656,
10260	5983, 5984, 5985, 5993, 6136, 6332,
<code>\theparagraph</code>	6349, 6360, 6361, 6451, 6477, 6616,
8392, 8396	6696, 6707, 6742, 6934, 7006, 7204,
<code>\thepart</code>	7256, 7257, 7258, 7355, 7523, 7524,
1563, 1630, 1662, 1700, 8370	7525, 7526, 7579, 7580, 7581, 8847,
<code>\theplainsproblem</code>	9011, 9457, 9575, 9577, 9578, 9820,
9033, 9034	10267, 10268, 10269, 10344, 10345,
<code>\thesection</code>	10346, 10371, 10372, 10373, 10374
..	<code>\tl_const:Nn</code>
8380, 8384, 8388, 8392, 8396, 9034	
<code>\thesproblem</code>	1053, 6958, 6959, 9829, 9830
9030, 9034, 9134,	<code>\tl_count:N</code>
9144, 10018, 10039, 10059, 10081,	5435, 5441
10106, 10128, 10146, 10169, 10292	<code>\tl_count:n</code>
<code>\thesubparagraph</code>	5739, 5749, 5755
8396	<code>\tl_gclear:N</code>
<code>\thesubsection</code> ...	2366, 2426
8384, 8388, 8392, 8396	<code>\tl_gput_left:Nn</code>
<code>\thesubsubsection</code>	365, 367, 368
8388, 8392, 8396	<code>\tl_gput_right:Nn</code>
<code>\this</code> ...	
50-52, 98, 100, 156, 273, 4645,	335, 2528, 2535, 2592, 2624,
4983, 5067, 5068, 5073, 5098, 7179	2629, 2648, 2649, 2655, 2979, 8831
<code>\thisarchive</code>	<code>\tl_gset:Nn</code>
2968	257, 337, 360, 361,
<code>\thisargs</code>	933, 1032, 1457, 1460, 1492, 1499,
3786, 3817	2004, 2491, 4587, 4591, 6364, 7847,
<code>\thiscopyname</code>	7853, 8245, 9189, 9192, 10336, 10337
3456, 3481, 3523, 3547	<code>\tl_gset_eq:NN</code>
<code>\thisdeclname</code>	81, 1080, 1081, 1086, 1087, 2614, 3108
3783, 3814, 3888, 6312	<code>\tl_head:N</code>
<code>\thisdecluri</code>	4165, 4764, 4995, 5013
3782, 3813, 3887, 6313	<code>\tl_head:n</code>
<code>\thisdefiniens</code>	5740, 5750, 5756, 6895
3785, 3816	<code>\tl_if_empty:NTF</code>
<code>\thisfor</code>	298, 557,
7289	1463, 1511, 1520, 1593, 1613, 1856,
<code>\thismodulename</code> ..	1886, 1888, 1918, 1922, 1952, 1954,
2337, 2933, 2970, 3006	1967, 2318, 2329, 2354, 3038, 3155,
<code>\thismoduleuri</code>	3187, 3227, 3356, 3360, 3623, 3858,
2336, 2932,	3927, 4029, 4032, 4035, 4175, 4386,
2969, 3005, 3457, 3482, 3524, 3548	4389, 4392, 4400, 4468, 4545, 4548,
<code>\thisname</code>	4551, 4559, 5312, 5324, 5371, 5455,
7288	5511, 5554, 5570, 5605, 5623, 5782,
<code>\thisnotation</code> 3820, 4329, 4488, 6314, 6334	
<code>\thisnotationvariant</code>	
.....	
3819, 4328, 4487, 6311, 6333	
<code>\thisstyle</code>	
505, 508,	
509, 510, 543, 546, 547, 548, 549,	
557, 560, 561, 2934, 2971, 3007,	
3787, 3818, 3889, 4327, 4486, 6332	
<code>\thistitle</code>	
105, 2328, 2329, 2330,	
7287, 9083, 9130, 9131, 9135, 9136	
<code>\thistype</code>	
3784, 3815	
<code>\thisvarname</code>	
4326, 4485, 6331	
<code>\throwaway</code>	
144	

5972, 6231, 6235, 6239, 6244, 6343,	1333, 1336, 1361, 1376, 1385, 1602,
6352, 6358, 6555, 6568, 6585, 6602,	1751, 1756, 1764, 1813, 1873, 1904,
6705, 6741, 7183, 7293, 7454, 7503,	1957, 1986, 1992, 2361, 2394, 2408,
7743, 7745, 7748, 7751, 8113, 8715,	2421, 2483, 2514, 2561, 2826, 2832,
8859, 8945, 8968, 9075, 9078, 9084,	2833, 2834, 2906, 2968, 3043, 3172,
9130, 9135, 9175, 9178, 9181, 9186,	3199, 3207, 3248, 3263, 3376, 3387,
9286, 9346, 9394, 9483, 9499, 9607,	3388, 3389, 3390, 3391, 3397, 3456,
9609, 9612, 9644, 9646, 9648, 9656,	3457, 3481, 3482, 3523, 3524, 3547,
9745, 9747, 9750, 9778, 9780, 9782,	3548, 3602, 3608, 3612, 3782, 3813,
9790, 9896, 10231, 10287, 10294,	3849, 3887, 3912, 4039, 4185, 4187,
10305, 10310, 10313, 10363, 10390	4188, 4189, 4190, 4191, 4225, 4227,
\tl_if_empty:nTF ... 265, 274, 362,	4228, 4229, 4230, 4231, 4286, 4288,
901, 1100, 1102, 1106, 1110, 1112,	4289, 4290, 4291, 4292, 4351, 4396,
1128, 1133, 1162, 1246, 1304, 1322,	4448, 4449, 4450, 4451, 4452, 4510,
1415, 1472, 1966, 2013, 2135, 2295,	4555, 4586, 4588, 4595, 4607, 4608,
2723, 3014, 3034, 3046, 3100, 3223,	4609, 4628, 4644, 4649, 4650, 4651,
3283, 3660, 4064, 4105, 4146, 4163,	4656, 4732, 4759, 4777, 4813, 4814,
4799, 4802, 4848, 4858, 4948, 4950,	4818, 4821, 4828, 4829, 4865, 4905,
5034, 5050, 5059, 5112, 5146, 5210,	4979, 4985, 5038, 5064, 5067, 5070,
5231, 5264, 5399, 5419, 5702, 6009,	5093, 5094, 5100, 5109, 5114, 5153,
6169, 6547, 6562, 6575, 6648, 6682,	5154, 5158, 5211, 5213, 5221, 5223,
6787, 6835, 7166, 7309, 7450, 7499,	5226, 5232, 5234, 5237, 5249, 5250,
7513, 8257, 8777, 8786, 9876, 9900	5265, 5266, 5268, 5269, 5435, 5441,
\tl_if_empty_p:N 288	5454, 5460, 5463, 5466, 5488, 5510,
\tl_if_eq:NnTF .. 974, 1877, 4764,	5774, 5775, 5783, 5815, 5817, 5824,
4995, 5013, 7206, 9839, 9840, 9849	5860, 5872, 5874, 5895, 5927, 6018,
\tl_if_eq:NnTF 8887,	6019, 6035, 6036, 6050, 6051, 6064,
9090, 9093, 9111, 9116, 10020,	6065, 6078, 6079, 6128, 6150, 6151,
10025, 10042, 10062, 10067, 10084	6313, 6344, 6353, 6377, 6393, 6396,
\tl_if_eq:nTF .. 2506, 3197, 3205,	6405, 6408, 6414, 6421, 6440, 6543,
4710, 5750, 5756, 5811, 5849, 6910	6545, 6548, 6550, 6744, 6751, 6757,
\tl_if_exist:NnTF 307,	6796, 6810, 6811, 6823, 6862, 6875,
334, 347, 509, 548, 560, 671, 902,	6887, 6907, 6933, 6955, 6956, 6993,
1042, 1095, 1099, 1109, 1221, 1360,	6994, 7036, 7050, 7155, 7184, 7207,
1535, 1545, 1852, 2178, 2541, 2545,	7377, 7384, 7742, 8106, 8112, 8128,
2549, 2562, 6982, 6987, 8401, 10004	8370, 8371, 8375, 8376, 8380, 8381,
\tl_if_in:NnTF 2752, 2756, 2774	8384, 8385, 8388, 8389, 8392, 8393,
\tl_item:nn 5742	8396, 8397, 9012, 9084, 9128, 9159,
\tl_map_inline:nn 314, 318	9160, 9186, 9226, 9527, 9591, 9622,
\tl_new:N 1477, 1480,	9627, 9642, 9700, 9734, 9760, 9761,
2007, 2289, 2526, 2621, 2626, 2644,	9776, 9838, 10324, 10330, 10331,
2645, 2659, 3019, 4583, 4631, 4632	10385, 10386, 10387, 10393, 10458
\tl_put_left:Nn 4655, 5523, 6825, 6826, 9057, 10280	\tl_set_eq:NN 68, 253, 364, 366, 371,
\tl_put_right:Nn 1084, 3316, 3320, 3948,	923, 1059, 1070, 1072, 1234, 1351,
3949, 4637, 4654, 5060, 5158, 5212,	1925, 2328, 2371, 2403, 2431, 2580,
5229, 5230, 5240, 5241, 5254, 5500,	2831, 3036, 3225, 3783, 3784, 3785,
6457, 6463, 6475, 6484, 6760, 6765,	3786, 3814, 3815, 3816, 3817, 3873,
6768, 6857, 6865, 6870, 6878, 6941,	3888, 4317, 4326, 4466, 4485, 4911,
7140, 9844, 9856, 10348, 10349, 10350	4983, 5073, 5098, 5375, 5764, 5822,
\tl_set:Nn . 90, 91, 93, 94, 295, 308,	6127, 6331, 6398, 6410, 6434, 6580,
492, 497, 499, 536, 537, 827, 828,	6582, 6586, 6612, 6614, 6756, 7287,
830, 831, 1043, 1179, 1279, 1282,	7452, 7501, 7577, 9083, 9088, 9089
	\tl_set_rescan:Nnn 466
	\tl_tail:n 2723, 4874, 5726

<code>\tl_to_str:n</code>	68, 74, 104, 107, 128, 131, 143, 166, 173, 235, 315, 319, 320, 426, 439, 860, 934, 935, 1018, 1044, 1045, 1055, 1118, 1127, 1129, 1131, 1132, 1134, 1245, 1247, 1249, 1304, 1310, 1316, 1386, 1414, 1416, 1418, 1419, 1422, 1498, 1832, 1833, 1834, 1835, 1837, 2004, 2034, 2133, 2471, 2472, 2473, 2476, 2477, 2478, 2479, 2519, 2520, 2536, 2634, 2654, 2739, 2753, 2757, 2775, 2791, 2799, 3093, 3215, 3249, 3406, 3574, 3594, 3624, 4057, 4058, 4059, 4060, 4085, 4260, 4771, 4791, 4795, 4956, 5014, 5092, 5165, 5179, 5201, 5220, 5480, 5701, 5716, 6437, 6505, 6531, 6795, 6808, 6897, 6932, 7273, 8129, 8130, 8245, 8246, 8993, 9847, 9857, 9879, 9895, 9909, 9911, 9948, 9990	
<code>\tl_trim_spaces:N</code>	84	
<code>\l_tmpa_tl</code>	80, 81, 83, 1064, 1226, 1228, 1336, 1350, 1351, 2185, 2490, 2491, 4862, 5221, 5229, 5230, 5232, 5240, 5241, 5422, 5428, 5433, 5435, 5439, 5441, 5444, 5453, 5455, 5460, 5463, 5466, 5860, 5877, 5895, 5906, 6467, 6934, 6941, 6948, 7742, 7743, 9642, 9656, 9776, 9790, 9838, 9839, 9840, 9849, 10344, 10348, 10356	
<code>\l_tmpb_tl</code>	1333, 1341, 1345, 5422, 5428, 5430, 5444, 5454, 5458, 10345, 10349, 10357	
tmpc commands:		
<code>\l_tmpc_tl</code>	10346, 10350, 10358	
<code>\to</code>	8189, 8190, 8202	
<code>\today</code>	8805, 10448	
<code>\TODO</code>	4849, 8885	
token commands:		
<code>\c_math_subscript_token</code>	 43, 89, 4417, 5927, 6727, 6728, 6731, 6732, 6736, 8170, 8192, 8194	
<code>topsect</code>	60	
<code>\topsep</code>	7811, 9199, 9473, 9492, 9520, 9685, 10102, 10142, 10219	
<code>\trueFfalseT</code>	9814	
<code>\trueFfalseF</code>	9809	
<code>\type</code>	76, 119	
U		
<code>\underbrace</code>	6189, 6191, 8196	
<code>\univ</code>	56	
<code>\unless</code>	8468	
<code>\uppercase</code>	149, 1976, 6029	
<code>\url</code>	1451	
use commands:		
<code>\use:N</code>	375, 506, 510, 512, 544, 549, 551, 558, 561, 563, 889, 896, 1033, 1594, 1596, 1854, 1901, 1983, 2568, 3860, 7697, 7705, 7721, 7783, 7801, 7898, 7911, 7922, 7924, 7953, 7962, 8011, 8028, 8402, 8408	
<code>\use:n</code>	329, 2124, 2209, 2225, 2240, 4036, 4393, 4552, 4574, 4965, 5080, 5490, 5526	
<code>\use:nn</code>	 93, 304, 474, 477, 889, 896, 1596, 1902, 1984, 2517, 3042, 3159, 3257, 3821, 3854, 3922, 4097, 4330, 4729, 4752, 4756, 5032, 5106, 5202, 5536, 5726, 5777, 5867, 6040, 6113, 6315, 6335, 6668, 6698, 7040, 7146, 7147, 7370, 7481, 7515, 7551, 7556, 7563, 7566, 7977, 8863, 9228, 9987, 10326	
<code>\use_i:nn</code>	3159, 5080, 10349	
<code>\use_i:nnn</code>	882, 1252	
<code>\use_i:nnnn</code>	1428	
<code>\use_i:nnnnn</code>	1138	
<code>\use_ii:nn</code>	3147, 3165, 5123	
<code>\use_ii:nnn</code>	885, 889, 1255, 1258	
<code>\use_ii:nnnn</code>	1431	
<code>\use_ii:nnnnn</code>	1141	
<code>\use_iii:nnn</code>	892, 896, 1261, 1265	
<code>\use_iii:nnnnn</code>	1144	
<code>\use_iv:nnnn</code>	1444, 1447	
<code>\use_iv:nnnnn</code>	1147	
<code>\use_none:n</code>	7838, 7843	
<code>\use_none:nn</code>	7157	
<code>\use_v:nnnnn</code>	1150	
<code>\usebox</code>	8658	
<code>\usemodule</code>	16, 21, 22, 25, 35, 43, 95, 96, 98, 124, 128, 145, 416, 423, 2918	
<code>\usepackage</code>	7, 22, 2124, 8482	
<code>\useSGvar</code>	63, 112, 8878	
<code>\usestructure</code>	98, 156, 7218	
<code>\usetheme</code>	8482	
<code>\usetikzlibrary</code>	82, 140, 2132, 10503	
V		
<code>\varbind</code>	41, 97, 102, 157, 7407, 7422, 7488	
<code>\vardef</code>	33, 41, 97, 98, 148, 4299, 4414, 7515, 7551, 7556, 7563, 7566, 7977	
<code>\varemp</code>	105, 151, 5938, 8939	
<code>\Varname</code>	152, 6045	
<code>\varname</code>	152, 6045	
<code>\varnotation</code>	97, 153, 6320	
<code>\varref</code>	152, 6045	
<code>\varseq</code>	38, 98, 149, 150, 4417, 4455	

<code>\vbox</code>	144, 1899, 1980, 2054, 2067, 7700, 7913, 8108, 8569, 8665, 8671, 8687, 9268, 9328, 9376, 9427, 9593, 9624	<code>\vspace</code>	9921
W			
vbox commands:		<code>\wff</code>	19
<code>\vbox_set:Nn</code>	2702	<code>\withbrackets</code>	94, 155, 6991, 7002
<code>\vdash</code>	8221	X	
<code>\vfill</code>	8406, 9919, 9927, 10262	<code>\xdef</code>	9966, 10018, 10059, 10106, 10146, 10260
<code>\vM</code>	46	<code>\xspace</code>	
<code>\vMs</code>	48		139, 1535, 1545, 3904, 3905, 4666, 4667
<code>\vop</code>	39, 41	Y	
<code>\vphantom</code>	9911	<code>\yesFnoT</code>	9804
<code>\vrule</code>	8108, 8734, 9593, 9624	<code>\yesTnoF</code>	9799
<code>\vskip</code>	2067, 2068, 8108, 8733, 8735, 9593, 9624	<code>\yield</code>	158, 7689, 8090